

Spring Security

В ДЕЙСТВИИ



Лауренциу
Спилкэ



MANNING

Лауренциу Спилкэ

Spring Security в действии

Spring Security in Action

Second Edition

LAURENȚIU SPILCĂ

Spring Security в действии

ЛАУРЕНЦИУ СПИЛКЭ

Москва, 2025

<https://t.me/javallib>

Книга рассказывает о популярном фреймворке Spring Security, значительно облегчающем защиту корпоративных приложений, разработанных на Java. Этот набор инструментов и механизмов идеально интегрируется с приложениями Spring, обеспечивая защиту с первых шагов проекта и предоставляя встроенные функции, позволяющие реализовать надежную авторизацию и аутентификацию, а также защитить систему от кражи данных и вторжений.

Издание предназначено опытным разработчикам Java и Spring.

Отзывы на первое издание

«Одно из лучших руководств по Spring Security 6 – практичное и простое в освоении, охватывающее все аспекты, которые вам, вероятно, пригодятся в работе».

Амарджит Бханда, старший Java-разработчик

«Одна из лучших технических книг, которые я прочитала за последний год. Удивительно, насколько хорошо она рассказывает про фреймворк Spring Security!»

*Симона Сгуацца, младший научный сотрудник,
Университет прикладных наук и искусств Южной Швейцарии*

«Это прекрасный и емкий обзор фреймворка Spring Security!»

Сачин Хандиекар, ведущий инженер-программист в JPMC

«Незаменимое руководство по освоению тонкостей Spring Security. Абсолютно авторитетное, непревзойденное по глубине и ясности».

Наджиб Ариф, старший консультант в Thoughtworks

«Вы должны иметь эту книгу под рукой, если собираетесь использовать Spring Security. И почти каждому приложению Spring требуется Spring Security».

Луиджи Рубино, архитектор программного обеспечения в Unimatica S.p.A.

Оглавление

Часть I	ЗНАКОМСТВО СО SPRING SECURITY	24
1	■ Безопасность сегодня	25
2	■ Spring Security в действии	40
Часть II	НАСТРОЙКА АУТЕНТИФИКАЦИИ	70
3	■ Управление пользователями	71
4	■ Управление паролями	100
5	■ Безопасность веб-приложения начинается с фильтров	117
6	■ Реализация аутентификации	138
Часть III	МЕТОДИКА АВТОРИЗАЦИИ	177
7	■ Авторизация на уровне конечной точки: ограничение доступа	178
8	■ Автоматизация на уровне конечной точки: применение ограничений	201
9	■ Настройка защиты от CSRF	223
10	■ Настройка CORS	250
11	■ Авторизация на уровне методов	261
12	■ Фильтрация на уровне метода	290
Часть IV	ВНЕДРЕНИЕ OAUTH 2 И OPENID CONNECT ...	313
13	■ Что такое OAuth 2 и OpenID Connect?	314
14	■ Сервер авторизации OAuth 2	336
15	■ Реализация сервера ресурсов OAuth 2	361
16	■ Реализация клиента OAuth 2	392
Часть V	РЕАКТИВНЫЙ РЕЖИМ	415
17	■ Безопасность реактивных приложений	416
Часть VI	ТЕСТИРОВАНИЕ КОНФИГУРАЦИЙ БЕЗОПАСНОСТИ	443
18	■ Тестирование конфигураций безопасности	444

Содержание

<i>Предисловие</i>	12
<i>Введение</i>	14
<i>Благодарности</i>	16
<i>Об этой книге</i>	17
<i>Об авторе</i>	21
<i>О техническом редакторе</i>	22
<i>Об иллюстрации на обложке</i>	23
Часть I ЗНАКОМСТВО СО SPRING SECURITY	24
1 <i>Безопасность сегодня</i>	25
1.1 Что такое Spring Security.....	27
1.2 Что такое безопасность программного обеспечения?.....	30
1.3 Почему так важна безопасность?	36
1.4 Что вы узнаете из этой книги?	38
Заключение	39
2 <i>Spring Security в действии</i>	40
2.1 Приступаем к первому проекту.....	42
2.2 Обзор устройства классов Spring Security	47
2.3 Переопределение конфигураций по умолчанию	52
2.3.1 Настройка управления данными пользователя.....	53
2.3.2 Применение авторизации на уровне конечной точки.....	57
2.3.3 Разные способы настройки	61
2.3.4 Определение пользовательской логики аутентификации	63
2.3.5 Использование нескольких классов конфигурации	67
Заключение	69
Часть II НАСТРОЙКА АУТЕНТИФИКАЦИИ	70
3 <i>Управление пользователями</i>	71
3.1 Реализация аутентификации в Spring Security	72
3.2 Описание пользователя	75
3.2.1 Описание пользователей с помощью контракта UserDetails....	75
3.2.2 Детализация контракта GrantedAuthority	77
3.2.3 Минимальная реализация UserDetails.....	78

3.2.4	Использование конструктора для создания экземпляров типа <code>UserDetails</code>	81
3.2.5	Объединение нескольких ответственностей, связанных с пользователем.....	82
3.3	Инструктирование Spring Security о том, как управлять пользователями	86
3.3.1	Подробнее о контракте <code>UserDetailsService</code>	86
3.3.2	Реализация контракта <code>UserDetailsService</code>	87
3.3.3	Реализация контракта <code>UserDetailsManager</code>	90
	Заключение	99
4	Управление паролями	100
4.1	Использование кодировщиков паролей	100
4.1.1	Контракт <code>PasswordEncoder</code>	102
4.1.2	Реализация собственного класса <code>PasswordEncoder</code>	103
4.1.3	Выбор готовых реализаций <code>PasswordEncoder</code>	104
4.1.4	Несколько стратегий кодирования с помощью <code>DelegatingPasswordEncoder</code>	107
4.2	Использование модуля Spring Security Crypto	112
4.2.1	Генераторы ключей.....	112
4.2.2	Шифрование и дешифровка секретов с помощью шифраторов.....	114
	Заключение	116
5	Безопасность веб-приложения начинается с фильтров	117
5.1	Реализация фильтров в архитектуре Spring Security	120
5.2	Добавление фильтра перед существующим фильтром в цепочке.....	122
5.3	Добавление фильтра после существующего фильтра в цепочке.....	126
5.4	Добавление фильтра на место существующего	128
5.5	Реализации фильтров, предоставляемые Spring Security	135
	Заключение	137
6	Реализация аутентификации.....	138
6.1	Что такое <code>AuthenticationProvider</code>	140
6.1.1	Представление запроса во время аутентификации.....	141
6.1.2	Реализация пользовательской логики аутентификации	143
6.1.3	Применение пользовательской логики аутентификации	146
6.2	Использование <code>SecurityContext</code>	151
6.2.1	Использование стратегии хранения для контекста безопасности	152
6.2.2	Использование стратегии хранения для асинхронных вызовов	154
6.2.3	Использование стратегии хранения в автономных приложениях.....	157

6.2.4	Распространение контекста безопасности с помощью <i>DelegatingSecurityContextRunnable</i>	158
6.2.5	Распространение контекста безопасности с помощью <i>DelegatingSecurityContextExecutorService</i>	161
6.3	Подробнее о методе HTTP Basic и аутентификации на основе форм	163
6.3.1	Использование и настройка HTTP Basic	164
6.3.2	Аутентификация на основе форм	167
	Заключение	175

Часть III МЕТОДИКА АВТОРИЗАЦИИ.....177

7 Авторизация на уровне конечной точки: ограничение доступа.....178

7.1	Ограничение доступа на основе полномочий и ролей	180
7.1.1	Ограничение доступа для всех конечных точек на основе полномочий пользователя	182
7.1.2	Ограничение доступа для всех конечных точек на основе ролей пользователей	192
7.1.3	Ограничение доступа ко всем конечным точкам	197
	Заклучение	200

8 Автоматизация на уровне конечной точки: применение ограничений.....201

8.1	Использование метода <i>requestMatchers()</i> для выбора конечных точек	202
8.2	Выбор запросов для применения ограничений авторизации	208
8.3	Использование регулярных выражений с сопоставителями запросов	217
	Заклучение	222

9 Настройка защиты от CSRF.....223

9.1	Как работает защита CSRF в Spring Security	224
9.2	Использование защиты от CSRF на практике	231
9.3	Настройка защиты от CSRF	237
	Заклучение	249

10 Настройка CORS.....250

10.1	Как работает CORS?	251
10.2	Применение политик CORS с аннотацией <i>@CrossOrigin</i>	257
10.3	Применение CORS с использованием <i>CorsConfigurer</i>	259
	Заклучение	260

11 Авторизация на уровне методов.....261

11.1	Включение механизма безопасности метода	262
11.1.1	Что такое авторизация вызовов	263

11.1.2	Применение безопасности метода в вашем проекте.....	266
11.2	Применение правил предварительной авторизации.....	267
11.3	Применение правил поставторизации.....	273
11.4	Реализация разрешений для методов.....	277
	Заключение.....	289

12	Фильтрация на уровне метода.....	290
12.1	Применение предварительной фильтрации.....	292
12.2	Применение постфильтрации.....	299
12.3	Использование фильтрации в репозиториях Spring Data.....	304
	Заключение.....	311

Часть IV	ВНЕДРЕНИЕ OAUTH 2 И OPENID CONNECT.....	313
-----------------	--	------------

13	Что такое OAuth 2 и OpenID Connect?.....	314
13.1	Общий обзор OAuth 2 и OpenID Connect.....	316
13.2	Использование различных реализаций токенов.....	320
13.2.1	Использование непрозрачных токенов.....	321
13.2.2	Использование прозрачных токенов.....	322
13.3	Получение токенов с помощью различных типов грантов.....	324
13.3.1	Получение токена с использованием кода авторизации.....	325
13.3.2	Применение защиты PKCE совместно с кодом авторизации.....	327
13.3.3	Получение токена с предоставлением учетных данных клиента.....	329
13.3.4	Использование обновления токенов для получения новых токенов доступа.....	330
13.4	Что OpenID Connect добавляет к OAuth 2?.....	332
13.5	Недостатки OAuth 2.....	333
	Заключение.....	334

14	Сервер авторизации OAuth 2.....	336
14.1	Базовая аутентификация с использованием веб-токенов JSON.....	337
14.2	Подход с кодом авторизации в действии.....	346
14.3	Грант в форме клиентских данных.....	353
14.4	Использование непрозрачных токенов и интроспекции.....	355
14.5	Отзыв токенов.....	358
	Заключение.....	359

15	Реализация сервера ресурсов OAuth 2.....	361
15.1	Настройка проверки JWT.....	363
15.2	Использование настраиваемых JWT.....	371
15.3	Проверка токена с помощью интроспекции.....	377
15.4	Реализация многопользовательских систем.....	384
	Заключение.....	390

16	Реализация клиента OAuth 2	392
16.1	Реализация входа OAuth 2	394
16.1.1	Аутентификация с одним провайдером	394
16.1.2	Предоставление пользователю дополнительных возможностей	397
16.1.3	Собственный сервер авторизации	399
16.1.4	Добавление гибкости в конфигурации	402
16.1.5	Управление авторизацией в OAuth 2	405
16.2	Реализация клиента OAuth 2	406
	Заключение	413

Часть V РЕАКТИВНЫЙ РЕЖИМ

415

17	Безопасность реактивных приложений	416
17.1	Что такое реактивные приложения?	417
17.2	Управление пользователями в реактивных приложениях	423
17.3	Настройка правил авторизации в реактивных приложениях	428
17.3.1	Применение авторизации на уровне конечной точки в реактивных приложениях	429
17.3.2	Использование безопасности методов в реактивных приложениях	436
17.4	Создание реактивного сервера ресурсов OAuth 2	439
	Заклучение	442

Часть VI ТЕСТИРОВАНИЕ КОНФИГУРАЦИЙ БЕЗОПАСНОСТИ

443

18	Тестирование конфигураций безопасности	444
18.1	Использование фиктивных пользователей для тестов	447
18.2	Тестирование с пользователями из UserDetailsService	456
18.3	Использование собственных объектов аутентификации для тестирования	457
18.4	Тестирование безопасности метода	461
18.5	Тестирование аутентификации	463
18.6	Тестирование конфигураций CSRF	468
18.7	Тестирование конфигураций CORS	470
18.8	Тестирование безопасности Spring Security в реактивных приложениях	471
	Заклучение	473
	Приложение А. Ссылки на официальную документацию	474
	Приложение В. Дополнительная литература	475
	Предметный указатель	479

Предисловие

Я впервые познакомился с Лауренциу Спилкэ в 2022 г., когда мы дистанционно работали над презентацией Spring-One, и я был счастлив наконец встретиться с ним лично на Devoxx 2023 в Бельгии. Меня взволновала и обрадовала его просьба написать это предисловие. Я начну с отрывка из книги:

«Если вы неправильно применяете фреймворк, то получаете приложение, которое трудно поддерживать. Что еще хуже, иногда те, кто не использует фреймворк, считают, что это вина фреймворка».

Золотые слова! За последние годы я несколько раз слышал мнение о том, что Spring Security сложен для понимания и имеет слишком крутую кривую обучения. Так это или нет, если вы погрузитесь во внутреннее устройство фреймворка и по-настоящему хорошо разберетесь в архитектуре аутентификации, вы невольно изучите возможности этого мощного инструмента, и в конечном итоге вам станет проще его использовать. Эта книга попадает точно в цель и дает нам детальное описание архитектуры аутентификации Spring Security с помощью простых и понятных диаграмм, сопровождаемых подробными объяснениями каждого из основных компонентов, которые взаимодействуют в процессе аутентификации.

На протяжении всей книги Лауренциу умело использует аналогии, чтобы упростить рассматриваемую тему. Мне очень нравится аналогия, приведенная в главе про архитектуру аутентификации, поскольку она хорошо передает идею:

«Если вы знаете эту архитектуру, то вы похожи на шеф-повара, который знает все ингредиенты на кухне и может приготовить любой рецепт».

Диаграмма взаимодействий, использованная для демонстрации процесса аутентификации, превосходна. Она дает общее понимание потока аутентификации и подробно описывает назначение и зону ответственности каждого из основных компонентов по мере чтения книги.

Повествование логично и последовательно начинается с очень простых примеров и плавно переходит к более сложным, не перегружая читателя.

После подробного изучения архитектуры аутентификации автор переходит к авторизации. Я сразу вспоминаю вопрос, который часто встречаю в сообществе разработчиков: «В чем разница между полномочиями, ролью и разрешением?» Вы получите хороший ответ на этот вопрос, потому что в книге даны очень простые реальные примеры полномочий, ролей и разрешений, а также варианты их привязки к пользователю. Далее автор дает общие рекомендации по моделированию полномочий в вашем приложении на основе доступных функций и типов пользователей системы. Затем он демонстрирует применение общих принципов на примере определения правил авторизации в конфигурации Spring Security для ограниченного контролируемого доступа.

Часть IV охватывает темы OAuth 2 и OpenID Connect 1.0. Набор спецификаций OAuth 2 и OpenID Connect 1.0 очень обширен, поэтому новичкам сложно в полной мере понять их назначение и возможности. Тем не менее эта книга дает превосходный обзор основных понятий (роли, типы предоставления авторизации, форматы токенов доступа и т. д.), определенных в спецификациях, и их реализации в Spring Security и Spring Authorization Server. Я согласен с тем, что спецификация OAuth 2 очень похожа на систему доступа в офисное здание, где вам нужна карта ограниченного доступа, чтобы войти в конференц-зал внутри здания. На примере этого реального сценария безопасности в книге показаны различные части системы OAuth 2 и их предназначение. Затем автор переходит к простому примеру использования Spring Authorization Server вместе с поддержкой клиента и сервера ресурсов Spring Security. Автор начинает с простейших примеров, а затем демонстрирует применение клиентов при реализации прав доступа разного типа, например кода авторизации (с PKCE), учетных данных клиента и обновления токена. Далее демонстрируются общие сценарии конфигурации, например настройка непрозрачных токенов и использование интроспекции и отзыва токенов. На следующем шаге показаны более сложные сценарии настройки многопользовательской аренды для серверов ресурсов.

Подводя итог, можно сказать, что эту книгу должны прочитать все, кто хочет тщательно изучить архитектуру аутентификации Spring Security и внутренние компоненты фреймворка, потому что только так вы сможете в полной мере воспользоваться его возможностями.

*Джо Гранджа,
инженер по безопасности Spring, VMware by Broadcom*

Введение

Путь разработчика программного обеспечения – это захватывающее переплетение творчества, обучения, преподавания, но часто – переучивания. Моя карьера началась в 2007 г., и я сам не заметил, как из простого разработчика превратился в разработчика-наставника. Хотя обе роли имеют свое уникальное очарование, именно искусство передачи знаний, воспитания любопытства и наблюдения за моментами озарения у других учеников по-настоящему зажигает во мне огонь. Но давайте посмотрим правде в глаза: сферы разработки и обучения глубоко переплетены. Чтобы нести факел, освещающий путь другим, нужно самому хорошо разбираться в постоянно меняющемся мире прикладного программного обеспечения. С годами приходит ясное понимание: в то время как функциональные аспекты программного обеспечения служат его сердцем, нефункциональные атрибуты, такие как безопасность, производительность и удобство обслуживания, становятся его спасательным кругом. Разработчику проще обнаружить и исправить ошибку в коде функции, чем барахтаться в мутных водах уязвимостей безопасности или падения производительности. Неудивительно, что многие разработчики, независимо от опыта, часто испытывают растерянность, сталкиваясь с этими побочными сложностями.

Безопасность не просто важна для приложения – это неоспоримый императив. Spring Security является лидером в обширной области фреймворков безопасности, что неудивительно, если учитывать широкое распространение и надежность экосистемы Spring в области корпоративных приложений. Тем не менее остается очевидная проблема – крутая кривая обучения при освоении Spring Security. Обильные, но разрозненные ресурсы в сети часто выглядят как кусочки пазла, которые отказываются складываться в единое целое, что сбивает с толку даже самых упорных учеников.

Именно эти проблемы в сочетании с примерами неправильного или, что еще хуже, уязвимого применения Spring Security и побудили меня написать первое издание данной книги. Моя цель была ясна: показать путеводную звезду любому желающему освоить Spring Security, будь то новичок или опытный поклонник Spring.

Во втором издании мы глубоко погрузимся в Spring Security, размышляя об инновациях, изменениях и опыте, накопленном сообществом с момента первого издания. Я улучшил то, что было, доба-

вил то, чего не было, и сделал акцент на том, что наиболее актуально сегодня. Я искренне надеюсь, что второе издание будет не просто полезной книгой, но и надежным спутником на вашем пути к созданию безопасных, надежных приложений. Я представлял себе эту книгу как маяк, который гарантирует, что вы не только сэкономите время, но и будете работать с уверенностью, зная, что ваши приложения устойчивы к постоянно меняющимся угрозам цифровой сферы.

Благодарности

Создание этой книги было увлекательным путешествием, в которое я не смог бы отправиться без коллективной мудрости, поддержки и опыта множества исключительных людей из моего окружения.

Прежде всего сердечное спасибо Даниэле, моей жене и путеводной звезде. Ее проницательность, постоянная поддержка и непоколебимая вера в мои силы сыграли неоценимую роль на протяжении всего проекта.

Сотрудники издательства Manning, работавшие над этой книгой, заслуживают особого упоминания. Благодаря их приверженности своему делу она обрела свои лучшие качества. Среди них я хотел бы выразить особую признательность Марине Михелс и Жану-Франсуа Морену. Их профессионализм, поддержка и бесценные советы значительно обогатили эту книгу.

Сердечная благодарность моей подруге Иоане Гёз – талантливому художнику, создавшему иллюстрации. Ее умение переводить мои абстрактные мысли в восхитительные визуальные образы добавило страницам уникальное очарование и позволило читателям найти повод для улыбки среди технических диаграмм.

Эта книга стала значительно лучше благодаря дотошному вниманию и отзывам многочисленных рецензентов. Их проницательные и конструктивные отзывы сыграли важную роль в уточнении содержания. Особая благодарность преданным рецензентам из Manning: Амарджиту Бхандалу, Асифу Икбалу, Козимо Дамиано Прете, Джеффу Уильямсу, Давиду Аскерову, Джастину Райзеру, Луиджи Рубино, Маноджу Кумару, Маркусу Гезелю, Мишелю Адлучи, Микаэлю Бистрему, Майклу Малеву, Наджибу Арифу, Патрику Ванджау, Ричарду Мейнсену, Сачину Хандиекару, Симеону Лейзерзону и Симону Скавца, а также моим друзьям, которые поделились своим опытом.

Наконец, спасибо моим коллегам в Endava: ваша постоянная поддержка, идеи и непоколебимая вера в мои начинания заставляли меня двигаться вперед. Я дорожу вашей поддержкой и глубоко ценю ее.

Все, кто прикоснулся к этому проекту, в большом и малом, пожалуйста, знайте, что ваш вклад стал одной из нитей, из которых соткан гобелен этой книги. Спасибо!

Об этой книге

Безопасность имеет первостепенное значение в разработке программного обеспечения, и очень важно позаботиться о ней с первых шагов. Во втором издании этой книги мы глубоко погружаемся в тематику использования Spring Security для обеспечения безопасности ваших проектов на уровне приложений. Знание Spring Security и его правильное применение необходимы каждому разработчику. Углубляться в разработку приложений без этих знаний – слишком большой риск.

Кому следует прочитать эту книгу?

Эта книга адресована разработчикам, использующим фреймворк Spring для корпоративных приложений. Хотя я адаптировал эту книгу для начинающих осваивать Spring Security, читателю потребуется базовое знание фреймворка Spring, включая

- использование контекста Spring;
- создание конечных точек REST;
- работу с источниками данных.

Глава 15 посвящена конфигурации безопасности реактивных приложений. Следовательно, перед прочтением этой главы важно иметь представление о реактивных приложениях и их разработке с использованием Spring. По мере необходимости я буду предлагать вам ссылки на дополнительные ресурсы для закрепления или дополнения необходимых тем.

Все примеры в этой книге написаны на Java. Учитывая популярность Java в экосистеме Spring, предполагается, что читатели уже имеют практические навыки программирования на этом языке. Но при желании все примеры можно без труда адаптировать к программированию на Kotlin.

Если вы чувствуете, что вам нужно освежить в памяти базовые знания перед началом работы с этой книгой, я настоятельно рекомендую прочитать еще одну мою книгу – «Spring быстро» (Питер, 2023).

Как организована эта книга

Я создал эту книгу, чтобы познакомить вас с обширной экосистемой Spring Security, от фундаментальных понятий до продвинутых при-

емов работы. Каждая часть книги естественным образом перетекает в следующую, делая ваш путь последовательным и комфортным. Книга состоит из следующих частей:

Часть I. Знакомство с Spring Security

В этой части я познакомлю вас с современным ландшафтом безопасности и Spring Security. Мы зложим основу для дальнейшего изучения, обсудив ключевую роль безопасности в современной цифровой эпоху и то, как Spring Security решает эту задачу.

Часть II. Настройка аутентификации

Здесь мы погрузимся в детали процесса аутентификации. Я расскажу о таких важных темах, как управление пользователями, протоколы паролей, решающая роль фильтров в безопасности веб-приложений и реализация аутентификации.

Часть III. Настройка авторизации

Далее мы перейдем от аутентификации к авторизации. Вместе мы рассмотрим авторизацию на уровне конечных точек, меры защиты от таких угроз, как CSRF, и управление CORS, а также изучим более сложную авторизацию и фильтрацию на уровне методов.

Часть IV. Реализация OAuth 2 и OpenID Connect

В этой части я проведу вас по миру OAuth 2 и OpenID Connect. Вы узнаете об их значении и настроите серверы OAuth 2, серверы ресурсов и клиентов, тем самым укрепив безопасность вашего приложения.

Часть V. Переход к реактивному программированию

Здесь я познакомлю вас с парадигмой реактивного программирования, подробно описав, как защитить реактивные приложения, чтобы вы могли гарантировать, что ваши асинхронные операции останутся защищенными от несанкционированного доступа.

Часть VI. Тестирование конфигураций безопасности

Я отмечаю и объясняю необходимость тестирования перед развертыванием. Мы рассмотрим различные методы тестирования, позволяющие убедиться, что ваши конфигурации безопасности функционируют точно так, как задумано.

Приложения

Приложения содержат ссылки на ресурсы официальной документации и списки литературы для дополнительного чтения.

Хотя я задумал эту книгу как последовательность взаимосвязанных тем, читатели, имеющие некоторый опыт работы с Spring Security, могут сразу перейти к интересующим их главам. Однако имейте

в виду, что эти главы могут опираться на содержание предыдущих глав. Если вы уже знакомы с основами Spring Security, попробуйте начать с части III или IV для изучения основ OAuth 2 и OpenID. Если вы интересуетесь реактивным программированием, можете сразу перейти к части V.

Независимо от того, с чего вы начнете, убедитесь, что полностью усвоили текущую тему, прежде чем переходить к следующей главе.

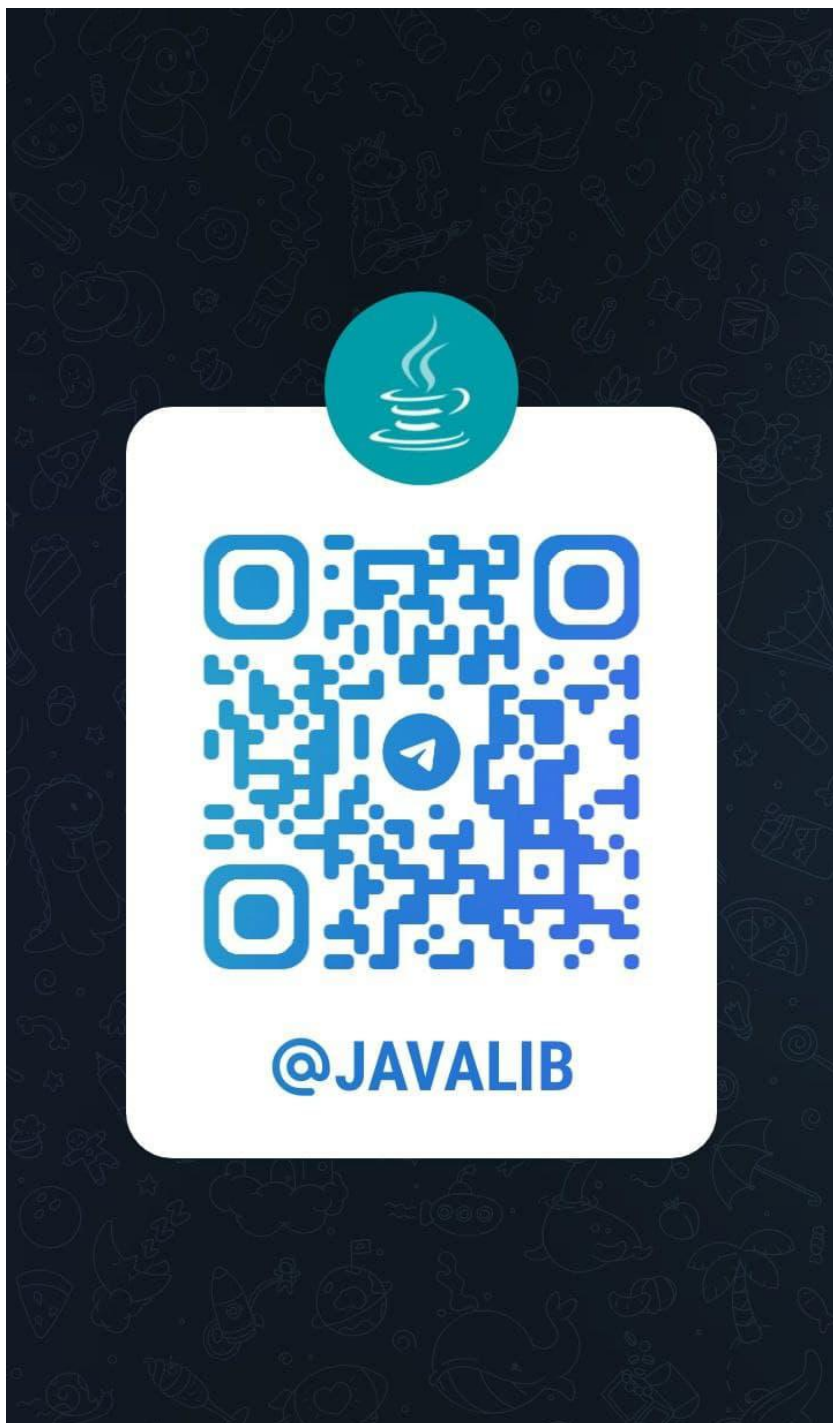
О примерах кода

В этой книге представлено более 70 проектов, над которыми мы будем работать, начиная с главы 2 и до главы 18. При работе над конкретным примером я упоминаю название проекта, который реализует пример. Я рекомендую вам попробовать написать свой собственный пример с нуля в соответствии с объяснениями в книге, а затем воспользоваться примером только для сравнения вашего решения с моим. Такой подход поможет вам лучше понять конфигурации безопасности, которые вы изучаете.

Каждый из проектов создан с помощью Maven, что упрощает их импорт в любую IDE. Для разработки проектов я использовал IntelliJ IDEA, но вы можете открывать и запускать их в Eclipse, STS, NetBeans или любой другой среде разработки по вашему выбору. Приложение поможет вам освежить знания о том, как создать проект Spring Boot.

Эта книга содержит множество примеров исходного кода как в виде отдельных листингов, так и в тексте. В обоих случаях исходный код выделен шрифтом фиксированной ширины. Во многих случаях исходный код был переформатирован; в него добавлены переносы строк и изменены отступы для оптимального размещения на странице книги. В редких случаях ширины страницы было недостаточно, и в листинги пришлось добавить маркеры продолжения строки (➡). Кроме того, из листинга удалены комментарии, если исходный код описан в тексте. Многие листинги сопровождаются пояснениями к коду.

Вы можете скопировать исполняемые фрагменты кода из liveBook (онлайн-версии этой книги) по адресу <https://livebook.manning.com/book/spring-security-in-action-second-edition>. Полный код примеров размещен в файловом архиве перевода книги на сайте издательства «ДМК Пресс».



<https://t.me/javallib>

Об авторе



Лауренциу Спилкэ начал свою карьеру в 2007 г. и сегодня является опытным разработчиком программного обеспечения и наставником. В настоящее время он занимает должность главного консультанта по разработке в Endava. В этой роли он возглавляет всемирно известные проекты, применяемые по всему миру. Лауренциу всегда ставил перед собой две цели: поставка первоклассного программного обеспечения и передача знаний коллегам-разработчикам.

На протяжении всей своей карьеры Лауренциу горячо верил, что необходимо заниматься не только созданием высококачественного программного обеспечения, но и развитием культуры обмена знаниями и непрерывного обучения. Это убеждение привело к разработке учебных курсов, сосредоточенных на технологиях Java. За последнее десятилетие он передал знания более чем 3000 студентов и активно участвовал в разработке учебных курсов на факультете математики и информатики Бухарестского университета.

Лауренциу также хорошо известен как автор трех книг, посвященных программированию на Java: *Spring Security in Action* (эта книга), *Spring Start Here* (в русском переводе – «Spring быстро») и *Troubleshooting Java* (в русском переводе – «Java: устранение проблем»). Эти книги раскрывают преподавательский талант автора и способствуют углубленному изучению языка Java и сопутствующих технологий.

Он выступает как наставник в разных странах мира. От Нью-Йорка и Сан-Франциско до исторических мест Варшавы, Белграда и Берлина – везде Лауренциу проводил презентации, обучающие программы и семинары, которые нашли благосклонный прием у международной аудитории.

Стремясь сделать знания максимально доступными, Лауренциу также создал канал на YouTube ([youtube.com/@laurspilca](https://www.youtube.com/@laurspilca)), посвященный Java и смежным технологиям. На этой платформе начинающие и опытные разработчики могут изучить широкий спектр тем, о которых рассказано в фирменном легком стиле Лауренциу.

В нерабочее время Лауренциу увлекается путешествиями, музыкой и подводным плаванием. Склонность Лауренциу к исследованиям и открытиям одинаково дает о себе знать как при исследовании глубин океана, так и при изучении тонкостей кода.

О техническом редакторе

Жан-Франсуа Морен – старший разработчик Java и архитектор в Университете Лавала в Квебеке, Канада. Он имеет степень бакалавра по математике, степень магистра по информатике и шесть сертификатов Sun/Oracle Java, включая Java SE 17 Developer и JAVA EE 6 Enterprise Architect. Он опытный преподаватель Java и постоянно сотрудничает с издательством Manning.

Об иллюстрации на обложке

Изображение на обложке книги называется «Homme de Murcie», или «Мужчина из Мурсии», и взято из коллекции Жака Грассе де Сен-Совера, опубликованной в 1797 г. Каждая иллюстрация прорисована до малейших деталей и раскрашена вручную.

В те дни по одежде можно было легко определить, где живут люди, чем они занимаются или какое положение занимают в обществе. Издательство Manning стремится отразить свое восхищение изобретательностью и инициативностью компьютерного бизнеса с помощью обложек книг, основанных на богатом разнообразии региональной культуры многовековой давности.

Часть I

Знакомство со Spring Security

Вы готовы к погружению в мир Spring Security? Давайте отправимся в это путешествие вместе! В первой части книги мы подготовим прочный фундамент будущих знаний.

Глава 1 познакомит вас с миром Spring Security. Постепенно мы углубимся в суть безопасности программного обеспечения, попытаюсь ответить на важные вопросы «Что такое безопасность ПО?» и «Почему она имеет первостепенное значение?». Кроме того, мы наметим траекторию обучения, чтобы вы знали, с чем столкнетесь в следующих главах.

Глава 2 посвящена практическим занятиям. Вы создадите свой первый проект Spring. Если вы задумывались об архитектуре и устройстве классов, заложенных в основу Spring Security, эта глава даст вам общее представление. Но речь идет не только о понимании базовых механизмов. Мы продвинемся на шаг вперед, пройдя через процесс переопределения конфигураций по умолчанию. Для этого мы детально изучим настройку профиля пользователя, улучшение авторизации в различных конечных точках, различные методы конфигурации, определение собственной логики аутентификации и эффективное использование нескольких классов конфигурации.

После прочтения первых двух глав вы получите не только ясное понимание теоретических основ Spring Security, но и работающее приложение, защищенное с его помощью. Иными словами, за короткий промежуток времени вы найдете ответы на ключевые вопросы «почему» и «как».

Безопасность сегодня



Темы этой главы:

- что такое Spring Security и какие задачи можно решить с его помощью;
- что такое безопасность ПО;
- почему безопасность ПО важна и почему заботиться о ней должны именно вы.

Разработчики все больше осознают потребность рынка в безопасном ПО и берут на себя ответственность за безопасность с самого начала разработки программного обеспечения. Как правило, приложения разрабатывают для того, чтобы решать определенные бизнес-задачи. Принято считать, что решение стандартной бизнес-задачи состоит из извлечения, обработки и отображения данных пользователю. Но эта традиционная точка зрения на функциональность ПО имеет досадный недостаток – она не раскрывает многие важные факторы, влияющие на использование приложения в реальной жизни. Даже если приложение работает правильно с точки зрения пользователя и делает то, что пользователь ожидает в плане функциональности, конечный результат зависит от множества скрытых аспектов.

Нефункциональные качества программного обеспечения, такие как производительность, масштабируемость, доступность и безопасность, а также некоторые другие, влияют на работу приложения в краткосрочной и долгосрочной перспективе. Если про-

игнорировать эти факторы на ранней стадии разработки, впоследствии они могут нанести ущерб пользователю приложения (рис. 1.1). Более того, пренебрежение нефункциональными факторами может спровоцировать сбой в других системах (например, невольное участие в распределенной атаке типа «отказ в обслуживании» [DDoS]). Скрытый характер нефункциональных факторов делает их еще более опасными.

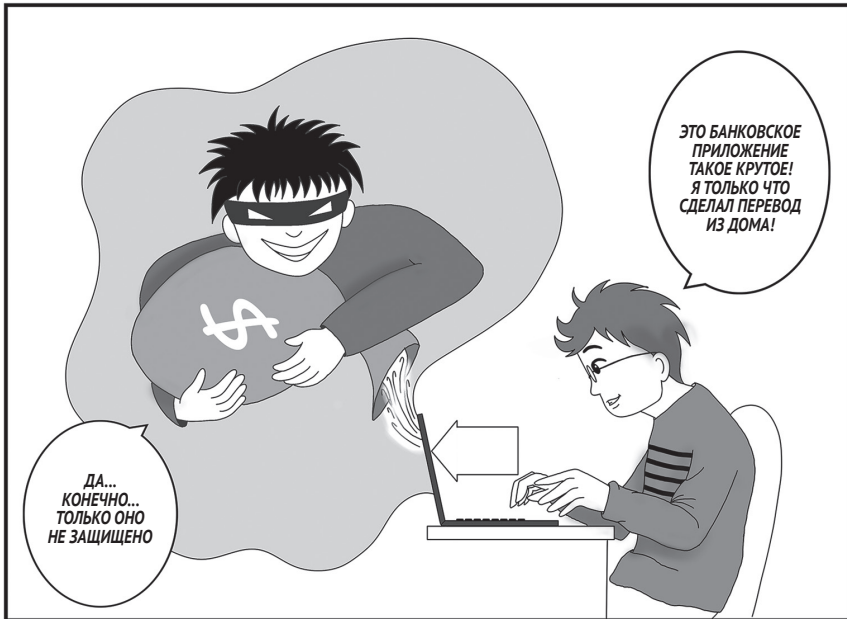


Рис. 1.1 Пользователи обычно сосредоточены на том, что должна делать система, – на ее функциональных аспектах. Иногда они могут уделять внимание такому нефункциональному показателю, как производительность, но редко обращают внимание на меры безопасности. Аспекты, которые не относятся напрямую к функциональности, часто остаются без внимания

При разработке приложения следует учитывать множество нефункциональных аспектов. На практике все они важны и требуют ответственного подхода при разработке ПО, но эта книга посвящена наиболее важному из них: безопасности. В данной книге вы узнаете, как защитить свое приложение с помощью Spring Security.

Эта глава в общих чертах обрисует ландшафт понятий, связанных с безопасностью. На протяжении всей книги мы будем работать над практическими примерами, и где это уместно, я буду ссылаться на примеры в этой главе. По возможности я буду стараться дать как можно более детальное описание. В этой и остальных главах вы найдете ссылки на дополнительные материалы (книги, статьи и документацию) для дальнейшего чтения.

1.1 Что такое Spring Security

В этом разделе мы обсудим взаимосвязь между Spring Security и Spring. Ее нужно хорошо понимать, прежде чем приступить к программированию. Если мы заглянем на официальный сайт (<https://spring.io/projects/spring-security>), то увидим, что Spring Security описывают как мощный фреймворк со множеством настроек для аутентификации и контроля доступа. Я бы сказал проще: это фреймворк, который значительно упрощает применение (или внедрение) средств безопасности для приложений Spring.

Spring Security является основным инструментом реализации безопасности на уровне приложений в Spring. Как правило, его цель – предложить вам настраиваемый способ реализации аутентификации, авторизации и защиты от распространенных атак. Spring Security – это программное обеспечение с открытым исходным кодом, выпущенное по лицензии Apache 2.0. Вы можете получить доступ к его исходному коду на GitHub по адресу <http://mng.bz/vPmJ>. Я настоятельно рекомендую вам также внести свой вклад в проект.

ПРИМЕЧАНИЕ Вы можете использовать Spring Security как для стандартных веб-сервлетов, так и для реактивных приложений, а также для приложений, не связанных с веб-технологиями. В этой книге мы будем использовать Spring Security совместно с последними версиями Java, Spring и Spring Boot (Java 21, Spring 6 и Spring Boot 3). Однако все примеры книги также работают с Java 17 (предыдущая версия с долгосрочной поддержкой).

Я могу предположить, что если вы открыли эту книгу, то вы работаете с приложениями Spring и заинтересованы в их защите. Spring Security, скорее всего, будет для вас лучшим выбором. Это практически готовая реализация безопасности на уровне приложений Spring. Однако Spring Security не защищает ваше приложение автоматически. Это не какая-то волшебная панацея, которая гарантирует отсутствие уязвимостей в приложении. Разработчикам нужно понимать, как конфигурировать и настраивать Spring Security в соответствии с потребностями своих приложений. Это зависит от многих факторов – от функциональных требований до архитектуры.

С технической точки зрения в реализации безопасности с помощью Spring Security в приложениях Spring нет ничего сложного. Если вам приходилось разрабатывать приложения Spring, то вы знаете, что философия фреймворка начинается с управления контекстом Spring. Вы определяете бины (bean) в контексте Spring, чтобы фреймворк управлял ими на основе указанных вами конфигураций.

Вы используете *аннотации*, чтобы сообщить Spring, что делать: раскрывать конечные точки, заключать методы в транзакции, перехватывать методы в аспектах и т. д. То же самое относится и к конфигурациям Spring Security, где вступает в игру часть фреймворка, связанная с безопасностью. Вы можете удобно использовать аннотации, бины и в целом стиль конфигурации в стиле Spring при определении безопасности на уровне приложения. В приложении Spring поведение, которое вам нужно защитить, определяется *методами*.

Чтобы наглядно представить, что такое безопасность на уровне приложения, вспомните свой дом и способ, которым вы разрешаете доступ к нему. Вы кладете ключ под коврик у входа? У вас вообще есть ключ от входной двери? Это зависит от вашего выбора. Та же концепция применима к приложениям, и Spring Security помогает вам разработать нужную функциональность. Это головоломка, которая допускает множество вариантов построения точного образа, описывающего вашу систему. Вы можете оставить свой дом полностью незащищенным или запретить входить в него вообще кому угодно, кроме себя.

Вы можете ограничиться ключом под ковриком или выбрать сложную систему безопасности, состоящую из сигнализации, видеокамер и замков. При разработке приложений ваш выбор столь же богат, но, как и в реальной жизни, чем больше сложности вы добавляете, тем дороже это обходится. В приложении эта стоимость сводится в основном к тому, как безопасность влияет на удобство обслуживания и производительность.

Но как тогда использовать Spring Security с приложениями Spring? Как правило, на уровне приложения разработчики наиболее часто принимают решение о том, разрешено ли кому-либо выполнять действие или использовать часть данных. На основе выбранной конфигурации вы пишете компоненты Spring Security, которые перехватывают запросы и проверяют наличие разрешений на доступ к защищенным ресурсам у источника запросов. Разработчик настраивает компоненты так, чтобы они делали именно то, что требуется. Если вы решили установить охранную систему, то сперва убедитесь, что она сконфигурирована защищать как окна, так и двери. Если вы забыли установить датчики на окна, это не вина системы, что она не сработала, когда кто-то полез через окно.

Другая область применения компонентов Spring Security охватывает хранение данных, а также передачу данных между различными частями системы. Перехватывая вызовы к этим частям системы, компоненты могут воздействовать на данные. Например, когда речь идет о хранении данных, эти компоненты могут применять алгоритмы шифрования или хеширования. Шифрование обеспечивает доступ к данным только для привилегированных сущностей.

В приложении Spring разработчик должен самостоятельно добавить и настроить компонент для выполнения упомянутой части работы везде, где это необходимо. Spring Security предоставляет нам *контракт*, благодаря которому мы знаем, в какой реализации нуждается фреймворк, и мы пишем эту реализацию в соответствии со структурой приложения. То же самое можно сказать и о передаче данных.

На практике вы столкнетесь со случаями, когда два взаимодействующих компонента не доверяют друг другу. Как первый компонент может узнать, что именно второй компонент отправил определенное сообщение и оно не было отправлено кем-то другим? Представьте, что вы разговариваете по телефону с кем-то, кому вы должны предоставить личную информацию. Как вы убедитесь, что на другом конце провода действительно находится доверенное лицо, имеющее право получить эти данные, а не кто-то посторонний? Та же ситуация актуальна и для вашего приложения. Spring Security предоставляет компоненты, способные решать эту задачу несколькими способами, но вы должны четко понимать, какой вариант выбрать, как его настроить и затем применить его в своей системе. Если коротко, Spring Security перехватывает сообщения и обеспечивает проверку безопасности связи до того, как приложение использует какие-либо отправленные или полученные данные.

Назначение любого фреймворка, включая Spring, – сделать так, чтобы вам пришлось писать меньше кода для реализации желаемой функциональности. В этом и заключается смысл Spring Security. Он дополняет Spring, избавляя вас от написания лишнего кода при реализации одного из самых важных аспектов приложения – безопасности. Spring Security предоставляет готовую функциональность, что позволяет избежать написания шаблонного кода или многократного написания одной и той же логики от приложения к приложению. Однако он также позволяет вам настраивать любой из его компонентов, тем самым обеспечивая большую гибкость. Кратко резюмируем сказанное:

- мы используем Spring Security для встраивания безопасности на уровне приложения с помощью стандартных средств Spring, под которыми я подразумеваю использование аннотаций, биннов, языка выражений Spring (SpEL) и т. д.;
- Spring Security – это фреймворк, позволяющий вам создавать безопасность на уровне приложений. Однако вам как разработчику предстоит понять принцип работы и правильно использовать Spring Security. Фреймворк сам по себе не защищает приложение или конфиденциальные данные ни в состоянии хранения, ни в процессе передачи;
- в этой книге вы найдете информацию, необходимую для эффективного использования Spring Security.

Альтернативы Spring Security

Эта книга рассказывает о Spring Security, но, как и в случае с любым решением, я всегда предпочитаю иметь общее представление о наборе инструментов. Никогда не забывайте изучать альтернативы. Со временем я убедился в том, что в технологиях безопасности нет однозначно правильного или неправильного решения. Поговорка «Все относительно» применима и здесь!

У вас не будет большого выбора альтернатив Spring Security, когда дело касается защиты приложения Spring. Один из вариантов, заслуживающих внимания, – это Apache Shiro (<https://shiro.apache.org>). Он обеспечивает гибкость настроек и легко интегрируется с приложениями Spring и Spring Boot. Apache Shiro иногда является хорошей альтернативой подходу Spring Security.

Если вы уже работали с Spring Security, то вам не составит труда освоить Apache Shiro. Он предлагает собственные аннотации и дизайн для веб-приложений на основе HTTP-фильтров, что значительно упрощает работу с веб-приложениями. Кроме того, с помощью Shiro вы можете защитить не только веб-приложения, но и другие объекты – от небольших командных и мобильных приложений до крупных корпоративных систем. И хотя он прост, его возможностей достаточно для решения широкого спектра задач – от аутентификации и авторизации до криптографии и управления сессиями.

Впрочем, вы можете обнаружить, что функциональности Apache Shiro недостаточно для нужд вашего приложения. Spring Security – это не просто молоток, а целый набор инструментов. Он предлагает более широкий спектр возможностей и разработан специально для приложений Spring. У него большое сообщество активных разработчиков, и он постоянно совершенствуется.

1.2 *Что такое безопасность программного обеспечения?*

Программные системы управляют большими объемами данных, значительную часть которых можно считать конфиденциальной, особенно в некоторых странах, где действует, например, Европейский общий регламент по защите данных (GDPR). Любая информация, которую вы как пользователь считаете личной, является конфиденциальной для вашего ПО. Конфиденциальной может считаться и относительно безвредная информация, такая как номер телефона, адрес электронной почты или идентификационный номер, хотя мы обычно в первую очередь думаем о более рискованных данных, таких как номер и фамилия владельца кредитной карты. Приложение должно гарантировать, что нет никакой возможности доступа к этой

информации, ее изменения или перехвата. Ни одна сторона, кроме пользователей, для которых эти данные предназначены, не должна иметь возможности каким-либо образом взаимодействовать с ней. В широком смысле в этом и заключается суть безопасности.

ПРИМЕЧАНИЕ GDPR вызвал много шума во всем мире после своего введения в 2018 г. В целом он представляет собой набор европейских законов, которые относятся к защите данных и дают людям больше контроля над своими личными данными. GDPR применяется к владельцам систем, у которых есть пользователи в Европе. Владельцы таких приложений рискуют получить значительные штрафы, если не будут соблюдать установленные правила.

Безопасность принято применять слоями, причем каждый слой требует своего подхода. Сравните эти слои с защищенным замком (рис. 1.2). Хакеру необходимо обойти несколько препятствий, чтобы получить ресурсы, управляемые приложением. Чем лучше вы защи-

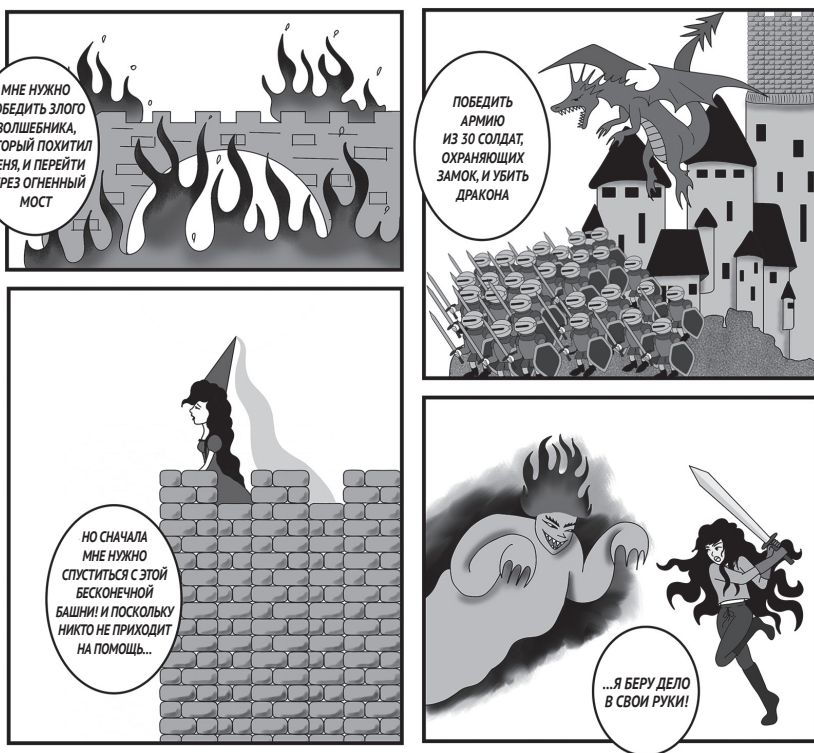


Рис. 1.2 Темный волшебник (хакер) должен обойти несколько препятствий (слоев безопасности), чтобы украсть Волшебный меч (пользовательские ресурсы) у Принцессы (вашего приложения)

тите каждый слой, тем ниже вероятность того, что человек с недобрыми намерениями сможет получить доступ к данным или выполнить несанкционированные операции.

Безопасность – сложная тема. В случае программной системы безопасность существует не только на уровне приложения. Например, на уровне сети учитываются особые риски и применяются специальные приемы, в то время как безопасность хранения данных организована совсем иначе. Аналогично применяется другая философия развертывания и т. д. Spring Security – это фреймворк, который обеспечивает безопасность на уровне приложения. В этом разделе я дам общий обзор безопасности на уровне приложения и поясню, что из этого следует.

Безопасность на уровне приложения (рис. 1.3) охватывает все, что приложение должно делать для защиты среды, в которой оно выполняется, а также данных, которые оно обрабатывает и хранит. Обратите внимание, что речь идет не только о данных, которые затрагивает и использует приложение. Приложение может содержать уязвимости, которые позволяют злоумышленнику повлиять на всю систему!

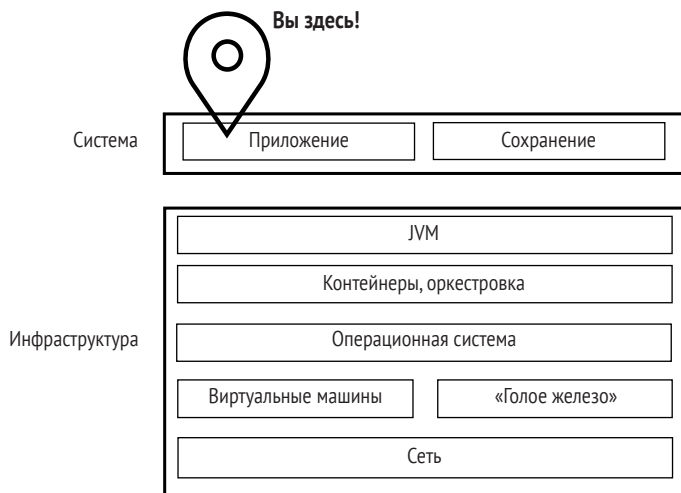


Рис. 1.3 Мы реализуем безопасность иерархически, причем каждый уровень строится на основе предыдущего. Эта книга описывает применение Spring Security – фреймворка, который обеспечивает безопасность на уровне приложения (самый верхний уровень)

Для лучшего понимания проанализируем несколько практических случаев. Мы рассмотрим ситуацию, в которой развернута система, как на рис. 1.4. Такая ситуация характерна для системы, разработанной с использованием микросервисной архитектуры, особенно если вы развертываете ее в нескольких зонах доступности в облаке.

ПРИМЕЧАНИЕ Если вы интересуетесь построением эффективных облачных приложений Spring, я настоятельно рекомендую книгу *Cloud Native Spring in Action* (Thomas Vitale, Manning, 2022). В этой книге автор освещает все необходимые аспекты, которые должен знать профессионал для разработки качественных облачных приложений Spring.

При такой архитектуре сервисов и микросервисов мы можем столкнуться с различными уязвимостями, поэтому нам следует проявлять осторожность. Как я уже говорил, безопасность – это сквозная задача, которую нужно решать на всех уровнях. Планируя меры безопасности на одном из уровней, старайтесь предполагать, насколько это возможно, что вышележащего уровня не существует. Вернемся к аналогии с замком на рис. 1.2. Если вы отвечаете за уровень с 30 стражниками, ваша задача – подготовить их максимально хорошо. И вы делаете это, хотя прежде чем добраться до них, кому-то нужно будет перейти огненный мост.

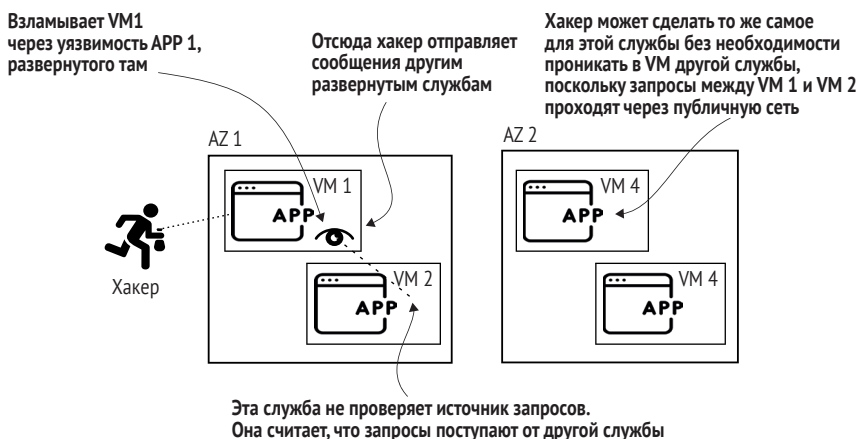


Рис. 1.4 Если злоумышленнику удастся получить доступ к виртуальной машине (VM), то при отсутствии защиты на уровне приложений хакер может получить контроль над другими приложениями в системе. Если в системе организована связь между двумя разными зонами доступности (availability zones, AZ), злоумышленнику будет легче перехватывать сообщения. Эта уязвимость позволяет ему похищать данные или выдавать себя за пользователей

Имея это в виду, давайте предположим, что злоумышленник может войти в виртуальную машину (VM), на которой размещено первое приложение. Давайте также предположим, что второе приложение не проверяет запросы, отправленные первым приложением. Злоумышленник может воспользоваться этой уязвимостью и управлять вторым приложением, выдавая себя за первое.

Допустим также, что мы развернули две службы в двух разных местах. Тогда злоумышленнику не нужно проникать в одну из VM, поскольку он может вклиниться в канал обмена данными между двумя приложениями.

ПРИМЕЧАНИЕ *Зона доступности* (AZ на рис. 1.4) с точки зрения развертывания в облаке – это отдельный центр обработки данных. Этот центр обработки данных расположен достаточно далеко географически (и имеет другие зависимости) от других центров обработки данных того же региона, так что в случае отказа одной зоны доступности вероятность сбоя других зон минимальна. С точки зрения безопасности особого внимания требует трафик между двумя разными центрами обработки данных, поскольку он часто проходит через общедоступную сеть.

Ранее я упоминал об аутентификации и авторизации. Они присутствуют в большинстве приложений. С помощью *аутентификации* приложение идентифицирует пользователя (человека или другое приложение), т. е. отличает его от множества других пользователей. Идентификация дает возможность узнать, что разрешено делать данному пользователю – а это уже *авторизация*. Я буду очень подробно говорить об аутентификации и авторизации начиная с главы 3 и далее на протяжении всей книги.

При разработке приложений мы часто сталкиваемся с необходимостью реализовать авторизацию в различных сценариях. Рассмотрим другую ситуацию: многие приложения ограничивают доступ пользователей к определенным функциям. Сначала нам нужно определить, *от кого* приложение получило запрос на доступ к функции – это аутентификация. Нам также необходимо знать привилегии, чтобы принять решение о допуске пользователя к этой функции. По мере усложнения системы вы будете сталкиваться с различными ситуациями, которые требуют разных способов аутентификации и авторизации.

Рассмотрим ситуацию, когда вам нужно авторизовать определенный компонент системы для доступа к подмножеству данных или выполнения операций от имени пользователя. Допустим, принтеру нужен доступ для чтения документов пользователя, отправленных на печать. Стоит ли вам просто поделиться учетными данными пользователя с принтером? Но это дает принтеру больше прав, чем нужно! Вдобавок так вы раскрываете учетные данные пользователя. Есть ли правильный способ предоставить принтеру доступ к документам, не выдавая его за пользователя? На самом деле это фундаментальные вопросы, с которыми приходится сталкиваться при разработке приложений. В этой книге вы не только найдете на

них ответы, но и познакомитесь с примерами применения Spring Security.

В зависимости от архитектуры аутентификация и авторизация могут быть организованы как на уровне всей системы, так и на уровне любого из компонентов. И как вы увидите далее в этой книге, с Spring Security иногда предпочтительно использовать авторизацию даже для разных уровней одного и того же компонента. В главе 11 мы подробнее поговорим о безопасности методов, которая основана на таком подходе. Структура безопасности приложения становится еще сложнее, когда у вас есть предопределенный набор ролей и полномочий.

Я также хотел бы обратить ваше внимание на хранение данных. Данные в состоянии покоя (хранения) увеличивают ответственность приложения. Ваше приложение не должно хранить все свои данные в читаемом формате. Иногда приложению необходимо хранить данные либо в зашифрованном виде с помощью закрытого ключа, либо в хешированном виде. *Секреты*, такие как учетные данные и закрытые ключи, также могут считаться данными в состоянии покоя. Их следует тщательно хранить, обычно в *хранилище секретов* (secret vault).

ПРИМЕЧАНИЕ С точки зрения безопасности данные можно разделить на два типа: *в состоянии покоя* (at rest) и *в процессе передачи* (in transition). В этом контексте к данным в состоянии покоя относят данные в хранилище компьютера, или, другими словами, «неподвижные» данные. К данным в процессе передачи относят все данные, которые передаются из одной точки в другую. Поэтому следует применять различные меры безопасности в зависимости от типа данных.

Наконец, исполняемое приложение также должно управлять своей внутренней памятью. Это может показаться странным, но данные, хранящиеся в области памяти, выделенной приложению, также могут содержать уязвимости. Иногда конструкция класса позволяет приложению в течение длительного времени хранить в памяти конфиденциальные данные, такие как учетные данные или закрытые ключи. В подобных случаях пользователь, у которого есть привилегия сделать дамп памяти, может извлечь критические данные и затем использовать их в своих целях.

С помощью этих кратких примеров я постарался показать, что мы подразумеваем под безопасностью приложений, а также проиллюстрировать сложность данной задачи. Безопасность программного обеспечения – запутанная тема. Разработчику, желающему стать экспертом в области безопасности, необходимо понимать (а также применять), а затем тестировать решения для всех слоев, которые

взаимодействуют в системе. Однако в этой книге мы сосредоточимся только на безопасности в контексте Spring Security. Вы узнаете, где этот фреймворк применяется, а где нет, как он помогает и почему вам следует его использовать. Конечно, впереди вас ждут практические примеры, которые вы сможете адаптировать к своим собственным уникальным вариантам использования.

1.3 *Почему так важна безопасность?*

Лучший способ осознать важность безопасности – представить себя на месте пользователя. Как и любой другой человек, вы используете приложения, и они имеют доступ к вашим данным. Они могут хранить, использовать и изменять ваши данные или даже раскрывать их. Вспомните обо всех приложениях, которые вы используете, от электронной почты до управления банковскими счетами. Как бы вы оценили конфиденциальность данных, которыми оперируют все эти системы? Как насчет действий, которые вы можете выполнять с помощью этих систем? Некоторые действия и данные важнее других. Вас не очень волнуют некоторые из них, в то время как другие более значимы. Возможно, для вас не так важно, могут ли посторонние прочесть некоторые из ваших писем. Но я уверен, что вы расстроитесь, если узнаете, что кто-то может опустошить ваши банковские счета.

Итак, вы представили себя на месте пользователя. Теперь попытайтесь увидеть более объективную картину. Те же данные или действия могут иметь другую степень конфиденциальности для других людей. Некоторые люди могут беспокоиться о конфиденциальности переписки по электронной почте намного больше вас. Ваше приложение должно гарантировать необходимую степень защиты данных. Любая утечка, которая позволяет использовать данные и функции, а также приложение, для воздействия на другие системы, считается уязвимостью, и вам нужно ее устранить. Пробелы в безопасности имеют свою цену, которую, я уверен, вы не готовы платить. Сейчас вы могли подумать о деньгах. Но величина ущерба, причиненного злоумышленниками, не обязательно связана с деньгами напрямую. Речь идет не только о хищении денег с банковского счета или использовании услуги без оплаты за нее. Имидж бренда или компании также ценен, и потеря хорошего имиджа обходится очень дорого – иногда даже дороже, чем расходы, напрямую связанные со взломом системы! Доверие пользователей к вашему приложению – один из его самых ценных активов.

Рассмотрим несколько вымышленных ситуаций. Постарайтесь поставить себя на место пользователя и представить его реакцию.

Как это может повлиять на организацию, ответственную за программное обеспечение?

- Приложение бэк-офиса должно управлять внутренними данными организации, но каким-то образом информация стала известна посторонним.
- Пользователи приложения для совместных поездок обнаружили, что с их счетов списываются деньги за поездки, которые они не совершали.
- После обновления пользователи мобильного банковского приложения видят транзакции, принадлежащие другим пользователям.

В первой ситуации может пострадать организация, использующая программное обеспечение, а также ее сотрудники. В некоторых случаях компания может потерять значительную сумму денег или понести репутационный ущерб. В этой ситуации у пользователей нет возможности сменить приложение, но организация может решить сменить поставщика своего программного обеспечения.

Во втором случае пользователи, вероятно, решат сменить поставщика услуг. Имидж компании, разрабатывающей приложение, серьезно пострадает. Прямые денежные затраты в этом случае намного меньше, чем затраты в плане имиджа. Даже если платежи будут возвращены пострадавшим пользователям, приложение все равно потеряет часть клиентов. Это повлияет на доходность бизнеса и может даже привести к банкротству. А в третьем случае банк может столкнуться не только с утратой доверия клиентов, но и с юридическими последствиями.

В большинстве этих сценариев вложения в безопасность обойдутся дешевле, чем потенциальные издержки уязвимости вашей системы. Во всех примерах злоумышленникам достаточно лишь небольшой уязвимости. Для первого примера это может быть взломанная аутентификация или *подделка межсайтовых запросов* (cross-site request forgery, CSRF). Для второго и третьего примеров это может быть отсутствие контроля доступа к методам. И для всех примеров это может быть комбинация уязвимостей.

Конечно, мы можем пойти еще дальше и обсудить безопасность в системах, связанных с обороной. Если вы считаете деньги важными, то как насчет стоимости человеческих жизней? А что, если пострадает медицинская система? Как насчет систем, которые управляют ядерными реакторами? Вы можете существенно снизить риски, позаботившись о безопасности на самых первых этапах разработки вашего приложения и выделяя достаточно времени специалистам по безопасности для разработки и тестирования механизмов безопасности.

ПРИМЕЧАНИЕ Урок, извлеченный из опыта тех, кто потерпел неудачу до вас, заключается в том, что стоимость ущерба от хакерской атаки обычно выше, чем затраты на избежание уязвимости.

В оставшейся части этой книги вы увидите примеры применения Spring Security для предотвращения ситуаций, подобных упомянутому выше. Я думаю, что никакие слова о значении безопасности не будут лишними, но мир вокруг нас не идеален. Когда вам придется идти на компромисс в отношении безопасности вашей системы, постарайтесь правильно оценить свои риски.

1.4 *Что вы узнаете из этой книги?*

Эта книга опирается на практический подход к изучению Spring Security. На протяжении оставшейся части книги мы будем изучать принцип работы Spring Security с помощью примеров различной степени сложности. Чтобы извлечь максимальную пользу из этой книги, вы должны быть знакомы с программированием на Java, а также в общих чертах знать фреймворк Spring. Если вы еще не использовали Spring или пока вам недостает опыта работы с ним, я рекомендую вам сначала прочитать другую мою книгу «Spring быстро» (Питер, 2023). После прочтения этой книги вы можете углубить свои знания с помощью книг «Spring в действии. Шестое издание» (Крейг Уоллс, ДМК, 2022), а также «Spring Boot по-быстрому» (Марк Хеклер, Питер, 2022).

В этой книге вы изучите:

- архитектуру и основные компоненты Spring Security и их применение для защиты вашего приложения;
- аутентификацию и авторизацию с помощью Spring Security, включая потоки OAuth 2 и OpenID Connect, и их применение в готовом к производству приложении;
- подходы к реализации безопасности с помощью Spring Security на разных уровнях вашего приложения;
- различные стили конфигурации и лучшие практики их использования в вашем проекте;
- использование Spring Security для реактивных приложений;
- тестирование реализаций безопасности.

Чтобы сделать процесс обучения более наглядным, каждая новая концепция сопровождается несколькими простыми примерами.

После прочтения этой книги вы научитесь применять Spring Security в наиболее распространенных сценариях и поймете, для каких случаев он подходит лучше всего. Я также настоятельно рекомендую

вам поработать над всеми примерами, которые сопровождают теоретическую часть.

Заключение

- Spring Security – приоритетный способ защиты приложений Spring. Он очень гибок и может применяться к разным стилям и архитектурам.
- Безопасность системы должна быть организована послойно, и для каждого слоя необходимо применять разные технологии защиты.
- Безопасность – это сквозная задача, которую следует начинать решать с первых шагов и на всем протяжении проекта.
- Обычно стоимость ущерба от хакерской атаки выше, чем затраты на изначальное устранение уязвимостей.
- Иногда даже самые незначительные ошибки могут причинить большой вред. Например, раскрытие конфиденциальных данных через журналы или сообщения об ошибках – это распространенный способ взлома приложений.

Spring Security в действии

Темы этой главы:

- создание вашего первого проекта со Spring Security;
- проектирование простых функций с использованием базовых компонентов для аутентификации и авторизации;
- базовая концепция и как ее использовать в данном проекте;
- применение базовых контрактов и понимание взаимосвязи между ними;
- написание пользовательских реализаций для основной функциональности;
- переопределение конфигураций Spring Boot по умолчанию для Spring Security.

Spring Boot появился как очередной этап эволюции разработки приложений с фреймворком Spring. Spring Boot устраняет необходимость писать с нуля все конфигурации и поставляется с набором предварительных настроек, поэтому вы можете переопределить только те конфигурации, которые не соответствуют вашим потребностям. Такой подход иногда называют *соглашением по конфигурации* (convention over configuration). На момент написания этой книги разработчики приложений активно использовали уже третью версию Spring Boot.

До Spring Boot разработчикам приходилось заново писать десятки строк кода для всех приложений, над которыми они работали. Эта ситуация была менее заметна в эпоху монолитных архитектур, когда такие конфигурации писали только один раз в начале разработки и потом к ним почти не возвращались. С появлением микросервисных архитектур разработчики сполна ощутили все неудобство шаблонного кода, который приходилось писать при настройке каждой службы. Если вам это кажется забавным, просто почитайте главу 3 книги *Spring in Practice* (Willie Wheeler and Joshua White, Manning, 2013). В этой главе показан процесс разработки веб-приложения с помощью Spring 3. Обратите внимание, сколько конфигураций там пришлось написать для одного небольшого одностраничного веб-приложения. Глава доступна по адресу <http://mng.bz/46la>.

По этой причине с появлением микросервисных архитектур резко возросла популярность Spring Boot. Этот инструмент выполняет автоматическую настройку проекта и сокращает время, необходимое для окончательной настройки. Можно сказать, что он реализует на практике современную философию разработки ПО.

В данной главе мы разработаем первое приложение, использующее Spring Security. Это отличный выбор для реализации безопасности на уровне приложения, созданного с помощью фреймворка Spring. Мы будем использовать Spring Boot и обсудим переопределение настроек по умолчанию. Изучение стандартных конфигураций дает отличное введение в Spring Security и заодно иллюстрирует концепцию аутентификации.

Начиная с первого проекта мы будем подробно обсуждать различные варианты аутентификации. В главах с 3 по 6 мы перейдем к более конкретным конфигурациям функций безопасности, которые вы увидите в этом первом примере. Вы также увидите различные способы применения этих конфигураций в зависимости от типа архитектуры. В текущей главе мы рассмотрим следующие шаги:

- 1 Создание начального проекта только с использованием Spring Security и веб-зависимостей, чтобы посмотреть, как он себя ведет, если не добавлена конфигурация. Вы сможете понять, чего следует ожидать от конфигурации по умолчанию для аутентификации и авторизации.
- 2 Добавление функциональности управления пользователями путем изменения настроек по умолчанию для определения пользователей и паролей.
- 3 По умолчанию приложение аутентифицирует все конечные точки. Это поведение тоже можно настроить.
- 4 Применение разных стилей одних и тех же конфигураций. Это поможет вам ближе познакомиться с современной практикой разработки приложений.

2.1 *Приступаем к первому проекту*

Давайте создадим первый проект, который заодно послужит первым практическим примером. Это будет небольшое веб-приложение, предоставляющее конечную точку REST. Вы увидите, как Spring Security без особых усилий защищает эту конечную точку с помощью аутентификации HTTP Basic. При использовании HTTP Basic веб-приложение аутентифицирует пользователя с помощью набора учетных данных (имя пользователя и пароль), которые приложение получает в заголовке HTTP-запроса.

ПРИМЕЧАНИЕ В конфигурации по умолчанию приложение использует два разных механизма аутентификации: HTTP Basic и Form Login. Я предпочитаю не спешить и расскажу про Form Login в последующих главах. Но если вы попытаетесь получить доступ к приложению через адрес в браузере, вы обнаружите, что ваше приложение реализует более приятную форму для аутентификации пользователя и не отображает по этой причине уродливое поле HTTP Basic. Я не хочу, чтобы это стало для вас неожиданностью, если вы решите поэкспериментировать с браузером, но пока мы ограничимся механизмом HTTP Basic.

После создания проекта и указания правильных зависимостей Spring Boot применяет конфигурации по умолчанию, включая запрос имени пользователя и пароля при запуске приложения.

ПРИМЕЧАНИЕ Проекты Spring Boot можно создать разными способами. Некоторые среды разработки предлагают возможность создания проекта напрямую. Для получения более подробной информации я рекомендую книги «Spring Boot по-быстрому» (Марк Хеклер, Питер, 2022) и *Spring Boot in Practice* (Somnath Musib, Manning, 2022) или другую мою книгу «Spring быстро» (Питер, 2023).

Примеры в этой книге основаны на исходном коде, доступном в файловом архиве книги. В каждом примере я также указываю зависимости, которые вам необходимо добавить в файл `pom.xml`. Вы можете (и я рекомендую вам это сделать) скачать исходный код проектов по адресу <https://www.manning.com/downloads/2105>. Готовые проекты помогут вам разобраться с примерами. Вы также можете использовать их для проверки своих окончательных решений.

ПРИМЕЧАНИЕ Примеры в этой книге не зависят от выбранного вами инструмента сборки. Вы можете использовать как Maven, так и Gradle. Я выполнял сборку всех примеров с помощью Maven просто для однообразия.

Первый проект также является самым маленьким. Это простое приложение, предоставляющее конечную точку REST, которую можно вызвать, а затем получить ответ, как показано на рис. 2.1. Этого проекта достаточно, чтобы вы изучили первые шаги разработки приложения с использованием Spring Security и Spring Boot. Пример иллюстрирует базовую архитектуру Spring Security для аутентификации и авторизации.

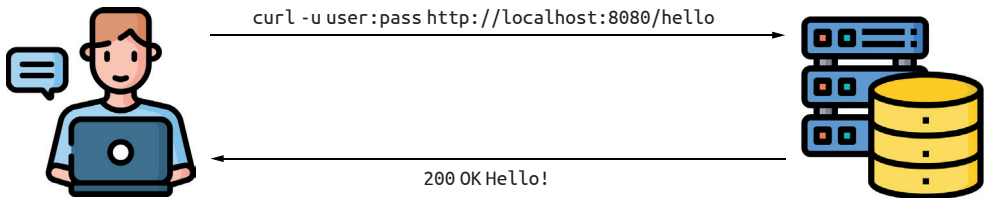


Рис. 2.1 Наше первое приложение использует механизм HTTP Basic для аутентификации и авторизации пользователя при доступе к конечной точке. Оно предлагает конечную точку REST по указанному маршруту (/hello). При успешном запросе оно выдает сообщение о состоянии HTTP 200 вместе с телом ответа. Этот экземпляр иллюстрирует механизмы аутентификации и авторизации по умолчанию, добавленные с помощью Spring Security

Создайте пустой проект и назовите его `ssia-ch2-ex1`. (Вы также найдете этот пример с тем же именем в других проектах.) Единственные зависимости, которые вам нужно написать для нашего первого проекта, – это `spring-boot-starter-web` и `spring-boot-starter-security`, как показано в листинге 2.1. После создания проекта обязательно добавьте эти зависимости в файл `pom.xml`. Основная цель работы над этим проектом – увидеть поведение приложения с конфигурацией по умолчанию, добавленной с помощью Spring Security. Вам предстоит разобраться, из каких компонентов состоит конфигурация и в чем заключается их назначение.

Листинг 2.1 Зависимости Spring Security для нашего первого веб-приложения

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
    
```

Мы могли бы запустить приложение прямо сейчас. Spring Boot применяет для нас настройку по умолчанию контекста Spring на основе зависимостей, которые мы добавляем в проект. Однако мы не

узнаем о безопасности ничего интересного, если у нас не будет хотя бы одной защищенной конечной точки. Давайте создадим простую конечную точку и вызовем ее, чтобы посмотреть, что произойдет. Для этого мы добавим класс в пустой проект и назовем этот класс `HelloController`. Затем добавим класс в пакет с именем `controllers` в основном пространстве имен проекта Spring Boot.

ПРИМЕЧАНИЕ Spring Boot сканирует компоненты только в пакете (и его подпакетах), содержащем класс, аннотированный `@SpringBootApplication`. Если вы аннотируете классы любым из стереотипных компонентов в Spring за пределами основного пакета, вы должны явно объявить местоположение с помощью аннотации `@ComponentScan`.

В листинге 2.2 класс `HelloController` определяет контроллер REST и конечную точку REST для нашего примера.

Листинг 2.2 Класс `HelloController` и конечная точка REST

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello!";
    }
}
```

Аннотация `@RestController` регистрирует компонент в контексте и сообщает Spring, что приложение использует этот экземпляр как веб-контроллер. Кроме того, аннотация указывает, что приложение должно задать тело HTTP-ответа из возвращаемого значения метода. Аннотация `@GetMapping` сопоставляет путь `/hello` с реализованным методом через запрос GET. После запуска приложения, помимо других строк в консоли, вы должны увидеть что-то, похожее на такую строку:

```
Using generated security password: 93a01cf0-794b-4b98-86ef-54860f36f7f3
```

Каждый раз, когда вы запускаете приложение, оно генерирует новый пароль и выводит этот пароль в консоли, как показано в предыдущем фрагменте кода. Вы должны использовать этот пароль для вызова любой из конечных точек приложения с аутентификацией HTTP Basic. Сначала попробуем вызвать конечную точку без использования заголовка `Authorization`:

```
curl http://localhost:8080/hello
```

ПРИМЕЧАНИЕ В этой книге мы используем cURL для вызова конечных точек во всех примерах. Я считаю cURL наиболее удобочитаемым решением. Но при желании вы можете использовать инструмент по своему выбору. Возможно, вы предпочитаете более удобный графический интерфейс. В этом случае Postman, Insomnia или Bruno – отличный выбор. Если в используемой вами операционной системе не установлен ни один из этих инструментов, установите его самостоятельно.

Так выглядит ответ на вызов:

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/hello"
}
```

Статус ответа – HTTP 401 Unauthorized. Это вполне ожидаемый результат, поскольку мы не использовали надлежащие учетные данные для аутентификации. При отсутствии дополнительных настроек Spring Security ожидает имя пользователя по умолчанию (user) с предоставленным паролем (в моем случае начинающимся с 93a01).

```
curl -u user:93a01cf0-794b-4b98-86ef-54860f36f7f3 http://localhost:8080/hello
```

Ответ на вызов:

Hello!

ПРИМЕЧАНИЕ Код состояния HTTP 401 Unauthorized немного неоднозначен. Обычно он используется для обозначения неудачной аутентификации, а не авторизации. Разработчики используют его при проектировании приложения для таких случаев, как отсутствие или неверные учетные данные. Для неудачной авторизации мы, вероятно, использовали бы статус 403 Forbidden. Обычно HTTP 403 означает, что сервер идентифицировал вызывающую сторону запроса, но у нее нет необходимых привилегий для вызова, который она пытается сделать.

После отправки правильных учетных данных мы видим в теле ответа значение, возвращенное методом `HelloController`, который мы определили ранее.

Вызов конечной точки с аутентификацией HTTP Basic

При вызове cURL вы можете задать базовое имя пользователя и пароль HTTP с помощью флага `-u`. За кулисами cURL кодирует строку `<username>:<password>` в Base64 и отправляет ее как значение заголовка `Authorization` с префиксом строки `Basic`. Разумеется, проще использовать флаг `-u`. Но также важно знать, как выглядит реальный запрос. Поэтому давайте попробуем вручную создать заголовок `Authorization`.

Сначала возьмите строку `<username>:<password>` и закодируйте ее с помощью Base64. Перед тем как приложение отправит вызов, нужно сформировать правильное значение для заголовка `Authorization`. Вы можете сделать это с помощью инструмента Base64 в консоли Linux. Вы также можете найти веб-страницу, которая кодирует строки в Base64, например <https://www.base64encode.org>. Этот фрагмент показывает команду в консоли Linux или Git Bash (параметр `-n` означает, что не нужно добавлять завершающий символ новой строки):

```
echo -n user:93a01cf0-794b-4b98-86ef-54860f36f7f3 | base64
```

Выполнение этой команды возвращает следующую строку в кодировке Base64:

```
dXNlcjo5M2EwMWNmMC03OTRiLTRiOTgtODZlZi01NDg2MGYzNmY3ZjM=
```

Теперь вы можете подставить значение в кодировке Base64 в качестве значения заголовка `Authorization` для вызова. Этот вызов должен сгенерировать тот же результат, что и вызов с опцией `-u`:

```
curl -H "Authorization: Basic
dXNlcjo5M2EwMWNmMC03OTRiLTRiOTgtODZlZi01NDg2MGYzNmY3ZjM="
localhost:8080/hello
```

Результат вызова:

```
Hello!
```

В этом проекте конфигурации безопасности по умолчанию не заслуживают обсуждения. Конфигурации по умолчанию обычно используют для того, чтобы убедиться, что заданы правильные зависимости. Они мало на что влияют в плане аутентификации и авторизации. Текущая реализация – совсем не то, что нам хотелось бы видеть в готовом к производству приложении. Но проект по умолчанию – отличный пример, с которого вы можете начать.

Благодаря этому примеру мы по крайней мере знаем, что Spring Security работает. Следующий шаг – изменить конфигурации так, чтобы они соответствовали требованиям нашего проекта. Сначала мы подробнее рассмотрим, какие настройки Spring Boot имеют отношение к Spring Security, а затем посмотрим, как можно переопределить конфигурации.

2.2 Обзор устройства классов Spring Security

Этот раздел посвящен основным компонентам архитектуры, которые принимают участие в процессе аутентификации и авторизации. Вам придется переопределить эти предварительно настроенные компоненты, чтобы они соответствовали потребностям вашего приложения. Я начну с описания того, как работает архитектура Spring Security в плане аутентификации и авторизации, а затем мы применим новые знания к следующим проектам в этой главе. Детальное обсуждение всех аспектов получилось бы слишком объемным и трудным для понимания, поэтому сейчас я расскажу о каждом компоненте лишь в общих чертах. Подробности о каждом из них вы узнаете в следующих главах.

В разделе 2.1 вы уже видели некие зачатки логики аутентификации и авторизации. У нас был пользователь по умолчанию, и мы получали случайный пароль каждый раз, когда запускали приложение. Мы могли использовать имя этого пользователя по умолчанию и пароль для вызова конечной точки. Но *где* реализована вся эта логика? Как вы, вероятно, уже знаете, Spring Boot настраивает некоторые компоненты, исходя из того, какие зависимости вы используете (т. е. соглашение по конфигурации, которое мы обсуждали в начале этой главы).

На рис. 2.2 показана общая картина взаимосвязей между основными компонентами в архитектуре Spring Security. В первом проекте эти компоненты имеют предварительно настроенную реализацию. В этой главе я покажу, какие настройки Spring Boot задает в вашем приложении с точки зрения Spring Security. Мы также обсудим взаимосвязи между сущностями, которые являются частью представленного потока аутентификации.

На рис. 2.2 показано, что

- 1 *фильтр аутентификации* (authentication filter) делегирует запрос менеджеру аутентификации и на основе ответа настраивает контекст безопасности;
- 2 *менеджер аутентификации* (authentication manager) использует провайдер аутентификации для обработки аутентификации;
- 3 *провайдер аутентификации* (authentication provider) реализует логику аутентификации;
- 4 *служба сведений о пользователе* (user details service) реализует ответственность за управление пользователями, которую провайдер аутентификации использует в логике аутентификации;
- 5 *кодировщик паролей* (password encoder) реализует управление паролями, которое провайдер аутентификации использует в логике аутентификации;
- 6 *контекст безопасности* (security context) сохраняет данные аутентификации после процесса аутентификации. Контекст

безопасности будет хранить данные до тех пор, пока действие не завершится. Обычно в приложении типа «один поток на запрос» это означает, что приложение отправит ответ обратно клиенту.

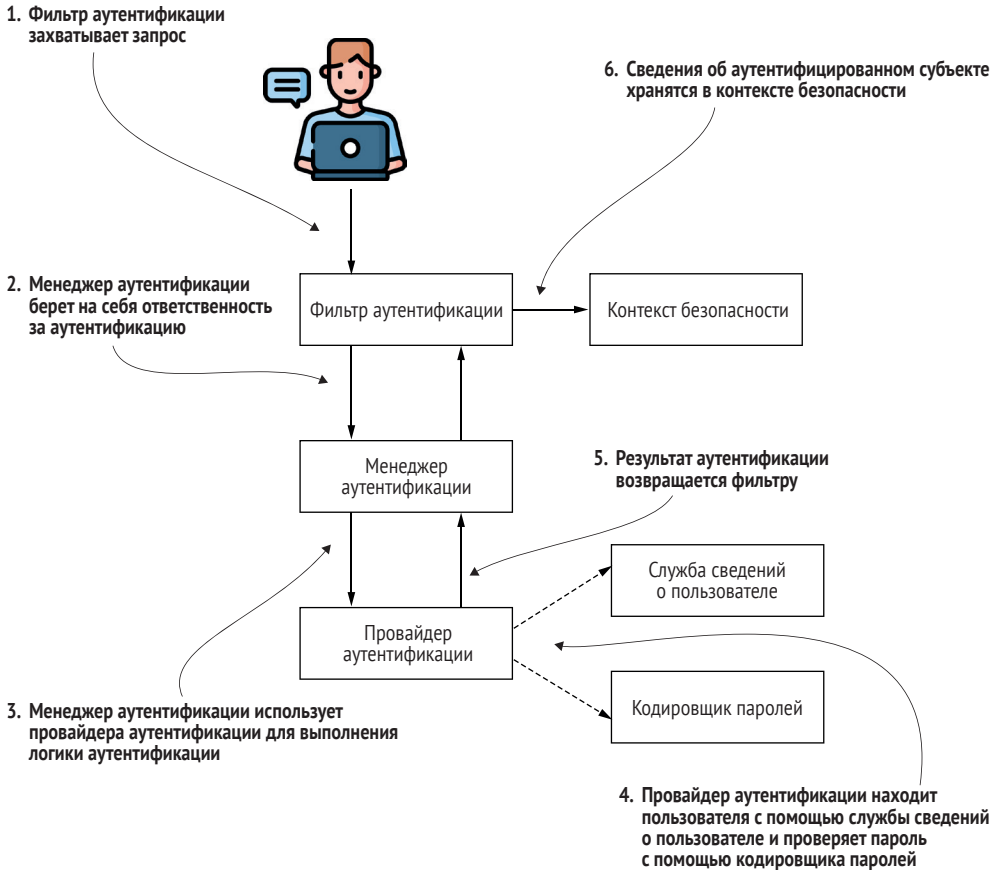


Рис. 2.2 Здесь изображены основные элементы, участвующие в процессе аутентификации Spring Security, и показаны связи между ними. Эта архитектура нужна для выполнения аутентификации с использованием Spring Security. На протяжении всей книги мы будем часто ссылаться на нее при рассмотрении различных стратегий аутентификации и авторизации

Далее я расскажу о следующих автоматически настроенных компонентах:

- `UserDetailsService`;
- `PasswordEncoder`.

Объект, реализующий интерфейс `UserDetailsService` с Spring Security, управляет сведениями о пользователях. До сих пор мы использовали реализацию по умолчанию, предоставляемую Spring Boot. Она регистрирует учетные данные по умолчанию только во

внутренней памяти приложения. Эти учетные данные по умолчанию – пользователь `user` с паролем по умолчанию, который является универсальным уникальным идентификатором (universally unique identifier, UUID). Пароль по умолчанию генерируется случайным образом при загрузке контекста Spring (при запуске приложения). В это время приложение выводит пароль в консоль, где вы можете его увидеть. Благодаря этому вы можете использовать его в примере, который мы недавно рассмотрели.

Эта реализация служит только для проверки работоспособности приложения и позволяет нам увидеть, что зависимость присутствует. Реализация хранит учетные данные в памяти – приложение не сохраняет учетные данные. Этот механизм подходит для примеров или проверки начальных версий приложения, но вам следует избегать его в приложении, готовом к производству.

Далее, у нас есть `PasswordEncoder`. Он делает две вещи:

- кодирует пароль (обычно с использованием алгоритма шифрования или хеширования);
- проверяет, совпадает ли пароль с существующим закодированным значением.

Даже если это не так очевидно, как в случае объекта `UserDetailsService`, `PasswordEncoder` является обязательным для потока аутентификации Basic. Простейшая реализация оперирует паролями в виде обычного текста и не кодирует их. Мы более подробно обсудим реализацию этого объекта в главе 4. На данный момент вы должны знать, что по умолчанию `PasswordEncoder` работает вместе с `UserDetailsService`. Когда мы заменяем реализацию `UserDetailsService`, мы также должны указать `PasswordEncoder`.

Spring Boot тоже выбирает метод аутентификации при настройке значений по умолчанию: аутентификация доступа HTTP Basic. Это самый простой метод аутентификации доступа.

Для аутентификации типа Basic клиенту достаточно отправить имя пользователя и пароль через заголовок HTTP Authorization. В значении заголовка клиент добавляет префикс Basic, за которым следует строка, содержащая имя пользователя и пароль, разделенные двоеточием (:) в кодировке Base64.

ПРИМЕЧАНИЕ Аутентификация HTTP Basic не обеспечивает конфиденциальность учетных данных. Base64 – это всего лишь метод кодирования для удобства передачи; это не метод шифрования или хеширования. Если сетевой трафик будет перехвачен, любой желающий может увидеть учетные данные. Как правило, мы не используем аутентификацию HTTP Basic без протокола HTTPS (как минимум) для соблюдения конфиденциальности. Вы можете прочитать подробное определение HTTP Basic в RFC 7617 (<https://tools.ietf.org/html/rfc7617>).

AuthenticationProvider определяет логику аутентификации, делегируя управление пользователями и паролями. Реализация AuthenticationProvider по умолчанию использует реализации по умолчанию, предоставленные для UserDetailsService и PasswordEncoder. Из этого следует, что ваше приложение неявно защищает все конечные точки. Поэтому единственное, что нам нужно было сделать для нашего примера, – это добавить конечную точку. Кроме того, есть только один пользователь, который может получить доступ ко всем конечным точкам, так что можно сказать, что в этом случае с авторизацией особо ничего не нужно делать.

HTTP или HTTPS?

Вы могли заметить, что в представленных примерах я использую протокол HTTP. Однако на практике ваши приложения взаимодействуют только по HTTPS. Для примеров, которые мы обсуждаем в этой книге, конфигурации, связанные с Spring Security, не отличаются, независимо от того, используем ли мы HTTP или HTTPS. Мы не будем настраивать HTTPS для конечных точек в примерах, чтобы вы могли сосредоточиться на аспектах, связанных с Spring Security. Но если хотите, вы можете включить HTTPS для любой из конечных точек.

Существует несколько шаблонов для настройки HTTPS в системе. Иногда разработчики настраивают HTTPS на уровне приложения; в других случаях они могут использовать сетку служб или выбрать настройку HTTPS на уровне инфраструктуры. С помощью Spring Boot вы можете легко включить HTTPS на уровне приложения, как показано ниже.

В любом из этих сценариев конфигурации вам нужен сертификат, подписанный центром сертификации (certification authority, CA). Благодаря этому сертификату клиент, вызывающий конечную точку, знает, что ответ пришел от сервера аутентификации и никто не перехватил канал связи. Вы можете купить такой сертификат, если он вам нужен. Если вам нужно настроить HTTPS только для тестирования вашего приложения, достаточно сгенерировать самоподписанный сертификат с помощью такого инструмента, как OpenSSL (<https://www.openssl.org/>). Давайте сгенерируем самоподписанный сертификат, а затем сконфигурируем его в проекте:

```
openssl req -newkey rsa:2048 -x509 -keyout key.pem -out cert.pem -days 365
```

После запуска команды openssl в терминале вам будет предложено ввести пароль и данные о вашем CA. Поскольку это всего лишь самоподписанный сертификат для тестирования, вы можете ввести туда любые данные; просто запомните пароль. Команда выводит два файла: key.pem (закрытый ключ) и cert.pem (открытый сертификат). Мы будем использовать эти файлы в дальнейшем при генерации нашего самоподписанного сертификата для включения HTTPS. В большинстве случаев сертификат представлен в формате Public Key Cryptography Standards #12 (PKCS12). Реже используют формат Java KeyStore (JKS). В нашем

примере применяется формат PKCS12 (для более глубокого изучения криптографии я рекомендую отличную книгу Дэвида Вонга «Реальная криптография» [Питер, 2024]):

```
openssl pkcs12 -export -in cert.pem -inkey key.pem -out certificate.p12 -name "certificate"
```

Вторая команда получает в качестве входных данных два файла, сгенерированных первой командой, и выводит самоподписанный сертификат.

Учтите, что если вы запускаете эти команды в оболочке Bash в системе Windows, вам может потребоваться добавить winpty перед ними:

```
winpty openssl req -newkey rsa:2048 -x509 -keyout key.pem -out cert.pem -days 365
winpty openssl pkcs12 -export -in cert.pem -inkey key.pem -out certificate.p12 -name "certificate"
```

Наконец, имея самоподписанный сертификат, вы можете настроить HTTPS для своих конечных точек. Скопируйте файл certificate.p12 в папку ресурсов проекта Spring Boot и добавьте следующие строки в файл application.properties:

```
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:certificate.p12
server.ssl.key-store-password=12345
```

❶ Значение пароля то же самое, которое вы указали при запуске второй команды для генерации файла сертификата PKCS12.

Пароль (в моем случае 12345) был запрошен в приглашении после запуска команды для генерации сертификата. Вот почему вы не видите его в команде. Теперь добавим тестовую конечную точку в наше приложение, а затем вызовем ее с помощью HTTPS:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello!";
    }
}
```

Если вы используете самоподписанный сертификат, вам следует настроить инструмент, который вы применяете для вызова конечной точки, так чтобы он пропускал проверку подлинности сертификата. Если инструмент проверяет подлинность сертификата, он не распознает его как подлинный, и вызов не будет работать. В случае использования cURL добавьте опцию -k, чтобы пропустить проверку подлинности сертификата:

```
curl -k -u user:93a01cf0-794b-4b98-86ef-54860f36f7f3
https://localhost:8080/hello
```


Ответ на вызов:

Hello!

Помните, что даже если вы используете HTTPS, связь между компонентами вашей системы не является надежной. Много раз я слышал, как люди говорили: «Я больше не буду шифровать данные. Я буду использовать HTTPS!» Хотя HTTPS полезен для защиты канала связи, это всего лишь один из кирпичиков в стене безопасности системы. Всегда относиться к безопасности своей системы ответственно и заботиться обо всех задействованных уровнях.

2.3 *Переопределение конфигураций по умолчанию*

Теперь, когда вы знаете настройки по умолчанию своего первого проекта, пришло время посмотреть, как их можно заменить. Вы должны понимать, какие у вас есть варианты переопределения компонентов по умолчанию, потому что именно так вы подключаете свои пользовательские реализации и применяете безопасность в соответствии с задачами и потребностями вашего приложения. И как вы узнаете в этом разделе, конфигурации желательно разрабатывать так, чтобы их было легко обслуживать. В проектах, над которыми вам предстоит работать, вы часто будете сталкиваться с несколькими способами переопределения конфигурации. Такая гибкость создает путаницу. Я часто вижу сочетание разных стилей настройки разных частей Spring Security в одном приложении, что нежелательно. Вам нужно научиться выбирать один вариант и придерживаться его на всем протяжении разработки приложения.

В некоторых случаях разработчики выбирают для настройки использование бинов (bean-компонентов) в контексте Spring. В других случаях они переопределяют различные методы для той же цели. Такое разнообразие подходов, вероятно, возникло из-за высокой скорости, с которой развивалась экосистема Spring. Смесь стилей в конфигурации проекта нежелательна, так как это делает код сложным для понимания и влияет на удобство обслуживания приложения. Знание всех доступных вариантов и способов их использования — это ценный навык, который помогает лучше понять, как следует настраивать безопасность на уровне приложения в проекте. Далее вы узнаете, как настроить `UserDetailsService` и `PasswordEncoder`. Эти два компонента обычно участвуют в аутентификации, и большинство приложений настраивают их в зависимости от своих потребностей. Хотя мы обсудим детали их настройки в главах 3 и 4, сейчас

важно увидеть, как подключить пользовательскую реализацию. Все реализации, которые мы используем в этой главе, предоставлены Spring Security.

2.3.1 *Настройка управления данными пользователя*

Первым компонентом, о котором мы говорили в этой главе, был `UserDetailsService`. Как вы уже видели, приложение использует этот компонент в процессе аутентификации. В этом разделе вы научитесь определять пользовательский компонент типа `UserDetailsService`. Мы сделаем это, чтобы переопределить стандартный компонент, настроенный Spring Boot. В главе 3 более подробно говорится о том, как создать собственную реализацию или использовать предопределенную конфигурацию, предоставленную Spring Security. В этой главе я не буду подробно описывать реализации, предоставляемые Spring Security, или создавать собственную реализацию. Для начала я ограничусь реализацией, предоставляемой Spring Security, под названием `InMemoryUserDetailsManager`. На следующем примере вы научитесь подключать этот тип объекта к вашей архитектуре.

ПРИМЕЧАНИЕ Интерфейсы в Java определяют контракты между объектами. При разработке класса приложения мы применяем интерфейсы для разделения объектов, которые используют друг друга. Чтобы выделить эту характеристику интерфейса при обсуждении их в этой книге, я в основном называю их *контрактами*.

Для демонстрации переопределения компонента с помощью выбранной нами реализации мы внесем изменения в первый пример. Они позволят нам иметь собственные управляемые учетные данные для аутентификации. Но мы не будем разрабатывать свой класс, а воспользуемся реализацией, предоставленной Spring Security.

В этом примере мы используем реализацию `InMemoryUserDetailsManager`. Хотя она немного шире, чем просто `UserDetailsService`, на данный момент мы используем ее точно так же, как `UserDetailsService`. Эта реализация сохраняет учетные данные в памяти, а затем Spring Security может использовать их для аутентификации запроса.

ПРИМЕЧАНИЕ Реализация `InMemoryUserDetailsManager` не предназначена для приложений, готовых к производству, но это отличный инструмент для обучающих примеров или тестирования идей. В некоторых случаях все, что вам нужно, – это пользователи. Вам незачем тратить время на реализацию этой части функциональности. В нашем случае мы используем ее, только чтобы понять, как переопределить реализацию `UserDetailsService` по умолчанию.

Начнем с определения класса конфигурации. Обычно мы объявляем классы конфигурации в отдельном пакете с именем `config`. В листинге 2.3 показано определение класса конфигурации. Вы также можете найти пример в проекте `ssia-ch2-ex2`.

Листинг 2.3 Класс конфигурации для bean-компонента `UserDetailsService`

```
@Configuration ❶
public class ProjectConfig {

    @Bean ❷
    UserDetailsService userDetailsService() {
        return new InMemoryUserDetailsManager();
    }
}
```

- ❶ Аннотация `@Configuration` сообщает, что это класс конфигурации.
- ❷ Аннотация `@Bean` предписывает Spring добавить возвращаемое значение как bean-компонент в контекст Spring.

Мы аннотируем класс как `@Configuration`. Аннотация `@Bean` предписывает Spring добавить экземпляр, возвращаемый методом, в контекст Spring. Если вы выполните код из листинга 2.3, вы больше не увидите автоматически сгенерированный пароль в консоли. Приложение теперь использует экземпляр типа `UserDetailsService`, который вы добавили в контекст, вместо автоматически настроенного по умолчанию. Но в то же время вы больше не сможете получить доступ к конечной точке по двум причинам:

- у вас нет пользователей;
- у вас нет `PasswordEncoder`.

На рис. 2.2 показано, что аутентификация также зависит от `PasswordEncoder`. Давайте последовательно решим обе проблемы. Нам нужно:

- 1 Создать хотя бы одного пользователя с набором учетных данных (имя пользователя и пароль).
- 2 Добавить пользователя, которым будет управлять наша реализация `UserDetailsService`.
- 3 Определить компонент типа `PasswordEncoder`, который наше приложение может использовать для сравнения введенного пароля с сохраненным, которым управляет `UserDetailsService`.

Сначала мы объявляем и добавляем набор учетных данных, которые можно использовать для аутентификации, в экземпляр `InMemoryUserDetailsManager`. В главе 3 мы подробнее обсудим создание пользователей и управление ими. На данный момент мы воспользуемся предопределенным конструктором для создания объекта типа `UserDetails`.

ПРИМЕЧАНИЕ Иногда я использую `var` в коде. В Java 10 появилось зарезервированное имя типа `var`, и вы можете использовать его только для локальных объявлений. Хотя в некоторых случаях способ использования `var` в этой книге можно считать дурным тоном с точки зрения чистого кодирования, он позволяет сделать синтаксис короче, а также скрыть тип переменной. Такой подход помогает вам сосредоточиться на том, что актуально для данного примера. Мы обсудим типы, скрытые `var`, в последующих главах, поэтому вам не нужно задумываться об этом типе, пока не придет время проанализировать его должным образом.

При создании экземпляра мы должны указать имя пользователя, пароль и по крайней мере одно полномочие. *Полномочие* (authority) – это действие, разрешенное для данного пользователя, и мы можем использовать для этого любую строку. В листинге 2.4 я называю полномочие `read`, но поскольку мы не будем использовать его в данный момент, это имя не имеет особого значения.

Листинг 2.4 Создание пользователя с помощью класса-конструктора `User` для `UserDetailsService`

```
@Configuration
public class ProjectConfig {

    @Bean
    UserDetailsService userDetailsService() {
        var user = User.withUsername("john")           ❶
                        .password("12345")             ❶
                        .authorities("read")           ❶
                        .build();                       ❶

        return new InMemoryUserDetailsManager(user);  ❷
    }
}
```

- ❶ Создаем пользователя с заданным именем пользователя, паролем и списком полномочий.
- ❷ Добавляем пользователя для управления `UserDetailsService`.

ПРИМЕЧАНИЕ Класс `User` находится в пакете `org.springframework.security.core.userdetails`. Это реализация конструктора, которую мы используем для создания объекта, представляющего пользователя. Кроме того, если я не показываю, как написать класс в листинге кода, то, как правило, это означает, что Spring Security предоставляет его в готовом виде.

Как показано в листинге 2.4, мы должны предоставить значения имени пользователя и пароля и хотя бы одно полномочие. Однако

этого все еще недостаточно для вызова конечной точки. Нам также нужно объявить `PasswordEncoder`.

При использовании `UserDetailsService` по умолчанию также автоматически настраивается `PasswordEncoder`. Но поскольку мы переопределили `UserDetailsService`, нам также нужно объявить `PasswordEncoder`. Если запустить код примера сейчас, вы увидите исключение при вызове конечной точки. При попытке выполнить аутентификацию Spring Security обнаруживает, что не знает, как управлять паролем, и терпит неудачу. В следующем фрагменте кода показано, как выглядит вывод исключения, и вы должны увидеть его в консоли вашего приложения. Клиент получает сообщение HTTP 401 Unauthorized и пустое тело ответа:

```
curl -u john:12345 http://localhost:8080/hello
```

Результат вызова в консоли приложения:

```
java.lang.IllegalArgumentException:
There is no PasswordEncoder mapped for the id "null"
    at
org.springframework.security.crypto.
    ➔password.DelegatingPasswordEncoder$
    ➔UnmappedIdPasswordEncoder.matches(
    ➔DelegatingPasswordEncoder.java:289)
    ➔~[spring-security-crypto-6.0.0.jar:6.0.0]
        at org.springframework.security.crypto.
    ➔password.DelegatingPasswordEncoder.matches(
    ➔DelegatingPasswordEncoder.java:237)
    ➔~[spring-security-crypto-6.0.0.jar:6.0.0]
```

Чтобы решить эту проблему, добавим компонент `PasswordEncoder` в контекст, как мы сделали с `UserDetailsService`. Для этого bean-компонента мы используем существующую реализацию `PasswordEncoder`:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
}
```

ПРИМЕЧАНИЕ Экземпляр `NoOpPasswordEncoder` обрабатывает пароли как обычный текст. Он не шифрует и не хеширует их. Для сопоставления паролей `NoOpPasswordEncoder` просто сравнивает строки, используя базовый метод `equals(Object o)` класса `String`. Вам не следует использовать этот тип `PasswordEncoder` в готовом к производству приложении. `NoOpPasswordEncoder` – хороший вариант для примеров, где вы не хотите фокусироваться на алгоритме хеширова-

ния пароля. Поэтому разработчики класса поместили его как `@Deprecated`, и ваша среда разработки будет отображать его имя зачеркнутым.

Полный код класса конфигурации показан в листинге 2.5.

Листинг 2.5 Полное определение класса конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    UserDetailsService userDetailsService() {
        var user = User.withUsername("john")
            .password("12345")
            .authorities("read")
            .build();

        return new InMemoryUserDetailsManager(user);
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

❶ Новый метод, аннотированный `@Bean`, для добавления `PasswordEncoder` в контекст.

Попробуем вызвать конечную точку с новым пользователем, имеющим имя пользователя `john` и пароль `12345`:

```
curl -u john:12345 http://localhost:8080/hello
Hello!
```

ПРИМЕЧАНИЕ У читателей, знающих о важности модульных и интеграционных тестов, мог возникнуть вопрос, почему я не сопровождаю свои примеры тестами. На самом деле в файловом архиве есть соответствующие интеграционные тесты Spring Security для всех примеров, представленных в этой книге. Но чтобы помочь вам сосредоточиться на ключевых темах каждой главы, я выделил тему тестирования интеграций Spring Security в отдельную главу 18.

2.3.2 Применение авторизации на уровне конечной точки

В предыдущем разделе мы добавили новое управление пользователями и теперь можем обсудить метод аутентификации и конфигурацию для конечных точек. Вы узнаете много нового о конфигурации

авторизации в главах с 7 по 12. Но прежде чем углубляться в детали, необходимо понять общую картину. Лучший способ добиться этого – наш первый пример. При конфигурации по умолчанию все конечные точки предполагают, что у вас есть действительный пользователь, управляемый приложением. Кроме того, по умолчанию ваше приложение использует аутентификацию HTTP Basic, но вы можете легко переопределить эту конфигурацию.

Как вы узнаете в следующих главах, аутентификация HTTP Basic не подходит для большинства архитектур приложений, и ее приходится менять на другую. Аналогично не все конечные точки приложения должны быть защищены, а для тех, которые защищены, нам могут потребоваться другие методы аутентификации и правила авторизации. Чтобы настроить обработку аутентификации и авторизации, нужно определить bean-компонент типа `SecurityFilterChain`. Рассмотрим код, показанный в листинге 2.6. Вы по-прежнему найдете его в проекте `ssia-ch2-ex3`.

Листинг 2.6 Определение bean-компонента `SecurityFilterChain`

```
@Configuration
public class ProjectConfig {

    @Bean
    SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        return http.build();
    }
    // Часть кода пропущена
}
```

Затем можно изменить конфигурацию, используя различные методы объекта `HttpSecurity`, как показано в следующем листинге.

Листинг 2.7 Использование параметра `HttpSecurity` для изменения конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());           ❶

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()              ❷
        );
    }
}
```

```
        return http.build();
    }

    // Часть кода пропущена
}
```

- ❶ Приложение использует аутентификацию HTTP Basic.
- ❷ Все запросы требуют аутентификации.

Код в листинге 2.7 настраивает авторизацию конечной точки с тем же поведением, что и по умолчанию. Вы можете снова вызвать конечную точку, чтобы увидеть, ведет ли она себя так же, как в предыдущем тесте из раздела 2.3.1. Достаточно небольших изменений, чтобы сделать все конечные точки доступными без необходимости в учетных данных. В листинге 2.8 показано, как это сделать.

Листинг 2.8 Использование `permitAll()` для изменения конфигурации авторизации

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().permitAll()
        );

        return http.build();
    }

    // Часть кода пропущена
}
```

- ❶ Ни один из запросов не требует аутентификации.

Теперь мы можем вызвать конечную точку `/hello` без необходимости в учетных данных. Вызов `permitAll()` в конфигурации вместе с методом `anyRequest()` делает все конечные точки доступными без необходимости ввода учетных данных:

```
curl http://localhost:8080/hello
```

Тело ответа вызова:

Hello!

В этом примере мы использовали два метода конфигурации:

- `httpBasic()` помог нам настроить способ аутентификации. Вызвав этот метод, вы указали приложению принять HTTP Basic в качестве способа аутентификации;
- `authorizeHttpRequests()` помог нам настроить правила авторизации на уровне конечной точки. Вызвав этот метод, вы указали приложению, как авторизовать запросы, полученные на определенных конечных точках.

Для обоих методов вам нужно было использовать объект `Customizer` в качестве параметра. `Customizer` – это контракт, который вы реализуете для определения настройки любого элемента Spring Security: аутентификация, авторизация или определенные механизмы защиты, такие как CSRF или CORS (о них говорится в главах 9 и 10). В следующем фрагменте кода показано определение интерфейса `Customizer`. Обратите внимание, что `Customizer` – это функциональный интерфейс (поэтому мы можем использовать лямбда-выражения для его реализации), а метод `withDefaults()`, который я использовал в листинге 2.8, на самом деле является просто ничего не делающей реализацией `Customizer`:

```
@FunctionalInterface
public interface Customizer<T> {
    void customize(T t);

    static <T> Customizer<T> withDefaults() {
        return (t) -> {
        };
    }
}
```

В более ранних версиях Spring Security можно было применять конфигурации без объекта `Customizer`, используя синтаксис цепочки, как показано в следующем фрагменте кода. Заметьте, что вместо предоставления объекта `Customizer` методу `authorizeHttpRequests()` конфигурация просто следует за вызовом метода:

```
http.authorizeHttpRequests()
    .anyRequest().authenticated()
```

Причина, по которой сохранился этот подход, заключается в том, что объект `Customizer` обеспечивает большую гибкость при перемещении конфигурации туда, где это необходимо. Конечно, в простых примерах использование лямбда-выражений удобно. Но в реальных приложениях конфигурации могут сильно разрастаться. В таких случаях возможность перемещать эти конфигурации в отдельные классы упрощает поддержку и тестирование.

Цель этого примера – дать вам представление о том, как переопределять конфигурации по умолчанию. Мы подробно рассмотрим авторизацию в главах с 7 по 10.

ПРИМЕЧАНИЕ В более ранних версиях Spring Security класс конфигурации безопасности требовал расширения класса с именем `WebSecurityConfigurerAdapter`. Но больше так не делается. Если ваше приложение использует старую кодовую базу или вам нужно обновить старую кодовую базу, я рекомендую вам также прочитать первое издание этой книги.

2.3.3 Разные способы настройки

При создании конфигураций с помощью Spring Security часто вызывает путаницу наличие нескольких способов настройки одних и тех же компонентов. В этом разделе говорится о разных способах настройки `UserDetailsService` и `PasswordEncoder`. Важно знать, с какими параметрами вам придется иметь дело в вашем приложении, чтобы определить, какие примеры из этой книги и других источников, таких как блоги и статьи, вам подходят. Также важно понимать, как и когда использовать их в своем приложении. В последующих главах будут представлены различные примеры, которые расширяют информацию этого раздела.

Возьмем первый проект. После того как мы создали приложение по умолчанию, нам удалось переопределить `UserDetailsService` и `PasswordEncoder`, добавив новые реализации как бины в контексте Spring. Теперь рассмотрим другой способ создания тех же конфигураций для `UserDetailsService` и `PasswordEncoder`.

С помощью бина `SecurityFilterChain` мы можем напрямую задать как `UserDetailsService`, так и `PasswordEncoder`, как показано в листинге 2.9. Этот пример можно найти в проекте `ssia-ch2-ex3`.

Листинг 2.9 Настройка `UserDetailsService` с помощью компонента `SecurityFilterChain`

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );
    }
}
```

```

var user = User.withUsername("john")           ❶
    .password("12345")
    .authorities("read")
    .build();

var userDetailsService =                       ❷
    new InMemoryUserDetailsManager(user);

http.userDetailsService(userDetailsService);  ❸

return http.build();
}

// Часть кода пропущена
}

```

- ❶ Определяем пользователя со всеми его данными.
- ❷ Объявляем `UserDetailsService` для хранения пользователей в памяти и добавляем пользователя, управляемого этим `UserDetailsService`.
- ❸ `UserDetailsService` теперь настроен с помощью bean-компонента `SecurityFilterChain`.

Вы можете заметить, что в листинге 2.9 мы объявляем `UserDetailsService` так же, как в листинге 2.5. Разница в том, что теперь это делается локально внутри метода bean-компонента, создающего `SecurityFilterChain`. Мы также вызываем метод `userDetailsService()` из `HttpSecurity` для регистрации экземпляра `UserDetailsService`. В листинге 2.10 показано полное определение класса конфигурации.

Листинг 2.10 Полное определение класса конфигурации

```

@Configuration
public class ProjectConfig {

    @Bean
    SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );

        var user = User.withUsername("john")           ❶
            .password("12345")
            .authorities("read")
            .build();

        var userDetailsService =                       ❷
            new InMemoryUserDetailsManager(user);
    }
}

```

```
http.userService(userDetailsService); ❸

return http.build();
}

@Bean
PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
}
}
```

- ❶ Создаем нового пользователя.
- ❷ Добавляем пользователя для управления нашим UserDetailsService.
- ❸ Настраиваем UserDetailsService.

Оба варианта конфигурации правильные. Первый вариант, где мы добавляем бины в контекст, позволяет вам внедрять значения в другой класс, где они вам потенциально могут понадобиться. Но если вам это не нужно, второй вариант будет столь же хорош.

2.3.4 Определение пользовательской логики аутентификации

Как вы уже заметили, компоненты Spring Security обеспечивают большую гибкость за счет различных способов адаптации их к архитектуре наших приложений. К настоящему моменту вы узнали о назначении UserDetailsService и PasswordEncoder в архитектуре Spring Security. Вы также увидели несколько способов их настройки. Пришло время узнать о настройке компонента AuthenticationProvider, который делегирует им полномочия, как показано на рис. 2.3. AuthenticationProvider реализует логику аутентификации и делегирует UserDetailsService и PasswordEncoder для управления пользователями и паролями. Поэтому можно сказать, что в этом разделе мы на один шаг углубляемся в архитектуру аутентификации, чтобы узнать, как реализовать пользовательскую логику аутентификации с помощью AuthenticationProvider.

Сейчас я покажу вам только общую картину, чтобы вы лучше поняли взаимосвязь между компонентами в архитектуре. Но мы рассмотрим этот вопрос более подробно в главах с 3 по 6.

Я рекомендую вам присмотреться к системе *ответственностей* (responsibilities), выстроенной в архитектуре Spring Security. Эта архитектура не привязана к жестким ответственным. Такой подход – одна из причин, по которой Spring Security является настолько гибким и простым для интеграции с вашими приложениями. Вы можете изменить архитектуру системы в зависимости от своих потребностей. Но будьте осторожны с этими подходами, поскольку иногда они могут усложнить ваше решение. Например, вы можете пере-

определить исходный `AuthenticationProvider` таким образом, что вам больше не понадобится `UserDetailsService` или `PasswordEncoder`. В листинге 2.11 показано создание настраиваемого провайдера аутентификации. Этот пример можно найти в проекте `ssia-ch2-ex4`.

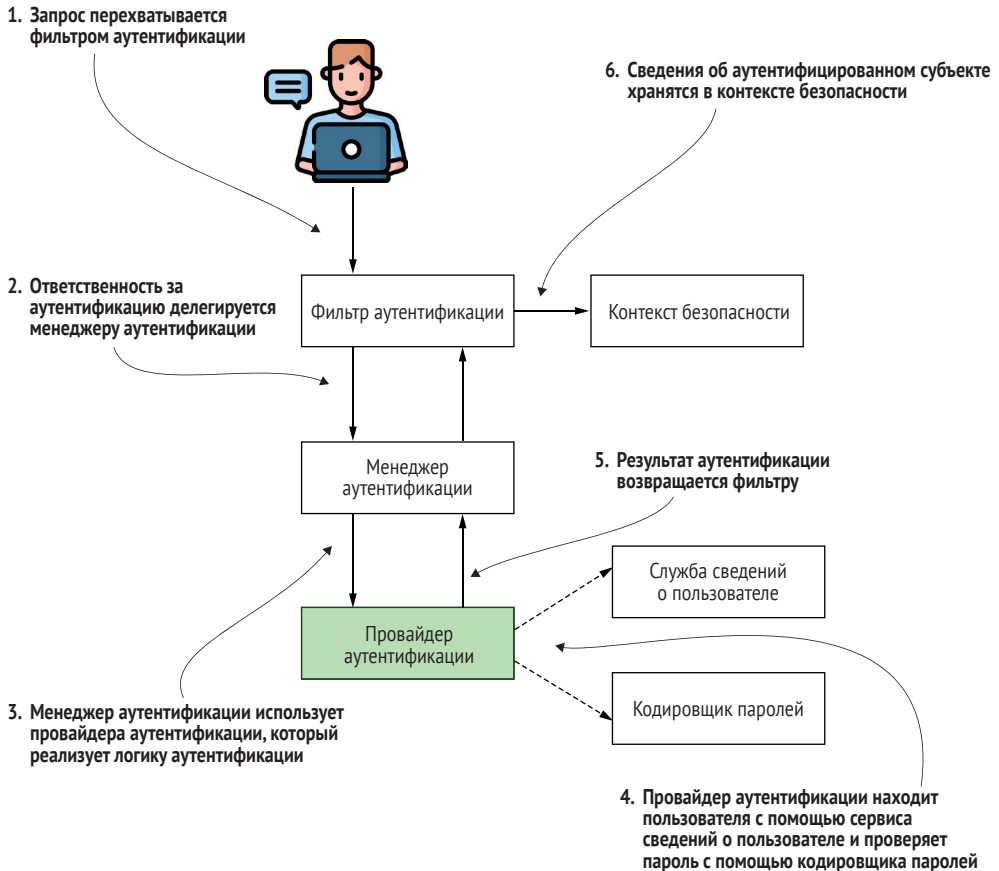


Рис. 2.3 `AuthenticationProvider` реализует логику аутентификации. Он получает запрос от `AuthenticationManager` и делегирует поиск пользователя компоненту `UserDetailsService`, а проверку пароля – компоненту `PasswordEncoder`

Листинг 2.11 Реализация интерфейса `AuthenticationProvider`

```
@Component
public class CustomAuthenticationProvider implements AuthenticationProvider {

    @Override
    public Authentication authenticate(Authentication authentication) throws
        AuthenticationException {

        // Здесь размещается логика аутентификации
```

```
}  
  
@Override  
public boolean supports(Class<?> authenticationType) {  
    // Здесь определяется тип реализации Autentification  
}  
}
```

Метод `authenticate(Authentication authentication)` представляет всю логику аутентификации, поэтому мы добавим реализацию, подобную той, что в листинге 2.12. Я подробно объясню использование метода `supports()` в главе 6. На данный момент я рекомендую вам просто не обращать внимания на его реализацию. Она не имеет значения для текущего примера.

Листинг 2.12 Реализация логики аутентификации

```
@Override  
public Authentication authenticate(  
    Authentication authentication)  
    throws AuthenticationException {  
  
    String username = authentication.getName();  
    String password = String.valueOf(  
        authentication.getCredentials());  
  
    if ("john".equals(username) &&  
        "12345".equals(password)) {  
        return new UsernamePasswordAuthenticationToken(  
            username,  
            password,  
            Arrays.asList());  
    } else {  
        throw new AuthenticationCredentialsNotFoundException("Error!");  
    }  
}
```

- ❶ Метод `getName()` наследуется `Authentication` от интерфейса `Principal`.
- ❷ Это условие обычно вызывает `UserDetailsService` и `PasswordEncoder` для проверки имени пользователя и пароля.

Здесь условие `if-else` заменяет ответственности `UserDetailsService` и `PasswordEncoder`. Вам не обязательно использовать два компонента, но если вы работаете с пользователями и паролями для аутентификации, я настоятельно рекомендую вам разделить логику их управления. Применяйте ее так, как это задумано архитектурой Spring Security, даже если вы переопределяете реализацию аутентификации.

Возможно, вам будет полезно заменить логику аутентификации, реализовав собственный `AuthenticationProvider`. Если реализация

по умолчанию не полностью соответствует требованиям вашего приложения, вы можете реализовать пользовательскую логику аутентификации. Полная реализация `AuthenticationProvider` выглядит так, как в следующем листинге.

Листинг 2.13 Полная реализация провайдера аутентификации

```
@Component
public class CustomAuthenticationProvider
    implements AuthenticationProvider {

    @Override
    public Authentication authenticate(
        Authentication authentication)
        throws AuthenticationException {

        String username = authentication.getName();
        String password = String.valueOf(authentication.getCredentials());

        if ("john".equals(username) &&
            "12345".equals(password)) {
            return new UsernamePasswordAuthenticationToken(
                username, password, Arrays.asList());
        } else {
            throw new AuthenticationCredentialsNotFoundException("Error!");
        }
    }

    @Override
    public boolean supports(Class<?> authenticationType) {
        return UsernamePasswordAuthenticationToken
            .class
            .isAssignableFrom(authenticationType);
    }
}
```

Вы можете зарегистрировать `AuthenticationProvider` в классе конфигурации с помощью метода `HttpSecurity authenticationProvider()`, показанного в листинге 2.14.

Листинг 2.14 Регистрация новой реализации `AuthenticationProvider`

```
@Configuration
public class ProjectConfig {

    private final CustomAuthenticationProvider authenticationProvider;

    public ProjectConfig(
        CustomAuthenticationProvider authenticationProvider) {

        this.authenticationProvider = authenticationProvider;
    }
}
```

```
}

@Bean
SecurityFilterChain configure(HttpSecurity http) throws Exception {
    http.httpBasic(Customizer.withDefaults());

    http.authenticationProvider(authenticationProvider);

    http.authorizeHttpRequests(
        c -> c.anyRequest().authenticated()
    );

    return http.build();
}
}
```

Теперь вы можете вызвать конечную точку, доступную только распознанному пользователю john с паролем 12345, как определено логикой аутентификации:

```
curl -u john:12345 http://localhost:8080/hello
```

Ответ конечной точки:

```
Hello!
```

В главе 6 вы узнаете больше об AuthenticationProvider и о том, как переопределить его поведение в процессе аутентификации. Там же мы обсудим интерфейс Authentication и его реализации, такие как UserPasswordAuthenticationToken.

2.3.5 Использование нескольких классов конфигурации

В предыдущих примерах мы использовали только класс конфигурации. Но хорошей практикой является разделение ответственностей даже для классов конфигурации. Нам действительно нужно это разделение, потому что конфигурация начинает становиться более сложной. В готовом к производству приложении у вас, вероятно, будет больше объявлений, чем в наших первых примерах. Вам также будет полезно иметь более одного класса конфигурации, чтобы сделать проект читаемым.

Хорошей практикой является наличие только одного класса для каждой ответственности. В нашем примере мы можем отделить конфигурацию управления пользователями от конфигурации авторизации. Мы делаем это, определяя два класса конфигурации: UserManagementConfig (определен в листинге 2.15) и WebAuthorizationConfig (определен в листинге 2.16). Этот пример можно найти в проекте ssia-ch2-ex5.

Листинг 2.15 Определение класса конфигурации для управления пользователями и паролями

```
@Configuration
public class UserManagementConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var userDetailsService = new InMemoryUserDetailsManager();
        var user = User.withUsername("john")
            .password("12345")
            .authorities("read")
            .build();

        userDetailsService.createUser(user);
        return userDetailsService;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

В этом случае класс `UserManagementConfig` содержит только два компонента, отвечающих за управление пользователями: `UserDetailsService` и `PasswordEncoder`. В следующем листинге показано это определение.

Листинг 2.16 Определение класса конфигурации для управления авторизацией

```
@Configuration
public class WebAuthorizationConfig {

    @Bean
    SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );

        return http.build();
    }
}
```

Здесь класс `WebAuthorizationConfig` должен определить компонент типа `SecurityFilterChain` для настройки правил аутентификации и авторизации.

Заклучение

- Spring Boot предоставляет ряд конфигураций по умолчанию при добавлении Spring Security к зависимостям приложения.
- Вы реализуете следующие базовые компоненты для аутентификации и авторизации: `UserDetailsService`, `PasswordEncoder` и `AuthenticationProvider`.
- Вы можете определить пользователей с помощью класса `User`. У пользователя должны быть как минимум имя пользователя, пароль и полномочия. Полномочия – это действия, которые вы разрешаете пользователю выполнять в контексте приложения.
- Простая реализация `UserDetailsService`, которую предоставляет Spring Security, – это `InMemoryUserDetailsManager`. Вы можете добавлять пользователей в такой экземпляр `UserDetailsService` для управления пользователем в памяти приложения.
- `NoOpPasswordEncoder` – это реализация контракта `PasswordEncoder`, который использует пароли в открытом тексте. Эта реализация хороша для изучения примеров и (возможно) проверки работоспособности идей, но не для приложений, готовых к производству.
- Вы можете использовать контракт `AuthenticationProvider` для реализации пользовательской логики аутентификации в приложении.
- Существует несколько способов написания конфигураций, но в рамках приложения следует придерживаться одного выбранного подхода. Это помогает сделать ваш код чище и проще для понимания.

Часть II

Настройка аутентификации

Аутентификация находится на переднем рубеже защиты любого приложения, определяя, кто с ним взаимодействует. Во второй части книги вы изучите суть этого механизма.

Глава 3 познакомит вас с механизмом управления пользователями Spring Security, включая основные контракты `UserDetails` и `GrantedAuthority`, а также с другими нюансами работы Spring Security с пользователями.

Глава 4 рассказывает о безопасности паролей. Вы изучите контракт `PasswordEncoder`, создадите собственный контракт и научитесь применять модуль `Crypto` фреймворка Spring Security для шифрования и генерации ключей.

Глава 5 знакомит с ключевой ролью фильтров в Spring Security. Вы научитесь интегрировать, упорядочивать и применять различные фильтры, повышая уровень безопасности вашего приложения.

Глава 6 объединяет знания из предыдущих глав в стройную систему. Вы откроете для себя суть `AuthenticationProvider`, изучите логику пользовательской аутентификации и познакомитесь с различными методами аутентификации при входе, включая `HTTP Basic` и формы.

К концу этой части вы будете иметь четкое представление о сложных уровнях и механизмах аутентификации в приложениях Spring.

Управление пользователями

Темы этой главы:

- описание пользователя с помощью интерфейса `UserDetails`;
- использование `UserDetailsService` в потоке аутентификации;
- создание пользовательской реализации `UserDetailsService`;
- создание пользовательской реализации `UserDetailsManager`;
- использование `JdbcUserDetailsManager` в потоке аутентификации.

Один из моих коллег по университету неплохо готовит. Он не шеф-повар в дорогом ресторане, но очень увлечен кулинарией. Однажды я спросил, как ему удастся помнить так много рецептов. Он сказал мне, что это легко. «Вам не нужно заучивать весь рецепт, но нужно помнить, как основные ингредиенты сочетаются друг с другом. Это своего рода контракты, которые говорят вам, что вы можете смешивать, а что нет. А в самом рецепте нужно запомнить лишь некоторые характерные приемы».

Эта аналогия хорошо отражает принцип работы архитектуры. В любом надежном фреймворке применяются контракты, отделяю-

щие реализации фреймворка от приложения, построенного на нем. В Java для определения контрактов мы используем интерфейсы. Программист похож на шеф-повара, который знает, как сочетаются ингредиенты, чтобы выбрать правильную реализацию. Программист знает абстракции структуры и использует их для интеграции.

В этой главе мы подробно рассмотрим одну из фундаментальных ролей, с которой вы столкнулись в первом примере из главы 2, – службу `UserDetails`. Наряду с `UserService` мы обсудим следующие интерфейсы (контракты):

- `UserDetails` – описывает пользователя для Spring Security;
- `GrantedAuthority` – позволяет нам определять действия, которые может выполнять пользователь;
- `UserDetailsManager` – расширяет контракт `UserService`. Помимо унаследованного поведения, он также описывает такие действия, как создание пользователя и изменение или удаление пароля пользователя.

Из главы 2 вы уже имеете представление о ролях, которые `UserDetailsService` и `PasswordEncoder` играют в процессе аутентификации. Но пока мы обсудили только, как подключить экземпляр, определенный вами, вместо использования экземпляра по умолчанию, настроенного Spring Boot. Нам нужно обсудить ряд технических деталей, а именно:

- реализации, предоставляемые Spring Security, и их применение;
- как определить пользовательскую реализацию для контрактов и когда это делать;
- способы реализации интерфейсов, которые вы встретите в реальных приложениях;
- лучшие практики использования этих интерфейсов.

Мы начнем с того, как Spring Security понимает определение пользователя. Для этого обсудим контракты `UserDetails` и `GrantedAuthority`. Далее мы подробно рассмотрим контракт `UserDetailsService` и то, как его расширяет `UserDetailsManager`. Вы примените реализации интерфейсов (например, `InMemoryUserDetailsManager`, `JdbcUserDetailsService` и `LdapUserDetailsService`). Если эти реализации не подходят для вашей системы, вы напишете собственную реализацию.

3.1 Реализация аутентификации в Spring Security

В предыдущей главе мы приступили к изучению Spring Security. В первом примере мы наблюдали, как Spring Boot определяет значения по умолчанию, от которых зависит поведение приложения. Вы

также узнали, как переопределять эти значения по умолчанию с помощью различных способов, которые часто встречаются в реальных приложениях. Однако на тот момент знакомство было слишком поверхностным, чтобы вы получили представление о том, что нам предстоит делать. В этой главе, а также в главах 4 и 5 мы обсудим более подробно эти интерфейсы, а также различные реализации и то, где вы можете найти их в реальных приложениях.

На рис. 3.1 представлен поток аутентификации в Spring Security. Эта схема является основой процесса аутентификации, реализованного Spring Security. Важно понимать ее, потому что вы будете иметь с ней дело в любой реализации Spring Security. Вы заметите, что мы обсуждаем части данной схемы почти во всех главах этой книги. Вы будете видеть ее так часто, что, вероятно, выучите ее наизусть, и это хорошо. Если вы знаете эту архитектуру, вы похожи на шеф-повара, который знает все сочетания ингредиентов и может приготовить блюдо по любому рецепту.

На рис. 3.1 окрашенные поля представляют компоненты, с которых мы начинаем: `UserDetailsService` и `PasswordEncoder`. Эти два компонента привязаны к части потока, которую я обычно называю «частью управления пользователем». В этой главе `UserDetailsService` и `PasswordEncoder` – это компоненты, которые напрямую работают с данными пользователя и его учетными данными. Мы подробно обсудим `PasswordEncoder` в главе 4.

В рамках управления пользователями мы используем интерфейсы `UserDetailsService` и `UserDetails Manager`. Интерфейс `UserDetailsService` отвечает только за извлечение пользователя по его имени. Это единственное действие, необходимое фреймворку для завершения аутентификации. `UserDetailsManager` добавляет поведение, связанное с добавлением, изменением или удалением пользователя, что необходимо большинству приложений. Разделение между двумя контрактами является прекрасным примером принципа разделения интерфейсов. Разделение интерфейсов увеличивает гибкость, поскольку фреймворк не заставляет вас реализовывать поведение, если ваше приложение в нем не нуждается. Если приложению нужно только аутентифицировать пользователей, то реализации контракта `UserDetailsService` достаточно для обеспечения желаемой функциональности. Для управления пользователями компонентам `UserDetailsService` и `UserDetailsManager` нужен способ их представления.

Spring Security предлагает контракт `UserDetails`, который вы должны реализовать для описания пользователя в форме, понятной фреймворку. Как вы узнаете в этой главе, в Spring Security у пользователя есть набор *привилегий* (*privilege*), представляющих собой действия, которые разрешено выполнять пользователю. Мы будем много работать с этими привилегиями в главах с 7 по 12 при изуче-

нии авторизации. Но сейчас ограничимся тем, что Spring Security представляет действия, которые пользователь может выполнять с помощью интерфейса `GrantedAuthority`. Эти привилегии часто называют *полномочиями* (authority), и у пользователя их может быть несколько. На рис. 3.2 схематически показана взаимосвязь между компонентами потока аутентификации, относящимися к части управления пользователями.

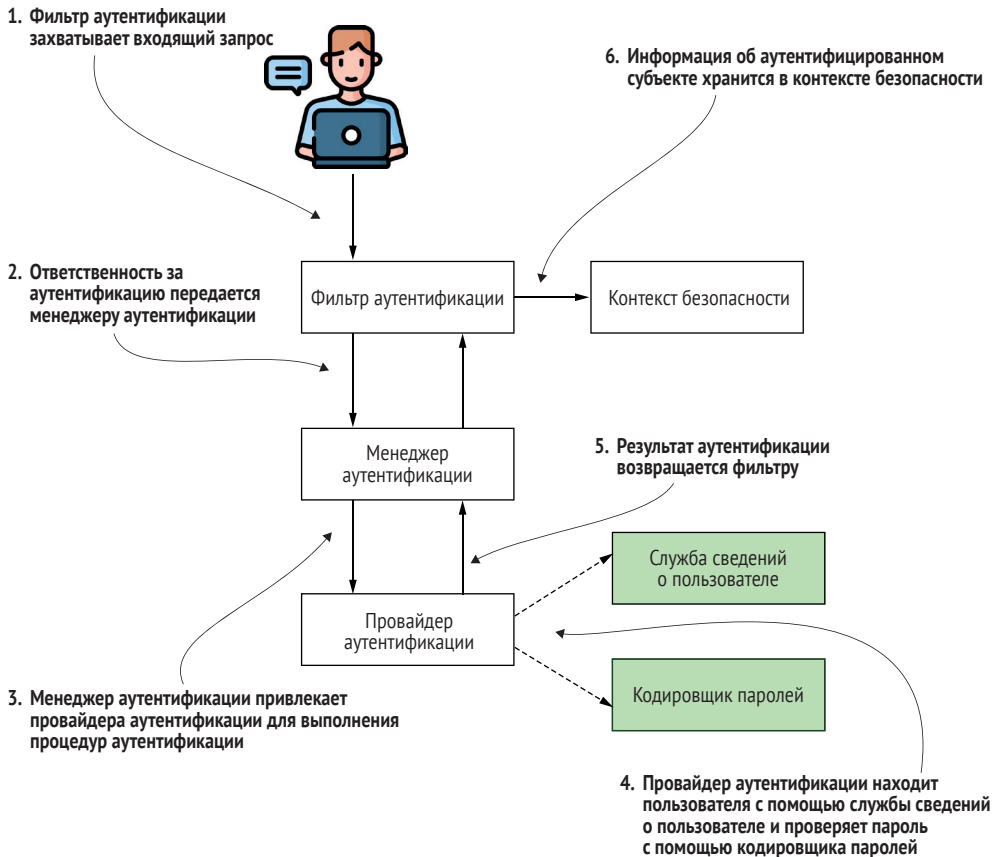


Рис. 3.1 Поток аутентификации Spring Security. `AuthenticationFilter` захватывает входящий запрос и передает задачу аутентификации компоненту `AuthenticationManager`. Тот, в свою очередь, использует провайдер аутентификации для выполнения процесса аутентификации. Для проверки имени пользователя и пароля `AuthenticationProvider` задействует `UserDetailsService` и `PasswordEncoder`

Понимание связей между этими объектами в архитектуре Spring Security и способов их реализации открывает перед вами обширный выбор возможностей при работе с приложениями. Любой из этих вариантов может быть правильным фрагментом пазла в приложении, над которым вы работаете, и ваша задача – сделать свой выбор

мудро. Но чтобы иметь возможность выбирать, вам сначала нужно знать, из чего вы можете выбирать.

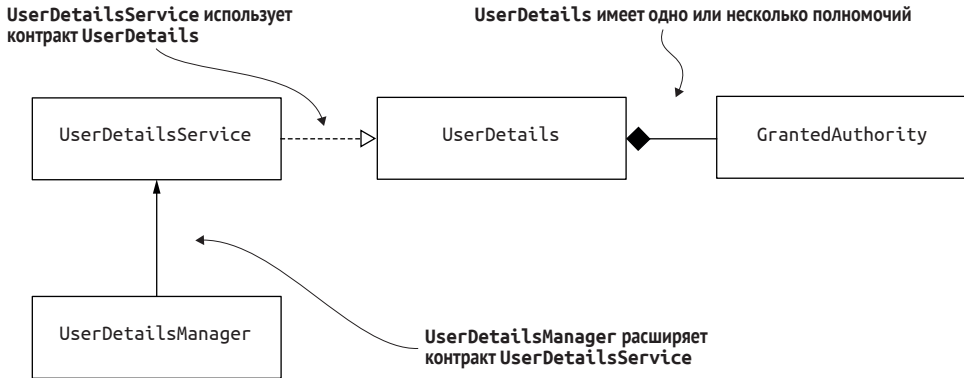


Рис. 3.2 Зависимости между компонентами, участвующими в управлении пользователями. `UserDetailsService` извлекает данные пользователя, выполняя поиск пользователя по имени. Пользователь характеризуется контрактом `UserDetails`. Каждый пользователь обладает одним или несколькими полномочиями, которые отображаются интерфейсом `GrantedAuthority`. Для добавления таких операций, как создание, удаление или изменение пароля пользователя, применяется контракт `UserDetailsManager`, который расширяет интерфейс `UserDetailsService` нужной функциональностью

3.2 Описание пользователя

В этом разделе вы узнаете, как описывать пользователей приложения способом, который понятен Spring Security. Этот навык является важным шагом в построении потока аутентификации. На основе описания пользователя приложение принимает решение – разрешен ли вызов определенной функции. Для работы с пользователями вам сначала нужно научиться определять прототип пользователя в вашем приложении. В этом разделе на примере показано, как настроить схему для ваших пользователей в приложении Spring Security.

Для работы со Spring Security определение пользователя должно соответствовать контракту `UserDetails`. Этот контракт делает представление пользователя понятным Spring Security. Класс вашего приложения, который описывает пользователя, должен реализовать этот интерфейс, и таким образом фреймворк понимает его.

3.2.1 Описание пользователей с помощью контракта `UserDetails`

В этом разделе вы узнаете, как реализовать интерфейс `UserDetails` для описания пользователей в приложении. Мы обсудим методы,

объявленные контрактом `UserDetails`, чтобы понять, как и почему мы реализуем каждый из них. Начнем с рассмотрения интерфейса, представленного в листинге 3.1.

Листинг 3.1 Интерфейс `UserDetails`

```
public interface UserDetails extends Serializable {  
    String getUsername();  
    String getPassword();  
    Collection<? extends GrantedAuthority> getAuthorities();  
    boolean isAccountNonExpired();  
    boolean isAccountNonLocked();  
    boolean isCredentialsNonExpired();  
    boolean isEnabled();  
}
```

- ❶ Эти методы возвращают учетные данные пользователя.
- ❷ Возвращает действия, которые приложение позволяет пользователю выполнять, как коллекцию экземпляров `GrantedAuthority`.
- ❸ Эти четыре метода включают или отключают учетную запись по разным причинам.

Методы `getUsername()` и `getPassword()` возвращают, как и ожидалось, имя пользователя и пароль. Приложение использует эти значения в процессе аутентификации, и это единственные сведения, связанные с аутентификацией из этого контракта. Все остальные пять методов относятся к авторизации пользователя для доступа к ресурсам приложения.

Как правило, приложение позволяет пользователю выполнять некоторые действия, которые имеют смысл в контексте приложения. Например, пользователь должен иметь возможность читать, записывать или удалять данные. Говорят, что у пользователя имеются или отсутствуют привилегии на выполнение действия, а полномочие отражает привилегию, которую имеет пользователь. Мы реализуем метод `getAuthorities()` для получения ссылки на группу полномочий, предоставленных пользователю.

ПРИМЕЧАНИЕ Как вы узнаете в главе 6, Spring Security использует полномочия для ссылки либо на детальные привилегии, либо на роли, которые являются группами привилегий. Чтобы облегчить чтение, в этой книге я называю детальные привилегии полномочиями.

Кроме того, как видно из контракта `UserDetails`, с пользователем могут произойти следующие события:

- его учетная запись может быть просрочена;
- его учетная запись может быть заблокирована;

- его учетные данные могут быть просрочены;
- его учетная запись может быть отключена.

Предположим, вы решили реализовать эти ограничения пользователя в логике своего приложения. В этом случае вам нужно реализовать методы `isAccountNonExpired()`, `isAccountNonLocked()`, `isCredentialsNonExpired()` и `isEnabled()`, чтобы для отключенных ограничений возвращалось значение `true`. Не во всех приложениях есть учетные записи, срок действия которых истекает или которые блокируются при определенных условиях. Если вам не нужно реализовывать эту функциональность в своем приложении, вы можете просто заставить эти четыре метода возвращать `true`.

ПРИМЕЧАНИЕ Названия последних четырех методов в интерфейсе `UserDetails` могут показаться странными. Можно подумать, что они выбраны неразумно с точки зрения чистого кодирования и удобства обслуживания. Например, имя `isAccountNonExpired()` выглядит как двойная негативная коннотация (не просрочен) и на первый взгляд может вызвать путаницу. Но давайте внимательно проанализируем все четыре имени методов. Они названы так, потому что все они возвращают `false`, когда авторизация должна быть неудачной, и `true` в противном случае. Это правильный подход, поскольку человеческий разум склонен ассоциировать слово `false` (ложь) с негативом, а `true` (истина) – с позитивом.

3.2.2 Детализация контракта *GrantedAuthority*

Как вы заметили в определении интерфейса `UserDetails` в разделе 3.2.1, действия, доступные пользователю, называются полномочиями. В главах с 7 по 12 мы будем писать конфигурации авторизации на основе этих полномочий. Поэтому важно знать, как их определять.

Без полномочий все пользователи были бы равны. Хотя существуют простые приложения, в которых пользователи равны, в большинстве практических сценариев приложение определяет несколько категорий пользователей. Приложение может иметь пользователей, которым разрешено только читать определенную информацию, в то время как другие могут также изменять данные. И вам нужно сделать так, чтобы ваше приложение предоставляло пользователям функциональные возможности в соответствии с полномочиями. Для описания полномочий в Spring Security используют интерфейс `GrantedAuthority`.

Но перед обсуждением реализации `UserDetails` нужно разобраться с интерфейсом `GrantedAuthority`. Мы используем этот интерфейс в определении сведений о пользователе. Он представляет собой

привилегию, предоставленную пользователю. Пользователь должен иметь по крайней мере одно полномочие. Вот реализация определения `GrantedAuthority`:

```
public interface GrantedAuthority extends Serializable {  
    String getAuthority();  
}
```

Чтобы создать полномочие, вам нужно только найти имя для этой привилегии, чтобы вы могли ссылаться на него позже при написании правил авторизации. Например, пользователь может читать записи, извлекаемые приложением, или удалять их. Вы пишете правила авторизации на основе имен, которые вы даете этим действиям.

В этой главе мы реализуем метод `getAuthority()`, возвращающий имя полномочия в виде строки. Интерфейс `GrantedAuthority` имеет только один абстрактный метод, и в этой книге вы часто будете встречать примеры, в которых мы используем лямбда-выражение для его реализации. Альтернативный способ – использовать класс `SimpleGrantedAuthority` для создания экземпляров полномочий. Этот класс предлагает способ создания неизменяемых экземпляров типа `GrantedAuthority`. Вы указываете имя полномочия при создании экземпляра. В следующем фрагменте кода показаны два примера реализации `GrantedAuthority`. Здесь мы используем лямбда-выражение, а затем класс `SimpleGrantedAuthority`:

```
GrantedAuthority g1 = () -> "READ";  
GrantedAuthority g2 = new SimpleGrantedAuthority("READ");
```

3.2.3 Минимальная реализация *UserDetails*

В этом разделе вы напишете свою первую реализацию контракта `UserDetails`. Начнем с базовой реализации, в которой каждый метод возвращает статическое значение. Затем мы изменим ее на версию, с которой вы, скорее всего, столкнетесь на практике. Она позволяет вам иметь несколько разных экземпляров пользователей. Когда вы освоите реализацию интерфейсов `UserDetails` и `GrantedAuthority`, мы сможем написать простейшее определение пользователя для приложения.

Итак, давайте реализуем минимальное описание пользователя с помощью класса `DummyUser`, показанного в листинге 3.2. Я использую этот класс в основном для демонстрации реализации методов контракта `UserDetails`. Экземпляры этого класса всегда ссылаются только на одного пользователя "bill", у которого есть пароль "12345" и привилегии с именем "READ".

Листинг 3.2 Класс `DummyUser`

```
public class DummyUser implements UserDetails {

    @Override
    public String getUsername() {
        return "bill";
    }

    @Override
    public String getPassword() {
        return "12345";
    }

    // Часть кода не показана

}
```

Класс в листинге 3.2 реализует интерфейс `UserDetails` и нуждается в реализации всех его методов. Здесь показаны реализации `getUsername()` и `getPassword()`. В данном примере эти методы возвращают только фиксированное значение для каждого из свойств.

Далее мы добавляем определение списка полномочий. В следующем листинге показана реализация метода `getAuthorities()`. Этот метод возвращает коллекцию только с одной реализацией интерфейса `GrantedAuthority`.

Листинг 3.3 Реализация метода `getAuthorities()`

```
public class DummyUser implements UserDetails {

    // Часть кода не показана

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(() -> "READ");
    }

    // Часть кода не показана

}
```

Наконец, вам нужно добавить реализацию для последних четырех методов интерфейса `User Details`. Для класса `DummyUser` они всегда возвращают `true`, то есть пользователь всегда активен и его учетную запись можно использовать. Примеры реализации показаны в листинге 3.4.

Листинг 3.4 Реализация последних четырех методов интерфейса UserDetails

```
public class DummyUser implements UserDetails {  
  
    // Часть кода не показана  
  
    @Override  
    public boolean isAccountNonExpired() {  
        return true;  
    }  
  
    @Override  
    public boolean isAccountNonLocked() {  
        return true;  
    }  
  
    @Override  
    public boolean isCredentialsNonExpired() {  
        return true;  
    }  
  
    @Override  
    public boolean isEnabled() {  
        return true;  
    }  
  
    // Часть кода не показана  
}
```

Конечно, эта минимальная реализация означает, что все экземпляры класса представляют одного и того же пользователя. Это хорошее начало для понимания контракта, но так не делают в реальном приложении. Для реального приложения вам следует создать класс, который вы можете использовать для генерации экземпляров, представляющих разных пользователей. В этом случае ваше определение будет иметь по крайней мере имя пользователя и пароль в качестве атрибутов в классе, как показано в листинге 3.5.

Листинг 3.5 Более реалистичная реализация интерфейса UserDetails

```
public class SimpleUser implements UserDetails {  
  
    private final String username;  
    private final String password;  
  
    public SimpleUser(String username, String password) {  
        this.username = username;  
        this.password = password;  
    }  
}
```

```
@Override
public String getUsername() {
    return this.username;
}

@Override
public String getPassword() {
    return this.password;
}

// Часть кода не показана
}
```

3.2.4 *Использование конструктора для создания экземпляров типа UserDetails*

Некоторые приложения просты и не нуждаются в пользовательской реализации интерфейса `UserDetails`. В этом разделе мы рассмотрим использование класса-построителя, предоставляемого `Spring Security`, для создания простых экземпляров пользователя. Вместо того чтобы объявлять еще один класс в вашем приложении, вы быстро получаете экземпляр, представляющий вашего пользователя с помощью класса-построителя `User`.

Класс `User` из пакета `org.springframework.security.core.userdetails` – это простой способ создания экземпляров типа `UserDetails`. Используя этот класс, вы можете создавать неизменяемые экземпляры `UserDetails`. Вам необходимо указать как минимум имя пользователя и пароль, и имя пользователя не должно быть пустой строкой. Следующий листинг демонстрирует применение построителя. При создании пользователя этим способом вам не нужно иметь пользовательскую реализацию контракта `UserDetails`.

Листинг 3.6 Создание пользователя с помощью класса-построителя `User`

```
UserDetails u = User.withUsername("bill")
    .password("12345")
    .authorities("read", "write")
    .accountExpired(false)
    .disabled(true)
    .build();
```

Опираясь на предыдущий листинг в качестве примера, углубимся в анатомию класса-построителя `User`. Метод `User.withUsername(String username)` возвращает экземпляр класса-построителя `UserBuilder`, вложенного в класс `User`. Еще один способ создания построителя – начать с другого экземпляра `UserDetails`. В листинге 3.7

первая строка создает `UserBuilder`, начиная с имени пользователя, заданного в виде строки. Далее я покажу, как создать построитель, начиная с уже существующего экземпляра `UserDetails`.

Листинг 3.7 Создание экземпляра `User.UserBuilder`

```
User.UserBuilder builder1 = User.withUsername("bill"); ❶

UserDetails u1 = builder1
    .password("12345")
    .authorities("read", "write")
    .passwordEncoder(p -> encode(p)) ❷
    .accountExpired(false)
    .disabled(true)
    .build(); ❸

User.UserBuilder builder2 = User.withUserDetails(u); ❹

UserDetails u2 = builder2.build();
```

❶ Создаем пользователя с заданным именем.

❷ Кодировщик пароля – это всего лишь функция, которая выполняет кодирование.

❸ В конце конвейера сборки вызываем метод `build()`.

❹ Вы также можете создать пользователя из существующего экземпляра `UserDetails`.

Из кода построителей в листинге 3.7 следует, что мы можем применить построитель для получения пользователя, представленного контрактом `UserDetails`. В конце конвейера сборки вы вызываете метод `build()`. Он применяет функцию, определенную для кодирования пароля, если вы ее указали, создает экземпляр `UserDetails` и возвращает его.

ПРИМЕЧАНИЕ Обратите внимание, что кодировщик пароля здесь указан как `Function<String, String>`, а не в форме интерфейса `PasswordEncoder`, предоставляемого `Spring Security`. Единственная ответственность этой функции – преобразовать пароль в заданную кодировку. В следующей главе мы подробно обсудим контракт `PasswordEncoder` из `Spring Security`, который мы использовали в главе 2.

3.2.5 Объединение нескольких ответственностей, связанных с пользователем

В предыдущем разделе вы узнали, как реализовать интерфейс `UserDetails`. На практике реализация часто бывает сложнее. В большинстве случаев пользователь связан с несколькими ответственностями. И если вы храните пользователей сначала в базе данных, а затем в приложении, вам также понадобится класс для представления

сущности сохранения. Или если вы извлекаете пользователей через веб-службу из другой системы, то вам, вероятно, понадобится объект передачи данных для представления экземпляров пользователей. Рассмотрим первый случай – простой и наиболее типичный. Предположим, что у нас есть таблица в базе данных SQL, где мы храним пользователей. Чтобы сделать пример короче, мы дадим каждому пользователю только одно полномочие. В листинге 3.8 показан класс-сущность, отображающий таблицу.

Листинг 3.8 Определение класса-сущности JPA User

```
@Entity
public class User {

    @Id
    private Long id;
    private String username;
    private String password;
    private String authority;

    // Геттеры и сеттеры не показаны
}
```

Если вы сделаете тот же класс также реализующим контракт Spring Security для сведений о пользователе, он станет сложнее. Что вы скажете о том, как выглядит код в листинге 3.9? По-моему, это беспорядок. Я бы в нем запутался.

Листинг 3.9 Класс User имеет две ответственности

```
@Entity
public class User implements UserDetails {

    @Id
    private int id;
    private String username;
    private String password;
    private String authority;

    @Override
    public String getUsername() {
        return this.username;
    }

    @Override
    public String getPassword() {
        return this.password;
    }

    public String getAuthority() {
```



```

    return this.authority;
}

@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(() -> authority);
}

// Часть кода не показана
}

```

Класс содержит аннотации JPA, геттеры и сеттеры, из которых оба – `getUsername()` и `getPassword()` – переопределяют методы в контракте `UserDetails`. У него есть метод `getAuthority()`, который возвращает `String`, а также метод `getAuthorities()`, который возвращает `Collection`. Метод `getAuthority()` – это просто геттер в классе, в то время как `getAuthorities()` реализует метод в интерфейсе `UserDetails`. Все становится еще сложнее при добавлении отношений с другими сущностями. К тому же этот код совсем не дружелюбен и сложен для понимания!

Как нам переписать этот код, чтобы он был чище? Причина путаницы в предыдущем примере – это смесь двух ответственностей. Хотя они действительно обе нужны в приложении, никто не говорит, что вы должны помещать их в один класс. Давайте попробуем разделить их, определив отдельный класс `SecurityUser`, который адаптирует класс `User`. Как показано в листинге 3.10, класс `SecurityUser` реализует контракт `UserDetails` и использует его для подключения нашего пользователя к архитектуре Spring Security. У класса `User` осталась только его ответственность за сущность JPA.

Листинг 3.10 Реализация класса `User` только как сущности JPA

```

@Entity
public class User {

    @Id
    private int id;
    private String username;
    private String password;
    private String authority;

    // Геттеры и сеттеры не показаны
}

```

Класс `User` в листинге 3.10 имеет только свою ответственность сущности JPA, и, таким образом, он становится более читабельным. Если вы прочитаете этот код, вы теперь можете сосредоточиться исключительно на деталях, связанных с сохранением, которые не важ-

ны с точки зрения Spring Security. В листинге 3.11 показана реализация класса `SecurityUser` для обертывания сущности `User`.

Листинг 3.11 Класс `SecurityUser`, реализующий контракт `UserDetails`

```
public class SecurityUser implements UserDetails {

    private final User user;

    public SecurityUser(User user) {
        this.user = user;
    }

    @Override
    public String getUsername() {
        return user.getUsername();
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(() -> user.getAuthority());
    }

    // Часть кода не показана
}
```

Как вы могли заметить, мы используем класс `SecurityUser` только для сопоставления сведений о пользователе в системе с контрактом `UserDetails`, понятным Spring Security. Чтобы отметить тот факт, что `SecurityUser` не имеет смысла без сущности `User`, мы объявляем поле как `final`. Вам необходимо предоставить пользователя через построитель. Класс `SecurityUser` адаптирует класс сущности `User` и добавляет необходимый код, связанный с контрактом Spring Security, не смешивая код с сущностью JPA, тем самым реализуя несколько различных задач.

ПРИМЕЧАНИЕ Существуют разные способы разделения ответственностей. Я не хочу сказать, что подход, выбранный мной в этом разделе, является лучшим или единственным. Обычно способ, выбираемый для реализации конструкции класса, сильно зависит от ситуации. Но основная идея неизменна: избегайте смешивания ответственностей и старайтесь писать максимально разделенный код, чтобы облегчить поддержку вашего приложения.

3.3 Инструктирование Spring Security о том, как управлять пользователями

Ранее вы реализовали контракт `UserDetailsService` для описания пользователей способом, понятным Spring Security. Но как Spring Security управляет пользователями? Откуда они извлекаются при сравнении учетных данных и как происходит добавление новых пользователей или изменение существующих? В главе 2 вы узнали, что фреймворк определяет конкретный компонент, которому процесс аутентификации делегирует управление пользователями: экземпляр `UserDetailsService`. Мы даже определили `UserDetailsService` для переопределения реализации по умолчанию, предоставляемой Spring Boot.

В этом разделе мы экспериментируем с различными способами реализации класса `UserDetailsService`. Вы поймете, как работает управление пользователями, реализуя ответственность, описанную контрактом `UserDetailsService` в нашем примере. Затем вы узнаете, как интерфейс `UserDetailsManager` добавляет новое поведение к контракту, определенному `UserDetailsService`. В конце этого раздела мы воспользуемся реализациями интерфейса `UserDetailsManager`, предоставленными Spring Security. Мы напишем пример проекта, в котором будем использовать одну из самых известных реализаций, предоставляемых Spring Security, – класс `JdbcUserDetailsManager`. Изучив пример, вы узнаете, как сообщить Spring Security, где искать пользователей (без чего не обойтись в потоке аутентификации).

3.3.1 Подробнее о контракте `UserDetailsService`

Далее речь пойдет об определении интерфейса `UserDetailsService`. Но прежде чем узнать, как и зачем нужно его реализовывать, следует понять контракт. Пришло время более подробно рассмотреть `UserDetailsService` и принцип работы с реализациями этого компонента. Интерфейс `UserDetailsService` содержит только один метод:

```
public interface UserDetailsService {  
  
    UserDetails loadUserByUsername(String username)  
        throws UsernameNotFoundException;  
}
```

Реализация аутентификации вызывает метод `loadUserByUsername(String username)` для получения сведений о пользователе с заданным именем (рис. 3.3). Имя пользователя, конечно же, считается уникальным. Пользователь, возвращаемый этим методом, является реализацией контракта `UserDetails`. Если имя пользователя не существует, метод выдает исключение `UsernameNotFoundException`.

ПРИМЕЧАНИЕ `UsernameNotFoundException` является исключением `RuntimeException`. Оператор `throws` в интерфейсе `UserDetailsService` предназначен только для целей документирования. Исключение `UsernameNotFoundException` наследуется напрямую от типа `AuthenticationException`, который является родителем всех исключений, связанных с процессом аутентификации. Исключение `AuthenticationException` далее наследует класс `RuntimeException`.

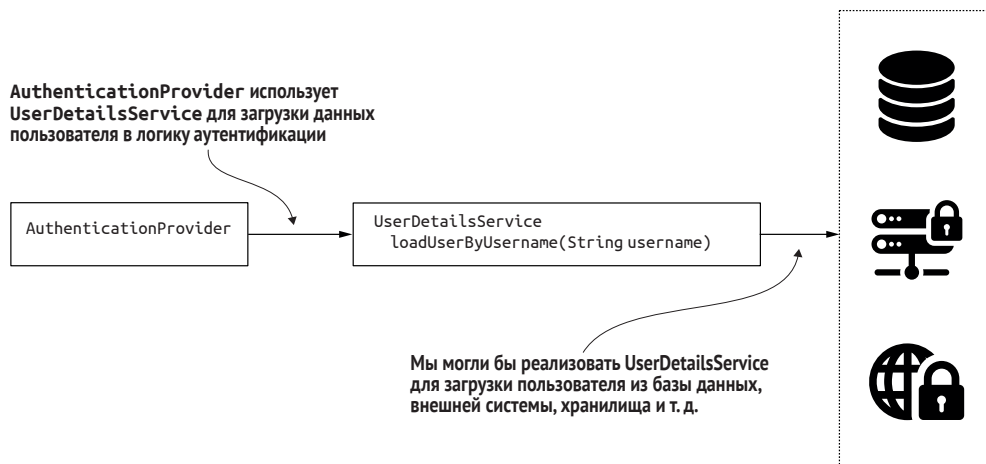


Рис. 3.3 AuthenticationProvider – элемент, отвечающий за выполнение процесса аутентификации и использующий UserDetailsService для сбора данных пользователя. Он вызывает метод `loadUserByUsername(String username)` для поиска пользователя по его имени

3.3.2 Реализация контракта UserDetailsService

В этом разделе мы рассмотрим практический пример, демонстрирующий реализацию `UserDetailsService`. Ваше приложение управляет сведениями об учетных данных и других аспектах пользователя. Они могут храниться в базе данных или обрабатываться другой системой, к которой вы получаете доступ через веб-службу или иными способами (рис. 3.3). Независимо от того, как это происходит в вашей системе, единственное, что требуется Spring Security от вас, – это реализация для извлечения пользователя по его имени.

В следующем примере мы разработали `UserDetailsService`, который имеет список пользователей в памяти. В главе 2 вы использовали реализацию по умолчанию `InMemoryUserDetailsManager`, которая делает то же самое. Поскольку вы уже знакомы с принципом работы этой реализации, я выбрал похожую функциональность, но на этот раз реализованную самостоятельно. Мы предоставляем список пользователей, когда создаем экземпляр нашего класса `UserDe-`

tailsService. Вы можете найти этот пример в проекте ssia-ch3-ex1. В пакете с именем model мы определяем UserDetails, как представлено в листинге 3.12.

Листинг 3.12 Реализация интерфейса UserDetails

```
public class User implements UserDetails {  
  
    private final String username;           ❶  
    private final String password;  
    private final String authority;         ❷  
  
    public User(String username, String password, String authority) {  
        this.username = username;  
        this.password = password;  
        this.authority = authority;  
    }  
  
    @Override  
    public Collection<? extends GrantedAuthority> getAuthorities() {  
        return List.of(() -> authority);    ❸  
    }  
  
    @Override  
    public String getPassword() {  
        return password;  
    }  
  
    @Override  
    public String getUsername() {  
        return username;  
    }  
  
    @Override  
    public boolean isAccountNonExpired() {   ❹  
        return true;  
    }  
  
    @Override  
    public boolean isAccountNonLocked() {  
        return true;  
    }  
  
    @Override  
    public boolean isCredentialsNonExpired() {  
        return true;  
    }  
  
    @Override  
    public boolean isEnabled() {  
        return true;  
    }  
}
```

- ❶ Класс `User` является неизменяемым. Вы задаете значения для трех атрибутов при создании экземпляра, и эти значения не могут быть изменены впоследствии.
- ❷ Для простоты примера у пользователя есть только одно полномочие.
- ❸ Возвращаем список, содержащий только объект `GrantedAuthority` с именем, предоставленным при создании экземпляра.
- ❹ Учетная запись не устаревает и не блокируется.

В пакете с именем `services` мы создаем класс с именем `InMemoryUserDetailsService`. В листинге 3.13 показана реализация этого класса.

Листинг 3.13 Реализация интерфейса `UserDetailsService`

```
public class InMemoryUserDetailsService implements UserDetailsService {  
    private final List<UserDetails> users; ❶  
  
    public InMemoryUserDetailsService(List<UserDetails> users) {  
        this.users = users;  
    }  
  
    @Override  
    public UserDetails loadUserByUsername(String username)  
        throws UsernameNotFoundException {  
  
        return users.stream()  
            .filter(  
                u -> u.getUsername().equals(username)  
            )  
            .findFirst() ❷  
            .orElseThrow(  
                () -> new UsernameNotFoundException("User not found") ❸  
            );  
    }  
}
```

- ❶ `UserDetailsService` управляет списком пользователей в памяти.
- ❷ Если такой пользователь есть, возвращаем его.
- ❸ Если пользователя с таким именем пользователя не существует, выдаем исключение.

Метод `loadUserByUsername(String username)` ищет в списке пользователей указанное имя пользователя и возвращает нужный экземпляр `UserDetails`. Если экземпляра с таким именем пользователя нет, выдает исключение `UsernameNotFoundException`. Теперь мы можем использовать эту реализацию в качестве нашего `UserDetailsService`. В следующем листинге показано, как мы добавляем его как компонент в класс конфигурации и регистрируем в нем одного пользователя.

Листинг 3.14 UserDetailsService зарегистрирован как бин в классе конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails u = new User("john", "12345", "read");
        List<UserDetails> users = List.of(u);
        return new InMemoryUserDetailsService(users);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

Наконец, мы создаем простую конечную точку и тестируем реализацию. В листинге 3.15 определена конечная точка.

Листинг 3.15 Определение конечной точки, используемой для тестирования реализации

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello!";
    }
}
```

При вызове конечной точки с помощью cURL мы видим, что для пользователя john с паролем 12345 мы получаем HTTP 200 OK. Если мы используем что-то другое, приложение возвращает ответ 401 Unauthorized:

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

```
Hello!
```

3.3.3 Реализация контракта UserDetailsManager

В этом разделе мы обсудим использование и реализацию интерфейса UserDetailsManager. Этот интерфейс расширяет и добавляет методы к контракту UserDetailsService. Spring Security нуждается

в контракте `UserDetailsService` для выполнения аутентификации. Но, как правило, в приложениях также необходимо управлять пользователями. В большинстве случаев приложение должно иметь возможность добавлять новых пользователей или удалять существующих. В этом случае мы реализуем более конкретный интерфейс, определенный Spring Security, – `UserDetails Manager`. Он расширяет `UserDetailsService` и добавляет больше операций, которые нам необходимо реализовать:

```
public interface UserDetailsManager extends UserDetailsService {  
    void createUser(UserDetails user);  
    void updateUser(UserDetails user);  
    void deleteUser(String username);  
    void changePassword(String oldPassword, String newPassword);  
    boolean userExists(String username);  
}
```

Объект `InMemoryUserDetailsManager`, который мы использовали в главе 2, на самом деле представляет собой `UserDetailsManager`. В то время мы рассматривали только его характеристики `UserDetailsService`. К этому разделу относится проект `ssia-ch3-ex2`.

Использование `JdbcuserdetailsManager` для управления пользователями

Помимо `InMemoryUserDetailsManager`, мы часто используем другую реализацию `UserDetail Manager` – `JdbcUserDetailsManager`. Этот класс управляет пользователями в базе данных SQL. Он подключается к базе данных напрямую через JDBC. Таким образом, `JdbcUserDetailsManager` не зависит от какой-либо другой структуры или спецификации, связанной с подключением к базе данных.

Чтобы понять, как работает `JdbcUserDetailsManager`, лучше всего применить его на практике. В следующем примере вы реализуете приложение, которое управляет пользователями в базе данных MySQL с помощью `JdbcUserDetailsManager`. На рис. 3.4 схематически показано, какое место реализация `JdbcUserDetailsManager` занимает в потоке аутентификации.

Мы начнем работу над нашим демонстрационным приложением, которое использует `JdbcUserDetailsManager`, с создания базы данных и двух таблиц. В данном случае мы назовем базу данных `spring`, одну из таблиц – `users`, а другую – `authorities`. Это имена таблиц по умолчанию, известные `JdbcUserDetailsManager`. Как вы узнаете в конце этого раздела, реализация `JdbcUserDetailsManager` является гибкой и позволяет вам переопределять эти имена по умолчанию, если вы этого хотите. Назначение таблицы `users` – хранить записи пользователей. Реализация `JdbcUserDetailsManager` ожидает три столбца

в таблице пользователей – имя пользователя, пароль и состояние (активен/неактивен), которое вы можете использовать для деактивации пользователя.



Рис. 3.4 Поток аутентификации Spring Security. Здесь мы используем `JdbcUserDetailsService` в качестве нашего компонента `UserDetailsService`. `JdbcUserDetailsService` использует базу данных для управления пользователями

Вы можете создать базу данных и ее структуру самостоятельно, используя либо инструмент командной строки для вашей системы управления базами данных (СУБД), либо клиентское приложение. Например, для работы с MySQL вы можете использовать MySQL Workbench. Но проще всего будет позволить Spring Boot самому запускать скрипты за вас. Для этого просто добавьте еще два файла в свой проект в папке ресурсов: `schema.sql` и `data.sql`. В файле `schema.sql` вы добавляете запросы, связанные со структурой базы данных, такие как создание, изменение или удаление таблиц. В файле `data.sql` вы добавляете запросы, которые работают с данными внутри таблиц, например `INSERT`, `UPDATE` или `DELETE`. Spring Boot автоматически запускает эти файлы для вас при запуске приложения. Более простое решение для создания примеров, которым нужны базы данных, – это использование базы данных H2 в памяти. В таком случае вам не придется устанавливать отдельное решение СУБД.

ПРИМЕЧАНИЕ Вы тоже можете использовать H2 при разработке приложений, представленных в этой книге, как я делаю в проекте ssia-ch3-ex2. Но в большинстве случаев я решил строить примеры с использованием внешней СУБД, чтобы было ясно, что это внешний компонент системы, и таким образом избежать путаницы.

Используйте код из листинга 3.16 для создания таблицы `users` на сервере MySQL. Вы можете добавить этот скрипт в файл `schema.sql` в вашем проекте Spring Boot.

Листинг 3.16 SQL-запрос для создания таблицы `users`

```
CREATE TABLE IF NOT EXISTS `spring`.`users` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `username` VARCHAR(45) NOT NULL,  
  `password` VARCHAR(45) NOT NULL,  
  `enabled` INT NOT NULL,  
  PRIMARY KEY (`id`));
```

Таблица `authorities` хранит полномочия для каждого пользователя. Каждая запись хранит имя и полномочия, предоставленные пользователю с этим именем пользователя.

Листинг 3.17 SQL-запрос для создания таблицы полномочий

```
CREATE TABLE IF NOT EXISTS `spring`.`authorities` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `username` VARCHAR(45) NOT NULL,  
  `authority` VARCHAR(45) NOT NULL,  
  PRIMARY KEY (`id`));
```

ПРИМЕЧАНИЕ Для простоты и чтобы вы могли сосредоточиться на конфигурациях Spring Security, которые мы обсуждаем, в примерах, предоставленных в этой книге, я пропускаю определения индексов или внешних ключей.

Чтобы гарантировать наличие пользователя для тестирования, вставьте запись в каждую из таблиц. Для этого достаточно добавить следующие запросы в файл `data.sql` в папке ресурсов проекта Spring Boot:

```
INSERT INTO `spring`.`authorities`  
(username, authority)  
VALUES  
('john', 'write');
```

```
INSERT INTO `spring`.`users`  
(username, password, enabled)  
VALUES  
('john', '12345', '1');
```

Для вашего проекта необходимо добавить как минимум зависимости, указанные в листинге 3.18. Проверьте файл `pom.xml`, чтобы убедиться, что вы добавили эти зависимости.

Листинг 3.18 Зависимости, необходимые для разработки учебного проекта

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>  
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>  
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-jdbc</artifactId>  
</dependency>  
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
</dependency>
```

ПРИМЕЧАНИЕ В вашем коде вы можете использовать любую технологию базы данных SQL, если добавите правильный драйвер JDBC к зависимостям.

Помните, вам необходимо добавить драйвер JDBC в соответствии с технологией базы данных, которую вы используете. Например, если вы используете MySQL, вам нужно добавить зависимость драйвера MySQL, как показано в следующем фрагменте кода:

```
<dependency>  
  <groupId>mysql</groupId>  
  <artifactId>mysql-connector-java</artifactId>  
  <scope>runtime</scope>  
</dependency>
```

Вы можете настроить источник данных в файле `application.properties` проекта или как отдельный компонент. Если вы решили использовать файл `application.properties`, вам нужно добавить в этот файл следующие строки:

```
spring.datasource.url=jdbc:h2:mem:ssia
spring.datasource.username=sa
spring.datasource.password=
spring.sql.init.mode=always
```

В классе конфигурации проекта вы определяете `UserDetailsService` и `PasswordEncoder`. `JdbcUserDetailsManager` требуется `DataSource` для подключения к базе данных. Источник данных может быть автоматически подключен через параметр метода (как показано в следующем листинге) или через атрибут класса.

Листинг 3.19 Регистрация `JdbcUserDetailsManager` в классе конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService(DataSource dataSource) {
        return new JdbcUserDetailsManager(dataSource);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

Для доступа к любой конечной точке приложения теперь необходимо использовать аутентификацию HTTP Basic с учетными данными одного из пользователей, сохраненных в базе данных. Чтобы убедиться в этом, создадим новую конечную точку, как показано в следующем листинге, а затем вызываем ее с помощью cURL.

Листинг 3.20 Тестовая конечная точка для проверки реализации

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello!";
    }
}
```

Если теперь вызывать конечную точку с правильным именем пользователя и паролем

```
curl -u john:12345 http://localhost:8080/hello
```

то получим ожидаемый на вызов:

Hello!

`JdbcUserDetailsManager` также позволяет настраивать используемые запросы. В предыдущем примере нам приходилось указывать точные имена для таблиц и столбцов, поскольку реализация `JdbcUserDetailsManager` ожидает их. Но может оказаться, что эти имена плохо подходят для вашего приложения. В листинге 3.21 показано, как переопределить запросы для `JdbcUserDetailsManager`.

Листинг 3.21 Изменение запросов `JdbcUserDetailsManager` для поиска пользователя

```
@Bean
public UserDetailsService userDetailsService(DataSource dataSource) {
    String usersByUsernameQuery =
        "select username, password, enabled from users where username = ?";
    String authsByUserQuery =
        "select username, authority from spring.authorities where username = ?";

    var userDetailsManager = new JdbcUserDetailsManager(dataSource);
    userDetailsManager.setUsersByUsernameQuery(usersByUsernameQuery);
    userDetailsManager.setAuthoritiesByUsernameQuery(authsByUserQuery);
    return userDetailsManager;
}
```

Точно так же мы можем изменить все запросы, используемые реализацией `JdbcUserDetailsManager`.

УПРАЖНЕНИЕ Напишите аналогичное приложение, для которого вы по-разному назовете таблицы и столбцы в базе данных. Переопределите запросы для реализации `JdbcUserDetailsManager` (например, аутентификация работает с новой структурой таблиц). Возможное решение можно найти в проекте `ssia-ch3-ex2`.

Использование `LdapuserdetailsManager` для управления пользователями

Spring Security также предлагает реализацию интерфейса `UserDetailsService` для LDAP. И хотя он менее популярен, чем `JdbcUserDetailsManager`, вы можете рассчитывать на него, если вам нужно интегрироваться с системой LDAP для управления пользователями. В проекте `ssia-ch3-ex3` вы найдете простую демонстрацию применения `LdapUserDetailsManager`. Поскольку я не могу использовать настоящий сервер LDAP для этой демонстрации, я настроил встроенный сервер в своем приложении Spring Boot. Чтобы настроить

встроенный сервер LDAP, я определил простой файл формата обмена данными LDAP (LDAP Data Interchange Format, LDIF). В листинге 3.22 показано содержимое моего файла LDIF.

Листинг 3.22 Определение файла LDIF

```
dn: dc=springframework,dc=org ❶
objectclass: top
objectclass: domain
objectclass: extensibleObject
dc: springframework

dn: ou=groups,dc=springframework,dc=org ❷
objectclass: top
objectclass: organizationalUnit
ou: groups

dn: uid=john,ou=groups,dc=springframework,dc=org ❸
objectclass: top
objectclass: person
objectclass: organizationalPerson
objectclass: inetOrgPerson
cn: John
sn: John
uid: john
userPassword: 12345
```

- ❶ Определяем базовую сущность.
- ❷ Определяем групповую сущность.
- ❸ Определяем пользователя.

В файле LDIF я добавляю только одного пользователя, для которого нам нужно протестировать поведение приложения в конце этого примера. Мы можем добавить файл LDIF непосредственно в папку ресурсов. Благодаря этому он автоматически попадает в classpath, поэтому мы можем легко сослаться на него позже. Я назвал файл LDIF server.ldif. Для работы с LDAP и для того, чтобы Spring Boot мог запустить встроенный сервер LDAP, необходимо добавить pom.xml в зависимости:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-ldap</artifactId>
</dependency>
<dependency>
  <groupId>com.unboundid</groupId>
  <artifactId>unboundid-ldapsdk</artifactId>
</dependency>
```

В файле application.properties также требуется добавить конфигурации для встроенного сервера LDAP, как показано в следующем

фрагменте кода. Значения, необходимые приложению для загрузки встроенного сервера LDAP, включают местоположение файла LDIF, порт для сервера LDAP и значения метки компонента базового домена (DN):

```
spring.ldap.embedded.ldif=classpath:server.ldif
spring.ldap.embedded.base-dn=dc=springframework,dc=org
spring.ldap.embedded.port=33389
```

После подготовки сервера LDAP для аутентификации нужно настроить приложение для его использования. В следующем листинге показано, как настроить `LdapUserDetailsManager`, чтобы ваше приложение могло аутентифицировать пользователей через сервер LDAP.

Листинг 3.23 Определение `LdapUserDetailsManager` в файле конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var cs = new DefaultSpringSecurityContextSource(
            "ldap://127.0.0.1:33389/dc=springframework,dc=org");
        cs.afterPropertiesSet();

        var manager = new LdapUserDetailsManager(cs);

        manager.setUsernameMapper(
            new DefaultLdapUsernameToDnMapper("ou=groups", "uid"));

        manager.setGroupSearchBase("ou=groups");

        return manager;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

- ❶ Добавляем реализацию `UserDetailsService` в контекст Spring.
- ❷ Создаем источник контекста для указания адреса сервера LDAP.
- ❸ Создаем экземпляр `LdapUserDetailsManager`.
- ❹ Настраиваем сопоставитель имен пользователей, чтобы указать `LdapUserDetailsManager`, как искать пользователей.
- ❺ Настраиваем базу поиска групп, необходимую приложению для поиска пользователей.

Давайте также создадим простую конечную точку для проверки конфигурации безопасности. Я добавил класс контроллера, как показано в следующем фрагменте кода:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello!";
    }
}
```

Теперь запустите приложение и вызовите конечную точку `/hello`. Если вы хотите, чтобы приложение разрешило вам вызывать конечную точку, необходимо пройти аутентификацию с пользователем `john`. Следующий фрагмент кода показывает результат вызова конечной точки с помощью `cURL`:

```
curl -u john:12345 http://localhost:8080/hello
```

Ответ на вызов:

Hello!

Заключение

- Интерфейс `UserDetails` – это контракт, который вы используете для описания пользователя в Spring Security.
- Интерфейс `UserDetailsService` – это контракт, который вы должны реализовать в архитектуре аутентификации для описания способа, которым приложение получает данные пользователя.
- Интерфейс `UserDetailsManager` расширяет `UserDetailsService` и добавляет поведение, связанное с созданием, изменением или удалением пользователя.
- Spring Security предоставляет несколько реализаций контракта `UserDetailsManager`. Среди них `InMemoryUserDetailsManager`, `JdbcUserDetailsManager` и `LdapUserDetailsManager`.
- Преимущества класса `JdbcUserDetailsManager` – в прямом использовании JDBC и отсутствии привязки приложения к другим фреймворкам.

Управление паролями

Темы этой главы:

- реализация и работа с PasswordEncoder;
- использование инструментов, предлагаемых модулем Spring Security Crypto.

В главе 3 мы обсуждали управление пользователями в приложении, реализованном с помощью Spring Security. Но что насчет паролей? Они, безусловно, являются неотъемлемой частью процесса аутентификации. В этой главе вы узнаете, как управлять паролями и секретами в приложении, разработанном с применением Spring Security. Мы обсудим контракт PasswordEncoder и инструменты, предлагаемые модулем Spring Security Crypto (SSCM) для управления паролями.

4.1 Использование кодировщиков паролей

После прочтения главы 3 вы должны иметь четкое представление о том, что такое интерфейс UserDetails и каковы способы использования его реализации. Но, как вы узнали в главе 2, различные субъекты могут управлять представлением пользователей во время процессов аутентификации и авторизации. Вы также узнали, что некоторые из них, такие как UserDetailsService и PasswordEncoder,

имеют значения по умолчанию. Теперь вы знаете, что можете переопределить значения по умолчанию. Мы продолжаем углубляться в изучение этих bean-компонентов и способов их реализации, поэтому следующим шагом мы проанализируем `PasswordEncoder`. Рисунок 4.1 напоминает вам о том, где `PasswordEncoder` включается в процесс аутентификации.

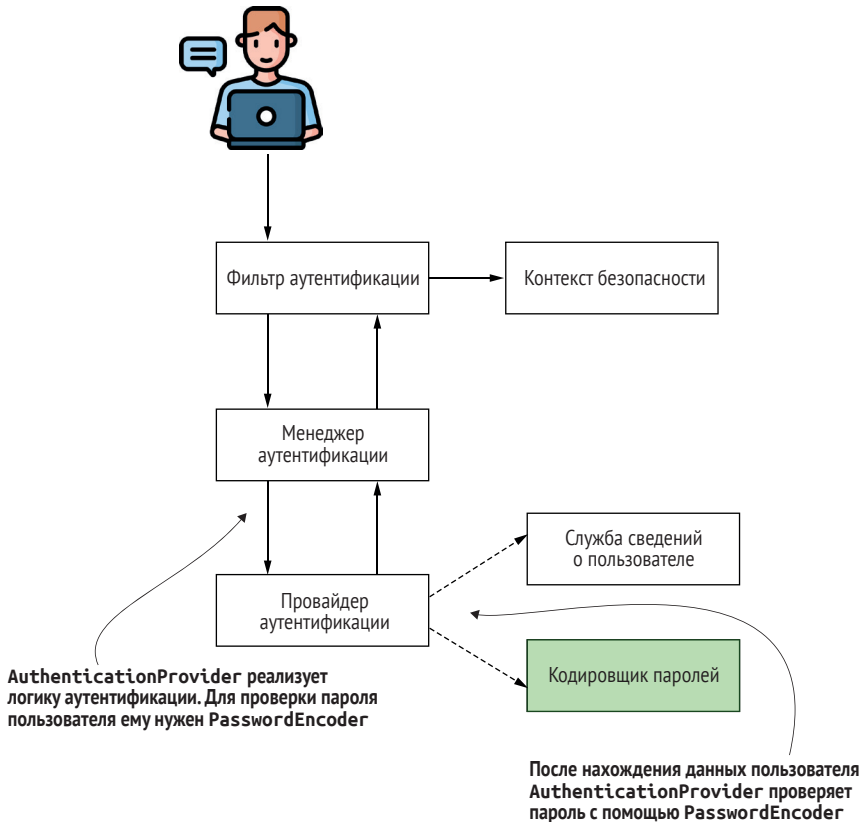


Рис. 4.1 Процесс аутентификации Spring Security. `AuthenticationProvider` использует `PasswordEncoder` для проверки пароля пользователя в процессе аутентификации

Поскольку система не должна оперировать паролями в виде простого текста, они обычно подвергаются своего рода преобразованию, что затрудняет их чтение и кражу. Для этой задачи Spring Security определяет отдельный контракт. Для наглядности в этой главе я привожу множество примеров кода, связанных с реализацией `PasswordEncoder`. Мы начнем с изучения контракта, а затем напишем свою реализацию в проекте. Затем в разделе 4.1.3 я дам вам список наиболее известных и широко используемых реализаций `PasswordEncoder`, предоставляемых Spring Security.

4.1.1 Контракт *PasswordEncoder*

В этом разделе мы обсудим определение контракта `PasswordEncoder`. Он нужен, чтобы сообщить Spring Security, как проверять пароль пользователя. В процессе аутентификации `PasswordEncoder` решает, является ли пароль действительным. Каждая система хранит пароли в закодированной форме, обычно в виде хешей, чтобы никто не смог их прочитать. `PasswordEncoder` тоже может кодировать пароли. Методы `encode()` и `matches()`, которые объявляются в контракте, на самом деле являются определением его ответственности. Оба они являются частями одного и того же контракта, поскольку они тесно взаимосвязаны. Способ, которым приложение кодирует пароль, связан со способом проверки пароля. Давайте взглянем на код интерфейса `PasswordEncoder`:

```
public interface PasswordEncoder {  
  
    String encode(CharSequence rawPassword);  
    boolean matches(CharSequence rawPassword, String encodedPassword);  
  
    default boolean upgradeEncoding(String encodedPassword) {  
        return false;  
    }  
}
```

Интерфейс определяет два абстрактных метода и один с реализацией по умолчанию. Об абстрактных методах `encode()` и `matches()` вы также чаще всего слышите при работе с реализацией `PasswordEncoder`.

Цель метода `encode(CharSequence rawPassword)` – вернуть результат преобразования заданной строки. С точки зрения функциональности Spring Security он используется для шифрования или хеширования заданного пароля. Впоследствии можно вызвать метод `matches(CharSequence rawPassword, String encodedPassword)`, чтобы проверить, соответствует ли закодированная строка исходному паролю. Вы будете использовать метод `matches()` в процессе аутентификации для проверки, соответствует ли предоставленный пароль одному из элементов набора известных учетных данных. Третий метод, называемый `upgradeEncoding(CharSequence encodedPassword)`, по умолчанию имеет в контракте значение `false`. Если вы переопределяете его для возврата `true`, то закодированный пароль повторно кодируется для лучшей безопасности.

В некоторых случаях кодирование закодированного пароля может усложнить получение открытого пароля из результата. В общем, это своего рода неопределенность, которая мне лично не нравится. Но фреймворк предоставляет такую возможность, если вы считаете, что это применимо к вашему случаю.

4.1.2 Реализация собственного класса *PasswordEncoder*

Как вы заметили, методы `matches()` и `encode()` сильно связаны между собой. После переопределения они должны сохранить взаимное соответствие с точки зрения функциональности: строка, возвращаемая методом `encode()`, всегда должна быть проверяемой с помощью метода `matches()` того же `PasswordEncoder`. В этом разделе вы реализуете контракт `PasswordEncoder` и определите два абстрактных метода, объявленных интерфейсом. Зная, как реализовать `PasswordEncoder`, вы можете выбрать, как приложение будет оперировать паролями для процесса аутентификации. Самая простая реализация – это кодировщик паролей, который возвращает пароли в виде обычного текста: то есть он не выполняет никакого кодирования пароля.

Операции с паролями в виде открытого текста – именно этим и занимается экземпляр `NoOpPasswordEncoder`. Мы использовали этот класс в нашем первом примере в главе 2. Если бы вы написали свой собственный класс, он выглядел бы примерно как в листинге 4.1.

Листинг 4.1 Простейшая реализация `PasswordEncoder`

```
public class PlainTextPasswordEncoder
    implements PasswordEncoder {

    @Override
    public String encode(CharSequence rawPassword) {
        return rawPassword.toString();
    }

    @Override
    public boolean matches(
        CharSequence rawPassword, String encodedPassword) {
        return rawPassword.equals(encodedPassword);
    }
}
```

❶ Мы не меняем пароль, а просто возвращаем его как есть.

❷ Проверяем, равны ли две строки.

Результат кодирования всегда совпадает с паролем. Поэтому для проверки их совпадения вам нужно только сравнить строки с помощью `equals()`. Простая реализация `PasswordEncoder`, использующая алгоритм хеширования SHA-512, выглядит, как показано в следующем листинге.

Листинг 4.2 Реализация `PasswordEncoder`, использующего SHA-512

```
public class Sha512PasswordEncoder
    implements PasswordEncoder {

    @Override
```

```

public String encode(CharSequence rawPassword) {
    return hashWithSHA512(rawPassword.toString());
}

@Override
public boolean matches(
    CharSequence rawPassword, String encodedPassword) {
    String hashedPassword = encode(rawPassword);
    return encodedPassword.equals(hashedPassword);
}

// Часть кода не показана
}

```

В листинге 4.2 мы используем метод для хеширования строкового значения, предоставленного с помощью SHA-512. Я опускаю реализацию этого метода в листинге 4.2, но вы можете найти его в листинге 4.3. Мы вызываем этот метод из метода `encode()`, который теперь возвращает значение хеша для входной строки. Чтобы проверить полученный хеш, метод `matches()` хеширует необработанный пароль и сравнивает его на предмет эквивалентности с хешем, проверку которого он выполняет.

Листинг 4.3 Реализация метода для хеширования ввода с помощью SHA-512

```

private String hashWithSHA512(String input) {
    StringBuilder result = new StringBuilder();
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-512");
        byte [] digested = md.digest(input.getBytes());
        for (int i = 0; i < digested.length; i++) {
            result.append(Integer.toHexString(0xFF & digested[i]));
        }
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("Bad algorithm");
    }
    return result.toString();
}

```

Позже вы узнаете о более удачных реализациях этой процедуры, поэтому пока не обращайте внимания на оптимальность кода в листингах 4.2 и 4.3.

4.1.3 Выбор готовых реализаций *PasswordEncoder*

Хотя очень полезно уметь реализовывать свой `PasswordEncoder`, вы также должны знать, что Spring Security предоставляет несколько готовых удобных реализаций. Если одна из них подходит для вашей

го приложения, вам не нужно ее переписывать. В этом разделе мы обсудим варианты реализации `PasswordEncoder`, которые предоставляет Spring Security:

- `NoOpPasswordEncoder` – не кодирует пароль, а сохраняет его в открытом виде. Мы используем эту реализацию только для примеров. Поскольку метод не хеширует пароль, вам никогда не следует использовать его в реальном сценарии;
- `StandardPasswordEncoder` – использует SHA-256 для хеширования пароля. Эта реализация устарела, и вам не следует использовать ее для новых проектов. Дело в том, что данный метод использует алгоритм хеширования, который больше не считается достаточно стойким, но вы можете встретить его в существующих приложениях. При обновлении существующих приложений желательно заменить этот метод на какой-то другой, более мощный кодировщик паролей;
- `Pbkdf2PasswordEncoder` – использует функцию деривации ключа на основе пароля 2 (PBKDF2);
- `BCryptPasswordEncoder` – использует функцию хеширования `bcrypt` для кодирования пароля;
- `SCryptPasswordEncoder` – использует функцию хеширования `scrypt` для кодирования пароля.

Более подробную информацию о хешировании и этих алгоритмах можно найти в главе 2 книги Дэвида Вонга «Реальная криптография» (Питер, 2024).

Давайте рассмотрим несколько примеров создания экземпляров различных реализаций `PasswordEncoder`. Итак, `NoOpPasswordEncoder` не кодирует пароль. Его реализация похожа на `PlainTextPasswordEncoder` из нашего примера в листинге 4.1. Поэтому мы используем данный кодировщик паролей только в учебных примерах. Кроме того, класс `NoOpPasswordEncoder` разработан как синглтон. Вы не можете вызвать его конструктор напрямую извне класса, но можете использовать метод `NoOpPasswordEncoder.getInstance()`, чтобы получить экземпляр класса следующим образом:

```
PasswordEncoder p = NoOpPasswordEncoder.getInstance();
```

Метод хеширования `StandardPasswordEncoder`, предоставляемый Spring Security, основан на алгоритме SHA-256. Вы можете передать в `StandardPasswordEncoder` так называемый *секрет*, используемый в процессе хеширования. Значение этого секрета является параметром конструктора. Если вы решите вызвать конструктор без аргументов, реализация будет использовать в качестве секрета пустую строку. Однако `StandardPasswordEncoder` уже устарел, и я не рекомендую использовать его в новых приложениях. Но вы можете встретить приложения, в которых он применяется, поэтому нужно уметь

с ним работать. Следующий фрагмент кода демонстрирует создание экземпляров этого кодировщика паролей:

```
PasswordEncoder p = new StandardPasswordEncoder();  
PasswordEncoder p = new StandardPasswordEncoder("secret");
```

Другой вариант, предлагаемый Spring Security, – это реализация `Pbkdf2PasswordEncoder`, основанная на алгоритме кодирования PBKDF2. Для создания экземпляров `Pbkdf2PasswordEncoder` применяется следующая опция:

```
PasswordEncoder p =  
    new Pbkdf2PasswordEncoder("secret", 16, 310000,  
Pbkdf2PasswordEncoder.  
SecretKeyFactoryAlgorithm.PBKDF2WithHmacSHA256);
```

PBKDF2 – довольно простая, медленно хеширующая функция, которая выполняет HMAC столько раз, сколько указано в аргументе итераций. Первые три параметра, полученные последним вызовом, – это значение ключа, используемого для процесса кодирования, количество итераций кодирования пароля и размер хеша. Второй и третий параметры могут влиять на надежность результата. Четвертый параметр задает ширину хеша. Вы можете выбрать следующие параметры:

- PBKDF2WithHmacSHA1;
- PBKDF2WithHmacSHA256;
- PBKDF2WithHmacSHA512.

Можно выбрать разное количество итераций, а также длину результата. Чем длиннее хеш, тем надежнее пароль (то же самое относится и к ширине хеша). Однако имейте в виду, что от этих значений зависит производительность: чем больше итераций, тем больше ресурсов потребляет ваше приложение. Вам следует найти разумный компромисс между ресурсами, потребляемыми для генерации хеша, и необходимой стойкостью кодировки.

ПРИМЕЧАНИЕ В этой книге я упоминаю несколько концепций криптографии, о которых вам, возможно, захочется узнать больше. В качестве источника актуальной информации о HMAC и других нюансах криптографии я рекомендую книгу Дэвида Вонга «Реальная криптография» (Питер, 2024). В главе 3 этой книги представлена подробная информация о HMAC.

Еще один отличный вариант, предлагаемый Spring Security, – это метод `BCryptPasswordEncoder`, который использует для кодирования пароля функцию стойкого хеширования `bcrypt`. Вы можете создать экземпляр `BCryptPasswordEncoder`, вызвав конструктор без аргументов. Однако у вас также есть возможность указать коэффициент

стойкости, определяющий логарифмические раунды (log-раунды), используемые в процессе кодирования. Более того, вы также можете изменить экземпляр `SecureRandom`, используемый для кодирования:

```

PasswordEncoder p = new BCryptPasswordEncoder();
PasswordEncoder p = new BCryptPasswordEncoder(4);

SecureRandom s = SecureRandom.getInstanceStrong();
PasswordEncoder p = new BCryptPasswordEncoder(4, s);

```

Указанное вами значение log-раундов влияет на количество итераций, применяемых операцией хеширования. Количество выполняемых итераций составляет $2^{\text{log-раунд}}$. Допустимое значение log-раундов лежит в диапазоне от 4 до 31. Вы можете указать его, вызвав один из перегруженных конструкторов, как показано в предыдущем фрагменте кода.

Последний вариант, который я вам представляю, – `SCryptPasswordEncoder` (рис. 4.2). Этот кодировщик паролей использует функцию хеширования `scrypt`. Вы можете создавать экземпляры `SCryptPasswordEncoder`, как показано на рис. 4.2.

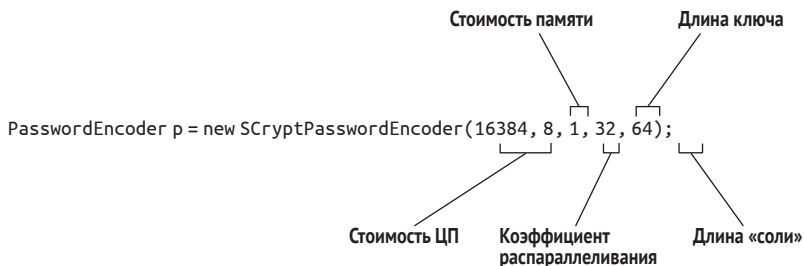


Рис. 4.2 Конструктор `SCryptPasswordEncoder` принимает пять параметров и позволяет вам настраивать нагрузку на ЦП, затраты памяти, длину ключа и длину соли¹

4.1.4 Несколько стратегий кодирования с помощью `DelegatingPasswordEncoder`

В этом разделе мы обсудим случаи, в которых поток аутентификации должен применять различные реализации для сопоставления паролей. Вы также узнаете, как применять полезный инструмент, который действует как `PasswordEncoder` в вашем приложении. Вместо того чтобы иметь собственную реализацию, этот инструмент делегирует полномочия другим объектам, реализующим интерфейс `PasswordEncoder`.

¹ Строка входных данных, которая передается хеш-функции вместе с входным массивом данных (прообразом) для вычисления хеша (образа).

В некоторых приложениях полезно иметь различные кодировщики паролей и выбирать нужный в зависимости от определенной конфигурации. Обычный сценарий, в котором я встречаю `DelegatingPasswordEncoder` в производственных приложениях, – это смена алгоритма кодирования начиная с определенной версии приложения. Представьте, что кто-то находит уязвимость в текущем алгоритме и вы хотите изменить его для вновь зарегистрированных пользователей, но при этом оставить без изменений для существующих учетных записей. В итоге у вас будет несколько видов хешей. Как справиться с этой ситуацией? Хотя это не единственное решение, хорошим выбором будет использование объекта `DelegatingPasswordEncoder`.

`DelegatingPasswordEncoder` – это реализация интерфейса `PasswordEncoder`, которая вместо своего алгоритма кодирования делегирует другому экземпляру реализации того же контракта. Хеш начинается с префикса, именующего алгоритм, используемый для определения этого хеша. `DelegatingPasswordEncoder` выполняет делегирование правильной реализации `PasswordEncoder` на основе префикса пароля. Звучит сложно, но на примере можно убедиться, что это проще, чем кажется. На рис. 4.3 представлена связь между экземплярами `PasswordEncoder`. `DelegatingPasswordEncoder` имеет список реализаций `PasswordEncoder`, которым он делегирует полномочия. `DelegatingPasswordEncoder` хранит каждый из экземпляров в карте сопоставлений. `NoOpPasswordEncoder` назначается ключу `noop`, в то время

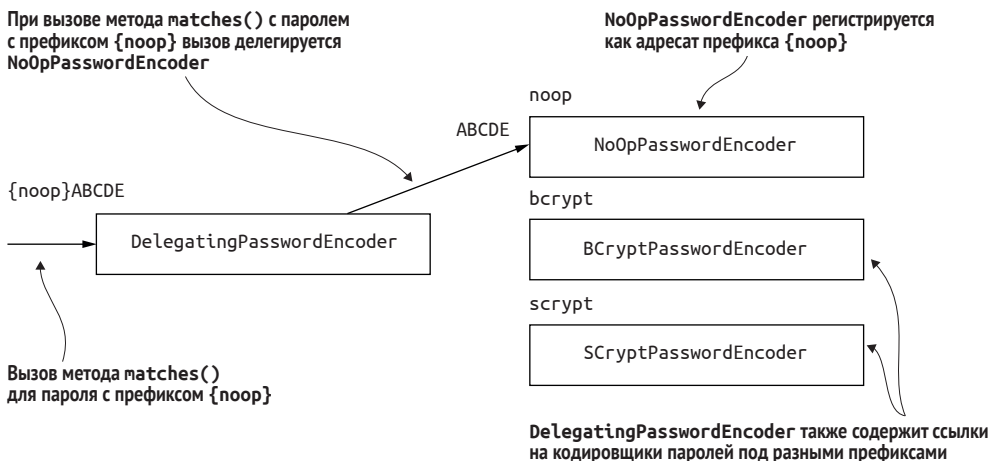


Рис. 4.3 В этом сценарии `DelegatingPasswordEncoder` привлекает `NoOpPasswordEncoder` для обработки паролей с префиксом `{noop}`, `BCryptPasswordEncoder` для паролей, начинающихся с `{bcrypt}`, и `SCryptPasswordEncoder` для паролей, начинающихся с `{scrypt}`. Когда пароль сопровождается префиксом `{noop}`, `DelegatingPasswordEncoder` направляет задачу версии `NoOpPasswordEncoder`

как реализация `BCryptPasswordEncoder` назначается ключу `bcrypt`. Когда пароль имеет префикс `{noop}`, `DelegatingPasswordEncoder` делегирует операцию реализации `NoOpPasswordEncoder`. Если префикс равен `{bcrypt}`, то действие делегируется реализации `BCryptPasswordEncoder`, как показано на рис. 4.4.

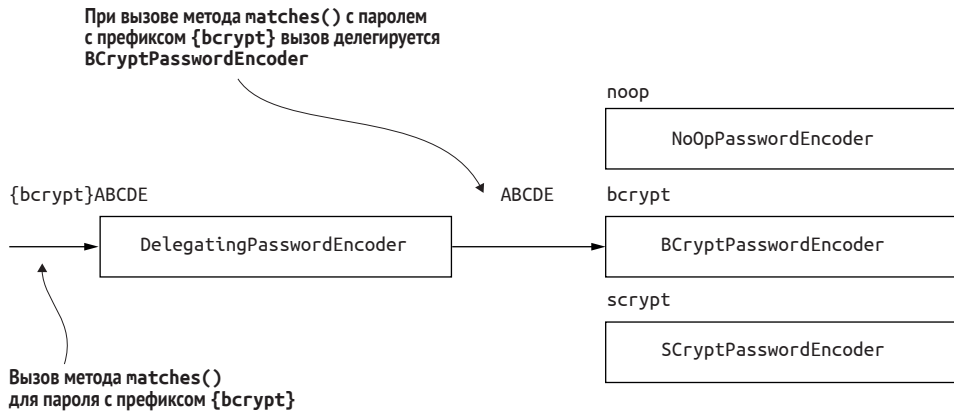


Рис. 4.4 Здесь `DelegatingPasswordEncoder` назначает задачу обработки паролей с префиксом `{noop}` классу `NoOpPasswordEncoder`, паролей с префиксом `{bcrypt}` – `BCryptPasswordEncoder` и паролей с префиксом `{scrypt}` – `SCryptPasswordEncoder`. Если пароль содержит префикс `{bcrypt}`, `DelegatingPasswordEncoder` направляет процесс в механизм `BCryptPasswordEncoder`

Давайте посмотрим, как определяют `DelegatingPasswordEncoder`. Мы начинаем с создания коллекции экземпляров желаемых реализаций `PasswordEncoder` и объединяем их в `DelegatingPasswordEncoder`, как в листинге 4.4.

Листинг 4.4 Создание экземпляра `DelegatingPasswordEncoder`

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public PasswordEncoder passwordEncoder() {
        Map<String, PasswordEncoder> encoders = new HashMap<>();

        encoders.put("noop", NoOpPasswordEncoder.getInstance());
        encoders.put("bcrypt", new BCryptPasswordEncoder());
        encoders.put("scrypt", new SCryptPasswordEncoder());

        return new DelegatingPasswordEncoder("bcrypt", encoders);
    }
}
```

`DelegatingPasswordEncoder` – это просто инструмент, который действует как `PasswordEncoder`, поэтому вы можете использовать его, когда нужно выбрать вариант из коллекции реализаций. В листинге 4.4 объявленный экземпляр `DelegatingPasswordEncoder` содержит ссылки на `NoOpPasswordEncoder`, `BCryptPasswordEncoder` и `SCryptPasswordEncoder` и делегирует реализацию по умолчанию `BCryptPasswordEncoder`. На основе префикса хеша `DelegatingPasswordEncoder` выбирает правильную реализацию `PasswordEncoder` для сопоставления пароля. Этот префикс имеет ключ, который идентифицирует необходимый кодировщик пароля. Если префикса нет, `DelegatingPasswordEncoder` использует кодировщик по умолчанию. Вариантом `PasswordEncoder` по умолчанию будет тот, который указан в качестве первого параметра при построении экземпляра `DelegatingPasswordEncoder`. Для кода в листинге 4.4 `PasswordEncoder` по умолчанию – `bcrypt`.

ПРИМЕЧАНИЕ Фигурные скобки являются частью префикса хеша, и они должны окружать имя ключа. Например, если заданный хеш – `{noop}12345`, то `DelegatingPasswordEncoder` делегирует `NoOpPasswordEncoder`, который мы зарегистрировали для префикса `noop`. Опять же, помните, что фигурные скобки в префиксе обязательны.

Если хеш выглядит как в следующем фрагменте кода, то кодировщиком пароля является тот, который мы назначаем префиксу `{bcrypt}`, то есть `BCryptPasswordEncoder`. И ему же приложение будет делегировать обработку пароля, если префикса вообще нет, потому что мы определили его как реализацию по умолчанию:

```
{bcrypt}$2a$10$xn3LI/AjqicFYZFruSwve.681477XaVNaUQbr1gioaWPn4t1KsnmG
```

Для удобства Spring Security предлагает способ создания `DelegatingPasswordEncoder`, который имеет карту сопоставления для всех стандартных реализаций `PasswordEncoder`. Класс `PasswordEncoderFactories` предоставляет статический метод `createDelegatingPasswordEncoder()`, который возвращает реализацию `DelegatingPasswordEncoder` с полным набором отображений `PasswordEncoder` и `bcrypt` в качестве кодировщика по умолчанию:

```
PasswordEncoder passwordEncoder =  
PasswordEncoderFactories.createDelegatingPasswordEncoder();
```

Кодирование, шифрование или хеширование?

В предыдущих разделах я часто использовал термины «кодирование», «шифрование» и «хеширование». Я хочу кратко прояснить эти термины и то, как они используются в книге.

Кодирование означает любое преобразование заданных входных данных. Например, если у нас есть функция x , которая переворачивает строку, функция $x \rightarrow y$, примененная к строке ABCD, дает DCBA.

Шифрование – это особый тип кодирования, в котором для получения выходных данных мы предоставляем как входное значение, так и ключ. Обладатель ключа имеет возможность впоследствии обратить функцию (получить входные данные из выходных данных). Простейшая форма представления шифрования в виде функции:

$(x, k) \rightarrow y$

где x – входные данные, k – ключ, а y – результат шифрования. Таким образом, человек, знающий ключ, может использовать известную функцию для получения входных данных из выходных: $(y, k) \rightarrow x$. Мы называем это обращение *функцией расшифровки*. Если ключ, используемый для шифрования, совпадает с ключом, используемым для расшифровки, мы обычно называем его *симметричным*.

Если применяются два разных ключа для шифрования $((x, k_1) \rightarrow y)$ и расшифровки $((y, k_2) \rightarrow x)$, то мы говорим, что шифрование выполняется с помощью *асимметричных* ключей. Тогда (k_1, k_2) называется парой ключей. Ключ k_1 , используемый для шифрования, также называется *открытым* (public) ключом, а ключ для расшифровки k_2 называется *закрытым* (private). Таким образом, только владелец закрытого ключа может расшифровать данные.

Хеширование – это особый тип кодирования, за исключением того, что функция является строго односторонней. То есть из выходных данных у функции хеширования вы не можете получить входные данные x . Однако всегда должен быть способ проверить, соответствует ли выходное значение y входным данным x , чтобы мы могли использовать хеширование как пару функций для кодирования и сопоставления. Если хеширование – это $x \rightarrow y$, то у нас также должна быть функция сопоставления $(x, y) \rightarrow \text{boolean}$.

Иногда функция хеширования может также использовать случайное значение, добавленное к входным данным: $(x, k) \rightarrow y$. Это значение называют *солью*. Соль делает функцию сильнее, усиливая сложность применения обратной функции для получения входных данных из результата.

Таблица 4.1 кратко описывает каждый из компонентов, которые мы обсудили и применили в книге к этому моменту.

Таблица 4.1. Интерфейсы, представляющие основные контракты для потока аутентификации в Spring Security

Контракт	Описание
UserDetails	Представляет пользователя в форме, понятной Spring Security
GrantedAuthority	Определяет действие в рамках назначения приложения, которое разрешено пользователю (например, чтение, запись, удаление и т. д.)
UserDetailsService	Представляет объект, используемый для получения данных пользователя по его имени
UserDetailsManager	Более конкретный контракт для UserDetailsService. Помимо извлечения пользователя по имени пользователя, его также можно использовать для мутации коллекции пользователей или конкретного пользователя
PasswordEncoder	Указывает, как шифруется или хешируется пароль, а также как проверить, соответствует ли заданная закодированная строка открытому паролю

4.2 Использование модуля Spring Security Crypto

В этом разделе мы обсудим модуль Spring Security Crypto (SSCM), который является частью Spring Security, ответственной за криптографию. Функции шифрования и дешифрования, а также генерации ключей не поставляются «из коробки» вместе с языком Java, что ограничивает разработчиков при добавлении зависимостей, которые обеспечивают более доступный подход к этим функциям.

Чтобы облегчить нам жизнь, Spring Security предоставляет собственное решение, которое сокращает зависимости наших проектов, устраняя необходимость использования отдельной библиотеки. Кодировщики паролей также являются частью SSCM, хотя мы рассматривали их отдельно в предыдущих разделах. В этом разделе мы обсудим, какие еще опции, связанные с криптографией, предлагает SSCM. Вы увидите примеры применения двух основных функций из SSCM:

- генераторы ключей – объекты, используемые для генерации ключей для алгоритмов хеширования и шифрования;
- шифровальщики – объекты, используемые для шифрования и дешифрования данных.

4.2.1 Генераторы ключей

В этом разделе мы обсудим генераторы ключей. *Генератор ключей* – это объект, используемый для генерации определенного вида ключа, обычно необходимого для алгоритма шифрования или хеширования. Реализации генераторов ключей, предлагаемые Spring Security, являются отличными рабочими инструментами. Намного удобнее использовать эти реализации, а не добавлять еще одну зависимость в свое приложение, и именно поэтому я рекомендую вам ознакомиться с ними. Рассмотрим несколько примеров кода, в которых создаются и применяются генераторы ключей.

Два интерфейса представляют два основных типа генераторов ключей: `BytesKeyGenerator` и `StringKeyGenerator`. Мы можем построить их напрямую, используя класс-фабрику `KeyGenerators`. Существует генератор строковых ключей, представленный контрактом `StringKeyGenerator`, для получения ключа в виде строки. Обычно этот ключ используют как значение соли для алгоритма хеширования или шифрования. Определение контракта `StringKeyGenerator` можно найти в следующем фрагменте кода:

```
public interface StringKeyGenerator {  
  
    String generateKey();  
  
}
```

Генератор имеет метод `generateKey()`, который возвращает строку, представляющую значение ключа. Следующий фрагмент кода демонстрирует пример получения экземпляра `StringKeyGenerator` и его использования для получения значения соли:

```
StringKeyGenerator keyGenerator = KeyGenerators.string();  
String salt = keyGenerator.generateKey();
```

Генератор создает 8-байтовый ключ и кодирует его как шестнадцатеричную строку. Метод возвращает результат этих операций как строку. Второй интерфейс, описывающий генератор ключей, – это `BytesKeyGenerator`, который определяется следующим образом:

```
public interface BytesKeyGenerator {  
  
    int getKeyLength();  
    byte[] generateKey();  
  
}
```

В дополнение к методу `generateKey()`, который возвращает ключ как `byte[]`, интерфейс определяет другой метод, который возвращает длину ключа в байтах. `BytesKeyGenerator` по умолчанию генерирует ключи длиной 8 байт:

```
BytesKeyGenerator keyGenerator = KeyGenerators.secureRandom();  
byte [] key = keyGenerator.generateKey();  
int keyLength = keyGenerator.getKeyLength();
```

В предыдущем фрагменте кода генератор ключей генерирует ключи длиной 8 байт. Если вы хотите указать другую длину ключа, сделайте это при получении экземпляра генератора ключей, указав желаемое значение в методе `KeyGenerators.secureRandom()`:

```
BytesKeyGenerator keyGenerator = KeyGenerators.secureRandom(16);
```

Ключи, сгенерированные `BytesKeyGenerator`, созданным с помощью метода `KeyGenerators.secureRandom()`, уникальны для каждого вызова метода `generateKey()`. В некоторых случаях предпочтительна реализация, которая возвращает одно и то же значение ключа для каждого вызова одного и того же генератора ключей. В этом случае мы можем создать `BytesKeyGenerator` с помощью метода `KeyGenerators.shared(int length)`. В этом фрагменте кода `key1` и `key2` имеют одинаковое значение:

```
BytesKeyGenerator keyGenerator = KeyGenerators.shared(16);  
byte [] key1 = keyGenerator.generateKey();  
byte [] key2 = keyGenerator.generateKey();
```

4.2.2 Шифрование и дешифровка секретов с помощью шифраторов

В данном разделе мы применим в примерах кода реализации шифраторов, которые предлагает Spring Security. *Шифратор* (encryptor) – это объект, который реализует алгоритм шифрования. Когда речь идет о безопасности, шифрование и дешифровка встречаются на каждом шагу, поэтому не сомневайтесь, что они понадобятся в вашем приложении.

Нам часто приходится шифровать данные либо при их передаче между компонентами системы, либо при их сохранении. Операции, выполняемые шифратором, – это шифрование и дешифровка. Существует два типа шифраторов, определенных SSCM: `BytesEncryptor` и `TextEncryptor`. Хотя у них схожие обязанности, они обрабатывают разные типы данных. `TextEncryptor` оперирует данными как строкой. Его методы получают строковые входные данные и возвращают строковые выходные данные, как следует из определения его интерфейса:

```
public interface TextEncryptor {  
  
    String encrypt(String text);  
    String decrypt(String encryptedText);  
  
}
```

`BytesEncryptor` более универсален. Он получает входные данные в виде массива байтов:

```
public interface BytesEncryptor {  
  
    byte[] encrypt(byte[] byteArray);  
    byte[] decrypt(byte[] encryptedByteArray);  
  
}
```

Давайте посмотрим, какие у нас есть варианты для создания и использования шифратора. Фабричный класс `Encryptors` предлагает несколько вариантов. Для `BytesEncryptor` мы могли бы использовать методы `Encryptors.standard()` или `Encryptors.stronger()`, например:

```
String salt = KeyGenerators.string().generateKey();
String password = "secret";
String valueToEncrypt = "HELLO";

BytesEncryptor e = Encryptors.standard(password, salt);
byte [] encrypted = e.encrypt(valueToEncrypt.getBytes());
byte [] decrypted = e.decrypt(encrypted);
```

За кулисами стандартный шифратор байтов использует 256-байтовый алгоритм AES для шифрования входных данных. Для создания более сильного экземпляра шифратора байтов нужно вызвать метод `Encryptors.stronger()`:

```
BytesEncryptor e = Encryptors.stronger(password, salt);
```

Разница невелика и скрыта за кадром, где 256-битное шифрование AES применяет *режим счетчика с аутентификацией Галуа* (Galois/Counter Mode, GCM). Стандартный режим использует цепочку блоков шифрования (CBC), которая считается более слабым методом.

`TextEncryptors` бывают трех основных типов. Их создают, вызывая `Encryptors.text()` или `Encryptors.delux()`. Помимо этих методов для создания шифраторов, есть также метод, который возвращает фиктивный `TextEncryptor`, который не шифрует значение. Вы можете использовать его для демонстрационных примеров или случаев, когда хотите протестировать производительность своего приложения, не тратя времени на шифрование. Метод, который возвращает этот пустой шифратор, – `Encryptors.noOpText()`. В следующем фрагменте кода показан пример использования `Text Encryptor`. Несмотря на вызов шифратора, в этом примере значения `encrypted` и `valueToEncrypt` одинаковы:

```
String valueToEncrypt = "HELLO";
TextEncryptor e = Encryptors.noOpText();
String encrypted = e.encrypt(valueToEncrypt);
```

Шифратор `Encryptors.text()` использует метод `Encryptors.standard()` для управления операциями шифрования, в то время как метод `Encryptors.delux()` использует экземпляр `Encryptors.stronger()` следующим образом:

```
String salt = KeyGenerators.string().generateKey();
String password = "secret";
String valueToEncrypt = "HELLO";
```



```
TextEncryptor e = Encryptors.text(password, salt);    ❶  
String encrypted = e.encrypt(valueToEncrypt);  
String decrypted = e.decrypt(encrypted);
```

❶ Создаем объект `TextEncryptor`, который использует соль и пароль.

Заключение

- `PasswordEncoder` выполняет одну из самых важных функций в логике аутентификации – работу с паролями.
- `Spring Security` предлагает несколько различных алгоритмов хеширования, что делает реализацию только вопросом выбора.
- Модуль `Spring Security Crypto (SSCM)` предлагает различные альтернативы для реализаций генераторов ключей и шифраторов.
- Генераторы ключей – это служебные объекты, которые помогают генерировать ключи, используемые криптографическими алгоритмами.
- Шифраторы – это служебные объекты, которые помогают применять шифрование и дешифровку данных.

Безопасность веб-приложения начинается с фильтров

Темы этой главы:

- работа с цепочкой фильтров;
- определение пользовательских фильтров;
- использование классов Spring Security, реализующих интерфейс Filter.

В Spring Security фильтры HTTP делегируют различные обязанности HTTP-запросу. Кроме того, они обычно управляют каждой ответственностью, которая возникает в связи с запросом. Таким образом, фильтры образуют цепочку ответственностей. Фильтр получает запрос, выполняет его логику и в конечном итоге делегирует запрос следующему фильтру в цепочке (рис. 5.1).

Давайте рассмотрим аналогию из обычной жизни. Когда вы приезжаете в аэропорт, то по пути от входа в терминал до посадки в самолет вы проходите через несколько фильтров (рис. 5.2). Сначала вы предъявляете свой билет, затем проверяют ваш паспорт, а дальше вы проходите проверку безопасности. На выходе из аэропорта могут быть применены дополнительные фильтры. Например, в некоторых случаях, прямо перед посадкой, ваш паспорт и визу проверяют снова. Это отличная аналогия с цепочкой фильтров в Spring Security. Точно так же вы настраиваете фильтры в цепочке фильтров. Spring

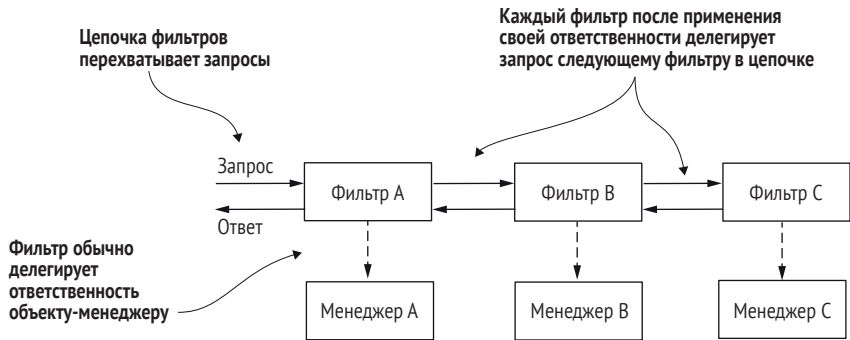


Рис. 5.1 Запрос передается в цепочку фильтров. Каждый фильтр привлекает менеджера для выполнения определенной логики по запросу, а затем передает ее следующему фильтру в цепочке



Рис. 5.2 В аэропорту перед посадкой в самолет вы проходите через ряд проверок. Аналогично Spring Security реализует последовательность фильтров, которые обрабатывают HTTP-запросы, получаемые приложением

Security предоставляет реализации фильтров, которые вы добавляете в цепочку посредством настройки, но вы также можете определить пользовательские фильтры.

В этой главе обсуждается применение Spring Security для настройки фильтров, которые являются частью архитектуры аутентификации и авторизации в веб-приложении. Например, вы можете расширить аутентификацию, добавив еще один шаг для пользователя, например проверку адреса электронной почты или использование одноразового пароля. Вы также можете добавить функциональность, относящуюся к аудиту событий аутентификации. Приложения используют аудит аутентификации в различных сценариях, от отладки до идентификации поведения пользователя. Современные технологии и алгоритмы машинного обучения позволяют изучить поведение пользователя и установить, что его учетная запись взломана или кто-то выдает себя за другого.

Умение настраивать цепочку ответственностей HTTP-фильтра является ценным навыком. На практике приложения поставляются с различными требованиями, где конфигурации по умолчанию больше не работают. Вам нужно будет добавить или заменить существующие компоненты цепочки. При реализации по умолчанию применяется метод аутентификации HTTP Basic, который ограничивается именем пользователя и паролем. Однако на практике встречаются более сложные сценарии. Возможно, вам нужно будет реализовать другую стратегию аутентификации, уведомить внешнюю систему о событии авторизации или зарегистрировать факт успешной или неудачной аутентификации, который позже будет использоваться при трассировке и аудите (рис. 5.3). Каким бы ни был ваш сценарий, Spring Security обеспечивает гибкое построение нужной вам цепочки фильтров.

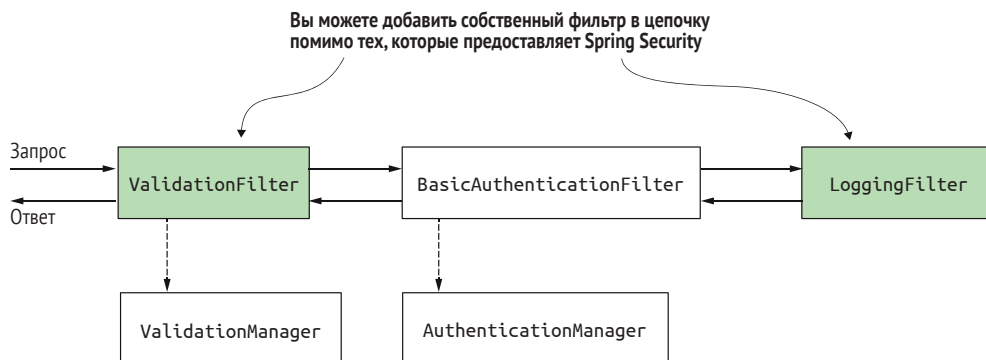


Рис. 5.3 У вас есть возможность персонализировать цепочку фильтров, вставляя новые фильтры до, после или на место текущих. Сделав это, вы сможете настроить не только процесс аутентификации, но и полную обработку запросов и ответов

5.1 Реализация фильтров в архитектуре Spring Security

В этом разделе говорится о том, как фильтры и цепочки фильтров работают в архитектуре Spring Security. Чтобы понять примеры реализации, над которыми мы будем работать в следующих разделах, сначала нужно рассмотреть этот вопрос в общих чертах. В главах 2 и 3 мы узнали, что фильтр аутентификации перехватывает запрос и делегирует ответственность за аутентификацию менеджеру авторизации. Если мы хотим добавить определенную логику, выполняемую перед аутентификацией, мы вставляем дополнительный фильтр перед фильтром аутентификации.

Фильтры в архитектуре Spring Security – это типичные HTTP-фильтры. Мы создаем их, реализуя интерфейс `Filter` из пакета `jakarta.servlet`. Как и для любого другого HTTP-фильтра, вам необходимо переопределить метод `doFilter()` для реализации его логики. Этот метод получает параметры `ServletRequest`, `ServletResponse` и `Filter Chain`:

- `ServletRequest` – представляет HTTP-запрос. Мы используем объект `ServletRequest` для получения сведений о запросе;
- `ServletResponse` – представляет HTTP-ответ. Мы используем объект `ServletResponse` для изменения ответа перед его отправкой обратно клиенту или далее по цепочке фильтров;
- `FilterChain` – представляет цепочку фильтров. Мы используем объект `FilterChain` для пересылки запроса следующему фильтру в цепочке.

ПРИМЕЧАНИЕ Начиная с Spring Boot 3 Jakarta EE заменяет старую спецификацию Java EE. Из-за этой замены у некоторых пакетов префикс `javax` изменился на `jakarta`. Например, такие типы, как `Filter`, `ServletRequest` и `ServletResponse`, ранее находились в пакете `javax.servlet`, но теперь вы найдете их в пакете `jakarta.servlet`.

Цепочка фильтров представляет собой набор фильтров, действующих в определенном порядке. Spring Security предлагает нам несколько готовых реализаций фильтров и их порядок. Ниже перечислены готовые фильтры:

- `BasicAuthenticationFilter` обслуживает аутентификацию HTTP Basic, если она присутствует;
- `CsrfFilter` заботится о защите от подделки межсайтовых запросов (CSRF), которую мы обсудим в главе 9;
- `CorsFilter` отвечает за правила авторизации совместного использования ресурсов между источниками (CORS), которые мы обсудим в главе 10.

Вам не нужно детально изучать все фильтры, так как вы, вероятно, не будете работать с ними напрямую из своего кода, но полезно понимать, как устроена цепочка фильтров, и знать несколько реализаций. В этой книге я объясняю только фильтры, которые необходимы для дальнейшего обсуждения.

Имейте в виду, что приложение не обязательно имеет экземпляры всех этих фильтров в цепочке. Цепочка длиннее или короче в зависимости от того, как построено приложение. Например, в главах 2 и 3 вы узнали, что вам нужно вызвать метод `httpBasic()` класса `HttpSecurity`, если вы хотите использовать метод аутентификации HTTP Basic. Если в приложении присутствует вызов метода `httpBasic()`, в цепочку добавляется экземпляр `BasicAuthenticationFilter`. Аналогично в зависимости от заданной вами конфигурации будет построена оставшаяся цепочка фильтров.

Вы добавляете новый фильтр в цепочку относительно другого (рис. 5.4). Вы можете добавить фильтр до, после или в позицию определенного фильтра. Каждая позиция, по сути, является индексом (числом), и поэтому иногда ее называют *порядком*.

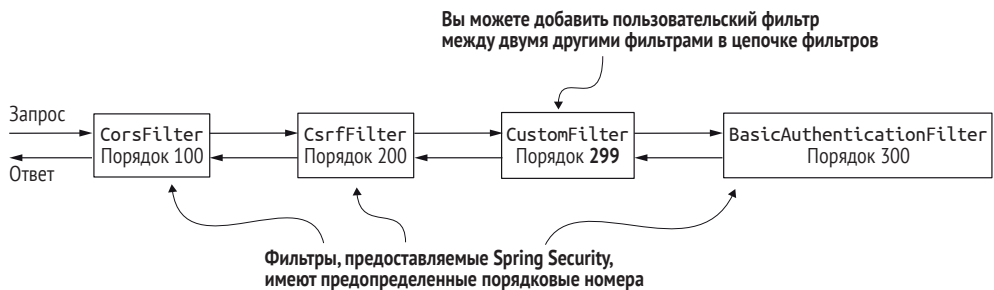


Рис. 5.4 Каждый фильтр имеет порядковый номер, который определяет порядок применения фильтров к запросу. Вы можете применять пользовательские фильтры вместе с фильтрами, предоставляемыми Spring Security

Если вы хотите узнать больше о фильтрах, предоставляемых Spring Security, и порядке их настройки, ознакомьтесь с описанием `SecurityWebFiltersOrder` по адресу <http://mng.bz/yZEG>.

Вы можете добавить два или более фильтров в одну и ту же позицию (рис. 5.5). В разделе 5.4 мы столкнемся с распространенным случаем, когда это происходит (и обычно создает путаницу среди разработчиков).

ПРИМЕЧАНИЕ Если несколько фильтров имеют одинаковую позицию, порядок, в котором они вызываются, не определен.

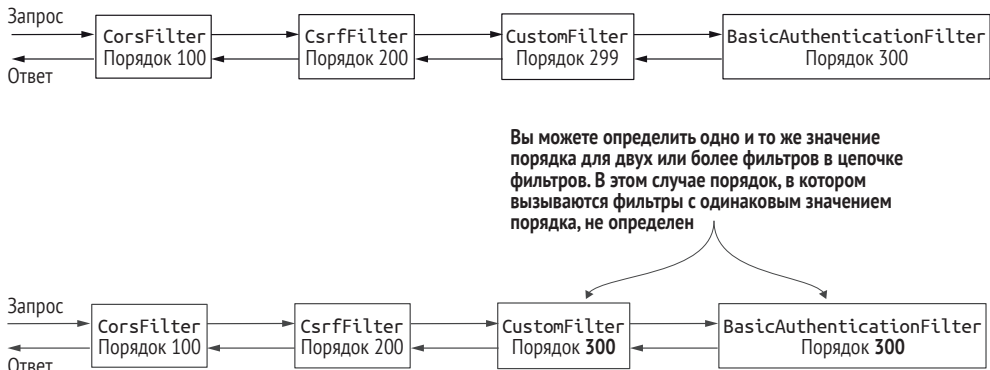


Рис. 5.5 У вас может быть несколько фильтров с одинаковым значением порядкового номера в цепочке. В этом случае Spring Security не гарантирует порядок, в котором они вызываются

5.2 Добавление фильтра перед существующим фильтром в цепочке

В этом разделе обсуждается применение пользовательского HTTP-фильтра перед существующим. Чтобы подойти к этой задаче с практической стороны, мы поработаем над учебным проектом, и вы научитесь легко реализовывать пользовательский фильтр и применять его в цепочке перед существующим фильтром. Впоследствии вы можете адаптировать этот пример к любой похожей задаче, с которой столкнетесь на практике.

В качестве первого примера реализации пользовательского фильтра рассмотрим тривиальный сценарий. Допустим, нам нужно гарантировать, что каждый запрос имеет заголовок с именем Request-Id (см. проект ssia-ch5-ex1). Мы предполагаем, что наше приложение использует этот заголовок для отслеживания запросов и что этот заголовок является обязательным. Мы должны проверить соблюдение этих требований, прежде чем приложение выполнит аутентификацию. Процесс аутентификации может содержать запрос к базе данных или другие ресурсоемкие действия, и нам не хотелось бы выполнять их напрасно, если формат запроса недопустим. Как этого избежать? Нам потребуются всего два шага, и цепочка фильтров будет выглядеть так, как показано на рис. 5.6.

- 1 *Реализуйте фильтр.* Создайте класс `RequestValidationFilter`, который проверяет наличие необходимого заголовка в запросе.
- 2 *Добавьте фильтр в цепочку фильтров.* Сделайте это в классе конфигурации, используя компонент `SecurityFilterChain`.

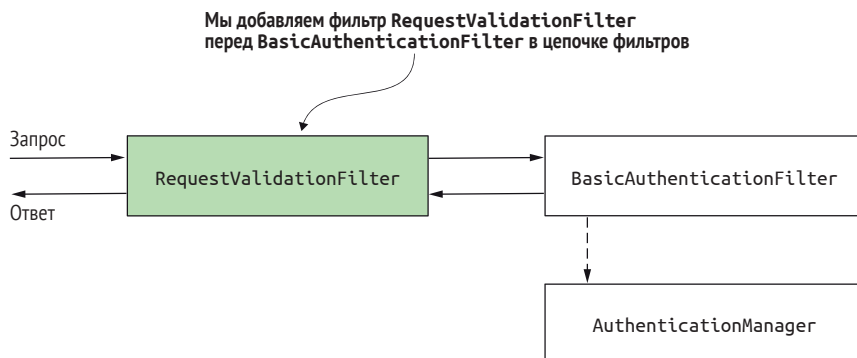


Рис. 5.6 В нашем примере мы добавляем RequestValidationFilter, который срабатывает до фильтра аутентификации. RequestValidationFilter гарантирует, что аутентификация не произойдет, если проверка запроса не пройдена. В нашем случае запрос должен иметь обязательный заголовок с именем Request-Id

Для выполнения шага 1 – реализации фильтра – мы определяем пользовательский фильтр. В листинге 5.1 показана его реализация.

Листинг 5.1 Реализация пользовательского фильтра

```
public class RequestValidationFilter
    implements Filter {
    ❶

    @Override
    public void doFilter(
        ServletRequest servletRequest,
        ServletResponse servletResponse,
        FilterChain filterChain)
        throws IOException, ServletException {
        // ...
    }
}
```

❶ Чтобы определить фильтр, этот класс реализует интерфейс Filter и переопределяет метод doFilter().

Внутри метода doFilter() мы пишем логику фильтра. В нашем примере мы проверяем, существует ли заголовок Request-Id. Если это так, мы пересылаем запрос следующему фильтру в цепочке, вызывая метод doFilter(). Если заголовок не существует, мы возвращаем в ответ HTTP-статус 400 Bad Request, не переходя к следующему фильтру в цепочке (рис. 5.7). В листинге 5.2 представлена логика фильтра.

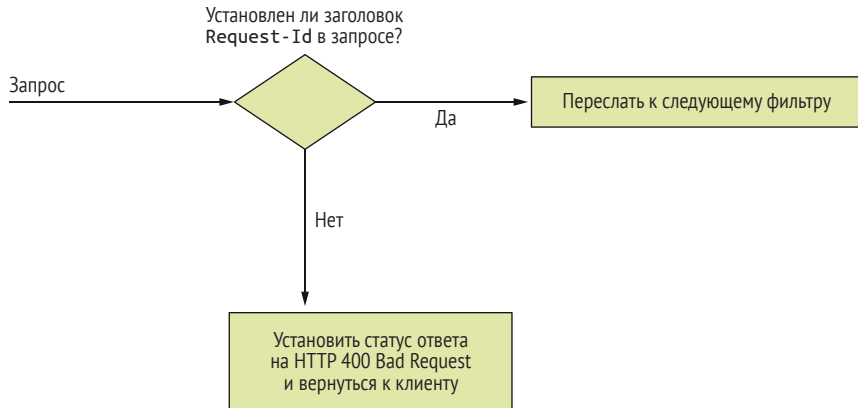


Рис. 5.7 Пользовательский фильтр, который мы добавляем перед аутентификацией, проверяет, существует ли заголовок Request-Id. Если заголовок присутствует в запросе, приложение пересылает запрос на аутентификацию дальше по цепочке. Если заголовок отсутствует, приложение возвращает HTTP-статус 400 Bad Request

Листинг 5.2 Реализация логики в методе doFilter()

```

@Override
public void doFilter(
    ServletRequest request,
    ServletResponse response,
    FilterChain filterChain)
    throws IOException,
        ServletException {

    var httpRequest = (HttpServletRequest) request;
    var httpResponse = (HttpServletResponse) response;

    String requestId = httpRequest.getHeader("Request-Id");

    if (requestId == null || requestId.isBlank()) {
        httpResponse.setStatus(HttpServletResponse.SC_BAD_REQUEST);
        return;
    }

    filterChain.doFilter(request, response);
  }

```

- ❶ Если заголовок отсутствует, HTTP-статус меняется на 400 Bad Request, и запрос не пересылается дальше.
- ❷ Если заголовок существует, запрос пересылается на следующий фильтр в цепочке.

Для реализации шага 2 мы используем метод `addFilterBefore()` объекта `HttpSecurity`, поскольку нам нужно, чтобы приложение выполнило данный пользовательский фильтр перед аутентификацией.

Этот метод получает два параметра:

- *экземпляр пользовательского фильтра, который мы хотим добавить в цепочку.* В нашем примере это экземпляр класса `RequestValidationFilter`, представленный в листинге 5.1;
- *тип фильтра, перед которым мы добавляем новый экземпляр.* В этом примере, поскольку требуется выполнить логику фильтра до аутентификации, нам нужно добавить экземпляр пользовательского фильтра перед фильтром аутентификации. Класс `BasicAuthenticationFilter` определяет тип фильтра аутентификации по умолчанию.

До сих пор мы называли фильтр, работающий в целом с аутентификацией, фильтром *аутентификации*. В следующих главах вы узнаете, что Spring Security также настраивает другие фильтры. В главе 9 мы обсудим защиту от подделки межсайтовых запросов (CSRF), а в главе 10 – обмен ресурсами между источниками (CORS). Эта функциональность также основана на фильтрах.

В листинге 5.3 показано добавление настраиваемого фильтра перед фильтром аутентификации в классе конфигурации. Чтобы упростить пример, мы используем метод `permitAll()`, разрешающий все неаутентифицированные запросы.

Листинг 5.3 Настройка пользовательского фильтра перед аутентификацией

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http.addFilterBefore(
            new RequestValidationFilter(), BasicAuthenticationFilter.class
        ).authorizeRequests(c -> c.anyRequest().permitAll());

        return http.build();
    }
}
```

- ❶ Добавляем в цепочку экземпляр пользовательского фильтра перед фильтром аутентификации.

Нам также нужен класс контроллера и конечная точка для проверки функциональности. Следующий листинг определяет класс контроллера.

Листинг 5.4 Класс контроллера

```
@RestController
public class HelloController {
```

```
@GetMapping("/hello")
public String hello() {
    return "Hello!";
}
}
```

Теперь вы можете запустить и протестировать приложение. Вызов конечной точки без заголовка генерирует ответ со статусом HTTP 400 Bad Request. Если вы добавите заголовок к запросу, статус ответа изменится на HTTP 200 OK, и вы также увидите тело ответа Hello!. Чтобы вызвать конечную точку без заголовка Request-Id, воспользуйтесь следующей командой cURL:

```
curl -v http://localhost:8080/hello
```

Этот вызов генерирует следующий ответ (здесь сокращен):

```
...
< HTTP/1.1 400
...
```

Чтобы вызвать конечную точку с заголовком Request-Id, воспользуйтесь следующей командой cURL:

```
curl -H "Request-Id:12345" http://localhost:8080/hello
```

Этот вызов вернет следующее тело ответа:

```
Hello!
```

5.3 *Добавление фильтра после существующего фильтра в цепочке*

В этом разделе говорится о том, как добавить фильтр в цепочку после существующего фильтра. Этот подход применяется, когда вы хотите выполнить некоторую логику после чего-то, уже существующего в цепочке фильтров. Предположим, что вам необходимо выполнить некоторые действия после процесса аутентификации. Например, это может быть уведомление другой системы после определенных событий аутентификации или просто для целей журналирования и трассировки (рис. 5.8). Как и в разделе 5.1, мы рассмотрим учебный пример. Впоследствии вы сможете адаптировать его к своим потребностям в реальном приложении.

В нашем примере мы регистрируем все успешные события аутентификации, добавляя пользовательский фильтр после фильтра аутентификации (рис. 5.8). Мы считаем, что прохождение фильтра аутентификации представляет собой событие успешной аутентифи-

кации, и хотим регистрировать такие события. Продолжая пример из раздела 5.1, мы заодно регистрируем идентификатор запроса, полученный через заголовок HTTP.

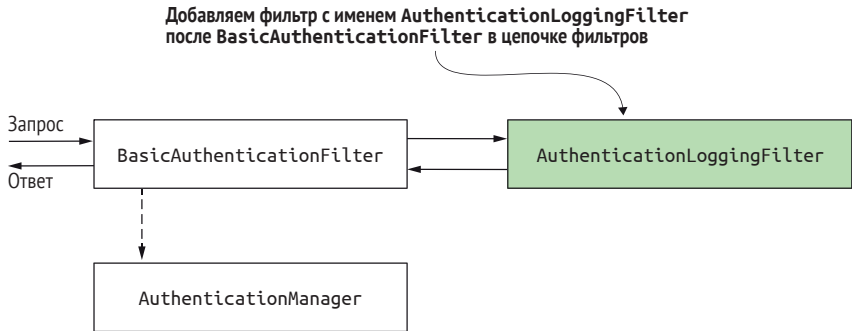


Рис. 5.8 Мы добавляем `AuthenticationLoggingFilter` после `BasicAuthenticationFilter` для регистрации запросов, которые завершились успешной аутентификацией в приложении

В листинге 5.5 показано определение фильтра, который регистрирует запросы, прошедшие фильтр аутентификации.

Листинг 5.5 Определение фильтра для регистрации запросов

```

public class AuthenticationLoggingFilter implements Filter {

    private final Logger logger =
        Logger.getLogger(
            AuthenticationLoggingFilter.class.getName());

    @Override
    public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain filterChain)
        throws IOException, ServletException {

        var httpRequest = (HttpServletRequest) request;

        var requestId =
            httpRequest.getHeader("Request-Id");

        logger.info("Successfully authenticated
            request with id " + requestId);

        filterChain.doFilter(request, response);
    }
}
  
```

- ❶ Получаем идентификатор запроса из заголовков запроса.
- ❷ Регистрируем событие со значением идентификатора запроса.
- ❸ Перенаправляем запрос следующему фильтру в цепочке.

Чтобы добавить пользовательский фильтр в цепочку *после* фильтра аутентификации, мы вызываем метод `addFilterAfter()` класса `HttpSecurity`. В следующем листинге показана его реализация.

Листинг 5.6 Добавление пользовательского фильтра в цепочку после существующего фильтра

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.addFilterBefore(
            new RequestValidationFilter(),
            BasicAuthenticationFilter.class)
            .addFilterAfter(
                new AuthenticationLoggingFilter(),
                BasicAuthenticationFilter.class)
            .authorizeRequests(c -> c.anyRequest().permitAll());

        return http.build();
    }
}
```

❶ Добавляем экземпляр `AuthenticationLoggingFilter` в цепочку фильтров после фильтра аутентификации.

После запуска приложения и вызова конечной точки мы наблюдаем, что для каждого успешного вызова конечной точки приложение выводит строку журнала в консоли. Для вызова

```
curl -H "Request-Id:12345" http://localhost:8080/hello
```

мы получаем тело ответа:

```
Hello!
```

В консоли вы можете увидеть примерно такие строки:

```
INFO 5876 --- [nio-8080-exec-2]
[CA]c.l.s.f.AuthenticationLoggingFilter:
[CA]Successfully authenticated request with id 12345
```

5.4 Добавление фильтра на место существующего

В этом разделе обсуждается добавление фильтра на позицию уже существующего в цепочке фильтров. Этот подход может быть осо-

бенно полезен при предоставлении другой реализации для ответственности, которая уже принята одним из фильтров, предлагаемых Spring Security. Типичным сценарием является аутентификация.

Предположим, что вместо потока аутентификации HTTP Basic вы хотите реализовать какой-то другой поток. Следовательно, вместо использования имени пользователя и пароля в качестве входных учетных данных, на основе которых приложение аутентифицирует пользователя, вам нужно применить другой подход. Вот несколько примеров сценариев, с которыми вы можете столкнуться на практике:

- идентификация на основе статического значения заголовка для аутентификации;
- использование симметричного ключа для подписи запроса на аутентификацию;
- использование одноразового пароля (one-time password, OTP) в процессе аутентификации.

В нашем первом сценарии (идентификация на основе статического ключа для аутентификации) клиент отправляет приложению строку в заголовке HTTP-запроса, который всегда одинаков. Приложение где-то хранит эти значения – скорее всего, в базе данных или хранилище секретов. На основе этого статического значения приложение идентифицирует клиента.

Этот подход (рис. 5.9) обеспечивает слабую безопасность аутентификации, но архитекторы и разработчики часто выбирают его для аутентификации вызовов между бэкенд-приложениями из-за его простоты. У этого подхода высокое быстродействие, поскольку не нужно выполнять сложные вычисления, как в случае применения криптографической подписи. Таким образом, статические ключи, используемые для аутентификации, представляют собой компромисс, при котором разработчики больше полагаются на инфраструктурные меры безопасности, но не оставляют конечные точки полностью незащищенными.

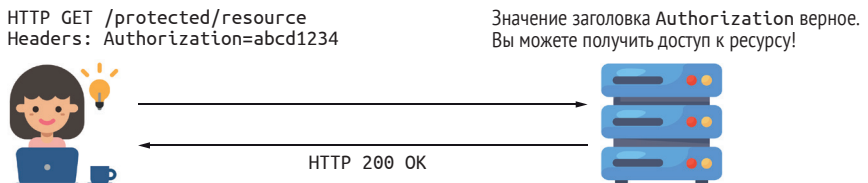


Рис. 5.9 Запрос содержит заголовок со значением статического ключа. Если это значение совпадает со значением, известным приложению, оно принимает запрос

В нашем втором сценарии, использующем симметричные ключи для подписи и проверки запросов, и клиент, и сервер знают значе-

ние ключа (клиент и сервер совместно используют ключ). Клиент использует этот ключ для подписи части запроса (например, для подписи значения определенных заголовков), а сервер проверяет, действительна ли подпись, используя тот же ключ (рис. 5.10). Сервер может хранить отдельные ключи для каждого клиента в базе данных или хранилище секретов. Аналогичным образом вы можете использовать пару асимметричных ключей.

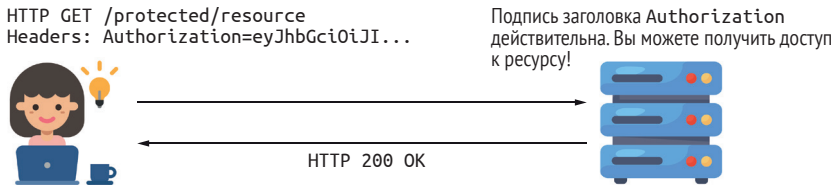


Рис. 5.10 Заголовок Authorization содержит значение, зашифрованное с помощью ключа, совместно используемого клиентом и сервером (или зашифрованное с помощью закрытого ключа, для которого сервер обладает соответствующим открытым ключом). Если приложение видит, что подпись действительна, оно продолжает обработку запроса

Наконец, в нашем третьем сценарии, использующем ОТР в процессе аутентификации, пользователь получает ОТР через сообщение или с помощью приложения поставщика аутентификации, такого как Google Authenticator (рис. 5.11).

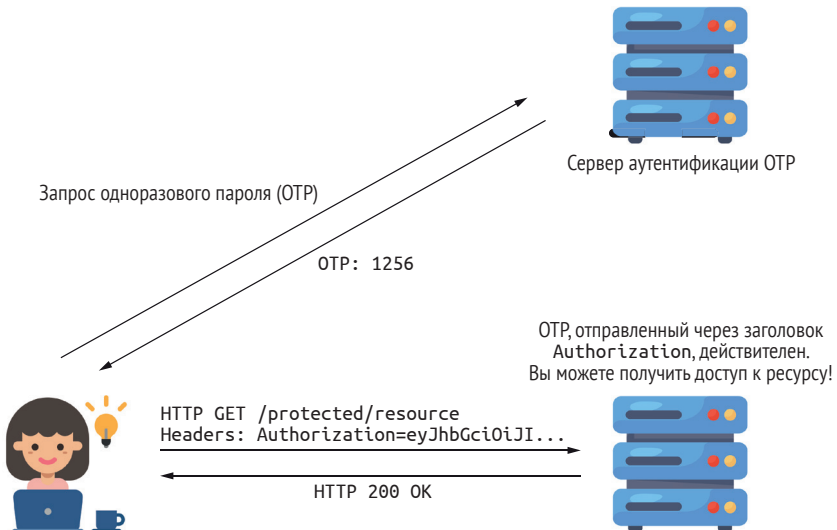


Рис. 5.11 Для входа в ресурс клиент должен использовать одноразовый пароль (ОТР). Этот ОТР поступает с внешнего сервера аутентификации. Обычно приложения используют данный метод для процессов входа, требующих многофакторной аутентификации

Теперь рассмотрим следующий пример реализации пользовательского фильтра. Для простоты мы сосредоточимся на конфигурации и рассмотрим простую логику аутентификации. В нашем сценарии у нас есть значение статического ключа, которое одинаково для всех запросов. Для аутентификации пользователь должен добавить правильное значение статического ключа в заголовок `Authorization`, как показано на рис. 5.12. Код для этого примера можно найти в проекте `ssia-ch5-ex2`.

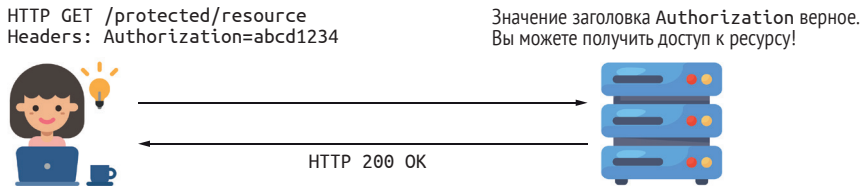


Рис. 5.12 Клиент добавляет статический ключ в заголовок `Authorization` HTTP-запроса. Сервер проверяет, знает ли он этот ключ, прежде чем авторизовать запросы

Начнем с реализации класса фильтра, называемого `StaticKeyAuthenticationFilter`. Этот класс считывает значение статического ключа из файла свойств и проверяет, равно ли ему значение заголовка `Authorization`. Если значения совпадают, фильтр пересылает запрос следующему компоненту в цепочке фильтров. Если нет, фильтр возвращает значение статуса HTTP ответа 401 Unauthorized и не пересылает запрос дальше по цепочке. В листинге 5.7 показано определение класса `StaticKeyAuthenticationFilter`.

Листинг 5.7 Определение класса `StaticKeyAuthenticationFilter`

```
@Component ①
public class StaticKeyAuthenticationFilter
    implements Filter { ②

    @Value("${authorization.key}") ③
    private String authorizationKey;

    @Override
    public void doFilter(ServletRequest request,
                        ServletResponse response,
                        FilterChain filterChain)
        throws IOException, ServletException {

        var httpRequest = (HttpServletRequest) request;
        var httpResponse = (HttpServletResponse) response;

        String authentication = ④
            httpRequest.getHeader("Authorization");
```



```

    if (authorizationKey.equals(authentication)) {
        filterChain.doFilter(request, response);
    } else {
        httpResponse.setStatus(
            HttpServletResponse.SC_UNAUTHORIZED);
    }
}
}
}

```

- ❶ Добавляем экземпляр класса в контекст Spring, чтобы иметь возможность вводить значения из файла свойств.
- ❷ Определяем логику аутентификации, реализуя интерфейс Filter и переопределяя метод doFilter().
- ❸ Берем значение статического ключа из файла свойств с помощью аннотации @Value.
- ❹ Берем значение заголовка Authorization из запроса для сравнения его со статическим ключом.

После определения фильтра мы добавляем его в цепочку фильтров в позицию класса Basic AuthenticationFilter с помощью метода addFilterAt() (рис. 5.13).

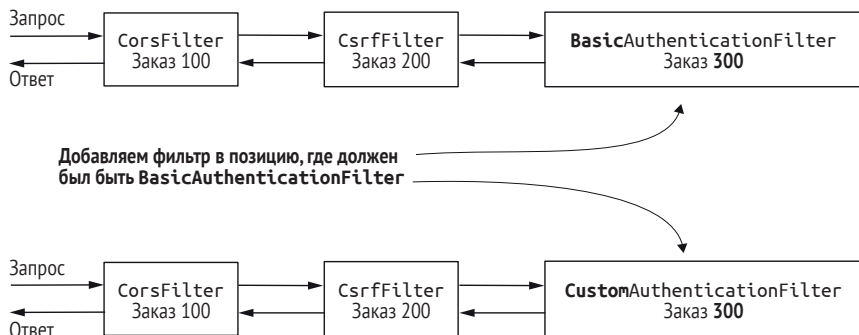


Рис. 5.13 Мы добавляем наш пользовательский фильтр аутентификации в то место, где был бы класс BasicAuthenticationFilter, если бы мы использовали HTTP Basic в качестве метода аутентификации. Это означает, что наш пользовательский фильтр имеет тот порядковый идентификатор

Но вспомните, о чем я говорил в разделе 5.1. При добавлении фильтра в определенную позицию Spring Security не предполагает, что это единственный фильтр в этой позиции. Вы можете добавить несколько фильтров в одно и то же место в цепочке. В этом случае Spring Security не гарантирует порядок, в котором они будут действовать. Я напоминаю об этом, потому что видел, как многие люди были сбиты с толку неожиданными последствиями. Некоторые разработчики думают, что новый фильтр заменяет старый. Это не так! Вы должны убедиться, что не добавляете фильтры, которые вам не нужны.

ПРИМЕЧАНИЕ Я советую вам не добавлять несколько фильтров в одну и ту же позицию в цепочке. Когда вы добавляете несколько фильтров в одно и то же место, порядок, в котором они используются, не определен. Лучше обеспечить определенный порядок, в котором вызываются фильтры. Наличие известного порядка упрощает понимание и поддержку вашего приложения.

В листинге 5.8 показано определение класса конфигурации, который добавляет фильтр. Обратите внимание, что мы не вызываем здесь метод `httpBasic()` из класса `HttpSecurity`, поскольку не хотим, чтобы экземпляр `BasicAuthenticationFilter` был добавлен в цепочку фильтров.

Листинг 5.8 Добавление фильтра в класс конфигурации

```
@Configuration
public class ProjectConfig {

    private final StaticKeyAuthenticationFilter filter;           ❶
    // Конструктор не показан

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http.addFilterAt(filter,                                ❷
            BasicAuthenticationFilter.class)
            .authorizeRequests(c -> c.anyRequest().permitAll());

        return http.build();
    }
}
```

❶ Внедряем экземпляр фильтра из контекста Spring.

❷ Добавляем фильтр в позицию фильтра базовой аутентификации в цепочке фильтров.

Для тестирования приложения нам также нужна конечная точка. Для этого мы определяем контроллер, как в листинге 5.4. Вам следует добавить значение для статического ключа на сервере в файле `.properties` приложения:

```
authorization.key=SD9cICj11e
```

ПРИМЕЧАНИЕ Хранение паролей, ключей или любых других данных производственного приложения, которые не должны быть общедоступны, в файле свойств – плохая идея! В наших примерах мы используем этот подход для простоты и для того, чтобы вы могли сосредоточиться на создаваемых

нами конфигурациях Spring Security. Но в реальных сценариях обязательно используйте секретное хранилище для хранения подобных сведений.

Теперь мы можем протестировать приложение. Ожидается, что приложение будет разрешать запросы с правильным значением для заголовка `Authorization` и отклонять остальные, возвращая статус HTTP 401 Unauthorized в ответе. В следующих фрагментах кода представлены вызовы `curl`, используемые для тестирования приложения. Если вы используете в заголовке `Authorization` значение, установленное на стороне сервера, вызов будет успешным, и вы увидите тело ответа. Вызов

```
curl -H "Authorization:SD9cICjl1e" http://localhost:8080/hello
```

возвращает ожидаемое тело ответа:

```
Hello!
```

В вызове, если заголовок `Authorization` отсутствует или неверен, статус ответа – HTTP 401 Unauthorized:

```
curl -v http://localhost:8080/hello
```

Статус ответа:

```
...  
< HTTP/1.1 401  
...
```

В этом случае, поскольку мы не настраиваем `UserDetailsService`, Spring Boot настраивает его автоматически, как вы узнали в главе 2. Но в нашем сценарии вам вообще не нужен `UserDetailsService`, поскольку концепция пользователя не существует. Мы только проверяем, знает ли пользователь, запрашивающий вызов конечной точки на сервере, заданное значение. Сценарии аутентификации реальных приложений обычно не так просты, и для них часто требуется `UserDetailsService`. Однако если вы уверены, что этот компонент не нужен, вы можете отключить автоконфигурацию. Чтобы отключить конфигурацию службы `UserDetailsService` по умолчанию, можно использовать атрибут `exclude` аннотации `@SpringBootApplication` в главном классе:

```
@SpringBootApplication(exclude =  
    {UserDetailsServiceAutoConfiguration.class })
```

5.5 Реализации фильтров, предоставляемые Spring Security

В этом разделе мы рассмотрим классы, предоставляемые Spring Security, реализующие интерфейс `Filter`. В примерах мы определяем фильтр, реализуя этот интерфейс напрямую.

Spring Security предлагает несколько абстрактных классов, реализующих интерфейс `Filter`, для которых вы можете расширить определения фильтров. Эти классы также добавляют функциональность, которую ваши реализации могут получить при их расширении. Например, вы можете расширить класс `GenericFilterBean`, позволяющий использовать параметры инициализации, которые вы определили в файле дескриптора `web.xml`, где это применимо. Более полезным классом, который расширяет `GenericFilterBean`, является `OncePerRequestFilter`. В общем случае при добавлении фильтра в цепочку фреймворк не гарантирует, что он будет вызван только один раз за запрос. Логика класса `OncePerRequestFilter`, как следует из названия, гарантирует, что метод `doFilter()` фильтра выполняется только один раз за запрос.

Если вам нужна такая функциональность в приложении, используйте готовые классы, предоставляемые Spring. Однако я всегда рекомендую максимально упрощать пользовательские реализации и не использовать фильтры без нужды. Слишком часто я видел, как разработчики расширяли класс `GenericFilterBean`, вместо того чтобы реализовывать интерфейс `Filter` в функциях, которые не требуют пользовательской логики, добавленной классом `GenericFilterBean`. Когда я спрашивал о причине такого решения, они затруднялись ответить. Вероятно, они просто скопировали реализацию, которую нашли в интернете.

Чтобы вам было ясно, как использовать такой класс, давайте рассмотрим еще один пример. Функциональность журналирования, которую мы реализовали в разделе 5.3, является отличным кандидатом для использования `OncePerRequestFilter`. Нам нужно избежать многократной регистрации одних и тех же запросов. Spring Security не гарантирует, что фильтр не будет вызван более одного раза, поэтому мы должны позаботиться об этом сами. Самый простой способ – реализовать фильтр с помощью класса `OncePerRequestFilter`. Я сделал это в отдельном проекте под названием `ssia-ch5-ex3`.

В листинге 5.9 вы найдете изменение, которое я внес в класс `AuthenticationLoggingFilter`. Вместо того чтобы реализовывать интерфейс `Filter` напрямую, как это было в примере в разделе 5.3, он

теперь расширяет класс `OncePerRequestFilter`. Метод, который мы здесь переопределяем, – это `doFilterInternal()`. Этот код вы можете найти в проекте `ssia-ch5-ex3`.

Листинг 5.9 Расширение класса `OncePerRequestFilter`

```
public class AuthenticationLoggingFilter
    extends OncePerRequestFilter { ❶

    private final Logger logger =
        Logger.getLogger(
            AuthenticationLoggingFilter.class.getName());

    @Override
    protected void doFilterInternal( ❷
        HttpServletRequest request, ❸
        HttpServletResponse response, ❹
        FilterChain filterChain) throws
        ServletException, IOException {

        String requestId = request.getHeader("Request-Id");

        logger.info("Successfully authenticated request with id " +
            requestId);

        filterChain.doFilter(request, response);
    }
}
```

- ❶ Вместо реализации интерфейса `Filter` он расширяет класс `OncePerRequestFilter`.
- ❷ Переопределяет `doFilterInternal()`, который заменяет назначение метода `doFilter()` интерфейса `Filter`.
- ❸ `OncePerRequestFilter` поддерживает только HTTP-фильтры. Вот почему параметры напрямую задаются как `HttpServletRequest` и `HttpServletResponse`.

Несколько коротких замечаний о классе `OncePerRequestFilter`, которые могут оказаться полезными:

- он поддерживает только HTTP-запросы, но на самом деле мы всегда их используем. Преимущество в том, что он приводит типы, и мы напрямую получаем запросы как `HttpServletRequest` и `HttpServletResponse`. Напомню, что с интерфейсом `Filter` нам пришлось привести запрос и ответ;
- вы можете реализовать логику, чтобы решить, применяется ли фильтр. Даже если вы добавили фильтр в цепочку, вы можете решить, что он не будет применяться к определенным запросам. Вы добиваетесь этого, переопределяя метод `shouldNotFilter(HttpServletRequest)`. По умолчанию фильтр применяется ко всем запросам;

- по умолчанию `OncePerRequestFilter` не применяется к асинхронным запросам или запросам на отправку ошибок. Вы можете изменить это поведение, переопределив методы `shouldNotFilterAsyncDispatch()` и `shouldNotFilterErrorDispatch()`.

Если какие-либо из вышеупомянутых характеристик `OncePerRequestFilter` выглядят полезными в вашей реализации, я рекомендую использовать этот класс для определения ваших фильтров.

Заключение

- Первый уровень архитектуры веб-приложения, который перехватывает HTTP-запросы, представляет собой цепочку фильтров. Что касается других компонентов в архитектуре Spring Security, вы можете настроить их в соответствии с вашими требованиями.
- Вы можете настроить цепочку фильтров, добавив новые фильтры до, после или в позицию существующего фильтра.
- Вы можете иметь несколько фильтров в одной и той же позиции существующего фильтра. В этом случае порядок выполнения фильтров не определен.
- Изменение цепочки фильтров помогает вам настроить аутентификацию и авторизацию в соответствии с требованиями вашего приложения.

Реализация аутентификации

Темы этой главы:

- реализация логики аутентификации с использованием пользовательского компонента `AuthenticationProvider`;
- использование методов аутентификации HTTP Basic и на основе форм;
- принцип работы компонента `Security-Context`.

В главах 3 и 4 было рассмотрено несколько компонентов, действующих в потоке аутентификации. Мы обсудили `UserDetails` и определение прототипа для описания пользователя в Spring Security. Затем мы использовали `UserDetails` в примерах, которые показали, как работают контракты `UserDetailsService` и `UserDetailsManager` и как они реализованы. Мы также обсудили и использовали ведущие реализации этих интерфейсов в примерах. Наконец, вы узнали, как `PasswordEncoder` управляет паролями и как его применять, а также изучили криптографический модуль Spring Security (SSCM) с его шифраторами и генераторами ключей.

Но слой `AuthenticationProvider` отвечает за логику аутентификации. Это место, где вы найдете условия и инструкции о том, как именно аутентифицировать запрос. В главе 5 говорилось о том, что компонент `AuthenticationManager` получает запрос от слоя фильтра HTTP и делегирует ответственность за аутентификацию компоненту

AuthenticationProvider. В этой главе мы рассмотрим процесс аутентификации, который имеет только два возможных исхода:

- *субъект, делающий запрос, не аутентифицирован.* Пользователь не распознан, и приложение отклоняет запрос, не делегируя ответственность процессу авторизации. Обычно в этом случае клиенту возвращается статус ответа HTTP 401 Unauthorized;
- *субъект, делающий запрос, аутентифицирован.* Данные о запрашивающей стороне сохраняются, чтобы приложение могло использовать их для авторизации. Как вы узнаете в этой главе, SecurityContext отвечает за данные, касающиеся текущего аутентифицированного запроса. Чтобы напомнить об участниках процесса и связях между ними, на рис. 6.1 показана схема, которую вы уже видели в главе 2.

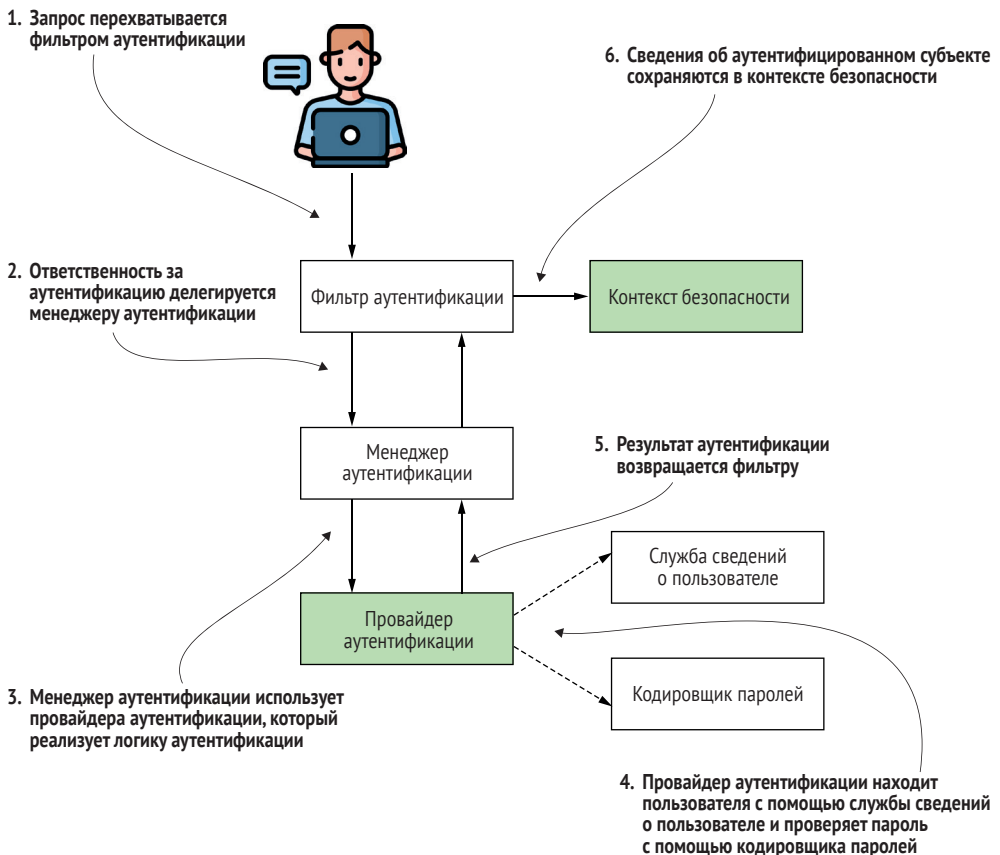


Рис. 6.1 Поток аутентификации в Spring Security. Этот процесс описывает метод, с помощью которого приложение распознает субъекта, отправляющего запрос. Элементы, которые находятся в центре внимания данной главы, закрашены. В этом контексте AuthenticationProvider отвечает за выполнение процедуры аутентификации, а SecurityContext сохраняет информацию о запросе, который был аутентифицирован

В этой главе рассматриваются оставшиеся части потока аутентификации (закрашенные блоки на рис. 6.1). Затем, в главах 7 и 8, вы узнаете, как работает авторизация, которая является процессом, следующим за аутентификацией в HTTP-запросе. Сначала нам нужно обсудить, как реализовать интерфейс `AuthenticationProvider`. Вам нужно знать, как Spring Security понимает запрос в процессе аутентификации.

Чтобы получить четкое представление о запросе аутентификации, мы начнем с интерфейса `Authentication`. Затем мы сможем пойти дальше и посмотреть, что происходит с деталями запроса после успешной аутентификации. После этого вы узнаете об интерфейсе `SecurityContext` и о том, как им оперирует Spring Security. Ближе к концу главы вы узнаете, как настраивать метод аутентификации HTTP Basic. Мы также обсудим еще один вариант аутентификации, который можно использовать в наших приложениях, – вход на основе форм.

6.1 Что такое `AuthenticationProvider`

При работе с корпоративными приложениями вы можете оказаться в ситуации, когда аутентификация по умолчанию на основе имени пользователя и пароля не применяется. Кроме того, когда дело доходит до аутентификации, ваше приложение может потребовать реализации нескольких сценариев (рис. 6.2). Например, иногда требуется, чтобы пользователь мог подтвердить свою личность, используя код, полученный в SMS или отображенный в пуш-сообщении. Или от вас потребуют реализовать сценарии аутентификации, где пользователь должен предоставить определенный вид ключа, сохраненного в файле. Вам может даже потребоваться использовать представление отпечатка пальца пользователя для реализации логики аутентификации. Фреймворк должен быть достаточно гибким, чтобы помочь реализовать любой из этих сценариев.

Фреймворк обычно предоставляет набор наиболее часто используемых реализаций, но, конечно, он не может охватить все возможные варианты. Что касается Spring Security, вы можете использовать контракт `AuthenticationProvider` для определения любой пользовательской логики аутентификации. В этом разделе вы научитесь представлять событие аутентификации, реализуя интерфейс `Authentication`, а затем создавать свою собственную логику аутентификации с помощью `AuthenticationProvider`. Чтобы достичь нашей цели:

- в разделе 6.1.1 мы проанализируем, как Spring Security представляет событие аутентификации;
- в разделе 6.1.2 мы обсудим контракт `AuthenticationProvider`, который отвечает за логику аутентификации;

- в разделе 6.1.3 вы напишете собственную логику аутентификации, реализовав контракт AuthenticationProvider в примере кода.

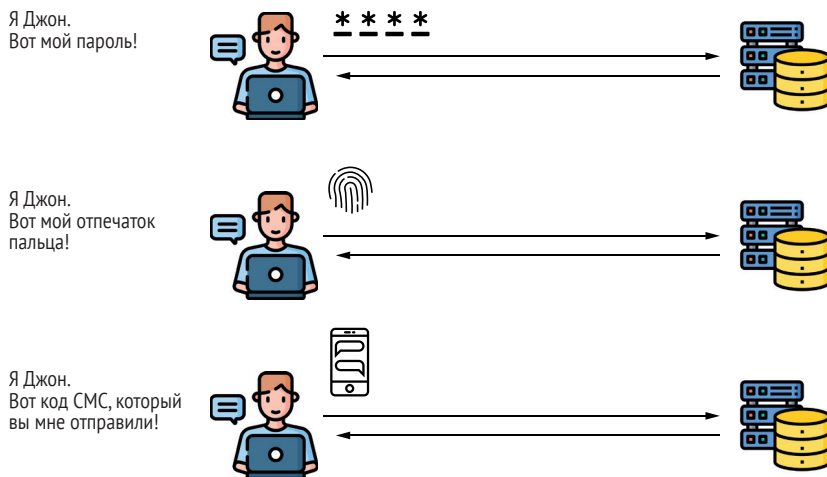


Рис. 6.2 Приложению могут потребоваться различные методы аутентификации. Хотя для большинства сценариев достаточно имени пользователя и пароля, существуют случаи, когда процесс аутентификации пользователя более сложен

6.1.1 Представление запроса во время аутентификации

В этом разделе говорится о том, как Spring Security понимает запрос во время процесса аутентификации. Вы должны знать об этом, прежде чем углубляться в реализацию собственной логики аутентификации. Как вы увидите в разделе 6.1.2, для реализации пользовательского AuthenticationProvider сначала нужно понять, как описать событие аутентификации. Поэтому сейчас мы рассмотрим контракт, представляющий аутентификацию, и обсудим методы, которые вам нужно знать.

Authentication – один из основных интерфейсов, участвующих в процессе аутентификации. Интерфейс Authentication представляет событие запроса аутентификации и содержит сведения о субъекте, который запрашивает доступ к приложению. Вы можете использовать информацию, связанную с событием запроса аутентификации, как во время процесса аутентификации, так и после него. Пользователь, запрашивающий доступ к приложению, называется *принципалом* (principal). Если вы когда-либо использовали Java Security в каком-либо приложении, вы, вероятно, знаете, что интерфейс с именем Principal представляет ту же концепцию. Интерфейс Authentication фреймворка Spring Security расширяет этот контракт (рис. 6.3).

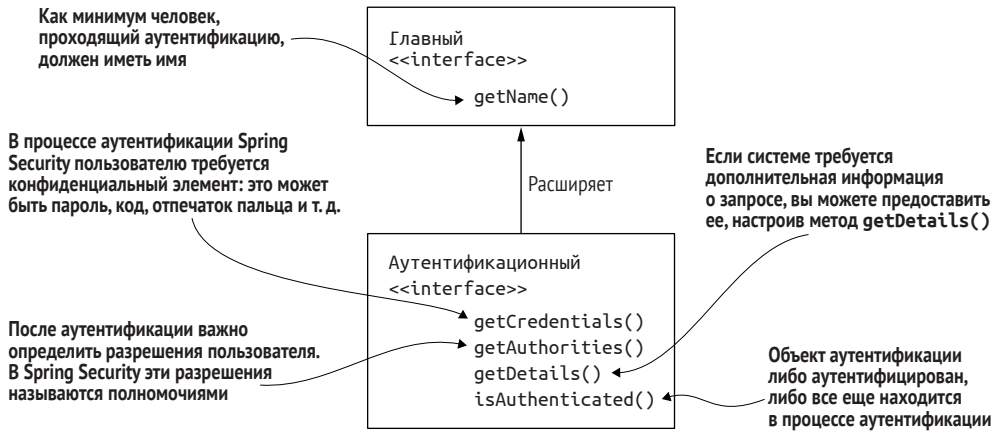


Рис. 6.3 Соглашение об аутентификации расширяет контракт `Principal`. Оно вводит дополнительные условия, такие как необходимость пароля или возможность предоставления дополнительных сведений относительно запроса аутентификации. Некоторые аспекты, такие как массив полномочий, специфичны для Spring Security

Контракт `Authentication` в Spring Security не только представляет принципа, но и добавляет информацию о том, завершается ли процесс аутентификации, а также набор полномочий. Тот факт, что этот контракт был разработан для расширения контракта `Principal` из Java Security, является плюсом с точки зрения совместимости с реализациями других фреймворков и приложений. Эта гибкость позволяет более легко переносить приложения, реализующие аутентификацию другим способом, в Spring Security.

Более детальное представление интерфейса `Authentication` вы найдете в листинге 6.1.

Листинг 6.1 Интерфейс аутентификации, объявленный в Spring Security

```
public interface Authentication extends Principal, Serializable {

    Collection<? extends GrantedAuthority> getAuthorities();
    Object getCredentials();
    Object getDetails();
    Object getPrincipal();
    boolean isAuthenticated();
    void setAuthenticated(boolean isAuthenticated)
        throws IllegalArgumentException;
}
```

На данный момент вам нужно изучить только следующие методы этого контракта:

- `isAuthenticated()` – возвращает `true`, если процесс аутентификации завершен, или `false`, если процесс аутентификации все еще выполняется;

- `getCredentials()` – возвращает пароль или любой секрет, используемый в процессе аутентификации;
- `getAuthorities()` – возвращает коллекцию предоставленных полномочий для аутентифицированного запроса.

Мы обсудим другие методы контракта аутентификации в последующих главах в соответствии с рассматриваемыми реализациями.

6.1.2 Реализация пользовательской логики аутентификации

В этом разделе рассматривается реализация пользовательской логики аутентификации. Мы проанализируем контракт Spring Security, связанный с этой ответственностью, чтобы понять его определение. Располагая необходимыми знаниями, мы перейдем к написанию пользовательской логики аутентификации на примере кода в разделе 6.1.3.

`AuthenticationProvider` в Spring Security отвечает за логику аутентификации. Реализация интерфейса `AuthenticationProvider` по умолчанию делегирует ответственность за поиск пользователя системы компоненту `UserDetailsService`. Она также использует `PasswordEncoder` для управления паролями в процессе аутентификации. В листинге 6.2 представлено определение `AuthenticationProvider`, которое необходимо для пользовательского провайдера аутентификации вашего приложения.

Листинг 6.2 Интерфейс `AuthenticationProvider`

```
public interface AuthenticationProvider {  
  
    Authentication authenticate(Authentication authentication)  
        throws AuthenticationException;  
  
    boolean supports(Class<?> authentication);  
}
```

Ответственность `AuthenticationProvider` тесно связана с контрактом `Authentication`. Метод `authenticate()` получает объект `Authentication` в качестве параметра и возвращает объект `Authentication`. Мы реализуем метод `authenticate()` для определения логики аутентификации. Вот краткое описание способа, которым мы должны реализовать метод `authenticate()`:

- метод должен выдавать исключение `AuthenticationException`, если аутентификация не удалась;
- если метод получает объект аутентификации, не поддерживаемый вашей реализацией `AuthenticationProvider`, то он должен возвращать `null`. Таким образом, у нас есть возможность ис-

пользовать несколько типов аутентификации, разделенных на уровне HTTP-фильтра;

- метод должен возвращать экземпляр `Authentication`, представляющий полностью аутентифицированный объект. Для этого экземпляра метод `isAuthenticated()` возвращает `true`, и он содержит все необходимые сведения об аутентифицированном объекте. Обычно приложение также удаляет из этого экземпляра конфиденциальные данные, такие как пароль. После успешной аутентификации пароль больше не требуется, и сохранение этих сведений может потенциально раскрыть их посторонним.

Второй метод в интерфейсе `AuthenticationProvider` – это `supports(Class<?> authentication)`. Вы можете заставить его возвращать `true`, если текущий `AuthenticationProvider` поддерживает тип, предоставленный как объект `Authentication`. Обратите внимание, что даже если этот метод возвращает `true` для объекта, все еще остается вероятность, что метод `authenticate()` отклонит запрос, вернув `null`. Spring Security работает очень гибко, позволяя пользователям реализовывать `AuthenticationProvider`, который может отклонять запрос аутентификации на основе его сведений, а не только на основе его типа.

Аналогом совместной работы менеджера и провайдера аутентификации может служить усовершенствованный замок для вашей двери. Вы можете открыть этот замок либо с помощью карты, либо с помощью старого физического ключа (рис. 6.4). Сам замок является менеджером аутентификации, который решает, открывать ли дверь. Чтобы принять это решение, он делегирует полномочия двум поставщикам аутентификации: одному, который знает, как проверить карту, или другому, который знает, как проверить физический ключ. Если вы хотите использовать карту, чтобы открыть дверь, поставщик аутентификации, который работает только с физическими ключами, жалуется, что он незнаком с этим типом аутентификации. Однако другой поставщик поддерживает этот тип аутентификации и проверяет, действительна ли карта для данной двери. В этом и заключается назначение методов `supports()`.

Помимо тестирования типа аутентификации, Spring Security добавляет еще один уровень гибкости. Замок двери может распознавать несколько видов карт. В этом случае, когда вы предъявляете карту, один из поставщиков аутентификации может сказать: «Я понимаю, что это карта. Но это не тот тип карты, который я могу проверить!» Это происходит, когда `supports()` возвращает `true`, а `authenticate()` возвращает `null`.

На рис. 6.5 показан альтернативный сценарий, когда один из объектов `AuthenticationProvider` распознает `Authentication`, но решает, что он недействителен. В этом случае результатом будет исключе-

ние `AuthenticationException`, которое заканчивается статусом 401 Unauthorized HTTP в ответе веб-приложения.

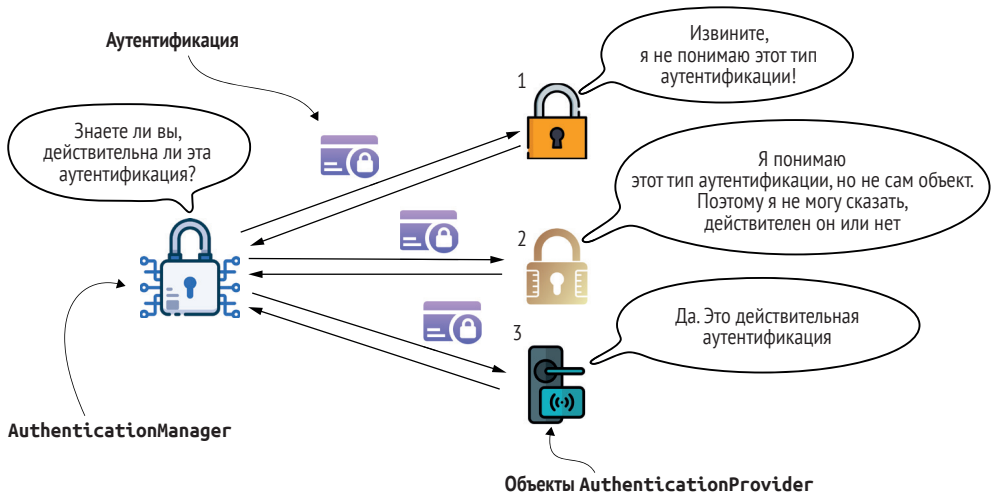


Рис. 6.4 `AuthenticationManager` делегирует полномочия одному из доступных провайдеров аутентификации. `AuthenticationProvider` может не поддерживать предоставленный тип аутентификации. Однако если он поддерживает тип объекта, он может не знать, как аутентифицировать этот конкретный объект. Аутентификация проверяется, и `AuthenticationProvider`, который может сказать, является запрос действительным или нет, возвращает ответ менеджеру аутентификации `AuthenticationManager`

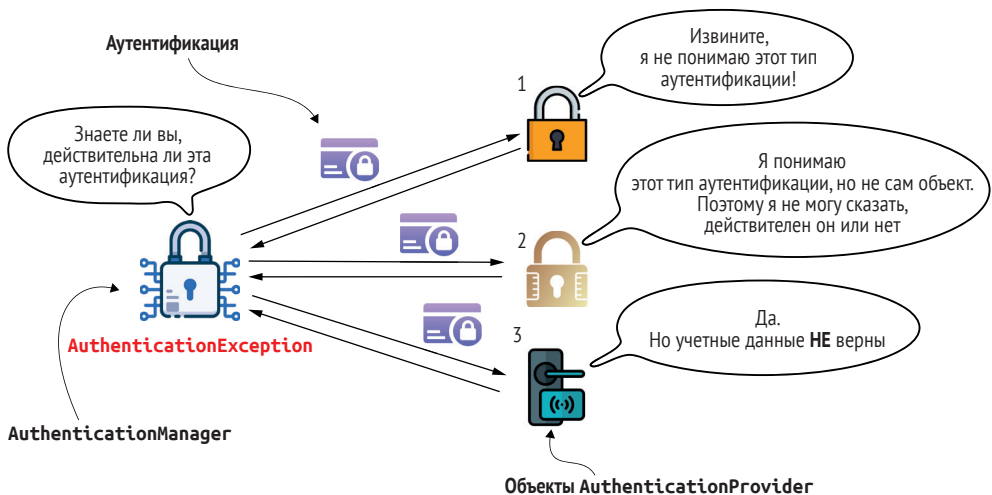


Рис. 6.5 Если ни один из объектов `AuthenticationProvider` не распознает аутентификацию или каждый из них отклоняет ее, результатом является `AuthenticationException`

6.1.3 Применение пользовательской логики аутентификации

В этом разделе вы познакомитесь с примером пользовательской логики аутентификации. Вы можете найти его в проекте `ssia-ch6-ex1`. Здесь применяются знания об интерфейсах `Authentication` и `AuthenticationProvider`, полученные в разделах 6.1.1 и 6.1.2. В листингах 6.3 и 6.4 показан пример реализации пользовательского `AuthenticationProvider`. Мы выполняем следующие шаги, также представленные на рис. 6.5:

- 1 Объявите класс, реализующий контракт `AuthenticationProvider`.
- 2 Решите, какие виды объектов `Authentication` поддерживаются новым `AuthenticationProvider`.
- 3 Реализуйте метод `supports(Class<?> c)`, чтобы указать, какой тип аутентификации поддерживается определяемым вами провайдером `AuthenticationProvider`.
- 4 Реализуйте метод `authenticate(Authentication a)` для реализации логики аутентификации.
- 5 Зарегистрируйте экземпляр новой реализации `AuthenticationProvider` с помощью `Spring Security`.

Листинг 6.3 Переопределение метода `supports()` в `AuthenticationProvider`

```
@Component
public class CustomAuthenticationProvider
    implements AuthenticationProvider {

    // Часть кода не показана

    @Override
    public boolean supports(Class<?> authenticationType) {
        return authenticationType
            .equals(UsernamePasswordAuthenticationToken.class);
    }
}
```

В листинге 6.3 мы определяем новый класс, реализующий интерфейс `AuthenticationProvider`. Мы помечаем класс с помощью аннотации `@Component`, чтобы иметь экземпляр его типа в контексте, управляемом `Spring`. Затем мы должны решить, какой тип реализации интерфейса `Authentication` поддерживает этот `AuthenticationProvider`. Это зависит от того, какой тип мы собираемся предоставить в качестве параметра для метода `authenticate()`. Если мы ничего не настраиваем на уровне фильтра аутентификации (как обсуждалось в главе 5), то тип определяет класс `UsernamePassword`

AuthenticationToken. Этот класс является реализацией интерфейса Authentication и представляет стандартный запрос аутентификации с именем пользователя и паролем.

С помощью этого определения мы заставили AuthenticationProvider поддерживать определенный тип ключа. После того как мы указали область действия нашего AuthenticationProvider, мы реализуем логику аутентификации, переопределяя метод authenticate(), как показано в следующем листинге.

Листинг 6.4 Реализация логики аутентификации

```
@Component
public class CustomAuthenticationProvider
    implements AuthenticationProvider {

    private final UserDetailsService userDetailsService;
    private final PasswordEncoder passwordEncoder;

    // Конструктор не показан

    @Override
    public Authentication authenticate(Authentication authentication) {
        String username = authentication.getName();
        String password = authentication.getCredentials().toString();

        UserDetails u = userDetailsService.loadUserByUsername(username);

        if (passwordEncoder.matches(password, u.getPassword())) {
            return new UsernamePasswordAuthenticationToken(
                username,
                password,
                u.getAuthorities());
        } else {
            throw new BadCredentialsException
                ("Something went wrong!");
        }
    }

    // Часть кода не показана
}
```

❶ Если пароль совпадает, он возвращает реализацию контракта Authentication с необходимыми данными.

❷ Если пароль не совпадает, он выдает исключение типа AuthenticationException. BadCredentialsException наследует от AuthenticationException.

Логика в листинге 6.4 проста и визуально представлена на рис. 6.6. Мы используем реализацию UserDetailsService для получения UserDetails. Если пользователь не существует, метод loadUserByUser-

`name()` должен выдать `AuthenticationException`. В этом случае процесс аутентификации останавливается, и HTTP-фильтр возвращает статус HTTP 401 Unauthorized. Если имя пользователя существует, мы можем дополнительно проверить пароль пользователя с помощью метода `matches()` `PasswordEncoder` из контекста. Если пароль не совпадает, то снова должно быть выдано исключение `AuthenticationException`. Если пароль правильный, `AuthenticationProvider` возвращает экземпляр `Authentication`, который помечен как «аутентифицированный» и содержит сведения о запросе.

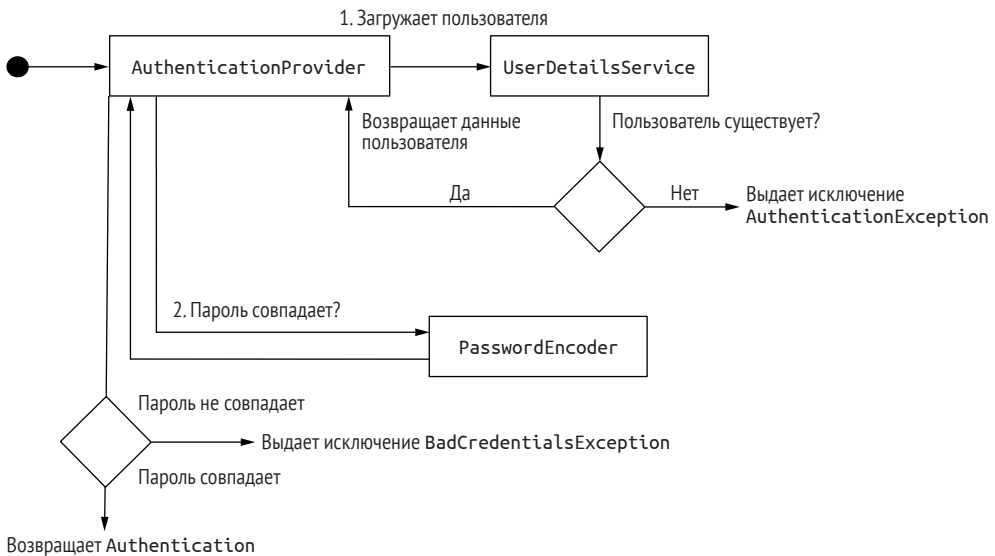


Рис. 6.6 `AuthenticationProvider` реализует специализированную процедуру аутентификации. Он подтверждает запрос аутентификации, извлекая сведения о пользователе через определенную реализацию `UserDetailsService`, и проверяет пароль с помощью `PasswordEncoder`. Если пользователь не найден или пароль не совпадает, `AuthenticationProvider` вызывает исключение `AuthenticationException`

Чтобы подключить новую реализацию `AuthenticationProvider`, мы определяем компонент `SecurityFilterChain`, как показано в листинге 6.5.

Листинг 6.5 Регистрация `AuthenticationProvider` в классе конфигурации

```

@Configuration
public class ProjectConfig {

    private final AuthenticationProvider authenticationProvider;

    // Конструктор не показан
  
```

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authenticationProvider(authenticationProvider);

    http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

    return http.build();
}

// Часть кода не показана
}
```

ПРИМЕЧАНИЕ В листинге 6.5 для внедрения зависимости применяется поле, объявленное с использованием интерфейса `AuthenticationProvider`. Spring распознает `AuthenticationProvider` как интерфейс (который является абстракцией). Однако Spring знает, что ему нужно найти экземпляр реализации в своем контексте для этого конкретного интерфейса. В нашем случае реализация – это экземпляр `CustomAuthenticationProvider`. Он является единственным экземпляром такого типа, который мы объявили и добавили в контекст Spring с помощью аннотации `@Component`. Для дополнительного чтения о внедрении зависимости я рекомендую еще одну свою книгу «Spring быстро» (Питер, 2022).

Вот и все! Вы успешно настроили реализацию `AuthenticationProvider`. Теперь вы можете настроить логику аутентификации для своего приложения там, где вам это нужно.

Как потерпеть фиаско при разработке приложения

Неправильное применение фреймворка приводит к тому, что приложение становится менее удобным для поддержки. Что еще хуже, иногда те, кто не справляется с использованием фреймворка, считают, что это вина фреймворка. Позвольте мне рассказать вам историю.

Однажды зимой руководитель разработки в компании, с которой я сотрудничал в качестве консультанта, позвонил мне с просьбой о помощи в реализации новой функции. Им нужно было применить пользовательский метод аутентификации в компоненте их системы, разработанной с помощью Spring на ранних этапах. К сожалению, при реализации класса приложения разработчики не учитывали должным образом архитектуру Spring Security.

Они использовали только цепочку фильтров, заново реализуя готовые функции из Spring Security в виде пользовательского кода.

Разработчики заметили, что со временем кастомизация (разработка пользовательских функций) приложения становится все сложнее. Однако никто не предпринял действий для правильной перепроектировки компонента и использования контрактов, как предполагалось в Spring Security. Большая часть трудностей возникла из-за незнания возможностей Spring. Один из ведущих разработчиков сказал: «Это вина проклятого Spring Security! Этот фреймворк трудно применять и еще труднее приспособить под свои нужды». Я был немного шокирован его высказыванием. Я знаю, что Spring Security иногда сложно понять и что этот фреймворк известен тем, что не имеет плавной кривой обучения. Но я никогда не сталкивался с ситуацией, в которой я не мог бы найти способ разработать простой в кастомизации класс с помощью Spring Security!

Мы вместе исследовали проблему, и я понял, что разработчики приложения использовали, возможно, только 10 % того, что может предложить Spring Security. Затем я провел двухдневный семинар по Spring Security, сосредоточившись на том, что мы можем сделать для конкретного системного компонента, который им нужно было изменить, и как это сделать.

Все закончилось решением полностью переписать большую часть пользовательского кода, чтобы правильно воспользоваться возможностями Spring Security и, таким образом, упростить будущее расширение приложения для удовлетворения их потребностей в реализации безопасности. Мы также обнаружили проблемы, не связанные с Spring Security, но это уже другая история.

Вот несколько уроков, которые можно извлечь из моего рассказа:

- фреймворк, особенно тот, который широко используется в приложениях, разработан большим коллективом умных людей, и трудно поверить, что он может быть реализован плохо, потому что разработчики не умеют программировать. Всегда начинайте анализ с себя, прежде чем делать вывод, что проблема связана с фреймворком;
- принимая решение об использовании фреймворка, убедитесь, что вы понимаете по крайней мере его основы;
- будьте аккуратны с ресурсами, которые вы используете для изучения фреймворка. Иногда статьи, которые вы найдете в интернете, показывают, как быстро решить проблему, но не обязательно – как правильно реализовать конструкцию класса;
- используйте несколько источников информации. Чтобы прояснить возможное недопонимание, попробуйте написать код в соответствии с той или иной концепцией, если не уверены в своем выборе;
- если вы решили использовать фреймворк, старайтесь использовать его строго по назначению. Например, предположим, что вы используете Spring Security и заметили, что объем написанного вами пользовательского кода больше, чем объем готовых решений. Вам следует задаться вопросом, почему это происходит.

Когда мы опираемся на функции, реализованные фреймворком, мы получаем несколько преимуществ. Мы знаем, что они проверены многими людьми и в них редко встречаются обновления с уязвимостями. Кроме того, хороший фреймворк основан на абстракциях, которые помогают вам создавать легко поддерживаемые приложения. Помните, что когда вы пишете собственные реализации, возрастает риск не заметить уязвимость.

6.2 Использование SecurityContext

В этом разделе мы обсудим контекст безопасности. Мы проанализируем, как он работает, как получить доступ к данным и как приложение управляет ими в различных сценариях, связанных с потоками. После прочтения этого раздела вы будете знать, как настраивать контекст безопасности для различных ситуаций. Вы сможете использовать сведения об аутентифицированном пользователе, сохраненные контекстом безопасности, при настройке авторизации в главах 7 и 8.

Вероятно, вам понадобятся сведения об аутентифицированном субъекте после процесса аутентификации. Допустим, для дальнейшей работы приложению необходимо знать имя или полномочия текущего аутентифицированного пользователя. Эта информация остается доступной после завершения процесса аутентификации? После того как `AuthenticationManager` успешно завершает процесс аутентификации, он сохраняет экземпляр `Authentication` для остальной части запроса (рис. 6.7). Экземпляр, хранящий объект аутентификации, называется *контекстом безопасности* (security context).

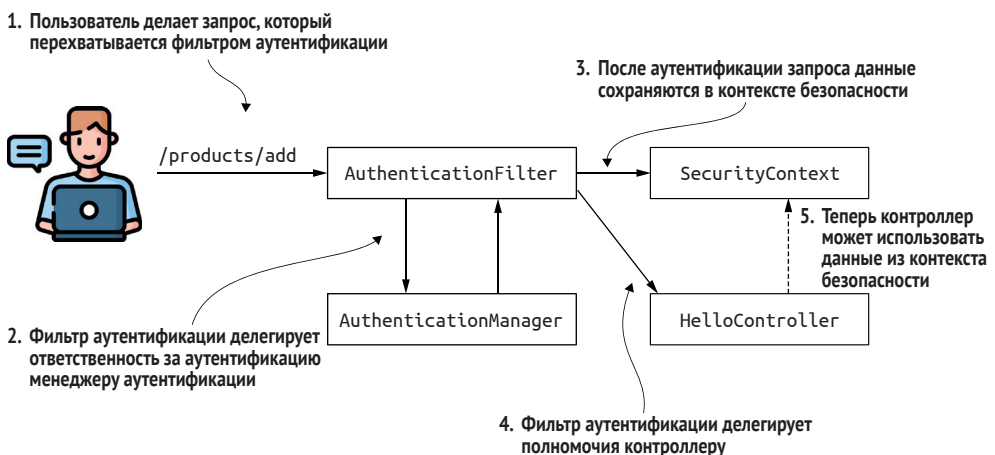


Рис. 6.7 После успешной аутентификации фильтр сохраняет сведения об аутентифицированной сущности в контексте безопасности. Оттуда контроллер, реализующий действие, сопоставленное с запросом, может получить доступ к этим сведениям при необходимости

Контекст безопасности Spring Security описывается интерфейсом `SecurityContext`. Его определение показано в листинге 6.6.

Листинг 6.6 Интерфейс `SecurityContext`

```
public interface SecurityContext extends Serializable {  
  
    Authentication getAuthentication();  
    void setAuthentication(Authentication authentication);  
}
```

Как следует из определения контракта, основная обязанность `SecurityContext` – хранить объект `Authentication`. Но как управляется сам `SecurityContext`? Spring Security предлагает три стратегии управления `SecurityContext` с объектом в роли менеджера. Он называется `SecurityContextHolder`:

- `MODE_THREADLOCAL` – позволяет каждому потоку хранить свои собственные сведения в контексте безопасности. В веб-приложении с одним потоком на запрос это распространенный подход, поскольку каждый запрос имеет отдельный поток;
- `MODE_INHERITABLETHREADLOCAL` – работает аналогично предыдущей стратегии, но также указывает Spring Security копировать контекст безопасности в следующий поток в случае асинхронного метода. Таким образом, можно сказать, что новый поток, запускающий метод `@Async`, наследует контекст безопасности. Аннотация `@Async` используется с методами, чтобы указать Spring вызвать аннотированный метод в отдельном потоке;
- `MODE_GLOBAL` – заставляет все потоки приложения видеть один и тот же экземпляр контекста безопасности.

В дополнение к этим трем стратегиям управления контекстом безопасности, предоставляемым Spring Security, в этом разделе также будет показано, что происходит, когда вы определяете собственные потоки, которые неизвестны Spring. Как вы узнаете, в этих случаях вам необходимо явно копировать детали из контекста безопасности в новый поток. Spring Security не может автоматически управлять объектами, которые не находятся в контексте Spring, но предлагает несколько отличных служебных классов.

6.2.1 *Использование стратегии хранения для контекста безопасности*

Первая стратегия управления контекстом безопасности – это `MODE_THREADLOCAL`, и она также является стратегией по умолчанию для управления контекстом безопасности, используемым Spring Secu-

urity. Согласно этой стратегии Spring Security использует ThreadLocal для управления контекстом. ThreadLocal – это реализация, предоставляемая JDK. Она работает как коллекция данных, но гарантирует, что каждый поток приложения может видеть только данные, хранящиеся в его выделенной части коллекции. Таким образом, каждый запрос имеет доступ к своему контексту безопасности. Ни один поток не будет иметь доступа к ThreadLocal другого потока. Это означает, что в веб-приложении каждый запрос может видеть только свой собственный контекст безопасности. Обычно эту стратегию применяют во внутренних (backend) веб-приложениях.

Эта функциональность схематически показана на рис. 6.8. Каждый запрос (А, В и С) имеет свой собственный выделенный поток (Т1, Т2 и Т3), поэтому каждый запрос видит только данные, хранящиеся в его собственном контексте безопасности. Однако это также означает, что если создается новый поток (например, когда вызывается асинхронный метод), новый поток также будет иметь свой собственный контекст безопасности. Данные из родительского потока (исходного потока запроса) не копируются в контекст безопасности нового потока.

ПРИМЕЧАНИЕ Здесь мы говорим только об архитектуре традиционного приложения сервлета, в которой каждому запросу назначен свой собственный поток. Она не применима к реактивным приложениям. Мы подробно обсудим безопасность реактивных приложений в главе 17.

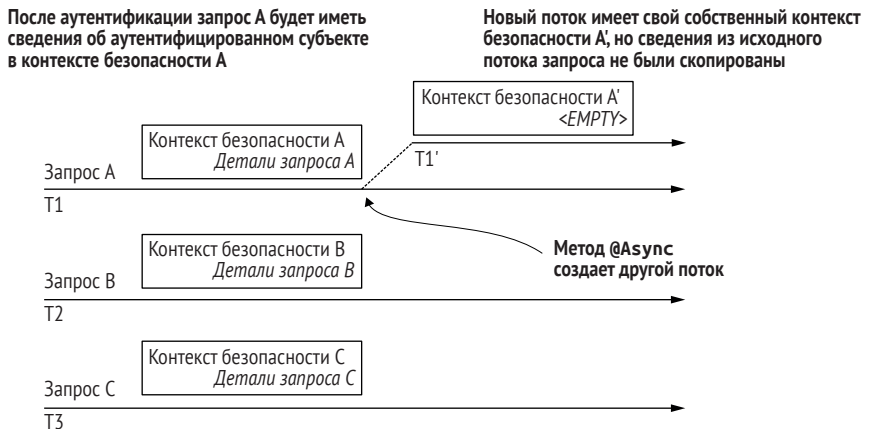


Рис. 6.8 Каждый запрос имеет свой собственный поток, представленный стрелкой. Каждый поток имеет доступ только к собственным деталям контекста безопасности. Когда создается новый поток (например, методом @Async), данные из родительского потока не копируются

Будучи стратегией по умолчанию для управления контекстом безопасности, этот процесс не требует явной настройки. Просто запросите контекст безопасности у хранителя с помощью статического метода `getContext()` в любое время после завершения процесса аутентификации. В листинге 6.7 показан пример получения контекста безопасности в одной из конечных точек приложения. Из контекста безопасности вы можете дополнительно получить объект `Authentication`, который хранит сведения об аутентифицированном субъекте. Примеры, обсуждаемые в этом разделе, можно найти в проекте `ssia-ch6-ex2`.

Листинг 6.7 Получение `SecurityContext` из `SecurityContextHolder`

```
@GetMapping("/hello")
public String hello() {
    SecurityContext context = SecurityContextHolder.getContext();
    Authentication a = context.getAuthentication();

    return "Hello, " + a.getName() + "!";
}
```

Получать аутентификацию из контекста еще удобнее на уровне конечной точки, поскольку Spring знает, что нужно вставить ее непосредственно в параметры метода. Вам не нужно каждый раз явно ссылаться на класс `SecurityContextHolder`. Этот подход представлен в листинге 6.8.

Листинг 6.8 Spring внедряет значение `Authentication` в параметр метода

```
@GetMapping("/hello")
public String hello(Authentication a) {
    return "Hello, " + a.getName() + "!";
}
```

❶ Spring Boot внедряет текущую аутентификацию в параметр метода.

При вызове конечной точки с правильным пользователем тело ответа содержит имя пользователя. Например:

```
curl -u user:99ff79e3-8ca0-401c-a396-0a8625ab3bad http://localhost:8080/hello
Hello, user!
```

6.2.2 Использование стратегии хранения для асинхронных вызовов

Вам не составит труда придерживаться стратегии по умолчанию для управления контекстом безопасности. Во многих случаях это единственное, что вам нужно. `MODE_THREADLOCAL` дает возможность

изолировать контекст безопасности для каждого потока, что делает контекст безопасности более естественным для понимания и управления. Однако бывают случаи, когда эта стратегия неприменима.

Ситуация усложняется, если нам приходится иметь дело с несколькими потоками на запрос. Посмотрите, что произойдет, если вы сделаете конечную точку асинхронной. Поток, который выполняет метод, больше не является тем же потоком, который обслуживает запрос. Эту ситуацию демонстрирует пример кода в листинге 6.9.

Листинг 6.9 Метод @Async, обслуживаемый другим потоком

```
@GetMapping("/bye")
@Async
public void goodbye() {
    SecurityContext context = SecurityContextHolder.getContext();
    String username = context.getAuthentication().getName();

    // какие-то действия с именем пользователя
}
```

❶ Метод с аннотацией @Async выполняется в отдельном потоке.

Чтобы включить функциональность аннотации @Async, я также создал класс конфигурации и аннотировал его как @EnableAsync:

```
@Configuration
@EnableAsync
public class ProjectConfig {

}
```

ПРИМЕЧАНИЕ Иногда встречаются примеры кода, в которых аннотации конфигурации размещены над главным классом. Например, вы можете заметить, что в некоторых примерах аннотация @EnableAsync расположена непосредственно над главным классом. Этот подход технически корректен, поскольку мы аннотируем главный класс приложения Spring Boot с помощью аннотации @SpringBootApplication, которая включает характеристику @Configuration. Однако в реальных приложениях принято разделять ответственности и никогда не использовать главный класс в качестве класса конфигурации. Чтобы сделать примеры в этой книге максимально понятными, я предпочитаю размещать эти аннотации над классом @Configuration, подобно тому, как это делается на практике.

Если вы попробуете код в его текущем виде, он выдаст исключение NullPointerException на строке, которая получает имя из аутентификации, а именно:

```
String username = context.getAuthentication().getName()
```


Это происходит потому, что метод теперь выполняется в другом потоке, который не наследует контекст безопасности. По этой причине объект `Authorization` имеет значение `null` и в контексте представленного кода вызывает исключение `NullPointerException`. В этом случае вы можете решить проблему, используя стратегию `MODE_INHERITABLETHREADLOCAL`. Ее можно выбрать либо путем вызова метода `SecurityContextHolder.setStrategyName()`, либо с помощью системного свойства `spring.security.strategy`. Если выбрана эта стратегия, фреймворк знает, что нужно скопировать сведения об исходном потоке запроса в недавно созданный поток асинхронного метода (рис. 6.9).

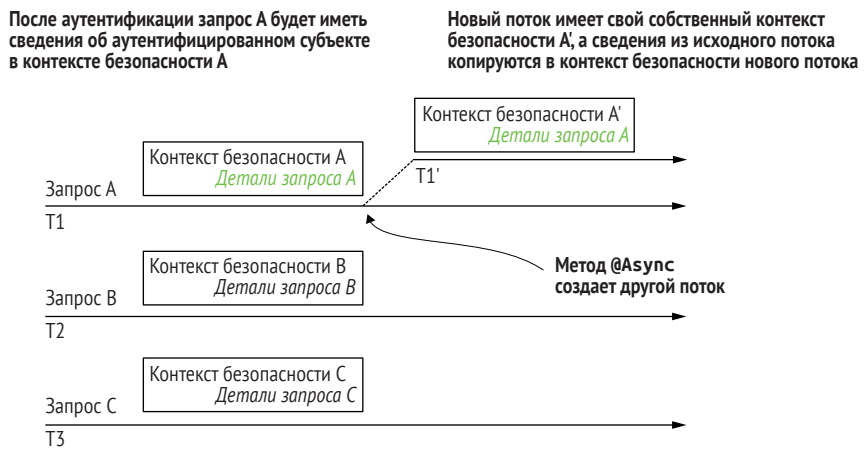


Рис. 6.9 При использовании стратегии `MODE_INHERITABLETHREADLOCAL` фреймворк копирует сведения о контексте безопасности исходного потока запроса в контекст безопасности нового потока

В листинге 6.10 показан способ установки стратегии управления контекстом безопасности путем вызова метода `setStrategyName()`.

Листинг 6.10 Использование `InitializingBean` для установки режима `SecurityContextHolder`

```

@Configuration
@EnableAsync
public class ProjectConfig {

    @Bean
    public InitializingBean initializingBean() {
        return () -> SecurityContextHolder.setStrategyName(
            SecurityContextHolder.MODE_INHERITABLETHREADLOCAL);
    }
}
  
```

После вызова конечной точки вы увидите, что контекст безопасности правильно распространяется в следующий поток Spring. Кроме того, Authentication больше не null.

ПРИМЕЧАНИЕ Этот прием работает только тогда, когда фреймворк сам создает поток (например, в случае метода `@Async`). Если поток создан вашим кодом, вы столкнетесь с той же проблемой даже со стратегией `MODE_INHERITABLETHREADLOCAL`. Фреймворк просто не будет знать о потоке, который создает ваш код. Мы обсудим решение этой проблемы в разделах 6.2.4 и 6.2.5.

6.2.3 Использование стратегии хранения в автономных приложениях

Если вам нужен контекст безопасности, общий для всех потоков приложения, измените стратегию на `MODE_GLOBAL` (рис. 6.10). Вы не будете использовать эту стратегию для веб-сервера, поскольку она не соответствует общей картине приложения. Веб-приложение бэкенда независимо управляет получаемыми запросами, поэтому действительно имеет смысл иметь отдельный контекст безопасности для каждого запроса вместо одного контекста для всех. Тем не менее общий контекст может быть правильным выбором для автономного приложения.

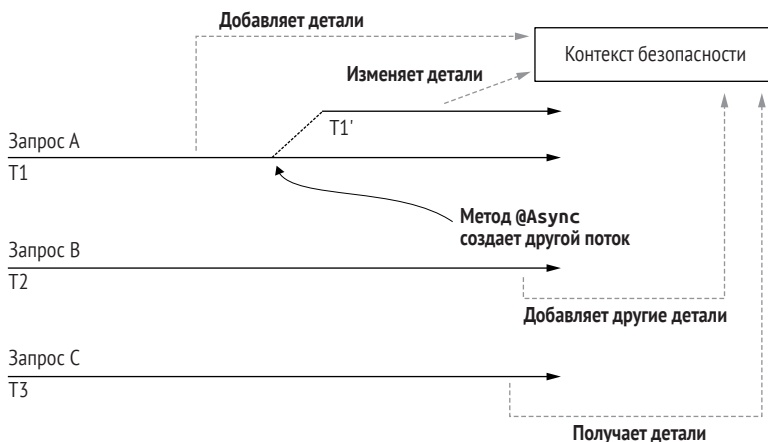


Рис. 6.10 При использовании стратегии управления контекстом безопасности `MODE_GLOBAL` все потоки получают доступ к одному и тому же контексту. Это подразумевает, что все они имеют доступ к одним и тем же данным и могут изменять эту информацию. Из-за этого может возникнуть ситуация «гонки», и вам необходимо позаботиться о синхронизации

Как показано в следующем фрагменте кода, вы можете изменить стратегию по аналогии с `MODE_INHERITABLETHREADLOCAL` с помощью

метода `SecurityContextHolder.setStrategyName()` или системного свойства `spring.security.strategy`:

```
@Bean
public InitializingBean initializingBean() {
    return () -> SecurityContextHolder.setStrategyName(
        SecurityContextHolder.MODE_GLOBAL);
}
```

Также следует помнить, что `SecurityContext` не является потоко-безопасным. Поэтому при использовании стратегии, в которой все потоки приложения могут получать доступ к объекту `SecurityContext`, вам необходимо позаботиться о безопасном параллельном доступе.

6.2.4 *Распространение контекста безопасности с помощью `DelegatingSecurityContextRunnable`*

Вы узнали, что контекстом безопасности можно оперировать с помощью трех стандартных режимов, предоставляемых Spring Security: `MODE_THREADLOCAL`, `MODE_INHERITEDTHREADLOCAL` и `MODE_GLOBAL`. По умолчанию фреймворк обеспечивает только предоставление контекста безопасности для потока запроса, и этот контекст безопасности доступен лишь для этого потока. Фреймворк не заботится о вновь созданных потоках (например, в случае асинхронного метода). Кроме того, вы узнали, что для этой ситуации необходимо явно задать другой режим для управления контекстом безопасности. Но мы все еще не раскрыли неопределенность: что происходит, когда ваш код запускает новые потоки, о которых не знает фреймворк? Иногда такие потоки называют *самоуправляемыми* (self-managed), потому что ими управляем мы, а не фреймворк. В этом разделе мы обсудим служебные инструменты Spring Security, которые помогают нам распространять контекст безопасности на вновь созданные потоки. Никакая конкретная стратегия `SecurityContextHolder` не предлагает вам готового решения для самоуправляемых потоков. В этом случае вам нужно самостоятельно позаботиться о распространении контекста безопасности. Одним из решений является использование `DelegatingSecurityContextRunnable` для декорирования задач, которые вы хотите выполнить в отдельном потоке. `DelegatingSecurityContextRunnable` расширяет `Runnable`. Вы можете использовать его после выполнения задачи, когда не ожидается возвращаемое значение. На случай возвращаемого значения существует альтернативный класс `DelegatingSecurityContextCallable<T>`, расширяющий `Callable<T>`. Оба класса представляют задачи, выполняемые асинхронно, как и любые другие `Runnable` или `Callable`. Более того, они

обязательно копируют текущий контекст безопасности для потока, который выполняет задачу. Как показано на рис. 6.11, эти объекты декорируют исходные задачи и копируют контекст безопасности в новые потоки.

DelegatingSecurityContextCallable декорирует задачу **Callable**, которая будет выполнена в отдельном потоке

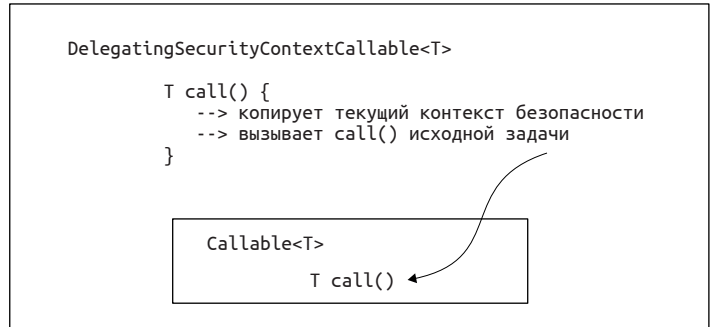


Рис. 6.11 **DelegatingSecurityContextCallable** разработан как декоратор объекта **Callable**. При создании такого объекта вы предоставляете вызываемую задачу, которую приложение выполняет асинхронно. **DelegatingSecurityContextCallable** копирует данные из контекста безопасности в новый поток, а затем выполняет задачу

Листинг 6.11 демонстрирует использование **DelegatingSecurityContextCallable**. Начнем с определения простого метода конечной точки, который объявляет объект **Callable**. Задача **Callable** возвращает имя пользователя из текущего контекста безопасности.

Листинг 6.11 Определение объекта **Callable** и выполнение его как задачи в отдельном потоке

```

@GetMapping("/ciao")
public String ciao() throws Exception {
    Callable<String> task = () -> {
        SecurityContext context = SecurityContextHolder.getContext();
        return context.getAuthentication().getName();
    };

    // Часть кода не показана
}
    
```

Продолжим пример, отправляя задачу в **ExecutorService**. Результат выполнения извлекается и возвращается конечной точкой как тело ответа.

Листинг 6.12 Определение ExecutorService и отправка задачи

```

@GetMapping("/ciao")
public String ciao() throws Exception {
    Callable<String> task = () -> {
        SecurityContext context = SecurityContextHolder.getContext();
        return context.getAuthentication().getName();
    };

    ExecutorService e = Executors.newCachedThreadPool();
    try {
        return "Ciao, " + e.submit(task).get() + "!";
    } finally {
        e.shutdown();
    }
}

```

Если вы запустите приложение как есть, вы не получите ничего, кроме `NullPointerException`. Внутри вновь созданного потока для запуска вызываемой задачи аутентификация больше не существует, а контекст безопасности пуст. Чтобы решить эту проблему, мы декорируем задачу с помощью `DelegatingSecurityContextCallable`, который предоставляет текущий контекст новому потоку, как показано в листинге 6.13.

Листинг 6.13 Запуск задачи, декорированной DelegatingSecurityContextCallable

```

@GetMapping("/ciao")
public String ciao() throws Exception {
    Callable<String> task = () -> {
        SecurityContext context = SecurityContextHolder.getContext();
        return context.getAuthentication().getName();
    };

    ExecutorService e = Executors.newCachedThreadPool();
    try {
        var contextTask = new DelegatingSecurityContextCallable<>(task);
        return "Ciao, " + e.submit(contextTask).get() + "!";
    } finally {
        e.shutdown();
    }
}

```

Если вызвать конечную точку сейчас, вы увидите, что Spring распространил контекст безопасности в поток, в котором выполняются задачи:

```
curl -u user:2eb3f2e8-debd-420c-9680-48159b2ff905 http://localhost:8080/ciao
```

Тело ответа для этого вызова:

Ciao, user!

6.2.5 Распространение контекста безопасности с помощью DelegatingSecurityContextExecutorService

При работе с потоками, которые наш код запускает, не сообщая фреймворку, мы должны позаботиться о передаче деталей из контекста безопасности в следующий поток. В разделе 6.2.4 мы применили технику копирования данных из контекста безопасности, используя саму задачу. Spring Security предоставляет несколько замечательных служебных классов, таких как `DelegatingSecurityContextRunnable` и `DelegatingSecurityContextCallable`. Эти классы декорируют задачи, которые вы выполняете асинхронно, а также берут на себя ответственность за копирование данных из контекста безопасности, чтобы ваша реализация могла получить к ним доступ из недавно созданного потока. Однако у нас есть второй вариант для работы с распространением контекста безопасности в новый поток, и это управление распространением из пула потоков, а не из самой задачи. В этом разделе вы узнаете, как применять эту технику, используя другие, столь же удобные служебные классы, предоставляемые Spring Security.

Альтернативой декорированию задач является использование определенного типа `Executor`. В следующем примере вы можете заметить, что задача остается простым объектом `Callable<T>`, но поток по-прежнему управляет контекстом безопасности. Распространение контекста безопасности происходит, потому что реализация, называемая `DelegatingSecurityContextExecutorService`, декорирует `ExecutorService`. `DelegatingSecurityContextExecutorService` также заботится о распространении контекста безопасности, как показано на рис. 6.12.

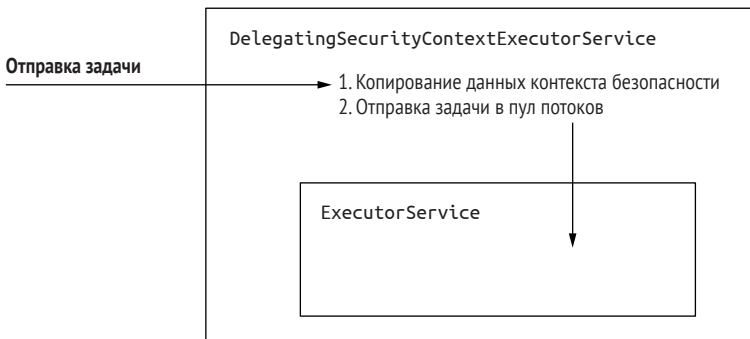


Рис. 6.12 `DelegatingSecurityContextExecutorService` декорирует `ExecutorService` и распространяет детали контекста безопасности в следующий поток перед отправкой задачи

Код в листинге 6.14 показывает, как использовать `DelegatingSecurityContextExecutorService` для декорирования `ExecutorService`, чтобы при отправке задачи он заботился о распространении деталей контекста безопасности.

Листинг 6.14 Распространение `SecurityContext`

```
@GetMapping("/hola")
public String hola() throws Exception {
    Callable<String> task = () -> {
        SecurityContext context = SecurityContextHolder.getContext();
        return context.getAuthentication().getName();
    };

    ExecutorService e = Executors.newCachedThreadPool();
    e = new DelegatingSecurityContextExecutorService(e);
    try {
        return "Hola, " + e.submit(task).get() + "!";
    } finally {
        e.shutdown();
    }
}
```

Вызовите конечную точку, чтобы убедиться, что `DelegatingSecurityContextExecutorService` правильно делегировал контекст безопасности:

```
curl -u user:5a5124cc-060d-40b1-8aad-753d3da28dca http://localhost:8080/hola
```

Тело ответа для этого вызова:

```
Hola, user!
```

ПРИМЕЧАНИЕ Из классов, связанных с поддержкой параллелизма для контекста безопасности, в первую очередь следует запомнить те, что представлены в табл. 6.1.

Spring предлагает различные реализации служебных классов, которые можно использовать в вашем приложении для управления контекстом безопасности при создании собственных потоков. В разделе 6.2.4 мы реализовали `DelegatingSecurityContextCallable`. В этом разделе мы используем `DelegatingSecurityContextExecutorService`. Если вам нужно реализовать распространение контекста безопасности для запланированной задачи, то наверняка вы будете рады услышать, что Spring Security также предлагает декоратор с именем `DelegatingSecurityContextScheduledExecutorService`. Он похож на `DelegatingSecurityContextExecutorService`, представлен-

ный в этом разделе, с той разницей, что он декорирует `ScheduledExecutorService`, позволяя вам работать с запланированными задачами.

Кроме того, для большей гибкости Spring Security поддерживает более абстрактную версию декоратора с именем `DelegatingSecurityContextExecutor`. Этот класс напрямую декорирует `Executor`, который является самым абстрактным контрактом этой иерархии пулов потоков. Воспользуйтесь им при разработке своего приложения, если нужно иметь возможность заменить реализацию пула потоков любым из вариантов, предоставляемых языком программирования.

Таблица 6.1. Объекты, ответственные за делегирование контекста безопасности отдельному потоку

Класс	Описание
<code>DelegatingSecurityContextExecutor</code>	Реализует интерфейс <code>Executor</code> и предназначен для декорирования объекта <code>Executor</code> с возможностью пересылки контекста безопасности потокам, созданным его пулом
<code>DelegatingSecurityContext-ExecutorService</code>	Реализует интерфейс <code>ExecutorService</code> и предназначен для декорирования объекта <code>ExecutorService</code> с возможностью пересылки контекста безопасности потокам, созданным его пулом
<code>DelegatingSecurityContext-ScheduledExecutorService</code>	Реализует интерфейс <code>ScheduledExecutorService</code> и предназначен для декорирования объекта <code>ScheduledExecutorService</code> с возможностью пересылки контекста безопасности потокам, созданным его пулом
<code>DelegatingSecurityContextRunnable</code>	Реализует интерфейс <code>Runnable</code> и представляет задачу, которая выполняется в другом потоке без возврата ответа. В отличие от обычного <code>Runnable</code> он также может распространять контекст безопасности для использования в новом потоке
<code>DelegatingSecurityContextCallable</code>	Реализует интерфейс <code>Callable</code> и представляет задачу, которая выполняется в другом потоке и в конечном итоге вернет ответ. В отличие от обычного <code>Callable</code> он также может распространять контекст безопасности для использования в новом потоке

6.3 Подробнее о методе HTTP Basic и аутентификации на основе форм

До сих пор мы использовали в качестве метода аутентификации только HTTP Basic, но в этой книге вы узнаете, что есть и другие варианты. Метод аутентификации HTTP Basic прост, что делает его отличным выбором для примеров и демонстрационных целей или тестирования идеи приложения. Но по той же причине он часто не подходит для применения в производстве.

В этом разделе вы узнаете больше о конфигурациях, основанных на HTTP Basic. Кроме того, мы поговорим о новом методе аутенти-

фикации, называемом `formLogin`. В оставшейся части этой книги мы обсудим другие методы аутентификации, которые хорошо соответствуют различным видам архитектур. Мы сравним их, чтобы вы освоили как лучшие приемы разработки, так и нежелательные способы аутентификации.

6.3.1 Использование и настройка HTTP Basic

Вы уже знаете, что HTTP Basic – это метод аутентификации по умолчанию, и мы видели, как он работает в различных примерах, в главе 3. В этом разделе мы добавим больше подробностей относительно настройки данного метода аутентификации.

Конфигурация по умолчанию, которую предоставляет аутентификация HTTP Basic, великолепно подходит для теоретических сценариев. Однако в более сложном приложении возникает потребность в настройке некоторых параметров. Допустим, вам нужно реализовать определенную логику для случая, когда процесс аутентификации завершается неудачей. Вам даже может потребоваться задать некоторые значения в ответе, отправляемом клиенту в этом случае. Давайте рассмотрим эти случаи на практических примерах. Я хочу еще раз обратить ваше внимание на явное определение метода, как показано в листинге 6.15. Вы можете найти этот пример в проекте `ssia-ch6-ex3`.

Листинг 6.15 Настройка метода аутентификации HTTP Basic

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

Метод `httpBasic()` экземпляра `HttpSecurity` можно вызвать с параметром типа `Customizer`. Этот параметр позволяет настроить некоторые конфигурации, связанные с методом аутентификации, например *имя области действия* (`realm name`), как показано в листинге 6.16. Вы можете рассматривать область действия как защищенное пространство, которое использует определенный метод аутентификации. Полное описание см. в RFC 2617 по адресу <https://tools.ietf.org/html/rfc2617>.

Листинг 6.16 Настройка имени области для ответа на неудачные аутентификации

```
@Bean
public SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.httpBasic(c -> {
        c.realmName("OTHER");
        c.authenticationEntryPoint(new CustomEntryPoint());
    });

    http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

    return http.build();
}
```

Листинг 6.16 демонстрирует пример изменения имени области. Используемое в нем лямбда-выражение фактически является объектом типа `Customizer<HttpBasicConfigurer<HttpSecurity>>`. Параметр типа `HttpBasicConfigurer<HttpSecurity>` позволяет нам вызывать метод `realmName()` для переименования области. Вы можете вызывать `cURL` с флагом `-v`, чтобы получить развернутый HTTP-ответ, в котором имя области действительно изменено. Однако обратите внимание, что вы найдете заголовок `WWW-Authenticate` в ответе только тогда, когда статус ответа HTTP равен 401 Unauthorized, а не тогда, когда он равен 200 OK. Вот вызов `cURL`:

```
curl -v http://localhost:8080/hello
```

и ответ на вызов:

```
/
...
< WWW-Authenticate: Basic realm="OTHER"
...
```

Кроме того, с помощью `Customizer` мы можем настроить ответ для неудачной аутентификации. Вам нужно сделать это, если клиент вашей системы ожидает какой-то конкретики в ответе в случае неудачной аутентификации. Возможно, вам потребуется добавить или удалить один или несколько заголовков. Или вы добавите логику, которая фильтрует тело сообщения, чтобы убедиться, что приложение не раскрывает клиенту никаких конфиденциальных данных.

ПРИМЕЧАНИЕ Всегда проявляйте осторожность в отношении данных, которые вы раскрываете за пределами системы. Одна из самых распространенных ошибок (которая также

входит в десятку самых уязвимых мест OWASP; см. <https://owasp.org/www-project-top-ten/>) – раскрытие конфиденциальных данных. Работа с данными, которые приложение отправляет клиенту при неудачной аутентификации, всегда является точкой риска раскрытия конфиденциальной информации.

Для настройки ответа на неудачную аутентификацию можно реализовать `AuthenticationEntryPoint`. Его метод `begin()` получает `HttpServletRequest`, `HttpServletResponse` и `AuthenticationException`, которые привели к сбою аутентификации. В листинге 6.17 показан способ реализации `AuthenticationEntryPoint`, который добавляет заголовок к ответу и устанавливает статус HTTP на 401 `Unauthorized`.

ПРИМЕЧАНИЕ Немного странно, что имя интерфейса `AuthenticationEntryPoint` не отражает его использование при *неудачной* аутентификации. В архитектуре Spring Security он используется напрямую компонентом `ExceptionHandlerTranslationManager`, который обрабатывает любые исключения `AccessDeniedException` и `AuthenticationException`, возникающие в цепочке фильтров. Вы можете рассматривать `ExceptionHandlerTranslationManager` как мост между исключениями Java и ответами HTTP.

Листинг 6.17 Реализация `AuthenticationEntryPoint`

```
public class CustomEntryPoint
    implements AuthenticationEntryPoint {

    @Override
    public void commence(
        HttpServletRequest httpServletRequest,
        HttpServletResponse httpServletResponse,
        AuthenticationException e)
        throws IOException, ServletException {

        httpServletResponse
            .addHeader("message", "Люк, я твой отец!");
        httpServletResponse
            .sendError(HttpStatus.UNAUTHORIZED.value());
    }
}
```

Затем вы можете зарегистрировать `CustomEntryPoint` с методом HTTP Basic в классе конфигурации. В листинге 6.18 представлен класс конфигурации для пользовательской точки входа.

Листинг 6.18 Настройка пользовательской точки входа AuthenticationEntryPoint

```
@Bean
public SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.httpBasic(c -> {
        c.realmName("OTHER");
        c.authenticationEntryPoint(new CustomEntryPoint());
    });

    http.authorizeHttpRequests().anyRequest().authenticated();

    return http.build();
}
```

Если вы теперь сделаете вызов конечной точки с неудачной аутентификацией, в ответе должен появиться добавленный вами заголовок:

```
curl -v http://localhost:8080/hello
```

Ответ на вызов:

```
...
< HTTP/1.1 401
< Set-Cookie: JSESSIONID=459BAFA7E0E6246A463AD19B07569C7B; Path=/; HttpOnly
< message: Люк, я твой отец!
...
```

6.3.2 Аутентификация на основе форм

При разработке веб-приложения вы, вероятно, захотите предоставить пользователям удобную форму входа, в которой пользователи будут вводить свои учетные данные. Кроме того, иногда нужно, чтобы аутентифицированные пользователи могли просматривать веб-страницы после входа в систему и выходить из нее. Для небольшого веб-приложения вы можете воспользоваться методом входа на основе форм. Далее вы научитесь применять и настраивать этот метод аутентификации. Мы напишем небольшое веб-приложение, использующее аутентификацию на основе форм. На рис. 6.13 схематически показан поток, который мы реализуем. Примеры в этом разделе являются частью проекта ssia-ch6-ex4.

ПРИМЕЧАНИЕ Я связываю этот метод с небольшим веб-приложением, поскольку таким образом мы используем сеанс на стороне сервера для управления контекстом безопасности. Для более крупных приложений, требующих горизонтальной

масштабируемости, использование сеанса на стороне сервера для управления контекстом безопасности нежелательно. Мы обсудим эти аспекты более подробно в главах 12–15 при работе с OAuth 2.

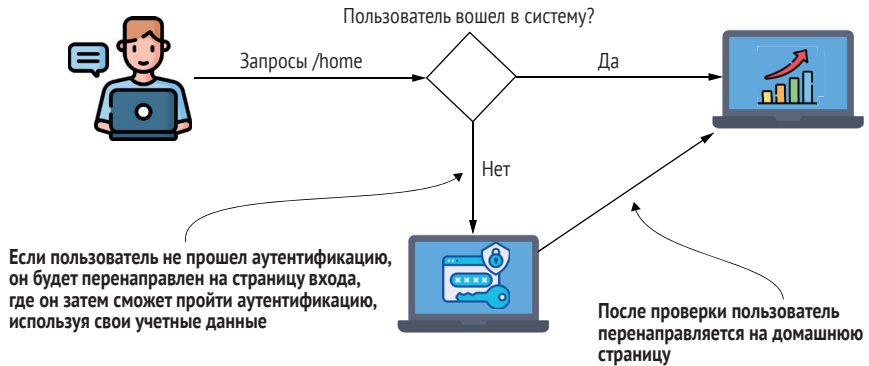


Рис. 6.13 Механизм входа на основе форм. Пользователя, который еще не прошел аутентификацию, направляют на страницу с формой входа, где он вводит свои учетные данные. После того как приложение выполнит проверку, пользователь будет перенаправлен на главную страницу приложения

Чтобы задать метод аутентификации на основе форм с использованием объекта `HttpSecurity` компонента `SecurityFilterChain` вместо `httpBasic()`, вызовите метод `formLogin()` параметра `HttpSecurity`. Это изменение показано в листинге 6.19.

Листинг 6.19 Изменение метода аутентификации на вход на основе форм

```

@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.formLogin(Customizer.withDefaults());

        http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

        return http.build();
    }
}
  
```

Даже при такой минимальной конфигурации Spring Security уже настроил форму входа, а также страницу выхода для вашего проекта.

Запуск приложения и доступ к нему с помощью браузера должны перенаправить вас на страницу входа (рис. 6.14).

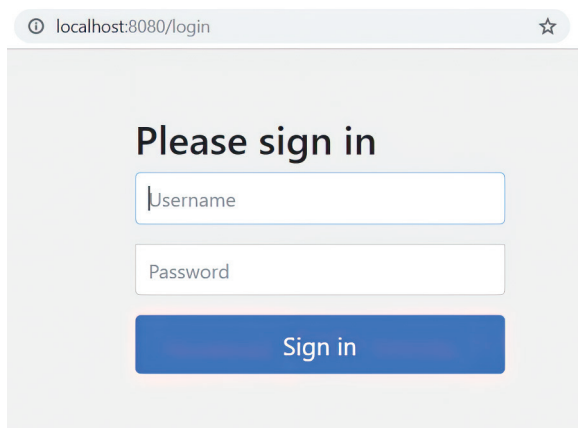


Рис. 6.14 Страница входа по умолчанию, автоматически настроенная Spring Security при использовании метода `formLogin()`

Если вы не регистрируете свой `UserDetailsService`, то можете войти, используя предоставленные по умолчанию учетные данные. Это, как мы узнали в главе 2, имя пользователя `user` и пароль `UUID`, который выводится в консоли при запуске приложения. Поскольку не определена другая страница, после успешного входа вы перенаправляетесь на страницу ошибки по умолчанию. Приложение использует ту же архитектуру аутентификации, с которой мы столкнулись в предыдущих примерах. Поэтому в соответствии с рис. 6.13 вам необходимо реализовать контроллер для домашней страницы приложения. Разница в том, что вместо простого ответа в формате JSON нам нужно, чтобы конечная точка возвращала код HTML, который может быть интерпретирован браузером как наша веб-страница. Поэтому мы будем придерживаться потока Spring MVC и визуализируем представление из файла после выполнения действия, определенного в контроллере. На рис. 6.15 представлен поток Spring MVC для визуализации домашней страницы приложения.

Чтобы добавить простую страницу в приложение, сначала необходимо создать HTML-файл в папке `resources/static` проекта. Я назвал этот файл `home.html`. Внутри него введите текст, который вы позже увидите в браузере. Вы можете просто добавить заголовок (например, `<h1>Добро пожаловать</h1>`). После создания HTML-страницы контроллер должен определить сопоставление пути с представлением. В листинге 6.20 показано определение метода действия для страницы `home.html` в классе контроллера.

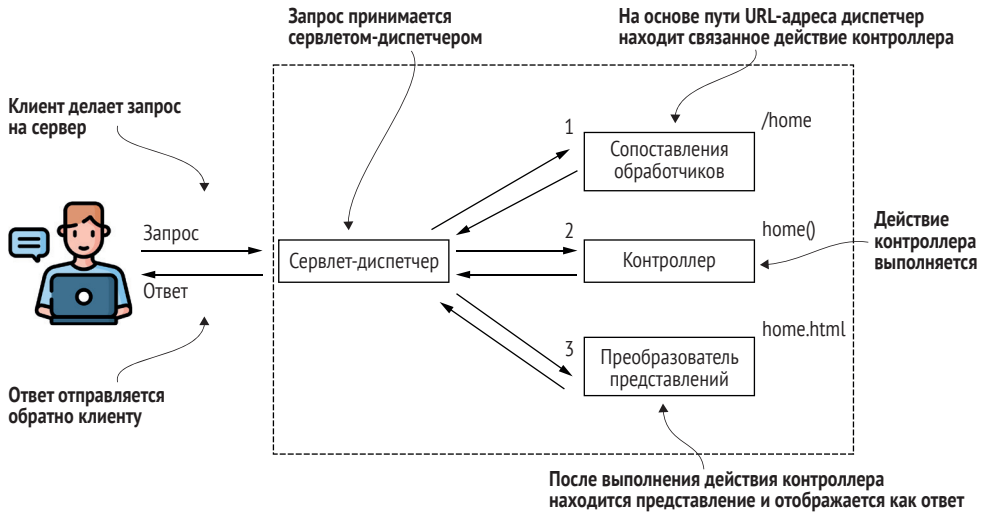


Рис. 6.15 Простое представление потока Spring MVC. Диспетчер находит действие контроллера, связанное с заданным путем (в данном случае /home). После выполнения действия контроллера отображается представление, а ответ отправляется клиенту

Листинг 6.20 Определение метода действия контроллера для страницы home.html

```
@Controller
public class HelloController {

    @GetMapping("/home")
    public String home() {
        return "home.html";
    }
}
```

Помните, что это не `@RestController`, а простой `@Controller`. Из-за этого Spring не отправляет значение, возвращаемое методом в HTTP-ответе. Вместо этого он находит и отображает представление с именем `home.html`.

При попытке доступа к пути `/home` вас сначала спросят, хотите ли вы войти в систему. После успешного входа вы будете перенаправлены на домашнюю страницу, где появится приветственное сообщение. Теперь вы можете получить доступ к пути `/logout`, который перенаправит вас на страницу выхода из системы (рис. 6.16).

После попытки доступа к пути без входа в систему пользователь автоматически перенаправляется на страницу входа. После успешного входа приложение перенаправляет пользователя обратно на путь, по которому он пытался получить доступ изначально. Если этот путь не существует, приложение отображает страницу ошибки по

умолчанию. Метод `formLogin()` возвращает объект типа `FormLoginConfigurer<HttpSecurity>`, что позволяет нам применить пользовательские настройки. Например, вы можете сделать это, вызвав метод `defaultSuccessUrl()`, как показано в листинге 6.21.

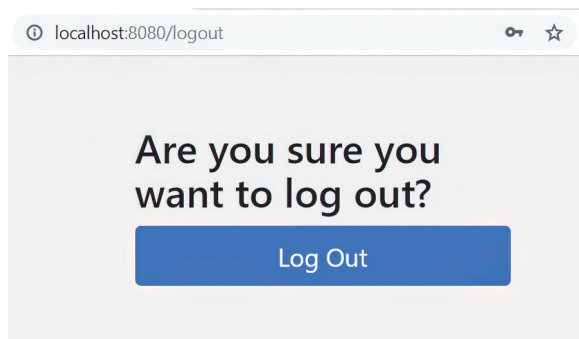


Рис. 6.16 Страница выхода из системы, настроенная Spring Security для метода аутентификации на основе форм

Листинг 6.21 Установка URL-адреса успешного входа по умолчанию для формы входа

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.formLogin(c -> c.defaultSuccessUrl("/home", true));

    http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

    return http.build();
}
```

Если вы хотите еще больше углубиться в кастомизацию, воспользуйтесь интерфейсами `AuthenticationSuccessHandler` и `AuthenticationFailureHandler`, поддерживающими более детальную настройку. Эти интерфейсы позволяют реализовать объект, с помощью которого вы затем применяете логику, выполняемую для аутентификации. Для настройки логики успешной аутентификации определите `AuthenticationSuccessHandler`. Метод `onAuthenticationSuccess()` получает запрос сервлета, ответ сервлета и объект `Authentication` в качестве параметров. В листинге 6.22 вы найдете пример реализации метода `onAuthenticationSuccess()` для выполнения различных перенаправлений в зависимости от предоставленных полномочий вошедшего в систему пользователя.

Листинг 6.22 Реализация AuthenticationSuccessHandler

```

@Component
public class CustomAuthenticationSuccessHandler
    implements AuthenticationSuccessHandler {

    @Override
    public void onAuthenticationSuccess(
        HttpServletRequest httpServletRequest,
        HttpServletResponse httpServletResponse,
        Authentication authentication)
        throws IOException {

        var authorities = authentication.getAuthorities();

        var auth =
            authorities.stream()
                .filter(a -> a.getAuthority().equals("read"))
                .findFirst();           ❶

        if (auth.isPresent()) {       ❷
            httpServletResponse
                .sendRedirect("/home");
        } else {
            httpServletResponse
                .sendRedirect("/error");
        }
    }
}

```

- ❶ Возвращаем пустой необязательный объект, если полномочие "read" не существует.
- ❷ Если полномочие "read" существует, перенаправляем на /home.

На практике иногда клиент ожидает определенный формат ответа в случае неудачной аутентификации. Он может ожидать другой код статуса HTTP, отличный от 401 Unauthorized, или поясняющую информацию в теле ответа. Наиболее типичный случай, который я встречал в приложениях, связан с отправкой идентификатора запроса. Этот идентификатор запроса имеет уникальное значение, используемое для отслеживания запроса среди нескольких систем, и приложение может отправить его в теле ответа в случае неудачной аутентификации. Другая ситуация – когда вы хотите очистить ответ, чтобы убедиться, что приложение не раскрывает конфиденциальные данные за пределами системы. Вы можете определить пользовательскую логику для неудачной аутентификации, просто зарегистрировав событие для дальнейшего расследования.

Если вы хотите настроить логику, которую приложение выполняет при неудачной аутентификации, это можно сделать аналогично удачному исходу, но с реализацией AuthenticationFailureHandler.

Например, если вы хотите добавить определенный заголовок для неудачной аутентификации, воспользуйтесь примером в листинге 6.23. Разумеется, вы можете реализовать здесь любую логику. В случае `AuthenticationFailureHandler` метод `onAuthenticationFailure()` получает запрос, ответ и объект `Authentication`.

Листинг 6.23 Реализация `AuthenticationFailureHandler`

```
@Component
public class CustomAuthenticationFailureHandler
    implements AuthenticationFailureHandler {

    @Override
    public void onAuthenticationFailure(
        HttpServletRequest httpRequest,
        HttpServletResponse httpResponse,
        AuthenticationException e) {

        try {

            httpResponse.setHeader("failed",
                LocalDateTime.now().toString());
            httpResponse.sendRedirect("/error");

        } catch (IOException ex) {
            throw new RuntimeException(ex);
        }
    }
}
```

Чтобы использовать два объекта, вам необходимо зарегистрировать их в методе `securityFilterChain()` объекта `FormLoginConfigurer`, возвращаемого методом `formLogin()`. В листинге 6.24 показано, как это сделать.

Листинг 6.24 Регистрация объектов обработчика в классе конфигурации

```
@Configuration
public class ProjectConfig {

    private final CustomAuthenticationSuccessHandler
authenticationSuccessHandler;
    private final CustomAuthenticationFailureHandler
authenticationFailureHandler;

    // Конструктор не показан

    @Bean
    public UserDetailsService uds() {
        var uds = new InMemoryUserDetailsManager();
    }
}
```

```

uds.createUser(
    User.withDefaultPasswordEncoder()
        .username("john")
        .password("12345")
        .authorities("read")
        .build()
);

uds.createUser(
    User.withDefaultPasswordEncoder()
        .username("bill")
        .password("12345")
        .authorities("write")
        .build()
);

return uds;
}

@Bean
public SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.formLogin(c ->
        c.successHandler(authenticationSuccessHandler)
        .failureHandler(authenticationFailureHandler)
    );

    http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

    return http.build();
}
}

```

На данный момент, если вы попытаетесь получить доступ к пути /home с помощью HTTP Basic с правильным именем пользователя и паролем, вы получите ответ со статусом HTTP 302 Found. Так приложение сообщает вам, что оно пытается выполнить перенаправление. Даже если вы указали правильное имя пользователя и пароль, оно не будет их учитывать и вместо этого попытается отправить вас на форму входа, как запрашивает метод `formLogin`. Однако вы можете изменить конфигурацию для поддержки как HTTP Basic, так и методов входа на основе форм, как в листинге 6.25.

Листинг 6.25 Совместное использование входа на основе форм и HTTP Basic

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

```

```
http.formLogin(c ->
    c.successHandler(authenticationSuccessHandler)
    .failureHandler(authenticationFailureHandler)
);

http.httpBasic(Customizer.withDefaults());

http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

return http.build();
}
```

Доступ к пути /home теперь работает как с входом на основе форм, так и с методами аутентификации HTTP Basic:

```
curl -u user:cdd430f6-8ebc-49a6-9769-b0f3ce571d19 http://
localhost:8080/home
```

Ответ на вызов конечной точки:

```
<h1>Добро пожаловать</h1>
```

Заключение

- `AuthenticationProvider` – это компонент, который позволяет реализовать пользовательскую логику аутентификации.
- При реализации пользовательской логики аутентификации рекомендуется разграничивать ответственности. Для управления пользователями `AuthenticationProvider` делегирует ответственность `UserDetailsService`, а для проверки пароля `AuthenticationProvider` делегирует ответственность `PasswordEncoder`.
- `SecurityContext` хранит сведения об аутентифицированном субъекте после успешной аутентификации.
- Вы можете использовать три стратегии для управления контекстом безопасности: `MODE_THREADLOCAL`, `MODE_INHERITABLETHREADLOCAL` и `MODE_GLOBAL`. Доступ из разных потоков к деталям контекста безопасности работает по-разному, в зависимости от выбранного вами режима.
- Помните, что при использовании локального режима общего потока он применяется только для потоков, управляемых Spring. Фреймворк не будет копировать контекст безопасности для потоков, которые им не управляются.
- Spring Security предлагает вам отличные служебные классы для управления потоками, созданными вашим кодом, о которых фреймворк теперь знает. Чтобы управлять `SecurityContext` для созданных вами потоков, вы можете использовать
 - `DelegatingSecurityContextRunnable`;

- `DelegatingSecurityContextCallable`;
- `DelegatingSecurityContextExecutor`.
- Spring Security автоматически настраивает форму для входа с возможностью выхода из системы с помощью метода аутентификации на основе формы `formLogin()`. Его легко использовать при разработке небольших веб-приложений.
- Метод аутентификации `formLogin` легко настраивается. Более того, вы можете использовать этот тип аутентификации вместе с методом HTTP Basic.

Часть III

Методика авторизации

После того как приложение идентифицировало источник вызова конечной точки, наступает решающий этап процедуры определения разрешений: авторизация. Правильная реализация имеет решающее значение, поскольку ошибки могут поставить под угрозу конфиденциальность пользователя и целостность данных. В этой части книги я проведу вас через сложные уровни авторизации и защиты от распространенных уязвимостей.

Глава 7 предлагает детально исследовать сферу ограничений доступа, уделяя особое внимание полномочиям и ролям пользователей, а также рассказывает об универсальном применении этих ограничений.

Глава 8 продолжает знакомство, представляя современные методы, такие как `requestMatchers()`, для выбора и применения ограничений авторизации. Она также знакомит с использованием регулярных выражений для более точного управления.

Глава 9 рассматривает насущную проблему *подделки межсайтовых запросов* (cross-site request forgery, CSRF). Изучив механизм работы Spring Security, вы сможете эффективно применять и настраивать защиту от CSRF.

Глава 10 знакомит с концепцией *обмена ресурсами с запросом происхождения* (cross-origin resource sharing, CORS), поясняя, как она работает, и проводя вас через применение политик CORS с помощью аннотаций и `CorsConfigurer`.

Глава 11 посвящена безопасности методов, в соответствии с которой отдельные функции в вашем приложении будут придерживаться строгих принципов авторизации. В главе рассматриваются правила предварительной авторизации и поставторизации, а также расширенные настройки разрешений для методов.

Глава 12 завершается методами фильтрации на уровне методов, охватывая различные аспекты от предварительной фильтрации до постфильтрации и интегрируя их в репозитории Spring Data.

К концу этой части вы накопите достаточные знания для разработки и реализации комплексных стратегий авторизации, благодаря чему ваше приложение будет как функциональным, так и защищенным.

Авторизация на уровне конечной точки: ограничение доступа

Темы этой главы:

- определение полномочий и ролей;
- применение правил авторизации на конечных точках.

Несколько лет назад я катался на лыжах в прекрасных Карпатах и стал свидетелем забавной сцены. Около 10, может быть, 15 человек стояли в очереди, чтобы попасть в кабинку горнолыжного подъемника, когда к турникету подошел известный певец в сопровождении двух телохранителей. Он уверенно шагнул вперед, намереваясь пройти без очереди, потому что был очень знаменит. Но в голове очереди его ждал сюрприз. «Ваш билет, пожалуйста!» – сказал контролер, управляющий посадкой. Затем он пояснил: «Во-первых, вам нужно приобрести билет, а во-вторых, здесь нет приоритетной очереди, извините. Очередь заканчивается там», – и указал певцу на конец очереди. Часто в жизни не важно, кто вы. То же самое можно сказать и о приложениях. Не важно, кем является человек, желающий получить доступ к определенной функции или данным, если он не авторизован!

До сих пор мы обсуждали только процесс аутентификации, в ходе которого приложение должно опознать вызывающую сторону. В предыдущих примерах мы не реализовывали никаких правил для принятия решения об одобрении запроса. Нас интересовало только

то, знает система пользователя или нет. В большинстве приложений не бывает так, чтобы все пользователи, опознанные системой, могли получить одинаковый доступ ко всем ресурсам в ней. В этой главе мы обсудим авторизацию. *Авторизация* – это процесс, в ходе которого система решает, имеет ли идентифицированный клиент разрешение на доступ к запрошенному ресурсу (рис. 7.1).

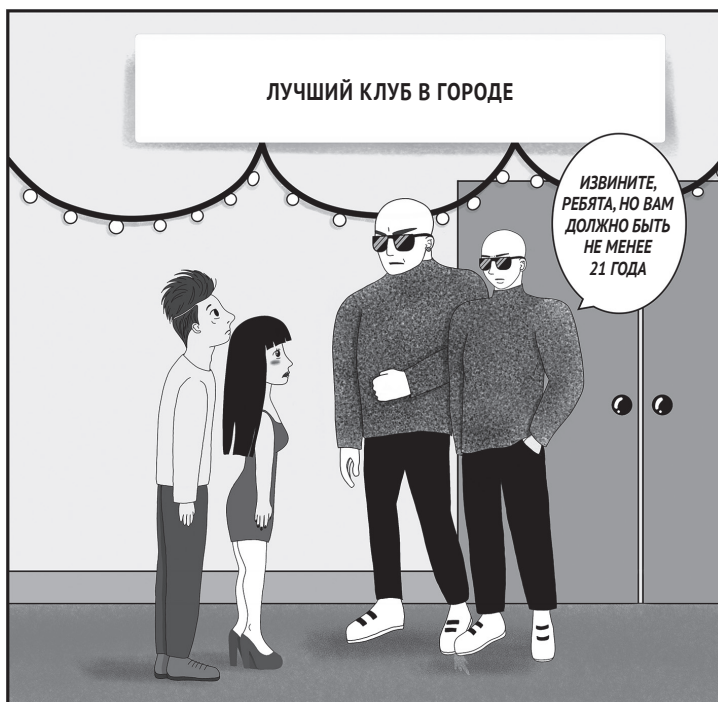


Рис. 7.1 Авторизация – это процесс, в ходе которого приложение решает, разрешено ли аутентифицированному субъекту получать доступ к запрашиваемому ресурсу. Авторизация всегда происходит после аутентификации

В Spring Security после завершения потока аутентификации приложение сразу делегирует запрос фильтру авторизации. Фильтр решает, разрешает или отклоняет запрос на основе настроенных правил авторизации (рис. 7.2).

В этой главе вы познакомитесь с основными принципами авторизации:

- научитесь определять полномочия и применять правила доступа ко всем конечным точкам на основе полномочий пользователя;
- узнаете, как группировать полномочия по ролям и как применять правила авторизации на основе ролей пользователя.

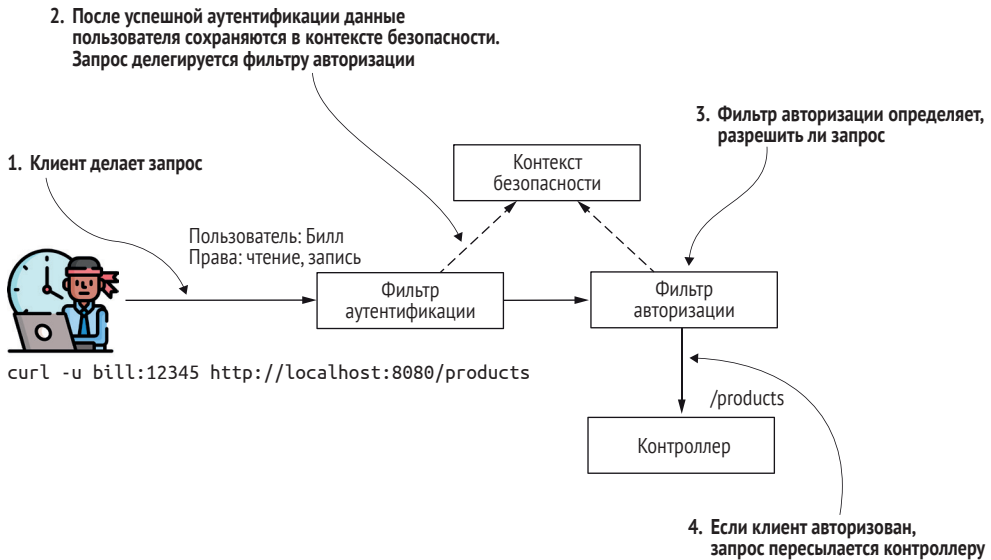


Рис. 7.2 При поступлении запроса фильтр аутентификации проверяет личность пользователя. После успешной проверки фильтр помещает данные пользователя в контекст безопасности и передает запрос фильтру авторизации. Этот фильтр оценивает, следует ли разрешить запрос. Он принимает решение на основании информации о пользователе, предоставленной в контексте безопасности

В главе 8 мы продолжим выбирать конечные точки, к которым будем применять правила авторизации. Но сначала вы узнаете, что такое полномочия и роли и по какому принципу они ограничивают доступ к приложениям.

7.1 Ограничение доступа на основе полномочий и ролей

В этом разделе вы узнаете о концепциях авторизации и ролей. Они нужны для защиты конечных точек вашего приложения. Без понимания концепций вы не сможете применять их в реальных сценариях, где разные пользователи имеют разные разрешения. В зависимости от привилегий, которые имеют пользователи, они могут выполнять только определенные действия. Приложение предоставляет привилегии в виде полномочий и ролей.

В главе 3 вы реализовали интерфейс `GrantedAuthority`. Этот контракт упоминался, когда мы обсуждали другой важный компонент: интерфейс `UserDetails`. Тогда мы не работали с `GrantedAuthority`, потому что, как вы узнаете в этой главе, этот интерфейс в основном связан с процессом авторизации. Но теперь пришло время вернуться к `GrantedAuthority`, чтобы изучить его назначение. На рис. 7.3 по-

казана связь между контрактом `UserDetails` и интерфейсом `GrantedAuthority`. После того как мы закончим обсуждение контракта, вы научитесь использовать эти правила по отдельности или для конкретных запросов.

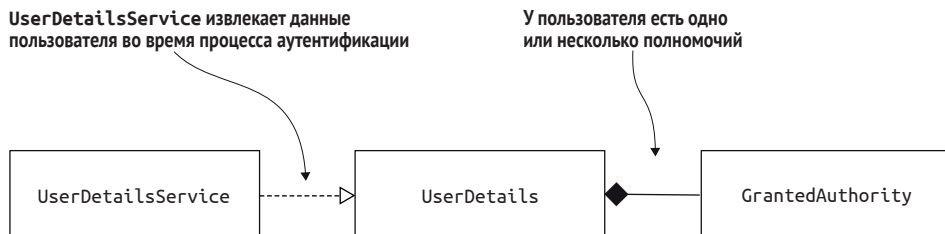


Рис. 7.3 Пользователь обладает одним или несколькими полномочиями (допустимыми действиями). На этапе аутентификации `UserDetailsService` извлекает исчерпывающие сведения о пользователе, включая его полномочия. После успешной аутентификации приложение использует эти полномочия, как показано в интерфейсе `GrantedAuthority`, для выполнения авторизации

В листинге 7.1 показано определение контракта `GrantedAuthority`. Полномочия – это действия, которые пользователь может выполнить с системным ресурсом. Полномочия имеют имя, которое метод `getAuthority()` объекта возвращает как строку. Мы используем имя полномочия при определении пользовательского правила авторизации. Часто правило авторизации может выглядеть следующим образом: «Джейн разрешено *удалять* записи о продуктах» или «Джону разрешено *читать* записи о документах». В этих случаях предоставленными полномочиями являются удаление и чтение. Приложение позволяет пользователям Джейн и Джону выполнять действия, которые часто так и называются – `read` (чтение), `write` (запись) и `delete` (удаление).

Листинг 7.1 Контракт `GrantedAuthority`

```
public interface GrantedAuthority extends Serializable {
    String getAuthority();
}
```

`UserDetails`, который является контрактом, описывающим пользователя в Spring Security, имеет коллекцию экземпляров `GrantedAuthority`, как показано на рис. 7.3. Вы можете предоставить пользователю одну или несколько привилегий. Метод `getAuthorities()` возвращает коллекцию экземпляров `GrantedAuthority`. В листинге 7.2 вы можете просмотреть этот метод в контракте `UserDetails`. Мы реализуем этот метод так, чтобы он возвращал все полномочия, предоставленные пользователю. После завершения аутентификации полномочия становятся частью сведений о пользователе,

который вошел в систему. Приложение может использовать их для предоставления разрешений.

Листинг 7.2 Метод `getAuthorities()` из контракта `UserDetails`

```
public interface UserDetails extends Serializable {  
    Collection<? extends GrantedAuthority> getAuthorities();  
  
    // Часть кода не показана  
}
```

7.1.1 Ограничение доступа для всех конечных точек на основе полномочий пользователя

В этом разделе говорится о том, как ограничить доступ к конечным точкам для определенных пользователей. До сих пор в наших примерах любой аутентифицированный пользователь мог вызвать любую конечную точку приложения. Теперь вы научитесь настраивать этот доступ. В реальных приложениях вы иногда можете вызывать некоторые конечные точки, даже если вы не аутентифицированы, в то время как для других вам нужны специальные привилегии (рис. 7.4). Мы рассмотрим несколько примеров, чтобы вы освоили различные способы применения этих ограничений с помощью Spring Security.

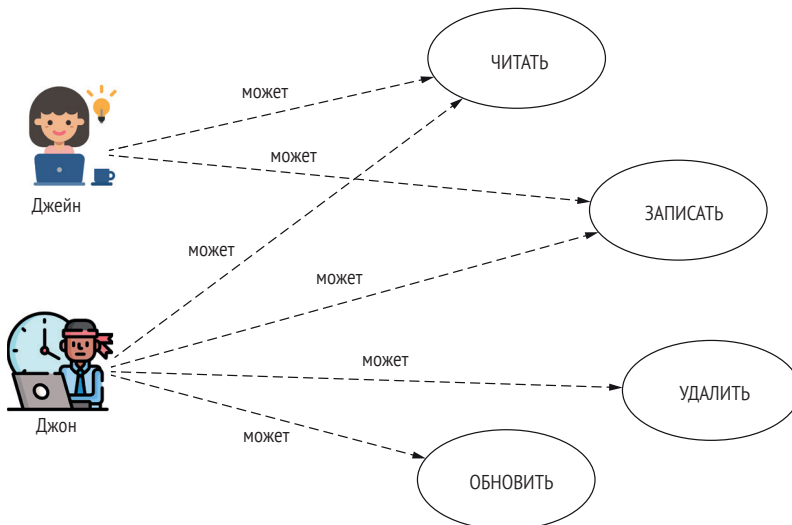


Рис. 7.4 Полномочия определяют допустимые операции, которые пользователи могут выполнять в приложении. Эти операции информируют о создании протоколов авторизации, ограничивая определенные запросы к конечным точкам для пользователей с назначенными полномочиями. Например, Джейн ограничена чтением и записью в конечной точке, тогда как Джон имеет возможность читать, писать, удалять и изменять данные в этой конечной точке

Теперь, когда вы вспомнили контракты `UserDetails` и `GrantedAuthority` и узнали больше о взаимосвязи между ними, пришло время написать небольшое приложение, которое применяет правило авторизации. В этом примере вы увидите несколько альтернативных способов настройки доступа к конечным точкам на основе полномочий пользователя. Мы начинаем новый проект, который я назвал `ssia-ch7-ex1`. Я покажу вам три способа, которыми вы можете настроить доступ с помощью следующих методов:

- `hasAuthority()` – получает в качестве параметров только одно полномочие, для которого приложение настраивает ограничения. Только пользователи, имеющие это полномочие, могут вызывать конечную точку;
- `hasAnyAuthority()` – может получать более одного полномочия, для которого приложение настраивает ограничения. Фактически пользователь может иметь любые из существующих полномочий и должен иметь по крайней мере одно из них, чтобы сделать запрос.

Я рекомендую использовать этот метод или метод `hasAuthority()` из-за их простоты, выбирая в зависимости от количества назначенных вами привилегий. Они просты для чтения в конфигурациях и делают ваш код более понятным;

- `access()` – предлагает неограниченные возможности для настройки доступа, поскольку приложение создает правила авторизации на основе пользовательского объекта с именем `AuthorizationManager`, который вы реализуете. Вы можете разработать любую реализацию для контракта `AuthorizationManager` в зависимости от ваших потребностей. Spring Security также предлагает несколько готовых реализаций. Наиболее распространенной реализацией является менеджер `WebExpressionAuthorizationManager`, который помогает вам применять правила авторизации на основе Spring Expression Language (SpEL). Но использование метода `access()` может сделать правила авторизации более сложными для чтения и понимания. По этой причине я рекомендую его как наименее предпочтительное решение и только если вы не можете применить методы `hasAnyAuthority()` или `hasAuthority()`.

Единственные зависимости, необходимые в вашем файле `pom.xml`, – это `spring-boot-starter-web` и `spring-boot-starter-security`. Их достаточно, чтобы использовать любое из трех вышеупомянутых решений. Этот пример можно найти в проекте `ssia-ch7-ex1`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Мы также добавляем конечную точку в приложение для проверки нашей конфигурации авторизации:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello!";
    }
}
```

В классе конфигурации мы объявляем `InMemoryUserDetailsManager` как наш `UserDetailsService` и добавляем двух пользователей, Джона и Джейн, обладающих разными полномочиями. В листинге 7.3 показано, как это сделать.

Листинг 7.3 Объявление `UserDetailsService` и назначение пользователей

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var manager = new InMemoryUserDetailsManager();

        var user1 = User.withUsername("john")
            .password("12345")
            .authorities("READ")
            .build();

        var user2 = User.withUsername("jane")
            .password("12345")
            .authorities("WRITE")
            .build();

        manager.createUser(user1);

        manager.createUser(user2);

        return manager;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
```

```

        return NoOpPasswordEncoder.getInstance();
    }
}

```

- ❶ `UserDetailsService`, возвращаемый методом, добавляется в `SpringContext`.
- ❷ Объявляем `InMemoryUserDetailsManager`, который хранит пару пользователей.
- ❸ Первый пользователь, Джон, имеет полномочия `READ`.
- ❹ Второй пользователь, Джейн, имеет полномочия `WRITE`.
- ❺ За добавление и управление пользователями отвечает `UserDetailsService`.
- ❻ Не забывайте, что также необходим `PasswordEncoder`.

Следующим шагом мы добавляем конфигурацию авторизации. В главе 2, когда мы работали над первым примером, вы видели, что мы можем сделать все конечные точки доступными для всех желающих. Для этого мы создали компонент `SecurityFilterChain` в контексте приложения, аналогично тому, как показано в листинге 7.4.

Листинг 7.4 Обеспечение доступа ко всем конечным точкам без аутентификации

```

@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().permitAll()
        );

        return http.build();
    }
}

```

- ❶ Разрешаем доступ для всех запросов.

Метод `authorizeHttpRequests()` позволяет нам указать правила авторизации для конечных точек. Метод `anyRequest()` указывает, что правило применяется ко всем запросам независимо от используемого URL или метода HTTP. Метод `permitAll()` разрешает доступ ко всем совпавшим запросам, аутентифицированным или нет.

Допустим, мы хотим сделать так, чтобы только пользователи с полномочиями `WRITE` могли получить доступ ко всем конечным точкам. В нашем примере это только Джейн. Мы можем достичь на-

шей цели и ограничить доступ на основе полномочий пользователя. Взгляните на код в листинге 7.5.

Листинг 7.5 Ограничение доступа только для пользователей с полномочиями WRITE

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest()
                .hasAuthority("WRITE")
        );

        return http.build();
    }
}
```

❶ Указываем условие, при котором пользователь имеет доступ к конечным точкам.

Вы можете видеть, что метод `permitAll()` заменен на метод `hasAuthority()`. Вы указываете имя полномочий, выданных пользователю, в качестве параметра метода `hasAuthority()`. Сначала приложению необходимо аутентифицировать запрос, а затем, на основе полномочий пользователя, приложение решает, разрешить ли вызов конечной точки.

Теперь мы можем начать тестировать приложение, вызывая конечную точку с каждым из двух пользователей. Когда мы вызываем конечную точку с пользователем Jane (Джейн), статус ответа HTTP 200 OK, и мы видим тело ответа «Hello!». Когда мы вызываем ее с пользователем John (Джон), статус ответа HTTP 403 Forbidden, и мы получаем пустое тело ответа. Например, вот вызов конечной точки с пользователем Jane

```
curl -u jane:12345 http://localhost:8080/hello
```

и ответ на вызов:

```
Hello!
```

Вызывая конечную точку с пользователем John

```
curl -u john:12345 http://localhost:8080/hello
```

мы получаем следующий ответ:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

Аналогично мы можем использовать метод `hasAnyAuthority()`. Этот метод имеет параметр `varargs`, благодаря чему может получать несколько имен полномочий. Приложение разрешает запрос, если у пользователя есть хотя бы одно из полномочий, предоставленных в качестве параметра метода. Вы можете заменить `hasAuthority()` в предыдущем листинге на `hasAnyAuthority("WRITE")`, в этом случае приложение будет работать точно так же. Однако если вы замените `hasAuthority()` на `hasAnyAuthority("WRITE", "READ")`, то будут приняты запросы от пользователей с любым из этих полномочий. В нашем случае приложение будет разрешать запросы как от Джона, так и от Джейн. В листинге 7.6 показано применение метода `hasAnyAuthority()`.

Листинг 7.6 Применение метода `hasAnyAuthority()`

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest()
                .hasAnyAuthority("WRITE", "READ");
        );

        return http.build();
    }
}
```

❶ Разрешаем запросы от пользователей с полномочиями `WRITE` и `READ`.

Теперь вы можете успешно вызвать конечную точку с любым из наших двух пользователей. Вот вызов для Джона:

```
curl -u john:12345 http://localhost:8080/hello
```


Тело ответа:

Hello!

А вот вызов для Джейн:

```
curl -u jane:12345 http://localhost:8080/hello
```

Тело ответа прежнее:

Hello!

Третий способ указать доступ на основе полномочий пользователя, применяемый на практике, – это метод `access()`. Однако метод `access()` более общий. Он получает в качестве параметра реализацию `AuthorizationManager`. Вы можете разработать любую реализацию этого объекта, которая может применять любую логику, определяющую правила авторизации. Этот метод мощный, и он относится не только к полномочиям. Однако данный метод также делает код более сложным для чтения и понимания. По этой причине я рекомендую его как последний вариант и только если вы не можете применить один из методов `hasAuthority()` или `hasAnyAuthority()`, представленных ранее в этом разделе.

Чтобы сделать этот метод более понятным, я сначала представляю его как альтернативу указанию полномочий с помощью методов `hasAuthority()` и `hasAnyAuthority()`. В этом примере вы будете использовать реализацию `AuthorizationManager`, где необходимо предоставить выражение SpEL в качестве параметра. Правило авторизации, которое мы определили, становится более сложным для чтения, и именно поэтому я не рекомендую данный подход для простых правил. Однако метод `access()` имеет преимущество, позволяя вам настраивать правила через реализацию `AuthorizationManager`, которую вы предоставляете в качестве параметра. И это действительно мощно! Как и в случае с выражениями SpEL, вы можете в принципе определить любое условие.

ПРИМЕЧАНИЕ В большинстве ситуаций требуемые ограничения можно реализовать с помощью методов `hasAuthority()` и `hasAnyAuthority()`, и я рекомендую вам использовать их. Используйте метод `access()` только в том случае, если два других варианта не подходят и вы хотите реализовать более общие правила авторизации.

Начнем с простого примера, удовлетворяющего тем же условиям, что и в предыдущих случаях. Если вам нужно только проверить, есть ли у пользователя определенные полномочия, выражение, которое

вам нужно использовать с методом `access()`, может быть одним из следующих:

- `hasAuthority('WRITE')` – указывает, что пользователю нужны полномочия `WRITE` для вызова конечной точки;
- `hasAnyAuthority('READ', 'WRITE')` – указывает, что пользователю нужны полномочия `READ` или `WRITE`. С помощью этого выражения вы можете перечислить все полномочия, для которых вы хотите разрешить доступ.

Обратите внимание, что эти выражения имеют то же имя, что и методы, представленные ранее в этом разделе. Следующий листинг демонстрирует, как использовать метод `access()`.

Листинг 7.7 Использование метода `access()` для настройки доступа к конечным точкам

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest()
                .access("hasAuthority('WRITE')")
        );

        return http.build();
    }
}
```

❶ Авторизуем запросы от пользователей с полномочиями `WRITE`.

Пример, представленный в листинге 7.7, демонстрирует, как метод `access()` усложняет синтаксис, если вы используете его для простых условий. В таком случае вам следует напрямую использовать метод `hasAuthority()` или `hasAnyAuthority()`. Но метод `access()` не всегда плох. Как было сказано ранее, он обеспечивает гибкость. Вы можете столкнуться с реальными ситуациями, когда он лучше подходит для написания более сложных выражений, на основе которых приложение предоставляет доступ. Вы не смогли бы реализовать эти сценарии без метода `access()`.

В листинге 7.8 показано применение метода `access()` с выражением, которое нелегко написать в противном случае. А именно конфигурация, представленная в листинге 7.8, определяет двух пользователей,

Джона и Джейн, которые имеют разные полномочия. Пользователь Джон имеет полномочия только на чтение, в то время как Джейн имеет полномочия на чтение, запись и удаление. Конечная точка должна быть доступна тем пользователям, у которых есть полномочия на чтение, но не тем, у кого есть полномочия на удаление.

ПРИМЕЧАНИЕ В приложениях Spring вы найдете различные стили и соглашения для именования полномочий. Некоторые разработчики используют все заглавные буквы, в то время как другие используют все строчные буквы. По моему мнению, все эти варианты приемлемы, если вы сохраняете их единообразие в своем приложении. В этой книге я использую разные стили в примерах, чтобы вы могли увидеть больше подходов, с которыми вы можете столкнуться на практике.

Конечно, это гипотетический пример, но он достаточно прост для понимания и достаточно сложен, чтобы показать, почему метод `access()` более мощный. Чтобы решить задачу с помощью метода `access()`, вы можете использовать реализацию `AuthorizationManager`, которая принимает выражение SpEL. Выражение SpEL должно отражать требование. Например:

```
"hasAuthority('read') and !hasAuthority('delete')"
```

Следующий листинг иллюстрирует применение метода `access()` с более сложным выражением. Вы можете найти этот пример в проекте с именем `ssia-ch7-ex2`.

Листинг 7.8 Применение метода `access()` с более сложным выражением

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var manager = new InMemoryUserDetailsManager();

        var user1 = User.withUsername("john")
            .password("12345")
            .authorities("read")
            .build();

        var user2 = User.withUsername("jane")
            .password("12345")
            .authorities("read", "write", "delete")
            .build();

        manager.createUser(user1);
    }
}
```

```

        manager.createUser(user2);

        return manager;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        String expression =
            ""hasAuthority('read') and      ❶
            !hasAuthority('delete')         ❷
            """;                             ❸

        http.authorizeHttpRequests(
            c -> c.anyRequest()
                .access(new WebExpressionAuthorizationManager(expression));
        );

        return http.build();
    }
}

```

- ❶ Указываем, что пользователь должен иметь полномочия на чтение, но не на удаление.

Теперь протестируем наше приложение, вызвав конечную точку /hello для Джона:

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

Hello!

Но при вызове конечной точки пользователем Джейн

```
curl -u jane:12345 http://localhost:8080/hello
```

тело ответа:

```

{
    "status":403,
    "error":"Forbidden",
    "message":"Forbidden",
    "path":"/hello"
}

```

Пользователь Джон имеет полномочия только на чтение и может успешно вызвать конечную точку. Однако Джейн также имеет полномочия на удаление и поэтому не имеет полномочий на вызов конечной точки. HTTP-статус для вызова Джейн – 403 Forbidden.

В предыдущих примерах было показано, как устанавливать ограничения в отношении полномочий, которые требуются пользователю для доступа к некоторым заданным конечным точкам. Конечно, мы еще не обсуждали, как на запрос влияет путь или метод HTTP. Вместо этого мы применили правила для всех запросов независимо от конечной точки. После обсуждения ролей пользователей я расскажу, как выбрать конечные точки, к которым вы применяете конфигурации авторизации.

7.1.2 Ограничение доступа для всех конечных точек на основе ролей пользователей

В этом разделе мы обсуждаем ограничение доступа к конечным точкам на основе ролей. *Роли* – это еще один способ обозначить, что может делать пользователь (рис. 7.5). Роли часто применяются в реальных приложениях, поэтому важно понимать суть роли и разницу

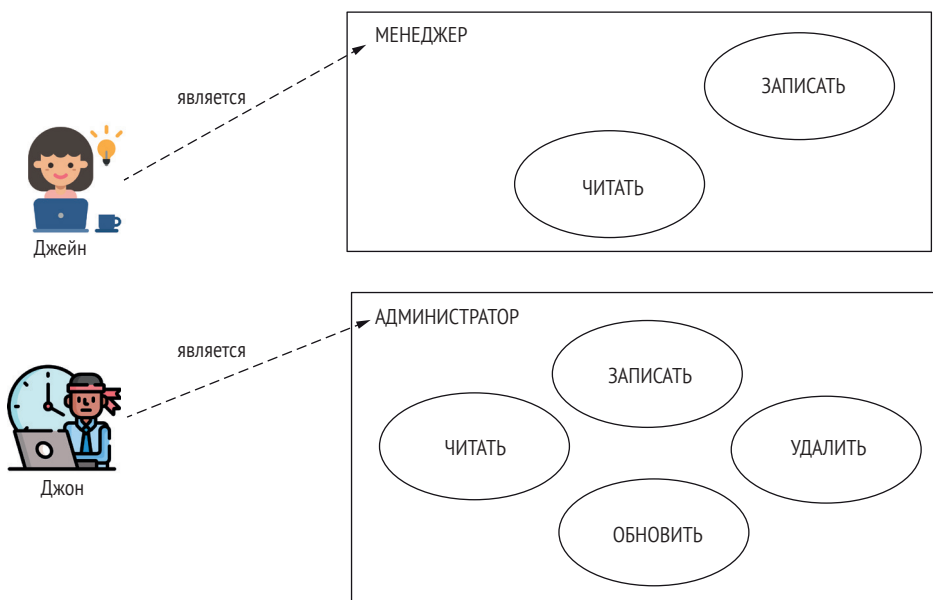


Рис. 7.5 Роли разделены на крупные сегменты. Каждый пользователь с определенной ролью может выполнять только действия, предоставленные этой ролью. Согласно этой философии при авторизации запрос разрешается на основе предназначения пользователя в системе. Только пользователи с определенной ролью могут вызывать определенную конечную точку

между ролями и полномочиями. В этом разделе мы применяем несколько примеров использования ролей, чтобы вы знали все практические сценарии, в которых приложение использует роли, и как писать конфигурации для этих случаев.

Spring Security рассматривает полномочия как тонко настраиваемые привилегии, к которым мы применяем ограничения. Роли подобны бейджам на груди пользователя. Они дают пользователю привилегии для *группы* действий. Некоторые приложения всегда предоставляют одни и те же группы полномочий определенным пользователям. Представьте, что в вашем приложении пользователь может иметь либо только полномочия на чтение, либо все (чтение, запись и удаление). В этом случае удобнее сделать так, чтобы пользователи, которые могут только читать, имели роль с именем `READER`, в то время как остальные имели роль `ADMIN`. Наличие роли `ADMIN` означает, что приложение предоставляет вам привилегии на чтение, запись, обновление и удаление. Потенциально у вас может быть больше ролей. Например, если в какой-то момент из запросов следует, что вам также нужен пользователь, которому разрешено только читать и записывать, вы можете создать третью роль с именем `MANAGER`.

ПРИМЕЧАНИЕ При использовании подхода с ролями в приложении вам больше не придется определять полномочия. Полномочия существуют в данном случае как концепция и могут появляться в требованиях к реализации. Но в приложении вам нужно определить роль только для охвата одного или нескольких таких действий, которые пользователь имеет привилегию выполнять.

Имена, которые вы даете ролям, похожи на имена для полномочий – вы свободны в своем выборе. Можно сказать, что роли являются более крупномасштабной градацией по сравнению с полномочиями. В любом случае, за кулисами роли представлены с использованием того же контракта `GrantedAuthority` в Spring Security. При определении роли ее имя должно начинаться с префикса `ROLE_`. На уровне реализации этот префикс указывает разницу между ролью и полномочиями. Пример, над которым мы работаем в этом разделе, находится в проекте `ssia-ch7-ex3`. В листинге 7.9 показано изменение, которое я внес в предыдущий пример.

Листинг 7.9 Настройка ролей для пользователей

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
```

```

var manager = new InMemoryUserDetailsManager();

var user1 = User.withUsername("john")
    .password("12345")
    .authorities("ROLE_ADMIN") b
    .build();

var user2 = User.withUsername("jane")
    .password("12345")
    .authorities("ROLE_MANAGER")
    .build();

manager.createUser(user1);
manager.createUser(user2);

return manager;
}

// Часть кода не показана
}

```

❶ При наличии префикса `ROLE_` `GrantedAuthority` теперь представляет роль.

Чтобы задать ограничения для ролей пользователей, вы можете использовать один из следующих методов:

- `hasRole()` – получает в качестве параметра имя роли, для которой приложение авторизует запрос;
- `hasAnyRole()` – получает в качестве параметров имена ролей, для которых приложение авторизует запрос;
- `access()` – использует `AuthorizationManager` для указания роли или ролей, для которых приложение авторизует запросы. Вы можете использовать `hasRole()` или `hasAnyRole()` как выражения SpEL вместе с реализацией `WebExpressionAuthorizationManager`.

Как вы заметили, имена методов для ролей похожи на имена методов для полномочий, представленные в разделе 7.1.1. Мы используем их аналогично, только вместо полномочий имеем дело с ролями. Мои рекомендации также схожи: используйте методы `hasRole()` или `hasAnyRole()` в качестве первого варианта и прибегайте к использованию `access()` только тогда, когда предыдущие два метода не применимы. Следующий листинг показывает, как сейчас выглядит метод `securityFilterChain()`.

Листинг 7.10 Настройка приложения для приема только запросов от администраторов

```

@Configuration
public class ProjectConfig {

    // Часть кода не показана
}

```

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeHttpRequests(
        c -> c.anyRequest().hasRole("ADMIN")
    );

    return http.build();
}
```

❶ Метод `hasRole()` теперь определяет роли, для которых разрешен доступ к конечной точке. Помните, что префикс `ROLE_` здесь не отображается.

ПРИМЕЧАНИЕ Важно отметить, что префикс `ROLE_` нужен только для объявления роли. При использовании роли мы ссылаемся на нее исключительно по имени.

При тестировании приложения мы увидим, что пользователь Джон может получить доступ к конечной точке, в то время как Джейн получает ответ HTTP 403 Forbidden. Чтобы вызвать конечную точку от имени Джона, выполните команду

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

Hello!

А для вызова конечной точки от имени Джейн введите команду

```
curl -u jane:12345 http://localhost:8080/hello
```

Тело ответа:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

При создании пользователей с помощью класса конструктора `User`, как мы делали в примере для этого раздела, мы указываем роль с помощью метода `roles()`. Этот метод создает объект `GrantedAuthority` и автоматически добавляет префикс `ROLE_` к указанным вами именам.

ПРИМЕЧАНИЕ Убедитесь, что параметр, который вы предоставляете для метода `roles()`, не включает префикс `ROLE_`. Если этот префикс случайно попадает в параметр `role()`, метод выдает исключение. Короче говоря, при использовании метода `authority()` добавьте префикс `ROLE_`. При использовании метода `roles()` *не* добавляйте префикс `ROLE_`.

В листинге 7.11 показан правильный способ использования метода `roles()` вместо метода `authority()` при проектировании доступа на основе ролей. Вы также можете сравнить этот листинг с листингом 7.9, чтобы увидеть разницу между использованием полномочий и ролей.

Листинг 7.11 Настройка ролей с помощью метода `roles()`

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var manager = new InMemoryUserDetailsManager();

        var user1 = User.withUsername("john")
            .password("12345")
            .roles("ADMIN")
            .build();

        var user2 = User.withUsername("jane")
            .password("12345")
            .roles("MANAGER")
            .build();

        manager.createUser(user1);
        manager.createUser(user2);

        return manager;
    }

    // Часть кода не показана
}
```

❶ Метод `roles()` определяет роли пользователя.

Подробнее о методе `access()`

В разделах 7.1.1 и 7.1.2 вы узнали, как использовать метод `access()` для применения правил авторизации, ссылающихся на полномочия и роли. В общем случае в приложении ограничения авторизации связаны с полномочиями и ролями. Однако важно помнить, что метод `access()` является универсальным и зависит только от того, какую реализацию кон-

тракта `AuthorizationManager` вы предоставляете в качестве параметра. Более того, в нашем примере мы использовали только реализацию `WebExpressionAuthorizationManager`, которая применяет ограничения авторизации на основе выражения SpEL. Здесь я стараюсь научить вас применять ее для полномочий и ролей, но на практике `WebExpressionAuthorizationManager` получает любое выражение SpEL. Оно не обязательно должно быть связано с полномочиями и ролями.

Простым примером может служить настройка доступа, в соответствии с которой конечная точка будет доступна только после 12:00. Чтобы решить эту задачу, можно использовать следующее выражение SpEL:

```
T(java.time.LocalDateTime).now().isAfter(T(java.time.LocalDateTime).of(12, 0))
```

Подробнее о выражениях SpEL говорится в документации фреймворка Spring: <http://mng.bz/M9J7>.

Можно сказать, что с помощью метода `access()` вы можете реализовать практически любое правило. Возможности безграничны. Просто не забывайте, что в приложениях мы всегда стремимся сохранять синтаксис максимально простым. Усложняйте свои конфигурации только тогда, когда у вас нет другого выбора. Вы найдете этот пример в проекте `ssia-ch7-ex4`.

7.1.3 Ограничение доступа ко всем конечным точкам

В этом разделе мы обсуждаем ограничение доступа для всех запросов. В разделе 5.2 вы узнали, что с помощью метода `permitAll()` вы можете разрешить доступ для всех запросов. Вы также узнали, что можете применять правила доступа на основе полномочий и ролей. Но у вас есть еще возможность отклонять все запросы. Метод `denyAll()` – это полная противоположность методу `permitAll()`. В следующем листинге вы увидите, как использовать метод `denyAll()`.

Листинг 7.12 Использование метода `denyAll()` для ограничения доступа к конечным точкам

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().denyAll()
        );
    }
}
```

```
);  
  
return http.basic();  
}  
}
```

❶ Используем `denyAll()` для ограничения доступа для всех.

Где можно использовать такое ограничение? Оно применяется не так часто, как другие методы, но бывают случаи, когда это необходимо. Позвольте мне привести вам пару примеров, чтобы прояснить этот момент.

Предположим, что у вас есть конечная точка, получающая адрес электронной почты в качестве переменной пути. Вы хотите разрешить запросы, которые имеют значение переменной адреса, заканчивающееся на `.com`. Вам не нужно, чтобы приложение принимало какой-либо другой формат адреса электронной почты. (В следующей главе вы узнаете, как применять ограничения для группы запросов на основе пути и метода HTTP и даже для переменных пути.) Поэтому вы применяете регулярное выражение для группировки запросов, которые соответствуют вашему правилу, а затем используете метод `denyAll()`, чтобы указать вашему приложению отклонить все эти запросы (рис. 7.6).

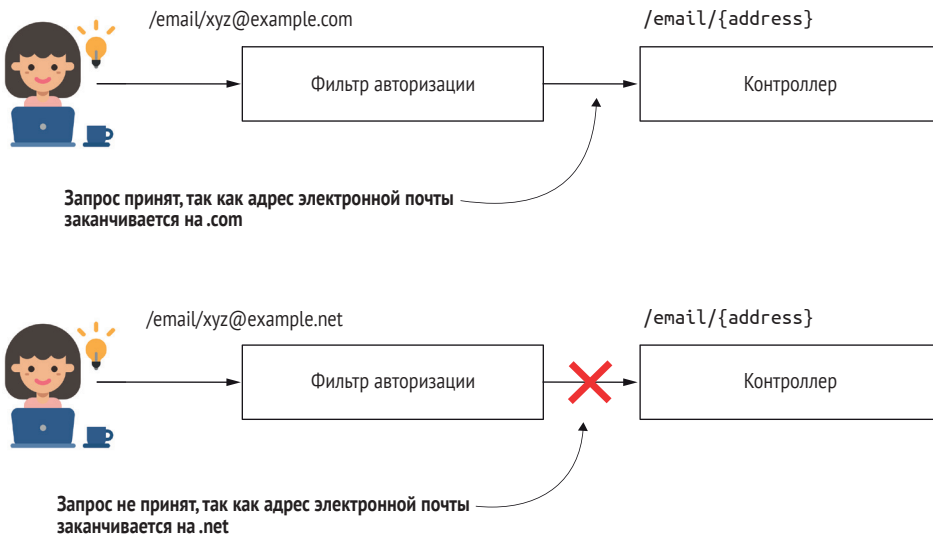


Рис. 7.6 Когда пользователь вызывает конечную точку со значением параметра, заканчивающимся на `.com`, приложение принимает запрос. Когда пользователь вызывает конечную точку и предоставляет адрес электронной почты, заканчивающийся на `.net`, приложение отклоняет вызов. Чтобы добиться такого поведения, вы можете использовать метод `denyAll()` для всех конечных точек, для которых значение параметра не заканчивается на `.com`

В качестве второго примера представьте приложение, действующее, как показано на рис. 7.7. Несколько служб реализуют варианты использования приложения, которые доступны путем вызова конечных точек, доступных по разным путям. Но для вызова конечной точки клиент запрашивает другую службу, которую мы можем называть *шлюзом*. В архитектуре есть две отдельные службы этого типа. На рис. 7.7 я назвал их шлюзом А и шлюзом В. Клиент запрашивает шлюз А, если он хочет получить доступ к пути `/products`. Но для пути `/articles` клиент должен запросить шлюз В. Каждая из служб шлюза разработана для отклонения всех запросов к другим путям, которые они не обслуживают. Этот упрощенный сценарий поможет вам понять метод `denyAll()`. В реальных приложениях вы можете найти похожие случаи, но в более сложных архитектурах.

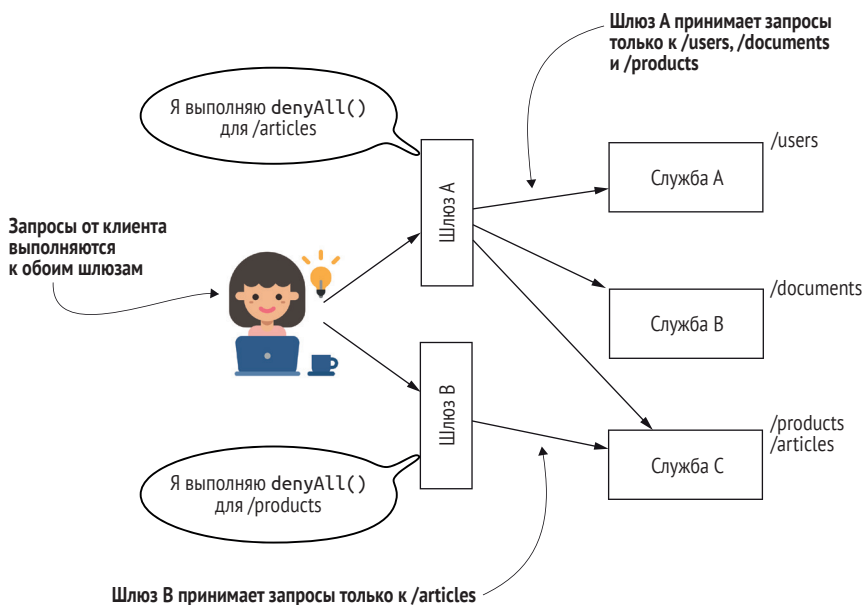


Рис. 7.7 Доступ осуществляется через шлюзы А и В. Каждый из шлюзов доставляет только запросы для определенных путей и отклоняет все остальные

При разработке приложений вы можете столкнуться с различными архитектурными требованиями, которые иногда выглядят странно. Фреймворк должен обеспечивать необходимую гибкость для любой ситуации, с которой вы можете столкнуться. По этой причине метод `denyAll()` так же важен, как и все другие варианты, которые вы узнали в этой главе.

Заключение

- Авторизация – это процесс, в ходе которого приложение решает, разрешен ли аутентифицированный запрос. Авторизация всегда происходит после аутентификации.
- Вы настраиваете реакцию приложения на запросы на основе полномочий и ролей аутентифицированного пользователя.
- В своем приложении вы также можете указать, что определенные запросы возможны для неаутентифицированных пользователей.
- Вы можете настроить свое приложение так, чтобы оно отклоняло любые запросы, используя метод `denyAll()`, или разрешало любые запросы, используя метод `permitAll()`.

Автоматизация на уровне конечной точки: применение ограничений

Темы этой главы:

- выбор запросов для применения ограничений с использованием методов сопоставления;
- изучение наилучших сценариев для каждого метода сопоставления.

В главе 7 вы узнали, как настроить доступ на основе полномочий и ролей. Но мы применили конфигурации ко всем конечным точкам без разбора. В этой главе вы узнаете, как применять ограничения авторизации к определенной группе запросов. В реальных приложениях вы вряд ли будете применять одни и те же правила для всех запросов. У вас есть конечные точки, которые могут быть вызваны только определенными пользователями, в то время как другие конечные точки могут быть доступны всем. В зависимости от бизнес-требований каждое приложение имеет свою собственную конфигурацию авторизации. В этой главе мы обсудим доступные варианты организации выборочной обработки запросов при написании конфигураций доступа.

Первым методом сопоставления запроса с конечной точкой, который вы использовали (хотя и не задумывались об этом), был метод `anyRequest()`. И поскольку он использовался в предыдущих главах, теперь вы знаете, что он относится ко всем запросам, независимо от

пути или метода HTTP. Это способ сказать «любой запрос» или, иногда, «любой другой запрос».

Сначала поговорим об отборе запросов по пути; затем добавим в сценарий отбора метод HTTP. Чтобы выбрать запросы, к которым применяется конфигурация авторизации, мы используем метод `requestMatchers()`.

8.1 *Использование метода `requestMatchers()` для выбора конечных точек*

В этом разделе вы узнаете, как использовать метод `requestMatchers()` в целом, чтобы в разделах 8.2–8.4 мы могли продолжить описывать различные подходы к выбору HTTP-запросов, к которым вам необходимо применить ограничения авторизации. К концу этой главы вы сможете применять метод `requestMatchers()` в любых конфигурациях авторизации, которые вам может потребоваться написать для требований вашего приложения. Давайте начнем с простого примера.

Мы создаем приложение, которое предоставляет две конечные точки: `/hello` и `/ciao`. Мы хотим сделать так, чтобы только пользователи с ролью ADMIN могли вызывать конечную точку `/hello`. Аналогично нам нужно, чтобы только пользователи с ролью MANAGER могли вызывать конечную точку `/ciao`. Вы найдете этот пример в проекте `ssia-ch8-ex1`. В листинге 8.1 мы определяем класс контроллера.

Листинг 8.1 Определение класса контроллера

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello!";
    }

    @GetMapping("/ciao")
    public String ciao() {
        return "Ciao!";
    }
}
```

В классе конфигурации мы объявляем `InMemoryUserDetailsManager` как наш экземпляр `UserDetailsService` и добавляем двух пользователей с разными ролями. Пользователь John (Джон) имеет роль ADMIN, а Jane (Джейн) – роль MANAGER. Чтобы указать, что только

пользователи с ролью ADMIN могут вызывать конечную точку `/hello` при авторизации запросов, мы используем метод `requestMatchers()`. В листинге 8.2 представлено определение класса конфигурации.

Листинг 8.2 Определение класса конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var manager = new InMemoryUserDetailsManager();

        var user1 = User.withUsername("john")
            .password("12345")
            .roles("ADMIN")
            .build();

        var user2 = User.withUsername("jane")
            .password("12345")
            .roles("MANAGER")
            .build();

        manager.createUser(user1);
        manager.createUser(user2);

        return manager;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.requestMatchers("/hello").hasRole("ADMIN")           ❶
                .requestMatchers("/ciao").hasRole("MANAGER")          ❷
        );

        return http.build();
    }
}
```

❶ Вызываем только путь `/hello`, если у пользователя есть роль ADMIN.

❷ Вызываем только путь `/ciao`, если у пользователя есть роль MANAGER.

Вы можете запустить и протестировать это приложение. Когда вы вызываете конечную точку `/hello` с пользователем `"john"`, вы получаете ожидаемый ответ. Но если вы вызываете ту же конечную точку с пользователем `"jane"`, приложение возвращает статус HTTP 403 Forbidden. Аналогично для вызова конечной точки `/ciao` вы можете использовать только `"jane"`, чтобы получить ответ. Для пользователя `"john"` приложение возвращает статус HTTP 403 Forbidden. Вы можете увидеть примеры вызовов с использованием cURL в следующих фрагментах кода. Чтобы вызвать конечную точку `/hello` от имени Джона, используйте команду

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

Hello!

Чтобы вызвать конечную точку `/hello` от имени Джейн, используйте команду

```
curl -u jane:12345 http://localhost:8080/hello
```

Тело ответа:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

Чтобы вызвать конечную точку `/ciao` от имени Джейн, используйте команду

```
curl -u jane:12345 http://localhost:8080/ciao
```

Тело ответа:

Ciao!

Чтобы вызвать конечную точку `/ciao` от имени Джона, используйте команду

```
curl -u john:12345 http://localhost:8080/ciao
```

Тело ответа:

```
{
  "status":403,
  "error":"Forbidden",

```

```
"message": "Forbidden",  
"path": "/ciao"  
}
```

Если вы теперь добавите любую другую конечную точку в свое приложение, она будет доступна по умолчанию для всех, даже неаутентифицированных пользователей. Предположим, вы добавили новую конечную точку `/hola`, как показано в листинге 8.3.

Листинг 8.3 Добавление новой конечной точки для пути `/hola` в приложение

```
@RestController  
public class HelloController {  
  
    // Часть кода не показана  
  
    @GetMapping("/hola")  
    public String hola() {  
        return "Hola!";  
    }  
}
```

При попытке вызова новой конечной точки вы увидите, что она доступна как с действительным пользователем, так и без него. Фрагменты кода ниже демонстрируют это поведение. Чтобы вызвать конечную точку `/hola` без аутентификации, используйте команду

```
curl http://localhost:8080/hola
```

Тело ответа:

```
Hola!
```

Чтобы вызвать конечную точку `/hola` от имени Джона, используйте команду

```
curl -u john:12345 http://localhost:8080/hola
```

Тело ответа:

```
Hola!
```

Вы можете сделать это поведение более заметным, используя метод `permitAll()`. Для этого нужно добавить метод сопоставления `anyRequest()` в конце цепочки конфигурации для авторизации запроса, как показано в листинге 8.4.

ПРИМЕЧАНИЕ Хорошей практикой является явное указание всех правил. Листинг 8.4 четко и недвусмысленно указы-

вает на намерение разрешить запросы к конечным точкам для всех, за исключением конечных точек /hello и /ciao.

Листинг 8.4 Явная маркировка дополнительных запросов как доступных без аутентификации

```
@Configuration
public class ProjectConfig {

    // Часть кода пропущена

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.requestMatchers("/hello").hasRole("ADMIN")
                .requestMatchers("/ciao").hasRole("MANAGER")
                .anyRequest().permitAll()
        );

        return http.build();
    }
}
```

- ❶ Метод `permitAll()` указывает, что все остальные запросы разрешены без аутентификации.

ПРИМЕЧАНИЕ При использовании сопоставлений для отбора запросов порядок правил должен идти от частного к общему. Вот почему метод `anyRequest()` не может быть вызван перед более конкретным методом `requestMatchers()`.

Отсутствующая и неудавшаяся аутентификация

Если вы разработали конечную точку, доступную для всех, вы можете вызвать ее, не предоставляя имя пользователя и пароль для аутентификации. В этом случае Spring Security не будет выполнять аутентификацию. Однако если вы предоставите имя пользователя и пароль, Spring Security оценит их в процессе аутентификации. Если они неверны (неизвестны системе), аутентификация не будет пройдена, а статус ответа будет 401 Unauthorized. Точнее, если вы вызовете конечную точку /hola для конфигурации, представленной в листинге 8.4, приложение вернет тело Hola!, как и ожидалось, а статус ответа будет 200 OK. Например:

```
curl http://localhost:8080/hola
```

Тело ответа:

Ho!a!

Но если вы вызовете конечную точку с недействительными учетными данными, статус ответа будет 401 Unauthorized. В следующем вызове я использую недействительный пароль:

```
curl -u bill:abcde http://localhost:8080/hola
```

Тело ответа:

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/hola"
}
```

Листинг 8.5 Делаем другие запросы доступными для всех аутентифицированных пользователей

```

@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.requestMatchers("/hello").hasRole("ADMIN")
                .requestMatchers("/ciao").hasRole("MANAGER")
                .anyRequest().authenticated()
        );

        return http.build();
    }
}

```

❶ Все другие запросы доступны только аутентифицированным пользователям.

Вы познакомились с использованием методов сопоставления для именования запросов, для которых вы хотите настроить ограничения авторизации. Теперь нужно более подробно рассмотреть варианты синтаксиса.

В большинстве практических сценариев несколько конечных точек могут иметь одинаковые правила авторизации, поэтому вам не нужно настраивать их для каждой конечной точки. Кроме того, иногда вам нужно указать метод HTTP, а не только путь, как мы делали до сих пор.

В других случаях вам нужно настроить правила для конечной точки только тогда, когда ее путь вызывается с помощью HTTP GET. В этом случае вам нужно будет определить разные правила для HTTP POST и HTTP DELETE. В следующем разделе мы рассмотрим каждый тип метода сопоставления и подробно обсудим эти аспекты.

8.2 Выбор запросов для применения ограничений авторизации

В этом разделе мы подробно рассмотрим настройку сопоставлений запросов. Использование метода `requestMatchers()` является распространенным способом выбора запросов для применения конфигурации авторизации. Я думаю, что у вас будет много возможностей

использовать этот метод для работы с запросами в разрабатываемых вами приложениях.

Этот сопоставитель использует стандартный синтаксис ANT (табл. 8.1) для именования путей. Синтаксис такой же, как тот, который вы используете при написании сопоставлений конечных точек с аннотациями, такими как `@RequestMapping`, `@GetMapping`, `@PostMapping` и т. д. Два метода, которые вы можете использовать для объявления сопоставителей MVC:

- `requestMatchers(HttpMethod method, String... patterns)` – позволяет указать как метод HTTP, к которому применяются ограничения, так и пути. Этот метод полезен, если вы хотите применить разные ограничения для разных методов HTTP для одного и того же пути;
- `requestMatchers(String... patterns)` – проще и легче в использовании, если вам нужно применить только ограничения авторизации на основе путей. Ограничения могут автоматически применяться к любому методу HTTP, используемому с путем.

В этом разделе мы рассмотрим несколько способов использования методов `requestMatchers()`. Чтобы продемонстрировать их, мы начнем с кода приложения, которое предоставляет несколько конечных точек.

Сейчас мы впервые объявим конечные точки, которые можно вызывать с помощью других методов HTTP, помимо GET. Вы могли заметить, что до сих пор я избегал использования других методов HTTP. Причина этого в том, что Spring Security по умолчанию применяет защиту от подделки межсайтовых запросов (cross-site request forgery, CSRF). В главе 9 мы обсудим, как Spring Security купирует эту уязвимость с помощью токенов CSRF. Но чтобы упростить текущий пример и иметь возможность вызывать все конечные точки, включая те, которые были доступны с помощью POST, PUT или DELETE, нам нужно отключить защиту от CSRF в нашем методе `securityFilterChain()`:

```
http.csrf(  
    c -> c.disable()  
);
```

ПРИМЕЧАНИЕ Мы отключаем защиту от CSRF сейчас только для того, чтобы вы могли сосредоточиться на обсуждаемой теме – методах сопоставления. Но не спешите считать это хорошим подходом. В главе 9 мы подробно поговорим о защите от CSRF, предоставляемой Spring Security.

Начнем с определения четырех конечных точек для использования в наших тестах:

- /a с использованием HTTP-метода GET;
- /a с использованием HTTP-метода POST;
- /a/b с использованием HTTP-метода GET;
- /a/b/c с использованием HTTP-метода GET.

С этими конечными точками мы сможем опробовать различные сценарии настройки авторизации. В листинге 8.6 приведены определения этих конечных точек. Вы найдете этот пример в проекте `ssia-ch8-ex2`.

Листинг 8.6 Определение четырех конечных точек, для которых мы настраиваем авторизацию

```
@RestController
public class TestController {

    @PostMapping("/a")
    public String postEndpointA() {
        return "Works!";
    }

    @GetMapping("/a")
    public String getEndpointA() {
        return "Works!";
    }

    @GetMapping("/a/b")
    public String getEnpointB() {
        return "Works!";
    }

    @GetMapping("/a/b/c")
    public String getEnpointC() {
        return "Works!";
    }
}
```

Нам также понадобится пара пользователей с разными ролями. Для простоты мы продолжим использовать `InMemoryUserDetailsManager`. В следующем листинге показано определение `UserDetailsService` в классе конфигурации.

Листинг 8.7 Определение `UserDetailsService`

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var manager = new InMemoryUserDetailsManager(); ❶
    }
}
```

```

var user1 = User.withUsername("john")
    .password("12345")
    .roles("ADMIN")
    .build();

var user2 = User.withUsername("jane")
    .password("12345")
    .roles("MANAGER")
    .build();

manager.createUser(user1);
manager.createUser(user2);

return manager;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
}
}

```

- ❶ Определяем `InMemoryUserDetailsManager` для хранения пользователей.
- ❷ Пользователь Джон имеет роль `ADMIN`.
- ❸ Пользователь Джейн имеет роль `MANAGER`.
- ❹ Не забудьте, что вам также нужно добавить `PasswordEncoder`.

Начнем с первого сценария. Для запросов, выполненных с использованием метода HTTP GET для пути `/a`, приложению необходимо аутентифицировать пользователя. Для того же пути запросы, использующие метод HTTP POST, не требуют аутентификации. Приложение отклоняет все остальные запросы. В следующем листинге показаны конфигурации, которые вам необходимо написать для реализации этого поведения.

Листинг 8.8 Конфигурация авторизации для первого сценария – путь `/a`

```

@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.requestMatchers(HttpMethod.GET, "/a")
                .authenticated()

```



```

        .requestMatchers(HttpMethod.POST, "/a")
        .permitAll()
        .anyRequest()
        .denyAll()
    );

    http.csrf(
        c -> c.disable()
    );

    return http.build();
}
}

```

- ❶ Для запросов пути /a, вызванных с помощью метода HTTP GET, приложению необходимо аутентифицировать пользователя.
- ❷ Разрешаем вызывать запросы пути /a с помощью метода HTTP POST любому желающему.
- ❸ Запрещаем любые другие запросы на любой другой путь.
- ❹ Отключаем CSRF, чтобы разрешить вызов пути /a с помощью метода HTTP POST.

В следующих фрагментах кода мы анализируем результаты вызовов конечных точек для конфигурации, представленной в листинге 8.8. Для вызова пути /a с использованием HTTP-метода POST без аутентификации используйте следующую команду cURL:

```
curl -XPOST http://localhost:8080/a
```

Тело ответа:

Works!

При вызове пути /a с использованием HTTP GET без аутентификации используйте команду

```
curl -XGET http://localhost:8080/a
```

Ответ:

```

{
  "status": 401,
  "error": "Unauthorized",
  "message": "Unauthorized",
  "path": "/a"
}

```

Если вы хотите получить ответ успешного вызова, вам необходимо выполнить аутентификацию с допустимым пользователем. Для следующего вызова

```
curl -u john:12345 -XGET http://localhost:8080/a
```

тело ответа:

Works!

Однако Джону не разрешено вызывать путь /a/b, поэтому аутентификация с его учетными данными для этого вызова генерирует статус 403 Forbidden:

```
curl -u john:12345 -XGET http://localhost:8080/a/b
```

Ответ:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/a/b"
}
```

Благодаря этому примеру вы теперь знаете, как различать запросы на основе метода HTTP. Но что, если несколько путей имеют одинаковые правила авторизации? Конечно, мы можем перечислить все пути, для которых мы применяем правила авторизации; однако если у нас слишком много путей, это делает чтение кода неудобным. Кроме того, мы можем знать с самого начала, что группа путей с одинаковым префиксом всегда имеет одинаковые правила авторизации. Мы хотим убедиться, что добавление нового пути в ту же группу также не изменит конфигурацию авторизации. Чтобы справиться с этими случаями, мы используем *выражения путей* (path expression). Рассмотрим следующий пример.

В текущем проекте нам нужно убедиться, что одни и те же правила применяются ко всем запросам для путей, начинающихся с /a/b. В нашем случае это пути /a/b и /a/b/c. Для этого мы используем оператор **. Вы можете найти этот пример в проекте ssia-ch8-ex3.

Листинг 8.9 Изменения в классе конфигурации для нескольких путей

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public void configure(HttpSecurity http)
        throws Exception {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.requestMatchers( "/a/b/**").authenticated() ❶
        )
    }
}
```

```

        .anyRequest().permitAll();

    });

    http.csrf(
        c -> c.disable()
    );

    return http.build();
}
}

```

❶ Выражение `/a/b/**` обозначает все пути с префиксом `/a/b`.

С конфигурацией, приведенной в листинге 8.9, вы можете вызвать путь `/a` без аутентификации, но для всех путей с префиксом `/a/b` приложению необходимо аутентифицировать пользователя. В следующих фрагментах кода представлены результаты вызова конечных точек `/a`, `/a/b` и `/a/b/c`. Чтобы вызвать путь `/a` без аутентификации, используйте команду

```
curl http://localhost:8080/a
```

Тело ответа:

Works!

Чтобы вызвать путь `/a/b` без аутентификации, используйте команду

```
curl http://localhost:8080/a/b
```

Ответ:

```

{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/a/b"
}

```

Чтобы вызвать путь `/a/b/c` без аутентификации, используйте команду

```
curl http://localhost:8080/a/b/c
```

Ответ:

```

{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/a/b"
}

```

Как показано в предыдущих примерах, оператор `**` обозначает произвольное количество имен путей. Вы можете использовать его, как мы сделали в последнем примере, чтобы сопоставлять запросы с путями, имеющими известный префикс. Вы также можете использовать его в середине пути, чтобы обозначить любые варианты имени пути или ссылаться на пути, заканчивающиеся определенным шаблоном, например `/a/**/c`. Поэтому `/a/**/c` будет соответствовать не только `/a/b/c`, но также `/a/b/d/c` и `a/b/c/d/e/c` и т. д. Если вы хотите указать только одно имя пути, то используйте один символ `*`. Например, `a/*/c` будет соответствовать `a/b/c` и `a/d/c`, но не `a/b/d/c`.

Поскольку мы обычно используем переменные пути, они могут быть полезны для применения правил авторизации к таким запросам. Вы даже можете применять правила, ссылающиеся на значение переменной пути. Вы помните, как в разделе 8.1 говорилось о методе `denyAll()` и ограничении всех запросов?

Давайте теперь перейдем к более реалистичной демонстрации того, что вы узнали. У нас есть конечная точка с переменной пути, и мы хотим отклонить все запросы, которые используют значение переменной, содержащее любые другие символы, кроме цифр. Вы можете найти этот пример в проекте `ssia-ch8-ex4`. В листинге 8.10 представлен код контроллера.

Листинг 8.10 Определение конечной точки с переменной пути в классе контроллера

```
@RestController
public class ProductController {

    @GetMapping("/product/{code}")
    public String productCode(@PathVariable String code) {
        return code;
    }
}
```

В следующем листинге показано, как настроить авторизацию так, чтобы всегда разрешались только вызовы, имеющие заданное цифровое значение, а все остальные вызовы запрещались.

Листинг 8.11 Настройка авторизации для разрешения только определенных цифр

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
```

```

http.httpBasic(Customizer.withDefaults());

http.authorizeHttpRequests(
    c -> c.requestMatchers("/product/{code:^(0-9)*$}")
        .permitAll()
        .anyRequest()
        .denyAll()

);

return http.build();
}
}

```

❶ Регулярное выражение применимо к строкам любой длины, содержащим любую цифру.

ПРИМЕЧАНИЕ При использовании выражений параметров с регулярным выражением убедитесь, что между именем параметра, двоеточием (:) и регулярным выражением нет пробела, как показано в листинге.

Запустив этот пример, вы можете увидеть результат, представленный в следующих фрагментах кода. Приложение обрабатывает вызов лишь в том случае, если значение переменной пути содержит только цифры. Чтобы вызвать конечную точку, используя значение 1234a, используйте команду

```
curl http://localhost:8080/product/1234a
```

Ответ:

```

{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/product/1234a"
}

```

Чтобы вызвать конечную точку со значением 12345, используйте команду

```
curl http://localhost:8080/product/12345
```

Ответ:

```
12345
```

Мы подробно обсудили несколько примеров проверки и сопоставления запросов с помощью метода `requestMatchers()`. Таблица 8.1 – это сводный перечень выражений пути, используемых

в этом разделе. Вы можете обратиться к ней позже, когда понадобится вспомнить любое из них.

Таблица 8.1. Общие выражения, используемые для сопоставления пути с помощью сопоставлений MVC

Выражение	Описание
/a	Только путь /a
/a/*	Оператор * заменяет одно имя пути. В этом случае он соответствует /a/b или /a/c, но не /a/b/c
/a/**	Оператор ** заменяет несколько имен путей. В этом случае /a, /a/b и /a/b/c являются соответствием для этого выражения
/a/{param}	Это выражение применяется к пути /a с заданным параметром пути
/a/{param:regex}	Это выражение применяется к пути /a с заданным параметром пути только в том случае, если значение параметра соответствует заданному регулярному выражению

8.3 Использование регулярных выражений с сопоставителями запросов

В этом разделе говорится о *регулярных выражениях* (regex). Вы уже должны знать, что такое регулярные выражения, но вам не нужно быть экспертом в этой области. Любая из книг, рекомендованных на сайте <https://www.regular-expressions.info/books.html>, является отличным источником информации, из которого вы можете узнать больше о предмете. Для написания регулярных выражений я также часто использую онлайн-генераторы, такие как <https://regexr.com/> (рис. 8.1).

В разделах 8.2 и 8.3 вы узнали, что в большинстве случаев для указания на запросы, к которым вы применяете конфигурацию авторизации, можно использовать синтаксис выражений пути. Однако в некоторых случаях у вас могут быть более конкретные требования, и вы не сможете удовлетворить их с помощью выражений пути. Например, такое требование: «Отклонять все запросы, если путь содержит определенные символы или знаки». Для этих сценариев вам нужно использовать более мощный инструмент, такой как регулярное выражение.

Регулярные выражения пригодны для представления любого формата строки, поэтому они предлагают безграничные возможности в вопросе реализации условий. Однако у них есть недостаток – их трудно читать, даже применительно к простым сценариям. По этой причине многие разработчики предпочитают использовать выражения пути и прибегают к регулярным выражениям, только когда не осталось других вариантов. Чтобы создать сопоставитель запросов регулярных выражений, вы можете использовать метод `requestMatchers()` с реализацией `RegexRequestMatcher` в качестве параметра.

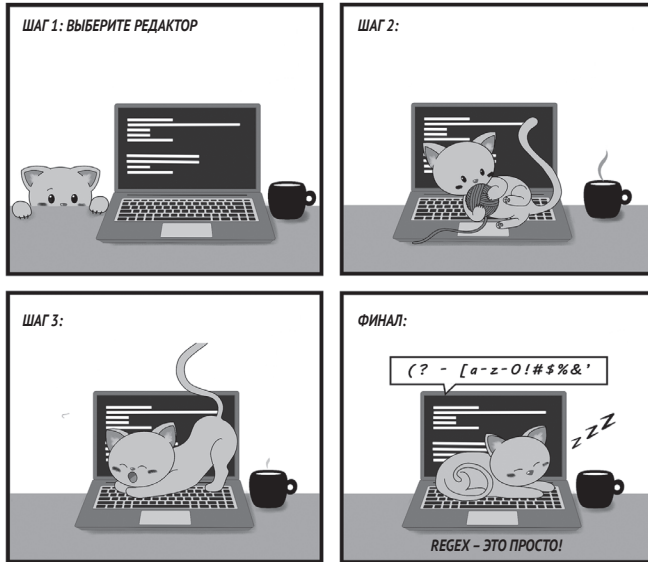


Рис. 8.1 Позволить своему коту играть на клавиатуре – не лучший способ генерации регулярных выражений. Чтобы научиться генерировать регулярные выражения, вы можете использовать онлайн-генератор, такой как <https://regexr.com/>

Для демонстрации работы сопоставителя регулярных выражений мы создадим приложение, которое поставляет видеоконтент своим пользователям. Приложение, которое транслирует видео, получает свой контент, вызывая конечную точку `/video/{country}/{language}`. Допустим, приложение получает страну и язык в двух переменных пути, откуда пользователь делает запрос. Мы считаем, что любой аутентифицированный пользователь может получить видеоконтент, если запрос поступает из США, Канады или Великобритании либо если он использует английский язык.

Код этого примера вы найдете в проекте `ssia-ch8-ex5`. Конечная точка, которую нам нужно защитить, имеет две переменные пути, как показано в листинге 8.12. Такое условие отбора сложно реализовать с помощью сопоставителей запросов.

Листинг 8.12 Определение конечной точки для класса контроллера

```
@RestController
public class VideoController {

    @GetMapping("/video/{country}/{language}")
    public String video(@PathVariable String country,
                       @PathVariable String language) {
        return "Video allowed for " + country + " " + language;
    }
}
```

Для условия с одной переменной пути мы можем написать регулярное выражение непосредственно в выражении пути. Я приводил такой пример в разделе 8.2, но не стал вдаваться в подробности в то время, потому что мы не обсуждали регулярные выражения.

Предположим, что у вас есть конечная точка `/email/{email}`. Вы хотите применить правило с использованием сопоставителя только к запросам, которые отправляют адрес, заканчивающийся на `.com`, в качестве значения параметра `email`. В этом случае вы пишете сопоставитель запроса, как представлено в следующем фрагменте кода. Полный код примера доступен в проекте `ssia-ch8-ex6`:

```
http.authorizeHttpRequests(
    c -> c.requestMatchers("/email/{email:^(?:(?:.+\\.com)}") ).permitAll()
    .anyRequest().denyAll();
);
```

Если вы протестируете такое ограничение, то увидите, что приложение принимает только адреса электронной почты, заканчивающиеся на `.com`. Например, чтобы вызвать конечную точку `jane@example.com`, вы вводите команду

```
curl http://localhost:8080/email/jane@example.com
```

Тело ответа:

```
Allowed for email jane@example.com
```

А чтобы вызвать конечную точку `jane@example.net`, вы используете команду

```
curl http://localhost:8080/email/jane@example.net
```

Тело ответа:

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/email/jane@example.net
}
```

Простота этого подхода еще больше объясняет, почему мы редко сталкиваемся с сопоставлениями на основе регулярных выражений. Однако, как я уже говорил ранее, условия иногда бывают очень замысловатыми. Вам будет удобнее использовать сопоставители регулярных выражений, если вы столкнетесь с условиями отбора наподобие следующих:

- конкретные конфигурации для всех путей, содержащих телефонные номера или адреса электронной почты;

- конкретные конфигурации для всех путей, имеющих определенный формат, включая все, что отправляется через все переменные пути.

Вернемся к нашему примеру сопоставителей регулярных выражений (ssia-ch8-ex6): когда вам нужно написать более сложное правило, в конечном итоге ссылающееся на большее количество шаблонов путей и несколько значений переменных пути, проще написать сопоставитель регулярных выражений. В листинге 8.13 представлено определение для класса конфигурации, который использует сопоставитель регулярных выражений для пути `/video/{country}/{language}`. Мы также добавляем двух пользователей с разными полномочиями для проверки реализации.

Листинг 8.13 Класс конфигурации, использующий сопоставитель регулярных выражений

```
@Configuration
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var uds = new InMemoryUserDetailsManager();

        var u1 = User.withUsername("john")
            .password("12345")
            .authorities("read")
            .build();

        var u2 = User.withUsername("jane")
            .password("12345")
            .authorities("read", "premium")
            .build();

        uds.createUser(u1);
        uds.createUser(u2);

        return uds;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http.httpBasic(Customizer.withDefaults());
    }
}
```

```

    http.authorizeHttpRequests(
        c -> c.regexMatchers(".*(us|uk|ca)+/(en|fr).*") ❶
            .authenticated()
            .anyRequest()
            .hasAuthority("premium"); ❷
    );
}
}

```

❶ Мы используем регулярное выражение для сопоставления путей, для которых пользователю нужно только пройти аутентификацию.

❷ Настраиваем другие пути, для которых пользователю нужен премиум-доступ.

Запуск и тестирование конечных точек подтверждают, что приложение правильно применило конфигурации авторизации. Пользователь Джон может вызвать конечную точку с кодом страны US и языком en, но он не может вызвать конечную точку для кода страны FR и языка fr из-за настроенных нами ограничений. Вызов конечной точки /video и аутентификация пользователя john для региона США и английского языка выглядит следующим образом:

```
curl -u john:12345 http://localhost:8080/video/us/en
```

Тело ответа:

```
Video allowed for us en
```

Вызов конечной точки /video и аутентификация пользователя john для региона FR и французского языка выглядит следующим образом:

```
curl -u john:12345 http://localhost:8080/video/fr/fr
```

Тело ответа:

```

{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/video/fr/fr"
}

```

Пользователь jane, имеющий премиум-доступ, успешно выполняет оба вызова. Для первого вызова это команда

```
curl -u jane:12345 http://localhost:8080/video/us/en
```

и тело ответа:

```
Video allowed for us en
```

Для второго вызова это команда

```
curl -u jane:12345 http://localhost:8080/video/fr/fr
```

и тело ответа:

```
Video allowed for fr fr
```

Регулярные выражения – это мощные инструменты. Вы можете использовать их при описании пути для любого заданного требования. Однако поскольку регулярные выражения трудно читать и они склонны становиться довольно длинными, их следует применять в последнюю очередь. Используйте их только в том случае, если стандартные выражения пути не позволяют решить задачу.

В этом разделе я использовал самый простой пример, который только можно представить, чтобы необходимое регулярное выражение было коротким. Но в более сложных случаях регулярное выражение может стать намного длиннее. Конечно, вы найдете экспертов, которые скажут, что любое регулярное выражение легко читается. Например, регулярное выражение, используемое для сопоставления адреса электронной почты, может выглядеть, как показано в следующем фрагменте кода. Легко ли вам прочитать и понять его?

```
(?:[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\.(?:[a-z0-9!#$%&'*/+=?^_`{|}~-]
+)*|"(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\\[\x01-
\x09\x0b\x0c\x0e-\x7f])*")@(?:(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0
-9](?:[a-z0-9-]*[a-z0-9])?)|[(?:(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0
-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)|[a-z0-9-]*[a-z0
-9](?:[a-z0-9-]*[a-z0-9])?|[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\\[\x01-
\x09\x0b\x0c\x0e-\x7f])+)\])
```

Заключение

- На практике для разных запросов применяются разные правила авторизации.
- Запросы, для которых настроены правила авторизации, определяются на основе пути и метода HTTP. Для этого используется метод `requestMatchers()`.
- Когда условия слишком сложны для реализации с помощью выражений пути, вы можете решить задачу с помощью более мощных регулярных выражений.

Настройка защиты от CSRF

Темы этой главы:

- что такое атаки CSRF;
- как работает защита от CSRF;
- настройка защиты от CSRF.

Из предыдущих глав вы узнали о цепочке фильтров и ее назначении в архитектуре Spring Security. Мы работали над несколькими примерами в главе 5, где настраивали цепочку фильтров. Но Spring Security также добавляет свои собственные фильтры в цепочку. В этой главе обсуждается фильтр, который настраивает защиту от CSRF (cross-site request forgery, подделка межсайтовых запросов). Вы научитесь настраивать фильтры, чтобы они идеально подходили для ваших сценариев.

Вы, вероятно, заметили, что в большинстве примеров до сих пор мы реализовывали наши конечные точки только с протоколом HTTP GET. Более того, когда нам нужно было настроить HTTP POST, нам также приходилось добавлять дополнительную инструкцию в конфигурацию для отключения защиты от CSRF. Причина, по которой вы не можете напрямую вызвать конечную точку с HTTP POST, заключается в защите от CSRF, которая включена по умолчанию в Spring Security.

Далее мы обсудим защиту от CSRF и ее использование в ваших приложениях. CSRF – это распространенный тип атаки, и уязвимые приложения могут заставить пользователей выполнять нежелательные действия в веб-приложении после аутентификации. Вы ведь не хотите, чтобы разрабатываемые вами приложения были уязвимы к CSRF и позволяли злоумышленникам обманом заставлять ваших пользователей выполнять нежелательные действия?

Поскольку важно понимать, как смягчить эти уязвимости, мы начнем с обзора того, что такое атаки CSRF и как они работают. Затем обсудим механизм специальных токенов, который Spring Security использует для борьбы с уязвимостями CSRF. Потом вы научитесь получать токен и использовать его для вызова конечной точки с помощью метода HTTP POST. Мы проверим этот способ посредством небольшого приложения с конечными точками REST. Вы увидите на примере, как Spring Security реализует свой механизм токенов CSRF, а затем я расскажу об использовании защиты в реальных приложениях. Наконец, вы узнаете о конфигурациях механизма токенов CSRF в Spring Security.

9.1 *Как работает защита CSRF в Spring Security*

В этом разделе говорится о том, как Spring Security реализует защиту от CSRF. Сначала важно понять базовый механизм защиты. Я часто сталкиваюсь со случаями, когда непонимание принципа работы защиты от CSRF приводит к ее неправильному использованию, либо потому что она отключена в ситуациях, где она должна быть включена, либо наоборот. Как и любую другую функцию в фреймворке, вы должны использовать ее правильно, чтобы извлечь максимальную пользу для приложения.

Например, рассмотрим сценарий, изображенный на рис. 9.1. У себя на работе вы используете веб-приложение для хранения и управления файлами. С помощью этого инструмента в веб-интерфейсе вы можете добавлять новые файлы, добавлять новые версии для своих записей и даже удалять их. Вы получаете электронное письмо с просьбой открыть страницу по определенной причине (например, акция в вашем любимом магазине). Вы открываете страницу, но она пуста или перенаправляет вас на известный веб-сайт (сайт вашего любимого магазина). Вы возвращаетесь к работе и обнаруживаете, что все ваши файлы исчезли!

Что случилось? Вы вошли в свое рабочее приложение, чтобы управлять своими файлами. Когда вы добавляете, изменяете или удаляете файл, веб-страница, с которой вы взаимодействуете, вызы-

вает определенные конечные точки на сервере для выполнения этих операций. Когда вы открыли чужую страницу, нажав на неизвестную ссылку в письме, эта страница вызвала бэкэнд вашего приложения и выполнила действия от вашего имени (т. е. удалила ваши файлы).

Это могло произойти, потому что вы уже выполнили вход в систему. Сервер полагал, что команды исходили от вас. Вы можете подумать, что постороннему человеку трудно убедить вас нажать на ссылку из чужого письма или сообщения, но поверьте мне, это случается сплошь и рядом. Большинство пользователей веб-приложений не знают о рисках безопасности. Поэтому будет разумнее, если вы – знающий все трюки злоумышленников – защитите своих пользователей и создадите безопасные приложения, а не будете полагаться на то, что пользователи ваших приложений защитят себя сами.

Атаки типа CSRF предполагают, что пользователь вошел в веб-приложение. Злоумышленник обманом заставляет пользователей открыть страницу, содержащую скрипты, которые выполняют действия в том же приложении, с которым работал пользователь. Поскольку пользователь уже вошел в систему (как мы предполагали с самого начала), поддельный код теперь может выдавать себя за пользователя и выполнять действия от его имени.

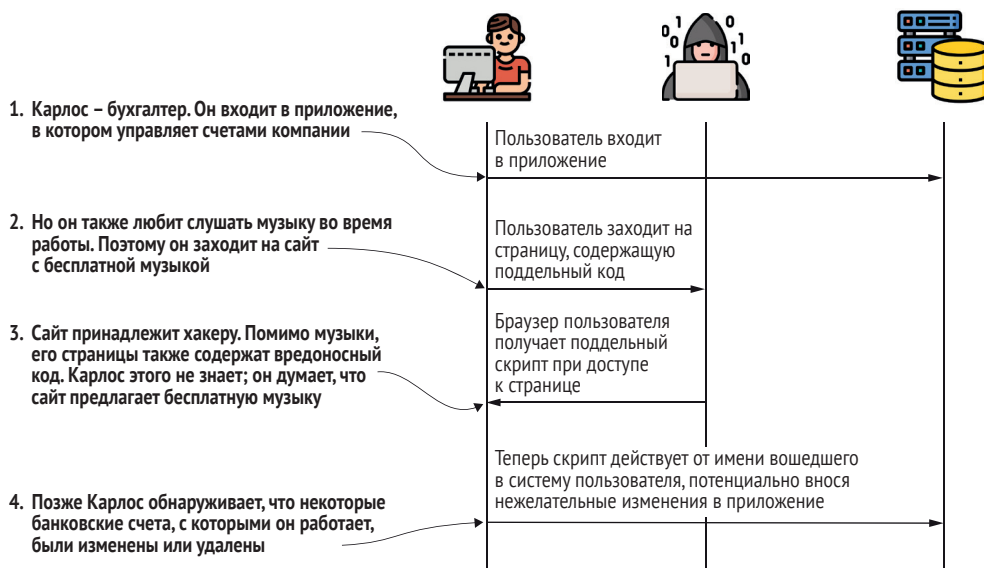


Рис. 9.1 После входа в учетную запись пользователь открывает страницу, содержащую поддельный код. Этот код выдает себя за пользователя и может выполнять нежелательные действия от его имени

Как защитить пользователей от таких сценариев? Защита от CSRF устроена таким образом, чтобы гарантировать, что только интер-

фейс веб-приложений может выполнять *операции мутации* (по соглашению, это HTTP-методы, отличные от GET, HEAD, TRACE или OPTIONS). Тогда чужая страница, например в нашем примере, не сможет действовать от имени пользователя.

Как этого добиться? Вы точно знаете, что перед выполнением любого действия, которое может изменить данные, пользователь должен отправить запрос с помощью HTTP GET, чтобы увидеть веб-страницу хотя бы один раз. Когда это происходит, приложение генерирует уникальный токен. Теперь приложение принимает только запросы на операции мутации (POST, PUT, DELETE и т. д.), которые содержат это уникальное значение в заголовке.

Приложение считает, что наличие токена в запросе является доказательством того, что мутирующий запрос выполняет именно приложение, а не другая система. Любая страница, содержащая мутирующие вызовы, такие как POST, PUT, DELETE и т. д., должна получать ответ через токен CSRF, и страница должна использовать этот токен при выполнении мутирующих вызовов.

Начальной точкой защиты от CSRF является фильтр в цепочке фильтров, называемый `CsrfFilter`. Он перехватывает запросы и пропускает только те из них, которые используют методы HTTP GET, HEAD, TRACE и OPTIONS. Для всех остальных запросов фильтр ожидает получить заголовок, содержащий токен. Если этот заголовок не существует или содержит неверное значение токена, приложение отклоняет запрос и возвращает статус ответа HTTP 403 `Forbidden`.

Что это за токен и откуда он берется? Это всего лишь строка символов. Вы должны добавить токен в заголовок запроса, если используете любой метод, кроме GET, HEAD, TRACE или OPTIONS. Если вы этого не сделаете, приложение не примет запрос, как показано на рис. 9.2.

`CsrfFilter` (рис. 9.3) использует компонент с именем `CsrfTokenRepository` для управления значениями токенов CSRF, который генерирует новые токены, хранит токены и в конечном итоге делает их недействительными. По умолчанию `CsrfTokenRepository` сохраняет токен в сеансе HTTP и генерирует токены как случайные строковые значения. В большинстве случаев этого достаточно, но, как вы узнаете в разделе 9.3, вы можете использовать собственную реализацию `CsrfTokenRepository`, если стандартная не соответствует требованиям, которые вам нужно реализовать.

В этом разделе я с помощью рисунков объяснил, как работает защита CSRF в Spring Security. Теперь я хочу подкрепить ваше понимание небольшим примером кода. Этот код является частью проекта с именем `ssia-ch9-ex1`. Давайте создадим приложение, которое предоставляет две конечные точки. Мы можем вызвать одну из них с помощью HTTP GET, а другую с помощью HTTP POST.

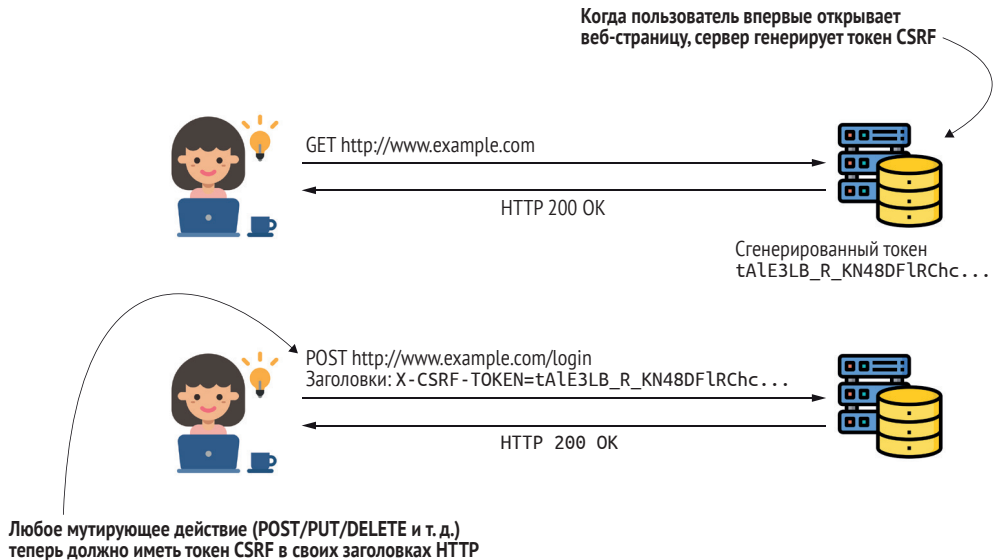


Рис. 9.2 Чтобы сделать запрос POST, клиенту необходимо добавить заголовок, содержащий токен CSRF. Приложение генерирует токен CSRF при загрузке страницы (через запрос GET), и токен добавляется ко всем запросам, которые могут быть сделаны с загруженной страницы. Таким образом, только легально загруженная страница может делать мутирующие запросы

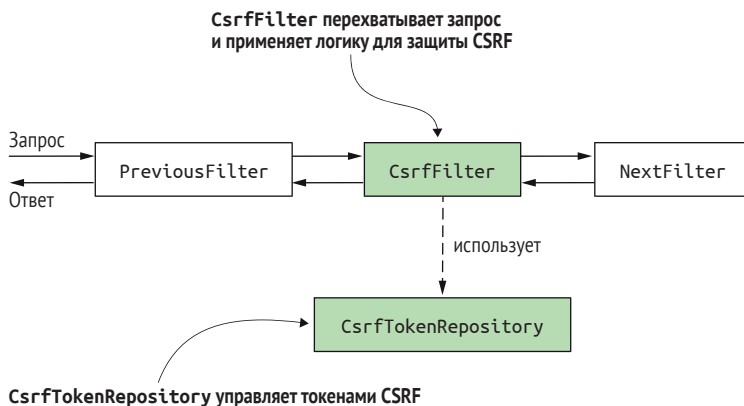


Рис. 9.3 CsrfFilter – один из фильтров в цепочке фильтров. Он получает запрос и в конечном итоге пересылает его следующему фильтру в цепочке. Для управления токенами CsrfFilter использует CsrfTokenRepository

Как вы уже знаете, вы не можете вызывать конечные точки с помощью POST напрямую, не отключив защиту CSRF. В этом примере вы узнаете, как вызвать конечную точку POST без отключения защиты. Вам нужно получить токен CSRF, чтобы вы могли добавить его в заголовок вызова, что вы и делаете с помощью HTTP POST.

Как вы узнали из этого примера, CsrfFilter добавляет сгенерированный токен CSRF к атрибуту HTTP-запроса с именем `_csrf`

(рис. 9.4). После срабатывания фильтра `CsrfFilter` мы можем извлечь атрибут и взять из него значение токена. Для этого небольшого приложения мы решили добавить пользовательский фильтр после `CsrfFilter`, как вы научились делать в главе 5. Этот пользовательский фильтр для печати токена CSRF в консоли приложения, который приложение генерирует, когда мы вызываем конечную точку с помощью HTTP GET. Затем мы можем скопировать значение токена из консоли и использовать его для выполнения вызова мутации с помощью HTTP POST. В листинге 9.1 показано определение класса контроллера с двумя конечными точками, которые мы используем для теста.

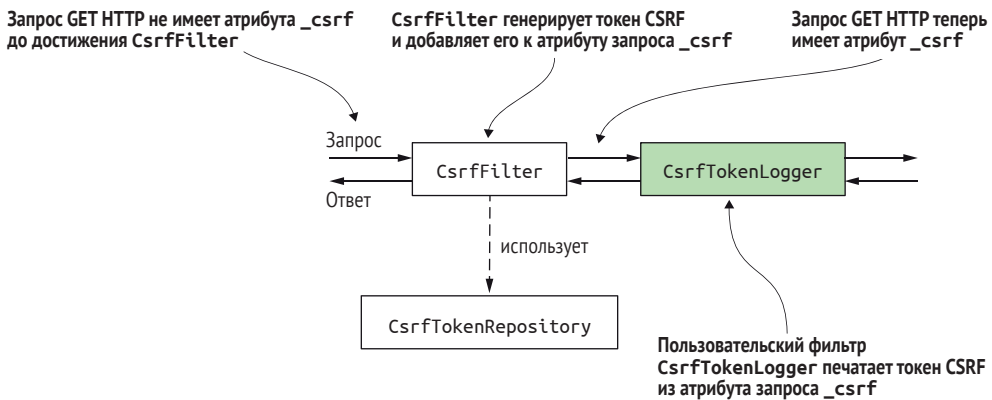


Рис. 9.4 Вставка `CsrfTokenLogger` (выделено) после `CsrfFilter` позволяет ему извлекать значение токена из атрибута `_csrf` запроса, куда `CsrfFilter` его помещает. `CsrfTokenLogger` выводит токен CSRF в консоль приложения, откуда его можно скопировать для формирования запроса HTTP POST к конечной точке

Листинг 9.1 Класс контроллера с двумя конечными точками

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String getHello() {
        return "Get Hello!";
    }

    @PostMapping("/hello")
    public String postHello() {
        return "Post Hello!";
    }
}
```

Листинг 9.2 определяет пользовательский фильтр, который мы используем для вывода значения токена CSRF в консоль. Я назвал

пользовательский фильтр `CsrfTokenLogger`. При вызове фильтр получает значение токена CSRF из атрибута запроса `_csrf` и выводит его в консоль. Имя атрибута запроса `_csrf` – это указатель на место, где `CsrfFilter` устанавливает значение сгенерированного токена CSRF как экземпляр класса `CsrfToken`. Этот экземпляр `CsrfToken` содержит строковое значение токена CSRF. Вы можете получить его, вызвав метод `getToken()`.

Листинг 9.2 Определение класса пользовательского фильтра

```
public class CsrfTokenLogger implements Filter {

    private Logger logger =
        Logger.getLogger(CsrfTokenLogger.class.getName());

    @Override
    public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain filterChain)
        throws IOException, ServletException {

        CsrfToken o =
            (CsrfToken) request.getAttribute("_csrf");    ❶

        logger.info("CSRF token " + token.getToken());

        filterChain.doFilter(request, response);
    }
}
```

❶ Берем значение токена из атрибута запроса `_csrf` и выводим его в консоль.

В классе конфигурации мы добавляем пользовательский фильтр. Следующий листинг представляет класс конфигурации. Обратите внимание, что я не отключаю защиту CSRF в листинге.

Листинг 9.3 Добавление пользовательского фильтра в класс конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain configure(HttpSecurity http)
        throws Exception {

        http.addFilterAfter(
            new CsrfTokenLogger(), CsrfFilter.class)
            .authorizeHttpRequests(
                c -> c.anyRequest().permitAll())
    }
}
```

```

    );
    return http.build();
}
}

```

Теперь мы можем протестировать конечные точки. Начнем с вызова конечной точки с помощью HTTP GET. Поскольку реализация интерфейса `CsrfTokenRepository` по умолчанию использует сеанс HTTP для хранения значения токена на стороне сервера, нам также нужно запомнить идентификатор сеанса. По этой причине я добавляю флаг `-v` к вызову, чтобы получить более детальный ответ, включающий идентификатор сеанса. Вызов конечной точки

```
curl -v http://localhost:8080/hello
```

возвращает следующий ответ (сокращен):

```

...
< Set-Cookie: JSESSIONID=21ADA55E10D70BA81C338FFBB06B0206;
...
Get Hello!

```

После запроса в консоли приложения вы можете найти строку журнала, содержащую токен CSRF:

```
INFO 21412 --- [nio-8080-exec-1] c.l.ssia.filters.CsrfTokenLogger :
CSRF token tAlE3LB_R_KN48DFlRChc...
```

ПРИМЕЧАНИЕ Вы можете спросить, как клиенты получают токен CSRF. Они не могут ни угадать его, ни прочитать в журналах сервера. Я разработал этот пример, чтобы вам было проще понять, как работает реализация защиты CSRF. Как вы увидите в разделе 9.2, бэкенд-приложение несет ответственность за добавление значения токена CSRF в ответ HTTP, который будет использовать клиент.

Если вы вызываете конечную точку с помощью метода HTTP POST без предоставления токена CSRF, статус ответа – 403 Forbidden, как показывает результат выполнения этой команды:

```
curl -XPOST http://localhost:8080/hello
```

Тело ответа:

```

{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}

```

Но если вы предоставите правильное значение токена CSRF, вызов будет успешным. Вам также необходимо указать идентификатор сеанса (JSESSIONID), поскольку реализация `CsrfTokenRepository` по умолчанию сохраняет значение токена CSRF в сеансе:

```
curl -X POST http://localhost:8080/hello  
-H 'Cookie: JSESSIONID=21ADA55E10D70BA81C338FFBB06B0206'  
-H 'X-CSRF-TOKEN: tA1E3LB_R_KN48DF1Rchc...'
```

Тело ответа:

Post Hello!

9.2 Использование защиты от CSRF на практике

В этом разделе мы обсудим применение защиты от CSRF в практических ситуациях. Вы уже знаете, как работает защита в Spring Security, и теперь осталось разобраться, где и как ее следует использовать в реальном мире. Какие приложения нуждаются в защите от CSRF?

Мы используем защиту от CSRF для веб-приложений, работающих в браузере, где следует ожидать, что операции мутации могут быть выполнены браузером, загружающим отображаемое содержимое приложения. Самый очевидный пример, который я могу здесь привести, – это простое веб-приложение, разработанное на стандартном потоке Spring MVC. Мы уже создали такое приложение при обсуждении входа с помощью формы в главе 6, и это веб-приложение фактически использовало защиту от CSRF. Вы заметили, что операция входа в этом приложении использовала HTTP POST? Тогда почему мы ничего не делали с CSRF в явном виде? Дело в том, что мы не применяли там никакой операции мутации.

При входе с помощью формы по умолчанию Spring Security правильно применяет за нас защиту от CSRF. Фреймворк заботится о добавлении токена CSRF в запрос на вход. Давайте теперь разработаем похожее приложение, чтобы более подробно рассмотреть, как работает защита. Как показано на рис. 9.5, в этом разделе мы

- создадим пример веб-приложения с формой входа;
- посмотрим, как реализация входа по умолчанию использует токены CSRF;
- реализуем вызов HTTP POST с главной страницы.

На этом примере вы убедитесь, что вызов HTTP POST не будет работать, пока мы правильно не используем токены CSRF, и узнаете, как применять токены CSRF в форме на такой веб-странице. Чтобы реализовать это приложение, мы начнем с создания нового проек-

та Spring Boot. Вы можете найти этот пример в проекте ssia-ch9-ex2. Следующий фрагмент кода представляет необходимые зависимости:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

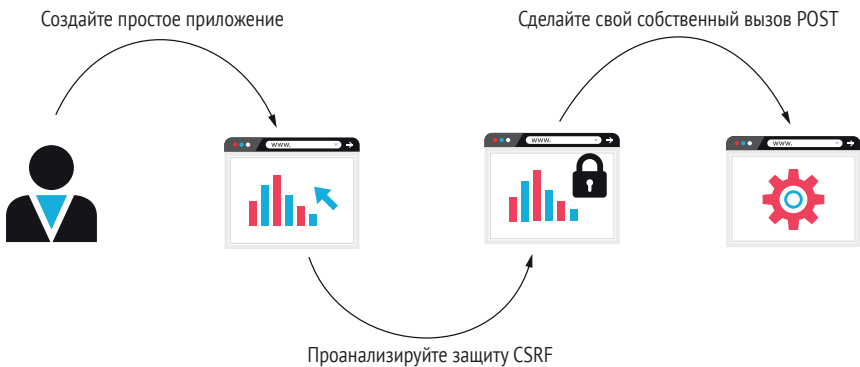


Рис. 9.5 План действий. В этом разделе мы начнем с создания и анализа простого приложения, чтобы понять, как Spring Security применяет защиту от CSRF, а затем напишем собственный вызов POST

Разумеется, нам еще нужно будет создать форму входа и хотя бы одного пользователя. В листинге 9.4 показан класс конфигурации, который определяет `UserDetailsService`, добавляет пользователя и настраивает метод `formLogin`.

Листинг 9.4 Определение класса конфигурации

```
public class ProjectConfig {

    @Bean
    public UserDetailsService uds() {
        var uds = new InMemoryUserDetailsManager();

        var u1 = User.withUsername("mary")
            .password("12345")
            .authorities("READ")
            .build();
    }
}
```

```

        uds.createUser(u1);
        return uds;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.formLogin(
            c -> c.defaultSuccessUrl("/main", true)
        );

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );

        return http.build();
    }
}

```

- ❶ Добавляем бин `UserDetailsService`, управляющий одним пользователем для тестирования приложения.
- ❷ Добавляем `PasswordEncoder`.
- ❸ Создаем бин типа `SecurityFilterChain` для установки метода аутентификации входа в форму и указываем, что только аутентифицированные пользователи могут получить доступ к любой из конечных точек.

Мы добавляем класс контроллера для главной страницы в пакет с именем `controllers` и в файл `main.html` в папке `resources/templates` проекта Maven. Файл `main.html` может оставаться пустым на данный момент, поскольку при первом выполнении приложения мы фокусируемся только на том, как страница входа использует токены CSRF. В следующем листинге представлен класс `MainController`, который обслуживает главную страницу.

Листинг 9.5 Определение класса `MainController`

```

@Controller
public class MainController {

    @GetMapping("/main")
    public String main() {
        return "main.html";
    }
}

```

После запуска приложения вы можете получить доступ к странице входа по умолчанию. Если вы проверите форму с помощью функции

inspect element вашего браузера, то увидите, что реализация формы входа по умолчанию отправляет токен CSRF. Вот почему ваш вход работает с включенной защитой от CSRF, даже если он использует запрос HTTP POST! На рис. 9.6 показано, как форма входа отправляет токен CSRF через скрытый ввод.

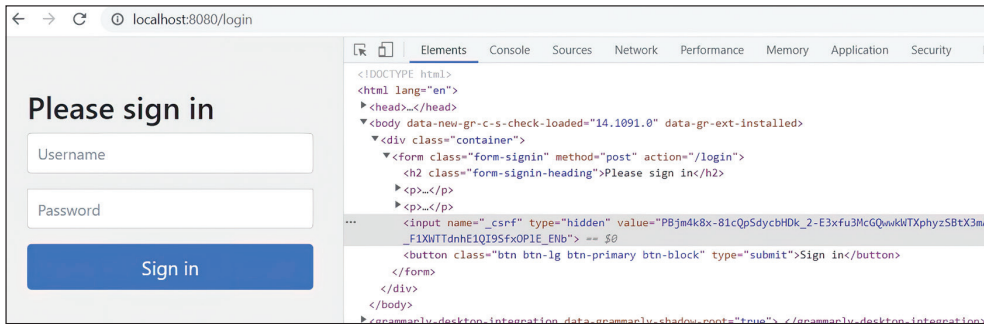


Рис. 9.6 Форма входа по умолчанию использует скрытый ввод для отправки токена CSRF в запросе. Вот почему запрос входа, который использует метод HTTP POST, работает с включенной защитой CSRF

Но как насчет разработки наших собственных конечных точек, которые используют POST, PUT или DELETE в качестве методов HTTP? В этом случае мы сами должны позаботиться об отправке значения токена, если включена защита от CSRF. Чтобы проверить это, добавим конечную точку с использованием HTTP POST в наше приложение. Мы вызываем эту конечную точку с главной страницы и создаем для нее второй контроллер, называемый `ProductController`. В этом контроллере мы определяем конечную точку `/product/add`, которая использует HTTP POST. Затем мы используем форму на главной странице для вызова этой конечной точки. В листинге 9.6 показано определение класса `ProductController`.

Листинг 9.6 Определение класса `ProductController`

```
@Controller
@RequestMapping("/product")
public class ProductController {

    private Logger logger =
        Logger.getLogger(ProductController.class.getName());

    @PostMapping("/add")
    public String add(@RequestParam String name) {
        logger.info("Adding product " + name);
        return "main.html";
    }
}
```

Конечная точка получает параметр запроса и выводит его в консоль приложения. Следующий листинг показывает определение формы в файле main.html.

Листинг 9.7 Определение формы на странице main.html

```
<form action="/product/add" method="post">
  <span>Name:</span>
  <span><input type="text" name="name" /></span>
  <span><button type="submit">Add</button></span>
</form>
```

Теперь вы можете перезапустить приложение и протестировать форму. Вы увидите, что при отправке запроса отображается страница ошибки по умолчанию, которая отражает статус HTTP 403 Forbidden в ответе от сервера (рис. 9.7). Причиной этого статуса является отсутствие токена CSRF.



Рис. 9.7 Если токен CSRF отсутствует, сервер отклонит все запросы, выполненные с использованием метода HTTP POST. Пользователь будет перенаправлен на стандартную страницу ошибки, которая отображает статус HTTP 403 Forbidden в ответе

Чтобы решить эту проблему и разрешить серверу запрос, нам нужно добавить токен CSRF в запрос, выполненный через форму. Простой способ сделать это – использовать скрытый компонент ввода, как вы видели в форме входа по умолчанию. Код скрытого ввода показан в листинге 9.8.

Листинг 9.8 Добавление токена CSRF к запросу, выполненному через форму

```
<form action="/product/add" method="post">
  <span>Name:</span>
  <span><input type="text" name="name" /></span>
  <span><button type="submit">Add</button></span>

  <input type="hidden" data-bbox="675 915 696 928"/>
```



```

th:name="${_csrf.parameterName}"      ❷
th:value="${_csrf.token}" />          ❸
</form>

```

- ❶ Используем скрытый ввод для добавления запроса к токenu CSRF.
- ❷ Префикс «th» позволяет Thymeleaf выводить значение токена.

ПРИМЕЧАНИЕ В этом примере мы используем Thymeleaf, поскольку он обеспечивает простой способ получения значения атрибута запроса в представлении. В нашем случае нам нужно вывести токен CSRF. Помните, что CsrfFilter добавляет значение токена в атрибут `_csrf` запроса. Вместо Thymeleaf можно использовать любую альтернативную утилиту по вашему выбору для вывода значения токена в ответе.

Перезапустите приложение и снова протестируйте форму. На этот раз сервер принимает запрос, и приложение выводит строку журнала в консоль, подтверждая, что выполнение прошло успешно. Кроме того, если вы проверите форму, то найдете скрытый ввод со значением токена CSRF (рис. 9.8).

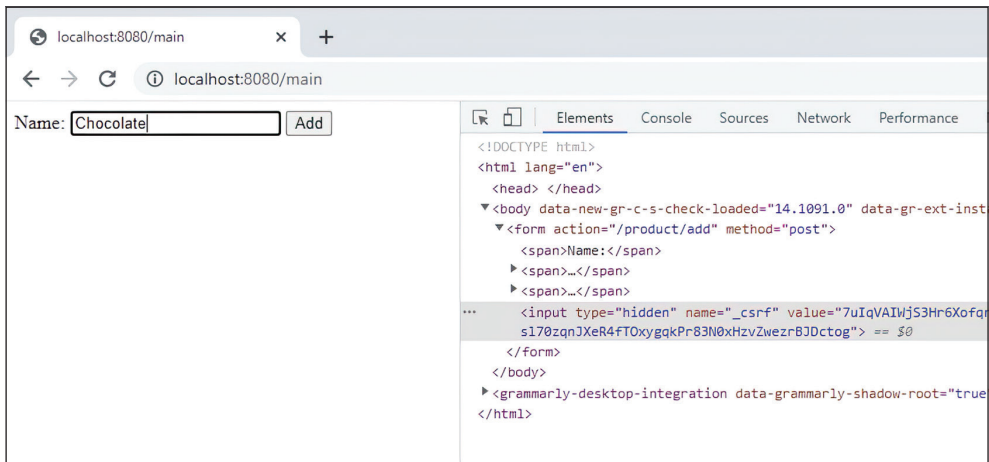


Рис. 9.8 Форма, определенная на главной странице, теперь отправляет значение токена CSRF в запросе. Поэтому сервер разрешает запрос и выполняет действие контроллера. В исходном коде страницы вы теперь можете найти скрытый ввод, используемый формой для отправки токена CSRF в запросе

После отправки формы вы должны найти в консоли приложения примерно такую строку:

```
INFO 20892 --- [nio-8080-exec-7] c.l.s.controllers.ProductController :
Adding product Chocolate
```

Конечно, для любого действия или асинхронного запроса JavaScript, который ваша страница использует для вызова мутирующего

действия, вам необходимо отправить действительный токен CSRF. Это наиболее распространенный способ убедить приложение, что запрос не исходит от третьей стороны. Сторонний запрос может попытаться выдать себя за пользователя, чтобы выполнить действия от его имени.

Токены CSRF хорошо работают в архитектуре, где один и тот же сервер отвечает как за фронтенд, так и за бэкенд, в основном из-за своей простоты. Но токены CSRF плохо работают, когда клиент не зависит от бэкенд-решения, которое он использует. Такой сценарий возникает, когда у вас есть мобильное приложение в качестве клиента или веб-фронтенд, разработанный независимо. В архитектурах веб-приложений часто встречается веб-клиент, разработанный с использованием фреймворка, такого как Angular, ReactJS или Vue.js, и именно поэтому вам нужно знать, как реализовать защиту от CSRF в подобных случаях. Мы обсудим более подробно эти типы архитектуры в четвертой части книги.

В главах 13–16 вы научитесь реализовывать спецификацию OAuth 2, которая обладает огромным преимуществом – возможностью разъединять компоненты. Это делает аутентификацию независимой от ресурсов, для которых приложение авторизует клиента.

ПРИМЕЧАНИЕ Это может показаться тривиальной ошибкой, но по моему опыту я слишком часто встречаю ее в приложениях – никогда не используйте HTTP GET с мутирующими операциями! Не реализуйте поведение, которое изменяет данные путем вызова конечной точки через HTTP GET. Помните, что вызовы конечных точек HTTP GET не требуют токена CSRF.

9.3 Настройка защиты от CSRF

В этом разделе вы узнаете, как настроить готовое решение защиты CSRF, предлагаемое Spring Security. Поскольку приложения предъявляют различные требования, любая реализация, предоставляемая фреймворком, должна быть достаточно гибкой, чтобы ее можно было легко адаптировать к различным сценариям. Механизм защиты CSRF в Spring Security не является исключением. В этом разделе на примерах показана реализация наиболее часто встречающихся потребностей в настройке механизма защиты от CSRF. Это:

- настройка путей, для которых применяется CSRF;
- управление токенами CSRF.

Мы используем защиту CSRF только тогда, когда страница, потребляющая ресурсы, созданные сервером, сама генерируется тем же сервером. Это может быть веб-приложение, в котором потребляе-

мые конечные точки предоставляются другим источником, как мы обсуждали в разделе 9.2, или мобильное приложение. В случае мобильных приложений вы можете использовать поток OAuth 2, который мы обсудим в главах 13–16.

По умолчанию защита от CSRF применяется к любому пути для конечных точек, вызываемых с помощью методов HTTP, отличных от GET, HEAD, TRACE или OPTIONS. Из главы 5 вы уже знаете, как полностью отключить защиту от CSRF. Но что, если вы хотите отключить ее только для некоторых путей вашего приложения? Вы можете быстро выполнить эту настройку с помощью объекта *Customizer*, аналогично тому, как мы настраивали HTTP Basic для входа в систему с помощью формы в главе 6.

Здесь мы создаем новый проект и добавляем только зависимости веб-интерфейса и безопасности, как показано в следующем фрагменте кода. Вы можете найти этот пример в проекте *ssia-ch9-ex3*. Вот эти зависимости:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

В этом приложении мы добавляем две конечные точки, вызываемые с помощью HTTP POST, но хотим отключить для одной из них защиту CSRF (рис. 9.9). В листинге 9.9 определяется класс контроллера, который я называл *HelloController*.

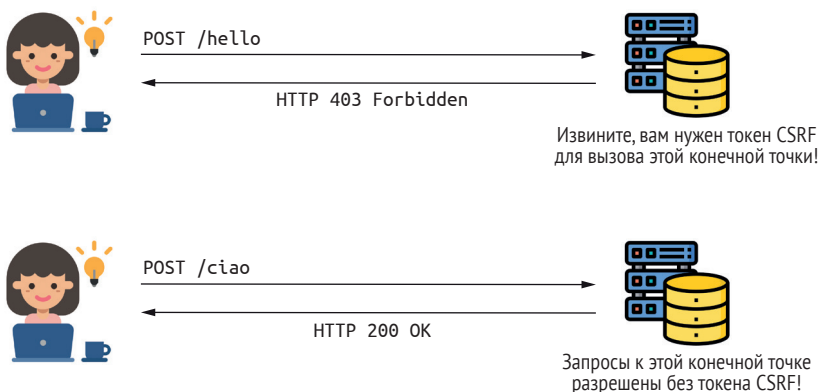


Рис. 9.9 Приложение требует токен CSRF для конечной точки `/hello`, вызываемой с помощью HTTP POST, но разрешает запросы HTTP POST к конечной точке `/ciao` без токена CSRF

Листинг 9.9 Определение класса `HelloController`

```
@RestController
public class HelloController {

    @PostMapping("/hello")           ❶
    public String postHello() {
        return "Post Hello!";
    }

    @PostMapping("/ciao")           ❷
    public String postCiao() {
        return "Post Ciao";
    }
}
```

❶ Путь `/hello` остается под защитой CSRF. Вы не можете вызвать конечную точку без действительного токена CSRF.

❷ Путь `/ciao` можно вызвать без токена CSRF.

Чтобы выполнить настройки защиты CSRF, вы можете использовать метод `csrf()` объекта `HttpSecurity` в методе `securityFilterChain()` с объектом `Customizer`. Этот подход показан в листинге 9.10.

Листинг 9.10 Объект `Customizer` для настройки защиты от CSRF

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.csrf(c -> {           ❶
            c.ignoringRequestMatchers("/ciao");
        });

        http.authorizeHttpRequests(
            c -> c.anyRequest().permitAll()
        );

        return http.build();
    }
}
```

❶ Параметром лямбда-выражения является `CsrfConfigurer`. Вызывая его методы, вы можете настроить защиту от CSRF различными способами.

Вызывая метод `ignoringRequestMatchers(String paths)`, вы можете указать выражения путей, указывающие на пути, которые вы хотите исключить из механизма защиты от CSRF. Более общий подход заключается в использовании `RequestMatcher`. Он позволяет изменять правила исключения с регулярными выражениями путей,

а также с произвольными регулярными выражениями. При использовании метода `ignoringRequestMatchers()` объекта `CsrfCustomizer` вы можете предоставить любой `RequestMatcher` в качестве параметра. Следующий фрагмент кода показывает, как использовать метод `ignoringRequestMatchers()` с `MvcRequestMatcher` вместо `ignoringRequestMatchers()` с путем, заданным как строковое значение:

```
HandlerMappingIntrospector i = new HandlerMappingIntrospector();
MvcRequestMatcher r = new MvcRequestMatcher(i, "/ciao");
c.ignoringRequestMatchers(r);
```

Или вы можете аналогичным образом использовать сопоставитель регулярных выражений:

```
String pattern = ".*[0-9].*";
String httpMethod = HttpMethod.POST.name();
RegexRequestMatcher r = new RegexRequestMatcher(pattern, httpMethod);
c.ignoringRequestMatchers(r);
```

Еще одна функциональная возможность, которую часто ожидают от приложения, – это настройка управления токенами CSRF. Как вы узнали, по умолчанию приложение хранит токены CSRF в сеансе HTTP на стороне сервера. Этот простой подход подходит для небольших приложений, но не очень хорош для приложений, которые обслуживают большое количество запросов и требуют горизонтального масштабирования. Сеанс HTTP является сохраняющим состояние и снижает масштабируемость приложения.

Предположим, вы хотите изменить способ, которым приложение управляет токенами, и хранить их где-то в базе данных, а не в сеансе HTTP. Spring Security предлагает три контракта, которые вам необходимо реализовать для этого:

- `CsrfToken` – описывает сам токен CSRF;
- `CsrfTokenRepository` – описывает объект, который создает, хранит и загружает токены CSRF;
- `CsrfTokenRequestHandler` – описывает объект, который управляет способом установки сгенерированного токена CSRF в запросе HTTP.

Объект `CsrfToken` имеет три основные характеристики, которые вам необходимо указать при реализации контракта (код в листинге 9.11 определяет контракт `CsrfToken`):

- имя заголовка в запросе, который содержит значение токена CSRF (по умолчанию с именем `X-CSRF-TOKEN`);
- имя атрибута запроса, который хранит значение токена (по умолчанию с именем `_csrf`);
- значение токена.

Листинг 9.11 Определение интерфейса `CsrfToken`

```
public interface CsrfToken extends Serializable {  
  
    String getHeaderName();  
    String getParameterName();  
    String getToken();  
}
```

Обычно вам нужен только экземпляр типа `CsrfToken` для хранения трех данных в атрибутах экземпляра. Для этой функциональности Spring Security предлагает реализацию `DefaultCsrfToken`, которую мы также используем в нашем примере. `DefaultCsrfToken` реализует контракт `CsrfToken` и создает неизменяемые экземпляры, содержащие требуемые значения: имя атрибута запроса и заголовка, а также сам токен.

Интерфейс `CsrfTokenRepository` – это контракт, представляющий компонент, управляющий токенами CSRF. Чтобы изменить способ управления токенами со стороны приложения, вам необходимо реализовать интерфейс `CsrfTokenRepository`, который позволяет вам подключать вашу пользовательскую реализацию к фреймворку. Давайте изменим текущее приложение, которое мы используем в этом разделе, чтобы добавить новую реализацию для `CsrfTokenRepository`, которая хранит токены в базе данных. На рис. 9.10 представлены компоненты, которые мы реализуем для этого примера, и связь между ними.

Мы определяем пользовательский `CsrfTokenRepository` для управления токенами CSRF по-разному

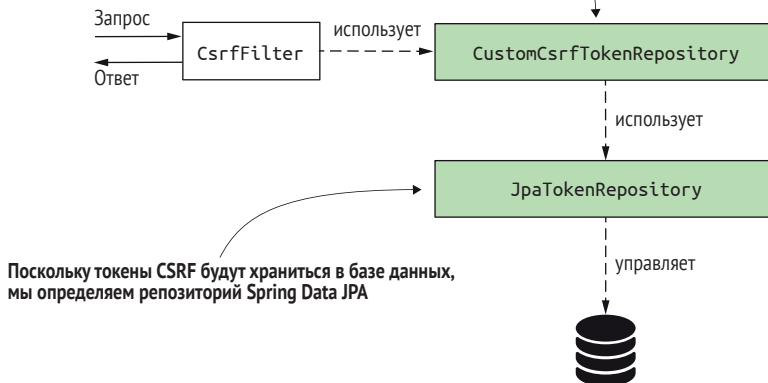


Рис. 9.10 `CsrfToken` использует пользовательскую реализацию `CsrfTokenRepository`. Эта пользовательская реализация нуждается в `JpaRepository` для управления токенами CSRF в базе данных

В нашем примере для хранения токенов CSRF мы используем таблицу в базе данных. Мы предполагаем, что у клиента есть уникальный идентификатор. Приложению нужен этот идентификатор для получения токена CSRF и его проверки. Обычно этот уникальный идентификатор будет получен во время входа пользователя в систему и должен меняться при каждом новом входе. Эта стратегия управления токенами похожа на их хранение в памяти. В этом случае вы используете идентификатор сессии. Таким образом, новый идентификатор пользователя просто заменяет идентификатор сессии.

Альтернативой этому подходу будет использование токенов CSRF с определенным сроком действия. При таком подходе токены истекают через заданное вами время. Вы можете хранить токены в базе данных, не привязывая их к определенному идентификатору пользователя. Вам нужно только проверить, существует ли токен, предоставленный через HTTP-запрос, и не истек ли его срок действия, чтобы разрешить запрос.

УПРАЖНЕНИЕ Разобравшись с примером, где мы используем идентификатор, которому назначен токен CSRF, реализуйте второй подход, в котором вы используете токены CSRF со сроком действия.

Чтобы сделать наш пример короче, мы сосредоточимся только на реализации `CsrfTokenRepository` и будем считать, что у клиента уже есть сгенерированный идентификатор. Для работы с базой данных нам нужно добавить еще пару зависимостей в файл `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
</dependency>
```

В файле `application.properties` нам нужно добавить свойства для подключения к базе данных:

```
spring.datasource.url=jdbc:mysql://localhost/spring
➡?useLegacyDatetimeCode=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.sql.init.mode=always
```

Чтобы приложение могло создать нужную таблицу в базе данных в момент первого запуска, добавьте файл `schema.xml` в папку ресурсов проекта. Этот файл должен содержать запрос на создание таблицы:

```
CREATE TABLE IF NOT EXISTS `spring`.`token` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `identifier` VARCHAR(45) NULL,
  `token` TEXT NULL,
  PRIMARY KEY (`id`));
```

Мы используем Spring Data с реализацией JPA для подключения к базе данных, поэтому нам нужно определить класс сущности и класс JpaRepository. В пакете с именем entities мы определяем сущность JPA, как показано в листинге 9.12.

Листинг 9.12 Определение класса сущности JPA

```
@Entity
public class Token {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String identifier;      ❶
    private String token;          ❷

    // Часть кода не показана
}
```

❶ Идентификатор клиента.

❷ Токен CSRF, сгенерированный приложением для клиента.

JpaTokenRepository, который является нашим контрактом JpaRepository, можно определить, как показано в листинге 9.13. Нам понадобится единственный метод findTokenByIdentifier(), получающий токен CSRF из базы данных для конкретного клиента.

Листинг 9.13 Определение интерфейса JpaTokenRepository

```
public interface JpaTokenRepository
    extends JpaRepository<Token, Integer> {

    Optional<Token> findTokenByIdentifier(String identifier);
}
```

Имея доступ к базе данных, мы теперь можем начать писать реализацию репозитория CsrfToken, которую я называю CustomCsrfTokenRepository. В листинге 9.14 показан код класса, который переопределяет три метода CsrfTokenRepository.

Листинг 9.14 Реализация контракта CsrfTokenRepository

```
@Component
public class CustomCsrfTokenRepository implements CsrfTokenRepository {
```



```

private final JpaTokenRepository jpaTokenRepository;

// Конструктор не показан

@Override
public CsrfToken generateToken(
    HttpServletRequest httpServletRequest) {
    // ...
}

@Override
public void saveToken(
    CsrfToken csrfToken,
    HttpServletRequest httpServletRequest,
    HttpServletResponse httpServletResponse) {
    // ...
}

@Override
public CsrfToken loadToken(
    HttpServletRequest httpServletRequest) {
    // ...
}
}

```

`CustomCsrfTokenRepository` внедряет экземпляр `JpaTokenRepository` из контекста Spring для получения доступа к базе данных. `CustomCsrfTokenRepository` использует этот экземпляр для извлечения или сохранения токенов CSRF в базе данных. Механизм защиты от CSRF вызывает метод `generateToken()`, когда приложению необходимо сгенерировать новый токен. Листинг 9.15 иллюстрирует реализацию этого метода для нашего упражнения. Мы используем класс `UUID` для генерации нового случайного значения `UUID` и сохраняем те же имена для заголовка запроса и атрибута, `X-CSRF-TOKEN` и `_csrf`, как в реализации по умолчанию, предлагаемой Spring Security.

Листинг 9.15 Реализация метода `generateToken()`

```

@Override
public CsrfToken generateToken(HttpServletRequest httpServletRequest) {
    String uuid = UUID.randomUUID().toString();
    return new DefaultCsrfToken("X-CSRF-TOKEN", "_csrf", uuid);
}

```

Метод `saveToken()` сохраняет сгенерированный токен для конкретного клиента. В случае реализации защиты CSRF по умолчанию приложение использует HTTP-сессию для идентификации токена CSRF. В нашем случае мы предполагаем, что у клиента есть уникальный идентификатор. Клиент отправляет значение своего уникального идентификатора в запросе с заголовком `X-IDENTIFIER`. В логике

метода мы проверяем, существует ли значение в базе данных. Если существует, мы обновляем базу данных новым значением токена. Если нет, мы создаем новую запись для этого идентификатора с новым значением токена CSRF. Следующий листинг представляет реализацию метода `saveToken()`.

Листинг 9.16 Реализация метода `saveToken()`

```
@Override
public void saveToken(
    CsrfToken csrfToken,
    HttpServletRequest httpServletRequest,
    HttpServletResponse httpServletResponse) {
    String identifier =
        httpServletRequest.getHeader("X-IDENTIFIER");

    Optional<Token> existingToken =
        jpaTokenRepository.findTokenByIdentifier(identifier);

    if (existingToken.isPresent()) {
        Token token = existingToken.get();
        token.setToken(csrfToken.getToken());
    } else {
        Token token = new Token();
        token.setToken(csrfToken.getToken());
        token.setIdentifier(identifier);
        jpaTokenRepository.save(token);
    }
}
```

- ❶ Получаем токен из базы данных по идентификатору клиента.
- ❷ Если идентификатор существует, обновляем значение токена новым сгенерированным значением.
- ❸ Если идентификатор не существует, создаем новую запись для идентификатора со сгенерированным значением для токена CSRF.

Метод `loadToken()` загружает данные токена (если они существуют) или возвращает `null` в противном случае. Реализация этого метода показана в листинге 9.17.

Листинг 9.17 Реализация метода `loadToken()`

```
@Override
public CsrfToken loadToken(
    HttpServletRequest httpServletRequest) {

    String identifier = httpServletRequest.getHeader("X-IDENTIFIER");

    Optional<Token> existingToken =
        jpaTokenRepository
            .findTokenByIdentifier(identifier);
```

```

if (existingToken.isPresent()) {
    Token token = existingToken.get();
    return new DefaultCsrfToken(
        "X-CSRF-TOKEN",
        "_csrf",
        token.getToken());
}

return null;
}

```

Для объявления компонента в классе конфигурации мы используем пользовательскую реализацию `CsrfTokenRepository`. Затем подключаем компонент к механизму защиты CSRF с помощью метода `csrfTokenRepository()` `CsrfConfigurer`. Класс конфигурации показан в листинге 9.18.

Листинг 9.18 Класс конфигурации для пользовательского `CsrfTokenRepository`

```

@Configuration
public class ProjectConfig {

    private final CustomCsrfTokenRepository customTokenRepository;

    // Конструктор не показан

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.csrf(c -> {
            c.csrfTokenRepository(customTokenRepository);
        });

        http.authorizeHttpRequests(
            c -> c.anyRequest().permitAll()
        );

        return http.build();
    }
}

```

❶ Используем объект `Customizer<CsrfConfigurer<HttpSecurity>>` для подключения новой реализации `CsrfTokenRepository` к механизму защиты от CSRF.

Последняя часть, которую нам нужно подключить, чтобы все работало правильно, – это обработчик `CsrfTokenRequest`. К счастью, мы можем использовать реализацию, предоставляемую Spring Security, – `CsrfTokenRequestAttributeHandler`. Эта реализация просто использует метод `generateToken()` `CsrfTokenRepository` для генерации

нового токена при вызове конечной точки с использованием метода HTTP GET. Затем она добавляет сгенерированный `CsrfToken` по запросу в качестве атрибута.

Вы можете кастомизировать простое поведение объекта обработчика `CsrfTokenRequestAttribute`, расширив его класс. Например, реализация по умолчанию, используемая Spring Security (названная `XorCsrfTokenRequestAttributeHandler`), имеет более сложное поведение. Эта реализация генерирует случайное значение с помощью объекта `SecureRandom`, а затем смешивает его массив байтов с токеном, сгенерированным `CsrfTokenRepository`, с помощью логической операции XOR.

Однако чтобы не усложнять наш пример и позволить вам сосредоточиться на конфигурации, мы настроим простой обработчик атрибутов `CsrfTokenRequestAttribute` для управления токеном CSRF в объекте запроса HTTP. В листинге 9.19 показана настройка обработчика атрибутов `CsrfTokenRequestAttribute` в классе конфигурации.

Листинг 9.19 Класс конфигурации для пользовательского `CsrfTokenRepository`

```
@Configuration
public class ProjectConfig {

    private final CustomCsrfTokenRepository customTokenRepository;

    // Конструктор не показан

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.csrf(c -> {
            c.csrfTokenRepository(customTokenRepository);
            c.csrfTokenRequestHandler(
                new CsrfTokenRequestAttributeHandler()
            );
        });

        http.authorizeHttpRequests(
            c -> c.anyRequest().permitAll()
        );

        return http.build();
    }
}
```

- ❶ Настройка объекта `CsrfTokenRequestAttributeHandler` для управления обработкой токена CSRF в HTTP-запросе.

В определении класса контроллера, представленном в листинге 9.9, мы также добавляем конечную точку, которая использует ме-

тод HTTP GET. Этот метод нам нужен для получения токена CSRF при тестировании нашей реализации:

```
@GetMapping("/hello")
public String getHello() {
    return "Get Hello!";
}
```

Теперь вы можете запустить приложение и протестировать новую реализацию для управления токеном. Мы вызываем конечную точку по протоколу HTTP GET для получения значения токена CSRF. При выполнении вызова мы должны использовать идентификатор клиента в заголовке X-IDENTIFIER. Новое значение токена CSRF генерируется и сохраняется в базе данных. Так выглядят вызов и ответ:

```
curl -H "X-IDENTIFIER:12345" http://localhost:8080/hello
Get Hello!
```

Если вы просмотрите содержимое таблицы токенов в базе данных, то обнаружите, что приложение добавило новую запись для клиента с идентификатором 12345. В моем случае сгенерированное значение для токена CSRF, которое я вижу в базе данных, – 2bc652f5-258b-4a26-b456-928e9bad71f8. Мы используем это значение для вызова конечной точки /hello с помощью метода HTTP POST, как показано в следующем фрагменте кода. Конечно, мы также должны предоставить идентификатор клиента, который используется приложением для извлечения токена из базы данных, чтобы сравнить его с тем, который мы предоставляем в запросе:

```
curl -XPOST -H "X-IDENTIFIER:12345" -H "X-CSRF-TOKEN:2bc652f5-258b-4a26-b456-928e9bad71f8" http://localhost:8080/hello
Post Hello!
```

Эти действия схематически показаны на рис. 9.11.

Если мы попытаемся вызвать конечную точку /hello с помощью POST без предоставления необходимых заголовков, то получим ответ со статусом HTTP 403 Forbidden. Чтобы убедиться в этом, вызовите конечную точку с помощью команды

```
curl -XPOST http://localhost:8080/hello
```

Тело ответа:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

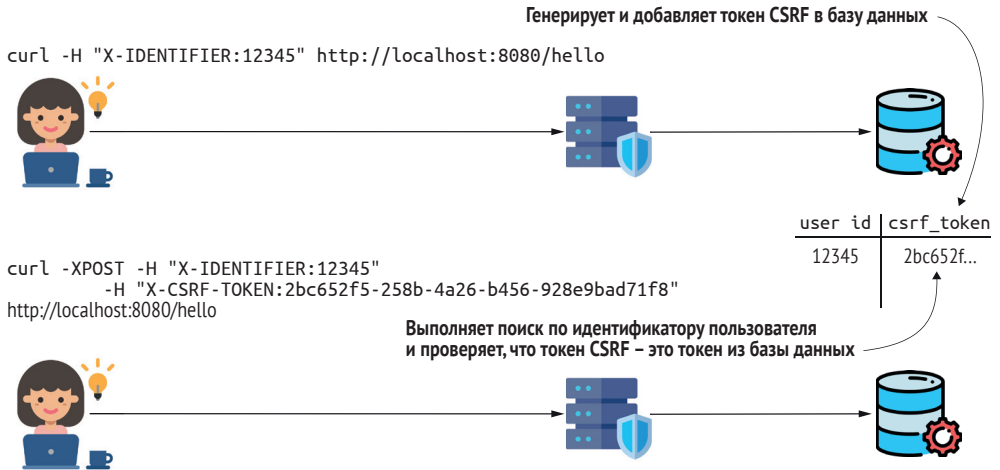


Рис. 9.11 Сначала запрос GET генерирует токен CSRF и сохраняет его значение в базе данных. Любой последующий запрос POST должен отправлять это значение. Затем `CsrfFilter` проверяет, совпадает ли значение в запросе со значением в базе данных. При совпадении запрос выполняется, в противном случае – отклоняется

Заклучение

- CSRF – это тип атаки, при которой пользователя обманом заставляют получить доступ к странице, содержащей скрипт подмены запроса. Этот скрипт может выдавать себя за пользователя, вошедшего в приложение, и выполнять действия от его имени.
- Защита от CSRF по умолчанию встроена в Spring Security.
- Начальной точкой логики защиты от CSRF в архитектуре Spring Security является HTTP-фильтр.
- Вы можете настроить защиту от CSRF. Spring Security предлагает три простых контракта, которые можно реализовать и подключить для определения пользовательских возможностей защиты от CSRF:
 - `CsrfToken` – описывает сам токен CSRF;
 - `CsrfTokenRepository` – описывает объект, который создает, хранит и загружает токены CSRF;
 - `CsrfTokenRequestHandler` – описывает объект, который управляет механизмом работы с токеном CSRF, сгенерированным при HTTP-запросе.

10

Настройка CORS

Темы этой главы:

- определение CORS;
- применение конфигураций CORS.

В этой главе мы обсудим *кросс-доменный обмен ресурсами* (CORS) и его применение совместно с Spring Security. Что такое механизм CORS и почему он заслуживает отдельного внимания? Необходимость в CORS возникает из-за веб-приложений. По умолчанию браузеры не разрешают запросы, сделанные для любого домена, кроме того, с которого загружен сайт. Например, если вы заходите на сайт с адреса `example.com`, браузер не позволит сайту делать запросы к `api.example.com`. Эта концепция схематически проиллюстрирована на рис. 10.1.

Вкратце можно сказать, что приложение использует механизм CORS, чтобы смягчить эту строгую политику и разрешить запросы, сделанные между разными источниками в определенных ситуациях. Вам нужно знать об этом, потому что вам, скорее всего, придется использовать механизм CORS для своих приложений, особенно в наши дни, когда фронтенд и бэкенд являются отдельными приложениями. Обычно фронтенд-приложение разрабатывают с использованием таких фреймворков, как Angular, ReactJS или Vue, и размещают на домене, например `example.com`, но оно вызывает конечные точки бэкенда, размещенного на другом домене, например `api.example.com`.

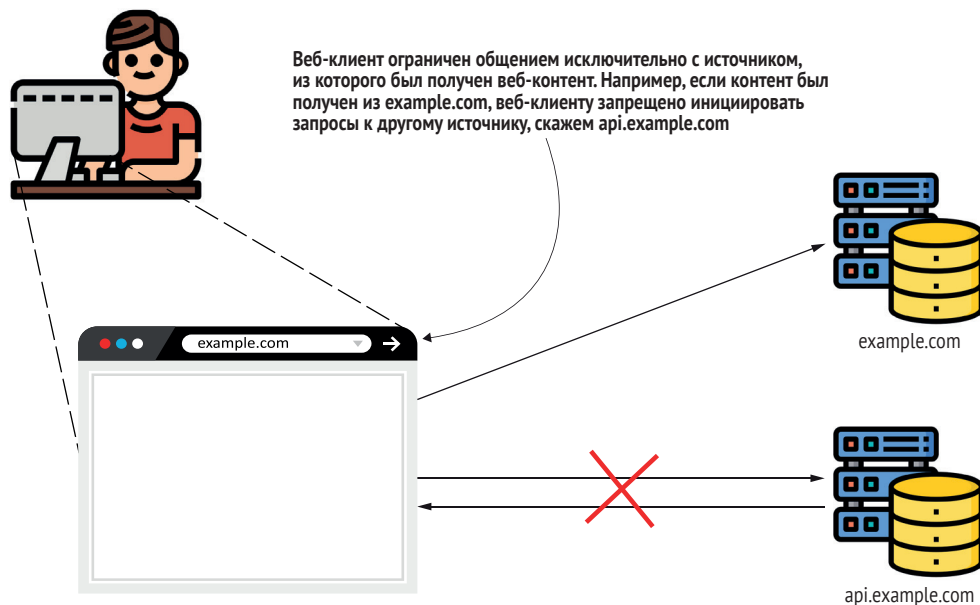


Рис. 10.1 Кросс-доменный обмен ресурсами (CORS). При доступе с `example.com` веб-сайт не может делать запросы к `api.example.com`, потому что они будут считаться кросс-доменными

В данной главе я приведу несколько примеров, из которых вы можете узнать, как применять политики CORS для своих веб-приложений. В этих примерах также показано, как избежать нарушений безопасности в ваших приложениях.

10.1 Как работает CORS?

В этом разделе мы обсудим применение CORS к веб-приложениям. Если вы являетесь владельцем `example.com` и по какой-то причине разработчики из `example.org` решают вызывать ваши конечные точки REST (`api.example.com`) со своего веб-сайта, они не смогут этого сделать. Та же ситуация может возникнуть, если домен загружает ваше приложение с помощью `iframe` (рис. 10.2).

ПРИМЕЧАНИЕ `iframe` – это элемент HTML, используемый для встраивания контента, сгенерированного веб-страницей, в другую веб-страницу (например, для интеграции контента с `example.org` внутри страницы с `example.com`).

Любая ситуация, в которой приложение выполняет вызовы между двумя разными доменами, запрещена. Конечно, вы можете столкнуться с ситуациями, в которых вам нужно выполнить такие вызовы

во что бы то ни стало, и в этом случае CORS позволит вам указать, с какого домена ваше приложение разрешает запросы и какие данные могут быть переданы. Механизм CORS работает на основе заголовков HTTP (рис. 10.3).

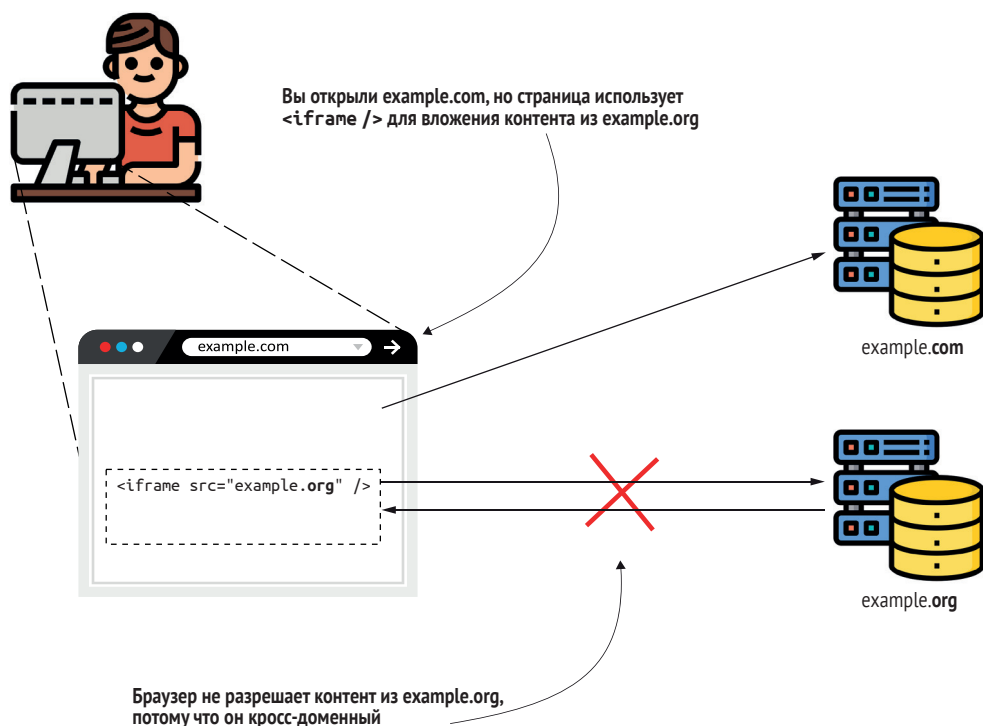


Рис. 10.2 Даже если страница example.org загружена в `iframe` из домена example.com, вызовы из контента, загруженного в example.org, не будут загружены. Более того, даже если приложение делает запрос, браузер не примет ответ

Наиболее важными заголовками являются:

- `Access-Control-Allow-Origin` – указывает внешние домены (источники), которые могут получить доступ к ресурсам на вашем домене;
- `Access-Control-Allow-Methods` – позволяет ссылаться только на некоторые методы HTTP в ситуациях, когда мы хотим разрешить доступ к другому домену, но только к определенным методам HTTP. Например, его используют, если нужно разрешить example.com вызывать некоторую конечную точку, но лишь с HTTP GET;
- `Access-Control-Allow-Headers` – добавляет ограничения на заголовки, которые можно использовать в определенном запросе. Допустим, вы не хотите, чтобы клиент мог отправлять определенный заголовок для данного запроса.

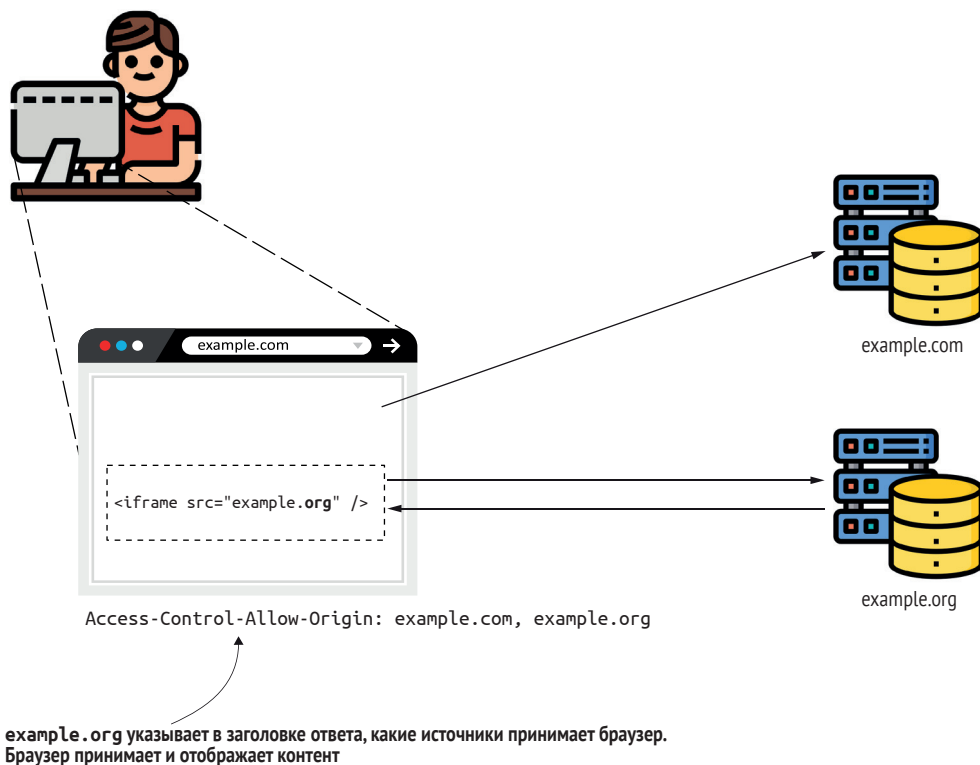


Рис. 10.3 Механизм кросс-доменных запросов. Сервер `example.org` добавляет заголовок `Access-Control-Allow-Origin`, чтобы указать источники запроса, для которых браузер должен принять ответ. Если домен, с которого был сделан вызов, указан в перечне источников, браузер принимает ответ

В Spring Security по умолчанию ни один из этих заголовков не добавляется в ответ. Итак, начнем с самого начала: что происходит, если вы делаете кросс-доменный запрос, но не настроили CORS в своем приложении? Когда приложение делает запрос, оно ожидает, что ответ будет содержать заголовок `Access-Control-Allow-Origin`, перечисляющий источники, разрешенные сервером. Если этого не произойдет, как в случае поведения Spring Security по умолчанию, браузер не примет ответ. Давайте продемонстрируем это на небольшом веб-приложении. Мы создаем новый проект, используя зависимости, представленные в следующем фрагменте кода (вы найдете этот пример в проекте `ssia-ch10-ex1`):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
```

```

    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

```

Мы определяем класс контроллера, который содержит действие для главной страницы и конечную точку REST. Поскольку класс является обычным классом Spring MVC `@Controller`, мы также должны явно добавить аннотацию `@ResponseBody` к конечной точке. В листинге 10.1 дано определение контроллера.

Листинг 10.1 Определение класса контроллера

```

@Controller
public class MainController {

    private Logger logger = ❶
        Logger.getLogger(MainController.class.getName());

    @GetMapping("/") ❷
    public String main() {
        return "main.html";
    }

    @PostMapping("/test")
    @ResponseBody
    public String test() { ❸
        logger.info("Test method called");
        return "HELLO";
    }
}

```

- ❶ Используем регистратор для наблюдения за вызовом метода `test()`.
- ❷ Определяем страницу `main.html`, которая делает запрос к конечной точке `/test`.
- ❸ Определяем конечную точку, которую мы вызываем из другого источника, чтобы посмотреть, как работает CORS.

Кроме того, нам нужно определить класс конфигурации, в котором мы отключаем защиту CSRF, чтобы упростить пример и сосредоточиться только на механизме CORS. С теми же целями мы разрешаем неаутентифицированный доступ ко всем конечным точкам. В следующем листинге дано определение этого класса конфигурации.

Листинг 10.2 Определение класса конфигурации

```

@Configuration
public class ProjectConfig {

    @Bean

```

```

public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.csrf(
        c -> c.disable()
    );

    http.authorizeHttpRequests(
        c -> c.anyRequest().permitAll()
    );

    return http.build();
}
}

```

Разумеется, нам также нужно разместить файл `main.html` в папке `resources/templates` проекта. Файл `main.html` содержит код JavaScript, который вызывает конечную точку `/test`. Чтобы имитировать кросс-доменный запрос, мы можем получить доступ к странице в браузере, используя домен `localhost`. Из кода JavaScript мы делаем вызов, используя IP-адрес `127.0.0.1`. Даже если `localhost` и `127.0.0.1` ссылаются на один и тот же хост, браузер видит их как разные строки и считает их разными доменами. В листинге 10.3 показано содержимое файла `main.html`.

Листинг 10.3 Страница `main.html`

```

<!DOCTYPE HTML>
<html lang="en">
  <head>
    <script>
      const http = new XMLHttpRequest();
      const url='http://127.0.0.1:8080/test'; ❶
      http.open("POST", url);
      http.send();

      http.onreadystatechange = (e) => {
        document
          .getElementById("output")
          .innerHTML = http.responseText; ❷
      }
    </script>
  </head>
  <body>
    <div id="output"></div>
  </body>
</html>

```

- ❶ Вызываем конечную точку, используя `127.0.0.1` в качестве хоста для имитации кросс-доменного запроса.
- ❷ Размещаем тело ответа в выходной блок `div` в теле страницы.

После запуска приложения и открытия страницы в браузере с адресом `localhost:8080` мы видим, что страница ничего не отображает. Мы ожидали увидеть HELLO на странице, потому что это то, что возвращает конечная точка `/test`. Однако когда мы проверяем консоль браузера, то видим ошибку, выведенную вызовом JavaScript. Ошибка выглядит так:

```
Access to XMLHttpRequest at 'http://127.0.0.1:8080/test' from origin  
'http://localhost:8080' has been blocked by CORS policy: No 'Access-  
Control-Allow-Origin' header is present on the requested resource.
```

(Доступ к XMLHttpRequest по адресу 'http://127.0.0.1:8080/test' из источника 'http://localhost:8080' заблокирован политикой CORS: в запрошенном ресурсе отсутствует заголовок 'Access-Control-Allow-Origin'.)

Сообщение об ошибке говорит нам, что ответ конечной точки не был принят, поскольку HTTP-заголовок `Access-Control-Allow-Origin` не существует. Это произошло, потому что мы ничего не настроили относительно CORS в нашем приложении Spring Boot, и по умолчанию оно не устанавливает никаких заголовков, связанных с CORS. Поэтому поведение браузера, не отображающего ответ, является правильным. Однако я прошу обратить внимание, что в консоли приложения видно, что метод был вызван. Вот что вы найдете в консоли приложения:

```
INFO 25020 --- [nio-8080-exec-2]  
➡c.l.s.controllers.MainController :  
➡Test method called
```

Этот аспект важен! Я часто встречаю разработчиков, которые рассматривают CORS как ограничение, похожее на авторизацию или защиту от CSRF. На самом деле CORS *помогает ослабить* жесткое ограничение для кросс-доменных вызовов. В некоторых ситуациях конечная точка может быть вызвана даже при наличии ограничений. Такое поведение встречается не всегда. Иногда браузер сначала выполняет вызов конечной точки с использованием метода HTTP OPTIONS, чтобы проверить, следует ли разрешить запрос. Эту проверку называют *предварительным запросом* (preflight request). Если предварительный запрос не выполняется, браузер не будет пытаться выполнить исходный запрос.

Ответственность за предварительный запрос и решение о том, следует ли его выполнять, возлагается на браузер. Вам не нужно реализовывать эту логику. Однако важно понимать этот механизм, чтобы не удивляться появлению кросс-доменных запросов бэкенда, даже если вы не указали никаких политик CORS для определенных доменов. Это также может произойти, если у вас есть клиентское приложение, разработанное с использованием фреймворка, такого

как Angular или ReactJS. На рис. 10.4 представлен этот поток запросов. Когда браузер пропускает предварительный запрос, если HTTP-метод – GET, POST или OPTIONS, у него есть только некоторые основные заголовки, как описано в официальной документации по адресу <https://fetch.spec.whatwg.org/#http-cors-protocol>.

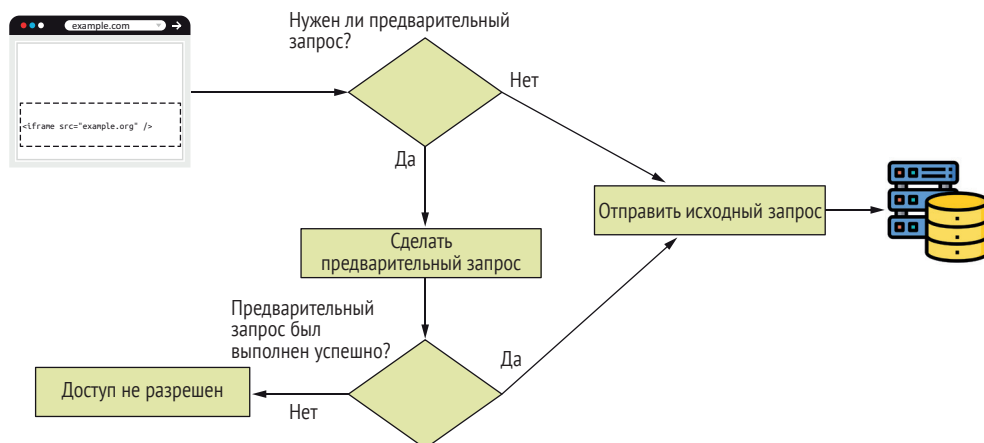


Рис. 10.4 В случае простых запросов браузер отправляет исходный запрос непосредственно на сервер. Браузер отклоняет ответ, если сервер не разрешает источник запроса. В некоторых случаях браузер отправляет предварительный запрос, чтобы проверить, принимает ли сервер источник. Если предварительный запрос проходит успешно, браузер отправляет исходный запрос

В нашем примере браузер делает запрос, но мы не принимаем ответ, если там не указан источник, как показано на рис. 10.1 и 10.2. Механизм CORS в конечном счете связан с браузером, а не способом защиты конечных точек. Единственное, что он гарантирует, – это то, что только разрешенные вами домены источника могут делать запросы с определенных страниц в браузере.

10.2 Применение политик CORS с аннотацией @CrossOrigin

В этом разделе обсуждается настройка CORS для разрешения запросов с разных доменов с помощью аннотации @CrossOrigin. Вы можете разместить аннотацию @CrossOrigin непосредственно над методом, который определяет конечную точку, и настроить ее путем указания разрешенных источников и методов. Как вы узнаете в этом разделе, преимущество использования аннотации @CrossOrigin заключается в том, что она упрощает настройку CORS для каждой конечной точки.

Давайте воспользуемся приложением, которое мы создали в разделе 10.1, чтобы продемонстрировать, как работает `@CrossOrigin`. Чтобы заставить сторонний источник работать в приложении, единственное, что вам нужно сделать, – это добавить аннотацию `@CrossOrigin` к методу `test()` в классе контроллера. В листинге 10.4 показано, как использовать аннотацию, чтобы сделать `localhost` разрешенным источником.

Листинг 10.4 Делаем `localhost` разрешенным источником

```
@PostMapping("/test")
@ResponseBody
@CrossOrigin("http://localhost:8080")    ❶
public String test() {
    logger.info("Test method called");
    return "HELLO";
}
```

❶ Разрешаем источник `localhost` для запросов разного происхождения.

Перезапустите и протестируйте приложение. Теперь на странице должна отображаться строка `HELLO`, возвращаемая конечной точкой `/test`.

Входным параметром `@CrossOrigin` является массив, позволяющий вам определять несколько источников, например `@CrossOrigin({"example.com", "example.org"})`. Вы также можете задать разрешенные заголовки и методы с помощью атрибутов `allowedHeaders` и `methods` аннотации. Как для источников, так и для заголовков вы можете использовать звездочку (*) для обозначения всех заголовков или всех источников. Однако я рекомендую вам проявлять осторожность при использовании этого подхода. Всегда лучше фильтровать источники и заголовки, которые вы хотите разрешить, и никогда не позволять ни одному постороннему домену использовать код, который обращается к ресурсам ваших приложений.

Разрешая все источники, вы подвергаете приложение риску выполнения запросов *межсайтового скриптинга* (cross-site scripting, XSS), что в конечном итоге может привести к DDoS-атакам. Лично я не разрешаю все источники даже в тестовых средах. Я знаю, что приложения иногда запускают на неправильно построенных инфраструктурах, которые используют одни и те же центры обработки данных как для тестирования, так и для производства. Разумнее рассматривать все слои, на которых применяется безопасность, независимо, как мы обсуждали в главе 1, и избегать предположения, что приложение не имеет особых уязвимостей, поскольку инфраструктура этого не позволяет.

Преимущество использования `@CrossOrigin` для указания правил непосредственно там, где определены конечные точки, заключает-

ся в том, что это создает хорошую прозрачность правил. Недостаток в том, что правила могут стать многословными, заставляя вас повторять большой объем кода. Отсюда вытекает риск того, что разработчик может забыть добавить аннотацию для недавно реализованных конечных точек. В разделе 10.3 мы обсуждаем применение централизованной конфигурации CORS, представленной в классе конфигурации.

10.3 Применение CORS с использованием CorsConfigurer

Хотя, как вы узнали в разделе 10.2, применять аннотацию `@Cross-Origin` очень легко, часто бывает удобнее определить конфигурацию CORS в одном месте. В этом разделе мы изменим пример, над которым мы работали в разделах 10.1 и 10.2, чтобы применить конфигурацию CORS в классе конфигурации с помощью настройщика. В листинге 10.5 показаны изменения, которые нам необходимо внести в класс конфигурации, чтобы определить источники, подлежащие разрешению.

Листинг 10.5 Централизованное определение конфигураций CORS в классе конфигурации

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.cors(c -> {
            CorsConfigurationSource source = request -> {
                CorsConfiguration config = new CorsConfiguration();
                config.setAllowedOrigins(
                    List.of("example.com", "example.org"));
                config.setAllowedMethods(
                    List.of("GET", "POST", "PUT", "DELETE"));
                config.setAllowedHeaders(List.of("*"));
                return config;
            };
            c.configurationSource(source);
        });

        http.csrf(
            c -> c.disable()
        );
    }
}
```



```

        http.authorizeHttpRequests(
            c -> c.anyRequest().permitAll()
        );

        return http.build();
    }
}

```

- ❶ Вызываем `cors()` для определения конфигурации CORS. Там мы создаем объект `CorsConfiguration`, в котором задаем разрешенные источники и методы.

Метод `cors()`, который мы вызываем из объекта `HttpSecurity`, получает в качестве параметра объект `Customizer<CorsConfigurer>`. Для этого объекта мы устанавливаем `CorsConfigurationSource`, возвращающий `CorsConfiguration` для HTTP-запроса. `CorsConfiguration` – это объект, который указывает, какие источники, методы и заголовки являются разрешенными. Если вы используете этот подход, вы должны как минимум указать источники и методы. Если вы укажете только источники, ваше приложение не разрешит запросы. Такое поведение возникает, потому что объект `CorsConfiguration` не определяет никаких методов по умолчанию.

В этом примере для простоты я предоставляю реализацию для `CorsConfigurationSource` как лямбда-выражение, используя bean-компонент `SecurityFilterChain` напрямую. Я настоятельно рекомендую разделять этот код по разным классам в ваших приложениях. В реальности у вас может быть гораздо более длинный код, который трудно читать, если он не разделен на классы конфигурации.

Заключение

- Механизм CORS предназначен для ситуаций, в которых веб-приложение, размещенное на определенном домене, пытается получить доступ к контенту из другого домена.
- По умолчанию браузер не допускает кросс-доменные запросы. Специальная конфигурация CORS позволяет вам разрешить вызов части ваших ресурсов из другого домена в веб-приложении, запущенном в браузере.
- Вы можете настроить CORS как для каждой конечной точки с помощью аннотации `@CrossOrigin`, так и централизованно в классе конфигурации с помощью метода `cors()` объекта `HttpSecurity`.

11

Авторизация на уровне методов

Темы этой главы:

- безопасность методов в приложениях Spring;
- предварительная авторизация на основе полномочий, ролей и разрешений;
- поставторизация на основе полномочий, ролей и разрешений.

В предыдущих главах мы рассмотрели различные способы настройки аутентификации. Мы начали с самого простого подхода HTTP Basic в главе 2, а затем я показал вам, как настроить форму входа в систему в главе 6. Однако с точки зрения авторизации мы обсуждали только конфигурацию на уровне конечной точки. Предположим, что ваше приложение не является веб-приложением – разве вы не можете использовать Spring Security для аутентификации и авторизации? Spring Security хорошо подходит и для ситуаций, когда ваше приложение не работает через конечные точки HTTP. В этой главе вы узнаете, как настроить авторизацию на уровне методов. Мы будем использовать этот подход для настройки авторизации как в веб-, так и в не веб-приложениях и назовем его *безопасностью метода* (method security, рис. 11.1).

В случае обычных, а не веб-приложений безопасность метода позволяет нам реализовывать правила авторизации, даже если у нас

нет конечных точек. В веб-приложениях этот подход дает нам гибкость в применении правил авторизации к разным уровням приложения, а не только на уровне конечной точки. Далее мы рассмотрим, как применять авторизацию на уровне метода с применением безопасности метода.

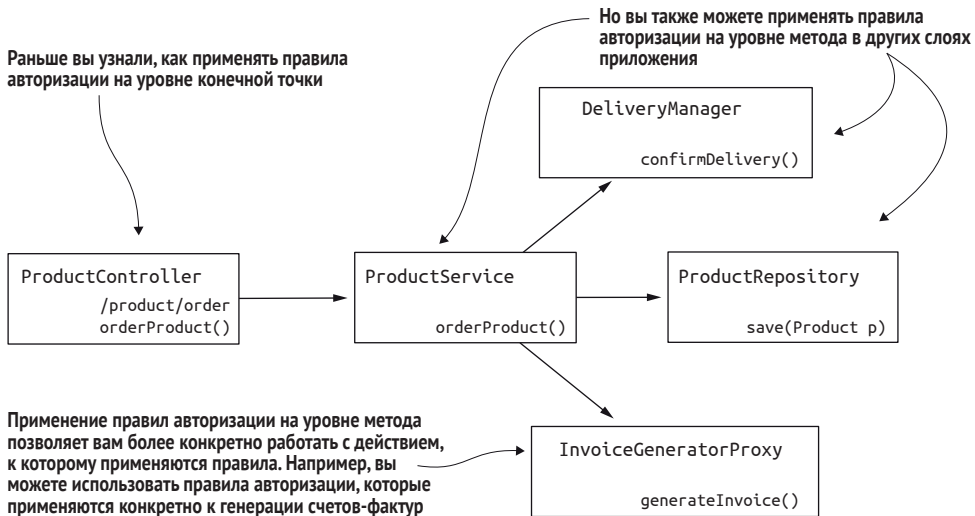


Рис. 11.1 Безопасность метода более детализована и позволяет применять правила авторизации на любом конкретно выбранном уровне вашего приложения

11.1 Включение механизма безопасности метода

В этом разделе будет показано, как включить авторизацию на уровне метода, и вы узнаете о вариантах применения различных правил авторизации в Spring Security. Этот подход обеспечивает большую гибкость в применении авторизации. Кроме того, это важный навык, позволяющий найти выход в ситуации, когда авторизацию нельзя настроить только на уровне конечной точки.

По умолчанию безопасность метода отключена, поэтому, если вы хотите использовать эту функцию, вам сначала нужно ее включить. Безопасность метода допускает несколько способов применения авторизации. Мы обсудим эти способы, а затем реализуем их в примерах в следующих разделах данной главы и в главе 12. Если коротко, глобальная безопасность методов позволяет сделать две основные вещи:

- *авторизацию вызовов* – принятие решения о том, может ли кто-то вызвать метод в соответствии с некоторыми правилами привилегий (предварительную авторизацию) или может ли кто-то

получить доступ к результату, который возвращен методом (поставторизацию);

- **фильтрацию** – принятие решения о том, что метод может получить через свои параметры (предварительную фильтрацию) и что вызывающая сторона может получить обратно от метода после выполнения метода (постфильтрацию). Мы обсудим и реализуем фильтрацию в главе 12.

11.1.1 Что такое авторизация вызовов

Одним из подходов к настройке правил авторизации, применяемых с безопасностью методов, является *авторизация вызовов* (call authorization). Этот подход заключается в разработке правил авторизации, которые решают, может ли метод быть вызван или которые разрешают вызов метода, но затем решают, может ли вызывающий получить доступ к значению, возвращаемому методом. Иногда нам нужно решить, может ли кто-то получить доступ к части логики в зависимости от предоставленных параметров или результата. Далее мы обсудим авторизацию вызовов, а затем опробуем ее на примерах.

Как работает безопасность методов? Каков механизм применения правил авторизации? Когда мы включаем безопасность методов в нашем приложении, мы фактически включаем аспект Spring. Этот аспект перехватывает вызовы метода, для которого мы применяем правила авторизации, и на основе этих правил он решает, перенаправлять ли перехваченный вызов методу (рис. 11.2).

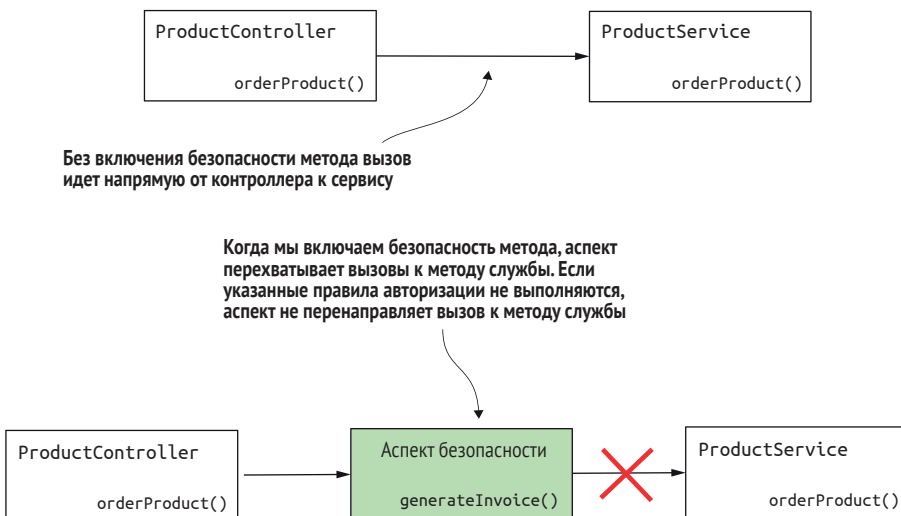


Рис. 11.2 Когда мы включаем глобальную безопасность методов, аспект перехватывает вызов защищенного метода. Если заданные правила авторизации не соблюдаются, аспект не делегирует вызов защищенному методу

Многие реализации в фреймворке Spring основаны на аспектно-ориентированном программировании (aspect-oriented programming, AOP). Безопасность методов – это лишь один из многих компонентов в приложениях Spring, основанных на аспектах. Если вам нужно освежить знания об аспектах и AOP, я рекомендую вам прочитать главу 6 еще одной моей книги – «Spring быстро» (Питер, 2022). Если коротко, мы делим авторизацию вызова на две категории:

- *предварительная авторизация* – фреймворк проверяет правила авторизации перед вызовом метода;
- *поставторизация* – фреймворк проверяет правила авторизации после выполнения метода.

Далее мы рассмотрим оба подхода и реализуем их с помощью нескольких примеров кода.

Использование предварительной авторизации для защиты доступа к методам

Допустим, у нас есть метод `findDocumentsByUser(String username)`, который возвращает вызывающей стороне документ для определенного пользователя. Вызывающая сторона предоставляет через параметры метода имя пользователя, для которого метод извлекает документы. Предположим, вам нужно гарантировать, что аутентифицированный пользователь может получать только свои собственные документы. Можем ли мы применить к этому методу такое правило, чтобы были разрешены только вызовы, которые получают имя аутентифицированного пользователя в качестве параметра? Да! За это отвечает предварительная авторизация.

Когда мы применяем правила авторизации, которые полностью запрещают кому-либо вызывать метод в определенных ситуациях, это называется *предварительной авторизацией* (preauthorization, рис. 11.3). Этот подход подразумевает, что фреймворк проверяет условия авторизации перед выполнением метода. Если у вызывающей стороны нет разрешений в соответствии с правилами авторизации, которые мы установили, фреймворк не делегирует вызов методу. Вместо этого фреймворк выдаст исключение с именем `AccessDeniedException`. Это, безусловно, наиболее часто используемый подход к глобальной безопасности методов.

Обычно нам нужно сделать так, чтобы функциональность была полностью недоступна, если не выполнены определенные условия. Вы можете применить условия, зависящие от аутентифицированного пользователя, а также от значений, полученных методом через его параметры.



Рис. 11.3 При предварительной авторизации правила авторизации проверяются перед дальнейшим делегированием вызова метода. Фреймворк не будет делегировать вызов, если правила авторизации не соблюдаются. Вместо этого он выдаст исключение стороне, вызывающей метод

Использование поставторизации для защиты вызова метода

Когда мы применяем правила авторизации, которые позволяют кому-либо вызывать метод, но не обязательно получать возвращаемый результат, такой подход называется *поставторизацией* (postauthorization, рис. 11.4). При поставторизации Spring Security проверяет соблюдение правил авторизации после выполнения метода. Этот тип авторизации применяется для ограничения доступа к результатам вызова метода в определенных условиях. Поскольку поставторизация происходит после выполнения метода, мы применяем правила авторизации к возвращаемому результату.

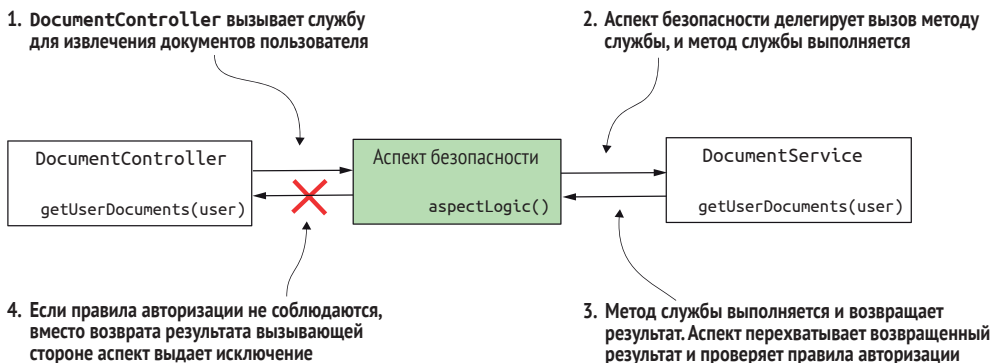


Рис. 11.4 При поставторизации аспект делегирует вызов защищенному методу. После того как защищенный метод завершает выполнение, аспект проверяет правила авторизации. Если правила не соблюдаются, вместо того чтобы возвращать результат вызывающей стороне, аспект выдает исключение

Обычно правила поставторизации применяют к результатам, возвращаемым методом после выполнения. Но будьте осторожны с поставторизацией! Если метод производит какие-то мутации во время выполнения, эти изменения произойдут независимо от исхода авторизации.

ПРИМЕЧАНИЕ Даже с аннотацией `@Transactional` изменения невозможно отменить, если поставторизация не удалась. Исключение, выдаваемое механизмом поставторизации, возникает после того, как менеджер транзакций фиксирует транзакцию.

11.1.2 Применение безопасности метода в вашем проекте

В этом разделе мы поработаем над проектом, чтобы применить функции предварительной и поставторизации, предлагаемые безопасностью метода. Механизм безопасности метода не включен по умолчанию в проекте Spring Security. Его нужно активировать самостоятельно. Это совсем не сложно – достаточно добавить аннотацию `@EnableMethodSecurity` в классе конфигурации.

Я создал для этого примера новый проект `ssia-ch11-ex1` и написал класс конфигурации `ProjectConfig`, как показано в листинге 11.1. В классе конфигурации мы добавляем аннотацию `@EnableMethodSecurity`. Нам доступны следующие три способа определения правил авторизации, которые мы обсуждаем в этой главе:

- аннотации до/после авторизации (включены по умолчанию);
- аннотации JSR 250, `@RolesAllowed`;
- аннотации `@Secured`.

Поскольку почти во всех случаях применяется подход аннотации до/после авторизации, мы обсудим его в этой главе. Он срабатывает после добавления аннотации `@EnableMethodSecurity`. Краткий обзор двух других вариантов будет представлен в конце данной главы.

Листинг 11.1 Включение безопасности метода

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {
}
```

Вы можете использовать глобальную безопасность метода с любым подходом к аутентификации, от базовой аутентификации HTTP до OAuth 2 (о котором вы узнаете в третьей части этой книги). Чтобы упростить задачу и сосредоточиться на новых деталях, мы рассмотрим безопасность метода с базовой аутентификацией HTTP. По этой причине файл `pom.xml` для проектов в этой главе нуждается

только в зависимостях web и Spring Security, как показано в следующем фрагменте кода:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

ПРИМЕЧАНИЕ В предыдущих версиях Spring Security мы использовали аннотацию `@EnableGlobal MethodSecurity`, а предварительная и поставторизация не были включены по умолчанию. Если вам нужно работать с авторизацией методов и предыдущей версией Spring Security (старше 6), вам может пригодиться глава 16 из первого издания Spring Security in Action.

11.2 Применение правил предварительной авторизации

В данном разделе мы изучим пример предварительной авторизации. Для этого мы продолжим проект `ssia-ch11-ex1`, начатый в разделе 11.1. Как было сказано в разделе 11.1, предварительная авторизация подразумевает определение правил авторизации, которые Spring Security применяет перед вызовом определенного метода. Если правила не соблюдаются, фреймворк не вызывает метод.

Приложение, которое мы реализуем в этом разделе, действует по простому сценарию. Оно предоставляет конечную точку `/hello`, возвращающую строку "Hello", за которой следует имя. Чтобы получить имя, контроллер вызывает служебный метод (рис. 11.5). Этот метод применяет правило предварительной авторизации для проверки наличия у пользователя полномочий на запись.

Я добавил `UserDetailsService` и `PasswordEncoder`, чтобы у меня были пользователи для аутентификации. Для тестирования приложения нам нужны два пользователя: один с полномочиями на запись и другой без них. Нам нужно убедиться, что первый пользователь может успешно вызвать конечную точку, в то время как для второго пользователя приложение выдает исключение авторизации при попытке вызвать метод. В следующем листинге показан полный код класса конфигурации, который определяет `UserDetailsService` и `PasswordEncoder`.

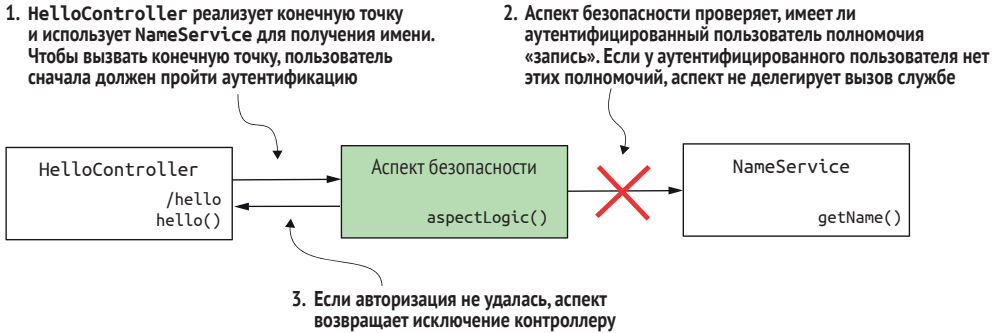


Рис. 11.5 Для вызова метода `getName()` `NameService` аутентифицированный пользователь должен иметь полномочия на запись. Если у пользователя нет этих полномочий, фреймворк не разрешит вызов и выдаст исключение

Листинг 11.2 Класс конфигурации для `UserDetailsService` и `PasswordEncoder`

```

@Configuration
@EnableMethodSecurity
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var service = new InMemoryUserDetailsManager();
        var u1 = User.withUsername("natalie")
            .password("12345")
            .authorities("read")
            .build();

        var u2 = User.withUsername("emma")
            .password("12345")
            .authorities("write")
            .build();

        service.createUser(u1);
        service.createUser(u2);

        return service;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
  
```

❶ Включаем безопасность метода для предварительной/поставторизации.

❷ Добавляем `UserDetailsService` в контекст Spring с двумя пользователями для тестирования.

❸ Добавляем `PasswordEncoder` в контекст Spring.

Чтобы определить правило авторизации для этого метода, мы используем аннотацию `@PreAuthorize`. Аннотация `@PreAuthorize` получает в качестве значения выражение Spring Expression Language (SpEL), описывающее правило авторизации. В этом примере мы применяем простое правило.

Вы можете определить ограничения для пользователей на основе их полномочий с помощью метода `hasAuthority()`. О нем говорилось в главе 7, где мы обсуждали применение авторизации на уровне конечной точки. Следующий листинг определяет класс службы, предоставляющий значение для имени.

Листинг 11.3 Класс службы, определяющий правило предварительной авторизации для метода

```
@Service
public class NameService {

    @PreAuthorize("hasAuthority('write')") ❶
    public String getName() {
        return "Fantastico";
    }
}
```

❶ Определяем правило авторизации. Только пользователи с полномочиями на запись могут вызывать метод.

Класс контроллера определяется в листинге 11.4. Он использует `NameService` как зависимость.

Листинг 11.4 Класс контроллера, реализующий конечную точку и использующий службу

```
@RestController
public class HelloController {

    private final NameService nameService; ❶

    // Конструктор не показан

    @GetMapping("/hello")
    public String hello() {
        return "Hello, " + nameService.getName(); ❷
    }
}
```

❶ Внедряем службу из контекста.

❷ Вызываем метод, для которого мы применяем правила предварительной авторизации.

Теперь вы можете запустить приложение и протестировать его поведение. Мы ожидаем, что только пользователь Эмма (Emma) бу-

дет иметь право вызывать конечную точку, поскольку у нее есть соответствующая авторизация. Следующий фрагмент кода представляет вызовы для конечной точки с нашими двумя пользователями, Эммой и Натали (Natalie). Чтобы вызвать конечную точку /hello и аутентифицироваться от имени Эммы, используйте следующую команду cURL:

```
curl -u emma:12345 http://localhost:8080/hello
```

Тело ответа:

```
Hello, Fantastico
```

Чтобы вызвать конечную точку /hello и аутентифицироваться от имени Натали, используйте следующую команду cURL:

```
curl -u natalie:12345 http://localhost:8080/hello
```

Тело ответа:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

Аналогично вы можете использовать любое другое выражение, которое мы обсуждали в главе 7, для аутентификации конечной точки. Кратко напомним их:

- `hasAnyAuthority()` – указывает несколько полномочий. Пользователь должен иметь хотя бы одно из этих полномочий для вызова метода;
- `hasRole()` – указывает роль, которую должен иметь пользователь для вызова метода;
- `hasAnyRole()` – указывает несколько ролей. Пользователь должен иметь хотя бы одну из них, чтобы вызвать метод.

Теперь расширим наш пример, чтобы показать, как можно использовать значения параметров метода для определения правил авторизации (рис. 11.6). Этот пример вы найдете в проекте с именем `ssia-ch11-ex2`.

Для этого проекта я определил тот же класс `ProjectConfig`, что и в нашем первом примере, чтобы мы могли продолжить работу с нашими двумя пользователями, Эммой и Натали. Теперь конечная точка принимает значение через переменную пути и вызывает класс службы, чтобы получить «секретные имена» для заданного имени пользователя. Конечно, в этом случае секретные имена – это

всего лишь придуманная мной характеристика пользователя, которую не все могут увидеть. Я определяю класс контроллера, как показано в следующем листинге.

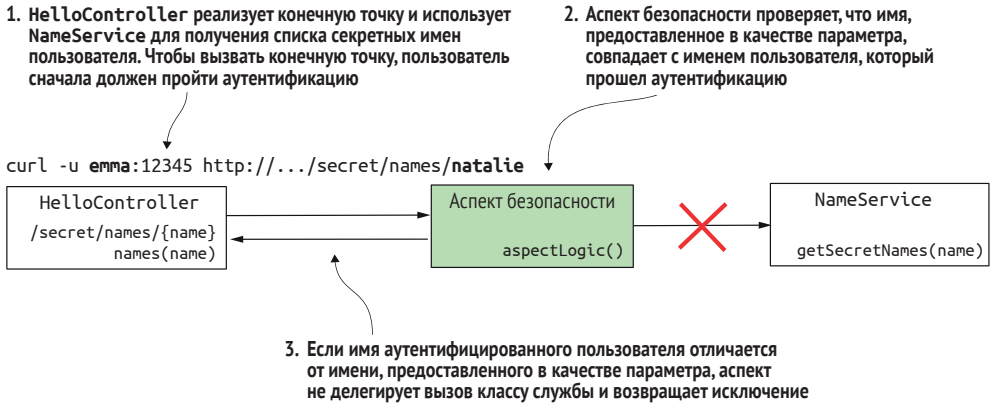


Рис. 11.6 В случае предварительной авторизации мы можем использовать в правилах авторизации значения параметров метода. В нашем примере только аутентифицированный пользователь может получить информацию о своих секретных именах

Листинг 11.5 Класс контроллера, определяющий конечную точку для тестирования

```

@RestController
public class HelloController {

    private final NameService nameService;

    // Конструктор не показан

    @GetMapping("/secret/names/{name}")
    public List<String> names(@PathVariable String name) {
        return nameService.getSecretNames(name);
    }
}
    
```

- ❶ Из контекста внедряем экземпляр класса службы, который определяет защищенный метод.
- ❷ Определяем конечную точку, которая принимает значение из переменной пути.
- ❸ Вызываем защищенный метод для получения секретных имен пользователей.

Теперь обратимся к реализации класса NameService в листинге 11.6. Выражение, которое мы используем для авторизации сейчас, – `#name == authentication.principal.username`. В этом выражении мы используем `#name` для ссылки на значение параметра метода `getSecretNames()` с именем `name`, и у нас есть прямой доступ к объекту аутентификации, который мы можем использовать для ссыл-

ки на текущего аутентифицированного пользователя. Выражение, которое мы используем, указывает, что метод может быть вызван только в том случае, если имя пользователя аутентифицированного пользователя совпадает со значением, отправленным через параметр метода. Другими словами, пользователь может получить только свои собственные секретные имена.

Листинг 11.6 Класс `NameService`, определяющий защищенный метод

```
@Service
public class NameService {

    private Map<String, List<String>> secretNames =
        Map.of(
            "natalie", List.of("Energico", "Perfecto"),
            "emma", List.of("Fantastico"));

    @PreAuthorize                                     ❶
        ("#name == authentication.principal.username")
    public List<String> getSecretNames(String name) {
        return secretNames.get(name);
    }
}
```

❶ Используем `#name` для представления значения параметров метода в выражении авторизации.

Запустим приложение и протестируем его, чтобы убедиться, что оно работает как задумано. Следующий фрагмент кода показывает поведение приложения при вызове конечной точки, предоставляя значение переменной пути, равное имени пользователя:

```
curl -u emma:12345 http://localhost:8080/secret/names/emma
```

Тело ответа:

```
["Fantastico"]
```

При аутентификации от имени Эммы мы пытаемся получить секретные имена Натали. Вызов не работает:

```
curl -u emma:12345 http://localhost:8080/secret/names/natalie
```

Тело ответа:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/secret/names/natalie"
}
```

Однако пользователь Натали может получить свои собственные секретные имена. Следующий фрагмент кода доказывает это:

```
curl -u natalie:12345 http://localhost:8080/secret/names/natalie
```

Тело ответа:

```
["Energico", "Perfecto"]
```

ПРИМЕЧАНИЕ Помните, что вы можете применить безопасность методов к любому слою вашего приложения. В этой главе вы найдете примеры правил авторизации, применяемые к служебным методам классов. Однако вы можете применять правила авторизации с безопасностью методов к любой части вашего приложения: контроллерам, репозиториям, менеджерам, прокси и т. д.

11.3 Применение правил поставторизации

Допустим, вы хотите разрешить вызов метода, но сделать так, чтобы при определенных обстоятельствах вызывающая сторона не получила возвращаемое значение. Когда мы применяем правило авторизации, соблюдение которого проверяется после вызова метода, это называется поставторизацией. Поначалу это кажется странным: почему кто-то может вызвать метод, но не получить результат? Здесь дело не в методе. Представьте, что этот метод извлекает некоторые данные из источника, скажем веб-службы или базы данных. Выполнение условий, добавляемых к процедуре авторизации, зависит от полученных данных. Поэтому мы разрешаем методу выполняться, но проверяем возвращаемый результат, и если он не соответствует критериям, то не позволяем вызывающей стороне получить доступ к возвращаемому значению.

Чтобы применить правила поставторизации с помощью Spring Security, мы используем аннотацию `@PostAuthorize`, которая похожа на `@PreAuthorize`, обсуждаемую в разделе 11.2. Аннотация получает SpEL в качестве значения, определяющего правило авторизации. Рассмотрим пример, демонстрирующий использование аннотации `@PostAuthorize` и определение правила поставторизации для метода (рис. 11.7).

В примере, для которого я создал проект с именем `ssia-ch11-ex3`, определяется объект `Employee`. У нашего `Employee` есть имя, список книг и список полномочий. Мы связываем каждый `Employee` с пользователем приложения. Чтобы сохранить сюжетную линию предыдущих примеров этой главы, мы определяем тех же пользователей, Эмму и Натали. Нам нужно сделать так, чтобы вызывающая сторона

метода получала данные о сотруднике, только если у сотрудника есть полномочия на чтение полученной записи. Поскольку мы не знаем полномочий, связанных с записью сотрудника, пока не получим ее, нам нужно применить правила авторизации после выполнения метода. По этой причине мы используем аннотацию `@PostAuthorize`.

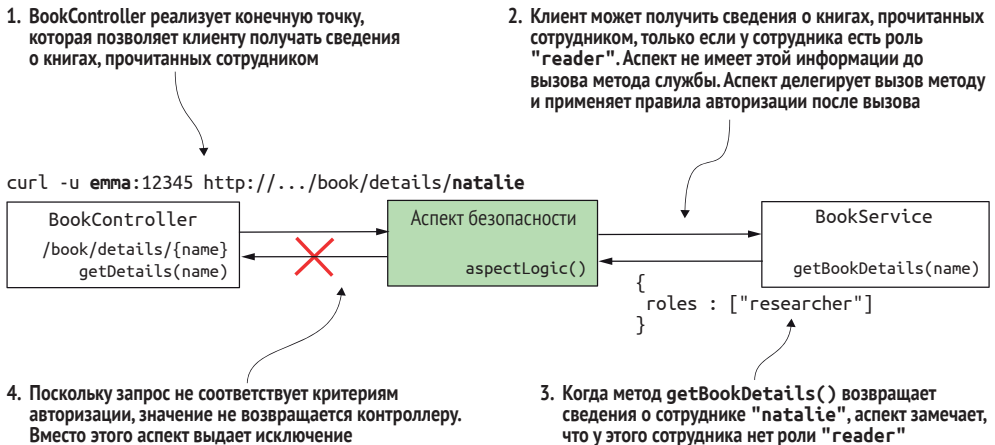


Рис. 11.7 С помощью поставоризации мы не защищаем метод от вызова, но защищаем возвращаемое значение от раскрытия, если определенные правила авторизации не соблюдаются

Класс конфигурации похож на тот, который мы использовали в предыдущих примерах. Но для вашего удобства я повторяю его в листинге 11.7.

Листинг 11.7 Включение безопасности метода и определение пользователей

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var service = new InMemoryUserDetailsManager();

        var u1 = User.withUsername(„natalie")
            .password("12345")
            .authorities("read")
            .build();

        var u2 = User.withUsername("emma")
            .password("12345")
            .authorities("write")
            .build();
    }
}
```

```
        service.createUser(u1);
        service.createUser(u2);

        return service;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

Нам также нужно объявить класс для представления объекта `Employee` с его именем, списком книг и списком ролей. Следующий листинг определяет класс `Employee`.

Листинг 11.8 Определение класса `Employee`

```
public class Employee {

    private String name;
    private List<String> books;
    private List<String> roles;

    // Конструктор, геттеры и сеттеры не показаны
}
```

На практике мы, скорее всего, получали бы данные о сотрудниках из базы данных. Чтобы сделать наш пример короче, я использую `Map` с парой записей, которые послужат нам источником данных. В листинге 11.9 вы найдете определение класса `BookService`. Класс `BookService` также содержит метод, для которого мы применяем правила авторизации. Обратите внимание, что выражение, которое мы используем с аннотацией `@PostAuthorize`, ссылается на значение, возвращаемое методом `returnObject`. Выражение поставторизации может использовать значение, возвращаемое методом, которое доступно после выполнения метода.

Листинг 11.9 Класс `BookService`, определяющий авторизованный метод

```
@Service
public class BookService {

    private Map<String, Employee> records =
        Map.of("emma",
            new Employee("Emma Thompson",
                List.of("Karamazov Brothers"),
                List.of("accountant", "reader")),
            "natalie",
            new Employee("Natalie Parker",
```



```

        List.of("Beautiful Paris"),
        List.of("researcher"))
    );
    @PostAuthorize("returnObject.roles.contains('reader')") ❶
    public Employee getBookDetails(String name) {
        return records.get(name);
    }
}

```

❶ Определяем выражение для поставторизации.

Мы также напишем контроллер и реализуем конечную точку для вызова метода, для которого мы применили правило авторизации. Следующий листинг представляет этот класс контроллера.

Листинг 11.10 Класс контроллера, реализующий конечную точку

```

@RestController
public class BookController {

    private final BookService bookService;

    // Конструктор не показан

    @GetMapping("/book/details/{name}")
    public Employee getDetails(@PathVariable String name) {
        return bookService.getBookDetails(name);
    }
}

```

Теперь вы можете запустить приложение и вызвать конечную точку, чтобы понаблюдать за поведением приложения. В следующих фрагментах кода вы найдете примеры вызова конечной точки. Любой из пользователей может получить доступ к данным Эммы, поскольку возвращенный список ролей содержит строку "reader" (читатель), но ни один пользователь не может получить данные для Натали. Вызывая конечную точку для получения данных об Эмме и аутентифицируясь от имени Эммы, мы используем следующую команду:

```
curl -u emma:12345 http://localhost:8080/book/details/emma
```

Тело ответа:

```

{
  "name": "Emma Thompson",
  "books": ["Karamazov Brothers"],
  "roles": ["accountant", "reader"]
}

```

Вызывая конечную точку для получения данных об Эмме и аутентифицируясь от имени Натали, мы используем команду

```
curl -u natalie:12345 http://localhost:8080/book/details/emma
```

Тело ответа:

```
{
  "name": "Emma Thompson",
  "books": ["Karamazov Brothers"],
  "roles": ["accountant", "reader"]
}
```

Вызывая конечную точку для получения данных о Натали и аутентифицируясь от имени Эммы, мы используем команду

```
curl -u emma:12345 http://localhost:8080/book/details/natalie
```

Тело ответа:

```
{
  "status": 403,
  "error": "Forbidden",
  "message": "Forbidden",
  "path": "/book/details/natalie"
}
```

Вызывая конечную точку для получения сведений о Натали и аутентифицируясь от имени Натали, мы используем команду

```
curl -u natalie:12345 http://localhost:8080/book/details/natalie
```

Тело ответа:

```
{
  "status": 403,
  "error": "Forbidden",
  "message": "Forbidden",
  "path": "/book/details/natalie"
}
```

ПРИМЕЧАНИЕ Вы можете использовать вместе как `@PreAuthorize`, так и `@PostAuthorize` для одного и того же метода, если ваша задача нуждается как в предварительной, так и в поставторизации.

11.4 Реализация разрешений для методов

До сих пор мы определяли правила с помощью простых выражений для предварительной авторизации и поставторизации. Теперь предположим, что требуется более сложная логика авторизации и вы не можете написать ее в одну строку. Действительно, неудобно писать огромные выражения SpEL. Я не рекомендую использовать длинные

выражения SpEL в любой ситуации, независимо от того, является это правилом авторизации или нет. Они значительно ухудшают читаемость кода, а это влияет на удобство поддержки приложения. Если вам нужно реализовать сложные правила авторизации, вместо написания длинных выражений SpEL вынесите логику в отдельный класс. Spring Security предлагает концепцию *разрешений* (permission), которая позволяет писать правила авторизации в отдельном классе, чтобы ваше приложение было легче читать и понимать.

В этом разделе мы применяем правила авторизации с использованием разрешений в проекте. Я назвал этот проект ssia-ch11-ex4. По сценарию проекта у нас есть приложение, управляющее документами. У каждого документа есть владелец, которым является пользователь, создавший документ. Чтобы получить сведения о существующем документе, пользователь должен быть либо администратором, либо владельцем документа. Для решения этой задачи мы реализуем оценщик разрешений. Следующий листинг определяет объект document, который является всего лишь простым объектом Java.

Листинг 11.11 Класс Document

```
public class Document {

    private String owner;

    // Конструктор, геттеры и сеттеры не показаны
}
```

Чтобы имитировать базу данных и сделать наш пример короче для вашего удобства, я создал класс-репозиторий, который управляет несколькими экземплярами документов в Map. Этот класс представлен в следующем листинге.

Листинг 11.12 Класс DocumentRepository, управляющий несколькими экземплярами Document

```
@Repository
public class DocumentRepository {

    private Map<String, Document> documents = ❶
        Map.of("abc123", new Document("natalie"),
              "qwe123", new Document("natalie"),
              "asd555", new Document("emma"));

    public Document findDocument(String code) {
        return documents.get(code); ❷
    }
}
```

- ❶ Идентифицируем каждый документ по уникальному коду и имени владельца.
- ❷ Получаем документ, используя его уникальный идентификационный код.

Служебный класс определяет метод, который использует репозиторий для получения документа по его коду. Метод в служебном классе – это тот, для которого мы применяем правила авторизации. Логика класса проста. Он определяет метод, который возвращает Document по его уникальному коду. Мы аннотируем этот метод с помощью @PostAuthorize и используем выражение SpEL hasPermission(). Этот метод позволяет нам ссылаться на внешнее выражение авторизации, которое мы реализуем далее в этом примере. Между тем обратите внимание, что параметры, которые мы предоставляем методу hasPermission(), – это returnObject, который представляет значение, возвращаемое методом, и имя роли, для которой мы разрешаем доступ, то есть 'ROLE_admin'. Определение этого класса представлено в следующем листинге.

Листинг 11.13 Класс DocumentService, реализующий защищенный метод

```
@Service
public class DocumentService {

    private final DocumentRepository documentRepository;

    // Конструктор не показан

    @PostAuthorize b
    ("hasPermission(returnObject, 'ROLE_admin')")
    public Document getDocument(String code) {
        return documentRepository.findDocument(code);
    }
}
```

- ❶ Используем выражение hasPermission() для ссылки на выражение авторизации.

Наша задача – реализовать логику разрешений. И мы делаем это, написав объект, который реализует контракт PermissionEvaluator. Данный контракт предоставляет два способа реализации логики разрешений:

- *по объекту и разрешению* – используется в текущем примере, предполагает, что оценщик разрешений получает два объекта: один, который подчиняется правилу авторизации, и один, который предлагает дополнительные сведения, необходимые для реализации логики разрешений;
- *по идентификатору объекта, типу объекта и разрешению* – предполагает, что оценщик разрешений получает идентификатор объекта, который он может использовать для получения необходимого объекта. Он также получает тип объекта, который может использоваться, если один и тот же оценщик разре-

ний применяется к нескольким типам объектов, и ему нужен объект, предлагающий дополнительные сведения для оценки разрешения.

В следующем листинге вы найдете контракт `PermissionEvaluator` с двумя методами.

Листинг 11.14 Определение контракта `PermissionEvaluator`

```
public interface PermissionEvaluator {  
    boolean hasPermission(  
        Authentication a,  
        Object subject,  
        Object permission);  
  
    boolean hasPermission(  
        Authentication a,  
        Serializable id,  
        String type,  
        Object permission);  
}
```

Для текущего примера достаточно использовать первый метод. У нас уже есть субъект, который в нашем случае является значением, возвращаемым методом. Мы также отправляем имя роли `'ROLE_admin'`, которая в соответствии со сценарием примера может получить доступ к любому документу. Конечно, в нашем примере мы могли бы напрямую использовать имя роли в классе оценщика разрешений и не отправлять его как значение объекта `hasPermission()`. Здесь мы отправляем значение только ради примера. В реальном сценарии, который может быть более сложным, у вас будет несколько методов, и детали процесса авторизации могут быть разными в зависимости от метода. По этой причине у вас есть параметр, с помощью которого вы можете отправлять необходимые данные для использования в логике авторизации с уровня метода.

Для лучшего понимания и во избежание путаницы я также хотел бы упомянуть, что вам не нужно передавать объект `Authentication`. Spring Security автоматически предоставляет это значение параметра при вызове метода `hasPermission()`. Фреймворк знает значение экземпляра аутентификации, поскольку он уже находится в `SecurityContext`. В листинге 11.15 показан класс `DocumentsPermissionEvaluator`, который в нашем примере реализует контракт `PermissionEvaluator` для определения пользовательского правила авторизации.

Листинг 11.15 Реализация правила авторизации

```
@Component  
public class DocumentsPermissionEvaluator  
    implements PermissionEvaluator {
```

❶

```

@Override
public boolean hasPermission(
    Authentication authentication,
    Object target,
    Object permission) {

    Document document = (Document) target; ❷
    String p = (String) permission; ❸

    boolean admin = ❹
        authentication.getAuthorities()
            .stream()
            .anyMatch(a -> a.getAuthority().equals(p));

    return admin || ❺
        document.getOwner()
            .equals(authentication.getName());
}

@Override
public boolean hasPermission(Authentication authentication,
                             Serializable targetId,
                             String targetType,
                             Object permission) {

    return false; ❻
}
}

```

- ❶ Реализуем контракт `PermissionEvaluator`.
- ❷ Приводим целевой объект к типу `Document`.
- ❸ Объект разрешения в нашем случае – это имя роли, поэтому мы приводим его к `String`.
- ❹ Проверяем, имеет ли пользователь роль, которую мы получили в качестве параметра.
- ❺ Если администратор или аутентифицированный пользователь является владельцем документа, предоставляем разрешение.
- ❻ Нам не нужно реализовывать второй метод, поскольку мы его не используем.

Чтобы Spring Security узнал о нашей новой реализации `PermissionEvaluator`, нам нужно определить bean-компонент `MethodSecurityExpressionHandler` в классе конфигурации. В следующем листинге показано, как определить обработчик `MethodSecurityExpression`, чтобы сделать известным пользовательский `PermissionEvaluator`.

Листинг 11.16 Настройка `PermissionEvaluator` в классе конфигурации

```

@Configuration
@EnableMethodSecurity
public class ProjectConfig {

    private final DocumentsPermissionEvaluator evaluator;

```

```
// Конструктор не показан

@Bean
protected MethodSecurityExpressionHandler createExpressionHandler() {
    var expressionHandler =
        new DefaultMethodSecurityExpressionHandler();

    expressionHandler.setPermissionEvaluator(
        evaluator);

    return expressionHandler;
}
// Не показано определение бинов UserDetailsService и PasswordEncoder
```

- ❶ Делаем возвращаемый объект `MethodSecurityExpressionHandler` компонентом в контексте Spring.
- ❷ Создаем обработчик выражений безопасности по умолчанию для настройки пользовательского оценщика разрешений.
- ❸ Настраиваем пользовательский оценщик разрешений.
- ❹ Возвращаем пользовательский обработчик выражений для добавления в контекст Spring.

ПРИМЕЧАНИЕ Здесь мы используем реализацию для `MethodSecurityExpressionHandler` с именем `DefaultMethodSecurityExpressionHandler`, которую предоставляет Spring Security. Вы также можете реализовать пользовательский обработчик `MethodSecurityExpressionHandler` для определения пользовательских выражений SpEL, которые вы используете для применения правил авторизации. Вам редко придется делать это в реальном сценарии, и по этой причине мы не будем рассматривать такой пользовательский объект в наших примерах. Я просто хотел, чтобы вы знали, что это возможно.

Я разделяю определение `UserDetailsService` и `PasswordEncoder`, чтобы вы могли сосредоточиться только на новом коде. В листинге 11.17 вы можете найти остальную часть класса конфигурации. Единственное, что важно отметить в отношении пользователей, – это их роли. Пользователь Натали – администратор и может получить доступ к любому документу. Пользователь Эмма – менеджер и может получить доступ только к своим собственным документам.

Листинг 11.17 Полное определение класса конфигурации

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {

    private final DocumentsPermissionEvaluator evaluator;

    // Конструктор не показан

    @Override
```

```
protected MethodSecurityExpressionHandler createExpressionHandler() {
    var expressionHandler =
        new DefaultMethodSecurityExpressionHandler();

    expressionHandler.setPermissionEvaluator(evaluator);

    return expressionHandler;
}

@Bean
public UserDetailsService userDetailsService() {
    var service = new InMemoryUserDetailsManager();

    var u1 = User.withUsername("natalie")
        .password("12345")
        .roles("admin")
        .build();

    var u2 = User.withUsername("emma")
        .password("12345")
        .roles("manager")
        .build();

    service.createUser(u1);
    service.createUser(u2);

    return service;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
}
}
```

Чтобы протестировать приложение, мы определяем конечную точку. В следующем листинге показан код этого определения.

Листинг 11.18 Определение класса контроллера и реализация конечной точки

```
@RestController
public class DocumentController {

    private final DocumentService documentService;

    // Конструктор не показан

    @GetMapping("/documents/{code}")
    public Document getDetails(@PathVariable String code) {
        return documentService.getDocument(code);
    }
}
```


Давайте запустим приложение и вызовем конечную точку, чтобы понаблюдать за его поведением. Пользователь Натали может получить доступ к документам независимо от их владельца. Пользователь Эмма может получить доступ только к тем документам, которыми владеет. При вызове конечной точки для документа, принадлежащего Натали, и аутентификации с пользователем "natalie" используйте команду

```
curl -u natalie:12345 http://localhost:8080/documents/abc123
```

Тело ответа:

```
{  
  "owner": "natalie"  
}
```

При вызове конечной точки для документа, принадлежащего Эмме, и аутентификации с пользователем "natalie" используйте команду

```
curl -u natalie:12345 http://localhost:8080/documents/asd555
```

Тело ответа:

```
{  
  "owner": "emma"  
}
```

При вызове конечной точки для документа, принадлежащего Эмме, и аутентификации с пользователем "emma" используйте команду

```
curl -u emma:12345 http://localhost:8080/documents/asd555
```

Тело ответа:

```
{  
  "owner": "emma"  
}
```

Вызывая конечную точку для документа, принадлежащего Натали, и аутентифицируясь с пользователем "emma", используйте команду

```
curl -u emma:12345 http://localhost:8080/documents/abc123
```

Тело ответа:

```
{  
  "status": 403,  
  "error": "Forbidden",  
}
```

```
"message": "Forbidden",
"path": "/documents/abc123"
}
```

Аналогичным образом для написания правила авторизации можно использовать второй метод `PermissionEvaluator`. Второй метод подразумевает использование идентификатора и типа субъекта вместо самого объекта. Допустим, мы хотим изменить текущий пример, чтобы применить правила авторизации до выполнения метода, используя `@PreAuthorize`. В этом случае у нас еще нет возвращаемого объекта. Но вместо самого объекта у нас есть код документа, который является его уникальным идентификатором. Следующий листинг показывает, как изменить класс оценщика разрешений для реализации нового сценария. Я выделил примеры в проект с именем `ssia-ch11-ex5`, который вы можете запустить отдельно.

Листинг 11.19 Изменения в классе `DocumentsPermissionEvaluator`

```
@Component
public class DocumentsPermissionEvaluator
    implements PermissionEvaluator {

    private final DocumentRepository documentRepository;

    // Конструктор не показан

    @Override
    public boolean hasPermission(Authentication authentication,
                                Object target,
                                Object permission) {
        return false; ❶
    }

    @Override
    public boolean hasPermission(Authentication authentication,
                                Serializable targetId,
                                String targetType,
                                Object permission) {

        String code = targetId.toString(); ❷
        Document document = documentRepository.findDocument(code);

        String p = (String) permission;

        boolean admin =
            authentication.getAuthorities()
                .stream()
                .anyMatch(a -> a.getAuthority().equals(p));

        return admin || ❸
    }
}
```

```

        document.getOwner().equals(
            authentication.getName());
    }
}

```

- ❶ Больше не определяем правила авторизации через первый метод.
- ❷ Вместо объекта у нас есть его идентификатор, и мы получаем объект, используя идентификатор.
- ❸ Если пользователь является администратором или владельцем документа, он может получить доступ к документу.

Конечно, нам также нужно использовать правильный вызов оценщика разрешений с аннотацией `@PreAuthorize`. В следующем листинге вы найдете изменение, которое я сделал в классе `DocumentService`, чтобы применить правила авторизации с новым методом.

Листинг 11.20 Класс `DocumentService`

```

@Service
public class DocumentService {

    private final DocumentRepository documentRepository;
    // Конструктор не показан
    @PreAuthorize
        ("hasPermission(#code, 'document', 'ROLE_admin')")
    public Document getDocument(String code) {
        return documentRepository.findDocument(code);
    }
}

```

- ❶ Применяем правила предварительной авторизации с использованием второго метода оценщика разрешений.

Вы можете перезапустить приложение и проверить поведение конечной точки. Вы должны увидеть тот же результат, что и в случае, когда мы использовали первый метод оценщика разрешений для реализации правил авторизации. Пользователь Натали является администратором и может получить доступ к сведениям о любом документе, в то время как пользователь Эмма может получить доступ только к документам, которыми владеет. При вызове конечной точки для документа, принадлежащего Натали, и аутентификации с пользователем "natalie" используйте команду

```
curl -u natalie:12345 http://localhost:8080/documents/abc123
```

Тело ответа:

```

{
  "owner": "natalie"
}

```

При вызове конечной точки для документа, принадлежащего Эмме, и аутентификации с пользователем "natalie" используйте команду

```
curl -u natalie:12345 http://localhost:8080/documents/asd555
```

Тело ответа:

```
{
  "owner": "emma"
}
```

При вызове конечной точки для документа, принадлежащего Эмме, и аутентификации с пользователем "emma" используйте команду

```
curl -u emma:12345 http://localhost:8080/documents/asd555
```

Тело ответа:

```
{
  "owner": "emma"
}
```

При вызове конечной точки для документа, принадлежащего Натали, и аутентификации с пользователем "emma" используйте команду

```
curl -u emma:12345 http://localhost:8080/documents/abc123
```

Тело ответа:

```
{
  "status": 403,
  "error": "Forbidden",
  "message": "Forbidden",
  "path": "/documents/abc123"
}
```

Использование аннотаций @Secured и @RolesAllowed

В этой главе мы обсуждали применение правил авторизации с глобальным методом безопасности. Мы начали с того, что узнали, что эта функциональность отключена по умолчанию и что вы можете включить ее с помощью аннотации @EnableMethodSecurity над классом конфигурации. Более того, при использовании предварительной авторизации и поставторизации вам не нужно указывать определенный способ применения правил авторизации с помощью атрибута аннотации @EnableMethodSecurity. Мы использовали аннотацию следующим образом:

@EnableMethodSecurity

Аннотация @EnableMethodSecurity имеет два атрибута, которые можно использовать для разрешения вариантов аннотации. Атрибут jsr250E-

nabled применяется для разрешения аннотации @RolesAllowed, а атрибут securedEnabled – для разрешения аннотации @Secured. Использование этих двух аннотаций менее эффективно, чем использование @PreAuthorize и @PostAuthorize, и вероятность того, что вы найдете их в реальных приложениях, невелика. Тем не менее я хотел бы ознакомить вас с ними, но не трата слишком много времени на детали.

Эти аннотации разрешают так же, как мы делали для предварительной авторизации и поставторизации, устанавливая атрибуты @EnableMethodSecurity в значение true. Можно активировать как @Secure, так и @RolesAllowed. Пример того, как это сделать, показан в следующем фрагменте кода:

```
@EnableMethodSecurity(
    jsr250Enabled = true,
    securedEnabled = true
)
```

После включения этих атрибутов можно использовать аннотации @RolesAllowed или @Secured, чтобы указать, какие роли или полномочия должен иметь вошедший в систему пользователь для вызова определенного метода. В следующем фрагменте кода показано, как использовать аннотацию @RolesAllowed, чтобы указать, что только пользователи с ролью ADMIN могут вызывать метод getName():

```
@Service
public class NameService {

    @RolesAllowed("ADMIN")
    public String getName() {
        return "Fantastico";
    }
}
```

Аналогично вы можете использовать вместо аннотации @RolesAllowed аннотацию @Secured:

```
@Service
public class NameService {

    @Secured("ROLE_ADMIN")
    public String getName() {
        return "Fantastico";
    }
}
```

Теперь можно протестировать пример. Выполните следующую команду:

```
curl -u emma:12345 http://localhost:8080/hello
```

Тело ответа:

```
Hello, Fantastico
```

Чтобы вызвать конечную точку и аутентифицироваться от имени пользователя Натали, выполните команду

```
curl -u natalie:12345 http://localhost:8080/hello
```

Тело ответа:

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

Вы можете найти полный код примера использования аннотаций `@RolesAllowed` и `@Secured` в проекте `ssia-ch9-ex6`.

Заключение

- Spring Security позволяет применять правила авторизации на любом уровне приложения, а не только на уровне конечной точки. Для этого мы активируем функционал безопасности метода.
- Функционал безопасности метода по умолчанию отключен. Чтобы включить его, мы используем аннотацию `@EnableMethodSecurity` над классом конфигурации вашего приложения.
- Вы можете применить правила авторизации, которые приложение проверяет перед вызовом метода. Если эти правила авторизации не соблюдаются, фреймворк не позволяет методу выполняться. Когда мы проверяем правила авторизации перед вызовом метода, это называется предварительной авторизацией.
- Для реализации предварительной авторизации мы используем аннотацию `@PreAuthorize`, а в качестве ее значения – выражение SpEL, которое определяет правило авторизации.
- Если мы только после вызова метода решаем, может ли вызывающая сторона использовать возвращаемое значение и может ли поток выполнения продолжиться, это называется поставторизацией.
- Для реализации поставторизации мы используем аннотацию `@PostAuthorize` со значением выражения SpEL, которое представляет правило авторизации.
- При разработке сложной логики авторизации вы должны вынести эту логику в отдельный класс, чтобы сделать ваш код более удобным для чтения. В Spring Security это обычно делается путем реализации `PermissionEvaluator`.
- Spring Security обеспечивает совместимость со старыми спецификациями, такими как аннотации `@RolesAllowed` и `@Secured`. Вы можете использовать их, но они менее эффективны, чем `@PreAuthorize` и `@PostAuthorize`, и вероятность того, что вы обнаружите их использование в Spring в современном приложении, очень мала.

12

Фильтрация на уровне метода

Темы этой главы:

- применение предварительной фильтрации для ограничения значений параметров, передаваемых методу;
- использование постфильтрации для ограничения результата, возвращаемого методом;
- интеграция фильтрации с Spring Data.

В главе 11 вы узнали, как применять правила авторизации с глобальной безопасностью методов. Мы работали над примерами с добавлением аннотаций `@PreAuthorize` и `@PostAuthorize`. Когда вы используете эти аннотации, приложение либо разрешает вызов метода, либо полностью отклоняет его. Предположим, вы не хотите запрещать вызов метода, но должны убедиться, что передаваемые входные параметры следуют некоторым правилам. Или, возможно, вам нужно сделать так, чтобы после срабатывания метода вызывающая сторона получала только разрешенную часть возвращаемого значения. Эта функциональная возможность называется *фильтрацией* и подразделяется на две категории:

- *предварительная фильтрация* (prefiltering) – фреймворк фильтрует значения параметров перед вызовом метода;

- *постфильтрация* (postfiltering) – фреймворк фильтрует возвращаемое значение после вызова метода.

Фильтрация работает иначе, чем авторизация вызова (рис. 12.1). При фильтрации фреймворк выполняет вызов и не выдает исключение, если параметр или возвращаемое значение не соответствует определенному вами правилу авторизации. Вместо этого он отфильтровывает элементы, которые не соответствуют указанным условиям.

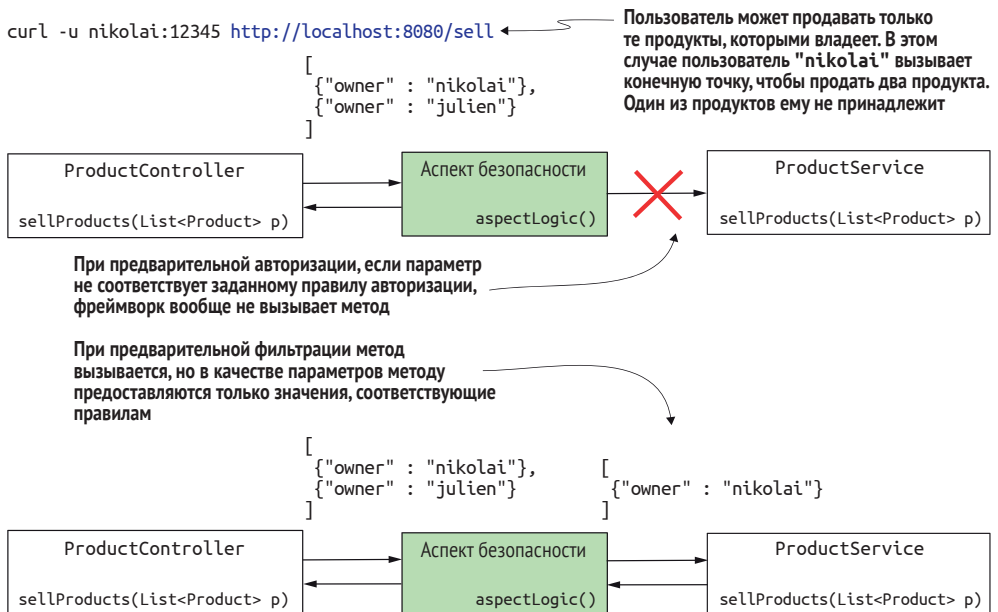


Рис. 12.1 Клиент вызывает конечную точку, предоставляя входное значение, которое не соответствует правилу авторизации. При предварительной авторизации метод вообще не вызывается, а вызывающий объект получает исключение. При предварительной фильтрации аспект всегда вызывает метод, но передает ему только значения, которые соответствуют заданным правилам

Важно сразу отметить, что фильтрация применяется только к коллекциям и массивам. Вы используете предварительную фильтрацию, только если метод получает в качестве параметра массив или коллекцию объектов. Фреймворк фильтрует эту коллекцию или массив в соответствии с определенными вами правилами. То же самое справедливо и для постфильтрации: вы можете применить этот подход, только если метод возвращает коллекцию или массив. Фреймворк фильтрует возвращаемое методом значение на основе указанных вами правил.

12.1 Применение предварительной фильтрации

В этом разделе обсуждается механизм предварительной фильтрации, а затем мы рассмотрим предварительную фильтрацию на примере. С помощью механизма фильтрации вы можете указать фреймворку проверять значения, отправленные через параметры метода. Фреймворк фильтрует значения, которые не соответствуют заданным критериям, и вызывает метод только со значениями, которые им соответствуют. Процесс предварительной фильтрации схематически показан на рис.12.2.

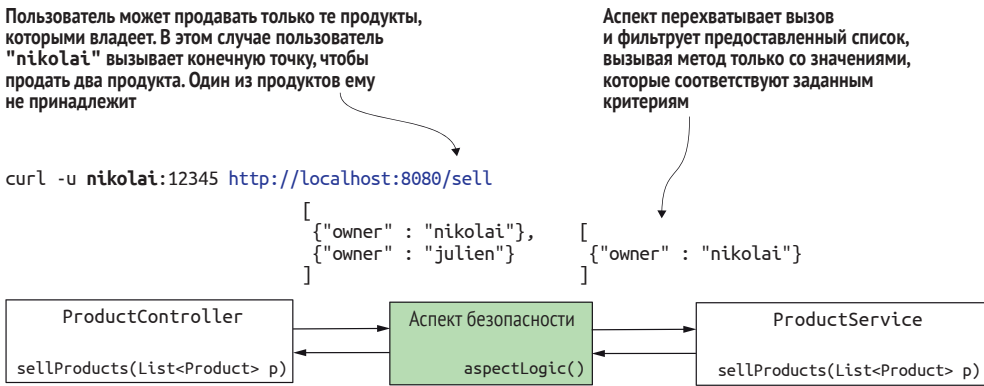


Рис. 12.2 При предварительной фильтрации аспект перехватывает вызов защищенного метода. Аспект фильтрует значения, которые вызывающий объект предоставляет в качестве параметра, и отправляет методу только значения, которые соответствуют определенным правилам

Предварительная фильтрация – это хорошая практика программирования, поскольку она отделяет правила авторизации от бизнес-логики, реализуемой методом. Допустим, вы хотите создать приложение, которое обрабатывает только определенные данные, принадлежащие аутентифицированному пользователю. Логiku обработки данных можно вызвать из нескольких мест. Тем не менее требование к приложению гласит, что могут быть обработаны только данные аутентифицированного пользователя, независимо от того, кто является вызывающей стороной. Вместо того чтобы проверять, что субъект, запрашивающий определенные действия, правильно авторизован на уровне фреймворка, вы можете применить собственные правила авторизации. Конечно, вы можете сделать это внутри метода. Но отделение логики авторизации от бизнес-логики повышает удобство обслуживания вашего кода и упрощает его чтение и понимание другими.

Как и в случае с авторизацией вызова, которую мы обсуждали в главе 11, Spring Security также реализует фильтрацию с помощью аспектов. Аспекты перехватывают определенные вызовы методов и могут дополнять их другими инструкциями. Для предварительной фильтрации аспект перехватывает методы, помеченные аннотацией `@PreFilter`, и фильтрует значения в коллекции, предоставленной в качестве параметра, в соответствии с критериями, которые вы определяете (рис. 12.3).

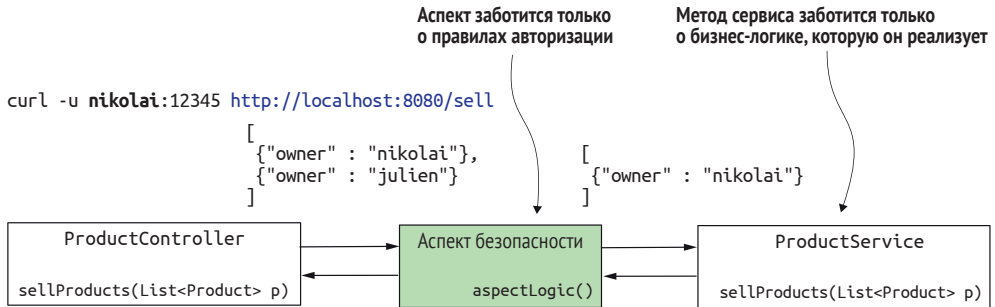


Рис. 12.3 С помощью предварительной фильтрации мы разделяем ответственность за авторизацию и реализацию бизнес-логики. Аспект, предоставляемый Spring Security, заботится только о правилах авторизации, а служебный метод заботится только о бизнес-логике варианта использования, который он реализует

Подобно аннотациям `@PreAuthorize` и `@PostAuthorize`, обсуждаемым в главе 11, мы устанавливаем правила авторизации как значение аннотации `@PreFilter`. В этих правилах, которые вы предоставляете как выражения SpEL, вы используете `filterObject` для ссылки на любой элемент внутри коллекции или массива, предоставленного в качестве параметра метода.

Чтобы рассмотреть применение предварительной фильтрации, давайте поработаем над проектом. Я назвал этот проект `ssia-ch12-ex1`. Допустим, у нас есть приложение для покупки и продажи неких продуктов, и его бэкэнд предоставляет конечную точку `/sell`. Фронтенд приложения вызывает эту конечную точку, когда пользователь продает продукт. Но вошедший в систему пользователь может продавать только те продукты, которыми он владеет. Давайте реализуем простой сценарий служебного метода, вызываемого для продажи продуктов, полученных в качестве параметра. Из этого примера вы узнаете, как применять аннотацию `@PreFilter`, поскольку именно ее мы используем, чтобы убедиться, что метод получает только продукты, принадлежащие текущему авторизованному пользователю.

После создания проекта мы пишем класс конфигурации, чтобы у нас было несколько пользователей для тестирования нашей реали-

зации. Вы найдете простое определение класса конфигурации в листинге 12.1. Класс конфигурации, который я называю `ProjectConfig`, объявляет только `UserDetailsService` и `PasswordEncoder`, и я аннотирую его с помощью `@EnableMethodSecurity`. Для аннотирования фильтрации нам все еще нужно использовать аннотацию `@EnableMethodSecurity` и включить аннотации до и после авторизации. Рассматриваемый `UserDetailsService` определяет двух пользователей, которые пригодятся при тестировании: Николай и Жюльен.

Листинг 12.1 Настройка пользователей и включение безопасности метода

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var uds = new InMemoryUserDetailsManager();

        var u1 = User.withUsername("nikolai")
            .password("12345")
            .authorities("read")
            .build();

        var u2 = User.withUsername("julien")
            .password("12345")
            .authorities("write")
            .build();

        uds.createUser(u1);
        uds.createUser(u2);

        return uds;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

Я описываю продукт с помощью класса модели, представленного в следующем листинге.

Листинг 12.2 Определение класса `Product`

```
public class Product {

    private String name;
```

```
private String owner;

// Конструктор, геттеры и сеттеры не показаны
}
```

❶ Атрибуту `owner` присвоено значение имени пользователя.

Класс `ProductService` определяет служебный метод, который мы защищаем с помощью `@PreFilter`. Код класса `ProductService` показан в листинге 12.3. В этом листинге перед методом `sellProducts()` вы можете видеть использование аннотации `@PreFilter`. Применяется выражение на языке Spring (SpEL) `filterObject.owner == authentication.name`, разрешающее только значения, в которых атрибут владельца `Product` равен имени пользователя, вошедшего в систему. С левой стороны оператора равенства в выражении SpEL мы используем `filterObject`. С помощью `filterObject` мы ссылаемся на объекты в списке как на параметры. Поскольку у нас есть список продуктов, `filterObject` в нашем случае имеет тип `Product`. По этой причине мы можем ссылаться на атрибут владельца продукта. С правой стороны оператора равенства в выражении мы используем объект аутентификации. В случае аннотаций `@PreFilter` и `@PostFilter` мы можем напрямую ссылаться на объект аутентификации, который доступен в `SecurityContext` после процедуры аутентификации (рис. 12.4).

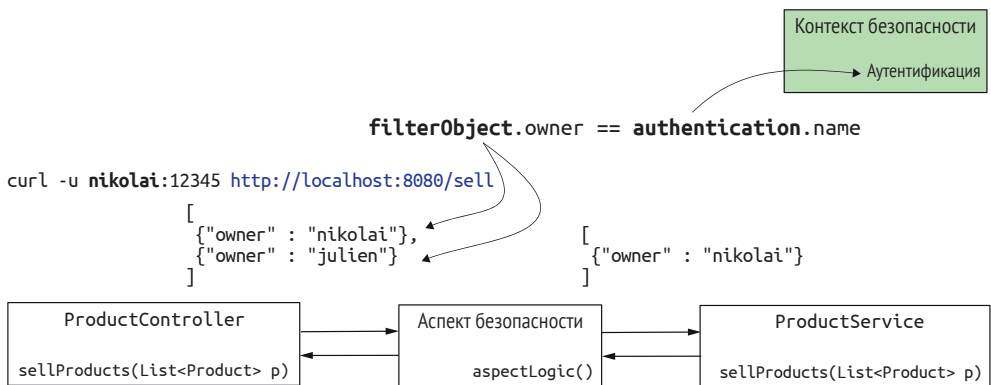


Рис. 12.4 При использовании предварительной фильтрации с помощью `filterObject` мы ссылаемся на объекты внутри списка, которые вызывающая сторона предоставляет в качестве параметра. Объект аутентификации сохраняется после процесса аутентификации в контексте безопасности

Метод службы возвращает список точно так же, как он его получает. Поэтому мы можем проверить его и убедиться, что фреймворк отфильтровал список так, как мы ожидали. Для этого мы проверяем список, возвращенный в теле ответа HTTP.

Листинг 12.3 Использование аннотации @PreFilter в классе ProductService

```
@Service
public class ProductService {

    @PreFilter("filterObject.owner == authentication.name")
    public List<Product> sellProducts(List<Product> products) {
        // продаем продукты и возвращаем список проданных продуктов
        return products;
    }
}
```

- ❶ Список, заданный в качестве параметра, содержит только продукты, принадлежащие аутентифицированному пользователю.
- ❷ Возвращаем список продуктов для целей тестирования.

Чтобы упростить наши тесты, я определяю конечную точку для вызова защищенного служебного метода. Листинг 12.4 определяет эту конечную точку в классе контроллера с именем `ProductController`. Здесь, чтобы сделать вызов конечной точки короче, я создаю список и напрямую предоставляю его в качестве параметра служебному методу. В реальном приложении этот список должен быть предоставлен клиентом в теле запроса. Вы также можете заметить, что я использую `@GetMapping` для операции, предполагающей мутацию, что нестандартно. Но знайте, что я делаю это, чтобы избежать работы с защитой от CSRF в нашем примере, и это позволяет вам сосредоточиться на рассматриваемой теме. Вы узнали о защите от CSRF в главе 9.

Листинг 12.4 Класс контроллера, реализующий конечную точку, которую мы используем для тестов

```
@RestController
public class ProductController {

    private final ProductService productService;

    // Конструктор не показан

    @GetMapping("/sell")
    public List<Product> sellProduct() {
        List<Product> products = new ArrayList<>();

        products.add(new Product("beer", "nikolai"));
        products.add(new Product("candy", "nikolai"));
        products.add(new Product("chocolate", "julien"));

        return productService.sellProducts(products);
    }
}
```

Теперь запустим приложение и посмотрим, что произойдет, когда мы вызовем конечную точку `/sell`. Взгляните на три продукта из списка, который мы предоставили в качестве параметра служебному методу. Я назначаю два продукта пользователю Николаю, а один – пользователю Жюльену. Когда мы вызываем конечную точку и аутентифицируемся от имени Николая, мы ожидаем увидеть в ответе только два продукта, связанных с ним. Когда мы вызываем конечную точку и аутентифицируемся от имени Жюльена, в ответе мы должны найти только один продукт, связанный с ней. Ниже показаны тестовые вызовы и их результаты. Чтобы вызвать конечную точку `/sell` и аутентифицироваться от имени Николая, используйте такую команду:

```
curl -u nikolai:12345 http://localhost:8080/sell
```

Тело ответа:

```
[  
  {"name": "beer", "owner": "nikolai"},  
  {"name": "candy", "owner": "nikolai"}  
]
```

Чтобы вызвать конечную точку `/sell` и аутентифицироваться от имени Жюльена, используйте

```
curl -u julien:12345 http://localhost:8080/sell
```

Тело ответа:

```
[  
  {"name": "chocolate", "owner": "julien"}  
]
```

Принимайте во внимание тот факт, что аспект изменяет заданную коллекцию. В нашем случае не ожидайте, что он вернет *новый* экземпляр `List`. Фактически это прежний экземпляр, из которого аспект удалил элементы, не соответствующие заданным критериям. Это важно учитывать. Вы обязательно должны убедиться, что предоставленный вами экземпляр коллекции *не* является неизменяемым. Попытка обработать неизменяемую коллекцию приводит к исключению во время выполнения, поскольку аспект фильтрации не сможет изменить содержимое коллекции (рис. 12.5).

Следующий листинг представляет тот же проект, над которым мы работали ранее в этом разделе, но я изменил определение `List` на неизменяемый экземпляр, возвращаемый методом `List.of()`, чтобы проверить, что произойдет в этой ситуации.

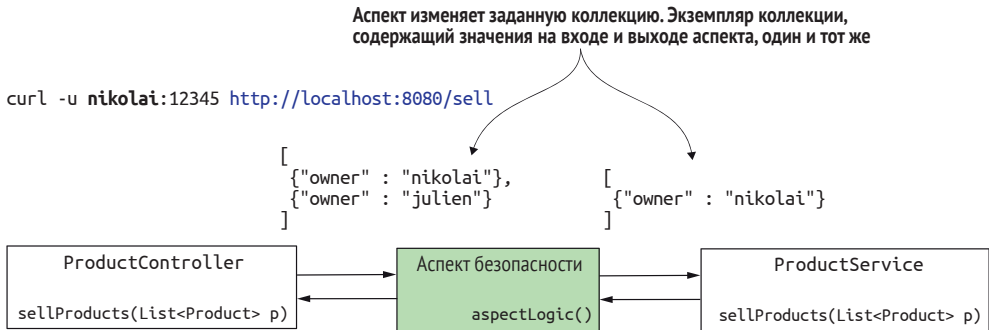


Рис. 12.5 Аспект перехватывает и изменяет коллекцию, указанную в качестве параметра. Вам необходимо предоставить изменяемый экземпляр коллекции, чтобы аспект мог ее изменить

Листинг 12.5 Использование неизменяемой коллекции

```
@RestController
public class ProductController {

    private final ProductService productService;

    // Конструктор не показан

    @GetMapping("/sell")
    public List<Product> sellProduct() {
        List<Product> products = List.of(
            new Product("beer", "nikolai"),
            new Product("candy", "nikolai"),
            new Product("chocolate", "julien"));

        return productService.sellProducts(products);
    }
}
```

❶ `List.of()` возвращает неизменяемый экземпляр списка.

Я выделил этот пример в папке проекта `ssia-ch12-ex2`, чтобы вы могли протестировать его самостоятельно. Запуск приложения и вызов конечной точки `/sell` приводит к ответу HTTP со статусом 500 Internal Server Error (внутренняя ошибка сервера) и исключению в журнале консоли, как показано ниже:

```
curl -u julien:12345 http://localhost:8080/sell
```

Тело ответа:

```
{
  "status":500,
  "error":"Internal Server Error",
  "path":"/sell"
}
```

В консоли приложения вы можете найти исключение, похожее на показанное в следующем фрагменте вывода:

```
java.lang.UnsupportedOperationException: null
    at java.base/java.util.ImmutableCollections.uoe(ImmutableCollections.
java:73) ~[na:na]
...
```

12.2 Применение постфильтрации

В этом разделе мы реализуем постфильтрацию. Предположим, перед нами стоит следующая задача. Приложение, имеющее интерфейс, выполненный в Angular, и бэкенд на основе Spring, управляет некоторыми продуктами. Пользователи владеют продуктами и могут получать сведения только о своих продуктах. Чтобы предоставить клиенту сведения о его продуктах, интерфейс вызывает конечные точки, предоставленные бэкендом (рис. 12.6).

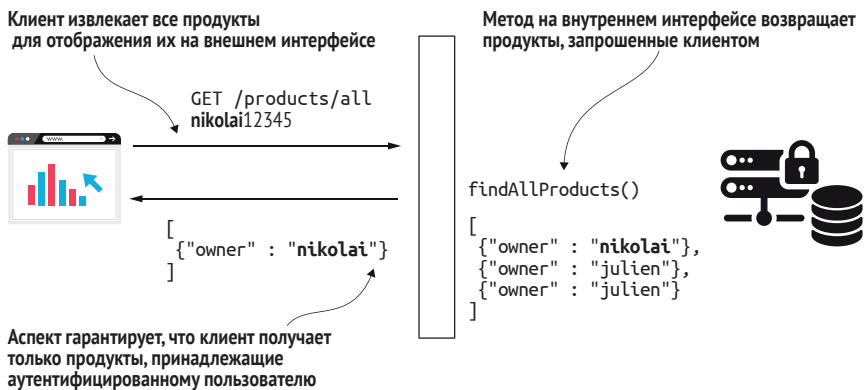


Рис. 12.6 Сценарий постфильтрации. Клиент вызывает конечную точку для извлечения данных, которые ему необходимо отобразить во внешнем интерфейсе. Применение постфильтрации гарантирует, что клиент получает только данные, принадлежащие текущему аутентифицированному пользователю

На стороне бэкенда в служебном классе разработчик разместил метод `List<Product> findProducts()`, который извлекает сведения о продукте. Клиентское приложение отображает эти сведения во внешнем интерфейсе. Как разработчик может сделать так, чтобы любой клиент, вызывающий этот метод, получал только продукты, которыми владеет он, а не продукты, принадлежащие другим клиентам? Способ реализации этой функции путем определения правил авторизации, отделенных от бизнес-правил приложения, назы-

вается *постфильтрацией*. В этом разделе обсуждается, как работает постфильтрация, и демонстрируется ее реализация в приложении.

Подобно предварительной фильтрации, постфильтрация также опирается на аспект. Этот аспект позволяет вызвать метод, но после того, как метод возвращает значение, аспект принимает возвращаемое значение и проверяет, что оно соответствует определенным вами правилам. Как и в случае предварительной фильтрации, постфильтрация изменяет коллекцию или массив, возвращаемые методом. Вы указываете критерии, которым должны следовать элементы внутри возвращаемой коллекции. Аспект постфильтрации отфильтровывает из возвращаемой коллекции или массивов те элементы, которые не соответствуют вашим правилам.

Чтобы применить постфильтрацию, вам нужно добавить аннотацию `@PostFilter`. Аннотация `@PostFilter` работает аналогично всем остальным пре-/постаннотациям, которые мы использовали в главе 11 и в этой главе. Мы предоставляем правило авторизации как выражение SpEL для значения аннотации, и затем это правило применяет аспект фильтрации, как показано на рис. 12.7. Кроме того, подобно предварительной фильтрации, постфильтрация работает только с массивами и коллекциями. Убедитесь, что вы применяете аннотацию `@PostFilter` только для методов, которые возвращают массив или коллекцию.

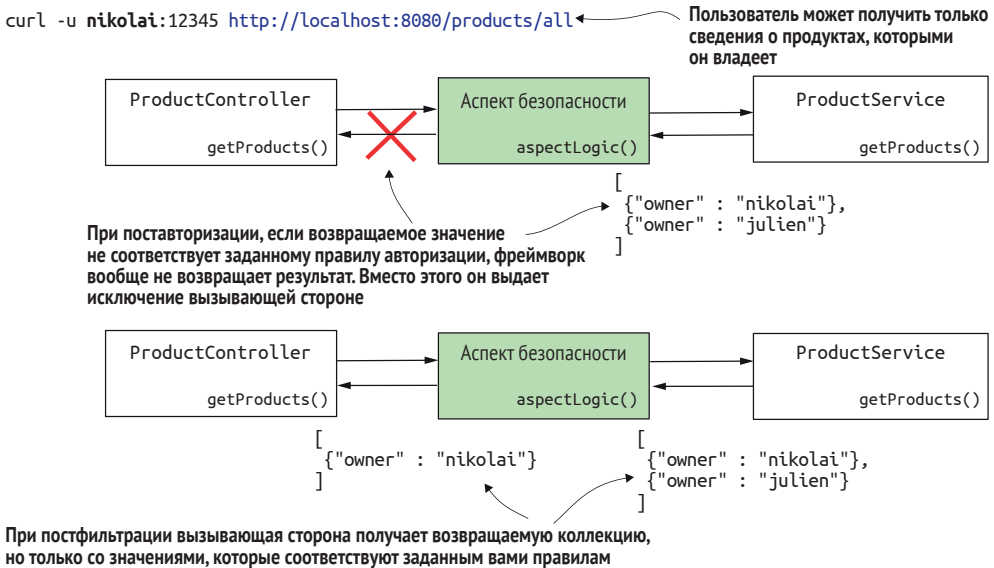


Рис. 12.7 Постфильтрация. Аспект перехватывает коллекцию, возвращаемую защищенным методом, и фильтрует значения, которые не соответствуют предоставленным вами правилам. В отличие от поставторизации, постфильтрация не выдает исключение вызывающему объекту, когда возвращаемое значение не следует правилам авторизации

Мы применим постфильтрацию в примере, для которого я создал проект с именем `ssia-ch12-ex3`. Чтобы быть последовательным, я сохранил тех же пользователей, что и в наших предыдущих примерах в этой главе, поэтому класс конфигурации не изменился. Для вашего удобства я повторяю конфигурацию, представленную в листинге 12.6.

Листинг 12.6 Класс конфигурации

```
@Configuration
@EnableMethodSecurity
public class ProjectConfig {

    @Bean
    public UserDetailsService userDetailsService() {
        var uds = new InMemoryUserDetailsManager();

        var u1 = User.withUsername("nikolai")
            .password("12345")
            .authorities("read")
            .build();

        var u2 = User.withUsername("julien")
            .password("12345")
            .authorities("write")
            .build();

        uds.createUser(u1);
        uds.createUser(u2);

        return uds;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

Следующий фрагмент кода показывает, что класс `Product` также остается неизменным:

```
public class Product {

    private String name;
    private String owner;

    // Конструктор, геттеры и сеттеры не показаны
}
```

В классе `ProductService` мы теперь реализуем метод, который возвращает список продуктов. В реальном сценарии предполагается,

что приложение будет считывать продукты из базы данных или любого другого источника данных. Чтобы сделать наш пример кратким и сосредоточиться на обсуждаемой теме, мы используем простую коллекцию, как показано в листинге 12.7.

Я аннотирую метод `findProducts()`, возвращающий список продуктов, аннотацией `@PostFilter`. В качестве значения аннотации я добавляю условие `filterObject.owner == authentication.name`, которое позволяет возвращать только те продукты, владелец которых совпадает с аутентифицированным пользователем (рис. 12.8). С левой стороны оператора равенства мы используем `filterObject` для ссылки на элементы внутри возвращаемой коллекции. С правой стороны оператора мы используем аутентификацию для ссылки на объект `Authentication`, сохраненный в `SecurityContext`.

```
curl -u nikolai:12345 http://localhost:8080/products/all
```

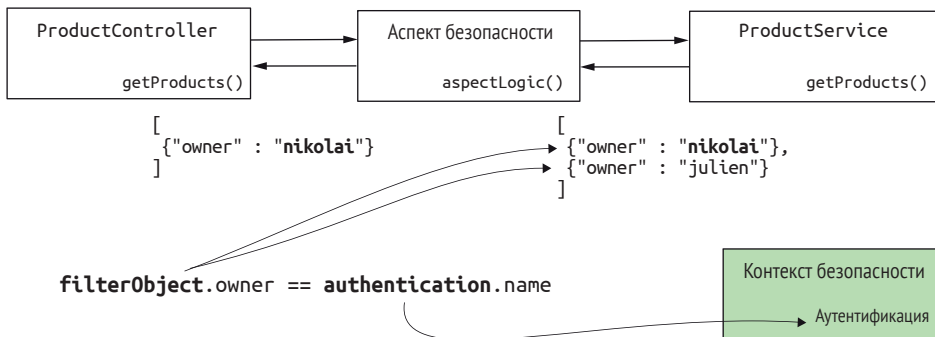


Рис. 12.8 В выражении SpEL, применяемом для авторизации, мы используем `filterObject` для ссылки на объекты в возвращаемой коллекции, а также аутентификацию для ссылки на экземпляр `Authentication` из контекста безопасности

Листинг 12.7 Класс `ProductService`

```
@Service
public class ProductService {

    @PostFilter("filterObject.owner == authentication.name") ❶
    public List<Product> findProducts() {
        List<Product> products = new ArrayList<>();

        products.add(new Product("beer", "nikolai"));
        products.add(new Product("candy", "nikolai"));
        products.add(new Product("chocolate", "julien"));

        return products;
    }
}
```

❶ Добавляем условие фильтрации объектов в коллекции, возвращаемой методом.

Кроме того, я определяю класс контроллера, чтобы сделать метод доступным через конечную точку. Код класса показан в листинге 12.8.

Листинг 12.8 Класс ProductController

```
@RestController
public class ProductController {

    private final ProductService productService;

    // Конструктор не показан

    @GetMapping("/find")
    public List<Product> findProducts() {
        return productService.findProducts();
    }
}
```

Пора запустить приложение и протестировать его поведение, вызвав конечную точку `/find`. Мы ожидаем увидеть в теле ответа HTTP только продукты, принадлежащие аутентифицированному пользователю. Следующие фрагменты кода показывают результат вызова конечной точки каждым из наших пользователей, Николаем и Жюльеном. Чтобы вызвать конечную точку `/find` и аутентифицироваться от имени Жюльена, выполните следующую команду cURL:

```
curl -u julien:12345 http://localhost:8080/find
```

Тело ответа:

```
[
  {"name": "chocolate", "owner": "julien"}
]
```

Чтобы вызвать конечную точку `/find` и аутентифицироваться от имени Николая, выполните следующую команду cURL:

```
curl -u nikolai:12345 http://localhost:8080/find
```

Тело ответа:

```
[
  {"name": "beer", "owner": "nikolai"},
  {"name": "candy", "owner": "nikolai"}
]
```

12.3 Использование фильтрации в репозиториях *Spring Data*

В этом разделе мы обсудим фильтрацию, применяемую в репозиториях *Spring Data*. Вам будет полезно понимать этот механизм, поскольку мы часто используем базы данных для сохранения данных приложения. Довольно часто встречаются приложения *Spring Boot*, которые используют *Spring Data* в качестве высокоуровневого слоя для подключения к базе данных, такой как SQL или NoSQL. Мы обсудим два подхода к применению фильтрации на уровне репозитория при использовании *Spring Data* и опробуем их на примерах.

Первый подход вы изучили ранее в этой главе: использование аннотаций `@PreFilter` и `@PostFilter`. Второй подход, который нам предстоит рассмотреть, – это прямая интеграция правил авторизации в запросы. Как вы узнаете в этом разделе, вам нужно быть внимательным при выборе способа применения фильтрации в репозиториях *Spring Data*. Как я уже сказал, у нас есть два варианта:

- использование аннотаций `@PreFilter` и `@PostFilter`;
- прямое применение фильтрации в запросах.

Использование аннотации `@PreFilter` в случае репозитория – это то же самое, что и применение этой аннотации на любом другом уровне вашего приложения. Но когда дело доходит до постфильтрации, ситуация меняется. Механизм `@PostFilter` в методах репозитория технически работает нормально, но это редко бывает хорошим выбором с точки зрения производительности.

Допустим, у вас есть приложение, управляющее документами вашей компании. Разработчику необходимо реализовать функцию, в которой все документы будут перечислены на веб-странице после входа пользователя в систему. Разработчик решает использовать метод `findAll()` репозитория *Spring Data* и аннотирует его с помощью `@PostFilter`, чтобы после фильтрации метод возвращал только те документы, которые принадлежат текущему вошедшему в систему пользователю. Этот подход явно неправильный, поскольку он заставляет приложение извлекать все записи из базы данных, а затем фильтровать их самостоятельно. Если у нас большое количество документов, вызов `findAll()` без разбиения на страницы может напрямую привести к исключению `OutOfMemoryError`. Даже если количество документов недостаточно велико для заполнения кучи, все равно фильтровать записи в приложении хуже, чем извлекать только то, что вам нужно из базы данных (рис. 12.9).

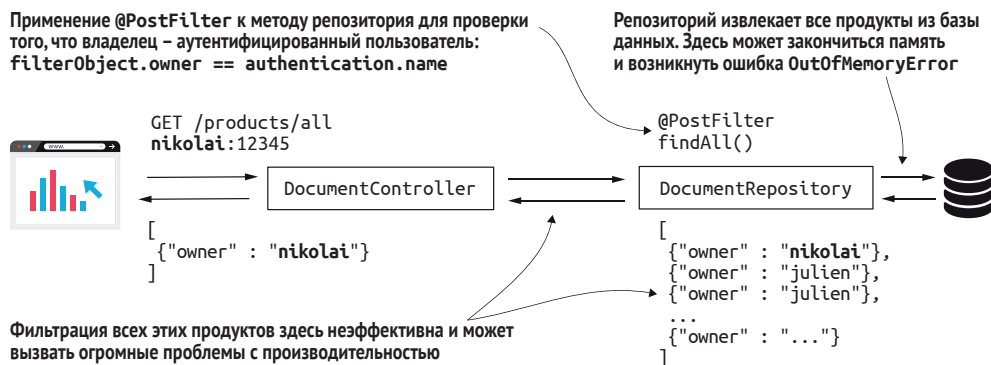


Рис. 12.9 Плохая структура приложения. Когда вам нужно применить фильтрацию на уровне репозитория, лучше сначала убедиться, что вы извлекаете только нужные вам данные. В противном случае ваше приложение может столкнуться с серьезными проблемами с памятью и производительностью

На уровне службы у вас нет другого выбора, кроме как отфильтровать записи в приложении. Тем не менее если на уровне репозитория известно, что вам нужно извлекать только записи, принадлежащие вошедшему в систему пользователю, вам следует реализовать запрос, который извлекает из базы данных только необходимые документы.

ПРИМЕЧАНИЕ В любой ситуации, когда вы извлекаете данные из источника данных, будь то база данных, веб-служба, входной поток или что-то еще, убедитесь, что приложение извлекает только нужные ему данные. Избегайте, насколько это возможно, необходимости фильтровать данные внутри приложения.

Давайте поработаем над приложением, в котором мы сначала используем аннотацию `@PostFilter` в методе репозитория Spring Data, а затем перейдем ко второму подходу, где мы применяем условие непосредственно в запросе. Таким образом, у нас будет возможность поэкспериментировать с обоими подходами и сравнить их.

Я создал новый проект с именем `ssia-ch12-ex4`, в котором использую тот же класс конфигурации, что и в наших предыдущих примерах в этой главе. Как и в предыдущих примерах, мы разрабатываем приложение, управляющее продуктами, но на этот раз мы извлекаем сведения о продукте из таблицы в базе данных. Для нашего примера мы разработаем функцию поиска продуктов (рис. 12.10). Мы создаем конечную точку, которая получает строку и возвращает список продуктов, в названиях которых есть заданная строка. Однако нам нужно убедиться, что возвращаются только продукты, связанные с аутентифицированным пользователем.

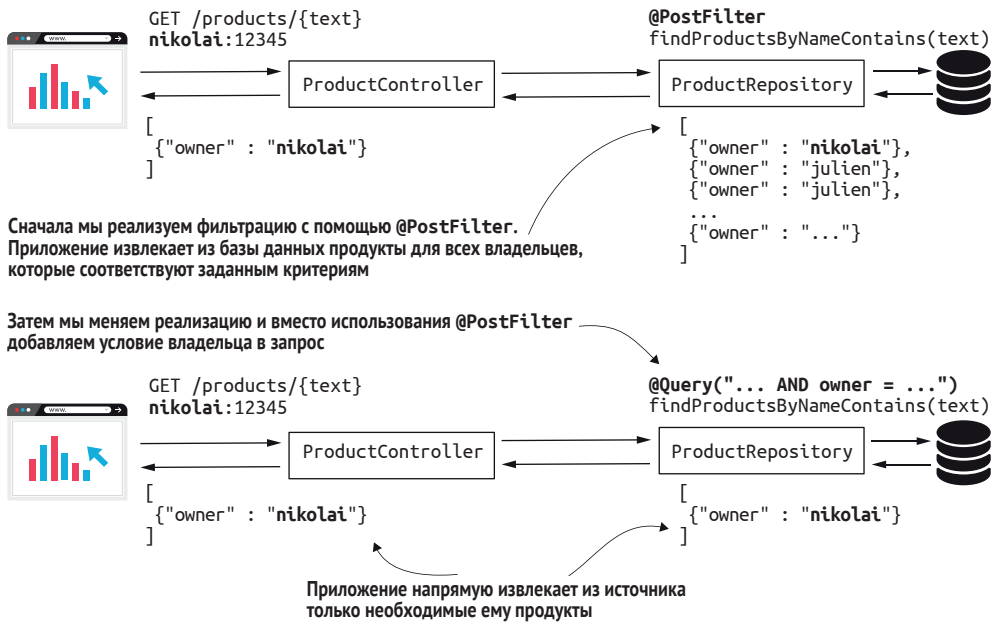


Рис. 12.10 В нашем сценарии мы начинаем с реализации приложения с использованием `@PostFilter` для фильтрации продуктов на основе их владельца. Затем мы изменяем код, чтобы добавить условие непосредственно в запрос. Во втором случае приложение получает из источника только необходимые записи

Для подключения к базе данных мы применяем Spring Data JPA. По этой причине нам также необходимо добавить в файл `pom.xml` зависимость `spring-boot-starter-data-jpa` и драйвер подключения в соответствии с технологией вашего сервера управления базами данных. Следующий фрагмент кода предоставляет зависимости, которые я использую в файле `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
```

В файле `application.properties` мы добавляем свойства, необходимые Spring Boot для создания источника данных. В следующем фрагменте кода показаны свойства, которые я добавил в свой файл `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost/  
spring?useLegacyDatetimeCode=false&serverTimezone=UTC  
spring.datasource.username=root  
spring.datasource.password=  
spring.datasource.initialization-mode=always
```

Нам также нужна таблица в базе данных для хранения сведений о продукте, которые извлекает наше приложение. Мы определяем файл `schema.sql`, в котором пишем скрипт для создания таблицы, и файл `data.sql`, в котором пишем запросы для вставки тестовых данных в таблицу. Вам нужно поместить оба файла (`schema.sql` и `data.sql`) в папку ресурсов проекта Spring Boot, чтобы они были найдены и выполнены при запуске приложения. В следующем фрагменте кода показан запрос для создания таблицы, который нам нужно записать в файл `schema.sql`:

```
CREATE TABLE IF NOT EXISTS `spring`.`product` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(45) NULL,  
  `owner` VARCHAR(45) NULL,  
  PRIMARY KEY (`id`));
```

В файле `data.sql` я пишу три оператора `INSERT`, как показано в следующем фрагменте кода. Эти операторы создают тестовые данные, которые нам понадобятся позже для проверки работы приложения:

```
INSERT IGNORE INTO `spring`.`product` (`id`, `name`, `owner`) VALUES ('1',  
'beer', 'nikolai');  
INSERT IGNORE INTO `spring`.`product` (`id`, `name`, `owner`) VALUES ('2',  
'candy', 'nikolai');  
INSERT IGNORE INTO `spring`.`product` (`id`, `name`, `owner`) VALUES ('3',  
'chocolate', 'julien');
```

ПРИМЕЧАНИЕ Напомню, что мы использовали те же имена для таблиц в других примерах в книге. Если у вас уже есть таблицы с такими же именами из предыдущих примеров, вам, вероятно, следует удалить их перед началом работы с этим проектом. Альтернативой является использование другой схемы.

Чтобы сопоставить таблицу продуктов в нашем приложении, нам нужно написать класс-сущность. В листинге 12.9 показано определение класса-сущности `Product`.

Листинг 12.9 Класс-сущность Product

```

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private String owner;

    // Геттеры и сеттеры не показаны
}

```

Для сущности `Product` мы также создаем интерфейс репозитория `Spring Data`, определенный в следующем листинге. Обратите внимание, что на этот раз мы используем аннотацию `@PostFilter` непосредственно в методе, объявленном интерфейсом репозитория.

Листинг 12.10 Интерфейс ProductRepository

```

public interface ProductRepository
    extends JpaRepository<Product, Integer> {

    @PostFilter("filterObject.owner == authentication.name") ❶
    List<Product> findProductByNameContains(String text);
}

```

❶ Применяем аннотацию `@PostFilter` к методу, объявленному репозиторием `Spring Data`

В листинге 12.11 показано определение класса контроллера, реализующего конечную точку, которую мы используем для тестирования поведения.

Листинг 12.11 Класс ProductController

```

@RestController
public class ProductController {

    private final ProductRepository productRepository;

    // Конструктор не показан

    @GetMapping("/products/{text}")
    public List<Product> findProductsContaining(
        @PathVariable String text) {

        return productRepository.findProductByNameContains(text);
    }
}

```

Запустив приложение, мы можем проверить, что происходит при вызове конечной точки `/products/{text}`. При поиске буквы `c` от имени аутентифицированного пользователя Николая ответ HTTP содержит только продукт `candy` (конфета). Продукт `chocolate` (шоколад) также содержит букву `c` в названии, но им владеет Жюльен, поэтому `chocolate` не появится в ответе. Вызовы и ответы на них вы найдете в приведенных ниже фрагментах кода. Чтобы вызвать конечную точку `/products` от имени аутентифицированного пользователя Николая, выполните следующую команду:

```
curl -u nikolai:12345 http://localhost:8080/products/c
```

Тело ответа:

```
[
  { "id": 2, "name": "candy", "owner": "nikolai" }
]
```

Чтобы вызвать конечную точку `/products` от имени аутентифицированного пользователя Жюльена, выполните следующую команду:

```
curl -u julien:12345 http://localhost:8080/products/c
```

Тело ответа:

```
[
  { "id": 3, "name": "chocolate", "owner": "julien" }
]
```

Ранее в этом разделе я говорил, что использование `@PostFilter` в репозитории – не лучший выбор. Вместо этого мы должны стараться выбирать из источника данных только нужные записи. Какие изменения нужно внести в код, чтобы выбирать только необходимые данные вместо фильтрации данных после выбора? Мы можем применить выражения SpEL непосредственно в запросах, используемых классами репозитория. Для этого мы выполняем два простых шага:

- 1 Добавляем объект типа `SecurityEvaluationContextExtension` в контекст Spring. Это можно сделать с помощью простого метода `@Bean` в классе конфигурации.
- 2 Настраиваем запросы в классах репозитория с помощью соответствующих условий выбора.

В нашем проекте, чтобы добавить бин `SecurityEvaluationContextExtension` в контекст, нам нужно изменить класс конфигурации, как показано в листинге 12.12. Чтобы не трогать предыдущий код, связанный с примерами в книге, я использую здесь другой проект под названием `ssia-ch12-ex5`.

Листинг 12.12 Добавление `SecurityEvaluationContextExtension` в контекст

```

@Configuration
@EnableMethodSecurity
public class ProjectConfig {

    @Bean
    public SecurityEvaluationContextExtension
        securityEvaluationContextExtension() {

        return new SecurityEvaluationContextExtension();

    }

    // Не показано объявление UserDetailsService и PasswordEncoder
}

```

❶ Добавляем `SecurityEvaluationContextExtension` в контекст Spring.

В интерфейсе `ProductRepository` мы добавляем запрос перед методом и настраиваем условие WHERE с помощью соответствующего выражения SpEL. Изменение показано в листинге 12.13.

Листинг 12.13 Использование SpEL в запросе в интерфейсе репозитория

```

public interface ProductRepository
    extends JpaRepository<Product, Integer> {

    Query("""SELECT p FROM Product p WHERE
           p.name LIKE %:text% AND
           p.owner=?#{authentication.name}
           """)
    List<Product> findProductByNameContains(String text);

}

```

❶ Используем SpEL в запросе для добавления условия владельца записи.

Теперь мы можем запустить приложение и протестировать его, вызвав конечную точку `/products/{text}`. Мы ожидаем, что поведение останется таким же, как и в случае, когда мы использовали `@PostFilter`. Но теперь из базы данных извлекаются только записи для нужного владельца, что делает приложение более быстрым и надежным. В следующих фрагментах кода представлены вызовы конечной точки. Чтобы вызвать конечную точку `/products` от имени аутентифицированного пользователя Николая, выполните команду

```
curl -u nikolai:12345 http://localhost:8080/products/c
```

Тело ответа:

```
[  
  {"id":2,"name":"candy","owner":"nikolai"}  
]
```

Чтобы вызвать конечную точку /products от имени аутентифицированного пользователя Жюльена, выполните команду

```
curl -u julien:12345 http://localhost:8080/products/c
```

Тело ответа:

```
[  
  {"id":3,"name":"chocolate","owner":"julien"}  
]
```

Заключение

- Фильтрация – это подход к авторизации, при котором фреймворк проверяет входные параметры метода или возвращаемое методом значение и исключает элементы, которые не соответствуют некоторым определенным вами критериям. С точки зрения авторизации фильтрация фокусируется на входных и выходных значениях метода, а не на самом выполнении метода.
- Фильтрацию используют, чтобы гарантировать, что метод не получит значений, отличных от тех, которые ему разрешено обрабатывать, и что он не может вернуть значения, которые вызывающая сторона метода не должна получать.
- При использовании фильтрации вы не ограничиваете доступ к методу, но ограничиваете данные, которые могут быть отправлены через параметры метода или содержимое ответа, возвращенного методом. Этот подход позволяет вам контролировать ввод и вывод метода.
- Для ограничения значений, которые могут быть отправлены через параметры метода, применяется аннотация `@PreFilter`. Аннотация `@PreFilter` получает условие, при котором значения разрешено отправлять в качестве параметров метода. Фреймворк отфильтровывает из коллекции, заданной в качестве параметра, все значения, которые не соответствуют заданному правилу.
- Для использования аннотации `@PreFilter` параметр метода должен быть коллекцией или массивом. Из выражения SpEL, которое

определяет правило, мы ссылаемся на объекты внутри коллекции с помощью `filterObject`.

- Для ограничения значений, возвращаемых методом, используется аннотация `@PostFilter`. При использовании этой аннотации метод также должен возвращать коллекцию или массив. Фреймворк фильтрует значения в возвращаемой коллекции в соответствии с правилом, которое вы определяете как значение аннотации `@PostFilter`.
- Вы также можете использовать аннотации `@PreFilter` и `@PostFilter` с репозиториями Spring Data. Но использование `@PostFilter` в методе репозитория Spring Data редко является хорошим выбором. Чтобы избежать проблем с производительностью, в этом случае фильтрация результата должна выполняться непосредственно на уровне запроса базы данных.
- Spring Security легко интегрируется с Spring Data, и этим можно воспользоваться, чтобы избежать проблем с `@PostFilter` при работе с репозиторием Spring Data.

Часть IV

Внедрение OAuth 2 и OpenID Connect

В эпоху, когда безопасные и бесшовные методы аутентификации имеют первостепенное значение, такие протоколы, как OAuth 2 и OpenID Connect, стали отраслевыми стандартами. В этой части книги раскрываются тонкости упомянутых протоколов, проливается свет на их механизмы, преимущества и потенциальные подводные камни.

Глава 13 задает тон, формируя всеобъемлющее представление о протоколах, подробно описывая различные типы предоставления токенов и выделяя потенциальные уязвимости в OAuth 2.

Глава 14 содержит подробное описание настройки надежного сервера авторизации Spring Security, включая определение данных клиента и управление криптографическими ключами.

Глава 15 дает руководство по созданию отказоустойчивого сервера ресурсов, подчеркивая интроспекцию токенов и обеспечение защиты ресурсов.

Глава 16 завершает эту часть, демонстрируя получение токена с сервера авторизации и доступ к ресурсам под защитой сервера ресурсов.

После прочтения этого раздела вы приобретете навыки интеграции OAuth 2 и OpenID Connect в свои приложения для защиты от несанкционированного доступа и обеспечения пользовательского опыта бесшовной авторизации.

К концу этой части вы приобретете опыт тщательной разработки и внедрения комплексных стратегий авторизации, благодаря чему ваше приложение останется как функциональным, так и защищенным.

13

Что такое OAuth 2 и OpenID Connect?

Темы этой главы:

- назначение токенов доступа;
- как токены выдаются и проверяются в системе OAuth 2;
- роли, задействованные в системе OAuth 2/OpenID Connect.

Предположим, вы работаете в крупной организации и используете несколько инструментов в своей повседневной работе. У вас задействованы приложения для отслеживания ошибок, приложения для документирования своей работы, приложения для регистрации рабочего времени и т. д. Для каждого из них требуется инструмент аутентификации. Станете ли вы использовать разные наборы учетных данных для этих приложений? Конечно, это может сработать, но такой подход будет обременительным для пользователя (вас), а также усложнит работу ваших приложений.

Для вас сложность заключается в том, что вам придется помнить разные учетные данные и по очереди входить в несколько приложений. Дополнительная сложность приложений возникает из-за необходимости хранить и защищать учетные данные и текущие сеансы аутентификации.

А как насчет управления ответственностью за хранение учетных данных и аутентификации в отдельном приложении? В этом случае

пользователям достаточно будет войти в систему только один раз и использовать все свои приложения, не утруждая себя повторной аутентификацией. Существует ли такое решение? Да. Вы можете реализовать аутентификацию по спецификации OAuth 2.

Но мы можем пойти еще дальше. Приложению, которое могут использовать публичные пользователи (пользователи за пределами организации – ваше приложение предназначено для всего мира), тоже понадобится аутентификация. Приложение может выполнять эту функцию, но

- реализация аутентификации в этом приложении требует больше усилий;
- пользователям нужен отдельный набор учетных данных для этого приложения;
- пользователи обычно не хотят заводить учетную запись в каждом небольшом приложении, которым они воспользовались.

Можно ли разрешить пользователям этого приложения входить в систему с учетными данными, которые у них уже есть? Могут ли пользователи вашего приложения входить в систему с учетными данными Facebook, GitHub, Twitter или Google? Наверняка вы уже встречали такую возможность. Приложения в интернете позволяют пользователям выбирать способ регистрации и использовать учетные записи в различных социальных сетях. Таким образом, приложение позволяет своим пользователям проходить аутентификацию с имеющимся набором учетных данных, без необходимости добавлять сложные механизмы аутентификации. Такой подход:

- снижает затраты (например, на реализацию и поддержку аутентификации в вашем приложении);
- избегает проблем с доверием пользователей (например, не нужно регистрироваться и создавать еще один набор учетных данных специально для вашего приложения);
- помогает пользователям минимизировать количество используемых ими учетных данных.

OAuth 2 – это спецификация, которая описывает разделение обязанностей по аутентификации в системе. Несколько клиентских приложений могут использовать общее приложение, выполняющее аутентификацию. Это помогает пользователям проходить аутентификацию быстрее, сохраняя их данные в большей безопасности и минимизируя трудозатраты на разработку приложений.

Мы начнем с раздела 13.1, где я представлю основных участников системы, в которой аутентификация и авторизация построены согласно спецификации OAuth 2. В разделе 13.1 вы узнаете о назначении всех частей системы OAuth 2, таких как пользователь, клиент, сервер авторизации и сервер ресурсов. В разделе 13.2 мы обсудим токены. Токены подобны ключам доступа к приложению. Вы узнае-

те, что можно использовать несколько типов токенов и когда лучше всего использовать каждый тип. В разделе 13.3 мы рассмотрим наиболее важные способы выдачи токенов (которые мы реализуем и протестируем в главе 14). Главу завершает раздел 13.4 с описанием возможных подводных камней, о которых вам следует помнить при внедрении OAuth 2.

Прежде чем начать, я хотел бы упомянуть, что в этой главе я предлагаю простой обзор основных понятий, которые нужно усвоить, прежде чем перейти к главам 14–16. Я не собираюсь делать из вас эксперта по OAuth 2 и OpenID Connect за одну главу. Я не думаю, что это возможно, поскольку эти две технологии настолько сложны, что заслуживают отдельных книг. Если вы хотите расширить свои знания по этой теме, я рекомендую книгу *OAuth 2 in Action* (Justin Richer and Antonio Sanso; Manning, 2017) и *OpenID Connect in Action* (Prabath Siriwardena; Manning, 2023).

13.1 Общий обзор OAuth 2 и OpenID Connect

Представьте, что вам нужно пройти собеседование в крупной корпорации. Вас вызвали в главный офис для личной беседы. Но не каждый может войти в офис компании. Для посетителей предназначены особые протоколы.

Чтобы попасть в здание и принять участие в собеседовании, вы должны сначала подойти к стойке регистрации и подтвердить свою личность, используя удостоверение личности. После идентификации вы получите на стойке регистрации карту доступа, которая позволит вам открывать строго определенные двери. Возможно, вы даже не сможете пользоваться всеми лифтами, а только определенными (рис. 13.1).

Процесс посещения офиса для собеседования очень похож на то, как работают аутентификация и авторизация в OAuth 2. Вы являетесь пользователем, которому необходимо получить доступ к определенной функции приложения (перейти в определенную комнату для беседы). Для этого вы используете свои учетные данные (идентификатор) для аутентификации на стойке регистрации (сервер авторизации). Как только вы подтвердите свою личность, у вас появится карта доступа (токен). Но вы можете использовать этот токен только для доступа к строго ограниченным ресурсам (например, лифтам и определенным дверям). Вы можете использовать карту доступа только в течение короткого периода времени. После собеседования вы должны вернуть карту на стойку регистрации.

В этом разделе мы обсудим взаимосвязанные ответственности в системе OAuth 2, и вы узнаете, чем это похоже на посещение штаб-

квартиры организации для собеседования. Мы также обсудим, что такое OAuth 2 как спецификация и разницу между OpenID Connect (протоколом) и OAuth 2 (спецификацией, на которой он основан). Я считаю, что важно хорошо понять концепции, лежащие в основе этого подхода к аутентификации и авторизации, прежде чем углубляться в его реализацию в главах 14–16.



Рис. 13.1 Спецификация OAuth 2 очень похожа на доступ в офисное здание

Сначала выясним, какие роли присутствуют в системе OAuth 2. На рис. 13.2 представлены основные акторы (actor, действующее лицо) системы OAuth 2. Под актором я подразумеваю любую сущность, которая играет роль в функциональности системы. В системе OAuth 2 вы найдете следующих акторов:

- **пользователь** – субъект, который использует приложение. Пользователи обычно работают с фронтенд-приложением, которое мы называем **клиентом**. Пользователи не всегда существуют в системе OAuth 2, как говорится в разделе 13.3.3, где вы узнаете о типе предоставления учетных данных клиента;
- **клиент** – приложение, которое вызывает бэкэнд и требует аутентификации и авторизации. Клиентом может быть веб-приложение, мобильное приложение или даже настольное

приложение или отдельная бэкэнд-служба. Система обычно не имеет пользователей, если клиент является бэкэнд-службой;

- *сервер ресурсов* – бэкэнд-приложение, которое авторизует и обслуживает вызовы, отправленные одним или несколькими клиентскими приложениями;
- *сервер авторизации* – приложение, которое реализует аутентификацию и безопасное хранение учетных данных.



Рис. 13.2 Действующие лица системы OAuth 2. Пользователи взаимодействуют через клиента, которому требуется авторизация для определенных операций на бэкэнд-сервисе, известном как сервер ресурсов. Для бэкэнд-авторизации первым шагом клиента является аутентификация сервером авторизации

Давайте посмотрим, как на самом деле происходят аутентификация и авторизация. Шаги просты:

- 1 Пользователь запускает клиентское приложение с определенной целью.
- 2 Клиентское приложение выполняет авторизацию для вызова сервера ресурсов и обслуживания запроса пользователя.
- 3 Чтобы получить авторизацию, клиент сначала запрашивает токен (так называемый *токен доступа*) у сервера авторизации. Этот токен – просто некоторая конкретная информация, которая помогает клиенту доказать, что сервер авторизации правильно его идентифицировал.
- 4 Клиент использует токен, который выдает сервер авторизации, чтобы авторизоваться при отправке запросов на свой бэкэнд (сервер ресурсов).

На рис. 13.3 схематически показана часть потока. Мы видим следующие пронумерованные шаги:

- 1 Пользователь пытается использовать клиентское приложение для достижения определенного результата.
- 2 Клиентское приложение знает, что не может вызвать свой бэкенд без предварительно полученного токена, который позволит пройти авторизацию. Клиент запрашивает такой токен доступа у сервера авторизации.
- 3 По запросу клиентского приложения сервер авторизации выдает токен и отправляет его клиентскому приложению.
- 4 Клиент использует токен для отправки запросов на свой бэкенд (сервер ресурсов).
- 5 Сервер ресурсов авторизует запрос клиента. В случае успешной авторизации сервер ресурсов выполняет запрос клиента и возвращает результат.
- 6 Клиент показывает результат пользователю.

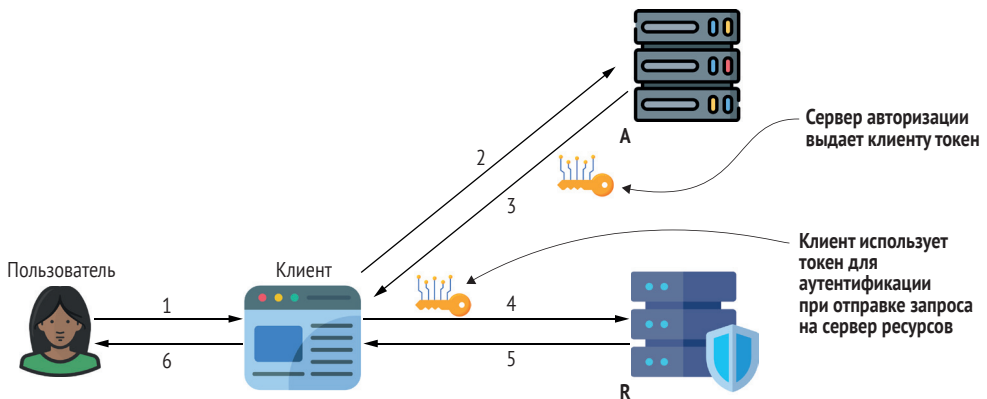


Рис. 13.3 Простейшее объяснение процедуры аутентификации OAuth 2 заключается в том, что клиент получает токен от сервера авторизации. Затем этот токен используется для авторизации запросов, отправляемых на бэкенд-приложение или сервер ресурсов

Но что именно представляет собой токен, выдаваемый сервером авторизации? Токеном может быть любой фрагмент данных (обычно строка символов), который позволяет клиенту доказать, что он (и/или пользователь) был идентифицирован сервером авторизации. Токен также является способом получить больше сведений как о пользователе, так и о клиенте, если это необходимо. Поскольку сервер авторизации теперь управляет всеми данными пользователя и клиента, бэкенду иногда необходимо получить часть этих данных с сервера авторизации и использовать их. Бэкенд получит такие данные с помощью токена. Иногда сам токен содержит необходимые

данные (как вы узнаете в разделе 13.2, такие токены называются прозрачными токенами); в противном случае бэкенду необходимо вызвать сервер авторизации, чтобы получить данные о клиенте и пользователе (т. е. непрозрачные токены). Кроме того, в отличие от физического ключа, токен доступа не имеет большого срока службы. Он истекает через короткий период времени (счет идет на минуты), после чего клиенту необходимо запросить у сервера авторизации новый токен. Таким образом, потерянный токен (например, потерянный ключ) не может быть использован не по назначению.

OAuth 2 описывает несколько потоков, в которых клиент может получить токен. Я называю эти потоки *типами грантов* и в разделе 13.3 рассказываю о наиболее распространенных типах грантов.

13.2 Использование различных реализаций токенов

Токены – это своеобразные карты доступа (рис. 13.4), которые клиент использует для авторизации при отправке запросов на бэкенд (сервер ресурсов). Токены являются неотъемлемой частью процес-

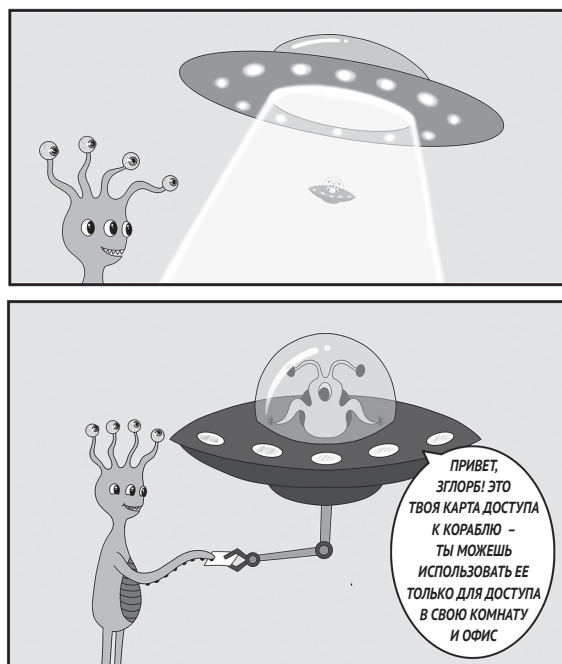


Рис. 13.4 Инопланетянин Зглорб (пользователь) хочет получить доступ на летающую тарелку (сервер ресурсов). Для этого он сначала проходит аутентификацию (сервер авторизации), а затем ему выдают карту доступа (токен). С помощью своей карты доступа Зглорб может получить доступ только к определенным помещениям (ресурсам) летающей тарелки

са аутентификации и авторизации OAuth 2, поскольку они используются для подтверждения подлинности аутентификации клиента и пользователя, но они также являются способом, с помощью которого бэкенд получает более подробную информацию о клиенте и пользователе.

В этом разделе мы обсудим, как классифицируются токены и как они используются в процессе авторизации в зависимости от типа.

Токены классифицируют по способу, которым они предоставляют серверу ресурсов данные для авторизации:

- *непрозрачные* – токены, которые не хранят данные. Для реализации авторизации серверу ресурсов обычно необходимо вызвать сервер авторизации, предоставить непрозрачный токен и получить данные. Это так называемый *вызов интроспекции* (introspection call);
- *прозрачные* – токены, которые хранят данные, что позволяет бэкенду немедленно реализовать авторизацию. Наиболее часто используемой реализацией прозрачного токена является JSON Web Token (JWT).

13.2.1 Использование непрозрачных токенов

Непрозрачные токены не содержат данных, которые бэкенд может использовать для идентификации пользователя или клиента или для применения правил авторизации. Непрозрачные токены являются просто доказательством попытки аутентификации. Когда сервер ресурсов получает непрозрачный токен, ему необходимо вызвать сервер авторизации, чтобы узнать, действителен ли токен, и получить больше информации, чтобы иметь возможность применить ограничения авторизации.

Непрозрачный токен буквально похож на ключ от сундука с сокровищами. Он не предоставляет никакой информации заранее; вы узнаете, сработает ли он, только когда попытаетесь открыть сундук с его помощью. Если ключ сработает, он даст вам доступ к содержимому сундука (в нашем случае к данным пользователя и клиента). Эту аналогию иллюстрирует рис. 13.5.

Сервер ресурсов вызывает конечную точку, предоставленную сервером авторизации, чтобы узнать, действителен ли непрозрачный токен, и получить необходимые данные о клиенте и пользователе, которому был выдан токен. Этот вызов называется интроспекцией токена (token introspection, рис. 13.6). Как только сервер ресурсов получит эти данные, он сможет применить ограничения авторизации.

Вы не знаете, подходящий ли это ключ, и вы не знаете, что находится внутри сундука, пока не попытаете открыть его



Если ключ откроет сундук, вы узнаете, что это подходящий ключ, и вы также узнаете, что находится внутри сундука

Рис. 13.5 Аналогия с непрозрачными токенами. Непрозрачный токен похож на ключ. Вы не узнаете, подходит ли он к сундуку, пока не попытаете. Если ключ подходит, вы получаете доступ к тому, что находится внутри сундука

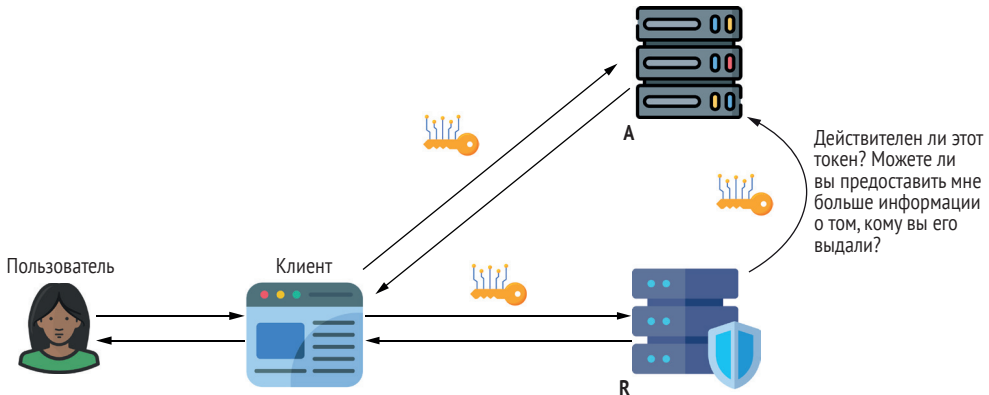


Рис. 13.6 Вызов интроспекции токена. Сервер ресурсов отправляет запрос на сервер авторизации, чтобы узнать, действителен ли непрозрачный токен, и узнать подробности о том, кому он был выдан

13.2.2 Использование прозрачных токенов

В отличие от непрозрачных токенов, которые мы обсуждали в разделе 13.2.1, прозрачные токены содержат информацию о клиенте и пользователе, которому сервер авторизации выдал токены в процессе аутентификации. Прозрачные токены можно сравнить с подписанными документами (рис. 13.7).

Наиболее распространенной реализацией прозрачного токена является JWT. Токен JWT состоит из трех частей (рис. 13.8):

- **заголовок** – обычно содержит данные о токене, такие как криптографический алгоритм, используемый для подписи токена, или идентификатор ключа, который сервер авторизации использовал для его подписи;
- **тело** – обычно содержит данные о субъекте, которому был выдан токен, такие как данные клиента и пользователя;
- **подпись** – криптографически сгенерированное значение, которое можно использовать для доказательства того, что сервер авторизации действительно выдал токен и что никто не изменил его содержимое (заголовок или тело) после его генерации.

В документе указаны данные о пользователе и клиенте



Подлинность документа можно установить, проверив подпись

Рис. 13.7 Прозрачный токен похож на подписанный документ. Он содержит необходимые сведения для сервера ресурсов, чтобы применить ограничения авторизации, и подпись для проверки его подлинности

Данные в заголовке и теле представлены в формате JavaScript Object Notation (JSON), а затем закодированы в Base64, чтобы сделать их меньше и проще для передачи. Три части разделены точками.

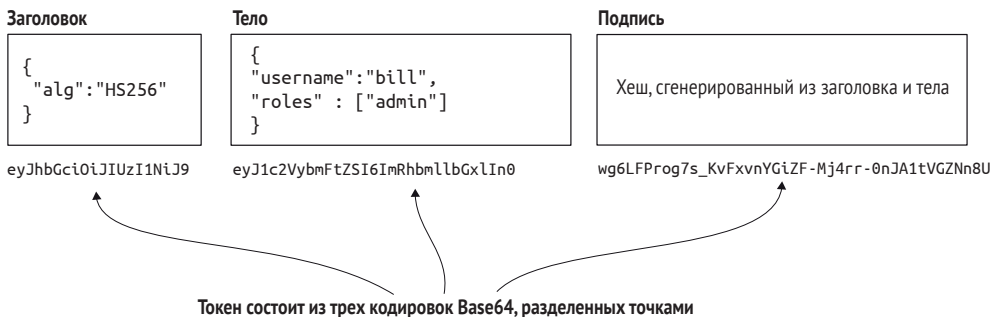


Рис. 13.8 Анатомия токена JWT. Заголовок и тело содержат сведения, необходимые ресурсному серверу для проверки подлинности токена и применения ограничений авторизации

Ниже показан пример токена JWT, где три части закодированы в Base64 и разделены точками:


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.  
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Когда следует использовать непрозрачные токены, а когда – прозрачные? Как я уже говорил вам ранее в этом разделе, сегодня чаще всего используются прозрачные токены, поскольку для их проверки не требуется интроспекция. Однако прозрачные токены содержат данные, и клиент отправляет эти данные по сети на свой бэкенд. Любой, кто получает токен, также может видеть данные, которые несет токен. В большинстве случаев это не проблема. Тем не менее я рекомендую всем избегать отправки слишком большого количества данных в токен.

Но что делать, если у вас есть большой объем данных или данные, которые небезопасно отправлять по сети в токене? В таком случае хорошей альтернативой могут быть непрозрачные токены. Я рекомендую вам сначала попробовать применить прозрачные токены и вернуться к непрозрачным только в том случае, если объем данных, которые должен переносить токен, слишком велик или вам нужно отправить более конфиденциальные данные и вы хотите избежать обмена ими через токен.

13.3 Получение токенов с помощью различных типов грантов

В этом разделе обсуждаются типы грантов¹. *Тип гранта* (grant type) – это разновидность процесса, с помощью которого клиент получает токен. В реальных приложениях вы встретите различные подходы, согласно которым клиент получает токен с сервера авторизации. Мы обсудим три наиболее часто используемых типа грантов. В конце этой главы вы узнаете, как клиент может повторно сгенерировать токен после истечения срока его действия.

ПРИМЕЧАНИЕ Вы все еще можете встретить приложения, реализующие два других типа грантов: *неявный* (implicit) и *грант в форме пароля* (password grant). Эти два типа грантов устарели, поскольку они были признаны недостаточно безопасными. Мы не будем обсуждать их в этой книге; я не рекомендую использовать их в приложениях. Вы всегда можете заменить любой из них одним из типов грантов, обсуждаемых в этом разделе. Если вы хотите узнать больше о гран-

¹ Грант – это данные, которые представляют из себя успешную авторизацию клиента владельцем ресурса, используемые клиентом для получения токена доступа (<https://habr.com/ru/companies/dododev/articles/520046/>).

те в форме пароля, прочитайте главу 12 первого издания этой книги. Мы также кратко рассмотрим неявный тип, и я поясню, почему он устарел, при обсуждении гранта в форме кода авторизации.

В разделе 13.3.1 обсуждается грант в форме кода авторизации, который является наиболее используемым типом гранта, когда системе необходимо разрешить пользователю пройти аутентификацию. В разделе 13.3.2 мы обсуждаем дополнение к коду авторизации – ключ подтверждения для обмена кодами (PKCE). В разделе 13.3.3 мы продолжаем рассматривать ситуацию, когда приложению необходимо получить токен без аутентификации пользователя, и заканчиваем главу вопросом повторной генерации токена в разделе 13.3.4.

13.3.1 *Получение токена с использованием кода авторизации*

Грант в форме кода авторизации является наиболее используемым типом предоставления доступа на сегодняшний день. Он используется, когда нашему приложению необходимо аутентифицировать пользователя (на рис. 13.9 показаны шаги процесса получения доступа):

- 1 Пользователь хочет что-то сделать в своем приложении. Допустим, девушка в левой части рисунка – бухгалтер Мэри, которая хочет видеть все счета, подлежащие оплате компаний.
- 2 Приложение, которое использует Мэри, является клиентом системы. В данном случае Мэри работает за своим компьютером, поэтому ее клиентское приложение – это веб-приложение. Но Мэри могла бы также использовать мобильную версию приложения. В обоих случаях применяется одинаковый тип гранта. Поскольку Мэри не входила в систему, приложение перенаправляет ее на страницу входа, размещенную на сервере авторизации.
- 3 Теперь Мэри видит страницу входа в своем браузере. Страница входа находится не в приложении, которое она запустила, а размещена в другой системе. Мэри понимает, что страница, на которую она была перенаправлена, принадлежит центральному приложению аутентификации, которое она использует для всех приложений, с которыми работает в компании. Мэри знает, что после ввода учетных данных браузер вернет ее в бухгалтерское приложение и она сможет видеть счета и работать с нужными ей данными. Она вводит правильные учетные данные и нажимает кнопку входа.
- 4 Поскольку предоставленные Мэри учетные данные верны, сервер авторизации перенаправляет ее обратно в бухгалтерское приложение. Сервер авторизации также предоставляет исход-

ному приложению (клиенту) уникальный код, называемый *кодом авторизации*. Клиент будет использовать этот код для получения токена доступа.

- 5 Клиент запрашивает токен доступа. Клиенту нужен этот токен для отправки запросов на свой бэкенд (сервер ресурсов).
- 6 Поскольку код авторизации правильный (тот же, что сервер предоставил на шаге 5), сервер авторизации отвечает токеном доступа.
- 7 Клиентское приложение использует токен доступа для отправки запроса на свой бэкенд и авторизации.

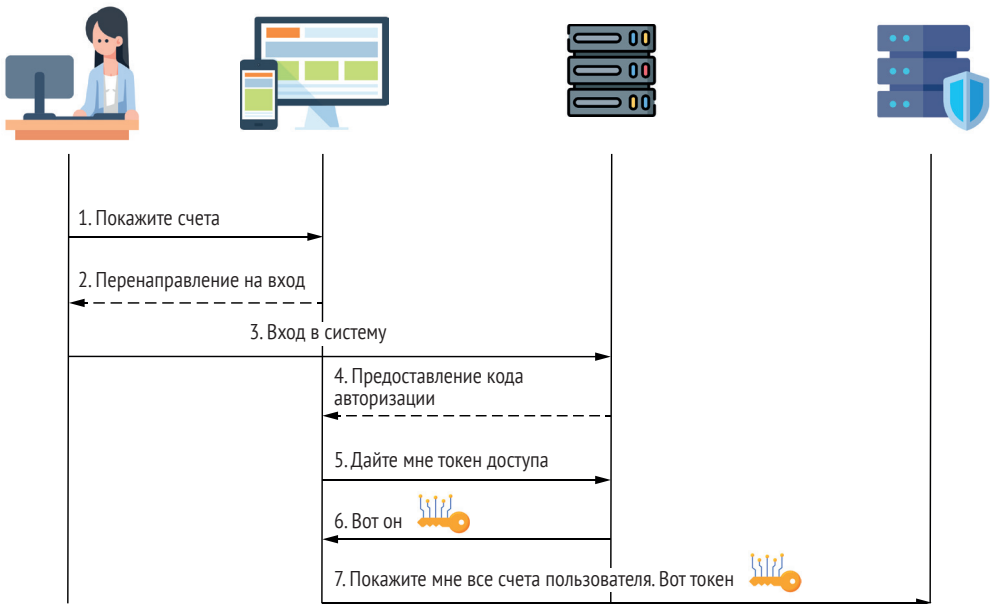


Рис. 13.9 Тип гранта – код авторизации. Пользователь должен войти в систему. После входа в систему сервер авторизации выдает клиенту код авторизации, который клиент затем использует для получения токена доступа. Этот токен доступа позволяет клиенту авторизовать свои запросы на сервере ресурсов.

Несколько замечаний, которые помогут вам лучше понять этот поток:

- обратите внимание на пунктирные стрелки. Важно помнить, что они представляют собой *перенаправления* в браузере, а не запросы или ответы. На шаге 2 клиентское приложение перенаправляет пользователя на страницу входа сервера авторизации (оно перенаправляет его в браузере на веб-страницу другого приложения). На шаге 4 сервер авторизации перенаправляет пользователя обратно в клиентское приложение, предоставляя код авторизации (обычно в качестве параметра запроса);

- Мэри (пользователь) не знает о шагах 4–7. После входа в систему она в конечном итоге увидит счета, отображаемые приложением-клиентом, которое получит их в ответ на шаг 7 после авторизации;
- помните, что не следует путать код авторизации и токен доступа. Токен доступа – это то, что клиенту в конечном итоге нужно для авторизации его бэкендом (шаг 7). Но чтобы получить токен доступа, клиент сначала получает код авторизации (шаги 4 и 5).

Кроме того, многие разработчики, впервые работающие с авторизацией и аутентификацией, путаются в шаге 4. Мне часто задают вопрос: «Почему сервер авторизации не возвращает токен доступа прямо на этом шаге?» Кажется странным, что клиенту нужно сделать еще один шаг, чтобы получить токен доступа, когда он мог бы получить его напрямую на шаге 4.

Но еще один шаг действительно нужен. На самом деле в первой версии OAuth сервер авторизации предоставлял токен доступа вместо кода авторизации на шаге 4. Это и есть упомянутый ранее неявный тип гранта, который устарел и не рекомендуется к применению. Дело в том, что перенаправление никак не защищено от перехвата, и злонамеренный субъект мог очень легко получить токен доступа. Возвращая код авторизации, сервер авторизации заставляет клиента повторно отправить запрос, в котором он должен снова пройти аутентификацию со своими учетными данными. Таким образом, если кто-то перехватит перенаправление и получит код авторизации, ему недостаточно будет получить токен доступа. Чтобы отправить запрос и получить токен, нужно также знать учетные данные клиента.

На рис. 13.10 показаны два шага, в которых код авторизации добавляет дополнительную защиту, чтобы не позволить злоумышленникам получить токен доступа.

13.3.2 Применение защиты PKCE совместно с кодом авторизации

Что, если злоумышленнику удастся получить учетные данные клиента? В этом случае он сможет получить токен доступа и отправлять запросы на сервер ресурсов. Есть ли способ предотвратить такую ситуацию? Да, *ключ подтверждения для обмена кодами* (proof key for code exchange, PKCE, обычно произносится как «пикси») – это улучшение, добавленное к потоку кода авторизации, чтобы сделать его более безопасным. В этом разделе мы обсудим, как PKCE работает в сценарии, когда кто-то может получить токен доступа, украв учетные данные клиента.

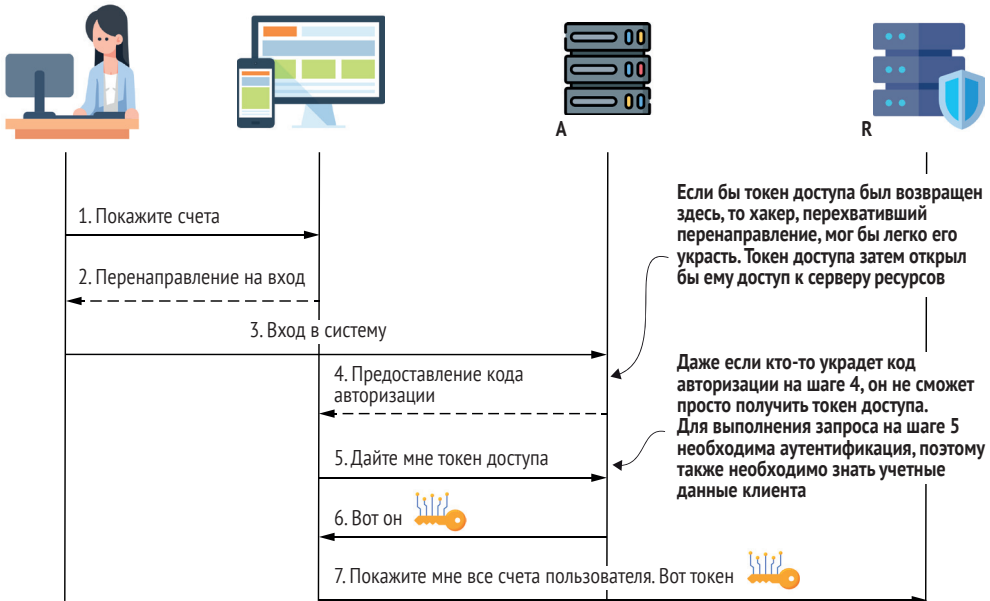


Рис. 13.10 После входа в систему и получения кода авторизации клиент должен сделать дополнительный запрос на получение токена доступа. Во время этого запроса клиент должен подтвердить свою личность, используя свои учетные данные. Этот метод увеличивает сложность для злоумышленников, которые пытаются незаконно получить токен доступа, так как им нужно будет перехватить код авторизации, а также знать учетные данные клиента

Использование PKCE влияет только на два шага гранта по типу кода авторизации, которые мы обсуждали в разделе 13.3.1. На рис. 13.11 я сделал стрелки, представляющие шаги 3 и 5, толще. Это два шага получения доступа с помощью кода авторизации, где применяется PKCE:

- 1 Сначала клиенту необходимо сгенерировать случайное значение. Оно может быть случайной строкой байтов. Это значение называется *верификатором* (verifier).
- 2 Далее клиент применяет хеш-функцию к случайно сгенерированному значению на шаге 1. Хеш-функция – это способ шифрования, при котором из выходного значения функции невозможно получить входное (глава 4). Результат применения хеш-функции к верификатору называется *испытанием*, или *вызовом* (challenge).

```
verifier = random();
challenge = hash(verifier);
```

На шаге 3 клиент отправляет вызов с логином пользователя. Сервер авторизации сохраняет вызов и ожидает, что верификатор с за-

просом, сделанным на шаге 5, получит токен доступа. Если верификатор, который клиент отправляет при запросе токена на шаге 5, совпадает с вызовом, который он отправил на шаге 3, то сервер авторизации знает, что клиентское приложение, которое запрашивает токен, является тем же, которое запрашивает у пользователя аутентификацию.

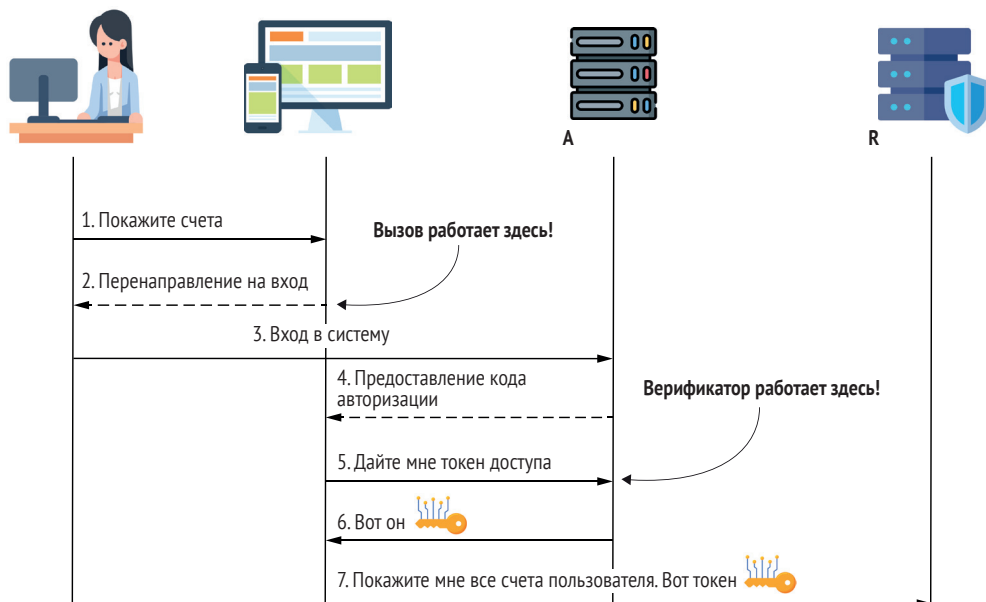


Рис. 13.11 Клиент отправляет вызов на шаге 3, а верификатор на шаге 5, чтобы доказать, что он является тем же клиентом, который изначально запросил вход в систему

Теперь никто не может получить токен доступа, даже если ему каким-то образом удастся получить код авторизации на шаге 4. Это связано с тем, что ему также необходимо знать значение верификатора. Он не может знать верификатор, поскольку клиент еще не отправил его по сети. И он не может получить верификатор, просто перехватив вызов (на шаге 3), поскольку вызов создается с помощью хеш-функции, а значит, невозможно извлечь из него исходное значение.

13.3.3 Получение токена с предоставлением учетных данных клиента

Иногда приложению необходимо получить авторизацию без участия пользователя. Если на сцене нет пользователя, приложению придется воспользоваться предоставлением учетных данных клиента, чтобы получить токен доступа. Такая ситуация обычно возникает,

когда службе необходимо вызвать другую службу по определенному событию, например по таймеру запланированного процесса. При использовании гранта с предоставлением учетных данных клиента приложению необходимо только пройти аутентификацию с использованием своих учетных данных клиента. На рис. 13.12 показана схема процесса с предоставлением учетных данных клиента:

- 1 Приложение запрашивает токен доступа с сервера авторизации. Приложение использует свои учетные данные для аутентификации.
- 2 Если учетные данные действительны, сервер авторизации выдает токен доступа.
- 3 Приложение использует токен доступа для авторизации при отправке запроса на сервер ресурсов.

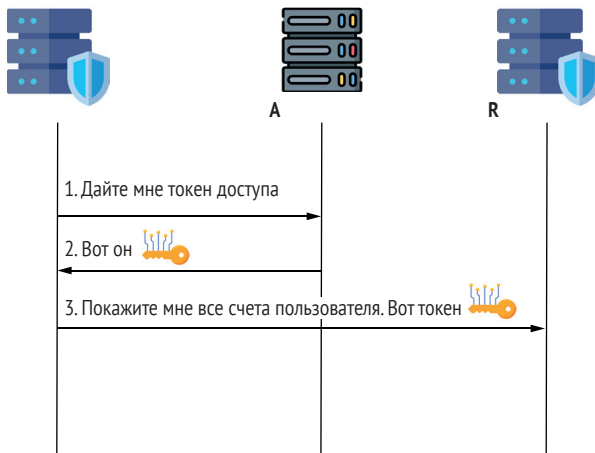


Рис. 13.12 Схема получения токена с предоставлением учетных данных клиента. Приложение получает токен доступа без необходимости аутентификации пользователя

13.3.4 Использование обновления токенов для получения новых токенов доступа

Одна важная вещь, которую вы должны помнить о токенах, заключается в том, что у них должен быть относительно короткий срок жизни. Точное время, в течение которого они остаются активными, обычно зависит от требований к приложению, но чаще оно составляет всего 15 минут, и я никогда не использовал токены, которые живут больше одного часа. В конце концов, все токены рано или поздно должны устареть. Когда срок действия токена истекает, сервер ресурсов больше его не принимает. В таком случае, когда у клиента есть токен и срок действия токена истек, у него есть два варианта:

- 1 Получить новый токен доступа, повторив все шаги получения. Этот путь подразумевает запрос пользователя на повторный вход в систему в случае предоставления кода авторизации.
- 2 Использовать *токен обновления* (refresh token) для получения нового токена доступа.

Токены обновления особенно полезны, когда клиент использует тип гранта, такой как код авторизации, который подразумевает, что пользователю необходимо войти в систему. Представьте, что у вас есть токены со сроком действия 15 минут. Понравится ли вам, если ваше приложение будет просить войти в систему снова и снова каждые 15 минут? Я был бы очень недоволен!

Вместо того чтобы просить пользователя входить в систему каждый раз, когда истекает срок действия токена доступа, приложение может использовать токен обновления для получения нового токена.

На рис. 13.13 показаны этапы процесса с использованием токена обновления:

- 1 Пользователь пытается получить некоторые данные, следовательно, приложение-клиент должно вызвать свой бэкэнд.
- 2 Поскольку токен доступа (полученный ранее) истек, клиенту необходимо получить новый. Клиент отправляет токен обновления, чтобы доказать, что он уже прошел аутентификацию ранее.
- 3 Сервер авторизации распознает токен обновления и предоставляет клиенту новый токен доступа.
- 4 Клиент может вызвать бэкэнд (сервер ресурсов) и авторизоваться с новым токеном доступа.



Рис. 13.13 Приложение-клиент может использовать токен обновления, чтобы получить новый токен доступа, когда старый истекает. Таким образом, приложение избегает необходимости в повторной аутентификации пользователя

13.4 Что OpenID Connect добавляет к OAuth 2?

Конечно, существует много путаницы вокруг OpenID Connect (иногда называемого OIDC) и OAuth 2 и различий между ними. Я обычно говорю своим студентам, чтобы они меньше заикливались на этой теме: «Если вы понимаете OAuth 2, вы также знаете, как использовать OpenID Connect».

На самом деле OIDC – это протокол, созданный на основе спецификации OAuth 2. По этой причине понимание OAuth 2 поможет вам легко освоить OIDC. Позвольте мне предложить вам аналогию спецификаций и протоколов.

Мы все пользуемся электрическими розетками каждый день. Электрические розетки выглядят по-разному в разных частях света. Иногда это становится настоящей головной болью во время путешествий. Вам могут понадобиться адаптеры, чтобы вы могли заряжать свои гаджеты, особенно если вы путешествуете между разными географическими регионами.

Но в основе своей все розетки работают одинаково. Есть электрические провода, по которым на розетку поступает электрическое напряжение. Вы можете определить принцип, согласно которому работают все электрические розетки в мире. Он состоит всего из двух пунктов:

- электрическая розетка имеет три провода, через два из которых протекает электрический ток: фаза и ноль. Иметь заземление желательно, но не обязательно;
- на электрическую розетку поступает стандартное напряжение, которое может быть около 120 или 230 В.

Ничего страшного, если вы не технический специалист; вам не нужно понимать эти два пункта. По крайней мере для изучения Spring Security. Просто поверьте мне на слово.

Проблема в том, что даже если все розетки в мире соответствуют этим условиям, у нас все равно будут возникать ситуации, когда нам понадобятся адаптеры для путешествий. Причина в том, что у розеток нет общего протокола. Поэтому нужны адаптеры для перехода с одной розетки на другую (например, при переезде из Северной Америки в Европу).

То же самое происходит с приложениями, аутентификацией и авторизацией. Даже если два приложения соответствуют спецификации OAuth 2, все равно может возникнуть ситуация, когда они не полностью совместимы и должны быть адаптированы, поскольку они используют разные протоколы. OpenID Connect – это протокол, который немного ограничивает свободу спецификации OAuth 2, вводя несколько изменений. Основные изменения:

- определенные значения для областей действия (например, *profile* или *openid*);

- использование дополнительного токена с именем ID, применяемого для хранения сведений об идентификации пользователя и клиента, которому был выдан токен;
- обычно *тип гранта* (grant type) также называют *поток* (flow) при его обсуждении в контексте OIDC, в то время как *сервер авторизации* (authorization server) обычно называют *поставщиком удостоверений* (identity provider), или *IdP*.

13.5 Недостатки OAuth 2

В этом разделе обсуждаются возможные уязвимости приложений, использующих аутентификацию и авторизацию OAuth 2. Важно понимать, что может пойти не так при применении OAuth 2, чтобы избежать этих ситуаций при разработке приложений. Конечно, как и все остальное в области программного обеспечения, OAuth 2 не является абсолютно надежным. У него есть свои уязвимости, о которых мы должны знать при создании наших приложений. Я перечислю некоторые из наиболее распространенных:

- *возможность подделки межсайтовых запросов (CSRF) на стороне клиента* – когда пользователь вошел в систему, возможна подделка запросов, если приложение не применяет какой-либо механизм защиты от CSRF. Мы подробно обсудили защиту CSRF, реализованную Spring Security, в главе 9;
- *кража учетных данных клиента* – хранение или передача учетных данных без защиты потенциально создает брешь в защите, которой могут воспользоваться злоумышленники;
- *повторное воспроизведение токенов* – как было сказано в разделе 13.2, токены – это «ключи», которые мы используем в архитектуре аутентификации и авторизации OAuth 2 для доступа к ресурсам. Вы отправляете их по сети, и иногда они могут быть перехвачены. Если их перехватили, они будут украдены и использованы повторно. Представьте, что вы потеряли ключ от входной двери вашего дома. Что может произойти? Кто-то другой воспользуется им, чтобы открыть дверь столько раз, сколько захочет (повторное воспроизведение);
- *перехват токенов* – злоумышленник вмешивается в процесс аутентификации и крадет токены, которые может использовать для доступа к ресурсам. Существует также потенциальная уязвимость использования токенов обновления, поскольку их тоже можно перехватить и использовать для получения новых токенов доступа. Я рекомендую прочитать полезную статью по адресу <http://mng.bz/am5z>.

Помните, OAuth 2 – это фреймворк. Уязвимости являются результатом неправильной реализации функциональности поверх него.

Использование Spring Security само по себе помогает частично купировать большинство этих уязвимостей в наших приложениях. При реализации приложения с помощью Spring Security, как вы видели в этой главе, нам нужно задать свои конфигурации или воспользоваться стандартными конфигурациями Spring Security.

Более подробную информацию об уязвимостях, связанных с фреймворком OAuth 2, и о том, как злонамеренный человек может их использовать, можно получить в третьей части книги *OAuth 2 In Action* (Justin Richer and Antonio Sanso; Manning, 2017), доступной по адресу <http://mng.bz/g7Ql>.

Заключение

- Фреймворк OAuth 2 описывает безопасные способы, с помощью которых бэкенд может аутентифицировать своих клиентов. OpenID Connect – это протокол, который реализует клиент OAuth 2, применяя некоторые ограничения для возможных реализаций.
- Четыре основных актора в системе OAuth 2:
 - *пользователь* – лицо, которое хочет воспользоваться функциональностью приложения;
 - *клиент* – приложение, которое должно быть авторизовано для доступа к ресурсу или варианту использования на данном бэкенде;
 - *сервер ресурсов* – бэкенд, которому необходимо авторизовать клиента для выполнения определенного варианта использования или доступа к ресурсу;
 - *сервер авторизации* – приложение, которое управляет данными пользователя и клиента, позволяя им проходить аутентификацию, и предоставляет токен, применяемый в качестве средства авторизации.
- Токен – это своего рода карта доступа (или ключ), которую клиент получает от сервера авторизации и использует для авторизации или доступа к определенному ресурсу на защищенном бэкенде (сервере ресурсов).
- Мы разделяем токены на две категории:
 - *непрозрачные* – токены, которые не содержат сведений о пользователе и клиенте, для которых они были выпущены. Для таких токенов сервер ресурсов всегда должен вызывать сервер авторизации, чтобы проверить токен и получить сведения, необходимые для авторизации запроса. Этот запрос на проверку токена называется интроспекцией;
 - *прозрачные* – токены, которые содержат сведения о пользователе и клиенте, которому они были выданы. Наиболее распро-

страненной реализацией непрозрачных токенов является JSON Web Token (JWT).

- Существует несколько потоков, в которых клиентское приложение может запрашивать у сервера авторизации токен. Эти потоки, в которых выдается токен, называются *типами грантов* (grant type). Наиболее распространенные типы грантов:
 - предоставление кода авторизации;
 - предоставление учетных данных клиента.
- Иногда к предоставлению кода авторизации добавляют дополнительный уровень безопасности с помощью ключа подтверждения для обмена кодами (PKCE). В этом случае приложение-клиент использует дополнительный ключ, чтобы злоумышленник не мог получить токены доступа, украв учетные данные клиента и код авторизации.
- В определенных случаях приложению может потребоваться получить новые токены доступа без повторной аутентификации пользователя. В таких случаях применяются токены обновления. Токены обновления – это специальные токены, которые можно использовать только для получения новых токенов доступа.

14

Сервер авторизации OAuth 2

Темы этой главы:

- реализация сервера авторизации Spring Security OAuth 2;
- гранты в форме кода авторизации и учетных данных клиента;
- настройка непрозрачных и прозрачных токенов доступа;
- отзыв токенов и интроспекция.

В главе 13 вы познакомились с OAuth 2 и OpenID Connect. Мы обсудили акторов системы, в которой аутентификация и авторизация основаны на спецификации OAuth 2. Одним из этих акторов был сервер авторизации. Его роль заключается в аутентификации пользователя и используемого им приложения (клиента), а также в выдаче токенов, которые служат доказательством аутентификации для доступа к ресурсам, защищенным бэкендом. Иногда клиент делает это от имени пользователя.

Экосистема Spring предлагает полностью настраиваемый способ реализации сервера авторизации OAuth 2/OpenID Connect. Сервер авторизации Spring Security – это де-факто способ реализации сервера авторизации с использованием Spring на сегодняшний день. В этой главе мы рассмотрим основные возможности, предлагаемые платформой, и построим собственный сервер авторизации. Рисунок 14.1 призван напомнить вам об акторах OAuth 2 и роли сервера авторизации, упомянутой в главе 13.

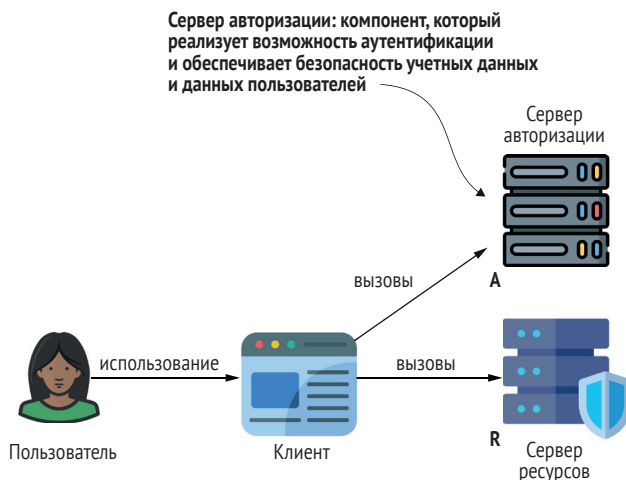


Рис. 14.1 Актеры в системе OAuth 2. Сервер авторизации защищает данные пользователя и клиента и выдает токены, которые клиент может использовать для авторизации при вызове конечных точек сервера ресурсов

Мы начнем с простого примера в разделе 14.1, в котором применяется конфигурация по умолчанию. Эта конфигурация подразумевает, что сервер авторизации будет выдавать прозрачные токены. В разделе 14.2 мы покажем, что наша реализация работает с грантом в форме кода авторизации, а затем, в разделе 14.3, мы также продемонстрируем работу с грантом в форме учетных данных клиента. В разделе 14.4 мы продолжим настройку сервера авторизации для работы с непрозрачными токенами и интроспекцией. Мы закончим эту главу разделом 14.5 про отзыв токенов.

Прежде чем мы начнем, я хотел бы сообщить вам, что текущий способ реализации сервера авторизации с помощью Spring Security полностью отличается от того, что было в прошлые годы. В этой главе мы обсудим новый подход, но у вас может возникнуть потребность узнать, как реализовать сервер авторизации с использованием старого способа (например, если вам нужно работать с существующим приложением, которое не было обновлено). В этом случае я рекомендую прочитать главу 13 первого издания книги.

14.1 Базовая аутентификация с использованием веб-токенов JSON

В этом разделе мы построим базовый сервер авторизации OAuth 2 с использованием фреймворка сервера авторизации Spring Security. Мы рассмотрим все основные компоненты, которые вам необходимо подключить к конфигурации, чтобы она работала, и обсудим

их по отдельности. Затем мы протестируем приложение, используя две самые важные разновидности гранта OAuth 2: код авторизации и учетные данные клиента. Вы найдете этот пример в проекте `ssia-ch14-ex1`.

Основные компоненты, которые вам необходимо настроить для правильной работы вашего сервера авторизации:

- 1 *фильтр конфигурации для конечных точек протокола* – помогает вам определять конфигурации, специфичные для функционала сервера авторизации, включая различные настройки (которые мы обсудим в разделе 14.3);
- 2 *фильтр конфигурации аутентификации* – как и в любом другом веб-приложении, защищенном с помощью Spring Security, вы будете использовать этот фильтр для определения конфигураций аутентификации и авторизации, а также конфигураций для любых других механизмов безопасности, таких как обмен ресурсами между источниками (CORS) и подделка межсайтовых запросов (CSRF) (см. главы 2–10);
- 3 *компоненты управления данными пользователя* – как и любой процесс аутентификации, реализованный с помощью Spring Security, они устанавливаются с помощью bean-компонента `UserDetailsService` и `PasswordEncoder`. Об их работе говорилось в главах 3 и 4;
- 4 *управление данными клиента* – сервер авторизации использует компонент `RegisteredClientRepository` для управления учетными данными клиента и другими данными;
- 5 *управление парами ключей* (используется для подписи и проверки токенов) – при использовании прозрачных токенов сервер авторизации использует закрытый ключ для подписи токенов. Сервер авторизации также предоставляет доступ к открытому ключу, который сервер ресурсов может использовать для проверки токенов. Сервер авторизации управляет парами закрытый–открытый ключ через компонент «источник ключа»;
- 6 *общие настройки приложения* – компонент `AuthorizationServerSettings` помогает вам настраивать общие параметры, такие как конечные точки, которые предоставляет приложение.

На рис. 14.2 показаны компоненты, которые нам необходимо подключить и настроить для работы минимального приложения сервера авторизации.

Для начала мы должны добавить необходимые зависимости в наш проект. В следующем фрагменте кода показаны зависимости, которые нужно добавить в ваш файл `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
```

```

</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-authorization-server</artifactId>
</dependency>

```

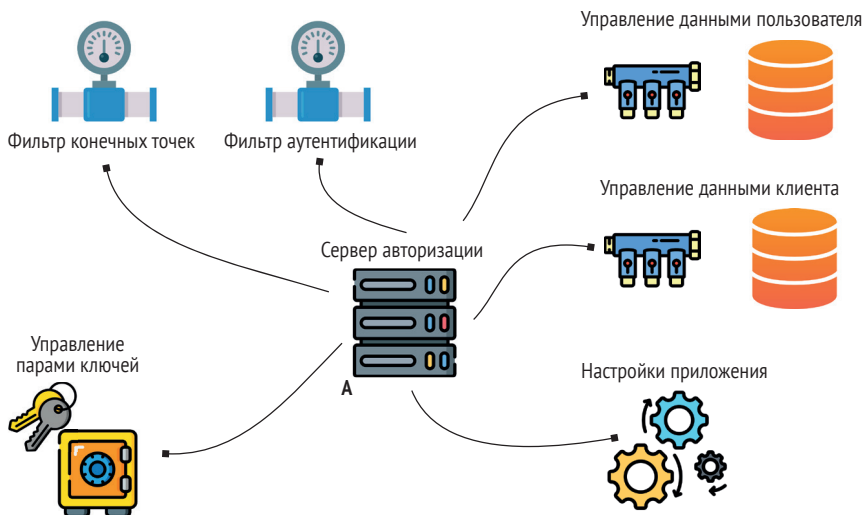


Рис. 14.2 Компоненты, которые нам нужно настроить и подключить для работы сервера авторизации, реализованного с помощью Spring Security

Мы размещаем конфигурации в стандартных классах конфигурации Spring, как в следующем фрагменте кода.

```

@Configuration
public class SecurityConfig {

}

```

Помните, что, как и для любого другого приложения Spring, компоненты могут быть определены в нескольких классах конфигурации или с использованием аннотаций стереотипов (в зависимости от ситуации). Если вам нужно освежить знания по управлению контекстом Spring, я рекомендую первую часть еще одной моей книги «Spring быстро» (Питер, 2022).

Давайте рассмотрим листинг 14.1, в котором представлен фильтр конфигурации для конечных точек протокола. Метод `applyDefaultSecurity()` – это вспомогательный метод, который мы используем для определения минимального набора конфигураций. Вы можете переопределить их позже. После вызова этого метода в листинге показано, как включить протокол OpenID Connect с помощью метода `oidc()` объекта-конфигуратора `OAuth2AuthorizationServerConfigurer`.

Кроме того, фильтр в листинге 14.1 указывает страницу аутентификации, на которую приложение должно перенаправить пользователя при запросе на вход в систему. Нам нужна эта конфигурация, поскольку мы намерены использовать грант в форме кода авторизации, следовательно, пользователи должны пройти аутентификацию. Путь по умолчанию в веб-приложении Spring – /login, поэтому если мы не настроим пользовательский путь отдельно, мы будем использовать в конфигурации сервера авторизации путь по умолчанию.

Листинг 14.1 Реализация фильтра для настройки конечных точек протокола

```
@Bean
@Order(1)
public SecurityFilterChain asFilterChain(HttpSecurity http)
    throws Exception {

    OAuth2AuthorizationServerConfiguration           ❶
        .applyDefaultSecurity(http);                ❶

    http.getConfigurer(
        OAuth2AuthorizationServerConfigurer.class)   ❷
        .oidc(Customizer.withDefaults());            ❷

    http.exceptionHandling((e) ->
        e.authenticationEntryPoint(
            new LoginUrlAuthenticationEntryPoint("/login")) ❸
    );

    return http.build();
}
```

- ❶ Вызов прикладного метода для применения конфигураций по умолчанию для конечных точек сервера авторизации.
- ❷ Включение протокола OpenID Connect.
- ❸ Указание страницы аутентификации для пользователей.

Код в листинге 14.2 настраивает аутентификацию и авторизацию. Эти конфигурации работают аналогично любому веб-приложению (как мы обсуждали в главах 2–10). В листинге 14.2 я настроил минимальные конфигурации:

- 1 Включение аутентификации через форму входа, чтобы приложение предоставляло пользователю простую страницу входа для аутентификации.
- 2 Указание того, что приложение разрешает доступ к любым конечным точкам только аутентифицированным пользователям.

Другие конфигурации, которые вы можете написать здесь, помимо аутентификации и авторизации, могут быть для определенных

механизмов защиты, таких как CSRF (обсуждается в главе 9) или CORS (обсуждается в главе 10).

Также обратите внимание на аннотацию `@Order`, которую я использовал в листингах 14.1 и 14.2. Эта аннотация необходима, поскольку у нас есть несколько экземпляров `SecurityFilterChain`, настроенных в контексте приложения, и нам нужно указать порядок, в котором они имеют приоритет в конфигурации.

Листинг 14.2 Реализация фильтра для конфигурации авторизации

```
@Bean
@Order(2)
public SecurityFilterChain defaultSecurityFilterChain(HttpSecurity
http)
    throws Exception {

    http.formLogin(Customizer.withDefaults());

    http.authorizeHttpRequests(
        c -> c.anyRequest().authenticated()
    );

    return http.build();
}
```

- ❶ Устанавливаем фильтр для интерпретации после конечных точек протокола.
- ❷ Включаем метод аутентификации формы входа.
- ❸ Настраиваем все конечные точки как требующие аутентификации.

Если вы ожидаете, что клиенты будут использовать сервер авторизации, который вы создаете для типов гранта, подразумевающих аутентификацию пользователя (например, с помощью кода авторизации), то ваш сервер должен управлять данными пользователя! К счастью, для управления данными пользователя вы можете использовать тот же подход, который вы изучили в главах 3 и 4. Все, что вам нужно, – это `UserDetailsService` и `PasswordEncoder`.

Листинг 14.3 представляет определение для этих двух компонентов. В этом примере мы используем для `UserDetailsService` реализацию в памяти, но я напомним, что вы узнали, как написать собственную реализацию для него, в главе 3. В большинстве случаев, как и для других веб-приложений, вы будете хранить такие данные в базе данных. Поэтому вы должны написать свою реализацию для контракта `UserDetailsService`.

Кроме того, в главе 4 было сказано, что `NoOpPasswordEncoder` следует использовать только с учебными примерами. `NoOpPasswordEncoder` никак не преобразует пароли, оставляя их в открытом виде и в распоряжении любого, кто может получить к ним доступ, что очень плохо. Всегда следует использовать кодировщик паролей с надежной хеш-функцией, например `BCrypt`.

Листинг 14.3 Определение управления данными пользователя

```

@Bean
public UserDetailsService userDetailsService() {
    UserDetails userDetails = User.withUsername("bill")
        .password("password")
        .roles("USER")
        .build();

    return new InMemoryUserDetailsManager(userDetails);
}

@Bean
public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
}

```

Для управления клиентскими данными серверу авторизации необходим компонент `RegisteredClientRepository`. Интерфейс `RegisteredClientRepository` работает аналогично интерфейсу `UserDetailsService`, но он предназначен для извлечения клиентских данных. Аналогично фреймворк предоставляет объект `RegisteredClient`, цель которого – описать клиентское приложение, известное серверу авторизации.

Если провести аналогию с тем, что вы узнали в главах 3 и 4, `RegisteredClient` для клиентов – это то же самое, что `UserDetails` для пользователей. Соответственно, `RegisteredClientRepository` работает с клиентскими данными так же, как `UserDetailsService` работает с пользовательскими данными (рис. 14.3).

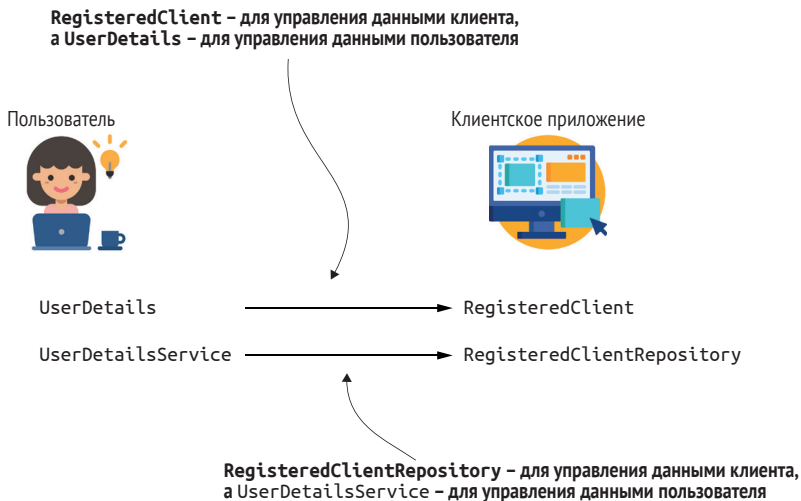


Рис. 14.3 Для управления данными клиента мы используем реализацию `RegisteredClientRepository`. Этот компонент использует объекты `RegisteredClient` для представления данных клиента

В этом примере мы будем использовать хранение пользователей в памяти, чтобы вы могли сосредоточиться на сервере авторизации. В реальном приложении вам, скорее всего, потребуется предоставить интерфейс для извлечения данных из базы данных. Чтобы этот механизм работал, вам необходимо реализовать интерфейс `RegisteredClientRepository` аналогично тому, как вы делали для интерфейса `UserDetailsService` в главе 3.

В следующем листинге показано определение `bean`-компонента `RegisteredClientRepository` с размещением данных в памяти. Метод создает один экземпляр `RegisteredClient` с требуемыми данными и сохраняет его в памяти для использования во время аутентификации сервером авторизации.

Листинг 14.4 Реализация управления данными клиента

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
    RegisteredClient registeredClient =
        RegisteredClient
            .withId(UUID.randomUUID().toString())
            .clientId("client")
            .clientSecret("secret")
            .clientAuthenticationMethod(
                ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
            .authorizationGrantType(
                AuthorizationGrantType.AUTHORIZATION_CODE)
            .redirectUri("https://www.manning.com/authorized")
            .scope(OidcScopes.OPENID)
            .build();
    return new InMemoryRegisteredClientRepository
        ➔(registeredClient);
}
```

- ❶ Создаем экземпляр `RegisteredClient`.
- ❷ Добавляем его для управления реализацией `RegisteredClientRepository` в памяти.

При создании экземпляра `RegisteredClient` мы указали следующие данные:

- *уникальный внутренний идентификатор* – значение, которое однозначно идентифицирует клиента и применяется только во внутренних процессах приложения;
- *идентификатор клиента* – внешний идентификатор клиента, аналогичный имени пользователя;
- *секрет клиента* – аналог пароля пользователя;
- *метод аутентификации клиента* – сообщает, какую аутентификацию ожидает сервер авторизации от клиента при отправке запросов на токены доступа;

- *тип гранта авторизации* – тип гранта, разрешенный сервером авторизации для этого клиента. Клиент может использовать несколько типов гранта;
- *URI перенаправления* – один из адресов URI, на который сервер авторизации может перенаправить клиента для предоставления кода авторизации в случае гранта в форме кода авторизации;
- *область действия* – определяет цель запроса токена доступа. Область действия можно использовать позже в правилах авторизации.

В этом примере клиент использует только грант в форме кода авторизации. Однако у вас могут быть клиенты, использующие несколько разных грантов. Если вы хотите, чтобы клиент мог использовать несколько типов гранта, вам необходимо указать их, как показано в следующем фрагменте кода. Клиент, определенный здесь, может использовать любой тип гранта (код авторизации, учетные данные клиента или токен обновления):

```
RegisteredClient registeredClient =
    RegisteredClient
        .withId(UUID.randomUUID().toString())
        .clientId("client")
        .clientSecret("secret")
        .clientAuthenticationMethod(
            ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .authorizationGrantType(
            AuthorizationGrantType.AUTHORIZATION_CODE)
        .authorizationGrantType(
            AuthorizationGrantType.CLIENT_CREDENTIALS)
        .authorizationGrantType(
            AuthorizationGrantType.REFRESH_TOKEN)
        .redirectUri("https://www.manning.com/authorized")
        .scope(OidcScopes.OPENID)
        .build();
```

Аналогично, многократно вызывая метод `redirectUri()`, вы можете указать несколько разрешенных URI для перенаправления. Клиент также может иметь доступ к нескольким областям действия. На практике приложение будет хранить все эти данные в базе данных, откуда ваша пользовательская реализация `RegisteredClientRepository` будет их извлекать.

Если сервер авторизации использует непрозрачные токены, то, помимо сведений о пользователе и клиенте, вы должны настроить управление парами ключей (обсуждается в главе 13). В случае непрозрачных токенов сервер авторизации использует закрытые ключи для подписи токенов и предоставляет клиентам открытые ключи, которые они могут использовать для проверки подлинности токенов.

JWKSouce – это объект, обеспечивающий управление ключами для сервера авторизации Spring Security. В листинге 14.5 показано, как настроить JWKSouce в контексте приложения. Для этого примера я создаю пару ключей программно и добавляю ее в набор ключей, которые может использовать сервер авторизации. Реальное приложение будет считывать ключи из места, где они безопасно хранятся (например, из хранилища, настроенного в среде).

Построение среды, которая идеально напоминает реальную систему, было бы слишком сложным, и я предпочитаю, чтобы вы сосредоточились на реализации сервера авторизации. Однако помните, что в реальном приложении нет смысла генерировать новые ключи каждый раз при перезапуске приложения (как в нашем случае). Если так сделать в реальном приложении, то каждый раз при новом развертывании уже выпущенные токены больше не будут работать (потому что их больше нельзя будет проверить с помощью существующих ключей).

Но в нашем примере программная генерация ключей работает и поможет нам продемонстрировать, как работает сервер авторизации. В реальном приложении вы должны хранить ключи в защищенном месте и считывать их оттуда.

Листинг 14.5 Реализация управления набором пар ключей

```
@Bean
public JWKSouce<SecurityContext> jwkSource()
    throws NoSuchAlgorithmException {

    KeyPairGenerator keyPairGenerator =
        KeyPairGenerator.getInstance("RSA");

    keyPairGenerator.initialize(2048);
    KeyPair keyPair = keyPairGenerator.generateKeyPair();

    RSAPublicKey publicKey =
        (RSAPublicKey) keyPair.getPublic();
    RSAPrivateKey privateKey =
        (RSAPrivateKey) keyPair.getPrivate();

    RSAKey rsaKey = new RSAKey.Builder(publicKey)
        .privateKey(privateKey)
        .keyID(UUID.randomUUID().toString())
        .build();

    JWKSet jwkSet = new JWKSet(rsaKey);
    return new ImmutableJWKSet<>(jwkSet);
}
```

- ❶ Программное создание пары открытого и закрытого ключей с использованием криптографического алгоритма RSA.

- ❷ Добавление пары ключей в набор, который сервер авторизации использует для подписи выданных токенов.
- ❸ Упаковка набора ключей в реализацию JWKSSource и возврат его для добавления в контекст Spring.

Наконец, последний компонент, который нам нужно добавить в нашу минимальную конфигурацию, – это объект `AuthorizationServerSettings` (листинг 14.6). Он позволяет настраивать все пути конечных точек, которые предоставляет сервер авторизации. Если вы создадите объект, как показано в следующем листинге, пути конечных точек получают некоторые значения по умолчанию, которые мы проанализируем позже в этом разделе.

Листинг 14.6 Установка общих настроек сервера авторизации

```
@Bean
public AuthorizationServerSettings authorizationServerSettings() {
    return AuthorizationServerSettings.builder().build();
}
```

Теперь мы можем запустить приложение и проверить, работает ли оно. В разделе 14.2 мы выполним последовательность авторизации. Затем в разделе 14.3 мы проверим, что обработка учетных данных клиента происходит, как запланировано, – с использованием кода авторизации.

14.2 Подход с кодом авторизации в действии

В этом разделе мы тестируем сервер авторизации, созданный в разделе 14.1. Мы ожидаем, что, используя зарегистрированные данные клиента, пройдем последовательность авторизации и получим токен доступа. Мы выполним следующие шаги:

- 1 поиск конечных точек, которые предоставляет сервер авторизации;
- 2 вызов конечной точки авторизации для получения кода авторизации;
- 3 использование кода авторизации для получения токена доступа.

Первый шаг – найти пути конечных точек, которые предоставляет сервер авторизации. Поскольку мы не настраивали пользовательские пути, мы должны использовать значения по умолчанию. Но какие это значения? Чтобы получить эти сведения, вызовем конечную точку конфигурации OpenID, как показано ниже. Этот запрос использует метод HTTP GET, и аутентификация не требуется:

`http://localhost:8080/.well-known/openid-configuration`

При вызове конечной точки конфигурации OpenID вы должны получить ответ, который выглядит так, как показано в листинге 14.7.

Листинг 14.7 Ответ на запрос конфигурации OpenID

```
{
  "issuer": "http://localhost:8080",
  "authorization_endpoint":
    "http://localhost:8080/oauth2/authorize",
  "token_endpoint": "http://localhost:8080/oauth2/token",
  "token_endpoint_auth_methods_supported": [
    "client_secret_basic",
    "client_secret_post",
    "client_secret_jwt",
    "private_key_jwt"
  ],
  "jwks_uri": "http://localhost:8080/oauth2/jwks",
  "userinfo_endpoint": "http://localhost:8080/userinfo",
  "response_types_supported": [
    "code"
  ],
  "grant_types_supported": [
    "authorization_code",
    "client_credentials",
    "refresh_token"
  ],
  "revocation_endpoint": "http://localhost:8080/oauth2/revoke",
  "revocation_endpoint_auth_methods_supported": [
    "client_secret_basic",
    "client_secret_post",
    "client_secret_jwt",
    "private_key_jwt"
  ],
  "introspection_endpoint":
    "http://localhost:8080/oauth2/introspect",
  "introspection_endpoint_auth_methods_supported": [
    "client_secret_basic",
    "client_secret_post",
    "client_secret_jwt",
    "private_key_jwt"
  ],
  "subject_types_supported": [
    "public"
  ],
  "id_token_signing_alg_values_supported": [
    "RS256"
  ],
  "scopes_supported": [
```



```

    "openid"
  ]
}

```

- ① Конечная точка авторизации, на которую клиент перенаправит пользователя для аутентификации.
- ② Конечная точка токена, которую клиент вызовет для запроса токена доступа.
- ③ Конечная точка набора ключей, через которую сервер ресурсов получит открытые ключи, необходимые для проверки токенов.
- ④ Конечная точка интроспекции, которую сервер ресурсов может вызвать для проверки непрозрачных токенов.

Посмотрите на рис. 14.4, чтобы вспомнить последовательность авторизации с кодом, о которой говорилось в главе 13. Мы используем ее сейчас, чтобы продемонстрировать, что созданный нами сервер авторизации работает нормально.

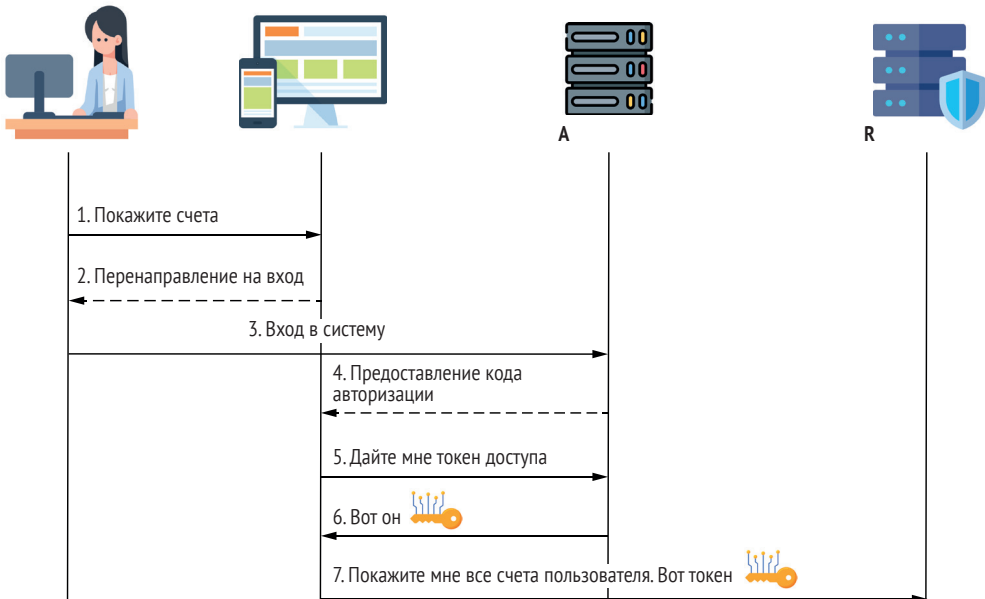


Рис. 14.4 Грант в форме кода авторизации. После успешной аутентификации клиент получает код авторизации. Затем этот код используется клиентом для получения токена доступа, который открывает путь к ресурсам, защищенным сервером ресурсов

Поскольку у нас нет готового клиента для нашего примера, нам нужно имитировать его действия. Теперь, когда вы знаете адрес конечной точки авторизации, вы можете поместить его в адресную строку браузера, чтобы смоделировать перенаправление пользователя клиентом. Следующий фрагмент показывает запрос авторизации:

```
http://localhost:8080/oauth2/authorize?
```

```
➡response_type=code&
```

```

➔client_id=client&
➔scope=openid&
➔redirect_uri=https://www.manning.com/authorized&
➔code_challenge=QYPAZ5NU8yvltlQ9erXrUYR-T5AGCjCF47vN-KsaI2A8&
➔code_challenge_method=S256

```

К запросу авторизации я добавил несколько параметров:

- `response_type=code` – этот параметр запроса указывает серверу авторизации, что клиент хочет использовать грант в форме кода авторизации. Помните, что клиент может настроить несколько типов гранта. Он должен сообщить серверу авторизации, какой тип гранта намерен использовать;
- `client_id=client` – идентификатор клиента похож на имя пользователя. Он уникально идентифицирует клиента в системе;
- `scope=openid` – указывает, какую область доступа клиент хочет получить при этой попытке аутентификации;
- `redirect_uri=https://www.manning.com/authorized` – указывает URI, на который сервер авторизации будет перенаправлять пользователя после успешной аутентификации. Этот URI должен быть одним из ранее настроенных для текущего клиента;
- `code_challenge=QYPAZ5NU8yvltlQ...` – при использовании механизма кода авторизации, улучшенного с помощью PKCE (обсуждается в главе 13), необходимо предоставить код вызова с запросом авторизации. При запросе токена клиент должен отправить проверочную пару верификации, чтобы доказать, что он является тем же приложением, которое изначально отправило этот запрос. Процесс PKCE включен по умолчанию;
- `code_challenge_method=S256` – этот параметр запроса указывает, какой метод хеширования использовался для создания вызова от верификатора. В этом случае S256 означает, что в качестве хеш-функции применялась SHA-256.

Я рекомендую использовать грант в форме кода авторизации с PKCE, но если вам действительно нужно отключить PKCE, вы можете сделать это, как показано в следующем фрагменте кода. Обратите внимание на метод `clientSettings()`, который принимает экземпляр `ClientSettings`, где вы можете указать, что вы отключаете ключ подтверждения для обмена кодами:

```

RegisteredClient registeredClient = RegisteredClient
    .withId(UUID.randomUUID().toString())
    .clientId("client")
    // ...
    .clientSettings(ClientSettings.builder()
        .requireProofKey(false)
        .build())
    .build();

```

В этом примере мы демонстрируем авторизацию с PKCE, что является стандартным и рекомендуемым способом. Отправляя запрос авторизации через адресную строку браузера, мы имитируем шаг 2 из рис. 14.4. Сервер авторизации перенаправит нас на свою страницу входа, и мы сможем пройти аутентификацию, используя имя пользователя и пароль. Это шаг 3 из рис. 14.4. На рис. 14.5 показана страница входа, которую сервер авторизации отображает пользователю.

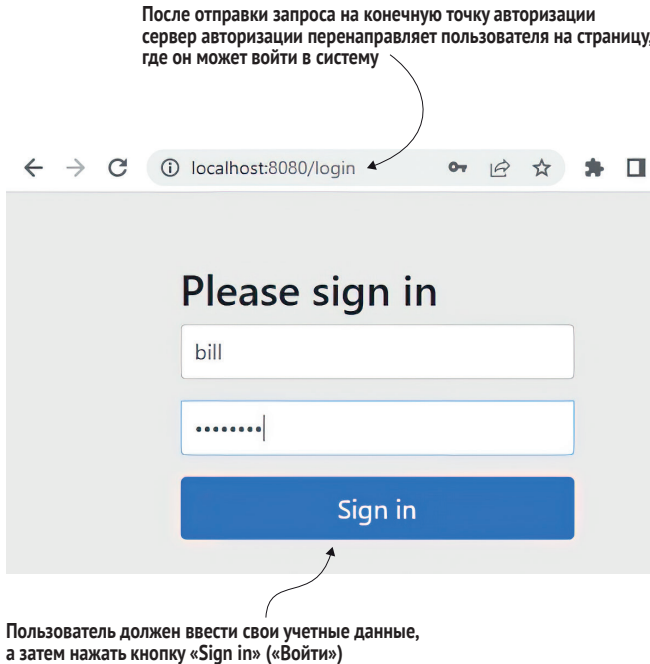


Рис. 14.5 Страница входа, которую сервер авторизации отображает пользователю в ответ на запрос авторизации

В нашем примере у нас есть только один пользователь (см. листинг 14.3). Его учетные данные – имя пользователя `bill` и пароль `password`. Как только пользователь вводит правильные учетные данные и нажимает кнопку **Sign in** (Войти), сервер авторизации перенаправляет пользователя на запрошенный URI перенаправления и предоставляет код авторизации (как показано на рис. 14.6; шаг 4 на рис. 14.4).

После получения кода авторизации клиент может запросить токен доступа через конечную точку токена. Ниже показана команда cURL для запроса токена. Запрос использует метод HTTP POST. Поскольку мы указали, что при регистрации клиента запрашивается аутентификация HTTP Basic, для запроса токена требуется аутентификация с помощью HTTP Basic с идентификатором клиента и секретом:

```
curl -X POST 'http://localhost:8080/oauth2/token?
↳client_id=client&
↳redirect_uri=https://www.manning.com/authorized&
↳grant_type=authorization_code&
↳code=ao2oz47zdM0D5gbAqtZVB...
↳code_verifier=qPsH306-... \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

Сервер авторизации перенаправляет на запрошенный URI перенаправления и предоставляет код авторизации

← → ↻ manning.com/authorized?code=tjt_grwvNynWhyjWJrtBVYnXmaKuX8nMYQc7a1hlpPBq6fddjdYKCEmHBxsWc0HkGS

404

We couldn't find what
you were looking for.

Помните, мы просто имитируем клиента, используя страницу на сайте Manning, которая не существует. Это нормально, что мы получаем ответ 404 (страница не найдена)

Рис. 14.6 После успешной аутентификации сервер авторизации направляет пользователя на указанный URI перенаправления и выдает код авторизации. Затем клиент использует этот код для получения токена доступа

Параметры запроса, которые мы использовали:

- `client_id=client` – необходим для идентификации клиента;
- `redirect_uri=https://www.manning.com/authorized` – URI перенаправления, через который сервер авторизации предоставил код авторизации после успешной аутентификации пользователя;
- `grant_type=authorization_code` – показывает, какую последовательность использует клиент для запроса токена доступа;
- `code=ao2oz47zdM0D5...` – значение кода авторизации, предоставленного сервером авторизации клиенту;
- `code_verifier=qPsH306ZDD...` – верификатор, на основе которого был создан запрос, отправленный клиентом при авторизации.

ПРИМЕЧАНИЕ Будьте внимательны к каждой детали. Если какие-либо значения не соответствуют тому, что знает приложение, или тому, что было отправлено в запросе авторизации, запрос токена не будет выполнен.

Ниже показано тело ответа на запрос токена. Теперь у клиента есть токен доступа, который он может использовать для отправки запросов на сервер ресурсов:

```
{
  "access_token": "eyJraWQiOiI4ODlhNGFmO...",
  "scope": "openid",
  "id_token": "eyJraWQiOiI4ODlhNGFmOS1...",
  "token_type": "Bearer",
  "expires_in": 299
}
```

Поскольку мы включили протокол OpenID Connect, то есть мы не полагаемся только на OAuth 2, токен идентификатора также присутствует в ответе. Если бы клиент был зарегистрирован с использованием процедуры токена обновления, этот токен обновления также был бы сгенерирован и отправлен в ответе.

Генерация верификатора кода и вызова

В примере, над которым мы работали в этом разделе, я использовал код авторизации с PKCE. В запросах авторизации и токена я использовал запрос и значение верификатора, которые я сгенерировал ранее. Я не уделял слишком много внимания этим значениям, потому что за них отвечает клиент, а не сервер авторизации или ресурсов. На практике ваш код JavaScript или мобильное приложение должны будут генерировать оба значения при использовании их в процедуре OAuth 2.

Но если вам интересно, здесь я объясню, как сгенерировал эти два значения. Вы найдете этот пример в проекте `ssia-ch14-ex2`.

Верификатор кода – это случайный 32-байтовый фрагмент данных. Чтобы упростить передачу через HTTP-запрос, эти данные должны быть закодированы в Base64 с использованием URL-кодировщика и без заполнения. Следующий фрагмент кода показывает, как это делается в Java:

```
SecureRandom secureRandom = new SecureRandom();
byte [] code = new byte[32];
secureRandom.nextBytes(code);
String codeVerifier = Base64.getUrlEncoder()
    .withoutPadding()
    .encodeToString(code);
```

Получив верификатор кода, мы используем хеш-функцию для генерации вызова. Следующий фрагмент кода показывает, как создать вызов с помощью хеш-функции SHA-256. Как и в случае с верификатором, вам нужно использовать Base64 для кодирования массива байтов в строковое значение, что упрощает передачу через HTTP-запрос:

```
MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
```

```
byte [] digested = messageDigest.digest(verifier.getBytes());
String codeChallenge = Base64.getUrlEncoder()
    .withoutPadding()
    .encodeToString(digested);
```

Теперь у вас есть верификатор и вызов. Вы можете использовать их в запросах авторизации и токена, как обсуждается в этом разделе.

14.3 Грант в форме клиентских данных

В этом разделе мы попробуем применить грант в форме клиентских учетных данных с помощью сервера авторизации, который мы реализовали в разделе 14.1. Помните, что предоставление клиентских данных – это процедура, которая позволяет клиенту получить токен доступа без аутентификации или пожелания пользователя. Предпочтительно, чтобы у вас не было клиента, способного одновременно использовать как зависящий от пользователя тип гранта (например, код авторизации), так и не зависящий от клиента (например, учетные данные клиента).

Как вы узнаете в главе 15, где мы обсуждаем сервер ресурсов, механизм авторизации может не различать токен доступа, полученный через предоставление кода авторизации, и полученный через предоставление клиентских учетных данных. Поэтому лучше всего использовать для таких случаев разные регистрации и желательно различать использование токенов через разные области действия.

В листинге 14.8 показан зарегистрированный клиент, способный использовать грант в форме клиентских учетных данных. Обратите внимание, что я также настроил другую область действия. В данном случае "CUSTOM" – это просто имя, которое я выбрал; вы можете выбрать любое имя для областей действия. Выбранное вами имя, как правило, должно сделать назначение области действия более понятным. Например, если этому приложению необходимо предоставлять клиентские учетные данные, чтобы получить токен для проверки состояния жизнеспособности сервера ресурсов, то, возможно, лучше назвать область действия "LIVENESS" (жизнеспособность), чтобы назначение токена было очевидным.

Вы можете найти пример, обсуждаемый в этом разделе, в проекте `ssia-ch14-ex3`.

Листинг 14.8 Настройка зарегистрированного клиента, работающего с предоставлением учетных данных

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
    RegisteredClient registeredClient =
```

```

RegisteredClient.withId(UUID.randomUUID().toString())
    .clientId("client")
    .clientSecret("secret")
    .clientAuthenticationMethod(
        ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
    .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
    .scope("CUSTOM")
    .build();

return new InMemoryRegisteredClientRepository(registeredClient);
}

```

- ① Разрешаем зарегистрированному клиенту использовать грант в форме учетных данных клиента.
- ② Настройка области действия в соответствии с целью запроса токена доступа.

На рис. 14.7 показан процесс использования учетных данных клиента, который мы обсуждали в главе 13. Чтобы получить токен доступа, клиент просто отправляет запрос и проходит аутентификацию, используя свои учетные данные (идентификатор клиента и секрет).

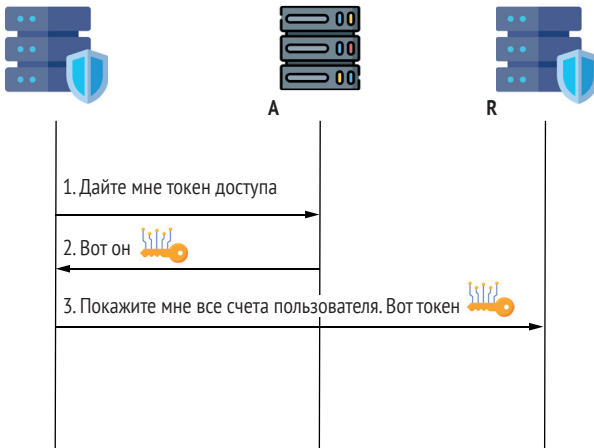


Рис. 14.7 Грант в форме учетных данных клиента. Приложение может получить токен доступа, только пройдя аутентификацию с предоставлением своих учетных данных клиента

Ниже показан запрос токена с помощью команды cURL. Если вы сравните его с запросом, который мы использовали в разделе 14.2, когда применяли грант в форме кода авторизации, вы увидите, что этот вариант проще. Клиенту нужно только упомянуть, что он предоставляет учетные данные клиента и область, в которой он запрашивает токен. Клиент использует свои учетные данные с HTTP Basic в запросе на аутентификацию:

```

curl -X POST 'http://localhost:8080/oauth2/token?
  ➤grant_type=client_credentials&
  ➤scope=CUSTOM' \
  --header 'Authorization: Basic Y2xpZW50bnNlY3JldA=='

```

В следующем фрагменте показано тело ответа HTTP, содержащее запрошенный токен доступа:

```
{
  "access_token": "eyJraWQiOiI4N2E3YjJiNS...",
  "scope": "CUSTOM",
  "token_type": "Bearer",
  "expires_in": 300
}
```

14.4 Использование непрозрачных токенов и интроспекции

К настоящему моменту в этой главе мы продемонстрировали тип предоставления кода авторизации (раздел 14.2) и тип предоставления учетных данных клиента (раздел 14.3). В обоих случаях нам удалось настроить клиентов, которые могут получать прозрачные токены доступа. Однако вы также можете легко настроить клиентов на использование непрозрачных токенов. В этом разделе я продемонстрирую настройку зарегистрированных клиентов для получения непрозрачных токенов и то, как сервер авторизации помогает с проверкой непрозрачных токенов. Пример, обсуждаемый в данном разделе, вы найдете в проекте `ssia-ch14-ex4`.

В листинге 14.9 показан код конфигурирования зарегистрированного клиента для использования непрозрачных токенов. Помните, что непрозрачные токены можно использовать с любым типом гранта. В этом разделе для простоты я буду использовать учетные данные клиента, чтобы вы могли сосредоточиться на обсуждаемой теме. Вы также можете генерировать непрозрачные токены с грантом в форме кода авторизации.

Листинг 14.9 Настройка клиентов для использования непрозрачных токенов

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
    RegisteredClient registeredClient =
        RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("client")
            .clientSecret("secret")
            .clientAuthenticationMethod(
                ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
            .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
            .tokenSettings(TokenSettings.builder()
                .accessTokenFormat(OAuth2TokenFormat.REFERENCE)
```

1


```

        .build()
        .scope("CUSTOM")
        .build();

    return new InMemoryRegisteredClientRepository(registeredClient);
}

```

❶ Конфигурирование клиента для использования непрозрачных токенов доступа.

Если вы запросите токен доступа, как вы научились делать в разделе 14.3, вы получите непрозрачный токен. Этот токен короче и не содержит данных. Ниже показана команда cURL с запросом токена доступа:

```

curl -X POST 'http://localhost:8080/oauth2/token?
➡grant_type=client_credentials&
➡scope=CUSTOM' \
--header 'Authorization: Basic Y2xpZW50bnNlY3JldA=='

```

Далее показан ответ, похожий на тот, который вы получили, когда ожидали прозрачные токены. Единственное отличие – это сам непрозрачный токен, который больше не является токеном JWT:

```

{
  "access_token": "iED8-...",
  "scope": "CUSTOM",
  "token_type": "Bearer",
  "expires_in": 299
}

```

В следующем фрагменте показан пример полностью непрозрачного токена. Обратите внимание, что он намного короче и не имеет такой же структуры, как JWT (отсутствуют три части, разделенные точками):

```

iED8-aUd5QLTfihDOTGUhKgKwzhJFzY
➡WnGdpNT2UZW03VVDqtMONNdozq1
➡r9r7RiP0aNWgJipcEu5HecAJ75V
➡yNJyNuJ-kaJvjpWL5Ns7Ndb7Uh6
➡DI6M1wMuUcUDEjJP

```

Поскольку непрозрачный токен не содержит данных, как кто-то может проверить его и получить больше сведений о клиенте (и потенциально о пользователе), для которого его сгенерировал сервер авторизации? Самый простой (и наиболее часто применяемый) способ – напрямую обратиться к серверу авторизации. Сервер авторизации предоставляет конечную точку, куда можно отправить запрос с токеном, и отвечает необходимыми сведениями о токене. Этот процесс называется *интроспекцией* (introspection, рис. 14.8).

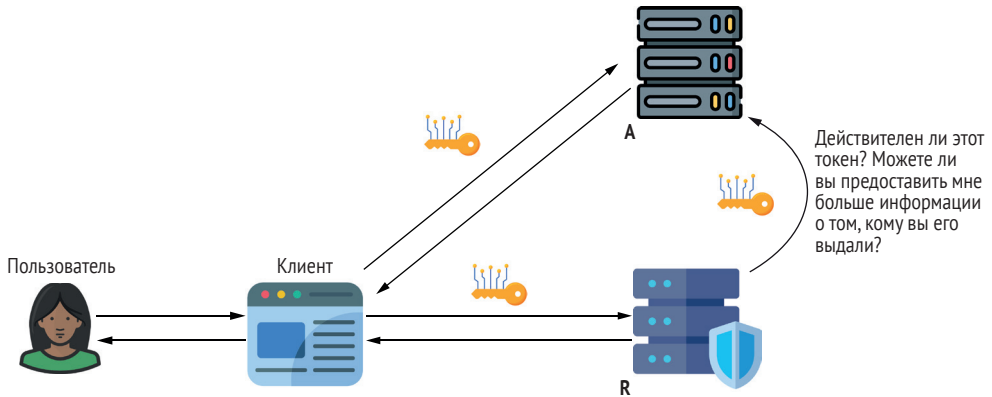


Рис. 14.8 Интроспекция токена. При использовании непрозрачных токенов сервер ресурсов должен отправлять запросы на сервер авторизации, чтобы выяснить, действителен ли токен, и получить более подробную информацию о том, кому он был выдан

В следующем примере командной строки показан запрос cURL к конечной точке интроспекции, которую предоставляет сервер авторизации. При отправке запроса клиент должен использовать HTTP Basic для аутентификации с использованием своих учетных данных. Клиент отправляет токен в качестве параметра запроса и получает сведения о токене в ответ:

```
curl -X POST 'http://localhost:8080/oauth2/introspect?token=iED8-...' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

В следующем фрагменте показан пример ответа на запрос интроспекции для действительного токена. Если токен действителен, его статус отображается как "active", а в ответе содержатся все сведения о токене, которые есть у сервера авторизации:

```
{
  "active": true,
  "sub": "client",
  "aud": [
    "client"
  ],
  "nbf": 1682941720,
  "scope": "CUSTOM",
  "iss": "http://localhost:8080",
  "exp": 1682942020,
  "iat": 1682941720,
  "jti": "ff14b844-1627-4567-8657-bba04cac0370",
  "client_id": "client",
  "token_type": "Bearer"
}
```

Если токен не существует или срок его действия истек, его активный статус – `false`, как показано ниже:

```
{
    "active": false,
}
```

По умолчанию время активности токена равно 300 с. В примерах имеет смысл сделать срок жизни токена более продолжительным. В противном случае у вас не будет достаточно времени, чтобы использовать токен для тестов, что весьма неудобно. В листинге 14.10 показано, как изменить время жизни токена. Я предпочитаю делать его очень продолжительным в примерах (например, 12 часов в данном случае), но помните, что никогда не следует делать его таким большим в реальном приложении. На практике обычно задают время жизни токена от 10 до 30 минут максимум.

Листинг 14.10 Изменение времени жизни токена доступа

```
RegisteredClient registeredClient = RegisteredClient
    .withId(UUID.randomUUID().toString())
    .clientId("client")
    // ...
    .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
    .tokenSettings(TokenSettings.builder()
        .accessTokenFormat(OAuth2TokenFormat.REFERENCE)
        .accessTokenTimeToLive(Duration.ofHours(12)) ❶
        .build())
    .scope("CUSTOM")
    .build();
```

❶ Задаем продолжительность жизни токена 12 часов.

14.5 Отзыв токенов

Предположим, вы обнаружили, что токен был украден. Как можно сделать токен недействительным для использования? *Отзыв токена* (token revocation) – это способ сделать токен, ранее выданный сервером авторизации, недействительным. Обычно срок действия токена доступа и без того короткий, поэтому украденный токен сложно использовать. Но иногда нужно быть особенно осторожным.

В следующем фрагменте показана команда `cURL`, которую можно использовать для отправки запроса на конечную точку отзыва токена, предоставляемую сервером авторизации. Для тестирования вы можете использовать любой из проектов, над которыми мы работали в этой главе. Функция отзыва активна по умолчанию на сервере авторизации Spring Security. Для запроса требуется только токен, ко-

торый вы хотите отозвать, и аутентификация HTTP Basic с учетными данными клиента. После отправки запроса токен больше нельзя использовать:

```
curl -X POST 'http://localhost:8080/oauth2/revoke?token=N7BruErWm-44-...' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

Если вы используете конечную точку интроспекции с токеном, который вы отозвали, то должны заметить, что токен больше не активен после отзыва (даже если его время жизни еще не истекло):

```
curl -X POST 'http://localhost:8080/oauth2/introspect?token=N7BruErWm-44-...' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

Использование отзыва токена иногда имеет смысл, но вряд ли стоит это делать постоянно. Помните, что применение функции отзыва также подразумевает, что вам нужно использовать интроспекцию (даже для прозрачных токенов) при каждом вызове, чтобы проверить, что токен все еще активен. Частое использование интроспекции может сильно повлиять на производительность системы. Вы всегда должны спрашивать себя: действительно ли мне нужен этот дополнительный уровень защиты?

Вспомните, о чем говорилось в первой главе. Иногда достаточно спрятать ключ под коврик; в других случаях вам нужны продвинутые, сложные и дорогие системы сигнализации. Выбор инструмента защиты зависит от того, что вы защищаете.

Заключение

- Фреймворк сервера авторизации Spring Security помогает вам создать собственный сервер авторизации OAuth 2/OpenID Connect с нуля.
- Поскольку сервер авторизации управляет данными пользователя и клиента, вы должны реализовать компоненты, определяющие, как приложение собирает эти данные:
 - для управления данными пользователя серверу авторизации, как и любому другому веб-приложению, нужен компонент Spring Security UserDetailsService;
 - для управления данными клиента сервер авторизации предоставляет другой контракт, который вам необходимо реализовать, – RegisteredClientRepository.
- Вы можете регистрировать клиентов, использующих различные процедуры аутентификации (типы грантов). Желательно, чтобы один и тот же клиент не использовал разные процедуры, основанные на разных грантах.

- При использовании прозрачных токенов (обычно JWT) необходимо также настроить компонент для управления парами ключей, которые сервер авторизации использует для подписи токенов. Этот компонент называется `JWKSSource`.
- При использовании непрозрачных токенов (токенов, не содержащих данных) сервер ресурсов должен использовать конечную точку интроспекции для проверки действительности токена и сбора данных, необходимых для авторизации.
- Иногда бывает необходимо аннулировать уже выданные токены. Сервер авторизации предлагает конечную точку отзыва токенов. При использовании функции отзыва токенов сервер ресурсов всегда должен проверять токены (даже прозрачные) на предмет их действительности.

15

Реализация сервера ресурсов OAuth 2

Темы этой главы:

- реализация сервера ресурсов Spring Security OAuth 2;
- использование токенов JWT с пользовательскими параметрами;
- настройка интроспекции для непрозрачных токенов или отзыва;
- реализация более сложных сценариев и многопользовательской среды.

В этой главе обсуждается защита приложения бэкенда в системе OAuth 2. То, что мы называем сервером ресурсов в терминологии OAuth 2, – это просто служба бэкенда (внутренняя служба). В главе 14 вы узнали, как отработать ответственность сервера авторизации с помощью Spring Security, а теперь пришло время обсудить, как использовать токен, который генерирует сервер авторизации.

В реальности вам не обязательно реализовать собственный сервер авторизации, как мы это сделали в главе 14. Ваша организация может использовать готовый сторонний продукт вместо создания собственного программного обеспечения. Вам доступно множество альтернатив, начиная от решений с открытым исходным кодом, таких как Keycloak, до корпоративных продуктов, таких как Okta,

Cognito или Azure AD. Пример с Keycloak показан в главе 18 первого издания книги.

Хотя у вас есть возможности настроить готовый сервер авторизации без необходимости создавать собственный, вам придется правильно реализовать аутентификацию и авторизацию на своем бэкенде. Я написал об этом отдельную главу; навыки, которые вы приобретете, прочитав ее, наверняка помогут вам в работе. Рисунок 15.1 напоминает вам об акторах OAuth 2 и о том, на каком этапе процесса обучения мы находимся.

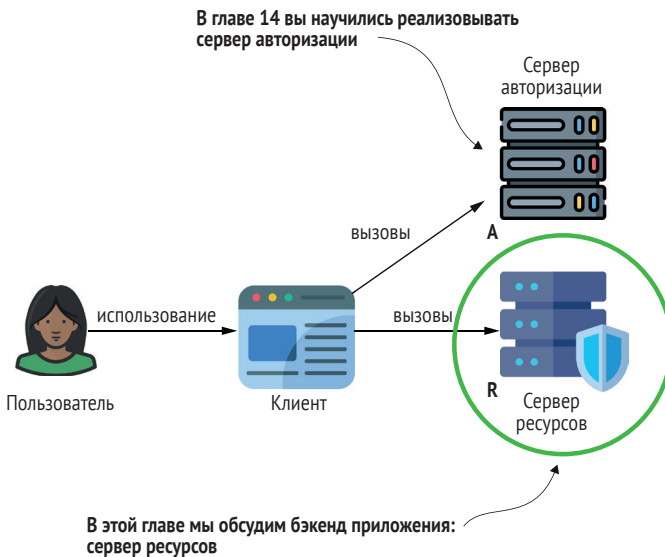


Рис. 15.1 В OAuth 2 бэкенд приложения называется сервером ресурсов, потому что он защищает ресурсы пользователей и клиентов (данные и действия, которые можно выполнять с данными)

Мы начнем эту главу в разделе 15.1 с обсуждения конфигурации сервера ресурсов для веб-токенов JSON (JWT). Сегодня вы чаще всего встретите JWT, используемые с системой OAuth 2; вот почему мы начинаем с них. В разделе 15.2 мы обсуждаем настройку JWT и использование пользовательских значений в установках тела или заголовка.

В разделе 15.3 мы обсуждаем конфигурирование сервера ресурсов для использования интроспекции при проверке токенов. Процесс интроспекции полезен при работе с непрозрачными токенами или когда вам нужно, чтобы ваша система могла отзывать токены до истечения срока их действия.

Мы закончим главу обсуждением более сложных случаев конфигурации, таких как мультитенантность, в разделе 15.4.

15.1 Настройка проверки JWT

В этом разделе мы обсуждаем конфигурирование сервера ресурсов для проверки и использования JWT – прозрачных токенов, включающих данные, которые сервер ресурсов использует для авторизации. Чтобы использовать JWT, серверу ресурсов необходимо будет доказать их подлинность, то есть убедиться, что ожидаемый сервер авторизации действительно выдал их в качестве доказательства аутентификации пользователя и/или клиента. Далее, серверу ресурсов необходимо будет прочитать данные в токене и использовать их для проверки соблюдения правил авторизации.

Мы научимся настраивать сервер ресурсов в полном цикле запуска, то есть реализуя его и настраивая с нуля. Начнем с создания нового проекта Spring Boot и добавления необходимых зависимостей. Затем мы создадим демонстрационную конечную точку (ресурс для использования в целях тестирования) и поработаем над конфигурацией для аутентификации и авторизации. Вот шаги, которые мы выполним:

- 1 добавим необходимые зависимости в проект (в файл `pom.xml`, поскольку мы используем Maven);
- 2 объявим фиктивную конечную точку, которую мы будем использовать для тестирования нашей реализации;
- 3 реализуем аутентификацию для JWT, настроив службу с помощью URI набора открытых ключей;
- 4 реализуем правила авторизации;
- 5 протестируем сервер путем:
 - a генерации токена с помощью сервера авторизации;
 - b использования токена для вызова фиктивной конечной точки, созданной на шаге 2.

В листинге 15.1 представлены необходимые зависимости. Помимо веб-зависимостей и Spring Security, мы также добавим *стартер сервера ресурсов* (resource server starter).

Листинг 15.1. Зависимости для реализации сервера ресурсов

```
<dependency> ❶
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
</dependency>
```



```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- ❶ Starter сервера ресурсов предоставляет необходимые зависимости для реализации приложения в качестве сервера ресурсов OAuth 2.

Подготовив зависимости, мы создадим фиктивную конечную точку, которую будем использовать для тестирования нашей реализации в конце процесса. В следующем листинге представлен простой контроллер, который предоставляет конечную точку, доступную по пути /demo.

Листинг 15.2 Объявление простой конечной точки для целей тестирования

```
@RestController
public class DemoController {

    @GetMapping("/demo")           ❶
    public String demo() {
        return "Demo";
    }
}
```

- ❶ Определение фиктивной конечной точки, которая понадобится для тестирования наших конфигураций после завершения процесса настройки.

Для этого примера вам нужно использовать сервер авторизации. Вы можете использовать сервер, который мы создали в главе 14 в проекте ssia-ch14-ex1.

Поскольку мы хотим запустить и сервер авторизации, и сервер ресурсов одновременно в одной системе, нам нужно будет настроить для них разные порты. Сервер авторизации имеет порт по умолчанию 8080, и мы можем изменить порт сервера ресурсов на другой. Я указал порт 9090, но вы можете использовать любой свободный порт в вашей системе. Следующий фрагмент кода показывает свойство, которое нужно добавить в файл application.properties для изменения порта:

```
server.port=9090
```

Запустите как сервер авторизации из проекта ssia-ch14-ex1, так и текущее приложение. Вы можете найти этот пример в проекте ssia-ch15-ex1. В главе 14 было сказано, что сервер авторизации OpenID Connect предоставляет URL, который вы можете использовать для получения его конфигурации (включая URL для авторизации, токена, набора открытых ключей и т. д.). Следующий фрагмент представляет так называемый общеизвестный URL:

```
http://localhost:8080/.well-known/openid-configuration
```

Эта ссылка вам нужна для получения информации об URL, предоставленном сервером авторизации для получения набора открытых ключей, который сервер ресурсов может использовать для проверки токенов. Сервер ресурсов должен вызвать эту конечную точку и получить набор открытых ключей. Затем сервер ресурсов использует один из этих ключей для проверки подписи токена доступа (рис. 15.2).

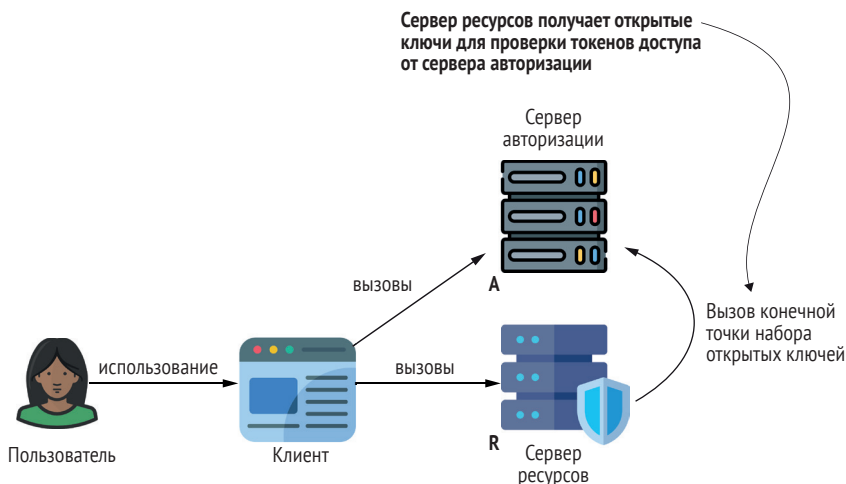


Рис. 15.2 Сервер ресурсов извлекает набор открытых ключей с сервера авторизации через конечную точку, предоставленную сервером авторизации. Затем сервер ресурсов использует эти ключи для проверки подписи токена доступа

Листинг 15.3 напоминает вам об ответе, который вы получаете при вызове открытой конечной точки конфигурации, предоставляемой сервером авторизации. Как вы могли заметить, в число предоставляемых данных входит URI набора открытых ключей. Этот параметр нам нужно настроить на сервере ресурсов, чтобы он мог проверять JWT.

Листинг 15.3 Ответ открытой конечной точки OpenID, содержащий URI набора ключей

```
{
  "issuer": "http://localhost:8080",
  "authorization_endpoint": "http://localhost:8080/oauth2/authorize",
  "device_authorization_endpoint":
    ➔ "http://localhost:8080/oauth2/device_authorization",
  "token_endpoint": "http://localhost:8080/oauth2/token",
  ...
}
```

```
"jwks_uri": "http://localhost:8080/oauth2/jwks",    ❶
...
}
```

❶ Конечная точка набора ключей предоставляет открытые части асинхронных пар ключей, настроенных на стороне сервера авторизации. Сервер авторизации использует закрытые части для подписи токенов. Сервер ресурсов может использовать открытые части для их проверки.

Чтобы задать URI набора открытых ключей, мы сначала объявим его в файле `application.properties` проекта. Класс конфигурации может внедрить его в поле атрибута, а затем использовать для настройки аутентификации сервера ресурсов:

```
keySetURI=http://localhost:8080/oauth2/jwks
```

В листинге 15.4 показан класс конфигурации, внедряющий значение URI набора открытого ключа в атрибут. Класс конфигурации также определяет компонент типа `SecurityFilterChain`. Приложение будет использовать компонент `SecurityFilterChain` для настройки аутентификации, как мы делали в предыдущих главах книги.

Листинг 15.4 Внедрение значения свойства в класс конфигурации

```
@Configuration
public class ProjectConfig {

    @Value("${keySetURI}")    ❶
    private String keySetUri;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        return http.build();
    }
}
```

❶ Внедряем значение URI набора ключа в атрибут файла конфигурации. Он понадобится вам для настройки цепочки фильтров.

Для настройки аутентификации мы будем использовать метод `oauth2ResourceServer()` объекта `HttpSecurity`. Этот метод похож на `httpBasic()` и `formLogin()`, которые мы использовали во второй и третьей частях данной книги.

Как и при использовании `httpBasic()` и `formLogin()`, вам необходимо обеспечить реализацию интерфейса `Customizer` для настройки аутентификации. В листинге 15.5 показано, как я использовал метод `jwt()` объекта `Customizer` для настройки аутентификации JWT. Затем я использовал `Customizer` в методе `jwt()` для настройки URI набора открытых ключей (применяя метод `jwtSetUri()`).

Листинг 15.5 Настройка аутентификации с помощью JWT

```

@Configuration
public class ProjectConfig {

    @Value("${keySetURI}")
    private String keySetUri;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2ResourceServer(
            c -> c.jwt(
                j -> j.jwkSetUri(keySetUri)
            )
        );

        return http.build();
    }
}

```

- ❶ Настройка приложения как сервера ресурсов OAuth 2.
- ❷ Настройка сервера ресурсов для использования JWT при аутентификации.
- ❸ Настройка URL набора открытых ключей, который сервер ресурсов будет использовать для проверки токенов.

Не забудьте сделать так, чтобы конечные точки требовали аутентификации. По умолчанию конечные точки не защищены, поэтому для проверки аутентификации вам сначала нужно убедиться, что ваша конечная точка /demo требует аутентификации. Следующий фрагмент кода настраивает правила авторизации приложения. Для этого примера мы можем заставить все конечные точки требовать аутентификацию:

```

http.authorizeHttpRequests(
    c -> c.anyRequest().authenticated()
);

```

В следующем листинге представлено полное содержимое класса конфигурации.

Листинг 15.6 Полный класс конфигурации

```

@Configuration
public class ProjectConfig {

    @Value("${keySetURI}")
    private String keySetUri;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)

```

```

throws Exception {

    http.oauth2ResourceServer(
        c -> c.jwt(
            j -> j.jwkSetUri(keySetUri)
        )
    );

    http.authorizeHttpRequests(
        c -> c.anyRequest().authenticated()
    );

    return http.build();
}
}

```

Теперь вам следует запустить приложение сервера ресурсов, которое вы только что создали. При этом сервер авторизации тоже должен работать. Вам понадобятся навыки, полученные в главе 14, чтобы сгенерировать токен доступа. Давайте вспомним шаги по использованию кода авторизации (но помните, что вы можете получить токен, используя любой другой тип гранта – для сервера ресурсов не имеет значения, как вы получили токен доступа, если он у вас есть).

Чтобы применить подход на основе кода авторизации, вам необходимо выполнить следующие действия (рис. 15.3):

- 1 перенаправьте пользователя для входа на сервер авторизации / конечной точки авторизации;
- 2 используйте учетные данные пользователя для аутентификации. Сервер авторизации отправит вас на URI перенаправления и предоставит код авторизации;
- 3 получите код авторизации, доступный после перенаправления, и используйте конечную точку /token для запроса нового токена доступа.

Во фрагменте вывода ниже показан URL-адрес, который вы можете использовать в своем браузере для перенаправления на конечную точку /authorize сервера авторизации. Помните, что вам нужно предоставить несколько параметров, и их значения должны соответствовать тому, что вы настроили на сервере авторизации. Вы должны отправить следующие параметры:

- `response_type` – используйте значение "code", если вы хотите использовать код авторизации;
- `client_id` – идентификатор клиента;
- `scope` – область, к которой вы хотите получить доступ. Это может быть любая из областей, настроенных на сервере авторизации;
- `redirect_uri` – URI, на который сервер авторизации перенаправляет клиента после успешной аутентификации. URI пере-

направления должен быть одним из настроенных на сервере авторизации;

- `code_challenge` – при использовании PKCE (ключ подтверждения для обмена кодами) необходимо предоставить код вызова из пары код вызова / верификатор;
- `code_challenge_method` – при использовании PKCE необходимо указать хеш-функцию, которую вы использовали для шифрования верификатора кода (например, SHA-256):

```
http://localhost:8080/oauth2/authorize?response_type=code&client_id=client&scope=openid&redirect_uri=https://www.manning.com/authorized&code_challenge=QYPAZ5NU8yvtlQ9erXrUYR-T5AGCjCF47vN-KsaI2A8&code_challenge_method=S256
```

ПРИМЕЧАНИЕ Помните, что для отправки запроса необходимо вставить URL-адрес авторизации в адресную строку браузера.

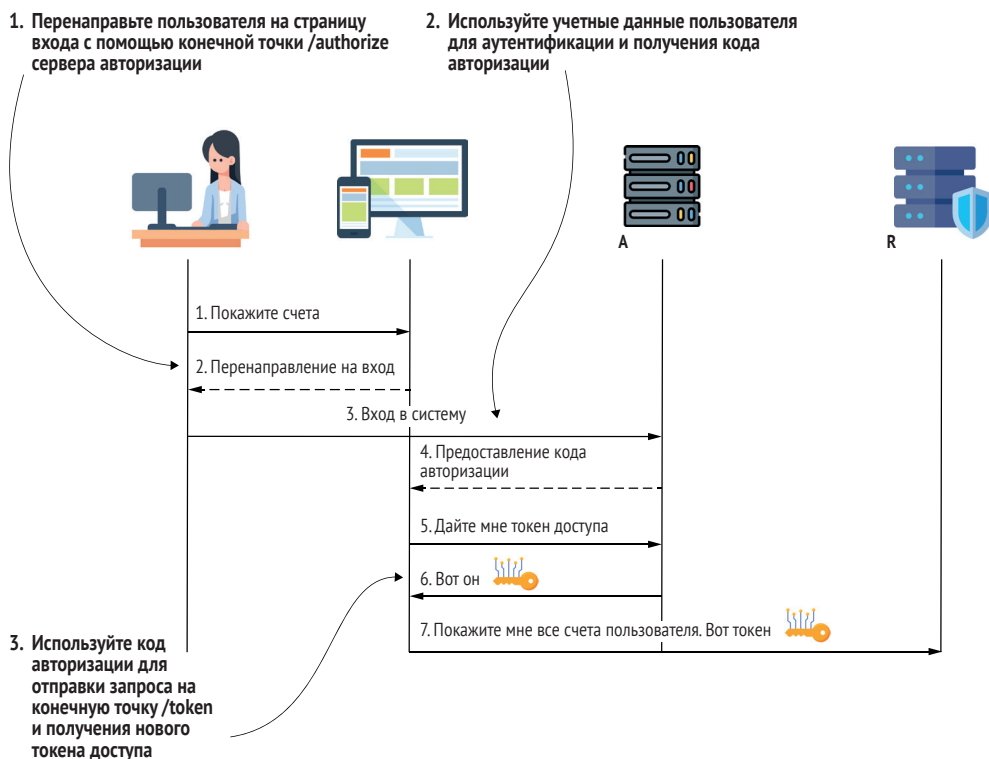


Рис. 15.3 Использование гранта в форме кода авторизации. Приложение-клиент перенаправляет пользователя на страницу входа сервера авторизации. После успешной аутентификации пользователя сервер авторизации перенаправляет пользователя обратно к приложению, предоставляя код авторизации. Приложение использует код авторизации для получения токена доступа

Войдите в систему, используя учетные данные пользователя, настроенные на сервере авторизации, а затем дождитесь перенаправления на запрошенный URI перенаправления. Сервер авторизации предоставляет код авторизации, который необходимо использовать в запросе к конечной точке /token.

В следующем фрагменте кода показан пример команды cURL, отправляющей запрос к конечной точке /token для получения токена доступа. Здесь я обрезал значение кода авторизации, чтобы оно поместилось на странице:

```
curl -X POST 'http://localhost:8080/oauth2/token? \
client_id=client& \
redirect_uri=https://www.manning.com/authorized& \
grant_type=authorization_code& \
code=IhKRpq7GJ7P5VQI_...& \
code_verifier=qPsH306-ZDDaOE8DFzVn05TkN3ZZoVmI_6x4LsVg1QI' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

Далее показано тело ответа после запроса конечной точки /token. Я также обрезал значения токенов во фрагменте:

```
{
  "access_token": "eyJraWQiOiI2Zjk5ZmE3MC...",
  "scope": "openid",
  "id_token": "eyJraWQiOiI2Zjk5ZmE3MC0xNTQ2LTRkMjM...",
  "token_type": "Bearer",
  "expires_in": 299
}
```

Теперь вы можете использовать токен доступа при вызове любой конечной точки, требующей аутентификации. Ниже показана команда cURL для отправки запроса на конечную точку /demo. Обратите внимание, что токен доступа должен быть отправлен в заголовке Authorization с использованием префикса Bearer (рис. 15.4). Префикс Bearer подразумевает, что тот, у кого есть значение токена доступа, может использовать его так же, как и любая другая сторона, у которой оно есть.

Эту команду cURL вы можете использовать для отправки запроса на конечную точку /demo с применением токена доступа от сервера авторизации:

```
curl 'http://localhost:9090/demo' \
--header 'Authorization: Bearer eyJraW...'
```

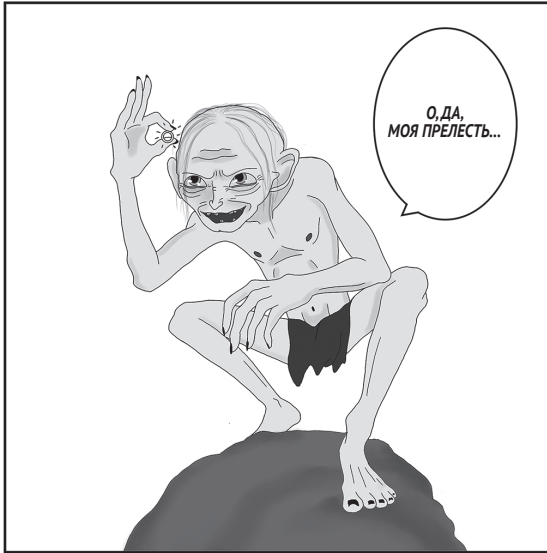


Рис. 15.4 Аналогия с романом «Властелин колец» Дж. Р. Р. Толкиена. Токен доступа – это большая ценность. Он дает доступ к различным ресурсам тому, кто им владеет

15.2 Использование настраиваемых JWT

Различные системы могут существенно отличаться друг от друга даже в отношении аутентификации и авторизации. Часто бывает так, что вам нужно передавать пользовательские значения между сервером авторизации и сервером ресурсов через токен доступа. Сервер ресурсов может использовать такие значения для применения различных правил авторизации.

В этом разделе мы реализуем пример, в котором серверы авторизации и ресурсов используют настраиваемые *утверждения* (claim) в токене доступа. Сервер авторизации формирует JWT, добавляя утверждение с именем "priority" в JWT. Сервер ресурсов считывает утверждение "priority" и добавляет его значение в экземпляр аутентификации в контексте безопасности. Сервер ресурсов может извлекать его оттуда при реализации любого правила авторизации.

Мы выполним следующие шаги:

- 1 изменим сервер авторизации, чтобы добавить пользовательское утверждение в токен доступа;
- 2 изменим сервер ресурсов, чтобы прочитать пользовательское утверждение и сохранить его в контексте безопасности;
- 3 реализуем правило авторизации, которое использует пользовательское утверждение.

Но сначала самое главное! Нам нужно добавить пользовательское значение в тело токена доступа в класс `SecurityConfig`. На сервере авторизации вы делаете это, добавляя `bean`-компонент типа `OAuth2-TokenCustomizer`. В следующем фрагменте кода показано определение такого `bean`-компонента. Для простоты и чтобы не отвлекать вас от примера, я добавил фиктивное значение в поле, которое назвал "priority". В реальных приложениях такие настраиваемые поля имели бы определенное назначение, и вам потенциально пришлось бы написать определенную логику для установки их значения:

```
@Bean
public OAuth2TokenCustomizer<JwtEncodingContext> jwtCustomizer() {
    return context -> {
        JwtClaimsSet.Builder claims = context.getClaims();
        claims.claim("priority", "HIGH");
    };
}
```

Благодаря этому минимальному изменению токены доступа теперь содержат настраиваемое поле "priority". Ниже показан токен доступа JWT, который я сгенерировал в формате, закодированном в Base64, а в листинге 15.7 показано декодированное тело, в котором можно увидеть поле "priority":

```
eyJraWQiOiI5ZTBjOTQ5Ny0zYmMyLTQ4Y2YtODU5MC04N2JmZjE2Zjc2OTAiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJiaWxsIiwiaXVkiOiY2xpZW50IiwibmJmIjoxNjg3MjYzMzI5LCJzY29wZSI6WyJvcGVuaWQiXSwiaXNzIjoiaHR0cDovL2xvY2FsaG9zdDo4MDgwIiwiaXhwIjoxNjg3MjYzNjI5LCJwcmVudCI6eSI6IkhJR0giLCJpYXQiOiJlE2ZD8HkXP0U7gm-dmea01vPgVypvcf3oR3uawIMdI_joQBsLY0zNWBIGktKn2w9-rvgtjD2xmhwZgSxRsDW_GZofqOzV9T-5llMuZlakF7SQLyI67UJZKuPTJK8hBd10hnurGo7ikPFDWhaqqyichKu_uI7SdFrQQVgVqbrmHii-syoURIrI9EwOhB036M7UPJnIwTOWc34fAoFHxqhPuGIVesHHX5qm6wx-8_Orjz96e0ujVSEuUGRNVtz35_SRjhozclZgIo3Rt9lUfLI7HSzulfXTcpxtxja-1E_l_dsk4VHSvLYJUzjLERp5kVJqS0_keaJt8JbDQ0new
```

Листинг 15.7 показывает декодированное тело ранее представленного токена доступа. Помните, что вы можете использовать онлайн-инструмент jwt.io, чтобы получить декодированную форму JWT. В качестве альтернативы вы можете индивидуально декодировать Base64 либо заголовок, либо тело токена доступа, используя любой другой декодер Base64. Следующий листинг демонстрирует, что наши изменения на сервере авторизации работают правильно.

Листинг 15.7 Раскодированное в Base64 тело настроенного токена доступа JWT

```
{
  "sub": "bill",
  "aud": "client",
```

```
"nbf": 1687263329,  
"scope": [  
  "openid"  
],  
"iss": "http://localhost:8080",  
"exp": 1687263629,  
"priority": "HIGH",  
"iat": 1687263329  
}
```

❶ Настраиваемое утверждение "priority", которое мы добавили к токену доступа.

На втором этапе мы вносим изменения на сервере ресурсов. Вы можете продолжить работу над примером, который мы использовали в разделе 15.1, но чтобы облегчить вам обучение, я создал отдельный проект для этого примера. Реализацию, которую мы обсудим далее в этом разделе, вы найдете в проекте `ssia-ch15-ex2`.

Список шагов, которые нам нужно выполнить, чтобы наш сервер ресурсов понимал пользовательские утверждения в токене доступа, следующий:

- 1 создать пользовательский объект аутентификации. Этот объект определит новую форму, включая пользовательские данные;
- 2 создать объект преобразователя аутентификации JWT. Этот объект определит логику для перевода JWT в пользовательский объект аутентификации;
- 3 настроить преобразователь аутентификации JWT, созданный вами на шаге 2, для использования механизмом аутентификации;
- 4 изменить конечную точку `/demo`, чтобы вернуть объект аутентификации из контекста безопасности;
- 5 протестировать конечную точку и убедиться, что объект аутентификации содержит пользовательское поле `"priority"`.

В листинге 15.8 представлено определение объекта аутентификации. Объект аутентификации должен быть любым классом, который напрямую или косвенно наследует класс `AbstractAuthenticationToken`. Поскольку мы используем JWT, удобнее расширять более конкретный `JwtAuthenticationToken`. Таким образом, вы напрямую расширите произвольную форму объекта аутентификации, разработанную для токенов доступа JWT.

Обратите внимание, что в листинге 15.8 добавлено поле с именем `"priority"`. Это поле будет содержать значение из пользовательского утверждения в теле токена доступа. Аналогичным образом вы можете добавить любые другие пользовательские данные, которые могут понадобиться вашему приложению для целей авторизации. Наличие этих данных непосредственно в объекте аутентификации из контекста безопасности упрощает написание конфигураций не-

зависимо от того, будем ли мы применять их на уровне конечной точки (главы 7 и 8) или на уровне метода (главы 11 и 12).

Листинг 15.8 Определение пользовательского объекта аутентификации

```
public class CustomAuthentication
    extends JwtAuthenticationToken { ❶

    private final String priority; ❷

    public CustomAuthentication(
        Jwt jwt,
        Collection<? extends GrantedAuthority> authorities,
        String priority) {

        super(jwt, authorities);
        this.priority = priority;
    }

    public String getPriority() {
        return priority;
    }
}
```

- ❶ Настройка объекта аутентификации путем расширения класса `JwtAuthenticationToken`.
- ❷ Добавление настраиваемого поля `"priority"`.

Теперь у вас есть пользовательская форма объекта аутентификации, и следующее, что вам нужно сделать, – это указать приложению, как преобразовать JWT в этот пользовательский объект. Вы можете сделать это, сконфигурировав специальный преобразователь `Converter`, как показано в листинге 15.9. Обратите внимание на два общих типа, которые мы использовали: `Jwt` и `CustomAuthentication`. Первый является входными данными для преобразователя, а второй – выходными. Таким образом, этот преобразователь изменяет объект `Jwt` (который является стандартным контрактом в Spring Security о том, как считывается токен доступа JWT) в пользовательский тип, который мы реализовали в листинге 15.8 (рис. 15.5).

Листинг 15.9 Преобразование токена доступа в объект аутентификации

```
@Component
public class JwtAuthenticationConverter
    implements Converter<Jwt, CustomAuthentication> {

    @Override
    public CustomAuthentication convert(Jwt source) {
        List<GrantedAuthority> authorities =
```

```

        List.of(() -> "read");

        String priority =
            String.valueOf(source.getClaims().get("priority"));

        return new CustomAuthentication(source,
                                       authorities,
                                       priority);
    }
}

```

- ❶ Получение значений приоритета из пользовательского утверждения токена.
- ❷ Установка значения приоритета в объекте аутентификации.

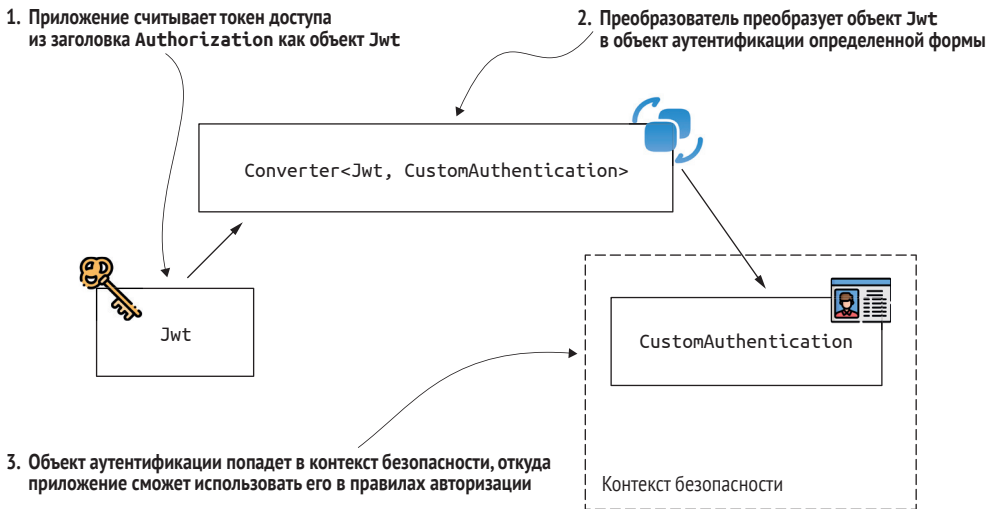


Рис. 15.5 Пользовательский преобразователь реализует логику для помещения информации из токена доступа в пользовательскую форму аутентификации

Вы также можете заметить в листинге 15.9, что я определил фиктивные полномочия. В реальном приложении вы бы либо взяли их из токена доступа (в случае если они управляются на уровне сервера авторизации), либо из базы данных или другой сторонней системы (в случае если они управляются бизнес-логикой). В данном случае я упростил пример и добавил фиктивные полномочия "read" для всех запросов. Но важно помнить, что это также место, где вы будете иметь дело с полномочиями (которые также должны оказаться в объекте аутентификации из контекста безопасности, поскольку они являются важными данными для правил авторизации в большинстве случаев).

В листинге 15.10 показано конфигурирование пользовательского преобразователя. В данном случае я использовал внедрение зависимости, чтобы получить bean-компонент преобразователя из кон-

текста Spring. Затем я использовал метод `jwtAuthenticationConverter()` конфигулятора аутентификации JWT.

Листинг 15.10 Настройка пользовательского преобразователя аутентификации

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    private final JwtAuthenticationConverter converter; ❶

    // Конструктор не показан

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2ResourceServer(
            c -> c.jwt(
                j -> j.jwkSetUri(keySetUri)
                    .jwtAuthenticationConverter(converter) ❷
            )
        );

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );

        return http.build();
    }
}
```

❶ Внедрение объекта преобразователя в поле класса.

❷ Настройка объекта преобразователя в механизме аутентификации.

Это вся конфигурация, которую нам нужно было применить для использования пользовательского утверждения токена доступа. Давайте протестируем нашу реализацию и убедимся, что она работает так, как и ожидалось. В следующем фрагменте кода показаны изменения, которые я внес в конечную точку `/demo`. Я заставил конечную точку `/demo` возвращать экземпляр аутентификации из контекста безопасности. Поскольку Spring знает, как автоматически вводить значение в параметр типа `Authentication`, мне просто нужно было добавить этот параметр, а затем заставить метод конечной точки возвращать его как есть:

```
@GetMapping("/demo")
public Authentication demo(Authentication a) {
    return a;
}
```

Если все работает как надо, то при отправке запроса на конечную точку /demo вы получите ответ с телом, похожим на представленное в следующем листинге. Обратите внимание, что пользовательский атрибут "priority" правильно отображается в объекте аутентификации со значением "HIGH".

Листинг 15.11 Ответ конечной точки /demo содержит значение поля priority

```
{
  "authorities": [
    {
      "authority": "read"
    }
  ],
  "details": {
    "remoteAddress": "0:0:0:0:0:0:1",
    "sessionId": null
  },
  "authenticated": true,
  "name": "bill",
  "priority": "HIGH", ❶
}
```

❶ Значение пользовательского утверждения отображается в экземпляре аутентификации.

15.3 Проверка токена с помощью интроспекции

В этом разделе мы обсудим использование интроспекции для проверки токена доступа. Если ваше приложение использует непрозрачные токены или если вам нужна система, в которой вы можете отзывать токены на уровне сервера авторизации, то для проверки токена необходимо использовать интроспекцию. Рисунок 15.6 напомним вам о процессе интроспекции, детально описанном в разделе 14.4.

Мы создадим сервер ресурсов для демонстрации механизма интроспекции. Чтобы достичь нашей цели, мы должны выполнить следующие шаги:

- 1 убедиться, что сервер авторизации распознает сервер ресурсов как клиента. Серверу ресурсов нужны учетные данные клиента, зарегистрированные на стороне сервера авторизации;
- 2 настроить аутентификацию на стороне сервера ресурсов для использования интроспекции;

- 3 получить токен доступа с сервера авторизации;
- 4 вызвать демонстрационную конечную точку, чтобы убедиться, что конфигурация работает так, как мы ожидали, с токеном доступа, который мы получили на шаге 3.

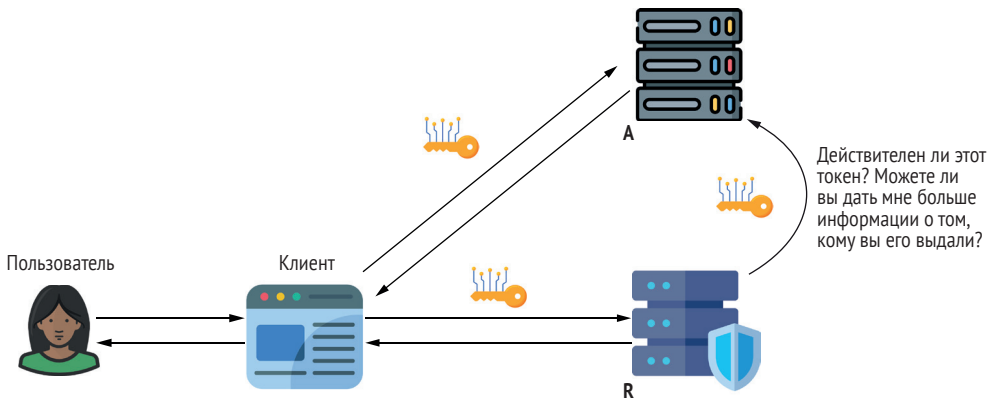


Рис. 15.6 Интроспекция токена. В ситуациях, когда сервер ресурсов не может полагаться на проверку токенов доступа на основе подписи (например, когда требуется отзыв токена) или когда токен не содержит подробной информации (как в случае с непрозрачными токенами), сервер ресурсов должен делать запросы к серверу авторизации. Это делается для подтверждения действительности токена и получения дополнительной информации о нем

Следующий фрагмент кода показывает вам пример создания экземпляра клиента, который мы регистрируем на стороне сервера авторизации. Этот клиент представляет наш сервер ресурсов. Как вы видите на рис. 15.6, сервер ресурсов отправляет запросы на сервер авторизации (для интроспекции), таким образом, он также становится клиентом сервера авторизации.

Для отправки запросов на интроспекцию серверу ресурсов нужны учетные данные клиента для аутентификации, как и любому другому клиенту. Для этого примера я изменю проект `ssia-ch14-ex4`, который мы создали при обсуждении непрозрачных токенов в главе 14:

```
RegisteredClient resourceServer =
    RegisteredClient.withId(UUID.randomUUID().toString())
        .clientId("resource_server")
        .clientSecret("resource_server_secret")
        .clientAuthenticationMethod(
            ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .authorizationGrantType(
            AuthorizationGrantType.CLIENT_CREDENTIALS)
        .build();
```

Помните, что пароли и данные конфигурации никогда не должны быть жестко закодированы, как я сделал в предыдущем фрагменте.

Я максимально упростил эти примеры, чтобы вы могли сосредоточиться на обсуждаемой теме. В реальном приложении вам следует поместить конфигурации в отдельные файлы и надежно сохранить секретные данные (например, учетные данные) в хранилище.

В листинге 15.12 показано, как добавить оба экземпляра клиентских данных (клиента и сервера ресурсов) в компонент `RegisteredClientRepository` сервера авторизации.

Листинг 15.12 Определение `RegisteredClientRepository`

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
    RegisteredClient registeredClient = ❶
        RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("client")
            .clientSecret("secret")
            .clientAuthenticationMethod(
                ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
            .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
            .tokenSettings(TokenSettings.builder()
                .accessTokenFormat(OAuth2TokenFormat.REFERENCE)
                .accessTokenTimeToLive(Duration.ofHours(12))
                .build())
            .scope("CUSTOM")
            .build();

    RegisteredClient resourceServer = ❷
        RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("resource_server")
            .clientSecret("resource_server_secret")
            .clientAuthenticationMethod(
                ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
            .authorizationGrantType(
                AuthorizationGrantType.CLIENT_CREDENTIALS)
            .build();

    return new InMemoryRegisteredClientRepository(
        registeredClient, ❸
        resourceServer);
}
```

- ❶ Определение экземпляра клиентских данных для клиентского приложения.
- ❷ Определение экземпляра клиентских данных для сервера ресурсов (который также становится клиентом при вызове конечной точки интроспекции).
- ❸ Добавление обоих экземпляров клиентских данных в репозиторий сервера авторизации.

Благодаря изменениям, внесенным в листинг 15.12, у нас теперь есть набор учетных данных, которые наш сервер ресурсов может использовать для вызова конечной точки интроспекции, предостав-

ленной сервером авторизации. Мы можем приступить к реализации сервера ресурсов. Вы можете найти этот пример в проекте `ssia-ch15-ex3`. В листинге 15.13 показано, как в файлах свойств я задал три основных значения, необходимых для интроспекции:

- URI интроспекции, предоставленный сервером авторизации, что позволяет серверу ресурсов проверять токены;
- идентификатор клиента сервера ресурсов, который позволяет серверу ресурсов идентифицировать себя при вызове конечной точки интроспекции;
- секрет клиента сервера ресурсов, который сервер ресурсов использует вместе со своим идентификатором клиента для аутентификации при отправке запросов в конечную точку интроспекции.

Кроме того, я изменил порт сервера на 9090, отличный от порта сервера приложений 8080, что позволило обоим приложениям работать одновременно.

Листинг 15.13 Файл `application.properties` сервера ресурсов

```
server.port=9090 ❶

introspectionUri=http://localhost:8080/
  ➔oauth2/introspect ❷
resourceserver.clientID=resource_server ❸
resourceserver.secret=resource_server_secret ❹
```

❶ Изменяем порт сервера ресурсов для одновременной работы сервера ресурсов и сервера авторизации.

❷ Конфигурируем URI интроспекции как свойство.

❸ Конфигурируем идентификатор клиента сервера ресурсов как свойство.

❹ Конфигурируем секрет клиента сервера ресурсов как свойство.

Затем можно перенести значения из файла свойств в поля класса конфигурации и использовать их для настройки аутентификации. В следующем листинге показан класс конфигурации, переносящий значения из файла свойств в поля.

Листинг 15.14 Ввод значений в поля класса конфигурации

```
@Configuration
public class ProjectConfig {

    @Value("${introspectionUri}") ❶
    private String introspectionUri;

    @Value("${resourceserver.clientID}") ❶
    private String resourceServerClientID;

    @Value("${resourceserver.secret}") ❶
```

```
private String resourceServerSecret;

}
```

- ❶ Перенос URI интроспекции, идентификатора клиента интроспекции и секрета интроспекции из файла свойств в поля класса конфигурации.

Используйте URI интроспекции и учетные данные для настройки аутентификации. Мы настраиваем аутентификацию аналогично тому, как делали это для токенов доступа JWT, – с помощью метода `oauth2ResourceServer()` объекта `HttpSecurity`. Однако мы вызываем другой метод конфигурации – объект-настройщик `oauth2ResourceServer()`: `opaqueToken()`. Для метода `opaqueToken()` мы настраиваем URI интроспекции и учетные данные. Конфигурирование показано в листинге 15.15.

Листинг 15.15 Конфигурирование аутентификации сервера ресурсов для непрозрачных токенов

```
@Configuration
public class ProjectConfig {

    @Value("${introspectionUri}")
    private String introspectionUri;

    @Value("${resourceserver.clientID}")
    private String resourceServerClientID;

    @Value("${resourceserver.secret}")
    private String resourceServerSecret;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2ResourceServer(
            c -> c.opaqueToken(❶

                o -> o.introspectionUri(introspectionUri) c
                    .introspectionClientCredentials( d
                        resourceServerClientID,
                        resourceServerSecret)
                    )
        );

        return http.build();
    }
}
```

- ❶ Настройка аутентификации сервера ресурсов для непрозрачных токенов.
 ❷ Настройка URI интроспекции, который сервер ресурсов должен использовать для проверки и получения сведений о токенах.
 ❸ Настройка учетных данных, которые сервер ресурсов должен использовать для аутентификации при вызове URI интроспекции сервера авторизации.

Не забудьте также добавить конфигурации авторизации. Следующий фрагмент кода демонстрирует стандартный способ, о котором вы узнали в главах 7 и 8, как сделать так, чтобы все конечные точки требовали аутентификации запросов:

```
http.authorizeHttpRequests(
    c -> c.anyRequest().authenticated()
);
```

В листинге 15.16 показано полное содержимое класса конфигурации.

Листинг 15.16. Полное содержимое класса конфигурации

```
@Configuration
public class ProjectConfig {

    @Value("${introspectionUri}")
    private String introspectionUri;

    @Value("${resourceserver.clientID}")
    private String resourceServerClientID;

    @Value("${resourceserver.secret}")
    private String resourceServerSecret;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2ResourceServer(
            c -> c.opaqueToken(
                o -> o.introspectionUri(introspectionUri)
                    .introspectionClientCredentials(
                        resourceServerClientID,
                        resourceServerSecret)
            )
        );

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );

        return http.build();
    }
}
```

❶ Добавление конфигурации авторизации конечной точки. Запросы для любой конечной точки требуют аутентификации.

Простой конечной точки /demo, такой как в следующем фрагменте кода, достаточно для проверки, что аутентификация работает правильно:

```
@RestController
public class DemoController {

    @GetMapping("/demo")
    public String demo() {
        return "Demo";
    }
}
```

Теперь вы можете запустить оба приложения: сервер авторизации и сервер ресурсов. Оба должны работать одновременно. Ниже показана команда cURL, которую вы можете использовать для отправки запроса на конечную точку /token. Для упрощения этого примера я использую клиентские данные, но вы можете использовать любой тип гранта, который изучили в главе 14, чтобы получить токен доступа. Помните, что конфигурация сервера ресурсов одинакова независимо от того, как вы получаете токен доступа:

```
curl -X POST 'http://localhost:8080/oauth2/token? \
client_id=client& \
grant_type=client_credentials' \
--header 'Authorization: Basic Y2xpZW50bnNlY3JldA=='
```

Если запрос будет успешным, в ответе вы получите токен доступа. Тело ответа показано в следующем фрагменте. Я обрезал значение токена, чтобы оно лучше помещалось на страницу:

```
{
  "access_token": "2zLyYA8b6Q54-...",
  "token_type": "Bearer",
  "expires_in": 43199
}
```

Точно так же, как и для токенов доступа JWT, при отправке запроса на защищенную конечную точку добавьте токен в качестве значения заголовка "Authorization". Значение токена доступа должно иметь префикс в виде строки "Bearer". Далее показана команда cURL, которую вы можете использовать для отправки запроса на конечную точку /demo. Если все работает правильно, вы получите строку "Demo" в теле ответа со статусом 200 OK:

```
curl 'http://localhost:9090/demo' \
--header 'Authorization: Bearer 2zLyYA8b6Q54-...'
```

15.4 Реализация многопользовательских систем

Реальные приложения не всегда идеальны. Иногда мы оказываемся в ситуации, когда нам нужно адаптировать приложение к нестандартным требованиям при интеграции со сторонними организациями. Кроме того, иногда нам нужно реализовать бэкенды, которые опираются на несколько серверов авторизации для аутентификации и авторизации (многопользовательские системы). Как нам быть в таких случаях?

К счастью, Spring Security обладает достаточной гибкостью для реализации любого сценария. В этом разделе мы обсудим конфигурации сервера ресурсов для более сложных случаев, таких как многопользовательские системы или взаимодействие с приложениями, которые отличаются от стандартных.

Обратимся к рис. 15.7, чтобы еще раз рассмотреть схему аутентификации Spring Security, которую мы подробно обсуждали в пер-

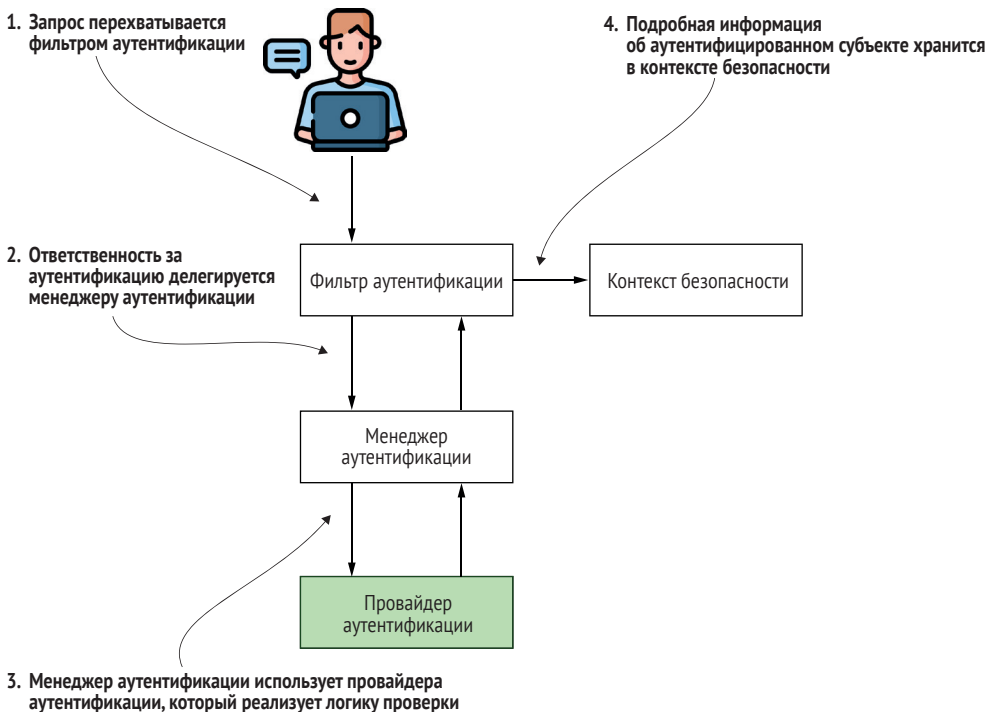


Рис. 15.7 Схема класса аутентификации. Во время процедуры аутентификации фильтр захватывает запрос и передает его компоненту диспетчера аутентификации. Затем этот диспетчер использует провайдера аутентификации, который выполняет необходимую логику. После успешной аутентификации приложение записывает данные аутентифицированного субъекта в контекст безопасности

вых двух частях этой книги. Фильтр перехватывает HTTP-запрос. Затем ответственность за аутентификацию делегируется менеджеру аутентификации. Менеджер аутентификации далее применяет провайдера аутентификации, который реализует логику аутентификации.

Почему важно помнить об этой схеме? Потому что для сервера ресурсов, как и для любого другого подхода к аутентификации, вам необходимо изменить провайдера аутентификации, если вы хотите сконфигурировать этот процесс под себя.

В случае сервера ресурсов Spring Security позволяет вам подключить к конфигурации компонент, называемый *распознавателем диспетчера аутентификации* (authentication manager resolver, рис. 15.8). Этот компонент дает возможность приложению в ходе работы решать, какой диспетчер аутентификации вызывать. Таким образом, вы можете делегировать аутентификацию любому пользовательскому диспетчеру, который может выбирать провайдера аутентификации.

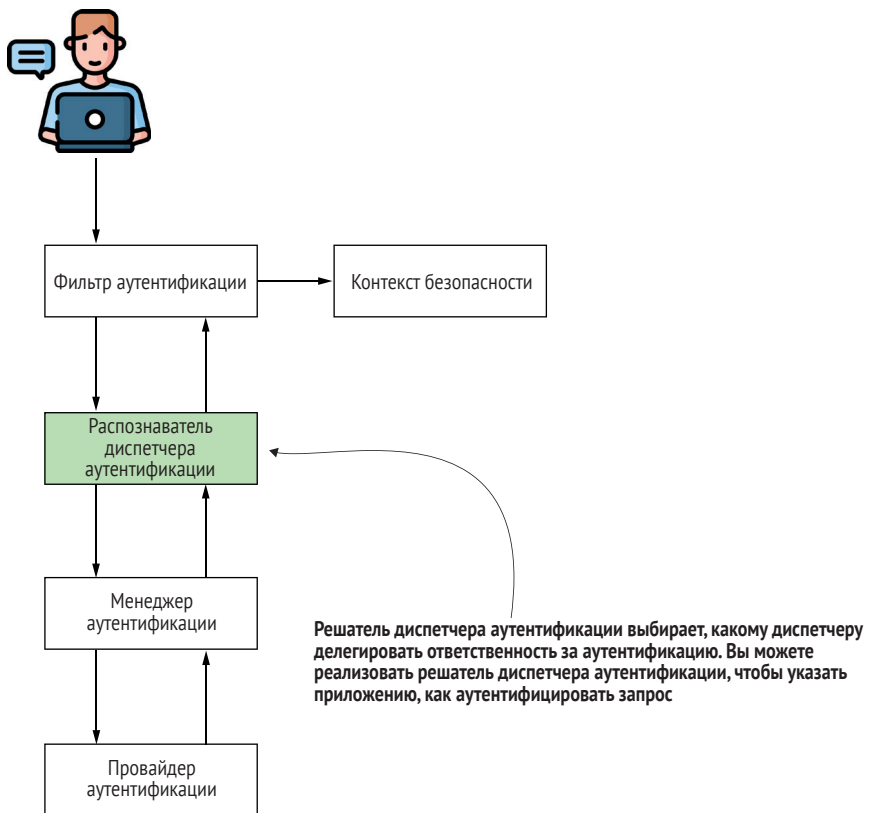


Рис. 15.8 Реализуя распознаватель диспетчера аутентификации, вы указываете своему приложению, какому диспетчеру делегировать ответственность за аутентификацию

Если вы хотите, чтобы ваше приложение использовало несколько серверов авторизации, все из которых применяют JWT, Spring Security даже предоставляет готовую реализацию распознавателя диспетчера аутентификации (рис. 15.9). В таком случае вам нужно только подключить пользовательскую реализацию `JwtIssuerAuthenticationManagerResolver`, которую предоставляет Spring Security.

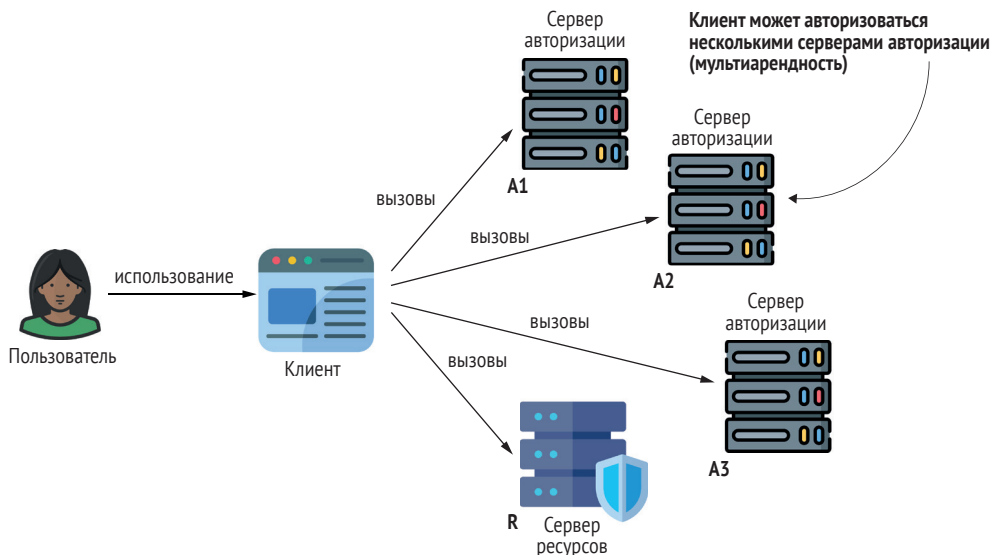


Рис. 15.9 Вашей системе могут понадобиться несколько серверов авторизации для аутентификации пользователей и клиентов

В листинге 15.17 показано, как использовать метод `authenticationManagerResolver()` при настройке аутентификации. В этом примере вы видите, что мне достаточно было создать экземпляр класса `JwtIssuerAuthenticationResolver`, для которого я предоставил все адреса серверов авторизации. Вы можете найти код этого примера в проекте `ssia-ch15-ex4`.

ПРИМЕЧАНИЕ Помните, что никогда не следует писать URL-адреса (или любые подобные пользовательские данные) непосредственно в коде. Я делаю это только в примерах, чтобы упростить код и помочь вам сосредоточиться на том, что вам необходимо изучить. Все настраиваемые параметры всегда должны быть записаны в файлах конфигурации или переменных среды.

Листинг 15.17 Работа с двумя серверами авторизации, использующими токены доступа JWT

```
@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2ResourceServer(
            j -> j.authenticationManagerResolver(
                authenticationManagerResolver())
        );

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );

        @Bean
        public AuthenticationManagerResolver<HttpServletRequest>
            authenticationManagerResolver() {
            var a = new JwtIssuerAuthenticationManagerResolver(
                "http://localhost:7070",
                "http://localhost:8080");
            return a;
        }
    }
}
```

С конфигурацией, подобной представленной на рис. 15.10, ваш сервер ресурсов работает с двумя серверами авторизации, работающими на портах 7070 и 8080.

Однако иногда все сложнее. Spring Security не может предоставить все мыслимые конфигурации «из коробки». В таком случае, когда вам нужно еще больше отклониться от стандартных возможностей сервера ресурсов, вы должны разработать свой собственный распознаватель менеджера авторизации.

Допустим, вам нужно, чтобы ваш сервер ресурсов работал как с JWT, так и с непрозрачными токенами с двумя разными серверами авторизации. Допустим также, что ваш сервер ресурсов различает запросы на основе значения параметра "type". Если значение параметра "type" равно "jwt", сервер ресурсов должен аутентифицировать запрос с помощью сервера авторизации, использующего токены доступа JWT; в противном случае он использует сервер авторизации с непрозрачными токенами доступа.

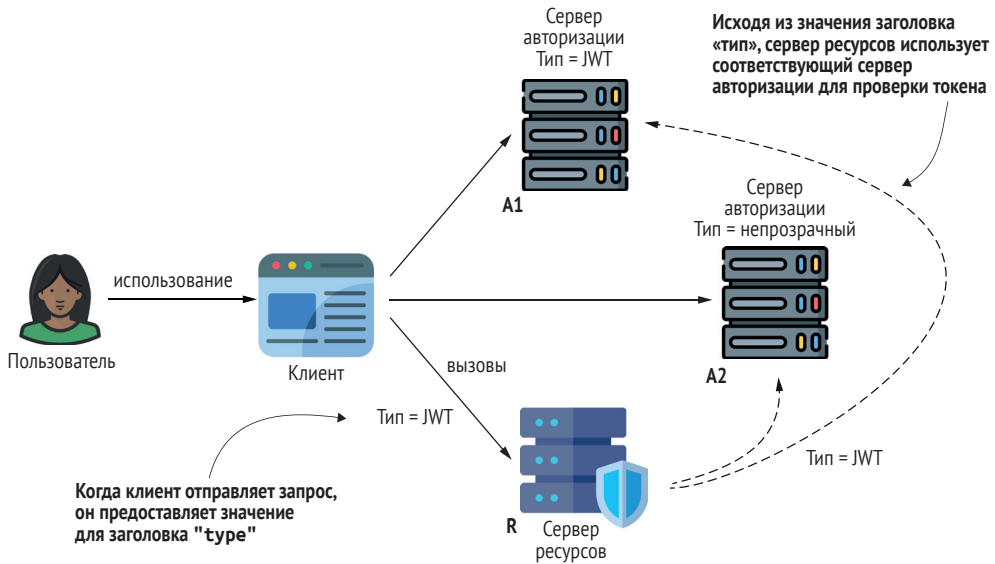


Рис. 15.10 Использование двух отдельных серверов авторизации, каждый из которых обрабатывает свой тип токена. В зависимости от конкретного значения в заголовке HTTP-запроса, используемого клиентом, сервер ресурсов определяет, какой сервер авторизации использовать для проверки токенов доступа

Этот сценарий реализован в коде из листинга 15.18. Сервер ресурсов обращается к разным серверам авторизации на основе значения заголовка "type" в HTTP-запросе. Для этого сервер ресурсов использует специально разработанный менеджер аутентификации, анализирующий значения этого заголовка.

Листинг 15.18 Использование как JWT, так и непрозрачных токенов

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public AuthenticationManagerResolver<HttpServletRequest>
        authenticationManagerResolver(
            JwtDecoder jwtDecoder,
            OpaqueTokenIntrospector opaqueTokenIntrospector
        ) {

        AuthenticationManager jwtAuth = new ProviderManager(
            new JwtAuthenticationProvider(jwtDecoder)
        );

        AuthenticationManager opaqueAuth = new ProviderManager(
            new OpaqueTokenAuthenticationProvider(opaqueTokenIntrospector)
        );
    }
}
```

```

    );

    return (request) -> {
        if ("jwt".equals(request.getHeader("type"))) {
            return jwtAuth;
        } else {
            return opaqueAuth;
        }
    };
}

@Bean
public JwtDecoder jwt'Decoder() {
    return NimbusJwtDecoder
        .withJwkSetUri("http://localhost:7070/oauth2/jwks")
        .build();
}

@Bean
public OpaqueTokenIntrospector opaqueTokenIntrospector() {
    return new SpringOpaqueTokenIntrospector(
        "http://localhost:6060/oauth2/introspect",
        "client", "secret");
}
}

```

- ❶ Определение менеджера аутентификации для сервера авторизации, управляющего токенами доступа JWT.
- ❷ Определение другого менеджера аутентификации для сервера авторизации, управляющего непрозрачными токенами.
- ❸ Определение логики распознавателя менеджера аутентификации для выбора менеджера на основе заголовка "type" HTTP-запроса.
- ❹ Настройка URI набора открытых ключей для менеджера аутентификации, работающего с сервером авторизации, управляющим токенами доступа JWT.
- ❺ Настройка URI интроспекции и учетных данных для менеджера аутентификации, работающего с сервером авторизации, управляющим непрозрачными токенами.

В следующем листинге показана оставшаяся часть конфигурации, предназначенная для настройки распознавателя менеджера авторизации с помощью параметра настройщика метода `authenticationManagerResolver()`.

Листинг 15.19 Конфигурирование `AuthenticationManagerResolver`

```

@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2ResourceServer(

```

```

        j -> j.authenticationManagerResolver(
            authenticationManagerResolver(
                jwtDecoder(),
                opaqueTokenIntrospector()
            ))
    );

    http.authorizeHttpRequests(
        c -> c.anyRequest().authenticated()
    );

    return http.build();
}

// Часть кода не показана
}

```

❶ Конфигурирование распознавателя пользовательского диспетчера аутентификации.

Даже в этом примере мы использовали реализации поставщика аутентификации, предлагаемые Spring Security: `JwtAuthenticationProvider` и `OpaqueTokenAuthenticationProvider`. В этом случае `JwtAuthenticationProvider` реализует логику аутентификации для работы со стандартным сервером авторизации с использованием токенов доступа JWT. `OpaqueTokenAuthenticationProvider` реализует логику аутентификации, работающую с непрозрачными токенами. Но реальные приложения могут быть устроены еще сложнее.

Если вам нужно реализовать что-то очень индивидуальное, например интеграцию с системой, которая не соответствует никакому стандарту, то вы даже можете создать свой собственный провайдер аутентификации.

Заключение

- Spring Security обеспечивает поддержку серверов ресурсов OAuth 2/OpenID Connect. Чтобы настроить аутентификацию как сервер ресурсов OAuth 2/OpenID Connect, используйте метод `oauth2ResourceServer()` объекта `HttpSecurity`.
- Если вы хотите использовать JWT, вам необходимо применить конфигурацию с помощью метода `jwt()` параметра настройщика `oauth2ResourceServer()`.
- Вы также можете использовать интроспекцию, если ваша система использует непрозрачные токены или если вы хотите иметь возможность отзываться JWT на стороне сервера авторизации. В таком случае вы должны настроить аутентификацию с помощью метода `opaqueToken()` параметра настройщика `oauth2ResourceServer()`.

- При использовании JWT необходимо настроить URI набора открытых ключей. Он предоставляется сервером авторизации. Сервер ресурсов вызывает этот URI, чтобы получить открытые части для пар ключей, заданных на стороне сервера авторизации. Сервер авторизации использует закрытые части для подписи токенов доступа, в то время как серверу ресурсов нужны открытые части для их проверки.
- При использовании интроспекции вам необходимо настроить URI интроспекции. Сервер ресурсов отправляет запросы на URI интроспекции, чтобы узнать у сервера авторизации, действителен ли токен, и получить более подробную информацию о нем. При вызове URI интроспекции сервер ресурсов выступает в качестве клиента для сервера авторизации, поэтому ему нужны собственные учетные данные для аутентификации.
- Spring Security дает возможность настраивать логику аутентификации с помощью компонента определителя диспетчера аутентификации. Вы определяете и настраиваете такой пользовательский компонент, когда вам необходимо решить нестандартные задачи, например поддерживать многопользовательскую среду или адаптировать ваше приложение к нестандартной реализации системы.

16

Реализация клиента OAuth 2

Темы этой главы:

- реализация входа OAuth 2;
- реализация клиента Spring Security OAuth 2;
- применение гранта в форме учетных данных клиента.

Часто бывает необходимо реализовать связь между внутренними приложениями, особенно если эти приложения включают в себя несколько служб. В таких случаях, когда аутентификация и авторизация системы построены на OAuth 2, рекомендуется аутентифицировать вызовы между приложениями, используя тот же подход. Хотя в некоторых случаях для простоты разработки выбирают методы аутентификации HTTP Basic и API Key (глава 6), использование механизма OAuth 2 более предпочтительно с точки зрения безопасности и целостности системы.

Помните основных акторов OAuth 2 (рис. 16.1)? Мы рассмотрели сервер авторизации в главе 14 и сервер ресурсов в главе 15. Эта глава посвящена приложению-клиенту. Мы обсудим применение Spring Security для реализации клиента OAuth 2, а также когда и как бэкэнд-приложение становится клиентом в системе OAuth 2.

Я должен сказать, что рис. 16.1 не полностью иллюстрирует то, о чем мы будем говорить. Мы начнем с обсуждения входа для пользователя, но дальше речь пойдет о том, как сделать бэкэнд-приложение клиен-

том для другого бэкэнд-приложения. Бэкэнд-приложения, разработанные с помощью Spring Security, также могут стать клиентами. На рис. 16.2 показан другой случай, который нам предстоит рассмотреть. Мы решим задачу установления связи между двумя бэкэнд-приложениями, сделав одно из них клиентом OAuth 2. В таком случае нам нужно использовать Spring Security для создания клиента OAuth 2.



Рис. 16.1 Акторы OAuth2. В этой главе мы обсудим бэкэнд-приложение, действующее как клиент в системе с аутентификацией и авторизацией на основе OAuth 2

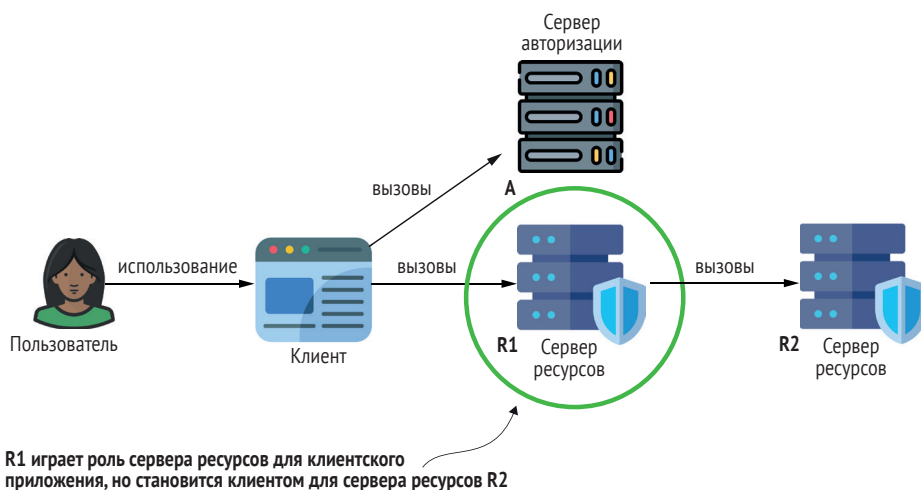


Рис. 16.2 Бэкэнд-приложение может стать клиентом для другого бэкэнд-приложения. Мы обсудим этот случай в текущей главе

В разделе 16.1 говорится о том, как легко реализовать вход OAuth 2 для веб-приложения Spring MVC с помощью Spring Security. Мы воспользуемся услугами внешнего провайдера авторизации, такого как Google и GitHub. Вы узнаете, как реализовать вход для вашего приложения, где пользователи могут аутентифицироваться со своей учетной записью Google или GitHub. Затем вы сможете организовать такой вход на основе собственного сервера авторизации.

В разделе 16.2 мы используем реализацию клиента через службу и рассматриваем применение гранта в форме учетных данных клиента.

16.1 Реализация входа OAuth 2

В этом разделе обсуждается реализация входа OAuth 2 для веб-приложения Spring. С помощью Spring Boot не составит труда настроить аутентификацию для стандартных случаев, когда сервер авторизации точно соответствует спецификациям OAuth 2 и OpenID Connect. Начнем с классического случая (который можно использовать с большинством известных провайдеров, таких как Google, GitHub, Facebook и Okta).

Затем я покажу вам, что скрывается за кулисами самостоятельно разработанной конфигурации, чтобы вы чувствовали себя уверенно при решении нестандартных задач. В конце этого раздела вы сможете реализовать вход для своего веб-приложения Spring с любым поставщиком OAuth 2 и даже разрешить своим пользователям выбирать между различными провайдерами при аутентификации.

16.1.1 Аутентификация с одним провайдером

В этом разделе мы рассмотрим простейший случай входа, когда пользователи могут входить в приложение только с одним провайдером аутентификации. Для этого примера в качестве провайдера аутентификации я выбрал Google.

Начнем с добавления нескольких ресурсов в проект для реализации простого веб-приложения Spring с упомянутыми возможностями входа. В листинге 16.1 показаны зависимости для демонстрационного приложения. Полный код приложения доступен в проекте ssia-ch16-ex1. Вы узнаете новую зависимость, которую мы не использовали в предыдущих главах, – зависимость клиента OAuth 2.

Листинг 16.1 Зависимости, необходимые для демонстрационного приложения

```
<dependency>  
  <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-oauth2-client
        ➡</artifactId>
    </dependency>
</dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
</dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

```

- ❶ Единственная новая зависимость, которую вы видите, – это зависимость клиента OAuth 2. Ее достаточно для получения доступа ко всем возможностям клиента OAuth 2, которые мы настраиваем в проекте.

Если вам нужно освежить знания о создании веб-приложений с помощью Spring Boot, перечитайте главы 7 и 8 еще одной моей книги «Spring быстро» (Питер, 2022). В следующем фрагменте кода показан простой контроллер нашего демонстрационного веб-приложения, у которого есть только домашняя страница:

```

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "index.html";
    }
}

```

Далее показан код небольшой демонстрационной HTML-страницы, к которой мы ожидаем получить доступ после успешного завершения аутентификации:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <h1>Home</h1>
</body>
</html>

```

В листинге 16.2 представлена конфигурация входа OAuth 2 в качестве метода аутентификации для веб-приложения. В соответствии с этой конфигурацией приложение автоматически будет применять грант в форме кода авторизации, перенаправляя пользователя для входа на определенный сервер авторизации и возвращая обратно

после успешной аутентификации. Этот процесс мы подробно обсуждали в главах 13–15 и несколько раз тестировали его с помощью команд cURL.

Листинг 16.2 Конфигурация входа OAuth 2

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2Login(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated());

        return http.build();
    }
}
```

❶ Чтобы задать аутентификацию через вход OAuth 2, мы используем метод `oauth2Login()`.

Я уверен, что вы подумали: разве нам не нужно указывать все те данные, о которых мы узнали в главах 13–15, такие как URL авторизации, URL токена, идентификатор клиента, секрет клиента и т. д.? Да, все эти данные необходимы. К счастью, Spring Security снова вам поможет. Если ваше приложение использует одного из провайдеров, которых Spring Security считает общеизвестными, большинство этих данных уже заполнены. Вам нужно только задать учетные данные клиента приложения. Spring Security считает общеизвестными следующих провайдеров аутентификации:

- Google;
- GitHub;
- Okta;
- Facebook.

Spring Security хранит предварительно настроенные данные для этих провайдеров в классе `CommonOAuth2Provider`. Поэтому вам достаточно настроить учетные данные клиента в свойствах вашего приложения. Ниже показаны два свойства, которые вам нужны для настройки идентификатора клиента и секретного ключа клиента при использовании Google (я обрезал значения своих учетных данных):

```
spring.security.oauth2.client.registration.google.client-id=790...
spring.security.oauth2.client.registration.google.client-secret=G0C...
```

Здесь и далее я подразумеваю, что вы зарегистрировали свое приложение в консоли разработчика Google – именно оттуда вы получаете уникальный набор учетных данных своего приложения. Если вы еще этого не сделали, но хотите настроить аутентификацию с помощью Google для своего приложения, изучите подробное руководство о том, как зарегистрировать свое приложение OAuth 2 в Google, по адресу <http://mng.bz/eEvz>. На рис. 16.3 показано, как приложение отображает логин Google при правильной настройке этого известного поставщика.

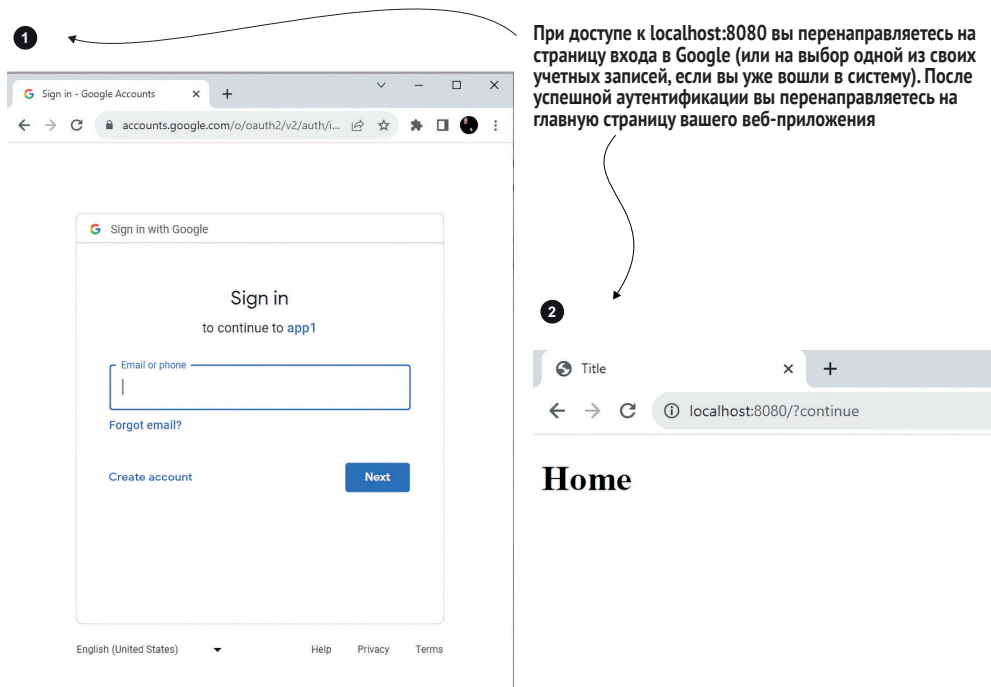


Рис. 16.3 При доступе к приложению браузер перенаправляет вас на страницу входа в Google. Если Google одобрит аутентификацию, вы будете перенаправлены обратно на главную страницу приложения

16.1.2 Предоставление пользователю дополнительных возможностей

Я уверен, что вы уже достаточно побродили по интернету, чтобы заметить, что многие приложения предлагают пользователю более одного способа входа в систему. Иногда вы даже можете выбрать одного из четырех или пяти поставщиков для входа в приложение. Такой подход удобен, поскольку не у всех пользователей есть учетная запись в конкретной социальной сети. У одних есть учет-

ная запись Facebook, а другие предпочитают использовать LinkedIn. Некоторые разработчики склонны входить в систему, используя свою учетную запись GitHub, но другие предпочитают вводить адрес Gmail.

С помощью Spring Security вы легко предоставите пользователям выбор из нескольких провайдеров аутентификации. Допустим, я хочу разрешить пользователям моего приложения входить в систему либо через Google, либо через GitHub. Мне нужно только настроить учетные данные для обоих провайдеров. В следующем фрагменте показаны свойства, которые необходимо прописать в файле `application.properties` для добавления GitHub в качестве метода аутентификации. Учтите, что вы должны также сохранить те, которые мы уже настроили для Google в разделе 16.1.1:

```
spring.security.oauth2.client.registration.github.client-id=03...
spring.security.oauth2.client.registration.github.client-secret=c5d...
```

Как и в случае с любым другим провайдером, вам сначала нужно зарегистрировать свое приложение, настроив идентификатор клиента и секрет в файле `application.properties`. Подход к регистрации приложения зависит от провайдера. Для GitHub по адресу <http://mng.bz/p1YG> доступна документация, в которой рассказывается, как зарегистрировать приложение.

Прежде чем попросить вас пройти аутентификацию, приложение предлагает вам два варианта входа, которые мы настроили ранее (рис. 16.4). Для входа вы должны выбрать Google или GitHub. Затем приложение перенаправит вас на страницу аутентификации выбранного провайдера аутентификации.

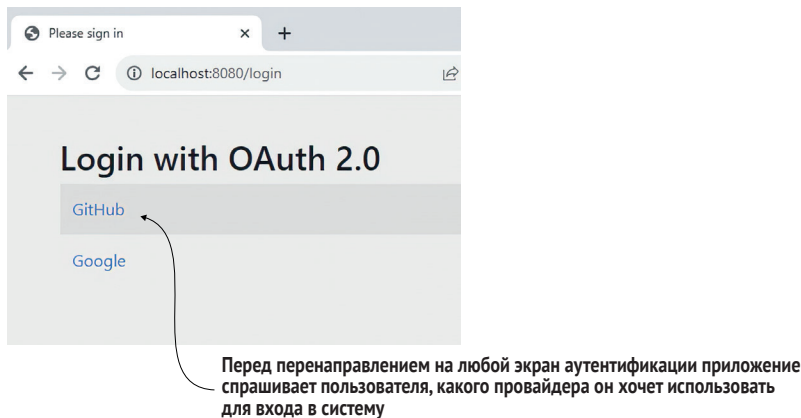


Рис. 16.4 При аутентификации приложение позволяет пользователю выбирать между GitHub и Google

16.1.3 Собственный сервер авторизации

Spring Security поддерживает список из четырех популярных провайдеров аутентификации, о чем говорилось в разделах 16.1.1 и 16.1.2. Но что, если вы хотите использовать провайдера, которого нет в списке «из коробки»? У вас есть много других альтернатив, таких как LinkedIn, Twitter, Yahoo и др. Вы можете использовать созданный вами сервер авторизации, о котором узнали в главе 14.

У нас есть возможность настроить вход OAuth 2 на работу с любым провайдером, включая ваш собственный сервер. В этом разделе мы воспользуемся сервером авторизации, который создали в главе 14, чтобы продемонстрировать пользовательскую конфигурацию входа OAuth 2. Чтобы разделить примеры, я скопировал содержимое проекта `ssiach14-ex1`, который мы обсуждали в главе 14, в проект для этой главы, который я назвал `ssia-ch16-ex1-as`.

Нам нужно только убедиться, что конфигурация нашего клиента соответствует целям этой главы. В листинге 16.2 показан код зарегистрированного клиента, настроенного на подключение к нашему серверу авторизации. Самое важное здесь – убедиться, что URI перенаправления указан правильно и соответствует приложению, для которого мы создаем механизм входа:

```
http://localhost:8080/login/oauth2/code/my_authorization_server
```

На рис. 16.5 проанализирована анатомия перенаправления. Обратите внимание, что стандартный URI перенаправления использует путь `/login/oauth2/code`, за которым следует имя сервера авторизации. В этом примере я дал серверу авторизации имя `my_authorization_server`.

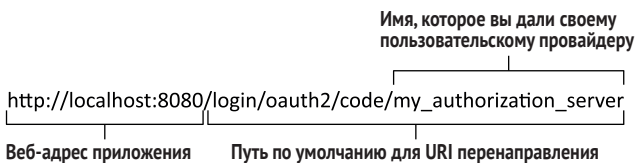


Рис. 16.5 Стандартный формат URI перенаправления. Последняя часть его пути – это имя провайдера

В листинге 16.3 показана часть конфигурации с сервера авторизации, который регистрирует данные клиента. Эти данные вам понадобятся позже в этом разделе; мы также настроим их на стороне приложения.

Листинг 16.3 Данные клиента, зарегистрированные на стороне сервера авторизации

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
    var registeredClient = RegisteredClient
        .withId(UUID.randomUUID().toString())
        .clientId("client")
        .clientSecret("secret")
        .clientAuthenticationMethod(
            ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .authorizationGrantType(
            AuthorizationGrantType.AUTHORIZATION_CODE)
        .redirectUri(
            "http://localhost:8080/login/oauth2/code/my_authorization_server")
        .scope(OidcScopes.OPENID)
        .build();

    return new InMemoryRegisteredClientRepository(registeredClient);
}
```

Напомню, что вы не можете запустить два приложения с одним и тем же номером порта в одной системе. Поскольку веб-приложение использует порт 8080, мы должны изменить порт сервера авторизации на другой. Как показано в следующем фрагменте кода, для этого примера я выбрал порт 7070 и настроил его в файле `application.properties`:

```
server.port=7070
```

Теперь мы можем перейти к настройке веб-приложения. Поскольку в наших примерах из разделов 16.1.1 и 16.1.2 мы использовали популярного провайдера, нам не нужно было его определять. Spring Security уже имеет все необходимые ему сведения о таких провайдерах. Но для использования другого провайдера потребуются дополнительные настройки. Spring Security необходимо знать следующее (как говорилось в главах 13 и 14):

- конечная точка авторизации, чтобы провайдер знал, куда перенаправить пользователя в процессе применения кода авторизации;
- конечная точка токена, которую приложение должно вызвать для получения токена доступа;
- конечная точка набора ключей, необходимая приложению для проверки токенов доступа.

Хорошая новость заключается в том, что если ваш провайдер (сервер авторизации) точно следует протоколу OpenID Connect, вам достаточно настроить URI эмитента. Затем приложение использует

URI эмитента, чтобы получить все необходимые ему данные, такие как авторизация, токен и URI набора ключей. Если сервер авторизации не соблюдает протокол OpenID Connect, вам придется явно задать эти три параметра в файле `application.properties`.

Поскольку серверы авторизации, которые мы создали в главе 14, корректно реализуют протокол OpenID Connect, мы можем положиться на URI эмитента. В следующем фрагменте кода показано, как настроить URI эмитента. Обратите внимание, что я дал имя провайдеру. Для этого примера я решил идентифицировать его с именем `my_authorization_server`, но вы можете выбрать любое имя для идентификации вашего провайдера:

```
spring.security.oauth2.client.provider.my_authorization_server.  
issueruri=http://127.0.0.1:7070
```

ПРИМЕЧАНИЕ Мы запускаем оба приложения, которые должны работать в локальной системе, – сервер авторизации и веб-приложение. Запуск этих приложений в одной системе и доступ к ним из браузера могут вызвать проблемы с файлами cookie, которые браузер использует для хранения сеанса пользователя. По этой причине я рекомендую вам использовать IP-адрес "127.0.0.1" для ссылки на одно приложение и DNS-имя "localhost" для ссылки на другое. Даже если они идентичны с точки зрения сети и ссылаются на одну и ту же систему (локальную систему), браузер будет считать их разными, что позволит ему правильно управлять сеансами. В этом примере я использую "127.0.0.1" для обозначения сервера авторизации и "localhost" для веб-приложения.

В листинге 16.4 показана конфигурация регистрации клиента. Помимо объявления поставщика, регистрация клиента также немного длиннее той, что мы написали в разделах 16.1.1 и 16.1.2, где мы использовали популярных поставщиков по умолчанию. Помимо идентификатора и секрета клиента, вам также необходимо заполнить следующие поля:

- *имя провайдера* (provider name) – имя, которое вы даете выбранному провайдеру, если оно не является общепринятым;
- *способ аутентификации клиента* (client authentication meeting) – метод аутентификации приложения для вызова защищенных конечных точек поставщика (обычно HTTP Basic);
- *URI перенаправления* (redirect URI) – адрес, на который провайдер перенаправит пользователя после правильной аутентификации. Этот URI должен соответствовать одному из зарегистрированных на стороне сервера авторизации (см. листинг 16.3);
- *запрошенная область веб-приложения* (web app's requested scope) – область, которую запрашивает веб-приложение, может

быть только одной из зарегистрированных на стороне сервера авторизации (см. листинг 16.3).

Листинг 16.4 Конфигурация регистрации клиента

```

spring.security.oauth2.client.
➔registration.my_authorization_server
.client-id=client ❶
spring.security.oauth2.client.
➔registration.my_authorization_server.
➔client-name=Custom ❷
Spring.security.oauth2.client.
➔registration.my_authorization_server.
➔client-secret=secret ❸
spring.security.oauth2.client.
➔registration.my_authorization_server.
➔provider=my_authorization_server ❹
spring.security.oauth2.client.
➔registration.my_authorization_server.
➔client-authentication-method=client_secret_basic ❺
spring.security.oauth2.client.
➔registration.my_authorization_server.redirect-uri=
➔http://localhost:8080/login/oauth2/code/my_authorization_server ❻
spring.security.oauth2.client.
➔registration.my_authorization_server.scope[0]=openid ❼

```

- ❶ Идентификатор клиента, зарегистрированный на стороне сервера авторизации.
- ❷ Отображаемое имя клиента.
- ❸ Секрет клиента, зарегистрированный на стороне сервера авторизации.
- ❹ Имя собственного провайдера.
- ❺ Метод аутентификации приложения для вызова защищенных конечных точек провайдера.
- ❻ URI, на который провайдер перенаправляет пользователя после успешной аутентификации.
- ❼ Область, запрашиваемая приложением.

Теперь вы можете запустить сервер авторизации и веб-приложение. Помните, что сначала необходимо запустить сервер авторизации. При запуске веб-приложения оно вызовет стартовый URI, чтобы получить остальные необходимые ему сведения. После запуска обоих приложений откройте веб-приложение в браузере, используя его адрес `http://localhost:8080`. На рис. 16.6 показано, что собственный провайдер теперь отображается в списке под именем `Custom` и может быть выбран пользователем для аутентификации.

16.1.4 Добавление гибкости в конфигурации

Часто нам нужно иметь больше гибкости, чем позволяют файлы свойств. Иногда нам нужно иметь возможность динамически изме-

нять учетные данные без повторного развертывания приложения. В других случаях мы хотим включать или выключать определенных провайдеров или даже предоставлять доступ к ним на основе определенной логики. В таких случаях простое добавление учетных данных в файл свойств и ожидание волшебства от Spring Boot больше не работает.

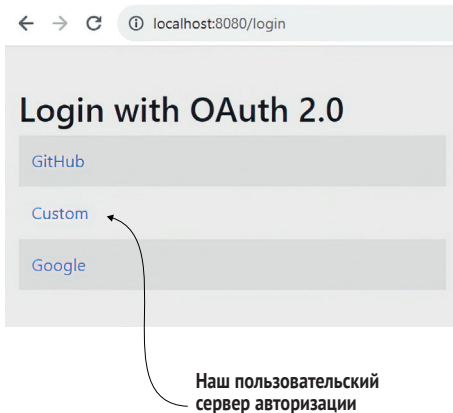


Рис. 16.6 Собственный сервер авторизации теперь отображается в списке провайдеров, которые пользователь может выбрать для аутентификации

Однако если вы знаете общие принципы авторизации, вы можете настроить данные провайдера по своему усмотрению. Единственные два типа, которые вы должны запомнить, – это

- `ClientRegistration` – с помощью этого объекта задают параметры, необходимые клиенту для использования сервера авторизации (учетные данные, URI перенаправления, URI авторизации и т. д.);
- `ClientRegistrationRepository` – этот контракт применяется для определения логики, которая извлекает клиентские регистрации. Например, вы можете создать репозиторий клиентских регистраций и сообщить своему приложению, что нужно получать клиентские регистрации из базы данных или хранилища.

В этом примере я все упрощаю. Я продолжу использовать файл `application.properties`, но с другими именами для свойств, чтобы доказать, что Spring Boot больше не выполняет настройку за нас. Но даже этот простой пример демонстрирует тот же подход, который вы бы использовали, если бы хотели сохранить детали в базе данных или получить их, вызвав заданную конечную точку. В любом подобном случае вы должны соответствующим образом реализовать контракт `ClientRegistrationRepository`.

Вы определяете компонент `ClientRegistrationRepository` как компонент Spring. Приложение будет использовать ваше хранилище для получения данных регистрации клиента. В листинге 16.5 показан пример кода, в котором я использовал хранение в памяти.

В этом примере я делаю три вещи:

- 1 ввожу значения учетных данных из файла свойств;
- 2 создаю объект `ClientRegistration` со всеми необходимыми данными;
- 3 настраиваю его в реализации `ClientRegistrationRepository` для хранения в памяти.

Этот пример можно найти в проекте `ssia-ch16-ex2`.

Листинг 16.5 Реализация собственной логики

```
@Configuration
public class SecurityConfig {

    @Value("${client-id}")
    private String clientId;

    @Value("${client-secret}")
    private String clientSecret;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2Login(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().authenticated()
        );

        return http.build();
    }

    @Bean
    public ClientRegistrationRepository clientRegistrationRepository() {
        return new InMemoryClientRegistrationRepository(
            this.googleClientRegistration());
    }

    private ClientRegistration googleClientRegistration() {
        return CommonOAuth2Provider.GOOGLE.getBuilder("google")
            .clientId(clientId)
            .clientSecret(clientSecret)
            .build();
    }
}
```

- ❶ Ввод значений учетных данных из файла свойств.
- ❷ Организация репозитория в памяти, который содержит данные регистрации клиента.
- ❸ Создание регистрации клиента на основе шаблона общеизвестного провайдера Google.

16.1.5 Управление авторизацией в OAuth 2

В этом разделе мы обсудим использование данных аутентификации. В большинстве случаев вашему приложению необходимо знать, кто вошел в систему. От этого зависит либо выбор варианта отображения, либо применение различных ограничений авторизации. К счастью, использование метода аутентификации `oauth2Login()` не отличается в этом отношении от любого другого метода аутентификации.

Помните порядок аутентификации Spring Security, который мы обсуждали начиная с главы 2 (показан на рис. 16.7)? Успешная аутентификация всегда заканчивается тем, что приложение добавляет данные аутентификации в контекст безопасности. Использование `oauth2Login()` не является исключением.

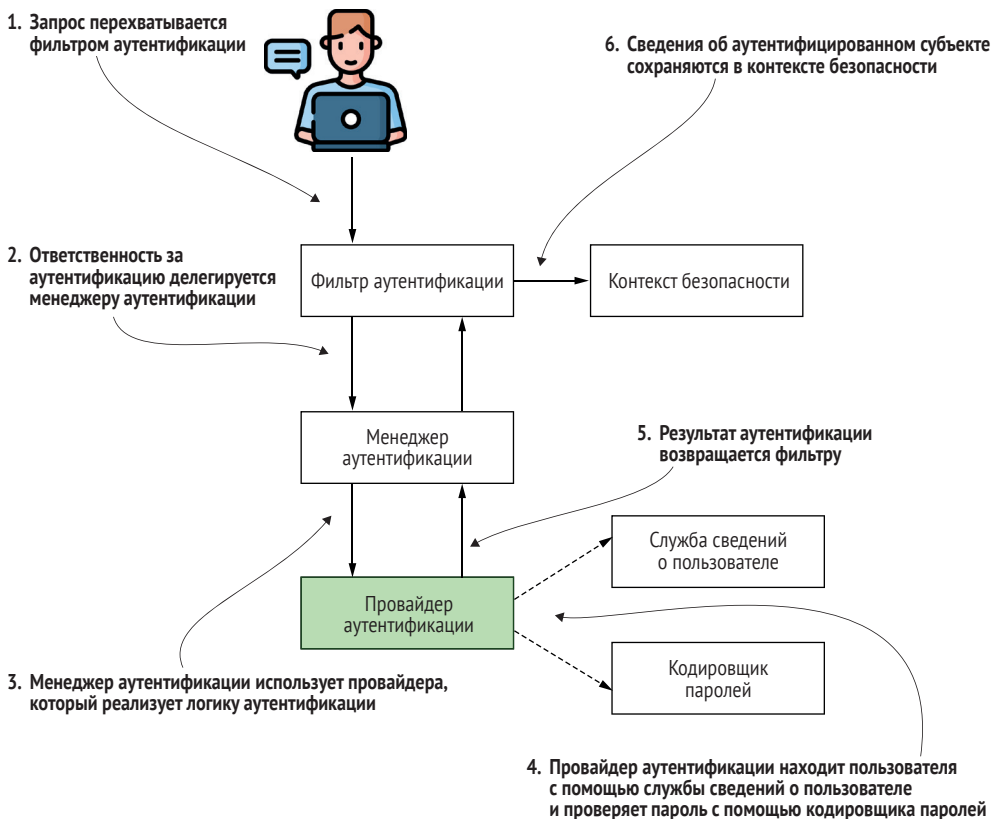


Рис. 16.7 Процесс аутентификации в Spring Security. Успешная аутентификация заканчивается тем, что приложение добавляет данные аутентифицированного субъекта в контекст безопасности

Зная, что данные аутентификации находятся в контексте безопасности, вы можете применять их точно так же, как и для любого дру-

того ранее обсуждавшегося метода аутентификации – `httpBasic()`, `formLogin()` или `oauth2ResourceServer()`:

- внедрить объект `Authentication` в качестве параметра метода (рис. 16.5);
- получить его из контекста безопасности в любом месте приложения (`SecurityContextHolder.getContext().getAuthentication()`);
- использовать предварительные и последующие аннотации, как говорилось в главах 11 и 12.

Для получения стандартных данных пользователя, таких как имя пользователя и полномочия, можно воспользоваться контрактом `Authentication`. Если вам нужны специальные данные, вы можете использовать прямую реализацию контракта, как показано в листинге 16.6. В случае OAuth 2 реализацию контракта определяет класс `OAuth2AuthenticationPrincipal`. Но в целях удобства обслуживания я рекомендую вам использовать контракт `Authentication` везде, где это возможно, и полагаться на собственную реализацию только в том случае, если у вас нет другого выбора (например, если вам нужно получить данные, которые вы уже не можете получить с помощью ссылки на контракт).

Листинг 16.6 Получение данных аутентификации

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String home(
        [CA]OAuth2AuthenticationToken authentication) { ❶
        // Здесь что-нибудь делаем с аутентификацией
        return "index.html";
    }
}
```

❶ Вставка данных аутентификации в параметр метода.

16.2 Реализация клиента OAuth 2

В этом разделе обсуждается реализация службы как клиента OAuth 2. В системах, основанных на службах, приложения часто взаимодействуют друг с другом. В таких случаях приложение, отправляющее запрос другому приложению, становится клиентом этого конкретного приложения. В большинстве случаев, если мы решаем реализовать аутентификацию для запросов через OAuth 2, приложение использует для получения токена доступа грант в форме учетных данных клиента.

Подход с предоставлением учетных данных клиента не подразумевает пользователя. По этой причине вам не понадобятся URI перенаправления и URI авторизации. Учетных данных клиента достаточно, чтобы позволить клиенту пройти аутентификацию и получить токен доступа, отправив запрос на URI токена. Рисунок 16.8 напоминает вам о гранте с предоставлением учетных данных клиента, который мы обсуждали в главе 13.



Рис. 16.8 Тип гранта – учетные данные клиента. Клиент отправляет запрос в конечную точку токена, используя свои учетные данные для аутентификации. После успешной аутентификации клиент получает токен доступа, который он может использовать для доступа к ресурсам на стороне сервера ресурсов

Рассмотрим простой пример, который иллюстрирует все, что вам нужно знать о реализации функционала клиента OAuth 2 с помощью Spring Security. Мы создадим приложение, которое использует учетные данные клиента для получения токена доступа с сервера авторизации. Чтобы упростить пример, в нем показано только получение токена доступа. Для этого примера не имеет значения, как вы создаете запрос. Если вы знаете, как получить токен доступа, вы можете отправить HTTP-запрос любым способом, поскольку любая технология позволяет вам легко добавить значение заголовка запроса (помните, что вы добавляете значение токена доступа к заголовку запроса Authorization с префиксом в виде строки "Bearer").

Итак, в этом примере мы настроим приложение для получения токена доступа с сервера авторизации OAuth 2 с помощью учетных данных клиента. Чтобы убедиться, что мы правильно получили токен доступа, мы вернем его в теле ответа демонстрационной конечной точки. На рис. 16.9 схематически показаны следующие шаги:

- 1 Пользователь (вы) вызывает демонстрационную конечную точку, которую мы назвали /token, с помощью cURL (или альтернативного инструмента, такого как Postman).
- 2 Инструмент (cURL), который имитирует приложение, отправляет запрос в приложение, которое мы создаем для этого примера.
- 3 Наше приложение предоставляет учетные данные клиента для получения токена доступа с сервера авторизации.
- 4 Приложение возвращает значение токена доступа клиенту в теле ответа HTTP.
- 5 Пользователь (вы) извлекает значение токена доступа из тела ответа HTTP.

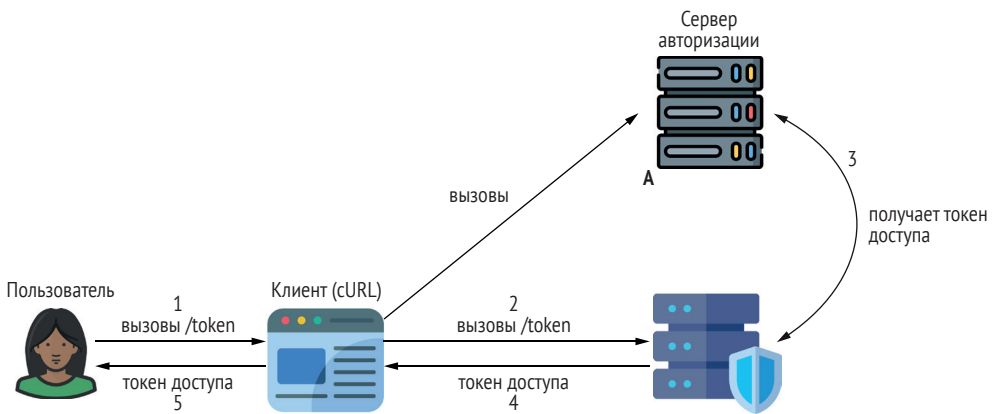


Рис. 16.9 В этом примере мы создаем приложение, способное извлекать токен доступа с сервера авторизации через предоставление учетных данных клиента. Чтобы убедиться, что приложение правильно извлекло токен доступа, приложение отправляет значение токена в ответ на вызов демонстрационной конечной точки. Я назвал эту демонстрационную конечную точку /token

Мы будем использовать тот же сервер авторизации, который вы создали в главе 14. Для этой главы его код размещен в проекте `ssia-ch16-ex1-as`. Не забудьте сначала добавить на сервер авторизации регистрацию клиента, позволяющую использовать его учетные данные. Вы можете изменить регистрацию, которую настроили ранее в главе 14 (как показано в листинге 16.7), или добавить вторую регистрацию клиента, которая соответствует этому требованию.

Листинг 16.7 Данные клиента, зарегистрированные на стороне сервера авторизации

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
    var registeredClient = RegisteredClient
```

```

        .withId(UUID.randomUUID().toString())
        .clientId("client")
        .clientSecret("secret")
        .clientAuthenticationMethod(
            ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .authorizationGrantType(
            AuthorizationGrantType.CLIENT_CREDENTIALS)
        .scope(OidcScopes.OPENID)
        .build();

        return new InMemoryRegisteredClientRepository(registeredClient);
    }

```

- ❶ Добавление регистрации клиента, которая позволяет использовать его учетные данные.

Аналогично другим способам аутентификации, Spring Security обеспечивает метод объекта `HttpSecurity` для настройки приложения как клиента OAuth 2. Вызовите метод `oauth2Client()`, как показано в листинге 16.8, чтобы настроить приложение как клиента OAuth 2.

Листинг 16.8 Настройка аутентификации клиента OAuth 2

```

@Configuration
public class ProjectConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.oauth2Client(Customizer.withDefaults());

        http.authorizeHttpRequests(
            c -> c.anyRequest().permitAll()
        );

        return http.build();
    }
}

```

- ❶ Вызов метода аутентификации `oauth2Client()` делает это приложение клиентом OAuth 2.

Приложению также необходимо знать определенные данные для отправки серверу авторизации запросов на токены доступа. Как вы узнали в разделе 16.1, мы предоставляем эти данные с помощью компонента `ClientRegistrationRepository`. Код в листинге 16.9 может показаться вам знакомым, потому что он напоминает код в листинге 16.4.

Однако, поскольку я не использую общеизвестного провайдера по умолчанию, мне пришлось указать больше данных, таких как об-

ласть действия, URI токена и метод аутентификации. Обратите внимание, что я настроил учетные данные клиента как тип гранта.

Листинг 16.9 Конфигурирование данных регистрации клиента для клиентского приложения

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public ClientRegistrationRepository clientRegistrationRepository() {
        ClientRegistration c1 =
            ClientRegistration.withRegistrationId("1")
                .clientId("client")
                .clientSecret("secret")
                .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
                .clientAuthenticationMethod(
                    ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
                .tokenUri("http://localhost:7070/oauth2/token")
                .scope(OidcScopes.OPENID)
                .build();

        var repository =
            new InMemoryClientRegistrationRepository(c1);
        return repository;
    }
}
```

Компонент менеджера клиентов делает запрос на получение токена доступа. Рисунок 16.10 иллюстрирует связь между контроллером и менеджером клиентов (для нашего примера).

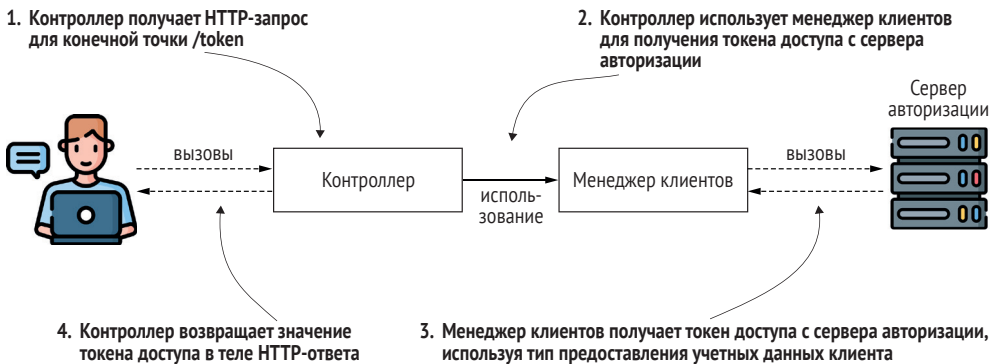


Рис. 16.10 Контроллер использует менеджера клиентов для получения токена доступа с сервера авторизации. Менеджер клиентов – это компонент Spring Security, отвечающий за подключение к серверу авторизации и правильное использование гранта для получения токена доступа

Класс `OAuth2AuthorizedClientManager` определяет менеджера клиентов. В листинге 16.10 мы настраиваем менеджера клиентов как компонент в контексте приложения.

Листинг 16.10 Реализация менеджера клиентов OAuth 2

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public OAuth2AuthorizedClientManager oAuth2AuthorizedClientManager(
        ClientRegistrationRepository clientRegistrationRepository,
        OAuth2AuthorizedClientRepository auth2AuthorizedClientRepository
    ) {

        var provider =
            OAuth2AuthorizedClientProviderBuilder.builder()
                .clientCredentials()
                .build();

        var cm = new DefaultOAuth2AuthorizedClientManager(
            clientRegistrationRepository,
            auth2AuthorizedClientRepository);

        cm.setAuthorizedClientProvider(provider);

        return cm;
    }
}
```

- ❶ Создаем объект провайдера для указания типов грантов, которые предполагается использовать.
- ❷ Создаем экземпляр менеджера клиентов, который будет обрабатывать логику клиентского запроса.
- ❸ Настройка провайдера для менеджера клиентов.

Теперь вы можете использовать менеджера клиентов везде, где вам нужно получить токен доступа. Как показано на рис. 16.10, я заставил контроллер использовать менеджер клиентов напрямую, чтобы упростить этот пример и помочь вам сосредоточиться на обсуждении реализации клиента OAuth 2. Реальное приложение, вероятно, будет более сложным. В проекте, который правильно разделяет ответственности объекта, менеджер клиентов, скорее всего, будет использоваться прокси-объектом, а не напрямую контроллером (рис. 16.11).

Листинг 16.11 демонстрирует добавление экземпляра менеджера клиентов и извлечение токена доступа с помощью конечной точки. При вызове конечной точки `/token`, которую предоставляет приложение, тело ответа должно содержать значение токена доступа.

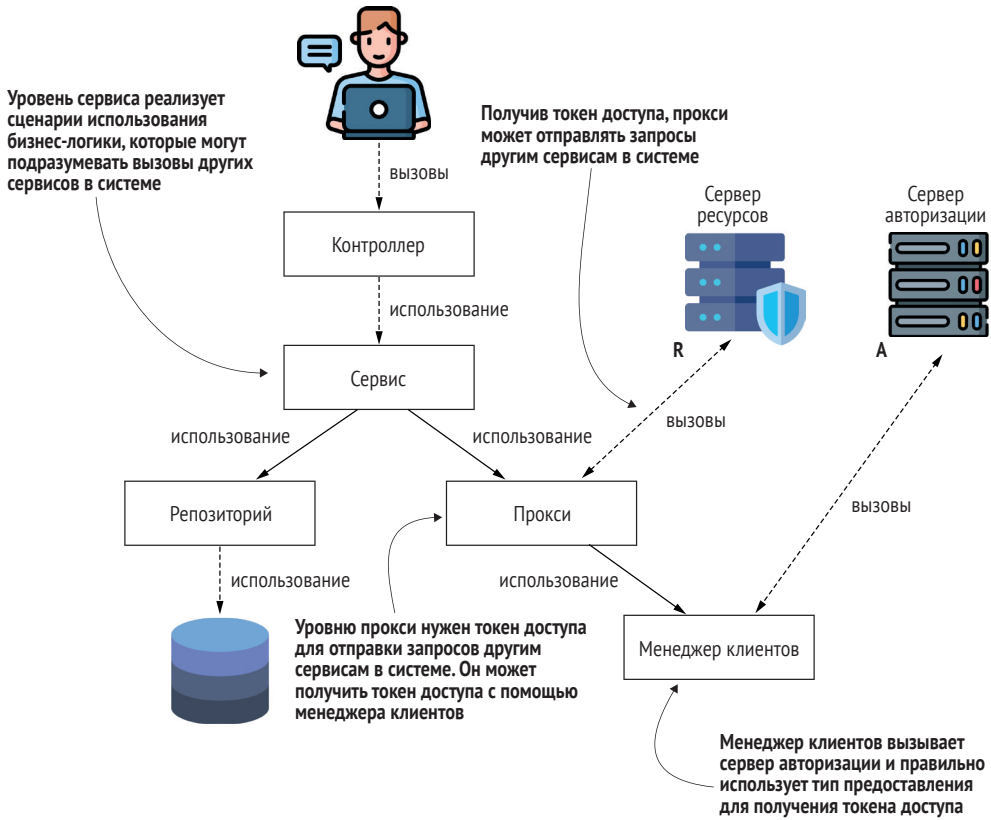


Рис. 16.11 Реальное приложение имело бы более четкое разделение ответственностей. В отличие от нашего примера, прокси-уровень использует токен, который он получает с помощью менеджера клиентов, для отправки запросов другому приложению в системе

Листинг 16.11 Использование менеджера клиентов OAuth 2 для получения токена

```
@RestController
public class DemoController {

    private final OAuth2AuthorizedClientManager clientManager;

    // Конструктор не показан

    @GetMapping("/token")
    public String token() {
        OAuth2AuthorizeRequest request = OAuth2AuthorizeRequest
            .withClientRegistrationId("1")
            .principal("client")
            .build();

        var client =
```

```

        clientManager.authorize(request);
    return client
        .getAccessToken().getTokenValue();
}
}

```

- ❶ Предоставление конечной точки GET по пути /token.
- ❷ Создание экземпляра запроса авторизации.
- ❸ Отправляя запрос, приложение возвращает значение токена доступа.
- ❹ Приложение возвращает значение токена доступа в теле ответа.

Используйте следующую команду cURL для вызова конечной точки, которую предоставляет приложение:

```
curl http://localhost:8080/token
```

Тело ответа должно содержать значение токена доступа, похожее на такое:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6ImJpbGwiLCJpYXQiOiE1MTYyMzkwMjJ9.zjL2JXw0TVgNgTMUKmP0-PTPkLULUVmV_5re50eZoHw
```

Заключение

- При реализации веб-приложения Spring нам часто приходится настраивать возможности аутентификации. Хотя мы можем быстро реализовать форму входа с помощью метода `formLogin()`, мы также можем разрешить пользователям проходить аутентификацию с помощью другой системы с зарегистрированной учетной записью.
- Наличие разных способов входа в систему полезно как для пользователя, так и для нашего приложения. Пользователям не нужно запоминать дополнительные учетные данные, а нашему приложению не нужно управлять учетными данными для всех своих пользователей.
- Spring Security по умолчанию рассматривает GitHub, Google, Facebook и Okta в качестве провайдеров аутентификации. Для этих провайдеров Spring Security уже владеет всеми данными для отправки запросов через фреймворк OAuth 2, поэтому вам нужно только задать клиентские учетные данные, предлагаемые поставщиком, чтобы сконфигурировать варианты входа в систему.
- Вы можете настроить свое приложение для использования других провайдеров, но вам придется в явном виде настроить все данные, необходимые приложению для формирования гранта, применяемого при запросе токенов доступа. Основные параметры, которые вам нужно задать, – это три URI: авторизации, токена и набора ключей.

- После того как пользователь входит в ваше приложение, даже если он аутентифицирован через внешнюю систему, приложение получает сведения о нем, сохраняя эти сведения в контексте безопасности. Этот процесс соответствует стандартной схеме аутентификации Spring Security. По данной причине вы можете настроить авторизацию аналогично всем другим методам аутентификации.
- Иногда бэкенд-служба становится клиентом для другого бэкенд-приложения. В таком случае приложению, которое хочет вызвать другое приложение и использовать подход OAuth 2, необходимо получить токен доступа для аутентификации. Приложение-клиент может использовать грант в форме учетных данных клиента для получения токена доступа.
- Spring Security предоставляет объект, называемый менеджером клиентов. Этот объект реализует логику для формирования гранта определенного типа и получения токена доступа. Прокси-уровень приложения, который отправляет запросы другому приложению и должен аутентифицировать запросы с помощью токенов доступа, будет использовать менеджер клиентов для получения токена доступа.

Часть V

Реактивный режим

Индустрия программного обеспечения всегда стремилась к более отзывчивым и эффективным приложениям, поэтому одной из наиболее важных парадигм стало реактивное программирование. В этой части книги вы изучите нюансы внедрения безопасности в реактивных приложениях с соблюдением баланса между отзывчивостью и надежной защитой.

В главе 17 обсуждается концепция реактивных приложений и выделяются их отличительные особенности. Глава проведет вас через тонкости управления пользователями в реактивной среде, подробно остановившись на настройке точных правил авторизации как на уровне конечной точки, так и через безопасность методов. Кроме того, вы получите представление о создании реактивного сервера ресурсов OAuth 2, объединяющего все лучшее из реактивного и безопасного миров.

После прочтения этой части вы будете готовы интегрировать механизмы безопасности в свои реактивные приложения, благодаря чему они останутся гибкими, не ставя под угрозу защиту пользователей.

17

Безопасность реактивных приложений

Темы этой главы:

- использование Spring Security с реактивными приложениями;
- реактивные приложения в системе, разработанной с использованием аутентификации OAuth 2.

Реактивность – это парадигма программирования, в которой мы применяем особый способ мышления при разработке наших приложений. Реактивное программирование – это мощный способ разработки веб-приложений, получивший широкое распространение в последнее десятилетие. Я бы даже сказал, что он стал модным несколько лет назад, когда на любой важной конференции было как минимум несколько докладов, посвященных реактивным приложениям. Однако, как и любую другую технологию в разработке программного обеспечения, реактивное программирование нельзя называть решением, применимым к любой ситуации.

В некоторых случаях реактивный подход очень полезен. В других случаях он может только усложнить вам жизнь. Но в целом реактивный подход существует, потому что он устраняет некоторые ограничения императивного программирования и применяется для обхода таких ограничений. Одно из них заключается в выполнении больших задач, которые могут быть фрагментированы. При императивном подходе вы даете приложению задачу для выполнения,

и приложение несет ответственность за ее решение. Если задача большая, то приложению может потребоваться значительное время для ее решения. Клиент, назначивший задачу, должен дожидаться полного решения задачи, прежде чем получить ответ. С помощью реактивного программирования вы можете разделить задачу так, чтобы приложение имело возможность одновременно приступить к нескольким подзадачам. В результате клиент быстрее получает обработанные данные.

В этой главе обсуждается безопасность на уровне приложения в реактивных приложениях с Spring Security. Как и в остальных случаях, безопасность является важным аспектом реактивных приложений. Но поскольку реактивные приложения разрабатываются по-другому, в Spring Security применяется адаптация функций, обсуждавшихся ранее в этой книге.

Мы начнем с краткого обзора реализации реактивных приложений с помощью фреймворка Spring в разделе 17.1. Затем применим функции безопасности, о которых вы узнали в этой книге, для защиты приложений. В разделе 17.2 мы обсудим управление пользователями в реактивных приложениях, а в разделе 17.3 мы продолжим применять правила авторизации. Наконец, в разделе 17.4 вы узнаете, как реализовать реактивные приложения в системе, разработанной на основе OAuth 2. Вы увидите, что меняется с точки зрения Spring Security, когда дело доходит до реактивных приложений, и, конечно же, вы научитесь применять новые знания на примерах.

17.1 Что такое реактивные приложения?

В этом разделе мы кратко обсудим реактивные приложения. Эта глава посвящена безопасности реактивных приложений, поэтому я хочу убедиться, что вы усвоили концепцию реактивных приложений, прежде чем углубляться в конфигурации Spring Security. Поскольку тема реактивных приложений обширна, я рассматриваю только основные аспекты реактивных приложений. Если вы еще не знаете, как работают реактивные приложения, или хотите узнать об этом более подробно, я рекомендую вам прочитать третью часть книги Крейга Уоллса «Spring в действии. Шестое издание» (ДМК, 2022).

При разработке приложений мы используем два способа достижения нужной функциональности. Эти два подхода заключаются в следующем:

- *при императивном подходе приложение обрабатывает все ваши данные одновременно.* Например, клиентское приложение вызывает конечную точку, предоставленную сервером, и отправляет все данные, которые необходимо обработать, на бэкэнд.

Допустим, вы реализуете функционал, в котором пользователь загружает файлы. Если пользователь выбирает несколько файлов и все они принимаются бэкенд-приложением для одновременной обработки, вы имеете дело с императивным подходом;

- *при реактивном подходе приложение получает и обрабатывает данные фрагментами.* Не обязательно, чтобы данные были полностью доступны для обработки с самого начала. Бэкенд получает и обрабатывает данные по мере их поступления. Допустим, пользователь выбирает несколько файлов, а бэкенд должен загрузить и обработать их. Бэкенд не ждет получения всех файлов, чтобы приступить к обработке. Он может получать файлы по одному и обрабатывать их, пока ожидает поступления остальных файлов.

На рис. 17.1 представлена аналогия двух подходов программирования. Представьте себе фабрику, разливающую молоко. Если фабрика получает все молоко утром и доставляет его после завершения розлива, мы говорим, что это нереактивный (императивный) подход. Если фабрика получает молоко в течение дня и доставляет заказы после завершения розлива достаточного количества молока – это реактивный подход. Очевидно, что для молочной фабрики выгоднее использовать реактивный подход, чем нереактивный.

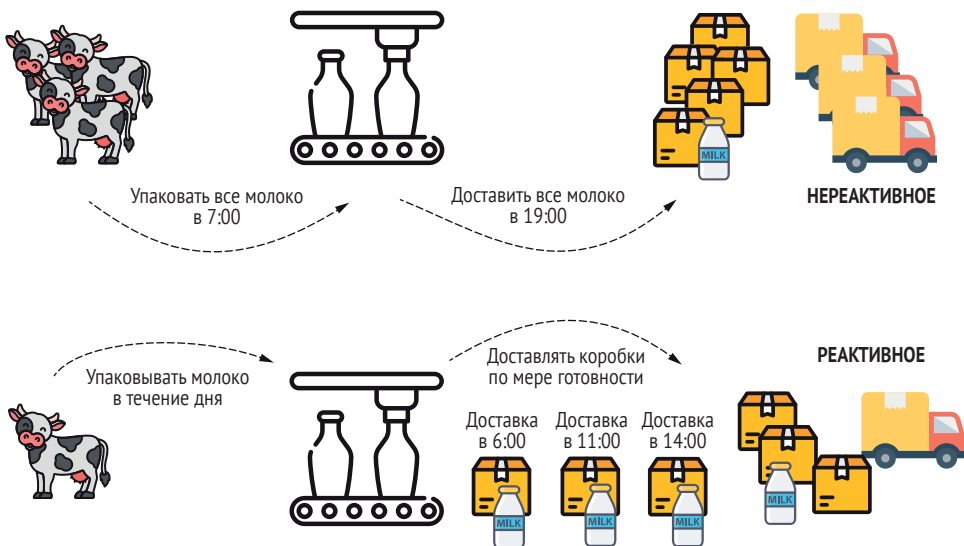


Рис. 17.1 Сравнение нереактивного подхода с реактивным. При нереактивном подходе молочная фабрика получает все молоко для розлива утром и развозит по заказчикам вечером. При реактивном подходе молоко упаковывают и передают в доставку по мере поступления. В данном случае реактивный подход лучше, так как он позволяет собирать молоко в течение дня и быстрее доставлять его клиентам

Спецификация Reactive Streams (<http://www.reactive-streams.org/>) описывает стандартный способ асинхронной обработки потоков для реализации реактивных приложений. Одной из реализаций этой спецификации является Project Reactor, который закладывает основы модели реактивного программирования Spring. Project Reactor предоставляет функциональный API для составления Reactive Streams.

Чтобы получить более прикладное представление о технологии, давайте начнем с создания простого реактивного приложения. Мы продолжим работу с этим приложением в разделе 17.2 при обсуждении управления пользователями в реактивных приложениях. Я создал новый проект с именем `ssia-ch17-ex1`, и сейчас мы разработаем реактивное веб-приложение, которое предоставляет демонстрационную конечную точку. В файле `pom.xml` мы должны добавить реактивную веб-зависимость, как показано в следующем фрагменте кода. Эта зависимость подключает Project Reactor и позволяет нам использовать связанные с ним классы и интерфейсы в нашем проекте:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Далее мы определяем простой `HelloController` для хранения определения нашей демонстрационной конечной точки. В листинге 17.1 показано определение класса `HelloController`. В определении конечной точки я использовал `Mono` в качестве возвращаемого типа. `Mono` – одно из основных понятий, определенных реализацией Reactor. При работе с Reactor вы будете часто использовать `Mono` и `Flux`, оба из которых определяют *издателей* (источники данных). В спецификации Reactive Streams издатель описывается интерфейсом `Publisher`. Этот интерфейс описывает один из основных контрактов, используемых с Reactive Streams. Другой контракт – `Subscriber` (*подписчик*). Этот контракт описывает компонент, потребляющий данные.

При проектировании конечной точки, которая что-то возвращает, конечная точка становится издателем, поэтому она должна вернуть реализацию `Publisher`. Если используется Project Reactor, это будет `Mono` или `Flux`. `Mono` – издатель для одного значения, а `Flux` – издатель для нескольких значений. Рисунок 17.2 описывает эти компоненты и отношения между ними.

Для большей наглядности вернемся к аналогии с молочной фабрикой. Молочная фабрика – это реактивная реализация бэкенда, которая предоставляет конечную точку получения молока для обработки. Эта конечная точка производит что-то (молоко в бутылках),

поэтому ей необходимо вернуть Publisher. Если получатель запрашивает более одной бутылки молока, то молочная фабрика должна вернуть Flux, который является реализацией Publisher Project Reactor, работающей со значениями от нуля и более.

Интерфейс Publisher – это контракт, определенный спецификацией Reactive Streams для представления сущности, которая производит значения

Любой реактивный поток нуждается как минимум в подписчике. Интерфейс Subscriber спецификации Reactive Streams определяет контракт, который должен реализовать любой подписчик

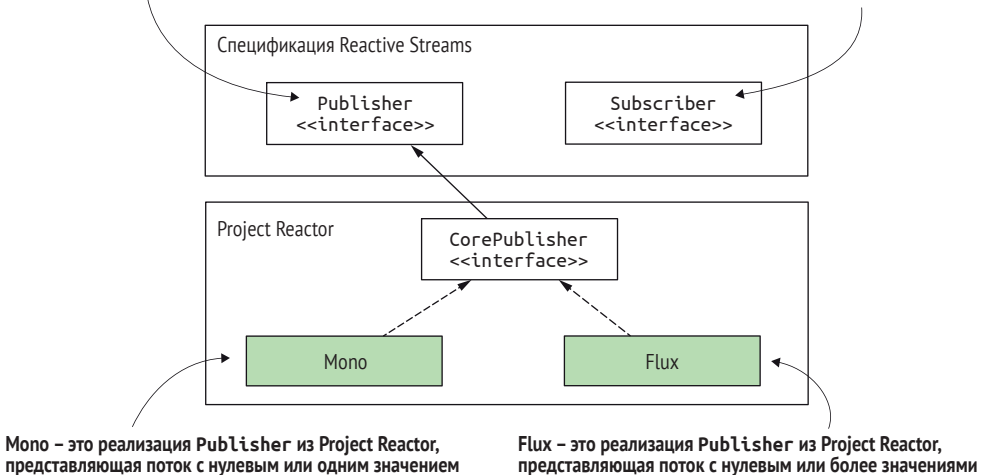


Рис. 17.2 В реактивном потоке издатель производит значения, а подписчик их потребляет. Контракты, определенные спецификацией Reactive Streams, описывают издателей и подписчиков. Project Reactor следует спецификации Reactive Streams и реализует контракты Publisher и Subscriber. На рисунке компоненты, применяемые в примерах этой главы, закрашены

Листинг 17.1 Определение класса HelloController

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public Mono<String> hello() {
        return Mono.just("Hello!");
    }
}
```

❶ Создаем и возвращаем источник потока Mono с одним значением в потоке.

Теперь вы можете запустить и протестировать приложение. Первое, что вы заметите, взглянув на терминал приложения, – Spring Boot больше не настраивает сервер Tomcat. Spring Boot по умолчанию настраивал Tomcat для веб-приложения, и вы могли заметить этот факт в любом из примеров, представленных ранее в этой книге.

Вместо этого Spring Boot теперь автоматически настраивает Netty как реактивный веб-сервер по умолчанию для проекта.

Второе, что вы могли заметить при вызове конечной точки, – ее поведение внешне не отличается от поведения конечной точки, разработанной с нереактивным подходом. Вы по-прежнему можете найти в теле ответа HTTP сообщение Hello!, которое конечная точка возвращает в своем определенном потоке Mono. Ниже показано поведение приложения при вызове конечной точки:

```
curl http://localhost:8080/hello
```

Тело ответа:

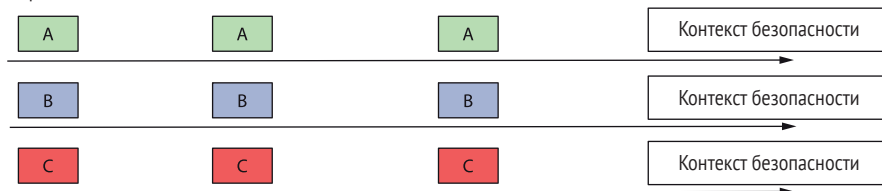
Hello!

Но чем реактивный подход отличается с точки зрения Spring Security? За кадром реактивная реализация использует несколько потоков для выполнения задач. Другими словами, она отказывается от философии «один поток на запрос», которую мы используем для веб-приложения, разработанного в соответствии с императивным подходом (рис. 17.3). Отсюда вытекают дополнительные различия:

- реализация SecurityContext по-другому работает в реактивных приложениях. Помните, что SecurityContext основан на Thread-Local, но теперь у нас есть более одного потока на запрос;

В нереактивном веб-приложении для каждого запроса выделяется поток. Мы всегда знаем, что в одном потоке мы всегда работаем с задачами для одного и того же запроса. Поэтому приложение управляет контекстом безопасности для каждого потока

Нереактивное



В реактивном приложении задача из запроса может управляться несколькими разными потоками. Таким образом, управление контекстом безопасности для каждого потока больше не подходит

Реактивное

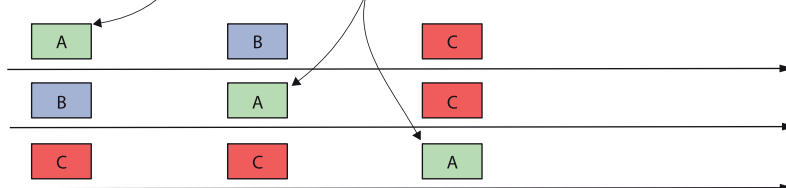


Рис. 17.3 Здесь каждая стрелка представляет временную шкалу отдельного потока, а прямоугольники – обработанные задачи из запросов A, B и C. Поскольку в реактивном приложении задачи из одного запроса могут обрабатываться в нескольких потоках, данные аутентификации больше не могут храниться на уровне потока

- из-за `SecurityContext` теперь меняется любая конфигурация авторизации. В главе 5 было сказано, что правила авторизации обычно полагаются на экземпляр `Authentication`, хранящийся в `SecurityContext`. Теперь затрагиваются конфигурации безопасности, применяемые на уровне конечной точки, а также глобальная функциональность безопасности метода;
- компонент `UserDetailsService`, отвечающий за получение данных пользователя, является источником данных. Из-за этого служба данных пользователя также должна поддерживать реактивный подход. (Мы узнали об этом контракте в главе 2.)

Рисунок 17.4 предлагает другой взгляд на этот подход. Представьте себе команду людей, работающих над набором задач. Каждый исполнитель может взять задачу в работу и оставить ее, когда она заблокирована. Продолжить эту задачу может уже другой поток. Поэтому контекст безопасности больше не может быть адресован потоку, но должен быть каким-то образом связан с задачей.



Рис. 17.4 Принцип работы реактивного приложения. Поток не принимает задачи запроса по порядку и не ждет, когда он будет заблокирован. Вместо этого все задачи из всех запросов находятся в очереди. Любой доступный поток может работать над задачами из любого запроса. Таким образом, независимые задачи могут выполняться параллельно, и потоки не простаивают

К счастью, Spring Security предлагает поддержку реактивных приложений и охватывает все случаи, когда вы больше не можете использовать подходы императивного программирования. В этой главе мы продолжим обсуждение способа реализации конфигураций безопасности с помощью Spring Security, но для реактивных приложений. Мы начнем с управления пользователями в разделе 17.2 и перейдем к применению правил авторизации конечных точек в разделе 17.3, где вы узнаете, как контекст безопасности работает в реактивных приложениях. Затем мы продолжим обсуждение безопасности реактивных методов, которая заменяет глобальную безопасность методов императивных приложений.

17.2 Управление пользователями в реактивных приложениях

Часто в приложениях аутентификация пользователя основана на паре учетных данных – имени пользователя и пароле. Этот подход является базовым, и мы обсуждали его, начиная с самого простого приложения, которое разработали в главе 2. Но в реактивных приложениях принцип работы компонента, отвечающего за управление пользователями, также меняется. В этом разделе мы обсуждаем реализацию управления пользователями в реактивном приложении.

Мы продолжаем работу над приложением в проекте `ssia-ch17-ex1`, которую мы начали в разделе 17.1, и добавляем `ReactiveUserDetailsService` в контекст приложения. Мы хотим убедиться, что конечная точка `/hello` может быть вызвана только аутентифицированным пользователем. Как следует из названия, контракт `ReactiveUserDetailsService` определяет службу сведений о пользователе для реактивного приложения.

Определение контракта такое же простое, как и для `UserDetailsService`. `ReactiveUserDetailsService` определяет метод, используемый Spring Security для извлечения пользователя по его имени. Разница в том, что метод, описанный `ReactiveUserDetailsService`, напрямую возвращает `Mono<UserDetails>`, а не `UserDetails`, как для `UserDetailsService`. Следующий фрагмент кода содержит определение интерфейса `ReactiveUserDetailsService`:

```
public interface ReactiveUserDetailsService {  
    Mono<UserDetails> findByUsername(String username);  
}
```

Как и в случае с `UserService`, вы можете написать собственную реализацию `ReactiveUserService`, чтобы предоставить Spring Security способ получения данных пользователя. Дабы упростить этот пример, мы используем реализацию, предоставленную Spring Security. `MapReactiveUserService` хранит данные пользователя в памяти (так же, как и `InMemoryUserDetailsManager`, о котором вы узнали в главе 2). Мы изменяем файл `pom.xml` проекта `ssia-ch17-ex1` и добавляем зависимость Spring Security, как показано ниже:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Затем мы создаем класс конфигурации и добавляем `ReactiveUserService` и `PasswordEncoder` в контекст Spring Security. Я назвал класс конфигурации `ProjectConfig`. Определение этого класса можно найти в листинге 17.2. Используя `ReactiveUserService`, мы определяем одного пользователя с именем `john`, паролем `12345` и полномочиями, которые я назвал `read` (чтение). Как вы могли заметить, это похоже на работу с `UserService`. Главное отличие `ReactiveUserService` заключается в том, что метод возвращает реактивный объект `Publisher`, содержащий `UserDetails` вместо самого экземпляра `UserDetails`. Spring Security берет на себя остальную часть обязанностей по интеграции.

Листинг 17.2 Класс `ProjectConfig`

```
@Configuration
public class ProjectConfig {

    @Bean
    public ReactiveUserService userDetailsService() {
        var u = User.withUsername("john")
            .password("12345")
            .authorities("read")
            .build();

        var uds = new MapReactiveUserService(u);

        return uds;
    }

    @Bean
```

```
public PasswordEncoder passwordEncoder() {  
    return NoOpPasswordEncoder.getInstance();  
}  
}
```

- ❶ Добавляем `ReactiveUserDetailsService` в контекст Spring.
- ❷ Создаем нового пользователя с его именем, паролем и полномочиями.
- ❸ Создаем `MapReactiveUserDetailsService` для управления экземплярами `UserDetails`.
- ❹ Добавляем `PasswordEncoder` в контекст Spring.

Теперь при запуске и тестировании приложения вы заметите, что вызвать конечную точку удастся только при аутентификации с использованием соответствующих учетных данных. В нашем случае мы можем использовать только имя пользователя `john` с его паролем `12345`, так как это единственная запись пользователя, которую мы добавили. Следующий фрагмент кода показывает поведение приложения при вызове конечной точки с действительными учетными данными:

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

Hello!

На рис. 17.5 изображена архитектура, которую мы используем в этом приложении. За кулисами `AuthenticationWebFilter` перехватывает HTTP-запрос. Этот фильтр делегирует ответственность за аутентификацию менеджеру аутентификации. Менеджер аутентификации реализует контракт `ReactiveAuthenticationManager`. В отличие от нереактивных приложений, у нас нет поставщиков аутентификации. `ReactiveAuthenticationManager` напрямую реализует логику аутентификации.

Если вы хотите создать собственную логику аутентификации, реализуйте интерфейс `ReactiveAuthenticationManager`. Архитектура для реактивных приложений не сильно отличается от архитектуры для нереактивных приложений, которую мы уже обсуждали в этой книге. Как показано на рис. 17.5, если аутентификация подразумевает учетные данные пользователя, то мы используем `ReactiveUserDetailsService` для получения данных пользователя и `PasswordEncoder` для проверки пароля.

Более того, фреймворк знает, что нужно ввести экземпляр аутентификации, когда вы его запрашиваете. Мы запрашиваем данные аутентификации, добавляя `Mono<Authentication>` в качестве параметра к методу в классе контроллера. В листинге 17.3 представлены изменения, внесенные в класс контроллера. Существенное изменение заключается в использовании реактивных издателей. Обратите

внимание, что нам нужно использовать `Mono<Authentication>` вместо простой аутентификации, как мы делали в неактивных приложениях.

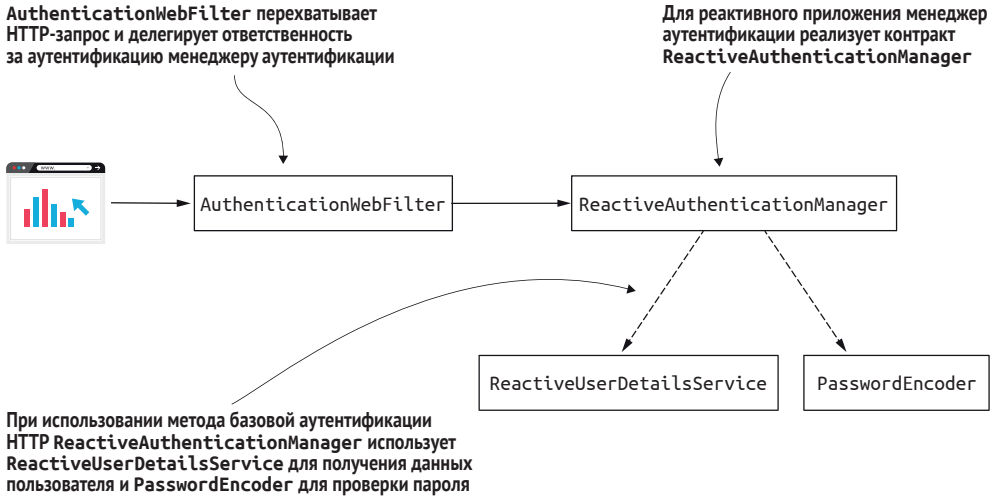


Рис. 17.5 AuthenticationWebFilter перехватывает запрос и делегирует ответственность за аутентификацию ReactiveAuthenticationManager. Если логика аутентификации подразумевает пользователей и пароли, ReactiveAuthenticationManager использует ReactiveUserDetailsService для поиска данных пользователя и PasswordEncoder для проверки пароля

Листинг 17.3 Класс HelloController

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public Mono<String> hello(
        Mono<Authentication> auth) { ❶

        Mono<String> message = ❷
            auth.map(a -> "Hello " + a.getName());

        return message;
    }
}
```

- ❶ Запрашиваем у фреймворка предоставление объекта аутентификации.
- ❷ Возвращаем имя субъекта в ответе.

Перезапустив приложение и вызвав конечную точку, вы обнаружите поведение, показанное ниже в запросе и ответе:

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

Hello john

И теперь вы, вероятно, задаетесь вопросом: откуда взялся объект `Authentication`? Поскольку это реактивное приложение, мы больше не можем позволить себе использовать `ThreadLocal`, потому что фреймворк предназначен для управления `SecurityContext`. Но Spring Security предлагает нам другую реализацию держателя контекста для реактивных приложений. Она называется `ReactiveSecurityContextHolder`. Мы используем ее для работы с `SecurityContext` в реактивном приложении. Таким образом, у нас все равно есть `SecurityContext`, но теперь он управляется по-другому. Рисунок 17.6 описывает завершение процесса аутентификации после того, как `ReactiveAuthenticationManager` успешно аутентифицирует запрос.

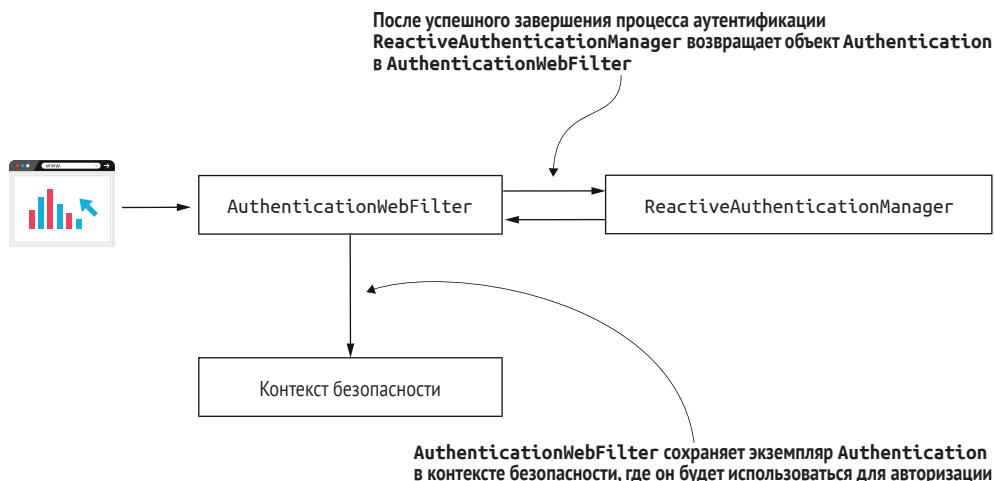


Рис. 17.6 После того как `ReactiveAuthenticationManager` успешно аутентифицирует запрос, он возвращает объект `Authentication` фильтру. Фильтр сохраняет экземпляр `Authentication` в `SecurityContext`

В листинге 17.4 показано, как переписать класс контроллера, если вы хотите получить данные аутентификации непосредственно из контекста безопасности. Этот подход является альтернативой решению фреймворку вводить их через параметр метода. Данное изменение представлено в проекте `ssia-ch17-ex2`.

Листинг 17.4 Работа с `ReactiveSecurityContextHolder`

```
@RestController
public class HelloController {
    @GetMapping("/hello")
```



```

public Mono<String> hello() {
    Mono<String> message =
        ReactiveSecurityContextHolder.getContext() ❶

        .map(ctx -> ctx.getAuthentication()) ❷

        .map(auth -> "Hello " + auth.getName()); ❸

    return message;
}
}

```

❶ Принимаем `Mono<SecurityContext>` из `ReactiveSecurityContextHolder`.

❷ Сопоставляем `SecurityContext` с объектом `Authentication`.

❸ Сопоставляем объект `Authentication` с возвращаемым сообщением.

Если вы перезапустите приложение и снова протестируете конечную точку, то увидите, что она ведет себя так же, как в предыдущих примерах этого раздела. Вот команда:

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

```
Hello john
```

Теперь вы знаете, что Spring Security предлагает реализацию для правильного управления `SecurityContext` в реактивной среде, и понимаете, как ваше приложение применяет правила авторизации. И эти новые знания открывают путь к настройке правил авторизации, которые мы обсудим в разделе 17.3.

17.3 Настройка правил авторизации в реактивных приложениях

В этом разделе мы обсудим настройку правил авторизации. Как вы уже знаете из предыдущих глав, авторизация следует за аутентификацией. В разделах 17.1 и 17.2 вы узнали, как Spring Security управляет пользователями и `SecurityContext` в реактивных приложениях. Но как только приложение завершает аутентификацию и сохраняет сведения об аутентифицированном запросе в `SecurityContext`, наступает время авторизации.

Как и для любого другого приложения, вам, вероятно, также потребуется настроить правила авторизации при разработке реактивных приложений. Чтобы вы научились задавать правила авторизации в реактивных приложениях, мы сначала рассмотрим в разделе 17.3.1 способ создания конфигураций на уровне конечной точки. Затем

в разделе 17.3.2 вы узнаете, как применить ее на любом другом уровне вашего приложения с помощью метода безопасности.

17.3.1 *Применение авторизации на уровне конечной точки в реактивных приложениях*

В этом разделе мы обсудим настройку авторизации на уровне конечной точки в реактивных приложениях. Это наиболее распространенный подход к настройке авторизации в веб-приложении. Вы уже могли это заметить, работая над предыдущими примерами. Настройка авторизации на уровне конечной точки имеет важное значение – ее используют почти в каждом приложении. Поэтому вам нужно знать, как применять ее и для реактивных приложений.

Из предыдущих глав вы узнали, как устанавливать правила авторизации, добавляя bean-компонент типа `SecurityFilterChain` в контекст приложения. Этот подход не работает в реактивных приложениях. Чтобы научиться правильно настраивать правила авторизации для уровня конечной точки в реактивных приложениях, мы начнем новый проект, который я назвал `ssia-ch17-ex3`.

В реактивных приложениях Spring Security использует контракт с именем `SecurityWebFilterChain` для применения конфигураций, которые мы использовали с помощью bean-компонента типа `SecurityFilterChain` в предыдущих главах. В реактивных приложениях мы добавляем в контекст Spring bean-компонент типа `SecurityWebFilterChain`. В качестве учебного примера мы создадим базовое приложение с двумя конечными точками, которые защищены независимо. В файле `pom.xml` нашего нового проекта `ssia-ch17-ex3` добавьте зависимости для реактивных веб-приложений и, конечно же, Spring Security:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Создайте класс контроллера для определения двух конечных точек, для которых мы настраиваем правила авторизации. Эти конечные точки доступны по путям `/hello` и `/ciao`. Чтобы вызвать конечную точку `/hello`, пользователю необходимо пройти аутентификацию, но вы можете вызвать конечную точку `/ciao` без аутентификации. В следующем листинге представлено определение контроллера.

Листинг 17.5 Класс `HelloController`, определяющий конечные точки для защиты

```

@RestController
public class HelloController {

    @GetMapping("/hello")
    public Mono<String> hello(Mono<Authentication> auth) {
        Mono<String> message = auth.map(a -> "Hello " + a.getName());
        return message;
    }

    @GetMapping("/ciao")
    public Mono<String> ciao() {
        return Mono.just("Ciao!");
    }
}

```

В классе конфигурации мы обязательно объявляем `ReactiveUserDetailsService` и `PasswordEncoder` для определения пользователя, как вы узнали в разделе 17.2. Код объявлений показан в листинге 17.6.

Листинг 17.6 Класс конфигурации, объявляющий компоненты для управления пользователями

```

@Configuration
public class ProjectConfig {

    @Bean
    public ReactiveUserDetailsService userDetailsService() {
        var u = User.withUsername("john")
            .password("12345")
            .authorities("read")
            .build();

        var uds = new MapReactiveUserDetailsService(u);

        return uds;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }

    // ...
}

```

В листинге 17.7 мы работаем с тем же классом конфигурации, который объявили в листинге 17.6, но опускаем объявление `ReactiveUserDetailsService` и `PasswordEncoder`, чтобы вы могли сосре-

доточиться на конфигурации авторизации, которую мы обсуждаем. В листинге 17.7 вы можете видеть, что мы добавляем в контекст Spring bean-компонент типа `SecurityWebFilterChain`. Метод получает в качестве параметра объект типа `ServerHttpSecurity`, который добавляет Spring. `ServerHttpSecurity` позволяет нам создать экземпляр `SecurityWebFilterChain`. `ServerHttpSecurity` предоставляет методы для настройки, аналогичные тем, которые вы использовали при настройке авторизации для неактивных приложений.

Листинг 17.7 Настройка авторизации конечной точки для реактивных приложений

```
@Configuration
public class ProjectConfig {

    // Omitted code

    @Bean
    public SecurityWebFilterChain securityWebFilterChain(
        ServerHttpSecurity http) {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeExchange(
            c -> c.pathMatchers(HttpMethod.GET, "/hello")
                .authenticated()
                .anyExchange()
                .permitAll()
        );

        return http.build();
    }
}
```

- ❶ Начинаем настройку авторизации конечной точки.
- ❷ Выбираем запросы, к которым мы применяем правила авторизации.
- ❸ Настраиваем выбранные запросы так, чтобы они были доступны только при аутентификации.
- ❹ Ссылается на любой другой запрос.
- ❺ Позволяет делать запросы без необходимости аутентификации.
- ❻ Создаем объект `SecurityWebFilterChain` для возврата.

Мы начинаем настройку авторизации с помощью метода `authorizeExchange()`. Мы вызываем этот метод аналогично тому, как вызываем метод `authorizeHttpRequests()` при настройке авторизации конечной точки для неактивных приложений. Затем мы продолжаем, используя метод `pathMatchers()`. Вы можете рассматривать этот метод как эквивалент использования `requestMatchers()` при настройке авторизации конечной точки для неактивных приложений.

Что касается нереактивных приложений, после использования метода для группировки запросов, к которым мы применяем правило авторизации, необходимо указать, что из себя представляет это правило. В нашем примере мы вызвали метод `authenticated()`, который утверждает, что принимаются только аутентифицированные запросы. Мы также использовали метод с именем `authenticated()` при настройке авторизации конечной точки для нереактивных приложений. Методы для реактивных приложений называются так же, чтобы сделать их более интуитивно понятными. Аналогично методу `authenticated()`, вы также можете вызывать следующие методы:

- `permitAll()` – настраивает приложение на разрешение запросов без аутентификации;
- `denyAll()` – отклоняет все запросы;
- `hasRole()` и `hasAnyRole()` – применяют правила на основе ролей;
- `hasAuthority()` и `hasAnyAuthority()` – применяют правила на основе полномочий.

Похоже, чего-то не хватает, не так ли? Есть ли у нас метод `access()`, который был предназначен для настройки правил авторизации в нереактивных приложениях? Да. Но он немного отличается, поэтому мы поработаем над отдельным примером, чтобы разобраться в различиях. Еще одно похожее название – метод `anyExchange()`, который берет на себя роль метода `anyRequest()` в нереактивных приложениях.

ПРИМЕЧАНИЕ Почему он называется `anyExchange()` и почему разработчики не сохранили имя метода `anyRequest()`? Почему `authorizeExchange()`, а не `authorizeHttpRequests()`? Это различие вытекает из терминологии, используемой в реактивных приложениях. Обычно мы называем коммуникацию между двумя компонентами реактивного типа *обмен данными* (*exchanging data*). Это название отражает специфику данных, отправляемых сегментированно в непрерывном потоке, а не как большой пакет в одном запросе.

Нам также нужно указать метод аутентификации, как мы это делаем для любой подобной конфигурации. Мы делаем это с тем же экземпляром `ServerHttpSecurity`, используя методы с тем же именем и тем же способом, которому вы научились для нереактивных приложений: `httpBasic()`, `formLogin()`, `csrf()`, `cors()`, – добавляя фильтры и настраивая цепочку фильтров и т. д. В конце мы вызываем метод `build()` для создания экземпляра `SecurityWebFilterChain`, который возвращаем, чтобы добавить в контекст Spring.

Ранее в этом разделе я говорил вам, что вы также можете использовать метод `access()` в конфигурации авторизации конечной точки реактивных приложений, как и для нереактивных приложений. Но как я сказал при обсуждении конфигурации нереактивных прило-

жений в главах 7 и 8, используйте метод `access()` только тогда, когда вы не можете применить свою конфигурацию иначе. Метод `access()` обеспечивает большую гибкость, но также делает конфигурацию вашего приложения более сложной для чтения. Всегда делайте выбор в пользу более простого решения.

Однако вы можете оказаться в ситуации, когда вам понадобится эта гибкость. Например, предположим, что вам нужно применить более сложное правило авторизации и использования `hasAuthority()` или `hasRole()` и сопутствующих методов недостаточно. Поэтому я также научу вас, как использовать метод `access()`. Я создал новый проект с именем `ssia-ch17-ex4` специально для этого примера. В листинге 17.8 вы можете увидеть, как я создал объект `SecurityWebFilterChain`, чтобы разрешить доступ к пути `/hello`, только если у пользователя есть роль администратора. Кроме того, доступ может быть разрешен лишь до полудня. Для всех остальных конечных точек я полностью закрываю доступ.

Листинг 17.8 Использование метода `access()` при реализации правил конфигурации

```
@Configuration
public class ProjectConfig {

    // Часть кода не показана

    @Bean
    public SecurityWebFilterChain
        securityWebFilterChain(ServerHttpSecurity http) {

        http.httpBasic(Customizer.withDefaults());

        http.authorizeExchange(
            c -> c.anyExchange()
                .access(this::getAuthorizationDecisionMono)
        );

        return http.build();
    }

    private Mono<AuthorizationDecision>
        getAuthorizationDecisionMono(
            Mono<Authentication> a,
            AuthorizationContext c) {

        String path = getRequestPath(c);

        boolean restrictedTime =
            LocalTime.now().isAfter(LocalTime.NOON);

        if(path.equals("/hello")) {
```

```

        return a.map(isAdmin())
            .map(auth -> auth && !restrictedTime)
            .map(AuthorizationDecision::new);
    }

    return Mono.just(new AuthorizationDecision(false));
}

// Часть кода не показана
}

```

- ❶ Для любого запроса применяем собственное правило авторизации.
- ❷ Метод, определяющий собственное правило авторизации, получает `Authentication` и контекст запроса в качестве параметров.
- ❸ Из контекста он получает путь запроса.
- ❹ Для пути `/hello` он применяет пользовательское правило авторизации.

Это может показаться сложным, но на самом деле все не так уж и сложно. Когда вы используете метод `access()`, вы предоставляете функцию, получающую все возможные сведения о запросе, которые являются объектом `Authentication` и `AuthorizationContext`. С помощью объекта `Authentication` вы получаете сведения об аутентифицированном пользователе: имя пользователя, роли или полномочия и другие пользовательские сведения в зависимости от того, как вы реализуете логику аутентификации. `AuthorizationContext` предоставляет информацию о запросе: путь, заголовки, параметры запроса, файлы cookie и т. д.

Функция, которую вы предоставляете в качестве параметра методу `access()`, должна возвращать объект типа `AuthorizationDecision`. Как вы догадались, `AuthorizationDecision` – это ответ, который сообщает приложению, разрешен ли запрос. Когда вы создаете экземпляр с помощью конструктора `new AuthorizationDecision(true)`, это означает, что вы разрешаете запрос. Если вы создаете его с помощью `new AuthorizationDecision(false)`, это означает, что вы запрещаете запрос. В листинге 17.9 вы найдете два метода, которые я пропустил в листинге 17.8 для вашего удобства: `getRequestPath()` и `isAdmin()`. Пропустив их, я помогаю вам сосредоточиться на логике, применяемой методом `access()`. Как видите, методы просты. Метод `isAdmin()` возвращает функцию, которая, в свою очередь, возвращает `true` для экземпляра `Authentication` с атрибутом `ROLE_ADMIN`. Метод `getRequestPath()` просто возвращает путь запроса.

Листинг 17.9 Определение методов `getRequestPath()` и `isAdmin()`

```

@Configuration
public class ProjectConfig {

    // Часть кода не показана

    private String getRequestPath(AuthorizationContext c) {

```

```
        return c.getExchange()
                .getRequest()
                .getPath()
                .toString();
    }

    private Function<Authentication, Boolean> isAdmin() {
        return p ->
            p.getAuthorities().stream()
                .anyMatch(e -> e.getAuthority().equals("ROLE_ADMIN"));
    }
}
```

Запуск приложения и вызов конечной точки либо приводит к статусу ответа 403 Forbidden, если какие-либо из примененных нами правил авторизации не выполнены, либо просто отображает сообщение в теле ответа HTTP:

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

```
Hello john
```

Что произошло за кадром в примерах из этого раздела? Когда аутентификация завершилась, другой фильтр перехватил запрос. `AuthorizationWebFilter` делегирует ответственность за авторизацию `ReactiveAuthorizationManager` (рис. 17.7).

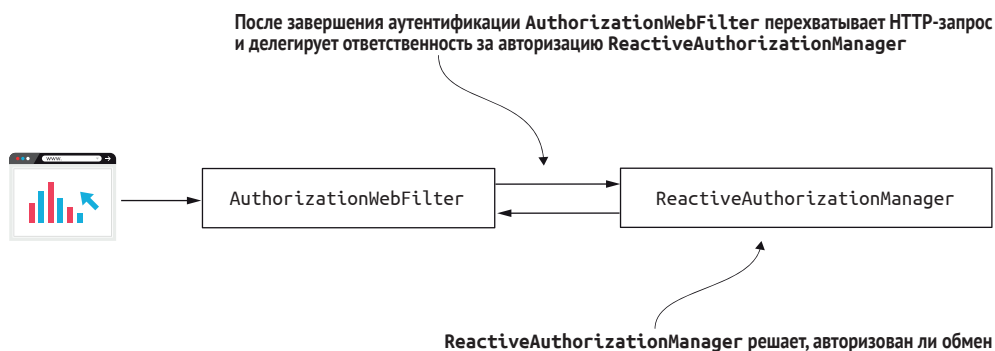


Рис. 17.7 После успешного завершения процесса аутентификации другой фильтр, названный `AuthorizationWebFilter`, перехватывает запрос. Этот фильтр делегирует ответственность за авторизацию `ReactiveAuthorizationManager`

Подождите! Означает ли это, что у нас есть только один `ReactiveAuthorizationManager`? Каким образом этот компонент узнает, как авторизовать запрос на основе сделанных нами конфигураций? Ответ на первый вопрос: нет, на самом деле существует несколько реализаций `ReactiveAuthorizationManager`. `AuthorizationWebFilter`

использует bean-компонент `SecurityWebFilterChain`, который мы добавили в контекст Spring. С помощью этого бина фильтр решает, какой реализации `ReactiveAuthorizationManager` делегировать ответственность за авторизацию (рис. 17.8).

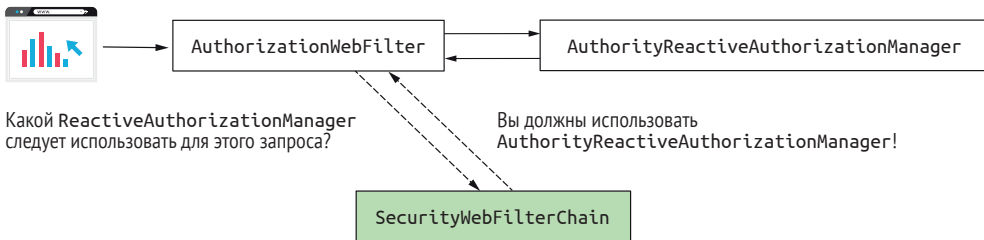


Рис. 17.8 `AuthorizationFilter` использует компонент `SecurityWebFilterChain` (закрашен), который мы добавили в контекст, чтобы знать, какой `ReactiveAuthorizationManager` использовать

17.3.2 Использование безопасности методов в реактивных приложениях

В этом разделе мы рассмотрим применение правил авторизации для всех слоев реактивных приложений. Для нереактивных приложений мы использовали безопасность методов, а в главах 11 и 12 вы узнали о различных подходах к применению правил авторизации на уровне методов. Возможность добавлять правила авторизации на уровнях, отличных от уровня конечной точки, обеспечивает большую гибкость и позволяет применять авторизацию для локальных (не веб-) приложений. Чтобы научиться использовать безопасность методов в реактивных приложениях, мы поработаем над отдельным примером в проекте `ssia-ch17-ex5`.

В отличие от глобальной безопасности методов при работе с нереактивными приложениями, мы будем называть наш новый подход безопасностью *реактивных* методов и применять правила авторизации непосредственно на уровне метода. В нашем примере мы используем аннотацию `@PreAuthorize` для проверки того, что у пользователя есть определенная роль для вызова тестовой конечной точки. Чтобы упростить пример, мы помещаем аннотацию `@PreAuthorize` непосредственно над методом, определяющим конечную точку. Но вы можете использовать ее так же, как было показано в главах 11 и 12 для нереактивных приложений: в любом другом методе компонента в вашем реактивном приложении. В листинге 17.10 показано определение класса контроллера. Обратите внимание, что мы используем `@PreAuthorize` аналогично тому, как делали в главе 11. Используя выражения SpEL, мы объявляем, что только администратор может вызывать аннотированный метод.

Листинг 17.10 Определение класса контроллера

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    @PreAuthorize("hasRole('ADMIN')") ❶
    public Mono<String> hello() {
        return Mono.just("Hello");
    }
}
```

❶ Используем `@PreAuthorize` для ограничения доступа к методу.

В листинге 17.11 вы найдете класс конфигурации, в котором мы используем аннотацию `@EnableReactiveMethodSecurity` для включения функции безопасности реактивного метода. По аналогии с безопасностью метода нам нужно явно использовать аннотацию для ее включения. Помимо этой аннотации, в классе конфигурации также присутствует обычное определение управления пользователями.

Листинг 17.11 Класс конфигурации

```
@Configuration
@EnableReactiveMethodSecurity ❶
public class ProjectConfig {

    @Bean
    public ReactiveUserDetailsService userDetailsService() {
        var u1 = User.withUsername("john")
            .password("12345")
            .roles("ADMIN")
            .build();

        var u2 = User.withUsername("bill")
            .password("12345")
            .roles("REGULAR_USER")
            .build();

        var uds = new MapReactiveUserDetailsService(u1, u2);

        return uds;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

❶ Включаем функцию безопасности реактивного метода.

Теперь вы можете запустить приложение и протестировать поведение конечной точки, вызвав ее для каждого пользователя. Вы должны заметить, что только Джон может вызвать конечную точку, потому что мы определили его как администратора. Билл – это просто обычный пользователь, поэтому если мы попытаемся вызвать конечную точку от имени Билла, то получим ответ со статусом HTTP 403 Forbidden. Вызов конечной точки /hello от имени Джона выглядит следующим образом:

```
curl -u john:12345 http://localhost:8080/hello
```

Тело ответа:

Hello!

Вызов конечной точки /hello от имени Билла выглядит так:

```
curl -u bill:12345 http://localhost:8080/hello
```

Тело ответа:

Access Denied

Внутренний механизм этой функциональности работает так же, как и для неактивных приложений. В главах 11 и 12 вы узнали, что аспект перехватывает вызов метода и обрабатывает авторизацию. Если вызов не соответствует указанным правилам предварительной авторизации, аспект не делегирует вызов методу (рис. 17.9).

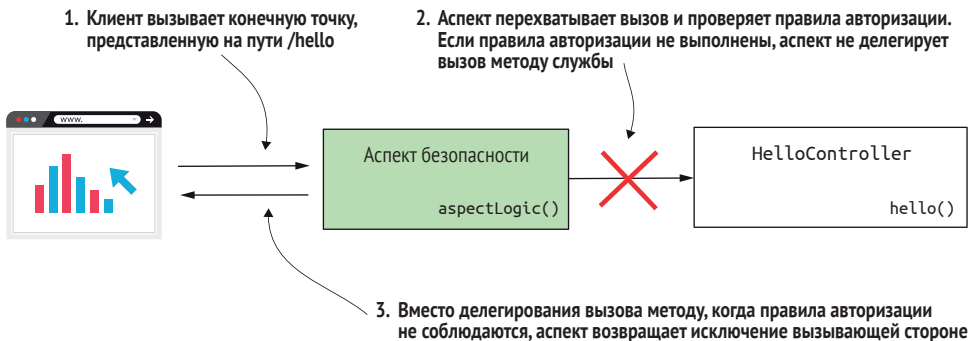


Рис. 17.9 При использовании безопасности метода аспект перехватывает вызов защищенного метода. Если вызов не соответствует правилам предварительной авторизации, аспект не делегирует вызов методу

17.4 Создание реактивного сервера ресурсов OAuth 2

Вы, вероятно, уже задааетесь вопросом, можем ли мы использовать реактивные приложения в системе, разработанной на основе фреймворка OAuth 2. В этом разделе мы обсудим реализацию сервера ресурсов как реактивного приложения. Вы узнаете, как настроить ваше реактивное приложение для аутентификации по протоколу OAuth 2. Поскольку использование OAuth 2 в настоящее время широко распространено, вы можете столкнуться с требованиями, когда ваше приложение сервера ресурсов должно быть спроектировано как реактивный сервер. Я создал новый проект с именем `ssia-ch17-ex6`, и в нем мы реализуем приложение реактивного сервера ресурсов. Вам нужно добавить зависимости в файл `pom.xml`, как показано в следующем фрагменте кода:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-oauth2</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Еще нам нужна конечная точка для тестирования приложения, поэтому мы добавляем класс контроллера. Он представлен в следующем фрагменте кода:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public Mono<String> hello() {
        return Mono.just("Hello!");
    }
}
```

А теперь самая важная часть примера: конфигурация безопасности. Для этого примера мы настраиваем сервер ресурсов на использование открытого ключа, предоставленного сервером авторизации для проверки подписи токена.

Для настройки метода аутентификации мы используем `SecurityWebFilterChain`, о котором вы узнали в разделе 17.3. Однако вместо метода `httpBasic()` мы вызываем метод `oauth2ResourceServer()`. Затем, вызывая метод `jwt()`, мы определяем тип используемого токена, а с помощью объекта `Customizer` указываем способ проверки подписи токена. В листинге 17.12 показано определение класса `configuration`.

Листинг 17.12 Определение конфигурации цепочки веб-фильтров безопасности

```
@Configuration
public class ProjectConfig {

    @Value("${jwk.endpoint}")
    private String jwkEndpoint;

    @Bean
    public SecurityWebFilterChain securityWebFilterChain(
        ServerHttpSecurity http) {

        http.oauth2ResourceServer(
            c -> c.jwt(
                j -> j.jwkSetUri(jwkEndpoint);
            )
        );

        http.authorizeExchange(
            c -> c.anyExchange().authenticated()
        );

        return http.build();
    }
}
```

① Настраиваем метод аутентификации сервера ресурсов.

② Указываем способ проверки токена.

Точно так же мы могли бы напрямую задать открытый ключ вместо указания URI, по которому он доступен. Единственное изменение заключается в вызове метода `publicKey()` экземпляра `jwtSpec` и предоставлении действительного открытого ключа в качестве параметра. Вы можете использовать любой из подходов, которые мы обсуж-

дали в главе 13, где мы подробно анализировали варианты создания сервера ресурсов для проверки токена доступа.

Далее мы изменяем файл `application.properties`, чтобы добавить значение для URI набора ключей, а также изменяем порт сервера на 9090. Таким образом, мы не помешаем серверу авторизации работать через порт 8080. Ниже показано содержимое файла `application.properties`:

```
server.port=9090
jwk.endpoint=http://localhost:8080/
➔auth/realms/master/protocol/openid-connect/certs
```

Давайте запустим приложение и убедимся, что оно ведет себя так, как мы ожидали. Сгенерируем токен доступа с помощью сервера авторизации:

```
curl -XPOST 'http://localhost:8080/auth/
➔realms/master/protocol/openid-connect/token' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=password' \
--data-urlencode 'username=bill' \
--data-urlencode 'password=12345' \
--data-urlencode 'client_id=fitnessapp' \
--data-urlencode 'scope=fitnessapp'
```

В теле HTTP-ответа получаем токен доступа, как показано далее:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI...",
  "expires_in": 6000,
  "refresh_expires_in": 1800,
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5c...",
  "token_type": "bearer",
  "not-before-policy": 0,
  "session_state": "610f49d7-78d2-4532-8b13-285f64642caa",
  "scope": "fitnessapp"
}
```

Используя токен доступа, мы вызываем конечную точку `/hello` нашего приложения следующим образом:

```
curl -H 'Authorization:
BearereyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJMSE9zT0VRSmJuTmJVb
jhQbVpYQTlUVW9QNTZowU90YzNWT2swa1V2ajVWIn...' \
'http://localhost:9090/hello'
```

Тело ответа:

Hello!

Заключение

- Реактивные приложения основаны на особых принципах обработки данных и обмена сообщениями с другими компонентами. Реактивные приложения могут быть наилучшим выбором в некоторых ситуациях, например когда мы можем разделить данные на меньшие сегменты для обработки и обмена.
- Как и в случае с любым другим приложением, вам также необходимо защитить реактивные приложения с помощью инструментов безопасности. Spring Security предлагает отличный набор инструментов, которые вы можете одинаково эффективно применять к неактивным и реактивным приложениям.
- Для управления пользователями в реактивных приложениях с помощью Spring Security мы используем контракт `ReactiveUserDetailsService`. Этот компонент имеет то же назначение, что и `UserDetailsService` для неактивных приложений: он сообщает приложению, как получить данные пользователя.
- Чтобы реализовать правила авторизации конечной точки для реактивного веб-приложения, вам необходимо создать экземпляр типа `SecurityWebFilterChain` и добавить его в контекст Spring. Экземпляр `SecurityWebFilterChain` создается с помощью конструктора `ServerHttpSecurity`.
- Как правило, имена методов, которые вы используете для определения конфигураций авторизации, совпадают с именами методов для неактивных приложений. Однако есть и незначительные различия в именах, связанные с реактивной терминологией. Например, вместо метода `authorizeHttpRequests()` для реактивных приложений применяется его аналог `authorizeExchange()`.
- Spring Security также предоставляет способ определения правил авторизации на уровне метода, называемый безопасностью реактивного метода. Он обеспечивает большую гибкость в применении правил авторизации на любом уровне реактивного приложения и похож на глобальную безопасность метода для неактивных приложений.

Часть VI

Тестирование конфигураций безопасности

В мире разработки программного обеспечения тестирование выступает в роли предшественника качества, гарантируя, что каждый фрагмент кода не только функционирует так, как задумано, но и легко интегрируется с другими компонентами. Это особенно важно при работе с конфигурациями безопасности, которые предлагает Spring Security. Эта заключительная часть книги посвящена внедрению лучших практик интеграционного тестирования в разработку приложений Spring Security, благодаря которым ваши приложения будут соответствовать требованиям правил безопасности.

Глава 18 послужит вам всеобъемлющим руководством по проверке конфигурации безопасности. В ней вы изучите сферы тестирования фиктивных пользователей, углубитесь в нюансы аннотации `@WithMockUser` и узнаете, как проверять управляемых пользователей. Глава охватывает тестирование безопасности методов, аутентификацию и уникальные проблемы, возникающие при тестировании реактивных приложений.

К концу этой части вы приобретете набор навыков для тщательного тестирования безопасности вашего приложения на всех уровнях и сможете обеспечить защищенное развертывание, готовое противостоять потенциальным уязвимостям.

18

Тестирование конфигураций безопасности

Темы этой главы:

- тестирование конфигураций Spring Security и безопасности конечных точек;
- создание фиктивных пользователей для тестов;
- тестирование конфигураций Spring Security и безопасности на уровне методов;
- тестирование реактивных реализаций Spring.

Легенда гласит, что написание модульных и интеграционных тестов началось со следующего короткого стишка:

99 маленьких ошибок в коде,
99 маленьких ошибок.
Мы нашли одну, исправили ее,
Осталось 113 маленьких ошибок.

(Анонимный автор)

Со временем программное обеспечение стало сложнее, а команды разработчиков – больше. Знать все функции, реализованные другими людьми, стало невозможно. Разработчикам нужен был способ убедиться, что они не навредят работе приложения при исправлении ошибок или внедрении новых функций.

При разработке приложений мы постоянно пишем тесты, чтобы убедиться, что реализуемые нами функции работают как задумано. Основная причина, по которой мы применяем модульные и интеграционные тесты, заключается в том, чтобы гарантировать, что мы не нарушим работоспособность существующих функций при изменении кода для исправления ошибки или внедрения новых функций. Этот подход называется *регрессионным тестированием* (regression testing).

В настоящее время, когда разработчик заканчивает вносить изменения, он загружает изменения на сервер, используемый командой для управления версиями кода. Это действие автоматически запускает инструмент непрерывной интеграции, который выполняет все запланированные тесты. Если какие-либо изменения нарушают существующую функциональность, тесты не проходят, и инструмент непрерывной интеграции уведомляет об этом команду (рис. 18.1). Благодаря этому значительно уменьшается вероятность внесения изменений, влияющих на работоспособность приложения.

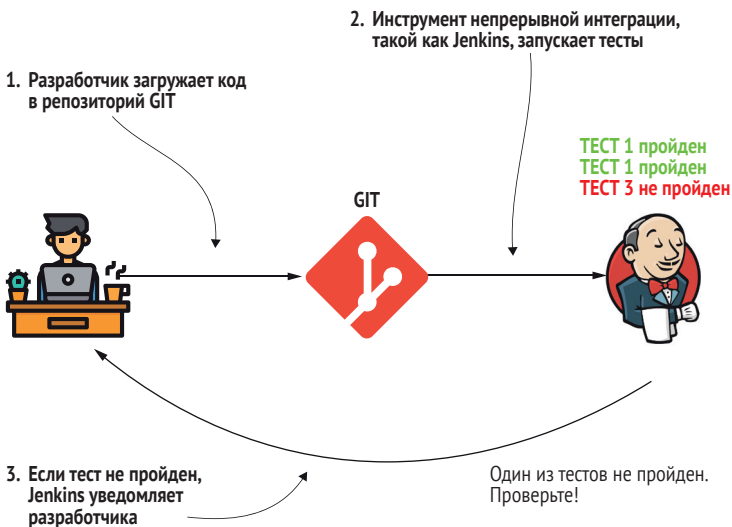


Рис. 18.1 Тестирование является частью процесса разработки. Каждый раз, когда разработчик загружает код, автоматически запускаются тесты. Если какой-либо тест не выполняется, инструмент непрерывной интеграции уведомляет разработчика

ПРИМЕЧАНИЕ Упомянув Jenkins на этом рисунке, я не говорю, что это единственный используемый инструмент непрерывной интеграции или что он лучший. У вас есть обширный выбор вариантов, таких как Bamboo, GitLab CI, CircleCI и т. д.

При тестировании не забывайте, что нужно проверять не только код вашего приложения. Обязательно тестируйте интеграции с применяемыми в коде фреймворками и библиотеками (рис. 18.2). Рано или поздно вы обновите текущий фреймворк или библиотеку до новой версии. При изменении версии зависимости следует убедиться, что ваше приложение по-прежнему хорошо работает с новой версией этой зависимости. Если обнаружится проблема интеграции, вы сможете вовремя внести необходимые изменения в код приложения.

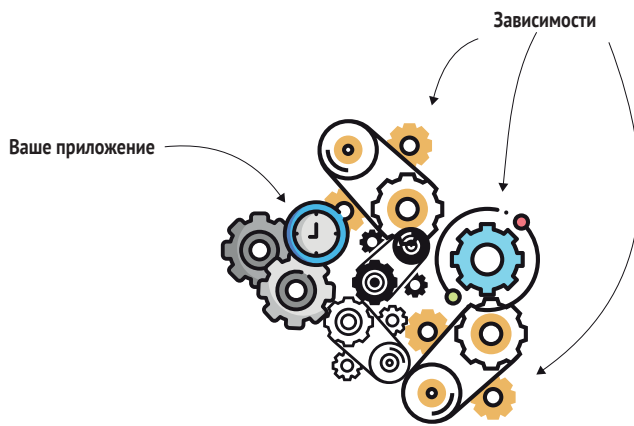


Рис. 18.2 Функциональность приложения основана на множестве зависимостей. Когда вы обновляете или изменяете зависимость, вы можете повлиять на функциональность. Выполнение интеграционных тестов зависимостей помогает вам быстро обнаружить, влияет ли изменение зависимости на функциональность вашего приложения

Вот почему вам нужно знать главную тему этой главы – тестирование интеграции вашего приложения с Spring Security. Фреймворк безопасности Spring Security, как и экосистема Spring в целом, быстро развивается. Скорее всего, вы регулярно обновляете свое приложение до новых версий, и вы, безусловно, хотите знать, не ведет ли обновление до определенной версии к уязвимостям, ошибкам или несовместимостям со сторонними решениями. Помните, что я говорил в первой главе: вам нужно ориентироваться на безопасность с самого начала проектирования приложения, и вы должны отнестись к этому серьезно. Разработка тестов для конфигураций безопасности должна быть обязательной задачей, без выполнения которой разработка приложения не может считаться завершенной. Вы не должны считать свою работу выполненной, если тесты безопасности не готовы.

В этой главе мы обсудим несколько методик тестирования интеграции приложения с Spring Security. Мы вернемся к некоторым примерам, над которыми работали в предыдущих главах, и вы узнаете,

как писать интеграционные тесты для реализованной функциональности. Тестирование в целом является важной темой. Но подробное изучение этого предмета дает много дополнительных преимуществ. В этой главе мы сосредоточимся на тестировании интеграции между приложением и Spring Security. Прежде чем перейти к примерам, я хотел бы порекомендовать несколько ресурсов, которые помогли мне хорошо понять этот предмет. Если вы тоже хотите разобраться в нем подробнее или просто освежить знания, я рекомендую прочитать следующие книги. Я уверен, что они окажутся очень полезными!

- *JUnit in Action, Third Edition*, авторы Cătălin Tudose и др. (Manning, 2020).
- *Принципы юнит-тестирования*, автор Владимир Хориков (Прогресс, 2021).
- *Testing Java Microservices*, авторы Alex Soto Bueno и др. (Manning, 2018).

Наше погружение в тему тестирования реализаций безопасности начинается с тестов для конфигураций авторизации. В разделе 18.1 вы узнаете, как пропустить аутентификацию и определить фиктивных пользователей для тестирования конфигурации авторизации на уровне конечной точки. Затем в разделе 18.2 вы узнаете, как тестировать конфигурации авторизации с пользователями из `UserDetailsService`. В разделе 18.3 мы обсудим, как настроить полный контекст безопасности, в случае если вам нужно использовать конкретные реализации объекта `Authentication`. И наконец, в разделе 18.4 вы примените подходы, изученные в предыдущих разделах, для тестирования конфигурации авторизации на уровне безопасности метода.

После того как мы закончим обсуждение тестирования авторизации, раздел 18.5 научит вас тестировать поток аутентификации. Затем в разделах 18.6 и 18.7 мы обсудим тестирование других конфигураций безопасности, таких как подделка межсайтовых запросов (CSRF) и совместное использование ресурсов между источниками (CORS). Глава заканчивается разделом 18.8, в котором обсуждаются интеграционные тесты Spring Security и реактивных приложений.

18.1 Использование фиктивных пользователей для тестов

В данном разделе обсуждается использование *фиктивных пользователей* (mock user) для тестирования конфигурации авторизации. Этот подход является наиболее простым и часто используемым методом тестирования конфигураций авторизации. При использова-

нии фиктивного пользователя тест полностью пропускает процесс аутентификации (рис. 18.3).

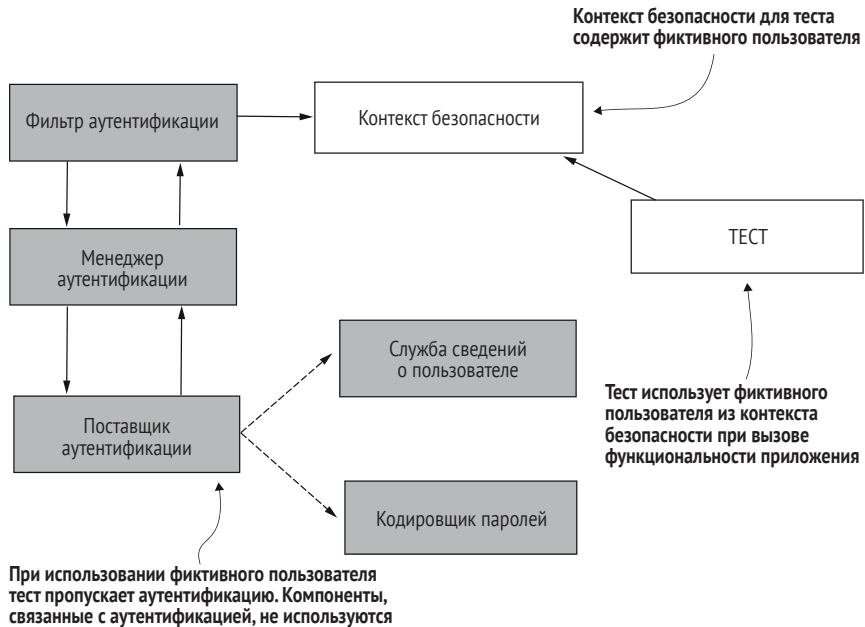


Рис. 18.3 Мы пропускаем компоненты, обозначенные закрашенными блоками, в потоке аутентификации Spring Security при выполнении теста. Тест напрямую использует фиктивный SecurityContext, который содержит фиктивного пользователя, определенного специально для тестирования функциональности

Тесты с пропуском аутентификации применяются очень часто. В самом деле, вам не нужно проверять процесс аутентификации каждый раз, когда вы хотите убедиться, что система правильно применяет правило авторизации. Помните, что аутентификация и авторизация зависят друг от друга, но они полностью разделены через контекст безопасности. Поэтому если вы хотите протестировать конфигурацию авторизации изолированно, вы можете определить фиктивный контекст безопасности и применять его для тестирования всех необходимых сценариев авторизации. Поскольку в большинстве случаев приложение реализует только ограниченное количество методов аутентификации (чаще всего только один), но имеет большое разнообразие правил авторизации, которые применяются к вариантам использования или конечным точкам, имеет смысл писать изолированные тесты авторизации. Вам незачем повторять тесты аутентификации каждый раз, когда вы проверяете, что авторизация для определенного элемента работает нормально.

Фиктивный пользователь действителен только для выполнения теста, и для этого пользователя вы можете настроить любые харак-

теристики, необходимые для проверки определенного сценария. Например, вы можете назначить пользователю определенные роли (ADMIN, MANAGER и т. д.) или использовать различные полномочия для проверки того, как ведет себя приложение в разных ситуациях.

ПРИМЕЧАНИЕ Важно знать, какие компоненты фреймворка участвуют в интеграционном тесте. Таким образом, вы будете знать, какую часть интеграции вы охватываете тестом. Например, фиктивный пользователь может применяться только для проверки авторизации. (В разделе 18.5 вы узнаете, как работать с аутентификацией.) Я часто вижу, как заблуждаются разработчики. Они думают, например, что при работе с фиктивным пользователем охватывают тестом пользовательскую реализацию `AuthenticationProvider`, что не так. Постарайтесь тщательно разобраться, что именно вы тестируете.

Чтобы показать, как написать такой тест, вернемся к простейшему примеру, над которым мы работали в этой книге, – проекту `ssia-ch2-ex1`. Этот проект предоставляет конечную точку для пути `/hello` только с конфигурацией Spring Security по умолчанию. Чего мы ожидаем?

- При вызове конечной точки без пользователя статус ответа HTTP должен быть 401 Unauthorized.
- При вызове конечной точки с аутентифицированным пользователем статус ответа HTTP должен быть 200 OK, а тело ответа должно быть Hello!.

Давайте же протестируем эти два сценария! Для написания тестов нам понадобится пара зависимостей в файле `pom.xml`. В следующем фрагменте кода показаны классы, которые мы используем в примерах этой главы. Вы должны убедиться, что они есть в вашем файле `pom.xml`, прежде чем начинать писать тесты. Вот зависимости:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

ПРИМЕЧАНИЕ Для примеров в этой главе при написании тестов я использовал JUnit. Однако ничего страшного, если вы продолжаете работать с JUnit 4. С точки зрения интегра-

ции Spring Security аннотации и остальные классы, которые вы изучите, работают так же. Глава 4 в книге JUnit in Action (Cătălin Tudose et al.; Manning, 2020), посвященная миграции с JUnit 4 на JUnit 5, содержит несколько интересных таблиц, которые показывают соответствие между классами и аннотациями версий 4 и 5. Вот ссылка на главу: <http://mng.bz/OPJn>.

В тестовой папке проекта Spring Boot Maven мы добавляем класс с именем `MainTests`. Мы пишем этот класс как часть основного пакета приложения. Имя основного пакета – `com.laurentiuspilca.ssia`. В листинге 18.1 дано определение пустого класса для тестов. Мы используем аннотацию `@SpringBootTest`, которая представляет собой удобный способ управления контекстом Spring для нашего тестового набора.

Листинг 18.1 Класс для написания тестов

```
@SpringBootTest ❶
public class MainTests {

}
```

❶ Делаем Spring Boot ответственным за управление контекстом Spring для тестов.

Удобный способ реализовать тест поведения конечной точки – использовать утилиту `MockMvc` Spring. В приложении Spring Boot вы можете автоматически настроить эту утилиту для тестирования вызовов конечной точки, добавив аннотацию к классу, как показано в следующем листинге.

Листинг 18.2 Добавление `MockMvc` для реализации тестовых сценариев

```
@SpringBootTest
@AutoConfigureMockMvc ❶
public class MainTests {

    @Autowired
    private MockMvc mvc; ❷

}
```

❶ Заставляет Spring Boot автоматически настроить `MockMvc`. В результате объект типа `MockMvc` добавляется в контекст Spring.

❷ Внедряет объект `MockMvc`, который мы используем для тестирования конечной точки.

Теперь, когда у нас есть инструмент, который мы можем использовать для тестирования поведения конечной точки, приступим к первому сценарию. При вызове конечной точки `/hello` без аутентифицированного пользователя статус ответа HTTP должен быть 401 `Unauthorized`.

Связь между компонентами этого теста показана на рис. 18.4. Тест вызывает конечную точку, но использует фиктивный `SecurityContext`. От нас зависит, что будет в него добавлено. С помощью нашего теста мы хотим убедиться, что если мы не добавим пользователя, способного вызвать конечную точку без аутентификации, приложение отклонит вызов с ответом HTTP 401 Unauthorized. Когда мы добавляем пользователя в `SecurityContext`, приложение принимает вызов и возвращает ответ HTTP со статусом 200 OK.

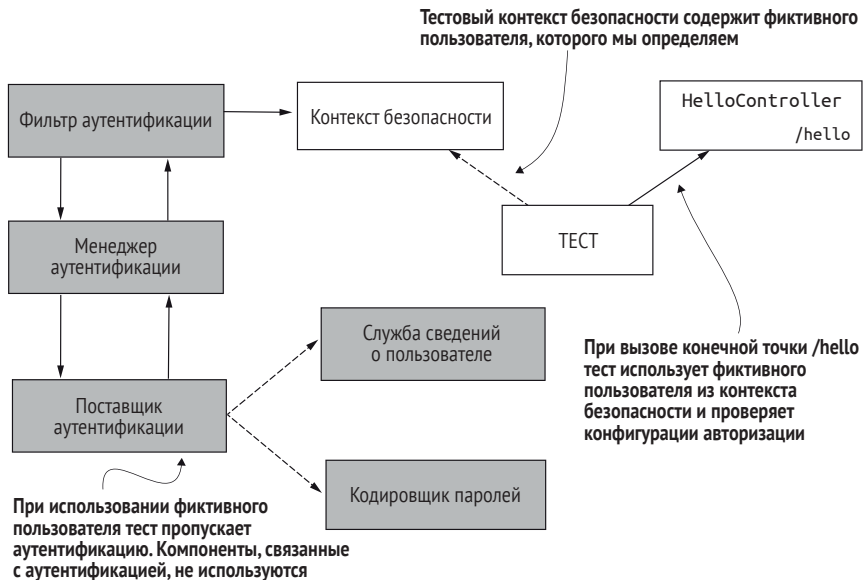


Рис. 18.4 При запуске теста мы пропускаем аутентификацию. Тест использует фиктивный `SecurityContext` и вызывает конечную точку `/hello`, предоставленную `HelloController`. Мы добавляем фиктивного пользователя в тестовый `SecurityContext`, чтобы проверить, что поведение соответствует правилам авторизации. Если фиктивный пользователь не определен, ожидается, что приложение не авторизует вызов, но если мы определяем пользователя, то вызов должен быть успешным

В листинге 18.3 представлена реализация этого сценария.

Листинг 18.3 Проверка предположения, что вы не можете вызвать конечную точку без аутентифицированного пользователя

```

@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;
  
```



```
@Test
public void helloUnauthenticated() throws Exception {
    mvc.perform(get("/hello"))
```

- ❶ При выполнении запроса GET для пути /hello мы ожидаем получить ответ со статусом Unauthorized.

Обратите внимание, что мы статически импортируем методы `get()` и `status()`. Вы можете найти метод `get()` и аналогичные методы, связанные с запросами, которые мы используем в примерах этой главы, в этом классе:

```
org.springframework.test.web.servlet.request.MockMvcRequestBuilders
```

Аналогично вы можете найти метод `status()` и подобные методы, связанные с результатом вызовов, которые мы используем в следующих примерах этой главы в этом классе:

```
org.springframework.test.web.servlet.result.MockMvcResultMatchers
```

Теперь вы можете запустить тесты и увидеть статус в вашей IDE. Обычно в любой IDE для запуска тестов можно щелкнуть правой кнопкой мыши по классу теста и выбрать **Run** (Запустить). IDE отображает успешный тест зеленым цветом, а неудачный – другим цветом (обычно красным или желтым).

ПРИМЕЧАНИЕ В проектах этой книги над каждым методом, реализующим тест, я также помещаю аннотацию `@DisplayName`. Эта аннотация позволяет применять более длинное, более подробное описание сценария теста. Чтобы занимать меньше места в книге и не мешать вам сосредоточиться на функциональности обсуждаемых нами тестов, я убрал аннотацию `@DisplayName` из листингов в книге.

Для тестирования второго сценария нам нужен фиктивный пользователь. Для проверки поведения вызова конечной точки /hello с аутентифицированным пользователем мы воспользуемся аннотацией `@WithMockUser`. Добавляя эту аннотацию над методом теста, мы указываем Spring настроить `SecurityContext`, содержащий экземпляр реализации `UserDetails`. По сути, это обход аутентификации. Теперь вызов конечной точки ведет себя так, как если бы пользователь, определенный аннотацией `@WithMockUser`, успешно прошел аутентификацию.

В этом простом примере нам не интересны данные о фиктивном пользователе, такие как его имя, роли или полномочия. Поэтому мы добавляем аннотацию `@WithMockUser`, которая предоставляет значения по умолчанию для атрибутов фиктивного пользователя. Далее в этой главе вы узнаете, как настраивать атрибуты пользователя для

тестовых сценариев, в которых их значения важны. Листинг 18.4 демонстрирует реализацию для второго тестового сценария.

Листинг 18.4 Использование @WithMockUser для определения фиктивного аутентифицированного пользователя

```
@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;

    // Часть кода не показана

    @Test
    @WithMockUser b
    public void helloAuthenticated() throws Exception {
        mvc.perform(get("/hello")) c
            .andExpect(content().string("Hello!"))
            .andExpect(status().isOk());
    }
}
```

- ❶ Вызываем метод с фиктивным аутентифицированным пользователем.
- ❷ В этом случае при выполнении запроса GET для пути /hello мы ожидаем, что статус ответа будет OK.

Запустите этот тест и убедитесь, что он успешно пройден. Однако в некоторых ситуациях нам нужно использовать конкретное имя или предоставить пользователю определенные роли либо полномочия для реализации теста. Допустим, мы хотим протестировать конечные точки, которые мы определили в ssia-ch5-ex2. В этом примере тело ответа зависит от имени аутентифицированного пользователя. Чтобы написать тест, нам нужно дать пользователю определенное имя. В следующем листинге показано, как настроить данные фиктивного пользователя, написав тест для конечной точки /hello в проекте ssia-ch5-ex2.

Листинг 18.5 Настройка данных для фиктивного пользователя

```
@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    // Часть кода не показана

    @Test
    @WithMockUser(username = "mary") b
    public void helloAuthenticated() throws Exception {
        mvc.perform(get("/hello"))
    }
}
```

```

        .andExpect(content().string("Hello, mary!"))
        .andExpect(status().isOk());
    }
}

```

❶ Задаем имя пользователя для фиктивной учетной записи.

На рис. 18.5 приводится сравнение того, как использование аннотаций для определения тестовой среды безопасности отличается от использования `RequestPostProcessor`. Фреймворк интерпретирует аннотации, такие как `@WithMockUser`, прежде чем он выполнит тестовый метод. Таким образом, тестовый метод создает тестовый запрос и выполняет его в уже настроенной среде безопасности. При использовании `RequestPostProcessor` фреймворк сначала вызывает тестовый метод и создает тестовый запрос. Затем фреймворк применяет `RequestPostProcessor`, который изменяет запрос или среду, в которой он выполняется, перед его отправкой. В этом случае фреймворк настраивает тестовые зависимости, такие как фиктивные пользователи и `SecurityContext`, после создания тестового запроса.

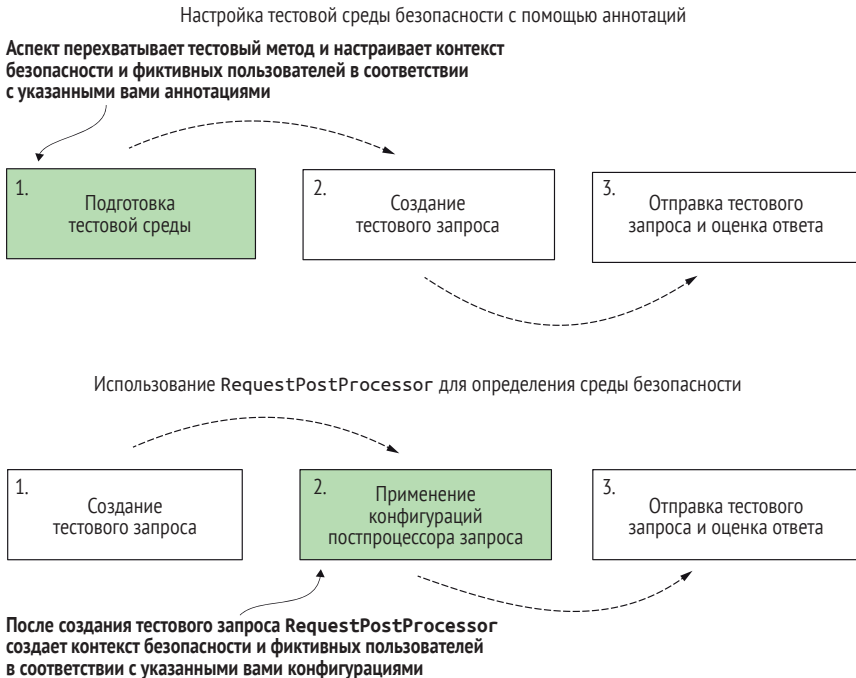


Рис. 18.5 Разница между использованием аннотаций и `RequestPostProcessor` для создания тестовой среды безопасности. При использовании аннотаций фреймворк сначала настраивает тестовую среду безопасности. При использовании `RequestPostProcessor` тестовый запрос создается, а затем изменяется для определения других ограничений, таких как тестовая среда безопасности. На рисунке закрашены блоки, в которых фреймворк применяет тестовую среду безопасности

Аналогично настройке имени пользователя, вы можете задать полномочия и роли для тестирования правил авторизации. Альтернативный подход к созданию фиктивного пользователя – использование `RequestPostProcessor`. Мы можем предоставить `RequestPostProcessor` метод `with()`, как показано в листинге 18.6. Класс `SecurityMockMvcRequestPostProcessors`, предоставляемый Spring Security, предлагает множество реализаций для `RequestPostProcessor`. Это помогает нам применять различные тестовые сценарии.

В этой главе мы также обсудим часто используемые реализации для `RequestPostProcessor`. Метод `user()` класса `SecurityMockMvcRequestPostProcessors` возвращает `RequestPostProcessor`, который мы можем использовать в качестве альтернативы аннотации `@WithMockUser`.

Листинг 18.6 Использование `RequestPostProcessor` для определения фиктивного пользователя

```
@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    // Часть кода не показана

    @Test
    public void helloAuthenticatedWithUser() throws Exception {
        mvc.perform(
            get("/hello")
                .with(user("mary")))
            .andExpect(content().string("Hello!"))
            .andExpect(status().isOk());
    }
}
```

❶ Вызываем конечную точку `/hello`, используя фиктивного пользователя с именем `"mary"`.

Как вы могли заметить, тесты для конфигураций авторизации пишутся быстро и просто! Большинство тестов, которые вы разрабатываете для интеграции Spring Security с функциональными возможностями вашего приложения, предназначены для конфигураций авторизации. Вы можете задаться вопросом, почему мы не тестировали аутентификацию. В разделе 18.5 мы обсудим тестирование аутентификации. Однако в целом, и как упоминалось ранее в этом разделе, имеет смысл тестировать авторизацию и аутентификацию отдельно. Обычно приложение имеет один способ аутентификации пользователей, но может предоставлять десятки конечных точек, для которых авторизация настроена по-разному. Вот почему чаще аутентификацию тестируют отдельно с помощью нескольких тестов, а затем применяют индивидуальные тесты для каждой конфигура-

ции авторизации конечных точек. Повторное тестирование аутентификации для каждой конечной точки – это пустая трата времени, если логика аутентификации не изменилась.

18.2 Тестирование с пользователями из UserDetailsService

В этом разделе обсуждается получение данных пользователя для тестов из UserDetailsService. Данный подход является альтернативой созданию фиктивного пользователя. Разница в том, что на этот раз вместо создания фиктивного пользователя нам нужно получить пользователя из заданного UserDetailsService. Мы используем этот подход, если нужно также протестировать интеграцию с источником данных, из которого приложение загружает данные пользователя (рис. 18.6).

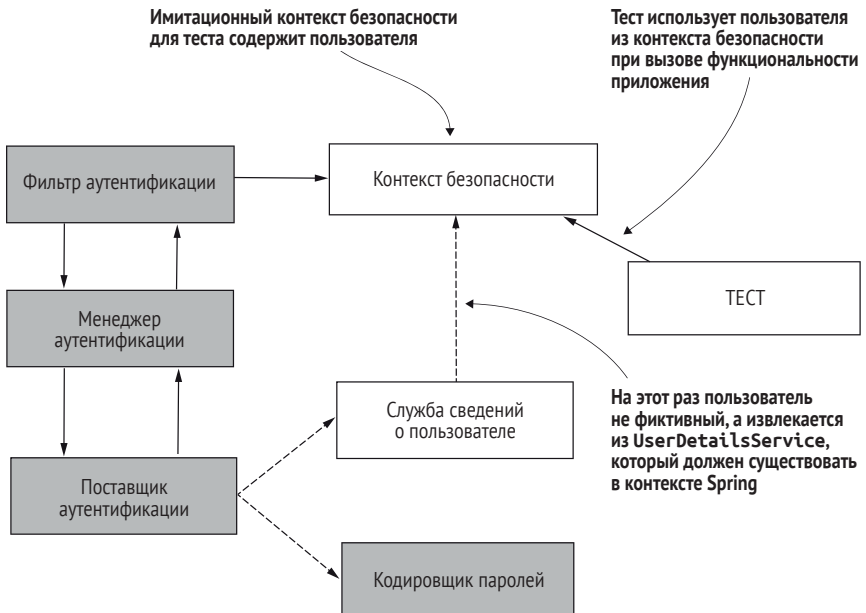


Рис. 18.6 Вместо создания фиктивного пользователя для теста при построении SecurityContext, используемого тестом, мы берем данные пользователя из UserDetailsService. Таким образом, вы можете протестировать авторизацию с использованием реальных пользователей, взятых из источника данных. Во время теста поток выполнения пропускает закрашенные компоненты

Чтобы продемонстрировать этот подход, откроем проект ssia-ch2-ex2 и создадим тесты для конечной точки, представленной по пути /hello. Мы применим компонент UserDetailsService, который

в данном проекте уже добавляется в контекст. Обратите внимание, что при таком подходе нам необходимо иметь компонент `UserDetailsService` в контексте. Чтобы указать пользователя, которого мы аутентифицируем из этого компонента, мы аннотируем тестовый метод с помощью `@WithUserDetails`. Чтобы найти пользователя под такой аннотацией, необходимо указать его имя. В листинге 18.7 представлена реализация теста для конечной точки `/hello` с использованием аннотации `@WithUserDetails` для определения аутентифицированного пользователя.

Листинг 18.7 Определение аутентифицированного пользователя с помощью аннотации `@WithUserDetails`

```
@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;

    @Test
    @WithUserDetails("john")
    public void helloAuthenticated() throws Exception {
        mvc.perform(get("/hello"))
            .andExpect(status().isOk());
    }
}
```

❶ Загружаем пользователя "john" с помощью `UserDetailsService` для запуска тестового сценария.

18.3 Использование собственных объектов аутентификации для тестирования

Обычно при тестировании с фиктивным пользователем не имеет значения, какой класс использует фреймворк для создания экземпляров аутентификации в `SecurityContext`. Но предположим, что у вас в контроллере есть некоторая логика, зависящая от типа объекта. Можно ли как-то заставить фреймворк создать объект `Authentication` для теста с использованием определенного типа? Ответ – да, и сейчас вы узнаете, как это делается.

Логика этого подхода проста. Мы определяем класс-фабрику, отвечающий за создание `SecurityContext`. Таким образом, мы имеем полный контроль над тем, как создается тестовый `SecurityContext`, включая то, что находится внутри него (рис. 18.7). Например, мы можем задать наличие собственного объекта `Authentication`.

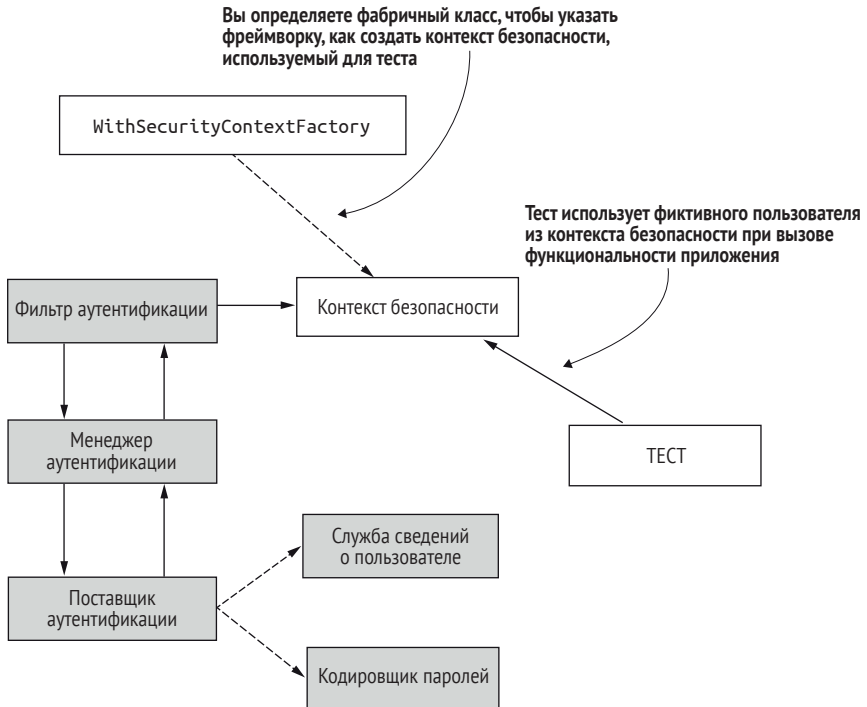


Рис. 18.7 Чтобы получить полный контроль над определением `SecurityContext` для теста, мы создаем класс-фабрику, который указывает тесту, как создавать `SecurityContext`. Таким образом, мы получаем большую гибкость и можем выбирать детали, такие как тип объекта для использования в качестве объекта `Authentication`. На схеме компоненты, исключенные из потока во время теста, выделены сплошной заливкой

Теперь откроем проект `ssia-ch2-ex4` и напишем тест, в котором мы настроим фиктивный `SecurityContext` и дадим инструкции фреймворку о том, как создать объект `Authentication`. Интересная особенность этого примера заключается в том, что мы используем его для проверки разработанного нами компонента `AuthenticationProvider`. Пользовательский `AuthenticationProvider`, который мы реализуем в нашем случае, аутентифицирует только пользователя по имени Джон. Однако, как и в двух других предыдущих подходах, которые мы обсуждали в разделах 18.1 и 18.2, текущий подход пропускает аутентификацию. По этой причине, как вы видите в конце примера, мы фактически можем дать любое имя нашему фиктивному пользователю. Мы выполняем три шага, чтобы добиться желаемого поведения (рис. 18.8):

- 1 пишем аннотацию для использования в тесте аналогично `@WithMockUser` или `@WithUserDetails`;
- 2 пишем класс, который реализует интерфейс `WithSecurityContextFactory`. Этот класс реализует метод `createSecurityContext`.

text(), который возвращает фиктивный SecurityContext, нужный фреймворку для теста;

- 3 связываем пользовательскую аннотацию, созданную на шаге 1, с классом-фабрикой, созданным на шаге 2, с помощью аннотации @WithSecurityContext.

Шаг 1

```
@WithCustomUser

@Retention(RetentionPolicy.RUNTIME)
public @interface WithCustomUser {

    String username();
}
```

Определяем пользовательскую аннотацию для использования в тестовых методах, для которых нам нужно настроить тестовый контекст безопасности

Шаг 2

```
WithSecurityContextFactory

public class CustomSecurityContextFactory
    implements WithSecurityContextFactory<WithCustomUser> {

    @Override
    public SecurityContext createSecurityContext() {
        ...
    }
}
```

Реализуем фабричный класс, в котором мы создаем пользовательский тестовый контекст безопасности. Фреймворк использует этот класс для создания контекста безопасности, когда находит тестовый метод, объявленный пользовательской аннотацией, созданной нами на шаге 1

Шаг 3

```
@WithCustomUser

@Retention(RetentionPolicy.RUNTIME)
@WithSecurityContext
(factory = CustomSecurityContextFactory.class)
public @interface WithCustomUser {

    String username();
}
```

Мы связываем пользовательскую аннотацию, созданную на шаге 1, с классом, созданным на шаге 2. Поэтому фреймворк знает, какой фабричный класс использовать при создании тестового контекста безопасности для метода, объявленного без пользовательской аннотации

Рис. 18.8 Чтобы тест мог использовать пользовательский SecurityContext, вам необходимо выполнить три шага, показанных на этом рисунке

Шаг 1: определение собственной аннотации

В листинге 18.8 вы можете найти определение пользовательской аннотации с именем @WithCustomUser, которую мы создаем для теста. В качестве свойств аннотации можно определить любые данные, необходимые для создания фиктивного объекта Authentication. В своем примере я добавил только имя пользователя. Также не забудьте использовать аннотацию @Retention (RetentionPolicy.RUNTIME), чтобы задать политику хранения для времени выполнения. Spring должен прочитать эту аннотацию во время выполнения. Чтобы разрешить Spring прочитать аннотацию, вам необходимо изменить ее политику хранения на RetentionPolicy.RUNTIME.

Листинг 18.8 Определение аннотации @WithCustomUser

```
@Retention(RetentionPolicy.RUNTIME)
public @interface WithCustomUser {

    String username();
}
```

Шаг 2: создание класса-фабрики для фиктивного SecurityContext

Второй шаг заключается в написании кода, который создает SecurityContext, нужный фреймворку для выполнения теста. На этом шаге мы решаем, какой тип аутентификации использовать для теста. Следующий листинг демонстрирует реализацию класса-фабрики.

Листинг 18.9 Реализация класса-фабрики для SecurityContext

```
public class CustomSecurityContextFactory ❶
    implements WithSecurityContextFactory<WithCustomUser> {

    @Override ❷
    public SecurityContext createSecurityContext(
        WithCustomUser withCustomUser) {
        SecurityContext context = ❸
            SecurityContextHolder.createEmptyContext();

        var a = new UsernamePasswordAuthenticationToken(
            withCustomUser.username(), null, null); ❹

        context.setAuthentication(a); ❺
        return context;
    }
}
```

- ❶ Реализуем аннотацию WithSecurityContextFactory и указываем свою аннотацию, которую мы используем для тестов.
- ❷ Реализуем createSecurityContext() для определения того, как создавать тестовый SecurityContext.
- ❸ Создаем пустой контекст безопасности.
- ❹ Создаем экземпляр Authentication.
- ❺ Добавляем фиктивный Authentication в SecurityContext.

Шаг 3: связывание собственной аннотации с классом-фабрикой

Используя аннотацию @WithSecurityContext, мы связали пользовательскую аннотацию, которую создали на шаге 1, с классом-фабрикой для SecurityContext, который мы реализовали на шаге 2. В листинге 18.10 показано изменение нашей аннотации @WithCustomUser для ее связывания с классом-фабрикой SecurityContext.

Листинг 18.10 Связывание пользовательской аннотации с классом-фабрикой `SecurityContext`

```
@Retention(RetentionPolicy.RUNTIME)
@WithSecurityContext(factory = CustomSecurityContextFactory.class)
public @interface WithCustomUser {

    String username();
}
```

После завершения этой подготовки мы можем написать тест для использования пользовательского `SecurityContext`. В листинге 18.11 показан код теста.

Листинг 18.11 Код теста, использующего пользовательский `SecurityContext`

```
@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;

    @Test
    @WithCustomUser(username = "mary")
    public void helloAuthenticated() throws Exception {
        mvc.perform(get("/hello"))
            .andExpect(status().isOk());
    }
}
```

❶ Выполняем тест с пользователем, имеющим имя "mary".

Запустив тест, вы увидите, что он успешно пройден. Вы можете подумать: «Постойте! В этом примере мы реализовали собственный класс `AuthenticationProvider`, который аутентифицирует только пользователя с именем `john`. Как тест может быть успешным для пользователя с именем `mary`?» Как и в случае `@WithMockUser` и `@WithUserDetails`, с этим методом мы *пропускаем логику аутентификации*. Поэтому вы можете использовать данный тест только для проверки всего, что связано с авторизацией и следует за ней.

18.4 Тестирование безопасности метода

В этом разделе обсуждается тестирование безопасности метода. Все тесты, которые мы написали до сих пор в этой главе, относятся к конечным точкам. Но что делать, если у вашего приложения нет конечных точек? Фактически, если это не веб-приложение, у него вообще

нет конечных точек! Однако мы можем использовать Spring Security с глобальными методами безопасности, как говорилось в главах 11 и 12. В таких ситуациях нам все равно нужно тестировать конфигурации безопасности.

К счастью, нам пригодятся те же подходы, которые мы обсуждали в предыдущих разделах. Мы все еще можем использовать `@WithMockUser`, `@WithUserDetails` или пользовательскую аннотацию, чтобы определить свой собственный `SecurityContext`. Но вместо использования `MockMvc` мы напрямую вводим из контекста компонент, определяющий метод, который вам нужно протестировать.

Теперь откроем проект `ssia-ch11-ex1` и создадим тесты для метода `getName()` в классе `NameService`. Мы защитили метод `getName()` с помощью аннотации `@PreAuthorize`. В листинге 18.12 показана реализация тестового класса с тремя тестами, а рис. 18.9 иллюстрирует три сценария, которые мы тестируем:

- 1 при вызове без аутентифицированного пользователя метод должен выдать исключение `AuthenticationException`;
- 2 при вызове с аутентифицированным пользователем, имеющим полномочия, отличные от ожидаемых (запись), метод должен выдать исключение `AccessDeniedException`;
- 3 вызов метода с аутентифицированным пользователем, имеющим нужные полномочия, возвращает ожидаемый результат.

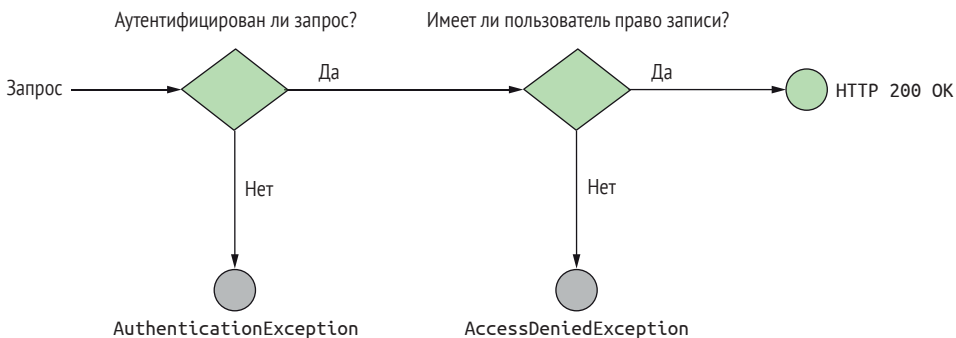


Рис. 18.9 Протестированные сценарии. Если HTTP-запрос не аутентифицирован, ожидаемым результатом является `AuthenticationException`. Если HTTP-запрос аутентифицирован, но у пользователя нет ожидаемых полномочий, ожидаемым результатом является `AccessDeniedException`. Если аутентифицированный пользователь имеет нужные полномочия, вызов успешен

Листинг 18.12 Реализация трех тестовых сценариев для метода `getName()`

```

@SpringBootTest
class MainTests {

    @Autowired

```

```
private NameService nameService;

@Test
void testNameServiceWithNoUser() {
    assertThrows(AuthenticationException.class,
        () -> nameService.getName());
}

@Test
@WithMockUser(authorities = "read")
void testNameServiceWithUserButWrongAuthority() {
    assertThrows(AccessDeniedException.class,
        () -> nameService.getName());
}

@Test
@WithMockUser(authorities = "write")
void testNameServiceWithUserButCorrectAuthority() {
    var result = nameService.getName();

    assertEquals("Fantastico", result);
}
}
```

Мы больше не настраиваем `MockMvc`, потому что нам не нужно вызывать конечную точку. Вместо этого мы напрямую внедряем экземпляр `NameService` для вызова тестируемого метода. Мы используем аннотацию `@WithMockUser`, как обсуждалось в разделе 18.1. Аналогично мы могли бы использовать `@WithUserDetails`, как обсуждалось в разделе 18.2, или разработать собственный способ создания `SecurityContext`, как показано в разделе 18.3.

18.5 Тестирование аутентификации

Далее мы обсудим тестирование аутентификации. В этой главе вы уже узнали, как задавать фиктивных пользователей и тестировать механизмы авторизации. Но можем ли мы аналогичным образом протестировать логику аутентификации? Мы не только можем, но и должны сделать это, если, например, у нас реализована пользовательская логика для аутентификации и нужно убедиться в правильной работе всей цепочки. При тестировании аутентификации проверочные запросы ничем не отличаются от обычных клиентских запросов, как показано на рис. 18.10.

В качестве примера вернемся к проекту `ssia-ch2-ex4`. Можем ли мы рассчитывать, что разработанный нами провайдер пользовательской аутентификации работает правильно? В этом проекте мы реализовали собственный `AuthenticationProvider` и с помощью

тестов хотим убедиться, что защитили логику пользовательской аутентификации.

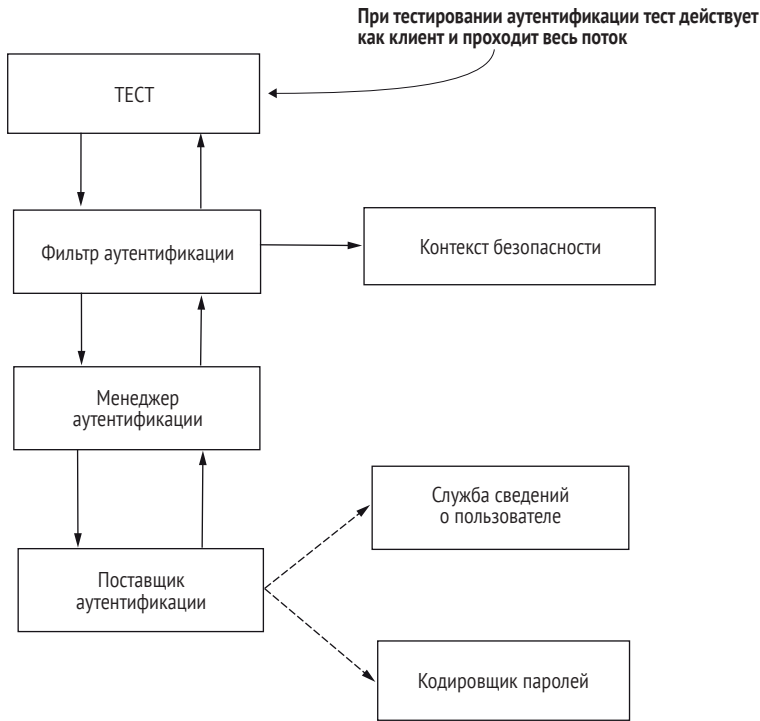


Рис. 18.10 При проверке аутентификации тест действует как обычный клиент и проходит через всю цепочку безопасности Spring Security, упомянутую в книге. Поэтому мы можем протестировать, например, объекты `AuthenticationProvider` собственной разработки

Логика, которую мы применим, проста. Принимается только один набор учетных данных: имя пользователя "john" и пароль "12345". Нам нужно доказать, что при использовании действительных учетных данных вызов будет успешным, тогда как при использовании других учетных данных статус ответа HTTP будет 401 Unauthorized. Давайте снова откроем проект `ssia-ch2-ex4` и добавим в него несколько тестов, чтобы убедиться, что аутентификация работает правильно (листинг 18.13).

Листинг 18.13 Тестирование аутентификации с помощью `httpBasic()` `RequestPostProcessor`

```

@SpringBootTest
@AutoConfigureMockMvc
public class AuthenticationTests {

```

```
@Autowired
private MockMvc mvc;

@Test
public void helloAuthenticatingWithValidUser() throws Exception {
    mvc.perform(
        get("/hello")
            .with(httpBasic("john","12345")))    ❶
        .andExpect(status().isOk());
    }

    @Test
    public void helloAuthenticatingWithInvalidUser() throws Exception {
        mvc.perform(
            get("/hello")
                .with(httpBasic("mary","12345")))    ❷
                .andExpect(status().isUnauthorized());
        }
    }
}
```

❶ Аутентификация с правильными учетными данными.

❷ Аутентификация с неправильными учетными данными.

Используя постпроцессор запросов `httpBasic()`, мы указываем тесту выполнить аутентификацию и проверяем поведение конечной точки при аутентификации с использованием действительных или недействительных учетных данных. Мы можем применить этот подход и для тестирования аутентификации с использованием формы входа. Давайте откроем проект `ssia-ch6-ex4`, где мы использовали форму входа для аутентификации, и добавим в него несколько тестов, чтобы убедиться, что аутентификация работает правильно. Мы тестируем поведение приложения в следующих ситуациях:

- при аутентификации с неверным набором учетных данных;
- при аутентификации с верным набором учетных данных, но когда у пользователя нет допустимых полномочий в соответствии с реализацией, которую мы написали в `AuthenticationSuccessHandler`;
- при аутентификации с верным набором учетных данных и пользователем, имеющим допустимые полномочия в соответствии с реализацией, которую мы написали в `AuthenticationSuccessHandler`.

В листинге 18.14 представлен код для первого случая. Если мы попытаемся пройти аутентификацию с использованием недопустимых учетных данных, приложение не аутентифицирует пользователя и добавит заголовок `"failed"` в ответ HTTP. Мы настроили приложение и добавили заголовок `"failed"` с `AuthenticationFailureHandler` при обсуждении аутентификации в главе 6.

Листинг 18.14 Тестирование аутентификации с неудавшимся входом через форму

```

@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;

    @Test
    public void loggingInWithWrongUser() throws Exception {
        mvc.perform(formLogin()
            .user("joeey").password("12345")
            .andExpect(header().exists("failed"))
            .andExpect(unauthenticated()));
    }
}

```

❶ Аутентификация с использованием входа через форму с недопустимым набором учетных данных.

В главе 6 мы настроили логику аутентификации с помощью `AuthenticationSuccessHandler`. В нашем случае, если у пользователя есть полномочия на чтение, приложение перенаправляет его на страницу `/home`. В противном случае приложение перенаправляет пользователя на страницу `/error`. В листинге 18.15 показана реализация этих двух сценариев.

Листинг 18.15 Тестирование поведения приложения при аутентификации пользователей

```

@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;

    // Часть кода не показана

    @Test
    public void loggingInWithWrongAuthority() throws Exception {
        mvc.perform(formLogin()
            .user("bill").password("12345")
            )
            .andExpect(redirectedUrl("/error"))
            .andExpect(status().isFound())
            .andExpect(authenticated());
    }

    @Test
    public void loggingInWithCorrectAuthority() throws Exception {

```

```

mvc.perform(formLogin()
    .user("john").password("12345")
    )
    .andExpect(redirectedUrl("/home"))      ❷
    .andExpect(status().isFound())
    .andExpect(authenticated());
}
}

```

- ❶ При аутентификации от имени пользователя, у которого нет полномочий на чтение, приложение перенаправляет пользователя на путь /error.
- ❷ При аутентификации от имени пользователя, у которого есть полномочия на чтение, приложение перенаправляет пользователя на путь /home.

Если приложение является сервером ресурсов OAuth 2/OpenID Connect (глава 15), вам понадобится токен для проверки аутентификации. Сервер ресурсов может использовать либо прозрачные JWT, либо непрозрачные токены. Spring Security поддерживает тестирование для обоих этих подходов. Аналогично методу `with(httpBasic())`, использованному ранее в этом разделе, вы можете применить `with(jwt())` для настройки фиктивного токена JWT для вашего теста или `with(opaqueToken())` для настройки фиктивного непрозрачного токена для вашего теста.

В следующем листинге показан пример теста, который вы можете найти в проекте `ssia-ch15-ex1`. Этот тест использует метод `with(jwt())` для установки фиктивного токена при проверке механизма аутентификации сервера ресурсов.

Листинг 18.16 Использование фиктивного JWT для проверки аутентификации сервера ресурсов

```

@SpringBootTest
@AutoConfigureMockMvc
class ApplicationTests {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void demoEndpointSuccessfulAuthenticationTest() throws Exception {
        mockMvc.perform(
            get("/demo").with(jwt())      ❶
        ).andExpect(status().isOk());
    }
}

```

- ❶ Настраиваем фиктивный JWT для проверки аутентификации сервера ресурсов с использованием прозрачных токенов.

У вас может возникнуть необходимость задать значения некоторых настраиваемых полей в токене, таких как полномочия. Вы мо-

жете использовать методы конфигурации, следующие за методом `jwt()`, для указания пользовательских полномочий или даже полной настройки JWT. Следующий код добавляет полномочия с именем "read" к фиктивному токenu, используемому при аутентификации:

```
jwt().authorities(() -> "read"))
```

Тест, аналогичный представленному в листинге 18.16, но для непрозрачных токенов, можно найти в проекте `ssia-ch15-ex3`. Следующий листинг демонстрирует код этого теста.

Листинг 18.17 Использование фиктивного непрозрачного токена для проверки механизма аутентификации сервера ресурсов

```
@SpringBootTest
@AutoConfigureMockMvc
class ApplicationTests {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void demoEndpointSuccessfulAuthenticationTest() throws Exception {
        mockMvc.perform(
            get("/demo").with(opaqueToken())
            .andExpect(status().isOk());
    }
}
```

❶ Использование фиктивного непрозрачного токена для проверки механизма аутентификации сервера ресурсов.

Даже с непрозрачным токеном вам могут потребоваться определенные полномочия в контексте безопасности, который возникает после аутентификации. Вы можете решать, какие полномочия будут у экземпляра аутентификации, добавленного в контекст безопасности. Для этого достаточно воспользоваться методом `opaqueToken()` с методом конфигурации `authority()`, как показано ниже. Следующая строка кода задает полномочия с именем "read" в экземпляре аутентификации, который будет добавлен в контекст безопасности теста:

```
opaqueToken().authorities(() -> "read"))
```

18.6 Тестирование конфигураций CSRF

В этом разделе мы обсудим тестирование конфигурации защиты от CSRF вашего приложения. Когда приложение содержит уязвимость

CSRF, злоумышленник может обмануть пользователя, заставив его выполнить нежелательные действия после входа в приложение. Как обсуждалось в главе 9, для купирования этих уязвимостей Spring Security использует токены CSRF. В соответствии с этим подходом для любой операции, потенциально способной внести изменения в данные (POST, PUT, DELETE), заголовок запроса должен содержать действительный токен CSRF. Конечно, рано или поздно вам нужно будет протестировать больше, чем простые запросы HTTP GET. В зависимости от того, как вы построите свое приложение, вам может потребоваться протестировать защиту CSRF. Вам нужно убедиться, что она работает, как задумано, и защищает конечную точку, которая выполняет изменения.

К счастью, Spring Security предоставляет простой механизм тестирования защиты от CSRF с помощью `RequestPostProcessor`. Давайте откроем проект `ssia-ch9-ex1` и проверим, что защита от CSRF включена для конечной точки `/hello` при вызове с помощью HTTP POST в следующих ситуациях:

- если мы не используем токен CSRF, статус ответа HTTP будет 403 Forbidden;
- если мы отправляем токен CSRF, статус ответа HTTP – 200 OK.

В следующем листинге показана реализация этих двух сценариев. Обратите внимание, что мы можем отправить токен CSRF, просто используя метод `csrf()` класса `RequestPostProcessor`.

Листинг 18.18 Реализация сценариев тестирования защиты от CSRF

```
@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;

    @Test
    public void testHelloPOST() throws Exception {
        mvc.perform(post("/hello"))
            .andExpect(status().isForbidden());
    }

    @Test
    public void testHelloPOSTWithCSRF() throws Exception {
        mvc.perform(post("/hello").with(csrf()))
            .andExpect(status().isOk());
    }
}
```

❶ При вызове конечной точки без токена CSRF статус ответа HTTP – 403 Forbidden.

❷ При вызове конечной точки с токеном CSRF статус ответа HTTP – 200 OK.

18.7 Тестирование конфигураций CORS

В этом разделе обсуждается тестирование конфигураций CORS. Как вы узнали в главе 10, если браузер загружает веб-приложение из одного источника (например, example.com), браузер не позволит приложению использовать ответ HTTP, который приходит из другого источника (например, example.org). Мы используем политики CORS, чтобы ослабить эти ограничения и настроить наше приложение для работы с несколькими источниками. Конечно, как и в случае любых других конфигураций безопасности, вам также необходимо протестировать политики CORS. В главе 10 вы узнали, что CORS рассматривает определенные заголовки в ответе, значения которых определяют, принимается ли ответ HTTP. Два из этих заголовков, связанных со спецификациями CORS, – это Access-Control-Allow-Origin и Access-Control-Allow-Methods. Мы использовали эти заголовки в главе 10, чтобы разрешить получать данные из нескольких источников для нашего приложения.

При написании тестов для политик CORS все, что нам нужно сделать, – это убедиться, что эти заголовки (и, возможно, другие заголовки, связанные с CORS, в зависимости от сложности наших конфигураций) существуют и имеют правильные значения. Для этой проверки мы можем имитировать действия браузера при выполнении предварительного запроса. Мы делаем запрос с использованием метода HTTP OPTIONS, запрашивая значение для заголовков CORS. Давайте откроем проект ssia-ch10-ex1 и напишем тест для проверки значений заголовков. В листинге 18.19 показан код теста.

Листинг 18.19 Реализация теста для политик CORS

```
@SpringBootTest
@AutoConfigureMockMvc
public class MainTests {

    @Autowired
    private MockMvc mvc;

    @Test
    public void testCORSForTestEndpoint() throws Exception {
        mvc.perform(options("/test")
            .header("Access-Control-Request-Method", "POST")
            .header("Origin", "http://www.example.com")
        )
        .andExpect(header().exists("Access-Control-Allow-Origin"))
        .andExpect(header().string("Access-Control-Allow-Origin", ""))
        .andExpect(header().exists("Access-Control-Allow-Methods"))
    }
}
```

```
        .andExpect(header().string("Access-Control-Allow-Methods", "POST"))
        .andExpect(status().isOk());
    }
}
```

- ❶ Выполняем запрос HTTP OPTIONS к конечной точке, запрашивая значение для заголовков CORS.
- ❷ Проверяем значения для заголовков в соответствии с конфигурацией, которую мы сделали в приложении.

18.8 Тестирование безопасности Spring Security в реактивных приложениях

В этом разделе мы обсудим тестирование интеграции Spring Security с функциональными возможностями реактивного приложения. Думаю, вы не удивитесь, узнав, что Spring Security также обеспечивает поддержку тестирования конфигураций безопасности для реактивных приложений. Как и в случае с нереактивными приложениями, безопасность для реактивных приложений имеет решающее значение. Поэтому тестирование их конфигураций безопасности столь же необходимо. Чтобы продемонстрировать тесты для конфигураций безопасности, мы вернемся к примерам, которые рассматривали в главе 17. В случае интеграции Spring Security с реактивными приложениями вам нужно знать два подхода к написанию тестов:

- использование фиктивных пользователей с аннотациями `@WithMockUser`;
- использование `WebTestClientConfigurer`.

Применение аннотации `@WithMockUser` не составляет сложности, поскольку она работает так же, как и для нереактивных приложений, о которых говорилось в разделе 18.1. Но сам тест отличается, поскольку, учитывая, что это реактивное приложение, мы больше не можем использовать `MockMvc`. Это изменение не связано с Spring Security. При тестировании реактивных приложений можно воспользоваться инструментом под названием `WebTestClient`. В листинге 18.20 представлен код простого теста, использующего фиктивного пользователя для проверки поведения реактивной конечной точки.

Листинг 18.20 Использование `@WithMockUser` при тестировании реактивных приложений

```
@SpringBootTest
@AutoConfigureWebTestClient
class MainTests {

    @Autowired
```

❶

❷

```

private WebTestClient client;

@Test
@WithMockUser
void testCallHelloWithValidUser() {
    client.get()
        .uri("/hello")
        .exchange()
        .expectStatus().isOk();
}
}

```

- ❶ Запрашиваем у Spring Boot автоматическое конфигурирование WebTestClient, который мы используем для тестов.
- ❷ Вводим экземпляр WebTestClient, сконфигурированный Spring Boot из контекста Spring.
- ❸ Используем аннотацию @WithMockUser для определения фиктивного пользователя при тестировании.
- ❹ Выполняем обмен данными и проверяем результат.

Как видите, использование аннотации @WithMockUser практически такое же, как и для неактивных приложений. Фреймворк создает SecurityContext с фиктивным пользователем. Приложение пропускает процесс аутентификации и использует фиктивного пользователя из SecurityContext теста для проверки правил авторизации.

Второй способ тестирования – с использованием WebTestClientConfigurer. Этот подход аналогичен использованию RequestPostProcessor в случае неактивного приложения. В случае реактивного приложения для используемого нами WebTestClient мы применяем WebTestClientConfigurer, который помогает изменять тестовый контекст. Например, мы можем определить фиктивного пользователя или отправить токен CSRF для проверки защиты от CSRF, как делали для неактивных приложений в разделе 18.6. В следующем листинге показано, как использовать WebTestClientConfigurer.

Листинг 18.21 Использование WebTestClientConfigurer для определения фиктивного пользователя

```

@SpringBootTest
@AutoConfigureWebTestClient
class MainTests {

    @Autowired
    private WebTestClient client;

    // Часть кода не показана

    @Test
    void testCallHelloWithValidUserWithMockUser() {
        client.mutateWith(mockUser())
            .get()
    }
}

```

```
        .uri("/hello")
        .exchange()
        .expectStatus().isOk();
    }
}
```

- ❶ Перед выполнением запроса GET изменяем вызов для использования фиктивного пользователя.

Для тестирования защиты от CSRF в вызове POST можно применить примерно такой код:

```
client.mutateWith(csrf())
    .post()
    .uri("/hello")
    .exchange()
    .expectStatus().isOk();
```

Заклучение

- Регулярное и полное тестирование – это лучшая практика безопасности. Мы разрабатываем тесты, чтобы убедиться, что наши новые реализации или исправления не наносят ущерб функциональности и безопасности приложения.
- Вам нужно тестировать не только свой код, но и интеграцию с библиотеками и фреймворками, которые вы используете.
- Spring Security предлагает отличную поддержку для реализации тестов ваших конфигураций безопасности.
- Вы можете тестировать авторизацию напрямую, используя фиктивных пользователей. Для этого предназначены специальные тесты авторизации без аутентификации, поскольку для проверки аутентификации обычно нужно намного меньше тестов, чем для авторизации.
- Проверка аутентификации в отдельных одиночных тестах значительно уменьшает трудозатраты и вычислительную нагрузку. Затем вы можете проверить работу механизма авторизации ваших конечных точек и методов при помощи более многочисленных специализированных тестов.
- Для тестирования конфигураций безопасности конечных точек в нереактивных приложениях Spring Security предлагает отличную поддержку тестов с помощью `MockMvc`.
- Для тестирования конфигураций безопасности конечных точек в реактивных приложениях Spring Security предлагает отличную поддержку тестов с помощью `WebTestClient`.
- Есть возможность разрабатывать тесты непосредственно для методов, для которых вы написали конфигурации безопасности, используя подход безопасности метода.

Приложение А

Ссылки на официальную документацию

Я рекомендую перед прочтением этой книги ознакомиться с официальной документацией фреймворков, которые мы будем использовать:

- фреймворк Spring: <https://docs.spring.io/spring-framework/reference/index.html>;
- Spring Security: <https://docs.spring.io/spring-security/reference/index.html>;
- сервер авторизации Spring Security: <https://docs.spring.io/spring-authorization-server/docs/current/api/>.

Если вы используете проекты с Java 17, изучите спецификации API, опубликованные по адресу <https://docs.oracle.com/en/java/javase/17/docs/api/>.

Если вы используете другую версию Java, пожалуйста, выберите спецификации API из списка по адресу <https://docs.oracle.com/en/java/javase/>.

В главе 18 мы используем Junit 5 для написания модульных и интеграционных тестов. Ознакомьтесь с официальной документацией JUnit 5, размещенной по адресу <https://junit.org/junit5/docs/current/user-guide/>.

Приложение В

Дополнительная литература

Я рекомендую прочитать несколько книг, которые помогут вам закрепить и дополнить знания, полученные в этой книге.

Про фреймворк Spring в целом

Крейг Уоллс: «Spring в действии. Шестое издание» (ДМК Пресс, 2022)

Книга Крейга Уоллса – это исчерпывающее руководство по Spring Framework, популярному фреймворку приложений Java. Она содержит детальное описание основных концепций и функции Spring, что весьма полезно для разработчиков Java, желающих использовать фреймворк при создании надежных масштабируемых приложений. Основные темы книги: внедрение зависимостей, аспектно-ориентированное программирование, доступ к данным и Spring MVC для веб-приложений. Книга хороша своими доходчивыми объяснениями и практическими примерами, которые помогают читателям понять не только то, как использовать Spring, но и почему рекомендуются определенные подходы. Это делает ее ценным источником знаний как для новичков, так и для опытных разработчиков, стремящихся развить навыки использования Spring. Вместе с моей книгой, которую вы сейчас читаете, они дают полное представление об основных функциях Spring и о том, как эффективно их защитить, разрабатывая надежные и безопасные приложения.

Марк Хеклер: «Spring Boot по-быстрому» (Питер, 2022)

Это полезное руководство, посвященное Spring Boot – инструменту, который упрощает разработку и развертывание приложений на основе фреймворка Spring. Данная книга ценна своим практическим подходом и дает читателям четкое представление об эффективной разработке микросервисов, веб-приложений и других программных продуктов с использованием Spring Boot. В ней рассматриваются такие ключевые функции, как автоматическая настройка, развер-

тывание автономного кода и использование встроенного сервера, что делает ее идеальным учебником для разработчиков, которые хотят быстро освоить Spring Boot. Читатели и рецензенты хвалят книгу за ее простые объяснения и практичные примеры, благодаря которым разработчики могут создавать надежные и эффективные приложения с минимальными затруднениями. Прочитав ее вместе с моей книгой «Spring Security в действии», вы научитесь быстро разрабатывать и защищать приложения Spring Boot, применяя лучшие практики безопасности с самого начала процесса разработки.

Лауренциу Спилкэ: «Spring быстро» (Питер, 2022)

Эта книга послужит отличной отправной точкой для разработчиков, впервые столкнувшихся с экосистемой Spring. Книга предлагает емкое и доступное введение в фреймворк Spring, уделяя особое внимание фундаментальным концепциям и методам, необходимым для создания современных приложений Java. Она охватывает такие важные темы, как внедрение зависимостей, доступ к данным, веб-разработка с помощью Spring MVC и безопасность, формируя прочную основу для понимания Spring. Книга особенно ценна своим акцентом на практических примерах и лучших практиках, помогая новичкам не только изучить основы, но и эффективно применять знания в реальных разработках. Эти качества делают книгу идеальным ресурсом как для начинающих программистов, так и для опытных разработчиков, ищущих структурированный и всесторонний вход в разработку Spring. В сочетании с книгой «Spring Security в действии» она сформирует у вас фундаментальное понимание основ Spring, а также основных мер безопасности, необходимых для безопасной разработки приложений.

О создании уровня хранения вашего приложения Java

Vlad Mihalcea: *High-Performance Java Persistence* (издание автора, 2019)

Это ключевой ресурс для разработчиков Java, посвященный оптимизации быстродействия баз данных. Книга погружает читателя в мир сохранения данных Java, в частности с Hibernate и JPA (Java Persistence API). Она особенно ценна своим глубоким исследованием проблем производительности и передовых методов обработки данных и знакомит читателя с ключевыми понятиями управления соединением, пакетной обработки, обработки транзакций и многими другими. Михалчи – активный участник разработки Hibernate. Он приводит реальные примеры и тесты, делая сложные концепции доступными и применимыми. Эта книга идеально подходит для разработчиков,

которые хотят освоить высокопроизводительные методы сохранения данных Java с гарантией, что их приложения для работы с базами данных будут эффективны, масштабируемы и надежны. Чтение этой книги в сочетании с «Spring Security в действии» позволяет глубоко понять принципы оптимизации взаимодействия с базами данных в приложениях Java, при условии сохранения безопасности операций и защиты этих взаимодействий.

Cătălin Tudose: Java Persistence with Spring Data and Hibernate, 3rd ed. (Manning, 2023)

Это емкое и обширное руководство посвящено интеграции JPA с Spring Data и Hibernate. Третье издание, обновленное в 2023 г., содержит подробный обзор механизмов эффективного управления сохранением данных в приложениях Java. Автор придерживается практического подхода, исследуя, как Spring Data упрощает доступ к данным и как Hibernate применяется для более сложных сценариев обработки данных. Книга охватывает такие темы, как сопоставление сущностей, запросы, управление транзакциями и репозитории Spring Data. Понятные объяснения автора и реальные примеры облегчают понимание сложных тем; благодаря такому подходу эта книга послужит отличным ресурсом как для начинающих, так и для опытных разработчиков Java, желающих углубить свои знания о сохранении данных Java с помощью Spring Data и Hibernate. Третье издание обновлено с учетом новейших функций и лучших практик, поэтому читатели изучат самые современные методы в этой области. В сочетании с книгой «Spring Security в действии» оно дает полное представление об эффективном и безопасном управлении сохранением данных в приложениях Java, охватывая как обработку данных, так и аспекты безопасности.

О тестировании приложений Spring

Cătălin Tudose: JUnit in Action, 3rd ed. (Manning, 2021)

Это авторитетное руководство по JUnit – популярному фреймворку тестирования Java. Обновленная с учетом последних нововведений в JUnit 5, книга представляет собой всестороннее исследование эффективных методов тестирования в Java. Она особенно ценна подробным описанием процесса разработки и запуска тестов, подтверждения корректности кода и организации тестов для максимальной эффективности. Автор выходит за рамки основ и углубляется в такие сложные темы, как имитация, разработка через тестирование (TDD), интеграционное тестирование и тестирование приложений Spring. Понятные практические примеры и объяснения делают эту книгу идеальным ресурсом для разработчиков Java всех уровней квалифи-

кации, которые хотят улучшить качество своих приложений и усовершенствовать процесс разработки с помощью надежного тестирования. Приоритетное внимание к версии JUnit 5 гарантирует, что читатели изучат самые современные методологии тестирования, поэтому можно сказать, что эта книга обязательна к прочтению всеми, кто хочет быть в курсе современных методов тестирования Java. Сочетание с книгой «Spring Security в действии» помогает понять, как эффективно тестировать приложения Java, включая функции безопасности, чтобы функциональность и безопасность приложений всегда оставались на высоком уровне.

Предметный указатель

- А**вторизация, 34, 179
 - вызовов, 262
 - последующая (поставторизация), 264
 - предварительная, 264
- Аннотация, 28
- Аутентификация, 34
 - менеджер, 47
 - провайдер, 47
 - фильтр, 47
- Б**езопасность метода, 261
- Бин, 27
- В**ерификатор, 328
- Выражение пути, 213
- Г**енератор ключей, 112
- З**она доступности, 33
- И**здатель, 419
- Испытание, 328
- К**люч шифрования
 - асимметричный, 111
 - закрытый, 111
 - открытый, 111
 - симметричный, 111
- Кодирование, 111
- Кодировщик паролей, 47
- Контекст, 27
 - безопасности, 47, 151
- Контракт, 29
- Кросс-доменный обмен ресурсами, 250
- М**ежсайтовый скриптинг, 258
- Метод, 28
- О**перация мутации, 226
- Ответственность, 63
- П**одделка межсайтовых запросов, 209
- Подписчик, 419
- Полномочия, 55, 74, 181
- Поставщик удостоверений, 333
- Предварительный запрос, 256
- Привилегии, 73
- Принципал, 141
- Р**азрешение, 278
- Реактивность, 416
- Регрессионное тестирование, 445
- Регулярные выражения, 217
- Роли, 192
- С**екрет, 35
 - хранилище, 35
- Соглашение по конфигурации, 40
- Соль, 111
- Т**ип гранта, 324
- Токен
 - доступа, 318
 - интроспекция, 356
 - непрозрачный, 321
 - обновления, 331
 - отзыв, 358
 - прозрачный, 321
 - утверждение, 371
- Ф**иктивный пользователь, 447
- Фильтрация, 263
 - последующая (постфильтрация), 291
 - предварительная, 290
- Х**еширование, 111
- Ц**епочка фильтров, 120
- Ш**ифратор, 114
- Шифрование, 111

Лауренциу Спилкэ

Spring Security в действии

<https://t.me/javallib>

Spring Security значительно упрощает защиту корпоративных приложений, разработанных на Java. Этот мощный фреймворк идеально интегрируется с приложениями Spring, обеспечивая защиту с первых шагов проекта и предоставляя готовые к использованию функции, которые помогут вам реализовать надежную авторизацию и аутентификацию, а также защитить от кражи данных и вторжений. Как и все остальное в экосистеме Spring, этот фреймворк бесплатен, имеет открытый исходный код и поддерживается замечательной командой VMWare.

Понятные объяснения и наглядные примеры научат вас создавать собственные серверы авторизации, настраивать защищенные конечные точки и предотвращать атаки с использованием межсайтового скриптинга и подделки запросов.

Рассматриваемые темы:

- собственные классы аутентификации и авторизации;
- конфигурации CRSF и CORS;
- подробное описание OAuth2/ OpenID Connect;
- безопасные реактивные приложения Spring;
- написание тестов для конфигураций безопасности;
- настройки безопасности с использованием нового фильтра Security-FilterChain.

Издание адресовано программистам Java и Spring.

Лауренциу Спилкэ — разработчик Java и Spring с многолетним опытом преподавания. Автор книг «Java: устранение проблем» (ДМК Пресс, 2023) и «Spring быстро» (Питер, 2022).

«Эта книга доступно рассказывает о безопасности и содержит полезные примеры кода на основе популярного в отрасли фреймворка Spring Security».

*Джош Лонг,
менеджер по продвижению
Spring*

«Легче всего изучать теорию, подкрепленную наглядными примерами».

*Ана-Мария Михальчану,
менеджер по продвижению
продукта*

«Автор обладает талантом рассказывать сложные вещи простыми словами. Это идеально подходит для книги о безопасности. Непременно прочитайте ее!»

*Томас Витале, автор книги
Cloud Native Spring in Action*

